

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++

Juan Carlos Morales
Doble titulación: Sistemas & Robótica

Bogotá, Colombia · 2026

Índice

I	Introduccion	6
1	Fundamentos Básicos	6
1.1	Entrada / Salida Rápida (Fast I/O)	6
1.2	Formateo de Salida	6
1.3	Condicionales y Ciclos	7
1.4	Conversiones, Parseos y Casteos	7
II	Estructuras de Datos	8
2	Basic Structures	8
2.1	Matrices (Arreglos 2D)	8
2.2	Vectores / Listas dinámicas	8
2.3	Listas enlazadas (LinkedList)	8
2.4	Pilas (Stack)	8
2.5	Colas (Queue)	8
2.6	Colas de prioridad (Priority Queue / Heap)	9
2.7	Comparadores en C++ (Comparators)	9
2.8	Vector circular (Circular Buffer)	10
3	Estructuras Hash/Tree	11
3.1	Conjuntos hash (HashSet)	11
3.2	Mapas / Diccionarios hash	12
3.3	Conjuntos ordenados (TreeSet)	12
3.4	Mapas ordenados (TreeMap)	13
4	Trees	13
5	Range Queries	13
5.1	Casos de uso	14
5.2	Prefix Sums	14
5.3	Árbol de Fenwick (BIT)	16
5.4	Sqrt Decomposition	18
5.5	Sparse Table (RMQ)	20
5.6	Segment Tree	21
5.7	RMQ (Range Minimum Query)	23
5.8	Mo's Algorithm	24
5.9	Wavelet Tree	26
5.10	Heavy-Light Decomposition	27
III	Grafos	29
5.11	¿Qué algoritmo usar?	29
6	Traversal	30
6.1	Representación de Grafos	30
6.2	BFS (Breadth-First Search)	31
6.3	BFS en Grafos Desconectados	34
6.4	BFS para Todos los Caminos	34
6.5	DFS	36
6.6	DFS para Conteo de Caminos	37
6.7	DFS + Cerramiento Transitivo	37
6.8	Flood Fill	38
6.9	Laberinto con BFS	39
6.10	Tour de Caballo de Ajedrez	40
6.11	Mínimo Número de Movimientos de Caballo	40
6.12	Serpientes y Escaleras	41

7	Conectividad	42
7.1	Componentes Conexas	42
7.2	Detección de Ciclos	42
7.3	Puentes	43
7.4	Puntos de Articulación	44
7.5	Componentes Biconectados	45
7.6	Tarjan	46
8	Topological Sort	46
8.1	Kahn's Algorithm	46
8.2	DFS + Orden Topológico	47
8.3	Detección de Ciclos con Topological Sort	48
9	Single Source Shortest Path (SSSP)	49
9.1	BFS para Caminos Más Cortos	49
9.2	0-1 BFS	49
9.3	Dijkstra	50
9.4	Bellman-Ford	51
9.5	Shortest Path en Grafos Binarios	52
9.6	Shortest Path desde Múltiples Orígenes	52
9.7	Shortest Path en Grafos Multietapa	53
9.8	Shortest Path con Exactamente k Aristas	54
10	All Pairs Shortest Path (APSP)	55
10.1	Floyd-Warshall	55
11	Minimum Spanning Tree (MST)	55
11.1	Disjoint Set Union (DSU)	55
11.2	Kruskal	56
11.3	Prim	57
12	Strongly Connected Components (SCC)	58
12.1	Kosaraju	58
12.2	Tarjan SCC	59
12.3	Condensation Graph	59
13	Bipartite Graphs	60
13.1	2-Coloring	60
13.2	Bipartite Matching	61
14	Trees	62
14.1	DFS en Árboles	62
14.2	BFS en Árboles	62
14.3	Diámetro de un Árbol	63
14.4	Centro de un Árbol	64
14.5	Combinatoria en Árboles Ordenados	65
14.6	Tree DP	66
14.7	Rerooting	66
15	Lowest Common Ancestor (LCA)	68
15.1	Binary Lifting	68
15.2	Euler Tour + RMQ	69
15.3	Distancia entre Dos Nodos	69
15.4	k -ésimo Ancestro	70
16	Tree Queries	71
16.1	Euler Tour	71
16.2	Subtree Queries	72
16.3	Path Queries	73
16.4	Heavy-Light Decomposition	74
16.5	Heavy-Light Decomposition + Fenwick	75
16.6	Heavy-Light Decomposition + Segment Tree	76

17 Disjoint Set Union	77
17.1 Union-Find	77
17.2 Path Compression	78
17.3 Union by Size / Rank	79
17.4 Unión de Conjuntos Disjuntos en un Árbol	79
17.5 DSU Offline	80
17.6 DSU Rollback	81
18 Graph Decomposition	82
18.1 Bridge Tree	82
18.2 Block-Cut Tree	83
18.3 Condensation Graph	84
19 Eulerian Graphs	85
19.1 Eulerian Path	85
19.2 Eulerian Circuit	86
19.3 Hierholzer	87
20 Network Flow	88
21 Network Flow	88
21.1 Ford-Fulkerson	88
21.2 Edmonds-Karp	89
21.3 Dinic	90
21.4 Maximum Bipartite Matching	92
22 2-SAT	93
22.1 Implication Graph	93
22.2 2-SAT con SCC	94
23 Backtracking y Problemas Clásicos	95
23.1 Problema de las N Reinas	95
23.2 Variaciones del Problema de las N Reinas	96
23.3 Tour de Caballo	97
24 Advanced Graph Algorithms	98
24.1 Centroid Decomposition	98
24.2 DSU on Tree	100
24.3 Hopcroft-Karp	101
24.4 Min-Cost Max-Flow	102
24.5 DSU Rollback	103
IV Ordenamiento y Búsqueda	104
25 Ordenamiento y Búsqueda (Sorting & Searching)	104
25.1 Número faltante (Missing Number)	104
25.2 Máximo / Mínimo (Array Max/Min)	105
25.3 Two Pointers	106
25.4 Sliding Window	106
25.5 Par de suma absoluta mínima	107
25.6 Par con diferencia dada (Difference Pair)	108
25.7 Búsqueda ternaria (Ternary Search)	109
25.8 Búsqueda de Fibonacci (Fibonacci Search)	109
25.9 Búsqueda exponencial (Exponential Search)	110
25.10 Búsqueda por saltos (Jump Search)	111
25.11 Meet in the Middle	111
25.12 Heap Sort	112
25.13 Merge Sort	113
25.14 Conteo de Inversiones (Counting Inversions)	114
25.15 QuickSelect	115
26 Búsqueda Binaria (Binary Search)	116

27 Búsqueda Binaria (Binary Search)	116
27.1 En arreglo ordenado	116
27.2 Menor x que cumple	117
27.3 Mayor x que cumple	117
27.4 Binary Search on Answer	118
27.5 Sobre reales	119
27.6 Con strings	120
 V Matemáticas	 122
28 Matemáticas (Math)	122
29 Teoría de Números (Number Theory)	122
 VI Geometría Computacional	 122
30 Line Sweep	122
30.1 Idea General y Manejo de Eventos	122
30.2 Intersección de Segmentos	123
30.3 Unión de Áreas de Rectángulos	124
30.4 Skyline Problem	125
30.5 Par de Puntos Más Cercano (Closest Pair)	126
 31 Primitivas Geométricas	 127
31.1 Vector Geométrico: Producto Punto y Producto Cruz	127
31.2 Orientación de Tres Puntos (Cross Product / CCW Test)	128
31.3 Distancia Euclidiana	129
31.4 Punto Medio de un Segmento	130
31.5 Ecuación de una Línea Dados Dos Puntos	130
31.6 División de un Segmento en una Razón Dada	131
31.7 Ángulo entre Dos Vectores	132
31.8 Tres Puntos Colineales	132
31.9 Distancia de un Punto a una Línea/Segmento	133
31.10 Reflexión de un Punto sobre una Línea	134
31.11 Rotación de un Punto (por Ángulo, alrededor de un Pivote)	135
 32 Triángulos	 136
32.1 Ángulos de un Triángulo	136
32.2 Fórmula de Herón (Área por Lados)	136
32.3 Círculo Circunscrito en un Triángulo	137
32.4 Punto Dentro de un Triángulo	138
 33 Círculos	 139
33.1 Longitud de Arco y Área de un Sector Circular	139
33.2 Intersección de Dos Círculos	139
 34 Polígonos	 139
34.1 Área de un Polígono (Shoelace)	139
34.2 Perímetro de un Polígono	139
34.3 Punto Dentro de un Polígono (Ray Casting)	139
34.4 Punto Dentro de un Polígono Convexo (Binary Search)	139
34.5 Teorema de Pick (Área en Malla de Puntos Enteros)	139
 35 Envoltente Convexa	 139
35.1 Algoritmo de Jarvis (Gift Wrapping)	139
35.2 Monotone Chain (Andrew)	139
35.3 Corte de un Polígono Convexo por una Línea	139

36 Ad-hoc y Combinatoria Geométrica	139
36.1 Perímetro Usando Bloques (Figuras en Grilla)	139
36.2 Número de Cuadrados en un Rectángulo $N \times M$	139
36.3 Número de Rectángulos en una Grilla $N \times M$	139
36.4 Suma de Distancias Manhattan entre Todos los Pares	139
36.5 Máximo de Cuadrados 2×2 en un Triángulo Isósceles Rectángulo	139
36.6 Máximo Número de Piezas con N Cortes (Pastelero Perezoso)	139
36.7 Altura Máxima de un Triángulo de Monedas	139
VII Programación Dinámica (Dynamic Programming)	139
VIII Algoritmos Voraces (Greedy)	139
IX Ad-hoc y Simulación	139
X String Processing	139
37 Cadenas (Strings)	139
XI BitWise	139
37.1 Casos de Uso	139
37.2 Representación Binaria de Enteros (con y sin signo)	140
37.3 Operaciones y Trucos Básicos	141
37.4 Desplazamientos Aritméticos vs. Lógicos (Arithmetic vs. Logical Shift)	142
37.5 Funciones Built-in de GCC	142
37.6 Conversión entre Bases Numéricas (Binario, Octal, Hexadecimal)	143
37.7 Overflow y Wraparound en Enteros	144
37.8 Operaciones Matemáticas en Binario (Suma, Resta, Multiplicación y Exponenciación Modular)	145
37.9 Bitset para Optimización (std::bitset)	147
37.10 Enumeración de Submáscaras (Submask Enumeration)	148
37.11 Suma sobre Subconjuntos (SOS DP)	148
37.12 Bitmask DP (Programación Dinámica sobre Subconjuntos)	149
37.13 DP con Perfil de Bits (Broken Profile DP)	151
37.14 XOR: Propiedades y Aplicaciones	152
37.15 Base Lineal de XOR (XOR Basis / Linear Basis)	153
37.16 Trie de Bits (Binary Trie / XOR Trie)	154
37.17 Código Gray (Gray Code)	155
37.18 Conteo de Pares con XOR/AND/OR Dado	156
37.19 Conteo de Bits Encendidos en un Rango $[0, N]$	157
37.20 Convolución XOR/AND/OR (Walsh-Hadamard Transform)	158
37.21 Meet in the Middle con Máscaras de Bits	160
37.22 Transformada de Möbius/Zeta sobre Subconjuntos	161
XII Game Theory	162

Parte I Introduccion

1 Fundamentos Básicos

1.1 Entrada / Salida Rápida (Fast I/O)

Usar lectura rápida para evitar TLE, hay 3 niveles de lectura rápida, estandarizar Tier 2 y usar Tier 3 (lector por bytes) solo cuando haya del orden de 10^6 – 10^7 lecturas.

Table 1: Casos donde se recomienda Fast I/O

Tipo de problema	Ejemplo
Arrays gigantes	Leer/ordenar/sumar 10^6 – 10^7 enteros
Muchos test cases (T grande)	$T \leq 10^5$, cada uno trivial pero se acumula
Grafos grandes	$M \sim 10^6$ – $2 \cdot 10^6$ aristas para BFS/DFS/Dijkstra/MST
Geometria con muchas consultas	L, S grandes + muchos test cases seguidos hasta EOF
Muchos tokens de texto	Strings/lineas que suman millones de caracteres
Judges clasicos "trampa de I/O"	SPOJ INTEST, ARRAY-SUB, etc.

Regla practica: si N/M es del orden de 10^4 – 10^5 , cin con `sync_with_stdio(false); cin.tie(nullptr);` alcanza de sobra. El FastReader es la "artilleria pesada" para cuando eso deja de ser suficiente.

Tier 1 — Básico: cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

Tier 2 — Rápido: desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

Tier 3 — Más rápido: lector por bytes con fread. Solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leidos = 0;
5
6     int leer() {
7         if (puntero == leidos) {
8             leidos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leidos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13
14    int nextInt() {
15        int resultado = 0, b = leer();
16        while (b <= ' ') b = leer(); // saltar blancos
17        bool negativo = (b == '-');
18        if (negativo) b = leer();
19        while (b >= '0' && b <= '9') {
20            resultado = resultado * 10 + (b - '0'); b = leer();
21        }
22        return negativo ? -resultado : resultado;
23    }
24}
```

```
23 long long nextLong() {
24     long long resultado = 0; int b = leer();
25     while (b <= ' ') b = leer();
26     bool negativo = (b == '-');
27     if (negativo) b = leer();
28     while (b >= '0' && b <= '9') {
29         resultado = resultado * 10 + (b - '0'); b = leer();
30     }
31     return negativo ? -resultado : resultado;
32 }
33
34 double nextDouble() {
35     double resultado = 0, divisor = 1; int b = leer();
36     while (b <= ' ') b = leer();
37     bool negativo = (b == '-');
38     if (negativo) b = leer();
39     while (b >= '0' && b <= '9') {
40         resultado = resultado * 10 + (b - '0'); b = leer();
41     }
42     if (b == '.') {
43         b = leer();
44         while (b >= '0' && b <= '9') {
45             resultado += (b - '0') / (divisor *= 10); b = leer();
46         }
47     }
48     return negativo ? -resultado : resultado;
49 }
50
51 string next() { // un token
52     string s; int b = leer();
53     while (b <= ' ') b = leer();
54     while (b > ' ') { s += (char) b; b = leer(); }
55     return s;
56 }
57
58 string nextLine() { // línea completa
59     string s; int b = leer();
60     while (b != '\n' && b != -1) {
61         if (b != '\r') s += (char) b; b = leer();
62     }
63     return s;
64 }
65
66 };
```

1.2 Formateo de Salida

La función `printf` utiliza una cadena de formato para indicar cómo debe interpretarse y mostrarse cada argumento. Cada especificador comienza con `%` y corresponde al argumento que aparece después de la cadena.

```
1 // %d -> entero (int)
2 printf("%d\n", 42);
3 // Entrada: 42
4 // Salida: 42
5
6 // %lld -> entero largo (long long)
7 long long x = 1234567890123LL;
8 printf("%lld\n", x);
9 // Entrada: 1234567890123
10 // Salida: 1234567890123
11
12 // %5d -> ancho mínimo de 5 caracteres, alineado a la derecha
13 printf("%5d\n", 42);
14 // Entrada: 42
15 // Salida: "  42"
16
17 // %-5d -> ancho mínimo de 5 caracteres, alineado a la izquierda
18 printf("%-5d\n", 42);
19 // Entrada: 42
20 // Salida: "42  "
21
22 // %05d -> ancho 5, completando con ceros
23 printf("%05d\n", 42);
24 // Entrada: 42
25 // Salida: "00042"
26
27 // %+d -> muestra siempre el signo
28 printf("%+d\n", 42);
29 // Entrada: 42
30 // Salida: "+42"
31
32 // %x -> hexadecimal usando letras minúsculas
33 printf("%x\n", 255);
34 // Entrada: 255
35 // Salida: ff
36
37 // %X -> hexadecimal usando letras mayúsculas
38 printf("%X\n", 255);
39 // Entrada: 255
40 // Salida: FF
41
```

```

42 // %.2f -> numero real con exactamente 2 decimales
43 printf("%.2f\n", 3.14159);
44 // Entrada: 3.14159
45 // Salida: 3.14
46
47 // %.0f -> numero real sin decimales (redondea)
48 printf("%.0f\n", 3.7);
49 // Entrada: 3.7
50 // Salida: 4
51
52 // %e -> notacion cientifica
53 printf("%e\n", 31415.9);
54 // Entrada: 31415.9
55 // Salida: 3.141590e+04
56
57 // %c -> un solo caracter
58 printf("%c\n", 'A');
59 // Entrada: 'A'
60 // Salida: A
61
62 // %s -> cadena de caracteres
63 printf("%s\n", "hola");
64 // Entrada: "hola"
65 // Salida: hola
66
67 // Se pueden combinar varios especificadores
68 printf("%d %lld %.2f\n", 42, 10000000000LL, 3.14159);
69 // Entrada: 42, 10000000000, 3.14159
70 // Salida: 42 10000000000 3.14
71
72 // %% -> imprime un '%' literal
73 printf("100%%\n");
74 // Entrada: no recibe ningun argumento para %
75 // Salida: 100%
76
77 // ! NO uses %n en C/C++ (puede ser peligroso)
78 // Para scanf, un double se lee con %lf

```

Estructura de un especificador

Un formato puede combinar varias partes:

```

1 // % [flags] [width] [.precision] [specifier]
2 // % 0 5 .2 f
3
4 printf("%05d\n", 42);
5 // % -> comienza el formato
6 // 0 -> rellena con ceros
7 // 5 -> ancho minimo de 5 caracteres
8 // d -> interpreta el argumento como int
9 //
10 // Salida: 00042

```

Los modificadores más utilizados en programación competitiva son:

Formato	Tipo	Ejemplo de salida
%d	int	42
%lld	long long	1234567890123
%c	char	A
%s	string/char*	hola
%f	double	3.140000
%.2f	double	3.14
%x	int	ff
%X	int	FF

Regla práctica: en programación competitiva, los formatos más importantes son %d para int, %lld para long long, %c para caracteres, %s para cadenas y %.2f (o una precisión diferente) para números reales.

1.3 Condicionales y Ciclos

```

1 // if / else: ejecuta un bloque segun una condicion
2 if (x > 0) cout << "pos"; // x > 0 -> "pos"
3 else if (x < 0) cout << "neg"; // x < 0 -> "neg"
4 else cout << "cero"; // x == 0 -> "cero"
5
6 // Operador ternario: condicion ? valor_si_true : valor_si_false
7 string s = (x >= 0) ? "no neg" : "neg";
8 // Si x >= 0, s = "no neg"; si no, s = "neg"

```

```

9
10 // switch: compara una expresion con diferentes valores
11 // Solo funciona directamente con tipos integrales como int o
   char,
12 // NO con string.
13 switch (x) {
14     case 1: /*...*/ break; // x == 1
15     case 2: case 3: /*...*/ break; // x == 2 o x == 3
16     default: /*...*/; // ningun case coincide
17 }
18 // ! Sin break, se continua ejecutando el siguiente case (fall-through)
19
20 // C++17: se puede declarar una variable dentro del if
21 if (int r = x % 2; r == 0) {}
22 // r solo existe dentro del if y sus bloques asociados
23
24 // for clasico: usa un indice y se repite mientras i < n
25 for (int i = 0; i < n; i++) {}
26
27 // range-based for: recorre directamente los elementos
28 // No proporciona el indice.
29 for (int v : datos) {}
30
31 // & obtiene una referencia al elemento original.
32 // Por tanto, las modificaciones afectan al vector.
33 for (int &v : datos) v *= 2;
34
35 // while: ejecuta mientras la condicion sea verdadera
36 while (c < n) c++;
37
38 // do-while: primero ejecuta el bloque y DESPUES verifica la
   condicion.
39 // Por eso se ejecuta AL MENOS una vez.
40 do {
41     c--;
42 } while (c > 0);
43
44 // break -> termina inmediatamente el ciclo o switch
45 // continue -> salta a la siguiente iteracion del ciclo
46
47 // ! C++ NO tiene break con etiqueta.
48 // Para salir de ciclos anidados se puede usar una bandera,
49 // encapsular la logica en una funcion y usar return, etc.

```

1.4 Conversiones, Parseos y Casteos

```

1 // string -> numero (parseo)
2 // Convierte una cadena de texto en el tipo numerico indicado.
3 int e = stoi("42");
4 // Entrada: "42" -> Salida: 42
5
6 long long l = stoll("10000000000");
7 // Entrada: "10000000000" -> Salida: 10000000000
8
9 double d = stod("3.14");
10 // Entrada: "3.14" -> Salida: 3.14
11
12 // Tambien se puede indicar la base numerica.
13 int bin = stoi("1010", nullptr, 2);
14 // "1010" en base 2 -> 10
15
16 int hex = stoi("ff", nullptr, 16);
17 // "ff" en base 16 -> 255
18
19
20 // numero -> string
21 // Convierte cualquier numero compatible en texto.
22 string s = to_string(42);
23 // Entrada: 42 -> Salida: "42"
24
25
26 // Casteo: fuerza la conversion de un tipo a otro.
27 // Al pasar de decimal a entero, se TRUNCa hacia 0.
28 int t = (int) 3.99;
29 // 3.99 -> 3
30
31 int r = static_cast<int>(3.99);
32 // 3.99 -> 3
33 // static_cast<int> es la forma recomendada en C++
34
35
36 // ! int / int = division ENTERA
37 double mal = 7 / 2;
38 // 7 y 2 son int -> 3
39 // Luego 3 se convierte a double -> 3.0
40
41 double bien = 7 / 2.0;
42 // 2.0 es double -> division decimal
43 // 7 / 2.0 -> 3.5
44

```

```

45 // caracteres
46 // Los char tienen un valor numerico asociado (ASCII).
47 int cod = (int) 'A';
48 // 'A' -> 65
49
50
51 char ch = (char) 66;
52 // 66 -> 'B'
53
54 // Truco muy usado para convertir un digito char a int.
55 int dig = '7' - '0';
56 // '7' tiene valor 55 y '0' tiene valor 48
57 // 55 - 48 -> 7

```

Parte II Estructuras de Datos

2 Basic Structures

```

1 int a[5] = {1,2,3,4,5}; // crear con valores: O(n)
2
3 int b[5];                // crear de tamaño 5
4                          // posiciones: b[0] ... b[4]
5                          // ! Valores no inicializados
6
7 b[0] = 10;               // INSERT: O(1)
8 b[1] = 20;               // INSERT: O(1)
9
10 int x = b[0];            // READ: O(1)
11
12 b[0] = 30;               // UPDATE: O(1)
13
14 for (int i = 0; i < 5; i++)
15     cout << b[i] << " "; // READ: recorrer -> O(n)
16
17 // DELETE: no existe metodo para eliminar
18 // Se puede sobrescribir o manejar logicamente
19 b[0] = 0;                // DELETE logico: O(1)
20
21 // ! El tamaño de un array estatico no cambia.
22 // Para mas posiciones se necesita otro arreglo.

```

2.1 Matrices (Arreglos 2D)

Arreglo bidimensional (filas $\times$ columnas). **Complejidad:** acceso $[i][j]$ $O(1)$; crear $n \times m$ en $O(nm)$.

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // CREATE: O(n*m)
2 // matriz 3x4, todas las posiciones inicializadas en 0
3
4 m[1][2] = 7;             // INSERT: O(1)
5
6 int x = m[1][2];         // READ: O(1)
7
8 m[1][2] = 10;            // UPDATE: O(1)
9
10 for (int i = 0; i < 3; i++)
11     for (int j = 0; j < 4; j++)
12         cout << m[i][j] << " ";
13 // READ: recorrer toda la matriz -> O(n*m)
14
15 // DELETE: no existe metodo para eliminar una posicion.
16 // Se puede sobrescribir el valor o manejarlo logicamente.
17 m[1][2] = 0;             // DELETE logico: O(1)
18
19 // ! El tamaño de la matriz puede cambiar con vector,
20 // pero un arreglo 2D estatico no puede cambiar de tamaño.

```

2.2 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice $O(1)$; agregar al final $O(1)$ amortizado; insertar/borrar en medio $O(n)$; buscar $O(n)$.

```

1 vector<int> v;            // CREATE: lista vacia
2
3 v.push_back(3);          // INSERT: final -> O(1) amortizado
4 v.push_back(7);          // INSERT: final -> O(1) amortizado
5
6 int x = v[0];            // READ: acceso -> O(1)

```

```

7 int n = v.size();        // READ: tamaño -> O(1)
8
9 v[0] = 10;               // UPDATE: modificar -> O(1)
10
11 v.insert(v.begin() + 1, 5); // INSERT: medio -> O(n)
12
13 v.erase(v.begin() + 1);  // DELETE: medio -> O(n)
14
15 // Buscar un elemento recorriendo el vector
16 for (int x : v)
17     if (x == 10) { /* ... */ }
18 // SEARCH: recorrer -> O(n)
19
20 // DELETE: eliminar el ultimo elemento
21 v.pop_back();            // DELETE: final -> O(1)
22
23 // ! vector mantiene acceso por índice O(1).
24 // ! Al crecer puede realocar su memoria internamente.

```

2.3 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos** $O(1)$; acceso por índice $O(n)$; buscar $O(n)$. Útil como cola o deque.

```

1 list<int> l;              // CREATE: lista vacia
2
3 l.push_front(1);          // INSERT: inicio -> O(1)
4 l.push_back(9);           // INSERT: final -> O(1)
5
6 int x = l.front();        // READ: primer elemento -> O(1)
7 int y = l.back();         // READ: ultimo elemento -> O(1)
8
9 l.front() = 5;            // UPDATE: primer elemento -> O(1)
10 l.back() = 10;           // UPDATE: ultimo elemento -> O(1)
11
12 l.pop_front();            // DELETE: inicio -> O(1)
13 l.pop_back();            // DELETE: final -> O(1)
14
15 // ! No existe acceso directo por índice.
16 // Avanzar hasta una posicion requiere recorrer nodos.
17 auto it = l.begin();
18 advance(it, 2);           // acceso al tercer elemento -> O(n)
19
20 // Buscar un elemento requiere recorrer la lista.
21 for (int x : l)
22     if (x == 10) { /* ... */ }
23 // SEARCH: recorrer -> O(n)
24
25 // ! list no permite l[i].
26 // Para acceso por índice, vector suele ser mejor.

```

2.4 Pilas (Stack)

Estructura **LIFO** (último en entrar, primero en salir). **Complejidad:** insertar, eliminar y consultar el tope $O(1)$. Para DFS iterativo, paréntesis balanceados y deshacer.

```

1 stack<int> pila;         // CREATE: pila vacia
2
3 pila.push(3);            // INSERT: O(1)
4 pila.push(7);            // INSERT: O(1)
5
6 int tope = pila.top();    // READ: consultar tope -> O(1)
7
8 pila.top() = 10;         // UPDATE: modificar tope -> O(1)
9
10 pila.pop();              // DELETE: eliminar tope -> O(1)
11
12 // ! Solo se puede acceder directamente al elemento superior.
13 // ! No existe acceso por índice.

```

2.5 Colas (Queue)

Estructura **FIFO** (primero en entrar, primero en salir). **Complejidad:** encolar y desencolar $O(1)$. Base del BFS.

```

1 queue<int> cola;         // CREATE: cola vacia
2
3 cola.push(3);            // INSERT: final -> O(1)

```

```

4 cola.push(7);           // INSERT: final -> O(1)
5
6 int frente = cola.front(); // READ: primero -> O(1)
7 int atrás = cola.back();  // READ: último -> O(1)
8
9 cola.front() = 10;       // UPDATE: modificar primero -> O(1)
10
11 cola.pop();             // DELETE: primero -> O(1)
12
13 // ! push agrega al final y pop elimina del frente.
14 // ! No existe acceso directo por índice.

```

2.6 Colas de prioridad (Priority Queue / Heap)

Devuelve siempre el elemento de mayor (o menor) prioridad. **Complejidad:** insertar y extraer $O(\log n)$; consultar el tope $O(1)$. Para Dijkstra, Prim, k-ésimo mayor.

```

1 priority_queue<int> pq; // CREATE: MAX-heap por defecto
2
3 pq.push(3);           // INSERT: O(log n)
4 pq.push(7);           // INSERT: O(log n)
5
6 int mx = pq.top();     // READ: mayor -> O(1)
7
8 pq.pop();             // DELETE: mayor -> O(log n)
9
10 // min-heap: el menor queda en el tope
11 priority_queue<int, vector<int>, greater<int>> pqMin;
12
13 pqMin.push(3);        // INSERT: O(log n)
14 int mn = pqMin.top(); // READ: menor -> O(1)
15 pqMin.pop();          // DELETE: O(log n)
16
17 // ! No existe acceso directo a elementos intermedios.
18 // ! priority_queue no tiene una operacion UPDATE directa.

```

2.7 Comparadores en C++ (Comparators)

Idea: un comparador es una función (lambda, functor, o `operator<` sobrecargado) que le dice a `sort`, `set`, `map`, `priority_queue`, etc. **cómo ordenar** dos elementos. Debe cumplir la propiedad de *orden estricto débil* (*strict weak ordering*): devolver `true` si el primer argumento va **estrictamente antes** que el segundo, y `false` en cualquier otro caso —incluyendo cuando se consideran “iguales” para efectos de orden. El error más común es usar `<=` en vez de `<`: eso rompe la propiedad y produce comportamiento indefinido (desde resultados mal ordenados hasta crashes).

¿Por qué usarlo? El orden por defecto casi nunca alcanza: necesitas ordenar por varios criterios a la vez, en orden descendente, por una clave derivada (no el valor mismo), o con un `struct`/`pair` propio que no debe usar el orden lexicográfico por defecto.

Casos de uso:

- Ordenar por múltiples campos (ej. por costo, y en empate por índice original).
- `priority_queue` como min-heap en vez de max-heap (o con un orden custom sobre un `struct`).
- `set`/`map` con orden propio sobre un tipo definido por ti.
- Ordenar **índices** según los valores de otro arreglo, sin mover el arreglo original.

Complejidad: el comparador en sí no cambia la complejidad del algoritmo que lo usa (`sort` sigue siendo $O(n \log n)$); pero un comparador que no sea un orden estricto débil puede producir UB o bucles infinitos.

Comparador con lambda para sort (multi-criterio, índices)

```

1 struct Persona {
2     string nombre;
3     int edad;
4 };
5
6 vector<Persona> personas = {
7     {"Ana", 30}, {"Luis", 25}, {"Eva", 25}
8 };
9
10 // ordena por edad ascendente; en empate, por nombre alfabético
11 sort(personas.begin(), personas.end(),
12     [](const Persona& a, const Persona& b) {
13         if (a.edad != b.edad)
14             return a.edad < b.edad;
15         return a.nombre < b.nombre;
16     });
17 // resultado: Eva(25), Luis(25), Ana(30)
18
19 // ordenar en orden DESCENDENTE: solo invierte la comparación
20 sort(personas.begin(), personas.end(),
21     [](const Persona& a, const Persona& b) {
22         return a.edad > b.edad;
23     });
24 // resultado: Ana(30), luego Luis/Eva(25) en su orden relativo
    previo

```

Ordenar índices en vez del arreglo (muy común):

```

1 vector<int> valores = {40, 10, 30, 20};
2
3 vector<int> idx(valores.size());
4 iota(idx.begin(), idx.end(), 0); // idx = [0, 1, 2, 3]
5
6 sort(idx.begin(), idx.end(),
7     [&](int a, int b) { return valores[a] < valores[b]; });
8 // idx queda [1, 3, 2, 0]
9 // (valores[1]=10, valores[3]=20, valores[2]=30, valores[0]=40)
10
11 for (int i : idx)
12     cout << valores[i] << " ";
13 // imprime: 10 20 30 40
14 // sin haber movido "valores" del lugar

```

operator< dentro de un struct (orden por defecto)

Idea: si sobrecargas `operator<` dentro de tu propio tipo, ese es el orden que usan automáticamente `sort`, `set`, `map` (como clave) y `priority_queue` **sin pasar ningún comparador extra**.

```

1 struct Punto {
2     int x, y;
3
4     bool operator<(const Punto& otro) const {
5         if (x != otro.x) return x < otro.x;
6         return y < otro.y;
7     }
8 };
9
10 set<Punto> s;
11 s.insert({3, 4});
12 s.insert({1, 2});
13 s.insert({1, 5});
14
15 for (auto& p : s)
16     cout << "(" << p.x << ", " << p.y << ") ";
17 // imprime en orden: (1,2) (1,5) (3,4)
18 // -> set los ordena automáticamente usando operator<

```

priority_queue con comparador custom

Idea: `priority_queue` es un **max-heap** por defecto (el elemento “mayor” según el comparador queda arriba). Para un min-heap con tipos simples, se usa `greater<T>`. Para un `struct` propio con una regla de prioridad distinta a `operator<`, se pasa un **functor** (struct con `operator()`).

```

1 // min-heap de enteros: el menor queda arriba
2 priority_queue<int, vector<int>, greater<int>> minHeap;
3 minHeap.push(5);
4 minHeap.push(1);
5 minHeap.push(3);
6
7 cout << minHeap.top();
8 // 1 (el menor, no el mayor)

```

```

1 struct Persona {
2     string nombre;
3     int edad;
4 };
5
6 // functor: queda arriba la persona con MENOR edad
7 // (priority_queue es max-heap, así que se invierte el sentido:
8 // operator() debe devolver true si "a" tiene MENOR prioridad que
9 // "b")
10 struct CompararPorEdad {
11     bool operator()(const Persona& a, const Persona& b) const {
12         return a.edad > b.edad;
13     }
14 };
15 priority_queue<Persona, vector<Persona>, CompararPorEdad> pq;
16 pq.push({"Ana", 30});
17 pq.push({"Luis", 20});
18 pq.push({"Eva", 25});
19
20 cout << pq.top().nombre;
21 // "Luis" (la persona mas joven queda en la cima)

```

Reglas de orden estricto débil (*strict weak ordering*)

Todo comparador `cmp(a, b)` debe cumplir:

- **Irreflexividad:** `cmp(a, a)` siempre `false`. Un elemento nunca va estrictamente antes que sí mismo.
- **Asimetría:** si `cmp(a, b)` es `true`, entonces `cmp(b, a)` debe ser `false`.
- **Transitividad:** si `cmp(a, b)` y `cmp(b, c)` son `true`, entonces `cmp(a, c)` también debe serlo.
- **Consistencia en los empates:** si ni `cmp(a, b)` ni `cmp(b, a)` son `true`, se consideran “equivalentes” para efectos de orden (aunque no sean idénticos como objetos).

El error más común:

```

1 // INCORRECTO: usa <= en vez de <
2 sort(v.begin(), v.end(), [](int a, int b) {
3     return a <= b; // viola irreflexividad: cmp(a,a) da true
4 });
5 // esto es Undefined Behavior: en el mejor caso ordena mal,
6 // en el peor caso crashea o hace un bucle infinito.
7
8 // CORRECTO:
9 sort(v.begin(), v.end(), [](int a, int b) {
10     return a < b;
11 });

```

Otro error común: comparadores que no son transitivos al combinar varios criterios sin desempate consistente (ej. comparar solo por `a.x < b.x` cuando a veces `a.x == b.x` pero querías tratar esos casos con otro criterio). Si dos elementos “empatan” en el primer criterio, siempre hay que **decidir** con un segundo criterio, o dejarlos expresamente como equivalentes.

Regla práctica: si tu comparador tiene la forma `return a.campo1 < b.campo1;` y sospechas que puede haber empates que te importan, agrega el siguiente criterio explícitamente (`if (a.campo1 != b.campo1) return ...; return a.campo2 < b.campo2;`) en vez

de dejar el desempate al azar del algoritmo de ordenamiento (que no es estable a menos que uses `stable_sort`).

2.8 Vector circular (Circular Buffer)

Idea: un arreglo de tamaño fijo (**cap**) que se comporta como una **cola doble**. En lugar de mover elementos cuando se inserta o elimina, se reutilizan las posiciones del arreglo de forma circular.

Se mantienen tres variables principales:

- **cap:** capacidad total del arreglo.
- **ini:** posición física donde comienza el primer elemento.
- **tam:** cantidad actual de elementos almacenados.

La posición lógica i no coincide necesariamente con la posición física del arreglo. Se obtiene mediante:

$$\text{posicionFisica}(i) = (\text{ini} + i) \% \text{cap}$$

Por ejemplo, con `cap = 5` y `ini = 3`:

Índice lógico	0	1	2	3	4
Índice físico	3	4	0	1	2

Cuando se llega al final del arreglo, el módulo hace que la siguiente posición vuelva al comienzo. Por eso **no es necesario mover los elementos**.

¿Por qué usarlo? En un arreglo normal, insertar al frente requiere desplazar todos los elementos y cuesta $O(n)$. En un vector circular solo se mueve `ini`, por lo que cuesta $O(1)$.

Casos de uso:

- Colas de tamaño fijo.
- Buffers de audio, video o red.
- Ventanas deslizantes.
- Sistemas donde llegan y salen datos continuamente.
- Simulaciones de procesos en una cola.
- Mantener únicamente los últimos k elementos.

Si el buffer está lleno y se ejecuta `pushBack`, el nuevo elemento **reemplaza al más antiguo**. Esto permite mantener automáticamente una ventana de tamaño fijo.

Complejidad: insertar, eliminar, consultar extremos, acceder por posición y comprobar el tamaño son $O(1)$.

```

1 struct VectorCircular {
2     vector<int> buf;
3     int cap, ini = 0, tam = 0;
4
5     VectorCircular(int c) : buf(c), cap(c) {}
6
7     // CREATE: crea un buffer con capacidad fija.
8     // Los elementos inicialmente no están ocupados.
9
10    bool empty() {
11        return tam == 0;
12    } // O(1)
13
14    bool full() {
15        return tam == cap;
16    } // O(1)
17

```

```

18 int size() {
19     return tam;
20 } // O(1)
21
22 int capacity() {
23     return cap;
24 } // O(1)
25
26 // Convierte una posicion logica en fisica.
27 int pos(int i) {
28     return (ini + i) % cap;
29 } // O(1)
30
31 // INSERT: agrega al final.
32 void pushBack(int x) {
33     buf[(ini + tam) % cap] = x;
34
35     if (tam < cap)
36         tam++;
37     else
38         ini = (ini + 1) % cap;
39 } // O(1)
40
41 // INSERT: agrega al inicio.
42 void pushFront(int x) {
43     ini = (ini - 1 + cap) % cap;
44     buf[ini] = x;
45
46     if (tam < cap)
47         tam++;
48 } // O(1)
49
50 // READ: consulta el primer elemento.
51 int front() {
52     return buf[ini];
53 } // O(1)
54
55 // READ: consulta el ultimo elemento.
56 int back() {
57     return buf[(ini + tam - 1) % cap];
58 } // O(1)
59
60 // READ: acceso por posicion logica.
61 int at(int i) {
62     return buf[(ini + i) % cap];
63 } // O(1)
64
65 // UPDATE: modifica un elemento.
66 void set(int i, int x) {
67     buf[(ini + i) % cap] = x;
68 } // O(1)
69
70 // DELETE: elimina el primer elemento.
71 int popFront() {
72     int v = buf[ini];
73     ini = (ini + 1) % cap;
74     tam--;
75     return v;
76 } // O(1)
77
78 // DELETE: elimina el ultimo elemento.
79 int popBack() {
80     tam--;
81     return buf[(ini + tam) % cap];
82 } // O(1)
83
84 // DELETE: elimina todos los elementos.
85 void clear() {
86     ini = 0;
87     tam = 0;
88 } // O(1)
89
90 // SEARCH: busca un elemento.
91 // A diferencia de las operaciones anteriores,
92 // hay que recorrer los elementos.
93 bool contains(int x) {
94     for (int i = 0; i < tam; i++)
95         if (at(i) == x)
96             return true;
97
98     return false;
99 } // O(n)
100
101 // READ: recorrer los elementos en orden logico.
102 void print() {
103     for (int i = 0; i < tam; i++)
104         cout << at(i) << " ";
105 } // O(n)
106 };

```

Ejemplo de funcionamiento:

```
1 VectorCircular v(5);
```

```

2 v.pushBack(10);
3 v.pushBack(20);
4 v.pushBack(30);
5 // [10, 20, 30, _, _]
6 // ini = 0, tam = 3
7
8 v.pushFront(5);
9 // [5, 10, 20, 30, _]
10 // ini cambia de 0 a 4
11
12 cout << v.front();
13 // 5
14
15 cout << v.back();
16 // 30
17
18 v.popFront();
19 // elimina 5
20 // [_ , 10, 20, 30, _]
21
22 v.pushBack(40);
23 v.pushBack(50);
24 // [_ , 10, 20, 30, 40, 50]
25 // El almacenamiento puede haber dado la vuelta.
26
27 v.pushBack(60);
28 // El buffer estaba lleno.
29 // Se agrega 60 y se elimina el mas antiguo.
30 // Resultado logico: [20, 30, 40, 50, 60]
31

```

Ventaja principal: el arreglo físico puede verse “desordenado”, pero los elementos siempre tienen un orden lógico determinado por `ini`. Por ejemplo:

```

1 // Memoria fisica:
2 // [40, 50, 20, 30, _]
3 //      ^
4 //      ini = 2
5
6 // Orden logico:
7 // 20 -> 30 -> 40 -> 50

```

No importa dónde estén físicamente los elementos: `at(0)` siempre representa el primero, `at(1)` el segundo, y así sucesivamente.

Diferencia con un vector normal:

Operación	vector	Circular
Agregar al final	$O(1)^*$	$O(1)$
Eliminar al final	$O(1)$	$O(1)$
Agregar al frente	$O(n)$	$O(1)$
Eliminar al frente	$O(n)$	$O(1)$
Acceso por índice	$O(1)$	$O(1)$
Buscar	$O(n)$	$O(n)$

*Amortizado.

Regla práctica: si necesitas una estructura de tamaño fijo donde los elementos entran y salen continuamente por los extremos, un vector circular evita los desplazamientos de un arreglo normal y mantiene las operaciones principales en $O(1)$.

3 Estructuras Hash/Tree

3.1 Conjuntos hash (HashSet)

Colección de elementos **sin duplicados** y sin orden garantizado. En C++ se implementa mediante `unordered_set`. Utiliza una tabla hash, por lo que las operaciones son $O(1)$ en promedio. Es útil cuando solo interesa saber si un elemento existe, sin necesidad de mantener los elementos ordenados.

Complejidad: insertar, buscar y borrar $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_set<int> s;           // CREATE: conjunto vacio
2
3 s.insert(3);                   // INSERT: O(1) prom.
4 s.insert(7);                   // INSERT: O(1) prom.
5 s.insert(3);                   // no se agrega: ya existe
6
7 bool hay = s.count(3);         // SEARCH: O(1) prom.
8 // hay = 1 si existe, 0 si no existe
9
10 auto it = s.find(7);           // SEARCH: O(1) prom.
11 // devuelve iterador al elemento o s.end()
12
13 s.erase(3);                   // DELETE: O(1) prom.
14
15 int n = s.size();              // READ: cantidad -> O(1)
16 bool vacio = s.empty();        // READ: ¿esta vacio? -> O(1)
17
18 // Recorrer todos los elementos
19 for (int x : s)
20     cout << x << " ";
21 // O(n), pero el orden NO esta garantizado.
22
23 // ! No existe acceso por indice: s[0] no es valido.
24 // ! Los elementos repetidos se ignoran.
25 // ! El orden puede cambiar entre ejecuciones.

```

Cuándo usarlo:

- Saber rápidamente si un elemento ya apareció.
- Eliminar duplicados.
- Mantener un conjunto de elementos visitados.
- Detectar elementos repetidos en $O(n)$ promedio.
- BFS/DFS: guardar vértices visitados cuando sea conveniente.

3.2 Mapas / Diccionarios hash

Estructura que almacena pares **clave** $\rightarrow$ **valor**. En C++ se implementa mediante `unordered_map`. Cada clave identifica un valor y **no puede repetirse**. Si se asigna un nuevo valor a una clave existente, se reemplaza el anterior.

Complejidad: insertar, acceder, buscar y borrar por clave $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_map<string,int> m;    // CREATE: mapa vacio
2
3 m["a"] = 1;                    // INSERT: O(1) prom.
4 m["b"] = 2;                    // INSERT: O(1) prom.
5
6 int v = m["a"];                // READ: acceder -> O(1) prom.
7 // v = 1
8
9 m["a"] = 10;                   // UPDATE: O(1) prom.
10 // la clave "a" ahora tiene valor 10
11
12 m.erase("a");                 // DELETE: O(1) prom.
13
14 bool existe = m.count("b");    // SEARCH: O(1) prom.
15 // existe = 1
16
17 auto it = m.find("b");         // SEARCH: O(1) prom.
18 if (it != m.end())
19     cout << it->second;        // valor asociado a "b"
20
21 // Recorrer todas las parejas
22 for (auto [clave, valor] : m)
23     cout << clave << " " << valor;
24 // O(n), sin orden garantizado.

```

Importante: usar `m[clave]` para consultar una clave que no existe puede **crear esa clave** con su valor por defecto. Para comprobar existencia sin insertar, es preferible usar `count()` o `find()`.

```

1 unordered_map<string,int> m;
2
3 int x = m["a"];
4 // Si "a" no existia, se crea:

```

```

5 // "a" -> 0
6 // ! operator[] puede INSERTAR una clave al consultar.
7
8 // count(): comprueba si existe SIN insertar
9 bool existe = m.count("b");
10 // Devuelve 0 si no existe, 1 si existe.
11
12 // find(): busca la clave SIN insertar
13 auto it = m.find("b");
14
15 if (it != m.end())
16     cout << it->second;        // valor asociado a "b"
17 else
18     cout << "No existe";
19
20 // ! count() solo indica si existe.
21 // ! find() permite acceder a la pareja clave -> valor.

```

Uso clásico: contar frecuencias.

```

1 unordered_map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;                // frecuencia de cada elemento
5
6 cout << freq[5];
7 // cantidad de veces que aparece 5

```

Cuándo usarlo:

- Contar frecuencias.
- Asociar identificadores con información.
- Buscar rápidamente un valor a partir de una clave.
- Problemas de conteo y frecuencia.
- Memorizar resultados de estados o funciones.

3.3 Conjuntos ordenados (TreeSet)

Conjunto de elementos **sin duplicados y ordenados**. En C++ se implementa mediante `set`, normalmente como un árbol balanceado.

A diferencia de `unordered_set`, los elementos siempre se mantienen ordenados según el comparador utilizado.

Complejidad: insertar, buscar y borrar $O(\log n)$. Consultar el menor o mayor elemento mediante `begin()` o `rbegin()` es $O(1)$.

```

1 set<int> s;                     // CREATE: conjunto vacio
2
3 s.insert(5);                   // INSERT: O(log n)
4 s.insert(1);                   // INSERT: O(log n)
5 s.insert(3);                   // INSERT: O(log n)
6 s.insert(3);                   // no se agrega: ya existe
7
8 int menor = *s.begin();        // READ: minimo -> O(1)
9 int mayor = *s.rbegin();       // READ: maximo -> O(1)
10
11 auto it = s.find(3);           // SEARCH: O(log n)
12 if (it != s.end())
13     cout << *it;
14
15 s.erase(3);                   // DELETE: O(log n)
16
17 int n = s.size();              // READ: cantidad -> O(1)
18
19 // Recorrer en orden ascendente
20 for (int x : s)
21     cout << x << " ";
22 // O(n)

```

lower_bound y upper_bound:

```

1 set<int> s = {1, 3, 5, 7, 9};
2
3 auto it1 = s.lower_bound(5);
4 // primer elemento >= 5
5 // resultado: 5
6
7 auto it2 = s.upper_bound(5);
8 // primer elemento > 5
9 // resultado: 7

```

Esto permite resolver consultas de **piso y techo** rápidamente:

```
1 // FLOOR: mayor elemento <= x
2 auto it = s.upper_bound(x);
3
4 if (it != s.begin()) {
5     --it;
6     cout << *it;
7 }
8
9 // CEILING: menor elemento >= x
10 auto it2 = s.lower_bound(x);
11
12 if (it2 != s.end())
13     cout << *it2;
```

Cuándo usarlo:

- Cuando se necesitan elementos únicos y ordenados.
- Obtener mínimo o máximo dinámicamente.
- Encontrar el siguiente elemento mayor o igual.
- Encontrar el anterior elemento menor o igual.
- Mantener un conjunto mientras se insertan y eliminan elementos.

Diferencia clave: si solo necesitas saber si existe un elemento, `unordered_set` suele ser más rápido. Si necesitas consultar orden, `lower_bound`, `upper_bound`, mínimo o máximo, `set` es la opción adecuada.

3.4 Mapas ordenados (TreeMap)

Mapa **clave** → **valor** cuyas claves se mantienen ordenadas. En C++ se implementa mediante `map`, usando un árbol balanceado.

Cada clave aparece una sola vez. Los valores pueden repetirse.

Complejidad: insertar, buscar, acceder y borrar por clave $O(\log n)$. `begin()` permite obtener la menor clave en $O(1)$.

```
1 map<string,int> m;           // CREATE: mapa vacío
2
3 m["b"] = 2;                  // INSERT: O(log n)
4 m["a"] = 1;                  // INSERT: O(log n)
5 m["c"] = 3;                  // INSERT: O(log n)
6
7 int v = m["a"];              // READ: O(log n)
8 // v = 1
9
10 m["a"] = 10;                 // UPDATE: O(log n)
11
12 m.erase("b");                // DELETE: O(log n)
13
14 auto it = m.find("c");        // SEARCH: O(log n)
15 if (it != m.end())
16     cout << it->second;
17
18 // Primera pareja: menor clave
19 auto primera = m.begin();     // O(1)
20 cout << primera->first;
21
22 // Última pareja: mayor clave
23 auto ultima = prev(m.end());  // O(1) amortizado
24 cout << ultima->first;
```

Las claves se recorren ordenadas:

```
1 map<int,string> m;
2
3 m[30] = "C";
4 m[10] = "A";
5 m[20] = "B";
6
7 for (auto [clave, valor] : m)
8     cout << clave << " " << valor;
```

```
10 // Salida:
11 // 10 A
12 // 20 B
13 // 30 C
```

`lower_bound` y `upper_bound` también funcionan sobre las claves:

```
1 map<int,string> m;
2 m[10] = "A";
3 m[20] = "B";
4 m[30] = "C";
5
6 auto it = m.lower_bound(15);
7 // primera clave >= 15
8 // apunta a 20
9
10 auto it2 = m.upper_bound(20);
11 // primera clave > 20
12 // apunta a 30
```

Uso clásico: contar frecuencias ordenadas.

```
1 map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;
5
6 // Las claves aparecen en orden creciente.
7 for (auto [x, cantidad] : freq)
8     cout << x << ": " << cantidad << "\n";
```

Cuándo usarlo:

- Cuando se necesitan pares clave-valor ordenados.
- Contar frecuencias y procesarlas en orden.
- Encontrar claves inmediatamente mayores o menores.
- Procesar eventos o valores manteniendo un orden dinámico.
- Cuando se necesitan operaciones de árbol como `lower_bound` y `upper_bound`.

Comparación rápida

Estructura	Duplicados	Orden	Buscar
<code>unordered_set</code>	No	No	$O(1)$ prom.
<code>unordered_map</code>	Claves: No	No	$O(1)$ prom.
<code>set</code>	No	Sí	$O(\log n)$
<code>map</code>	Claves: No	Sí	$O(\log n)$

Regla práctica:

- `unordered_set` → ¿Ya existe este elemento?
- `unordered_map` → ¿Qué valor corresponde a esta clave?
- `set` → Elementos únicos + ordenados.
- `map` → Claves → valores + claves ordenadas.

4 Trees

5 Range Queries

5.1 Casos de uso

Cuando el problema te pide:	Entonces usa:	Complejidad
Suma de rango sobre un arreglo que nunca cambia , con muchas consultas (ej. “suma de ventas entre el día l y r ”)	Prefix Sums	Build $O(n)$ Query $O(1)$
Suma de rango con actualizaciones puntuales frecuentes (ej. “suma el stock de la posición i , luego consulta un rango”)	Fenwick Tree (BIT)	Build $O(n)$ Query/Update $O(\log n)$
Sumar v a todo un rango $[l, r]$ y también consultar la suma de cualquier rango	FenwickRangeUpdate (doble BIT)	Query/Update $O(\log n)$
Range update + range query con operaciones más complejas: min/max en rango, asignar un valor a todo un rango, contar ceros, etc.	Segment Tree + Lazy Propagation	Query/Update $O(\log n)$
Mínimo, máximo o gcd de cualquier rango sobre un arreglo estático (ej. RMQ clásico, LCA) Query $O(1)$	Sparse Table	Build $O(n \log n)$
Mínimo o máximo de rango cuando el arreglo sí cambia con el tiempo	Segment Tree	Query/Update $O(\log n)$
Consultas 2D: cuántos puntos o cuánta suma hay dentro de un rectángulo $[x_1, x_2] \times [y_1, y_2]$	Fenwick 2D	Query/Update $O(\log n \log m)$
Encontrar el k -ésimo elemento, o “menor índice cuya suma acumulada alcanza X ” (frecuencias)	Fenwick con lowerBound	$O(\log n)$
Muchas queries de rango offline (las conoces todas de antemano) sobre un arreglo estático o con pocos updates	Mo's Algorithm	$O((n + q)\sqrt{n})$
LCA entre dos nodos, o mínimo en rango sobre un recorrido Euler de un árbol (valores consecutivos difieren en ± 1)	RMQ ± 1 (Euler Tour Sparse Table)	Build $O(n)$ Query $O(1)$
Cuántos elementos en el rango $[l, r]$ están dentro de un rango de valores $[a, b]$, o cuál es el k -ésimo menor en $[l, r]$	Wavelet Tree	Query $O(\log n)$
Queries o updates sobre caminos o subárboles de un árbol (ej. “suma el camino de u a v ”)	Heavy-Light Decomposition (+ Segment/Fenwick)	Query/Update $O(\log^2 n)$
Necesitas algo simple de implementar bajo presión, con balance razonable entre update y query, sin memorizar mucha teoría	Sqrt Decomposition	Query/Update $O(\sqrt{n})$

Table 2: Guía rápida: qué estructura de range queries usar según el problema, con su complejidad.

5.2 Prefix Sums

Idea: si un arreglo **no cambia**, puedes precalcular $\text{pref}[i] = \text{suma de los primeros } i \text{ elementos}$ en $O(n)$, una sola vez. Cualquier suma de rango $[l, r]$ se responde restando dos prefijos: $\text{pref}[r+1] - \text{pref}[l]$. Es la base conceptual de todo lo demás en esta sección: Fenwick y Segment Tree existen justamente para soportar lo que Prefix Sums no puede (actualizaciones), a costa de más complejidad de código.

¿Por qué usarlo? Es lo más simple y rápido posible para consultas de rango cuando el arreglo es estático. Cero estructuras de datos elaboradas, cero recursión, solo un arreglo auxiliar y una resta.

Casos de uso:

- Suma de rango sobre un arreglo que nunca se modifica, con muchas queries.

- Suma de submatriz (versión 2D) sobre una matriz estática.
- Aplicar B actualizaciones de rango **offline** y consultar el arreglo final una sola vez (Difference Array).
- Como paso final de un precómputo de teoría de números (criba de ϕ , μ , número de divisores, etc.) cuando el problema pide sumas sobre esos valores en un rango.

Complejidad: construir $O(n)$ (u $O(nm)$ en 2D), consulta $O(1)$. No soporta actualizaciones puntuales eficientes: modificar un elemento y volver a consultar obliga a reconstruir, $O(n)$.

Prefix Sum 1D

```

1 struct PrefixSum1D {
2     vector<long long> pref;
3     int n;
4
5     // CREATE: construye los prefijos a partir de a[0..n-1].
6     PrefixSum1D(vector<long long>& a) {
7         n = a.size();
8         pref.assign(n + 1, 0);
9
10        for (int i = 0; i < n; i++)
11            pref[i + 1] = pref[i] + a[i];
12    } // O(n)
13
14    // READ: suma del rango [l, r] (0-indexado, inclusivo).
15    long long rango(int l, int r) {
16        if (l > r) return 0;
17        return pref[r + 1] - pref[l];
18    } // O(1)
19 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 PrefixSum1D ps(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << ps.rango(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << ps.rango(1, 4);
10 // 2 + 9 + 1 + 7 = 19
11
12 cout << ps.rango(3, 3);
13 // 1

```

Prefix Sum 2D

```

1 struct PrefixSum2D {
2     vector<vector<long long>> pref;
3     int n, m;
4
5     // CREATE: construye la matriz de prefijos a partir de
6     // a[0..n-1][0..m-1] usando inclusion-exclusion.
7     PrefixSum2D(vector<vector<long long>>& a) {
8         n = a.size();
9         m = a[0].size();
10        pref.assign(n + 1, vector<long long>(m + 1, 0));
11
12        for (int i = 0; i < n; i++)
13            for (int j = 0; j < m; j++)
14                pref[i + 1][j + 1] = a[i][j] + pref[i][j + 1]
15                + pref[i + 1][j] - pref[i][j];
16    } // O(n * m)
17
18    // READ: suma del rectangulo [x1..x2] x [y1..y2] (0-indexado,
19    // inclusivo), usando inclusion-exclusion sobre los prefijos.
20    long long rango(int x1, int y1, int x2, int y2) {
21        return pref[x2 + 1][y2 + 1] - pref[x1][y2 + 1]
22        - pref[x2 + 1][y1] + pref[x1][y1];
23    } // O(1)
24 };

```

Ejemplo de funcionamiento:

```

1 vector<vector<long long>> a = {
2     {1, 2, 3},
3     {4, 5, 6},
4     {7, 8, 9}
5 };
6
7 PrefixSum2D ps(a);
8
9 cout << ps.rango(0, 0, 1, 1);
10 // 1+2+4+5 = 12
11
12 cout << ps.rango(1, 1, 2, 2);
13 // 5+6+8+9 = 28

```

Difference Array (updates de rango offline)

Idea: el truco inverso a Prefix Sums: para sumar v a todo un rango $[l, r]$, marcas $\text{diff}[l] += v$ y $\text{diff}[r+1] -= v$ en $O(1)$. Después de aplicar **todas** las actualizaciones, un solo recorrido con prefijos reconstruye el arreglo final. Sirve solo cuando todas las updates son **offline** (las conoces antes de necesitar consultar); si necesitas consultar *entre* actualizaciones, usa Fenwick-RangeUpdate en su lugar.

```

1 struct DifferenceArray {
2     vector<long long> diff;
3     int n;
4
5     // CREATE: reserva un arreglo de diferencias de tamaño n.
6     DifferenceArray(int n) : n(n) {
7         diff.assign(n + 1, 0);
8     }
9
10    // UPDATE: marca +v en todo el rango [l, r] (0-indexado).
11    void rangoUpdate(int l, int r, long long v) {
12        diff[l] += v;
13        if (r + 1 <= n)
14            diff[r + 1] -= v;
15    }
16
17    // READ: reconstruye el arreglo final tras aplicar todas
18    // las actualizaciones marcadas hasta el momento.
19    vector<long long> aplicar() {
20        vector<long long> a(n);
21        long long acc = 0;
22
23        for (int i = 0; i < n; i++) {
24            acc += diff[i];
25            a[i] = acc;
26        }
27
28        return a;
29    }
30 };
31
32 // 0(1)
33
34 // 0(n)

```

Ejemplo de funcionamiento:

```

1 DifferenceArray d(5);
2
3 d.rangoUpdate(1, 3, 4);
4 d.rangoUpdate(0, 4, 1);
5 d.rangoUpdate(2, 2, 10);
6
7 vector<long long> resultado = d.aplicar();
8 // resultado: [1, 5, 15, 5, 1]

```

Aplicación: suma de Euler Totient (ϕ) en un rango

Idea: $\phi(n)$ cuenta cuántos enteros en $[1, n]$ son coprimos con n . Calcular ϕ de un número es teoría de números ($O(\sqrt{n})$ factorizando); pero cuando el problema pide la **suma de $\phi(i)$ para i en un rango $[l, r]$** , con muchas queries, el patrón se resuelve en dos fases: (1) precalculas el arreglo de ϕ una sola vez con una criba (teoría de números), y (2) respondes las queries de rango con Prefix Sums sobre ese arreglo ya calculado (exactamente esta

subsección). Factorizar cada número del rango por separado en cada query sería $O(q\sqrt{n})$ y no escala; precalcular y sumar con prefijos reduce cada query a $O(1)$.

Criba lineal + Prefix Sums (rango $[1, N]$).

```

1 struct PhiRangeSum {
2     vector<long long> phi, pref;
3     int n;
4
5     // CREATE: calcula phi[1..n] con criba lineal y sus prefijos.
6     // phi[i] = cantidad de enteros en [1, i] coprimos con i.
7     PhiRangeSum(int n) : n(n) {
8         phi.assign(n + 1, 0);
9         pref.assign(n + 1, 0);
10
11        vector<int> primos;
12        vector<bool> compuesto(n + 1, false);
13        phi[1] = 1;
14
15        for (int i = 2; i <= n; i++) {
16            if (!compuesto[i]) {
17                primos.push_back(i);
18                phi[i] = i - 1;
19            }
20
21            for (int p : primos) {
22                if ((long long)i * p > n) break;
23
24                compuesto[i * p] = true;
25
26                if (i % p == 0) {
27                    phi[i * p] = phi[i] * p;
28                    break;
29                } else {
30                    phi[i * p] = phi[i] * (p - 1);
31                }
32            }
33        }
34
35        for (int i = 1; i <= n; i++)
36            pref[i] = pref[i - 1] + phi[i];
37    }
38
39    // READ: suma de phi(i) para i en [l, r].
40    long long rango(int l, int r) {
41        if (l > r) return 0;
42        return pref[r] - pref[l - 1];
43    }
44
45    // READ: valor individual de phi(i).
46    long long get(int i) {
47        return phi[i];
48    }
49 };
50
51 // 0(n)
52
53 // 0(1)

```

Ejemplo de funcionamiento:

```

1 PhiRangeSum pr(10);
2 // phi(1..10) = [1, 1, 2, 2, 4, 2, 6, 4, 6, 4]
3
4 cout << pr.get(7);
5 // phi(7) = 6 (7 es primo)
6
7 cout << pr.rango(1, 10);
8 // 1+1+2+2+4+2+6+4+6+4 = 32
9
10 cout << pr.rango(4, 8);
11 // 2+4+2+6+4 = 18

```

Criba segmentada (rango $[L, R]$ grande). Si R es muy grande (10^{12}) pero $R - L$ es manejable, no puedes cribar desde 1. En vez de eso, precalculas los primos hasta $\sqrt{R}$, y para cada número del segmento $[L, R]$ le quitas sus factores primos pequeños, aplicando $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$. Si al final queda un residuo > 1 , es un único primo grande (mayor a $\sqrt{R}$) que también se aplica a la fórmula.

```

1 struct PhiSegmentedRangeSum {
2     vector<long long> phi, pref;
3     long long L, R;
4
5     // CREATE: calcula phi(i) para cada i en [L, R] factorizando
6     // con

```

```

6 // los primos hasta sqrt(R), y sus prefijos sobre el segmento.
7
8 PhiSegmentedRangeSum(long long L, long long R) : L(L), R(R) {
9     int lim = (int)sqrt((double)R) + 1;
10
11     vector<int> primos;
12     vector<bool> compuesto(lim + 1, false);
13     for (int i = 2; i <= lim; i++) {
14         if (!compuesto[i]) primos.push_back(i);
15         for (int p : primos) {
16             if ((long long)i * p > lim) break;
17             compuesto[i * p] = true;
18             if (i % p == 0) break;
19         }
20     } // O(sqrt(R) log
21     log sqrt(R))
22
23     int sz = R - L + 1;
24     vector<long long> num(sz);
25     phi.assign(sz, 0);
26
27     for (int i = 0; i < sz; i++) {
28         num[i] = L + i;
29         phi[i] = L + i;
30     }
31
32     for (int p : primos) {
33         long long start = ((L + p - 1) / p) * p;
34
35         for (long long j = start; j <= R; j += p) {
36             int idx = j - L;
37             phi[idx] -= phi[idx] / p;
38
39             while (num[idx] % p == 0)
40                 num[idx] /= p;
41         }
42     }
43
44     // lo que quede > 1 es un primo grande (> sqrt(R)).
45     for (int i = 0; i < sz; i++)
46         if (num[i] > 1)
47             phi[i] -= phi[i] / num[i];
48
49     pref.assign(sz + 1, 0);
50     for (int i = 0; i < sz; i++)
51         pref[i + 1] = pref[i] + phi[i];
52 } // O((R-L+1) log
53     log R + sqrt(R))
54
55 // READ: suma de phi(i) para i en [l, r], con L <= l <= r <=
56 // R.
57 long long rango(long long l, long long r) {
58     return pref[r - L + 1] - pref[l - L];
59 } // O(1)
60
61 // READ: valor individual de phi(i), con L <= i <= R.
62 long long get(long long i) {
63     return phi[i - L];
64 } // O(1)
65 }

```

Ejemplo de funcionamiento:

```

1 PhiSegmentedRangeSum ps(95, 105);
2 // calcula phi(95), phi(96), ..., phi(105)
3 // sin cribar desde 1
4
5 cout << ps.get(100);
6 // phi(100) = 40
7
8 cout << ps.rango(95, 105);
9 // suma de phi(95) a phi(105)
10
11 cout << ps.rango(100, 102);
12 // phi(100) + phi(101) + phi(102)

```

Cuál variante usar:

Situación	Estructura	Complejidad
$[1, N]$, N chico/mediano	PhiRangeSum	$O(N)$
$[L, R]$ grande, $R - L$ chico	PhiSegmentedRangeSum	$O((R - L) \log \log R + \sqrt{R})$
ϕ cambia entre queries	Fenwick sobre ϕ	$O(\log N)$

Table 3: Variantes para la suma de ϕ .

Regla práctica: el cálculo de ϕ en sí (la criba, lineal o segmentada) es teoría de números y no cambia según la query. Una vez que tienes el arreglo de ϕ listo, trátalo como *cualquier otro arreglo estático*: Prefix Sums

resuelve el resto. No hace falta una estructura especial para ϕ más allá de construir bien el arreglo inicial.

5.3 Árbol de Fenwick (BIT)

Idea: un arreglo normal da suma de prefijo en $O(1)$ pero actualizar un elemento cuesta $O(n)$ (o al revés si guardas prefijos). El Fenwick equilibra ambas a $O(\log n)$ guardando *sumas parciales inteligentes*: la posición i almacena la suma de un bloque de tamaño $i \& (-i)$ (su bit menos significativo) que termina en i . Para consultar un prefijo saltas hacia atrás quitando el bit bajo ($i -= i \& (-i)$); para actualizar subes sumándolo ($i += i \& (-i)$). Cada camino toca a lo más $\log n$ posiciones (un bit por paso). Es 1-indexado.

¿Por qué usarlo? Es mucho más simple y liviano que un Segment Tree cuando solo necesitas sumas (o cualquier operación invertible) de prefijo/rango con actualizaciones puntuales. Menos memoria, menos código y una constante más pequeña.

Casos de uso:

- Suma de prefijo / suma de rango con updates frecuentes.
- Conteo de inversiones en un arreglo.
- Frecuencias acumuladas y búsqueda del k -ésimo elemento.
- Range update + range query cuando no hace falta un Segment Tree con Lazy Propagation.
- Consultas 2D: conteo/suma de puntos dentro de un rectángulo.

Complejidad: construir $O(n)$, update puntual $O(\log n)$, query de prefijo/rango $O(\log n)$, range update + range query $O(\log n)$ y versión 2D $O(\log n \cdot \log m)$.

BIT clásico (Point Update, Range Query)

```

1 struct FenwickTree {
2     vector<long long> t;
3     int n;
4
5     // CREATE: reserva un arbol de tamaño n (1-indexado).
6     FenwickTree(int n) : t(n + 1, 0), n(n) {}
7
8     // CREATE: construye el arbol a partir de un arreglo a[0..n-1]
9     // propagando cada valor a su "padre" una sola vez, en vez de
10    // hacer n updates (que serian O(n log n)).
11    void build(vector<long long> &a) {
12        for (int i = 1; i <= n; i++) {
13            t[i] += a[i - 1];
14
15            int j = i + (i & (-i));
16
17            if (j <= n)
18                t[j] += t[i];
19        }
20    } // O(n)
21
22    // UPDATE: suma v al elemento en la posición i.
23    void update(int i, long long v) {
24        for (; i <= n; i += i & (-i))
25            t[i] += v;
26    } // O(log n)
27
28    // UPDATE: fija el valor absoluto de la posición i en x.
29    void set(int i, long long x) {
30        update(i, x - get(i));
31    } // O(log n)
32
33    // READ: suma de prefijo [1..i].

```

```

34 long long query(int i) {
35     long long s = 0;
36
37     for (; i > 0; i -= i & (-i))
38         s += t[i];
39
40     return s;
41 } // O(log n)
42
43 // READ: suma del rango [l, r].
44 long long rango(int l, int r) {
45     if (l > r)
46         return 0;
47
48     return query(r) - query(l - 1);
49 } // O(log n)
50
51 // READ: valor puntual en la posición i.
52 long long get(int i) {
53     return rango(i, i);
54 } // O(log n)
55
56 // SEARCH: menor índice i tal que query(i) >= objetivo.
57 // Usa binary lifting sobre el árbol.
58 // Requiere que todos los valores sean >= 0.
59 int lowerBound(long long objetivo) {
60     int pos = 0;
61     long long ac = 0;
62
63     int LOG = 0;
64
65     while ((1 << LOG) <= n)
66         LOG++;
67
68     for (int k = LOG - 1; k >= 0; k--) {
69         int nx = pos + (1 << k);
70
71         if (nx <= n && ac + t[nx] < objetivo) {
72             pos = nx;
73             ac += t[nx];
74         }
75     }
76
77     return pos + 1;
78 } // O(log n)
79
80 // DELETE: reinicia el árbol a ceros.
81 void clear() {
82     fill(t.begin(), t.end(), 0);
83 } // O(n)
84 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 FenwickTree bit(5);
4 bit.build(a);
5
6 // arreglo logico:
7 // [5, 2, 9, 1, 7]
8 // posiciones 1..5
9
10 cout << bit.query(3);
11 // 5 + 2 + 9 = 16
12
13 cout << bit.rango(2, 4);
14 // 2 + 9 + 1 = 12
15
16 bit.update(2, 3);
17
18 // arreglo logico:
19 // [5, 5, 9, 1, 7]
20
21 cout << bit.get(2);
22 // 5
23
24 bit.set(4, 10);
25
26 // arreglo logico:
27 // [5, 5, 9, 10, 7]
28
29 cout << bit.lowerBound(15);
30 // menor índice cuyo prefijo alcanza 15
31 // -> índice 3 (5 + 5 + 9 = 19 >= 15)

```

BIT con Range Update + Range Query (Doble BIT)

Idea: con *dos* Fenwick y el truco de diferencias se puede sumar v a todo un rango $[l, r]$ en $O(\log n)$ y también consultar la suma de cualquier rango en $O(\log n)$. Esto permite resolver simultáneamente actualizaciones y consultas de rango.

```

1 struct FenwickRangeUpdate {
2     FenwickTree b1, b2;
3     int n;
4
5     // CREATE: reserva dos arboles auxiliares de tamaño n.
6     FenwickRangeUpdate(int n) : b1(n), b2(n), n(n) {}
7
8     // UPDATE: suma v a todos los elementos del rango [l, r].
9     void rangoUpdate(int l, int r, long long v) {
10         b1.update(l, v);
11         b1.update(r + 1, -v);
12
13         b2.update(l, v * (1 - 1));
14         b2.update(r + 1, -v * r);
15     } // O(log n)
16
17     // READ: suma de prefijo [1..i].
18     long long prefijo(int i) {
19         return b1.query(i) * i - b2.query(i);
20     } // O(log n)
21
22     // READ: suma del rango [l, r].
23     long long rango(int l, int r) {
24         if (l > r)
25             return 0;
26
27         return prefijo(r) - prefijo(l - 1);
28     } // O(log n)
29
30     // READ: valor puntual en la posición i.
31     long long get(int i) {
32         return rango(i, i);
33     } // O(log n)
34 };

```

Ejemplo de funcionamiento:

```

1 FenwickRangeUpdate f(5);
2 // arreglo logico inicial: [0, 0, 0, 0, 0]
3
4 f.rangoUpdate(2, 4, 3);
5 // arreglo logico: [0, 3, 3, 3, 0]
6
7 cout << f.rango(1, 5);
8 // 0+3+3+3+0 = 9
9
10 f.rangoUpdate(1, 3, 2);
11 // arreglo logico: [2, 5, 5, 3, 0]
12
13 cout << f.get(2);
14 // 5

```

BIT 2D (point update, sum de submatriz)

Idea: un BIT dentro de otro BIT. Cada update/query en x dispara un update/query completo en y , dando $O(\log n \cdot \log m)$ en vez de $O(nm)$ por consulta.

```

1 struct Fenwick2D {
2     vector<vector<long long>>> t;
3     int n, m;
4
5     // CREATE: reserva una matriz de arboles de tamaño n x m.
6     Fenwick2D(int n, int m) : t(n + 1, vector<long long>(m + 1,
7         0)),
8         n(n), m(m) {}
9
10    // UPDATE: suma v a la celda (x, y).
11    void update(int x, int y, long long v) {
12        for (int i = x; i <= n; i += i & (-i))
13            for (int j = y; j <= m; j += j & (-j))
14                t[i][j] += v;
15    } // O(log n * log m)
16
17    // READ: suma de la submatriz [1..x] x [1..y].
18    long long query(int x, int y) {
19        long long s = 0;

```

```

19     for (int i = x; i > 0; i -= i & (-i))
20         for (int j = y; j > 0; j -= j & (-j))
21             s += t[i][j];
22     return s;
23 }                                     // O(log n * log m)
24 )
25 // READ: suma del rectangulo [x1..x2] x [y1..y2].
26 long long rango(int x1, int y1, int x2, int y2) {
27     return query(x2, y2) - query(x1 - 1, y2)
28         - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
29 }                                     // O(log n * log m)
30 };
```

Ejemplo de funcionamiento:

```

1 Fenwick2D f(4, 4);
2
3 f.update(1, 1, 5);
4 f.update(2, 3, 7);
5 f.update(4, 4, 2);
6
7 cout << f.rango(1, 1, 2, 3);
8 // 5 + 7 = 12
9
10 cout << f.rango(3, 3, 4, 4);
11 // 2
```

Diferencia con un arreglo de prefijos simple:

Estructura	Construir	Query	Update
Prefijos simples	$O(n)$	$O(1)$	$O(n)$
FenwickTree	$O(n)$	$O(\log n)$	$O(\log n)$
FenwickRangeUpdate	$O(n)$	$O(\log n)$	$O(\log n)^*$
Fenwick2D	$O(nm)$	$O(\log n \log m)$	$O(\log n \log m)$

Table 4: Comparación de complejidades: Fenwick vs. prefijos simples.

*Range update + range query, con la variante de doble BIT.

Regla práctica: si el arreglo cambia con frecuencia y solo necesitas sumas (o cualquier operación invertible) de prefijo o rango, el Fenwick da el mejor balance entre simplicidad de código y velocidad. Si necesitas min/max, operaciones no invertibles, o updates de rango con lazy propagation general, usa Segment Tree en su lugar.

5.4 Sqrt Decomposition

Idea: divide el arreglo en bloques de tamaño $\approx \sqrt{n}$ y guarda un resumen (suma, mínimo, etc.) por bloque. Un update toca solo un bloque; una query de rango toca a lo más dos bloques *parciales* (recorridos elemento por elemento) y varios bloques *completos* en el medio (usados de golpe con su resumen precalculado). Como hay $\approx \sqrt{n}$ bloques y cada bloque tiene $\approx \sqrt{n}$ elementos, todo cuesta $O(\sqrt{n})$: ni tan lento como un arreglo plano, ni tan rápido como Fenwick/Segment Tree, pero mucho más simple de escribir y de adaptar a operaciones raras.

¿Por qué usarlo? Es la estructura más flexible cuando no tienes tiempo de pensar un Segment Tree con lazy propagation correcto: soporta casi cualquier operación (suma, min, max, contar algo con una condición) con el mismo esqueleto de código, aunque sea $O(\sqrt{n})$ en vez de $O(\log n)$. También es la base conceptual de Mo's Algorithm.

Casos de uso:

- Necesitas implementar algo rápido bajo presión, sin memorizar casos de Segment Tree con lazy propagation.

- Operaciones donde min/max cambian de forma que Fenwick no puede manejar (Fenwick solo sirve para operaciones invertibles como la suma).
- n no es tan grande ($\leq 10^5$ – 10^6) y $O(\sqrt{n})$ por operación es suficiente para el límite de tiempo.
- Base para Mo's Algorithm (ver esa sección).

Complejidad: construir $O(n)$, update puntual $O(1)$ (suma) u $O(\sqrt{n})$ (min/max), query de rango $O(\sqrt{n})$, range update + range query $O(\sqrt{n})$ con lazy por bloque.

Sqrt Decomposition clásico (point update, range query — suma)

```

1 struct SqrtDecomposition {
2     vector<long long> a, bloque;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtDecomposition(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloque.assign((n + tam - 1) / tam, 0);
13
14        for (int i = 0; i < n; i++)
15            bloque[i / tam] += a[i];
16    }                                     // O(n)
17
18    // UPDATE: suma v al elemento en la posición i.
19    void update(int i, long long v) {
20        a[i] += v;
21        bloque[i / tam] += v;
22    }                                     // O(1)
23
24    // READ: suma del rango [l, r] (0-indexado, inclusivo).
25    long long rango(int l, int r) {
26        long long s = 0;
27        int bl = l / tam, br = r / tam;
28
29        if (bl == br) {
30            for (int i = l; i <= r; i++)
31                s += a[i];
32            return s;
33        }
34
35        for (int i = l; i < (bl + 1) * tam; i++)
36            s += a[i];
37
38        for (int b = bl + 1; b < br; b++)
39            s += bloque[b];
40
41        for (int i = br * tam; i <= r; i++)
42            s += a[i];
43
44        return s;
45    }                                     // O(sqrt(n))
46 };
```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtDecomposition sd(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3, 8, 4] (posiciones 0..7)
5 // tam = 2 -> bloques: [5+2, 9+1, 7+3, 8+4] = [7, 10, 10, 12]
6
7 cout << sd.rango(1, 5);
8 // 2 + 9 + 1 + 7 + 3 = 22
9
10 sd.update(3, 10);
11 // arreglo logico: [5, 2, 9, 11, 7, 3, 8, 4]
12
13 cout << sd.rango(1, 5);
14 // 2 + 9 + 11 + 7 + 3 = 32
```

Sqrt Decomposition con range update + range query (lazy por bloque)

Idea: igual que arriba, pero cada bloque guarda un valor lazy pendiente que representa “sumar v a todo el bloque”

sin tocar cada elemento individualmente. Solo se aplica a los elementos reales cuando un update o query **parcial** necesita tocar ese bloque específico.

```

1 struct SqrtRangeUpdate {
2     vector<long long> a, bloqueSum, lazy;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtRangeUpdate(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        int nb = (n + tam - 1) / tam;
13        bloqueSum.assign(nb, 0);
14        lazy.assign(nb, 0);
15
16        for (int i = 0; i < n; i++)
17            bloqueSum[i / tam] += a[i]; // O(n)
18    }
19
20    // READ auxiliar: cantidad de elementos reales del bloque b.
21    int tamBloque(int b) {
22        return min(n, (b + 1) * tam) - b * tam; // O(1)
23    }
24
25    // UPDATE: suma v a todo el rango [l, r].
26    void rangoUpdate(int l, int r, long long v) {
27        int bl = l / tam, br = r / tam;
28
29        if (bl == br) {
30            for (int i = l; i <= r; i++)
31                a[i] += v;
32            bloqueSum[bl] += v * (r - l + 1);
33            return;
34        }
35
36        for (int i = l; i < (bl + 1) * tam; i++)
37            a[i] += v;
38        bloqueSum[bl] += v * ((bl + 1) * tam - l);
39
40        for (int b = bl + 1; b < br; b++)
41            lazy[b] += v;
42
43        for (int i = br * tam; i <= r; i++)
44            a[i] += v;
45        bloqueSum[br] += v * (r - br * tam + 1); // O(sqrt(n))
46    }
47
48    // READ: suma del rango [l, r].
49    long long rango(int l, int r) {
50        long long s = 0;
51        int bl = l / tam, br = r / tam;
52
53        if (bl == br) {
54            for (int i = l; i <= r; i++)
55                s += a[i] + lazy[bl];
56            return s;
57        }
58
59        for (int i = l; i < (bl + 1) * tam; i++)
60            s += a[i] + lazy[bl];
61
62        for (int b = bl + 1; b < br; b++)
63            s += bloqueSum[b] + lazy[b] * tamBloque(b);
64
65        for (int i = br * tam; i <= r; i++)
66            s += a[i] + lazy[br];
67
68        return s; // O(sqrt(n))
69    }
70 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SqrtRangeUpdate sr(a);
4 // tam = 2 -> bloques: [1+1, 1+1, 1+1] = [2, 2, 2]
5
6 sr.rangoUpdate(1, 4, 10);
7 // arreglo logico "efectivo": [1, 11, 11, 11, 11, 1]
8
9 cout << sr.rango(0, 5);
10 // 1+11+11+11+11+1 = 46
11
12 cout << sr.rango(2, 3);
13 // 11+11 = 22

```

Variante para mínimo / máximo (point update, range query)

Idea: a diferencia de Fenwick (que solo sirve para operaciones *invertibles* como la suma), Sqrt Decomposition soporta min/max con el mismo esqueleto: la única diferencia es que un update ya no puede ajustar el resumen del bloque en $O(1)$, porque quitar el valor viejo no dice si sigue habiendo otro elemento igual de chico. Hay que **reconstruir el mínimo de todo el bloque**, lo cual sigue siendo $O(\sqrt{n})$.

```

1 struct SqrtMinQuery {
2     vector<long long> a, bloqueMin;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtMinQuery(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloqueMin.assign((n + tam - 1) / tam, LLONG_MAX);
13
14        for (int i = 0; i < n; i++)
15            bloqueMin[i / tam] = min(bloqueMin[i / tam], a[i]); // O(n)
16    }
17
18    // UPDATE: fija el valor absoluto de la posición i en x.
19    // Reconstruye el mínimo de todo el bloque, ya que a
20    // diferencia
21    // de la suma, el mínimo no se puede ajustar incrementalmente.
22
23    void set(int i, long long x) {
24        a[i] = x;
25
26        int b = i / tam;
27        long long mn = LLONG_MAX;
28
29        for (int j = b * tam; j < min(n, (b + 1) * tam); j++)
30            mn = min(mn, a[j]);
31
32        bloqueMin[b] = mn; // O(sqrt(n))
33    }
34
35    // READ: mínimo del rango [l, r].
36    long long rango(int l, int r) {
37        long long mn = LLONG_MAX;
38        int bl = l / tam, br = r / tam;
39
40        if (bl == br) {
41            for (int i = l; i <= r; i++)
42                mn = min(mn, a[i]);
43            return mn;
44        }
45
46        for (int i = l; i < (bl + 1) * tam; i++)
47            mn = min(mn, a[i]);
48
49        for (int b = bl + 1; b < br; b++)
50            mn = min(mn, bloqueMin[b]);
51
52        for (int i = br * tam; i <= r; i++)
53            mn = min(mn, a[i]);
54
55        return mn; // O(sqrt(n))
56    }
57 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtMinQuery sm(a);
4 // tam = 2 -> bloques min: [2, 1, 3, 4]
5
6 cout << sm.rango(0, 4);
7 // min(5,2,9,1,7) = 1
8
9 sm.set(3, 100);
10 // arreglo logico: [5, 2, 9, 100, 7, 3, 8, 4]
11
12 cout << sm.rango(0, 4);
13 // min(5,2,9,100,7) = 2

```

Comparación con Fenwick y Segment Tree:

Estructura	Update	Query	Min/Max?
Sqrt Decomposition	$O(1) - O(\sqrt{n})$	$O(\sqrt{n})$	Sí
FenwickTree	$O(\log n)$	$O(\log n)$	No
Segment Tree	$O(\log n)$	$O(\log n)$	Sí

Table 5: Sqrt Decomposition vs. Fenwick vs. Segment Tree.

Regla práctica: usa Sqrt Decomposition cuando quieras algo simple y flexible de escribir rápido, sobre todo si la operación no es una suma (Fenwick no sirve) y no quieres arriesgarte con un Segment Tree + Lazy Propagation bajo presión de tiempo. Si n es muy grande o el límite de tiempo es ajustado, $O(\log n)$ de Segment Tree gana; si la operación es solo suma, Fenwick es más simple y más rápido.

5.5 Sparse Table (RMQ)

Idea: responde consultas de rango en $O(1)$ sobre un arreglo que **no cambia**, precalculando $sp[k][i] = \text{el resultado de combinar el bloque que empieza en } i \text{ y tiene largo } 2^k$. La clave está en la consulta: para el rango $[l, r]$ tomas $k = \lfloor \log_2(r - l + 1) \rfloor$ y lo cubres con *dos* bloques de largo 2^k —uno pegado a l y otro pegado a r — que **se solapan**. Si la operación es *idempotente* ($f(x, x) = x$, como min, max o gcd), contar el solape dos veces no cambia el resultado, así que la consulta son solo dos lecturas y una combinación. Para operaciones **no idempotentes** (como suma o xor) hace falta la variante *Disjoint Sparse Table*, que evita el solape por construcción.

¿Por qué usarlo? Es la estructura más rápida posible para consultas de rango ($O(1)$ por query) cuando el arreglo es estático. A cambio de esa velocidad, no soporta actualizaciones: reconstruir cuesta $O(n \log n)$, así que si el arreglo cambia, conviene un Segment Tree.

Casos de uso:

- RMQ clásico: mínimo o máximo de cualquier rango, arreglo estático.
- gcd o AND/OR de rango sobre un arreglo estático.
- LCA vía Euler Tour + RMQ ± 1 .
- Suma, xor o cualquier operación no idempotente de rango estático, en $O(1)$, usando Disjoint Sparse Table.
- Cualquier problema con muchísimas queries offline sobre datos que nunca se modifican.

Complejidad: construir $O(n \log n)$, consulta $O(1)$. (Si el arreglo *cambia*, usa Segment Tree: $O(\log n)$ por operación.)

Sparse Table clásico (operaciones idempotentes: min, max, gcd)

```
1 struct SparseTable {
2     vector<vector<long long>>> sp;
3     vector<int> lg;
4     int n;
5
6     // CREATE: precalcula la tabla a partir de un arreglo a[0..n-1].
7     // sp[k][i] = mínimo del bloque [i, i + 2^k - 1].
8     SparseTable(vector<long long>& a) {
```

```
9         n = a.size();
10
11         lg.assign(n + 1, 0);
12         for (int i = 2; i <= n; i++)
13             lg[i] = lg[i / 2] + 1;
14
15         int K = lg[n] + 1;
16         sp.assign(K, vector<long long>(n));
17         sp[0] = a;
18
19         for (int k = 1; k < K; k++)
20             for (int i = 0; i + (1 << k) <= n; i++)
21                 sp[k][i] = min(sp[k - 1][i],
22                               sp[k - 1][i + (1 << (k - 1))]);
23     }
24
25     // READ: mínimo del rango [l, r] (0-indexado, inclusivo).
26     // Cubre [l, r] con dos bloques de largo 2^k que se solapan;
27     // como min es idempotente, el solape no afecta el resultado.
28     long long query(int l, int r) {
29         int k = lg[r - l + 1];
30         return min(sp[k][l], sp[k][r - (1 << k) + 1]);
31     }
32 };
```

Ejemplo de funcionamiento:

```
1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SparseTable st(a);
4 // arreglo lógico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << st.query(0, 2);
7 // min(5, 2, 9) = 2
8
9 cout << st.query(2, 5);
10 // min(9, 1, 7, 3) = 1
11
12 cout << st.query(4, 4);
13 // min(7) = 7
```

Nota: para max o gcd solo cambia `min(...)` por `max(...)` o `__gcd(...)` en el constructor y en `query`. Si necesitas saber *qué índice* alcanza el mínimo (no solo el valor), guarda índices en vez de valores y compara `a[sp[k][i]]` en lugar de `sp[k][i]`.

Disjoint Sparse Table (operaciones no idempotentes: suma, xor)

Idea: en vez de dos bloques que se solapan, cada nivel k particiona el arreglo en bloques de largo 2^{k+1} y precalcula, para cada bloque, un prefijo hacia la izquierda desde su punto medio y un prefijo hacia la derecha desde ese mismo punto medio. Toda consulta $[l, r]$ cae exactamente en un nivel donde l y r quedan en mitades distintas del mismo bloque (el nivel lo da el bit más alto donde difieren l y r), así que se combina sin solapar y sirve para operaciones **no idempotentes** como la suma.

```
1 struct DisjointSparseTable {
2     vector<vector<long long>>> pre;
3     vector<long long> a;
4     int n, LOG;
5
6     // CREATE: precalcula prefijos izquierda/derecha por nivel,
7     // partiendo el arreglo en bloques de tamaño 2^(k+1).
8     DisjointSparseTable(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        LOG = 1;
12        while ((1 << LOG) < n) LOG++;
13        LOG++;
14
15        pre.assign(LOG, vector<long long>(n));
16
17        for (int k = 0; k < LOG; k++) {
18            int block = 1 << (k + 1);
19
20            for (int start = 0; start < n; start += block) {
21                int mid = min(start + block / 2, n);
22                int end = min(start + block, n);
23
24                if (mid > n) continue;
25            }
26        }
27    }
```

```

26 // prefijo hacia la izquierda desde mid-1
27 if (mid - 1 >= start) {
28     pre[k][mid - 1] = a[mid - 1];
29     for (int i = mid - 2; i >= start; i--)
30         pre[k][i] = a[i] + pre[k][i + 1];
31 }
32
33 // prefijo hacia la derecha desde mid
34 if (mid < end) {
35     pre[k][mid] = a[mid];
36     for (int i = mid + 1; i < end; i++)
37         pre[k][i] = pre[k][i - 1] + a[i];
38 }
39 }
40 }
41 // O(n log n)
42
43 // READ: suma del rango [l, r] (0-indexado, inclusivo).
44 // El nivel k lo da el bit mas alto donde l y r difieren:
45 // ahi es donde el bloque se parte en dos mitades disjuntas.
46 long long query(int l, int r) {
47     if (l == r) return a[l];
48
49     int k = 31 - __builtin_clz(1 ^ r);
50     return pre[k][l] + pre[k][r];
51 }
52 // O(1)
53 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 DisjointSparseTable dst(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << dst.query(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << dst.query(2, 5);
10 // 9 + 1 + 7 + 3 = 20
11
12 cout << dst.query(1, 1);
13 // 2

```

Diferencia entre las dos variantes:

Estructura	Construir	Query	Updates
Sparse Table (min/max/gcd)	$O(n \log n)$	$O(1)$	No
Disjoint Sparse Table (suma/xor)	$O(n \log n)$	$O(1)$	No
Segment Tree	$O(n)$	$O(\log n)$	Sí, $O(\log n)$

Table 6: Sparse Table vs. Disjoint Sparse Table vs. Segment Tree.

Regla práctica: si el arreglo **no cambia**, usa Sparse Table para min/max/gcd, o Disjoint Sparse Table si necesitas suma, xor, o cualquier operación no idempotente. En cuanto el arreglo **cambia** —aunque sea una sola actualización— reconstruir en $O(n \log n)$ deja de valer la pena y conviene un Segment Tree.

5.6 Segment Tree

Idea: un árbol binario implícito (guardado en un arreglo) donde cada nodo representa el resultado combinado de un rango del arreglo original. La raíz cubre todo $[0, n-1]$; cada nodo se parte en dos mitades hasta llegar a las hojas, que son elementos individuales. Un update baja por el árbol tocando $O(\log n)$ nodos (uno por nivel); una query de rango se descompone en a lo más $O(\log n)$ nodos que cubren exactamente el rango pedido, sin necesidad de bloques parciales como en Sqrt Decomposition. A diferencia de Fenwick, sirve para **cualquier** operación asociativa (min, max, gcd, and, or, no solo sumas), y con Lazy Propagation soporta actualizaciones de rango completo.

¿Por qué usarlo? Es la estructura más versátil de esta sección: $O(\log n)$ tanto para update como para query, funciona con casi cualquier operación asociativa,

y con lazy propagation resuelve range update + range query para operaciones que Fenwick no puede (min, max, asignar un valor a todo un rango, etc.).

Casos de uso:

- Min/max/gcd de rango con actualizaciones (a diferencia de Sparse Table, que exige arreglo estático).
- Range update + range query con operaciones no invertibles: asignar un valor a todo un rango, min/max con suma en rango, etc.
- Cualquier operación asociativa que no sea una suma simple (donde Fenwick ya alcanzaría y sería más simple).
- Base para variantes más avanzadas: Segment Tree 2D, Merge Sort Tree, Persistent Segment Tree, Segment Tree Beats.

Complejidad: construir $O(n)$, update puntual $O(\log n)$, query de rango $O(\log n)$, range update + range query con lazy propagation $O(\log n)$.

Segment Tree clásico (point update, range query — suma)

```

1 struct SegmentTree {
2     vector<long long> tree, a;
3     int n;
4
5     // CREATE: reserva el arbol (tamano 4n es cota segura) y
6     // construye recursivamente a partir de a[0..n-1].
7     SegmentTree(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tree.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    }
14    // O(n)
15
16    void build(int node, int l, int r) {
17        if (l == r) {
18            tree[node] = a[l];
19            return;
20        }
21
22        int mid = (l + r) / 2;
23        build(2 * node, l, mid);
24        build(2 * node + 1, mid + 1, r);
25        tree[node] = tree[2 * node] + tree[2 * node + 1];
26    }
27
28    void updateUtil(int node, int l, int r, int i, long long x) {
29        if (l == r) {
30            tree[node] = x;
31            return;
32        }
33
34        int mid = (l + r) / 2;
35        if (i <= mid)
36            updateUtil(2 * node, l, mid, i, x);
37        else
38            updateUtil(2 * node + 1, mid + 1, r, i, x);
39
40        tree[node] = tree[2 * node] + tree[2 * node + 1];
41    }
42
43    // UPDATE: fija el valor absoluto de la posicion i en x.
44    void update(int i, long long x) {
45        updateUtil(1, 0, n - 1, i, x);
46    }
47    // O(log n)
48
49    long long queryUtil(int node, int l, int r, int ql, int qr) {
50        if (qr < l || r < ql)
51            return 0;
52        // neutro para
53        // suma
54        if (ql <= l && r <= qr)
55            return tree[node];
56
57        int mid = (l + r) / 2;

```

```

55     return queryUtil(2 * node, 1, mid, ql, qr)
56         + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
57 }
58
59 // READ: suma del rango [l, r] (0-indexado, inclusivo).
60 long long rango(int l, int r) {
61     return queryUtil(1, 0, n - 1, l, r);
62 }
63 // O(log n)
};

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 SegmentTree st(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << st.rango(1, 3);
7 // 2 + 9 + 1 = 12
8
9 st.update(2, 100);
10 // arreglo logico: [5, 2, 100, 1, 7]
11
12 cout << st.rango(1, 3);
13 // 2 + 100 + 1 = 103

```

Nota: para min/max/gcd solo cambia el neutro del caso base (LLONG_MAX para min, 0 para gcd, etc.) y el operador de combinación (min(...), __gcd(...)) en build, updateUtil y queryUtil.

Segment Tree con Lazy Propagation (range update + range query — suma)

Idea: cada nodo guarda un valor lazy pendiente que representa “sumar v a todo este rango” sin aplicarlo todavía a los hijos. Se aplica (*se propaga*) solo cuando hace falta bajar más en el árbol para otra operación.

```

1 struct SegmentTreeLazyAdd {
2     vector<long long> tree, lazy, a;
3     int n;
4
5     // CREATE: reserva arbol + lazy, construye a partir de a[0..n-1].
6     SegmentTreeLazyAdd(vector<long long>& arr) {
7         a = arr;
8         n = a.size();
9         tree.assign(4 * n, 0);
10        lazy.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    } // O(n)
14
15    void build(int node, int l, int r) {
16        if (l == r) {
17            tree[node] = a[l];
18            return;
19        }
20
21        int mid = (l + r) / 2;
22        build(2 * node, l, mid);
23        build(2 * node + 1, mid + 1, r);
24        tree[node] = tree[2 * node] + tree[2 * node + 1];
25    }
26
27    // Aplica el lazy pendiente de este nodo y lo empuja a los hijos.
28    void propagar(int node, int l, int r) {
29        if (lazy[node] == 0)
30            return;
31
32        tree[node] += lazy[node] * (r - l + 1);
33
34        if (l != r) {
35            lazy[2 * node] += lazy[node];
36            lazy[2 * node + 1] += lazy[node];
37        }
38
39        lazy[node] = 0;
40    }
41
42    void rangoUpdateUtil(int node, int l, int r, int ql, int qr,
43        long long v) {
44        propagar(node, l, r);
45
46        if (qr < l || r < ql)
47            return;

```

```

48        if (ql <= l && r <= qr) {
49            lazy[node] += v;
50            propagar(node, l, r);
51            return;
52        }
53
54        int mid = (l + r) / 2;
55        rangoUpdateUtil(2 * node, l, mid, ql, qr, v);
56        rangoUpdateUtil(2 * node + 1, mid + 1, r, ql, qr, v);
57        tree[node] = tree[2 * node] + tree[2 * node + 1];
58    }
59
60    // UPDATE: suma v a todo el rango [l, r].
61    void rangoUpdate(int l, int r, long long v) {
62        rangoUpdateUtil(1, 0, n - 1, l, r, v);
63    } // O(log n)
64
65    long long queryUtil(int node, int l, int r, int ql, int qr) {
66        propagar(node, l, r);
67
68        if (qr < l || r < ql)
69            return 0;
70
71        if (ql <= l && r <= qr)
72            return tree[node];
73
74        int mid = (l + r) / 2;
75        return queryUtil(2 * node, l, mid, ql, qr)
76            + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
77    }
78
79    // READ: suma del rango [l, r].
80    long long rango(int l, int r) {
81        return queryUtil(1, 0, n - 1, l, r);
82    } // O(log n)
83
84 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SegmentTreeLazyAdd st(a);
4
5 st.rangoUpdate(1, 4, 10);
6 // arreglo logico efectivo: [1, 11, 11, 11, 11, 1]
7
8 cout << st.rango(0, 5);
9 // 1+11+11+11+11+1 = 46
10
11 cout << st.rango(2, 3);
12 // 11+11 = 22

```

Lazy Propagation con asignación (range assign + range query — mínimo)

Idea: en vez de *sumar* un valor al rango, lo *reemplaza* por completo (asigna x a todo $[l, r]$). El lazy aquí no se acumula como en la suma: el último assign pisa a cualquier assign anterior pendiente, así que se necesita una bandera aparte para distinguir “no hay lazy pendiente” de “hay que asignar 0”.

```

1 struct SegmentTreeAssignMin {
2     vector<long long> tree, lazy, a;
3     vector<bool> tieneLazy;
4     int n;
5
6     // CREATE: reserva arbol + lazy + banderas, construye desde a.
7
8     SegmentTreeAssignMin(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        tree.assign(4 * n, LLONG_MAX);
12        lazy.assign(4 * n, 0);
13        tieneLazy.assign(4 * n, false);
14        if (n > 0)
15            build(1, 0, n - 1);
16    } // O(n)
17
18    void build(int node, int l, int r) {
19        if (l == r) {
20            tree[node] = a[l];
21            return;
22        }
23
24        int mid = (l + r) / 2;
25        build(2 * node, l, mid);

```

```

25     build(2 * node + 1, mid + 1, r);
26     tree[node] = min(tree[2 * node], tree[2 * node + 1]);
27 }
28
29 // Empuja el assign pendiente de este nodo hacia sus hijos.
30 void propagar(int node) {
31     if (!tieneLazy[node])
32         return;
33
34     for (int hijo : {2 * node, 2 * node + 1}) {
35         tree[hijo] = lazy[node];
36         lazy[hijo] = lazy[node];
37         tieneLazy[hijo] = true;
38     }
39
40     tieneLazy[node] = false;
41 }
42
43 void rangoAssignUtil(int node, int l, int r, int ql, int qr,
44                     long long x) {
45     if (qr < l || r < ql)
46         return;
47
48     if (ql <= l && r <= qr) {
49         tree[node] = x;
50         lazy[node] = x;
51         tieneLazy[node] = true;
52         return;
53     }
54
55     propagar(node);
56     int mid = (l + r) / 2;
57     rangoAssignUtil(2 * node, l, mid, ql, qr, x);
58     rangoAssignUtil(2 * node + 1, mid + 1, r, ql, qr, x);
59     tree[node] = min(tree[2 * node], tree[2 * node + 1]);
60 }
61
62 // UPDATE: asigna x a todo el rango [l, r].
63 void rangoAssign(int l, int r, long long x) {
64     rangoAssignUtil(1, 0, n - 1, l, r, x);
65 }
66 // O(log n)
67
68 long long queryUtil(int node, int l, int r, int ql, int qr) {
69     if (qr < l || r < ql)
70         return LLONG_MAX;
71
72     if (ql <= l && r <= qr)
73         return tree[node];
74
75     propagar(node);
76     int mid = (l + r) / 2;
77     return min(queryUtil(2 * node, l, mid, ql, qr),
78               queryUtil(2 * node + 1, mid + 1, r, ql, qr));
79 }
80
81 // READ: mínimo del rango [l, r].
82 long long rango(int l, int r) {
83     return queryUtil(1, 0, n - 1, l, r);
84 }
85 // O(log n)
86 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SegmentTreeAssignMin st(a);
4
5 cout << st.rango(0, 5);
6 // min(5,2,9,1,7,3) = 1
7
8 st.rangoAssign(0, 2, 100);
9 // arreglo logico efectivo: [100, 100, 100, 1, 7, 3]
10
11 cout << st.rango(0, 2);
12 // min(100,100,100) = 100
13
14 cout << st.rango(0, 5);
15 // min(100,100,100,1,7,3) = 1

```

Comparación con las otras estructuras de rango:

Estructura	Update	Query	Min/Max?
Segment Tree	$O(\log n)$	$O(\log n)$	Sí
Segment Tree + Lazy	$O(\log n)$ range	$O(\log n)$ range	Sí
FenwickTree	$O(\log n)$ punto	$O(\log n)$ rango	No
Sqrt Decomposition	$O(1) - O(\sqrt{n})$	$O(\sqrt{n})$	Sí

Table 7: Segment Tree vs. Fenwick vs. Sqrt Decomposition.

Regla práctica: usa Segment Tree cuando necesites min/max/gcd (operaciones no invertibles que Fenwick no soporta), o cuando necesites range update + range query con una operación que no sea suma simple. Si la operación es solo suma y no necesitas range update, Fenwick es más simple y más rápido en la práctica (menor constante). Si no confías en escribir el lazy propagation correcto bajo presión, Sqrt Decomposition es una alternativa más simple a costa de $O(\sqrt{n})$ en vez de $O(\log n)$.

5.7 RMQ (Range Minimum Query)

Idea: RMQ es el problema general de "dado un arreglo, encuentra el mínimo en cualquier rango $[l, r]$ ". Ya tienes las dos estructuras que lo resuelven: **Sparse Table** si el arreglo es estático ($O(1)$ por query) y **Segment Tree** si cambia ($O(\log n)$ por query). Esta sección no repite esas estructuras, sino que muestra las dos aplicaciones clásicas donde un problema que *no parece* de rango se puede **reformular** como RMQ sobre un arreglo auxiliar: LCA en árboles, y LCP entre sufijos arbitrarios de un string.

¿Por qué usarlo? Reconocer cuándo un problema es "RMQ disfrazado" es lo que realmente separa resolver estos problemas en $O(1) - O(\log n)$ de resolverlos con fuerza bruta $O(n)$ por consulta. El patrón se repite: construyes un arreglo auxiliar donde la respuesta a tu pregunta real equivale al mínimo de un rango, y le aplicas Sparse Table o Segment Tree normal.

Casos de uso:

- LCA (ancestro común más bajo) de dos nodos en un árbol, con muchas queries: Euler Tour + RMQ.
- LCP (prefijo común más largo) entre dos sufijos *cualquiera* de un string (no necesariamente consecutivos en el Suffix Array): Suffix Array + Kasai + RMQ sobre el LCP Array.
- Cualquier problema donde la respuesta se pueda expresar como "el mínimo de algo" sobre un rango de índices auxiliares.

Complejidad: en ambas aplicaciones, el precómputo (Euler Tour o Suffix Array + Kasai) es $O(n)$ a $O(n \log^2 n)$ según el método de Suffix Array usado; una vez armado el arreglo auxiliar, el RMQ es $O(n \log n)$ de build y $O(1)$ por query con Sparse Table (ambos casos son estáticos: el árbol y el string no cambian).

LCA vía Euler Tour + RMQ (Sparse Table)

Idea: recorres el árbol con un Euler Tour, anotando el nodo visitado cada vez que **entras o vuelves** a él (arreglo de tamaño $2n - 1$) junto con su profundidad. El LCA de u y v es el nodo de **menor profundidad** que aparece entre la primera aparición de u y la primera aparición de v en ese recorrido — exactamente un RMQ (por profundidad) sobre el arreglo del Euler Tour.

```

1 struct LCA {
2     vector<int> euler, primeraApar, profundidad;
3     vector<vector<int>> sp; // sp[k][i] = nodo con menor
4                             profundidad
5     vector<int> lg;
6
7     // CREATE: construye el Euler Tour del arbol (arr. de tamaño

```

```

7 // 2n-1 y la Sparse Table sobre profundidad, para RMQ O(1).
8 LCA(vector<vector<int>>& adj, int raiz, int n) {
9     primeraApar.assign(n, -1);
10    profundidad.assign(n, 0);
11    euler.reserve(2 * n);
12
13    dfs(adj, raiz, -1, 0);
14    build();
15
16    // O(n log n)
17
18    void dfs(vector<vector<int>>& adj, int u, int p, int d) {
19        primeraApar[u] = euler.size();
20        profundidad[u] = d;
21        euler.push_back(u);
22
23        for (int v : adj[u]) {
24            if (v != p) {
25                dfs(adj, v, u, d + 1);
26                euler.push_back(u);
27            }
28        }
29    }
30
31    void build() {
32        int m = euler.size();
33        lg.assign(m + 1, 0);
34        for (int i = 2; i <= m; i++)
35            lg[i] = lg[i / 2] + 1;
36
37        int K = lg[m] + 1;
38        sp.assign(K, vector<int>(m));
39        sp[0] = euler;
40
41        for (int k = 1; k < K; k++)
42            for (int i = 0; i + (1 << k) <= m; i++) {
43                int l = sp[k - 1][i];
44                int r = sp[k - 1][i + (1 << (k - 1))];
45                sp[k][i] = profundidad[l] < profundidad[r] ? l : r;
46            }
47
48        // READ: LCA de u y v.
49        int query(int u, int v) {
50            int l = primeraApar[u], r = primeraApar[v];
51            if (l > r) swap(l, r);
52
53            int k = lg[r - l + 1];
54            int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
55            return profundidad[a] < profundidad[b] ? a : b;
56        }
57    };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //            / \
3 //          1   2
4 //         / \
5 //       3   4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 LCA lca(adj, 0, 5);
15
16 cout << lca.query(3, 4);
17 // 1 (padre comun de 3 y 4)
18
19 cout << lca.query(3, 2);
20 // 0 (raiz)
21
22 cout << lca.query(1, 4);
23 // 1 (uno es ancestro del otro)

```

LCP entre dos sufijos arbitrarios (Suffix Array + Kasai + RMQ)

Idea: el LCP Array (via Kasai) solo te da el LCP entre sufijos **consecutivos** en el Suffix Array. Para el LCP entre dos sufijos **cualquiera** i, j : los ubicas por su rango en el SA ($rnk[i], rnk[j]$) y el LCP entre ambos es el **mínimo** del LCP Array en el rango entre esas dos posiciones —otra vez RMQ, esta vez sobre el LCP Array.

```

1 struct SuffixLCPQuery {
2     vector<int> sa, rnk, lcp;
3     vector<vector<int>> sp;
4     vector<int> lg;
5     int n;
6
7     // CREATE: construye SA, LCP (Kasai) y Sparse Table sobre lcp,
8     // todo en un solo paso.
9     SuffixLCPQuery(const string& s) {
10         n = s.size();
11         sa = buildSuffixArray(s);
12         lcp = buildLCPArray(s, sa);
13
14         rnk.assign(n, 0);
15         for (int i = 0; i < n; i++)
16             rnk[sa[i]] = i;
17
18         buildSparse();
19     }
20
21     // O(n log^2 n)
22
23     void buildSparse() {
24         lg.assign(n + 1, 0);
25         for (int i = 2; i <= n; i++)
26             lg[i] = lg[i / 2] + 1;
27
28         int K = lg[n] + 1;
29         sp.assign(K, vector<int>(n));
30         sp[0] = lcp;
31
32         for (int k = 1; k < K; k++)
33             for (int i = 0; i + (1 << k) <= n; i++)
34                 sp[k][i] = min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
35     }
36
37     // READ: LCP entre el sufijo que empieza en i y el que
38     // empieza en j (0-indexado sobre el string original).
39     int lcpEntreSufijos(int i, int j) {
40         if (i == j) return n - i;
41
42         int a = rnk[i], b = rnk[j];
43         if (a > b) swap(a, b);
44
45         // el mínimo va de [a+1, b] en el LCP array
46         int k = lg[b - a];
47         return min(sp[k][a + 1], sp[k][b - (1 << k) + 1]);
48     };

```

Ejemplo de funcionamiento:

```

1 SuffixLCPQuery slq("banana");
2
3 cout << slq.lcpEntreSufijos(1, 3);
4 // sufijo desde 1: "anana"
5 // sufijo desde 3: "ana"
6 // LCP = "ana" -> 3
7
8 cout << slq.lcpEntreSufijos(0, 3);
9 // sufijo desde 0: "banana"
10 // sufijo desde 3: "ana"
11 // LCP = "" -> 0

```

Nota: buildSuffixArray y buildLCPArray son las funciones estándar de Suffix Array + Kasai (ver la sección de Algoritmos de Strings); aquí solo se reutilizan como precómputo antes de aplicar RMQ.

Regla práctica: cuando un problema pida "algo" entre dos elementos que no son adyacentes en su estructura original (dos nodos de un árbol, dos sufijos de un string, dos elementos de cualquier secuencia con una noción de "orden" y "profundidad/afinidad"), pregúntate si esa relación se puede expresar como el mínimo de un arreglo auxiliar construido con un recorrido lineal. Si sí, el problema se reduce a Sparse Table (estático) o Segment Tree (dinámico), ya cubiertos en las secciones anteriores.

5.8 Mo's Algorithm

Idea: responde q queries de rango **offline** (las conoces todas de antemano) en $O((n + q)\sqrt{n})$ total, sin ninguna estructura de datos elaborada. La clave es **ordenar las queries** —no procesarlas en el orden dado—

agrupándolas por bloque de tamaño $\sqrt{n}$ según su extremo izquierdo l , y dentro de cada bloque por su extremo derecho r (alternando la dirección en bloques pares/impares para aprovechar mejor la localidad). Luego mantienes un puntero $[curL, curR]$ que se **desliza** de una query a la siguiente con funciones **add/remove** en $O(1)$ cada una. Como los punteros nunca retroceden más de lo necesario gracias al orden elegido, el total de movimientos amortizado es $O((n+q)\sqrt{n})$ en vez de $O(nq)$ con fuerza bruta.

¿Por qué usarlo? Sirve para respuestas que **no tienen una estructura de rango obvia** (por ejemplo: contar elementos distintos, sumar cuadrados de frecuencias, contar pares con cierta propiedad) donde escribir un Segment Tree específico sería complicado o imposible, pero sí es fácil mantener la respuesta incrementalmente al agregar/quitar un elemento a la vez.

Casos de uso:

- Contar elementos distintos en un rango, con muchas queries.
- Sumas o conteos que dependen de *frecuencias* dentro del rango (no solo de valores individuales), donde **add/remove** son fáciles pero no hay forma clara de combinar dos sub-rangos con un Segment Tree.
- Todas las queries se conocen de antemano (offline). Si necesitas responder **online** (una query a la vez, sin ver las siguientes), Mo's no aplica: usa Segment Tree o Sqrt Decomposition.
- El arreglo tiene pocos updates puntuales intercalados con las queries: usa la variante *Mo's con updates* (3D Mo's).

Complejidad: $O((n+q)\sqrt{n})$ total para q queries sin updates; $O(n^{5/3})$ (aprox.) para la variante con updates, usando bloques de tamaño $n^{2/3}$.

Mo's clásico (queries offline, sin updates)

Ejemplo guía: contar cuántos valores **distintos** hay en cada rango $[l, r]$ preguntado.

```
1 struct MoAlgorithm {
2     int n, blockSize;
3     vector<int> a;
4     vector<long long> cnt;
5     int curL = 0, curR = -1;
6     long long curDistinct = 0;
7
8     // CREATE: reserva el arreglo de frecuencias segun el maximo
9     // valor posible, y fija el tamaño de bloque en  $\sqrt{n}$ .
10    MoAlgorithm(vector<int>& arr, int maxVal) {
11        a = arr;
12        n = a.size();
13        blockSize = max(1, (int)sqrt((double)n));
14        cnt.assign(maxVal, 0);
15    } // O(n)
16
17    // UPDATE: agrega el elemento en pos a la ventana actual.
18    void add(int pos) {
19        if (cnt[a[pos]]++ == 0)
20            curDistinct++;
21    } // O(1)
22
23    // UPDATE: quita el elemento en pos de la ventana actual.
24    void remove(int pos) {
25        if (--cnt[a[pos]] == 0)
26            curDistinct--;
27    } // O(1)
28
29    // READ: procesa todas las queries offline (l, r inclusive,
```

```
30    // 0-indexadas) y devuelve las respuestas en el orden
31    // original.
32    vector<long long> resolver(vector<array<int, 2>>& queries) {
33        int q = queries.size();
34        vector<int> idx(q);
35        iota(idx.begin(), idx.end(), 0);
36
37        sort(idx.begin(), idx.end(), [&](int i, int j) {
38            int bi = queries[i][0] / blockSize;
39            int bj = queries[j][0] / blockSize;
40            if (bi != bj) return bi < bj;
41            return (bi & 1) ? (queries[i][1] > queries[j][1])
42                           : (queries[i][1] < queries[j][1]);
43        });
44
45        vector<long long> respuestas(q);
46        for (int id : idx) {
47            int l = queries[id][0], r = queries[id][1];
48
49            while (curR < r) add(++curR);
50            while (curL > l) add(--curL);
51            while (curR > r) remove(curR--);
52            while (curL < l) remove(curL++);
53
54            respuestas[id] = curDistinct;
55        }
56        return respuestas; // O((n+q) sqrt(n))
57    };
```

Ejemplo de funcionamiento:

```
1 vector<int> a = {1, 2, 1, 3, 2, 4, 1};
2 // valores en [1, 4]
3
4 MoAlgorithm mo(a, 5);
5
6 vector<array<int, 2>> queries = {
7     {0, 2}, // [1, 2, 1] -> 2 distintos
8     {1, 4}, // [2, 1, 3, 2] -> 3 distintos
9     {0, 6} // arreglo completo -> 4 distintos
10 };
11
12 vector<long long> resp = mo.resolver(queries);
13 // resp = [2, 3, 4]
```

Mo's con updates (3D Mo's)

Idea: si además de queries hay **updates puntuales** intercaladas, se agrega una tercera dimensión al ordenamiento: el *tiempo* t (cuántas updates ya ocurrieron antes de cada query). Se ordenan las queries por (bloque(l), bloque(r), t), y además de deslizar $[curL, curR]$, se avanza o retrocede $curT$ aplicando o revertiendo updates una por una. El bloque óptimo cambia a $\approx n^{2/3}$ (en vez de $\sqrt{n}$) porque ahora hay tres dimensiones que balancear.

```
1 struct MoUpdates {
2     int n, blockSize;
3     vector<int> a;
4     vector<long long> cnt;
5     vector<pair<int, int>> updates; // (pos, valor); ver nota
6     // abajo
7     int curL = 0, curR = -1, curT = 0;
8     long long curDistinct = 0;
9
10    // CREATE: reserva frecuencias y fija el bloque en  $n^{2/3}$ .
11    MoUpdates(vector<int>& arr, int maxVal) {
12        a = arr;
13        n = a.size();
14        blockSize = max(1, (int)cbrt((double)n) *
15                        (int)cbrt((double)n));
16        cnt.assign(maxVal, 0);
17    } // O(n)
18
19    void add(int pos) {
20        if (cnt[a[pos]]++ == 0) curDistinct++;
21    } // O(1)
22
23    void remove(int pos) {
24        if (--cnt[a[pos]] == 0) curDistinct--;
25    } // O(1)
26
27    // CREATE: registra una update (aplicarla no es inmediato,
28    // solo queda en la lista para procesarse en su momento).
```

```

28 void agregarUpdate(int pos, int val) {
29     updates.push_back({pos, val});
30 } // O(1)
31
32 // UPDATE: aplica (o revierte, es la misma operacion) la
33 // update numero t. El truco: como updates[t].second guarda
34 // "el valor a poner", tras aplicarla queda guardado el valor
35 // viejo -> llamar de nuevo revierte exactamente al estado
36 // anterior.
37 void alternarUpdate(int t) {
38     int pos = updates[t].first;
39     int val = updates[t].second;
40
41     if (pos >= curL && pos <= curR)
42         remove(pos);
43
44     swap(a[pos], val);
45     updates[t].second = val;
46
47     if (pos >= curL && pos <= curR)
48         add(pos);
49 } // O(1)
50
51 // READ: procesa todas las queries offline (l, r, t) y
52 // devuelve las respuestas en el orden original.
53 vector<long long> resolver(vector<array<int, 3>>& queries) {
54     int q = queries.size();
55     vector<int> idx(q);
56     iota(idx.begin(), idx.end(), 0);
57
58     sort(idx.begin(), idx.end(), [&](int i, int j) {
59         int bi = queries[i][0] / blockSize;
60         int bj = queries[j][0] / blockSize;
61         if (bi != bj) return bi < bj;
62
63         int ri = queries[i][1] / blockSize;
64         int rj = queries[j][1] / blockSize;
65         if (ri != rj) return ri < rj;
66
67         return queries[i][2] < queries[j][2];
68     });
69
70     vector<long long> respuestas(q);
71     for (int id : idx) {
72         int l = queries[id][0], r = queries[id][1],
73             t = queries[id][2];
74
75         while (curR < r) add(++curR);
76         while (curL > l) add(--curL);
77         while (curR > r) remove(curR--);
78         while (curL < l) remove(curL++);
79
80         while (curT < t) alternarUpdate(curT++);
81         while (curT > t) alternarUpdate(--curT);
82
83         respuestas[id] = curDistinct;
84     }
85     return respuestas;
86 } // O(n^(5/3))
87 } // aprox.

```

Ejemplo de funcionamiento:

```

1 vector<int> a = {1, 2, 3};
2
3 MoUpdates mo(a, 5);
4 mo.agregarUpdate(0, 2); // update #0: a[0] = 2
5 mo.agregarUpdate(1, 1); // update #1: a[1] = 1
6
7 // queries: (l, r, cuantas updates ya se aplicaron)
8 vector<array<int, 3>> queries = {
9     {0, 2, 0}, // arreglo original [1,2,3] -> 3 distintos
10    {0, 2, 1}, // tras update #0: [2,2,3] -> 2 distintos
11    {0, 2, 2} // tras update #1: [2,1,3] -> 3 distintos
12 };
13
14 vector<long long> resp = mo.resolver(queries);
15 // resp = [3, 2, 3]

```

Nota: para problemas sobre **árboles** (queries sobre un camino o subárbol) existe *Mo's on Trees*, que aplanar el árbol con un Euler Tour y aplica el mismo algoritmo sobre ese arreglo; el resto del razonamiento (ordenar queries, deslizar punteros) es idéntico.

Comparación con otras estructuras de rango:

Situación	Estructura	Complejidad
Queries offline, sin updates	Mo's clásico	$O((n + q)\sqrt{n})$
Queries offline, con updates	Mo's con updates	$O(n^{5/3})$
Queries online (una a la vez)	Segment Tree / Sqrt Decomp.	$O(\log n)$ / $O(\sqrt{n})$

Table 8: Cuándo usar Mo's Algorithm frente a estructuras online.

Regla práctica: usa Mo's solo cuando (1) todas las queries son offline, y (2) la respuesta es fácil de mantener incrementalmente con **add/remove**, pero difícil de combinar con un Segment Tree (min/max/suma si se combinan fácil, así que ahí Mo's no aporta nada). Si el problema admite Segment Tree o Fenwick directamente, esas opciones son más simples y no necesitan que las queries sean offline.

5.9 Wavelet Tree

Idea: es un árbol binario sobre el **rango de valores** del arreglo (no sobre las posiciones, como Segment Tree). La raíz cubre $[lo, hi]$ de valores posibles; en cada nodo, los elementos con valor $\leq$ mediana van al hijo izquierdo y el resto al derecho, **preservando su orden relativo** (partición estable). Cada nodo guarda un arreglo $b[]$ con cuántos elementos, hasta cada posición, fueron hacia la izquierda. Con eso puedes, en cada nivel, saber exactamente qué sub-rango de posiciones le corresponde a cada hijo — y bajar por el árbol respondiendo preguntas sobre **valores dentro de un rango de posiciones** en $O(\log C)$, donde C es la cantidad de valores distintos.

¿Por qué usarlo? Es la única estructura de esta sección que responde eficientemente preguntas que combinan **posición y valor** a la vez: "el k -ésimo menor en $[l, r]$ ", "cuántos elementos en $[l, r]$ son $\leq X$ ", "cuántas veces aparece exactamente X en $[l, r]$ ". Ninguna de las estructuras anteriores (Fenwick, Segment Tree, Sparse Table) resuelve esto directamente sin re-plantear el problema.

Casos de uso:

- k -ésimo elemento más chico (o más grande) dentro de un rango $[l, r]$.
- Conteo de elementos en $[l, r]$ cuyo valor está en un rango $[a, b]$.
- Frecuencia de un valor específico dentro de un rango de posiciones (rank query).
- Arreglo **estático** — no soporta updates sin reconstruir (para eso existe la variante persistente, más avanzada).

Complejidad: construir $O(n \log C)$, cada query $O(\log C)$, donde C es la cantidad de valores distintos tras comprimir coordenadas.

Wavelet Tree: k -ésimo, conteo $\leq X$ y frecuencia

```

1 struct WaveletTree {
2     int lo, hi;
3     WaveletTree *izq, *der = nullptr;
4     vector<int> b; // b[i] = cuantos de a[0..i-1] fueron a la
5     izq.

```

```

6 // CREATE: construye recursivamente sobre el segmento [from,
7 // de un arreglo mutable, particionando por la mediana de
8 // [lo, hi]. Los valores deben venir ya comprimidos a [0, C
9 WaveletTree(int* from, int* to, int lo, int hi) : lo(lo), hi(hi) {
10     if (lo == hi || from == to) return;
11
12     int mid = (lo + hi) / 2;
13     auto valzq = [mid](int x) { return x <= mid; };
14
15     b.reserve(to - from + 1);
16     b.push_back(0);
17     for (auto it = from; it != to; it++)
18         b.push_back(b.back() + valzq(*it));
19
20     auto pivote = stable_partition(from, to, valzq);
21     izq = new WaveletTree(from, pivote, lo, mid);
22     der = new WaveletTree(pivote, to, mid + 1, hi);
23 } // O(n log C)
24
25 // READ: k-esimo valor mas chico en [l, r] (1-indexado, k
26 // desde 1).
27 int kth(int l, int r, int k) {
28     if (l > r) return -1;
29     if (lo == hi) return lo;
30
31     int enIzq = b[r] - b[l - 1];
32     int lb = b[l - 1], rb = b[r];
33
34     if (k <= enIzq)
35         return izq->kth(lb + 1, rb, k);
36     return der->kth(l - lb, r - rb, k - enIzq);
37 } // O(log C)
38
39 // READ: cuantos elementos de [l, r] tienen valor <= val.
40 int contarMenorIgual(int l, int r, int val) {
41     if (l > r || val < lo) return 0;
42     if (hi <= val) return r - l + 1;
43
44     int lb = b[l - 1], rb = b[r];
45     return izq->contarMenorIgual(lb + 1, rb, val)
46         + der->contarMenorIgual(l - lb, r - rb, val);
47 } // O(log C)
48
49 // READ: cuantas veces aparece exactamente val en [l, r].
50 int contarIgual(int l, int r, int val) {
51     if (l > r || val < lo || val > hi) return 0;
52     if (lo == hi) return r - l + 1;
53
54     int mid = (lo + hi) / 2;
55     int lb = b[l - 1], rb = b[r];
56
57     if (val <= mid)
58         return izq->contarIgual(lb + 1, rb, val);
59     return der->contarIgual(l - lb, r - rb, val);
60 } // O(log C)
61
62 ~WaveletTree() {
63     delete izq;
64     delete der;
65 }

```

Ejemplo de funcionamiento (con compresión de coordenadas):

```

1 vector<int> original = {40, 10, 30, 10, 20, 40, 30};
2
3 // comprimir a [0, C-1] para que el wavelet tree trabaje con
4 // valores pequenos y contiguos.
5 vector<int> ordenado = original;
6 sort(ordenado.begin(), ordenado.end());
7 ordenado.erase(unique(ordenado.begin(), ordenado.end()), ordenado
8 .end());
9
10 vector<int> comprimido(original.size());
11 for (int i = 0; i < (int)original.size(); i++)
12     comprimido[i] = lower_bound(ordenado.begin(), ordenado.end(),
13         original[i]) - ordenado.begin();
14
15 // comprimido: [3, 0, 2, 0, 1, 3, 2] (valores 10,20,30,40 ->
16 // 0,1,2,3)
17
18 WaveletTree wt(comprimido.data(),
19     comprimido.data() + comprimido.size(),
20     0, (int)ordenado.size() - 1);
21
22 // 3er elemento mas chico en el rango [2, 6] (1-indexado)
23 int idx = wt.kth(2, 6, 3);
24 cout << ordenado[idx];

```

```

22 // posiciones 2..6 (1-idx): 10,30,10,20,40,30 (valores 0-idx 2..6)
23
24 // -> valores reales: 30,10,20,40,30 ordenados: 10,20,30,30,40
25 // 3er mas chico = 30
26
27 cout << wt.contarMenorIgual(1, 7, 1);
28 // cuantos valores comprimidos <= 1 (o sea, <= 20) hay en todo el
29 // arreglo
30 // original: 40,10,30,10,20,40,30 -> <=20 son: 10,10,20 -> 3
31
32 cout << wt.contarIgual(1, 7, 0);
33 // cuantas veces aparece el valor comprimido 0 (o sea, 10) -> 2

```

Comparación con las otras estructuras de rango:

Pregunta	Estructura
Suma/min/max de rango	Fenwick / Segment Tree / Sparse Table
k-ésimo menor, o conteo por rango de valores , en un rango de posiciones	Wavelet Tree
Elementos distintos, moda, funciones no lineales de frecuencia	Mo's Algorithm

Table 9: Cuándo el problema necesita Wavelet Tree en vez de las otras estructuras.

Regla práctica: si el enunciado combina una condición sobre **valores** (“menor que”, “entre a y b ”, “el k -ésimo más chico”) con un rango de **posiciones**, y el arreglo es estático, es casi siempre Wavelet Tree. Si solo te piden agregar (suma/min/max) sin condición sobre el valor, usa las estructuras más simples de las secciones anteriores.

5.10 Heavy-Light Decomposition

Idea: descompone un árbol en **cadenas** de forma que cualquier camino desde la raíz hasta cualquier nodo cruza a lo más $O(\log n)$ cadenas distintas. En cada nodo, el hijo con el **subárbol más grande** (el hijo “pesado”) continúa la misma cadena que su padre; los demás hijos inician cadenas nuevas. Al numerar los nodos en el orden en que se visitan (DFS que siempre sigue primero al hijo pesado), cada cadena queda como un **rango contiguo** de posiciones — así que una query sobre un **camino** del árbol se convierte en $O(\log n)$ queries de rango sobre un Segment Tree normal, una por cada cadena que el camino atraviesa.

¿Por qué usarlo? Es la forma estándar de llevar range queries de un arreglo a un **árbol**: sumas/mínimos/updates en el camino entre dos nodos, o en el subárbol de un nodo. Sin HLD, un camino arbitrario no tiene ninguna estructura contigua sobre la cual aplicar Fenwick o Segment Tree directamente.

Casos de uso:

- Suma/mínimo/máximo en el camino entre dos nodos u y v .
- Actualizar todos los nodos (o aristas) de un camino $u-v$.
- Consultas o updates sobre el **subárbol** completo de un nodo (más simple: el subárbol siempre es un rango contiguo, sin necesidad de descomponer en cadenas).
- Cualquier problema de árbol que “se sentiría” como Range Queries si el árbol fuera un arreglo lineal.

Complejidad: construir $O(n)$ (dos DFS) más $O(n)$ del Segment Tree subyacente; query o update de **camino** $O(\log^2 n)$ ($O(\log n)$ cadenas $\times O(\log n)$ por consulta al Segment Tree); query o update de **subárbol** $O(\log n)$ (un solo rango contiguo).

HLD: suma en camino, suma en subárbol

Idea de la implementación: dos DFS iterativas (con pila explícita, para evitar stack overflow en árboles muy profundos o muy desbalanceados — un riesgo real con recursión cuando $n \geq 10^5$ y el árbol es una cadena larga). El primero (`dfsTamano`) calcula el tamaño de cada subárbol y decide el hijo pesado de cada nodo. El segundo (`dfsDescomponer`) recorre siempre primero al hijo pesado (así la cadena sigue consecutiva en las posiciones) y solo después explora los hijos livianos, cada uno iniciando su propia cadena nueva. Las posiciones resultantes (`pos[]`) alimentan un Segment Tree normal —aquí se reutiliza `SegmentTreeLazyAdd` de la sección anterior— sobre el cual se hacen las queries reales.

```
1 struct HLD {
2     vector<vector<int>> adj;
3     vector<int> padre, profundidad, tamano, pesado, cabeza, pos;
4     int n, timer = 0;
5     SegmentTreeLazyAdd* seg; // Segment Tree sobre el orden pos
6
7     // CREATE: dos DFS iterativas para calcular cadenas, luego
8     // arma
9     // el Segment Tree sobre el arreglo reordenado por pos[].
10    HLD(vector<vector<int>>& adjacency, vector<long long>&
11        valores,
12        int raiz) {
13        adj = adjacency;
14        n = adj.size();
15        padre.assign(n, -1);
16        profundidad.assign(n, 0);
17        tamano.assign(n, 1);
18        pesado.assign(n, -1);
19        cabeza.assign(n, raiz);
20        pos.assign(n, 0);
21
22        dfsTamano(raiz);
23        dfsDescomponer(raiz);
24
25        vector<long long> reordenado(n);
26        for (int v = 0; v < n; v++)
27            reordenado[pos[v]] = valores[v];
28
29        seg = new SegmentTreeLazyAdd(reordenado);
30    } // O(n)
31
32    // Fase 1: obtiene un orden donde cada padre aparece ANTES
33    // que
34    // sus hijos (equivalente a un preorden), usando una pila.
35    // Fase 2: recorre ese orden al revés (hijos antes que padres)
36    // para acumular tamano[] y decidir el hijo pesado, igual que
37    // haría la versión recursiva al "volver" de cada llamada.
38    void dfsTamano(int raiz) {
39        vector<int> orden;
40        stack<int> st;
41        st.push(raiz);
42        padre[raiz] = -1;
43        profundidad[raiz] = 0;
44
45        while (!st.empty()) {
46            int u = st.top();
47            st.pop();
48            orden.push_back(u);
49
50            for (int v : adj[u]) {
51                if (v != padre[u]) {
52                    padre[v] = u;
53                    profundidad[v] = profundidad[u] + 1;
54                    st.push(v);
55                }
56            }
57        }
58
59        for (int i = (int)orden.size() - 1; i >= 0; i--) {
60            int u = orden[i];
```

```
61            tamano[u] = 1;
62            int maxSub = 0;
63
64            for (int v : adj[u]) {
65                if (v == padre[u]) continue;
66
67                tamano[u] += tamano[v];
68                if (tamano[v] > maxSub) {
69                    maxSub = tamano[v];
70                    pesado[u] = v;
71                }
72            }
73        } // O(n)
74
75        // Pila de pares (nodo, cabeza de su cadena). Para que el
76        // hijo
77        // pesado quede en el tope de la pila (y por lo tanto se
78        // procese inmediatamente despues, como en la versión
79        // recursiva), se empuja de ULTIMO: los hijos livianos
80        // (cadena nueva) se empujan primero.
81        void dfsDescomponer(int raiz) {
82            stack<pair<int, int>> st;
83            st.push({raiz, raiz});
84
85            while (!st.empty()) {
86                auto [u, h] = st.top();
87                st.pop();
88
89                cabeza[u] = h;
90                pos[u] = timer++;
91
92                for (int v : adj[u]) {
93                    if (v == padre[u] || v == pesado[u]) continue;
94                    st.push({v, v}); // cadena nueva
95                }
96
97                if (pesado[u] != -1)
98                    st.push({pesado[u], h}); // misma cadena
99            } // O(n)
100
101            // READ: suma de todos los nodos en el camino u-v.
102            long long consultarCamino(int u, int v) {
103                long long resultado = 0;
104
105                while (cabeza[u] != cabeza[v]) {
106                    if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
107                        swap(u, v);
108
109                    resultado += seg->rango(pos[cabeza[u]], pos[u]);
110                    u = padre[cabeza[u]];
111                }
112
113                if (profundidad[u] > profundidad[v]) swap(u, v);
114                resultado += seg->rango(pos[u], pos[v]);
115                return resultado;
116            } // O(log^2 n)
117
118            // UPDATE: suma val a todos los nodos en el camino u-v.
119            void actualizarCamino(int u, int v, long long val) {
120                while (cabeza[u] != cabeza[v]) {
121                    if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
122                        swap(u, v);
123
124                    seg->rangoUpdate(pos[cabeza[u]], pos[u], val);
125                    u = padre[cabeza[u]];
126                }
127
128                if (profundidad[u] > profundidad[v]) swap(u, v);
129                seg->rangoUpdate(pos[u], pos[v], val);
130            } // O(log^2 n)
131
132            // READ: suma de todo el subarbol de u (rango contiguo por
133            // construcción, gracias al orden DFS de dfsDescomponer).
134            long long consultarSubarbol(int u) {
135                return seg->rango(pos[u], pos[u] + tamano[u] - 1);
136            } // O(log n)
137
138            // UPDATE: suma val a todo el subarbol de u.
139            void actualizarSubarbol(int u, long long val) {
140                seg->rangoUpdate(pos[u], pos[u] + tamano[u] - 1, val);
141            } // O(log n)
142        };
143    };
```

Ejemplo de funcionamiento:

```
1 // arbol:      0
2 //            /\
3 //           1 2 3
4 //          /\
5 //         4 5
```

```

7 vector<vector<int>> adj(6);
8 adj[0] = {1, 2, 3};
9 adj[1] = {0, 4, 5};
10 adj[2] = {0};
11 adj[3] = {0};
12 adj[4] = {1};
13 adj[5] = {1};
14
15 vector<long long> valores = {1, 2, 3, 4, 5, 6};
16 // nodo:      0 1 2 3 4 5
17
18 HLD hld(adj, valores, 0);
19
20 cout << hld.consultarCamino(4, 3);
21 // camino 4 -> 1 -> 0 -> 3 : valores 5+2+1+4 = 12
22
23 cout << hld.consultarSubarbol(1);
24 // subarbol de 1: nodos {1,4,5} : 2+5+6 = 13
25
26 hld.actualizarCamino(4, 5, 10);
27 // suma 10 a cada nodo del camino 4 -> 1 -> 5
28
29 cout << hld.consultarSubarbol(0);
30 // subarbol de 0 (todo el arbol) tras el update:
31 // 1+(2+10)+3+4+(5+10)+(6+10) = 51

```

Nota: al ser iterativas, `dfsTamano` y `dfsDescomponer` no tienen riesgo de stack overflow incluso en árboles con $n \geq 10^5$ y forma de “cadena larga” (el peor caso para recursión). `SegmentTreeLazyAdd` es exactamente el struct de la sección de Segment Tree — cualquier otra variante (min, max, assign) funciona igual de bien como motor subyacente.

Regla práctica: usa HLD en cuanto el problema pida algo sobre un **camino** entre dos nodos de un árbol con muchas queries. Si solo pide algo sobre **subárboles** (sin caminos arbitrarios), no hace falta la descomposición en cadenas completa: basta un solo DFS para las posiciones (`pos[]` y `tamano[]`) y un Segment Tree normal, ya que el subárbol siempre es un rango contiguo por sí solo.

Parte III Grafos

5.11 ¿Qué algoritmo usar?

La siguiente tabla resume situaciones típicas de Competitive Programming y el algoritmo que debe considerarse. La idea es reconocer las palabras clave o condiciones del enunciado y asociarlas rápidamente con una técnica.

Cuando el ejercicio diga / pida	Usa	Pista rápida
“Recorrer el grafo por niveles”	BFS	Usa una cola.
“Encontrar la distancia mínima” con aristas de igual costo	BFS	Cada arista cuesta lo mismo.
“Encontrar caminos mínimos” con pesos 0 y 1	0-1 BFS	Deque; pesos únicamente 0 o 1.
“Camino mínimo” con pesos no negativos	Dijkstra	Usa una cola de prioridad.
“Camino mínimo” con pesos negativos	Bellman-Ford	Permite detectar ciclos negativos.
“Camino mínimo entre todos los pares” y n pequeño	Floyd-Warshall	DP sobre todos los pares.
“¿Existe un camino entre u y v ?”	DFS / BFS	Problema de conectividad.
“Encontrar componentes conexas”	DFS / BFS	Ejecutar traversal desde cada nodo no visitado.
“Detectar si existe un ciclo” en grafo no dirigido	DFS / DSU	DFS para recorrer; DSU si se agregan aristas.
“Detectar ciclos” en grafo dirigido	DFS / Kahn	Usar estados en DFS o grados de entrada.
“Ordenar tareas con dependencias”	Topological Sort	El grafo debe ser dirigido y acíclico.
“Encontrar puentes”	Tarjan	DFS + tiempos de descubrimiento.
“Encontrar puntos de articulación”	Tarjan	Vértices cuya eliminación desconecta el grafo.
“Encontrar componentes fuertemente conexas”	Kosaraju / Tarjan	Grafo dirigido.
“Conectar todos los nodos con costo mínimo”	MST	Kruskal o Prim.
“MST procesando aristas ordenadas por peso”	Kruskal	Ordenar aristas + DSU.
“MST expandiendo desde un nodo”	Prim	Priority Queue + lista de adyacencia.
“¿Dos nodos pertenecen al mismo conjunto?”	DSU	Union-Find.
“Agrupar elementos mediante uniones”	DSU	union + find.
“Encontrar si un grafo es bipartito”	2-Coloring	Colorear con dos colores.
“Máximo número de parejas entre dos grupos”	Bipartite Matching	Emparejamiento entre conjuntos.
“Máximo flujo de una fuente a un destino”	Dinic	Network Flow.
“Separar fuente y sumidero con capacidad mínima”	Min-Cut	Por el teorema Max-Flow Min-Cut.
“Distancia entre dos nodos de un árbol”	LCA	$\text{dist}(u, v) = d[u] + d[v] - 2d[\text{LCA}(u, v)]$.
“Encontrar el ancestro k niveles arriba”	Binary Lifting	Saltar potencias de 2.
“Consultas sobre subárboles”	Euler Tour + Fenwick / Segment Tree	Convertir subárbol en intervalo.
“Consultas o actualizaciones sobre caminos de un árbol”	HLD	Descomponer el camino en cadenas.
“Encontrar el diámetro de un árbol”	BFS / DFS	Dos traversals desde extremos.
“Encontrar un camino que use todas las aristas”	Eulerian Path / Hierholzer	Trabaja con aristas, no con vértices.
“Encontrar todas las rutas posibles”	DFS + Backtracking	Explorar y deshacer decisiones.
“Rellenar una región conectada”	Flood Fill	DFS / BFS desde una celda inicial.
“Moverse por una cuadrícula buscando la menor cantidad de pasos”	BFS	Cada movimiento tiene costo 1.
“Variables booleanas con restricciones de la forma OR”	2-SAT + SCC	Construir grafo de implicaciones.
“Resolver muchas consultas sobre un árbol”	LCA / HLD	Depende de si son ancestros o caminos.

Table 10: Guía rápida para seleccionar algoritmos de grafos.

Condición del problema	Algoritmo	Complejidad
Aristas sin peso / costo 1	BFS	$O(n + m)$
Pesos únicamente 0 y 1	0-1 BFS	$O(n + m)$
Pesos no negativos	Dijkstra	$O((n + m) \log n)$
Puede haber pesos negativos	Bellman-Ford	$O(nm)$
Todos los pares y n pequeño	Floyd-Warshall	$O(n^3)$

Table 11: Selección del algoritmo para caminos mínimos.

6 Traversal

6.1 Representación de Grafos

Idea: antes de aplicar cualquier algoritmo sobre un grafo, primero debemos decidir cómo almacenar sus vértices y aristas. Las tres representaciones más comunes son la **lista de adyacencia**, la **matriz de adyacencia** y la **lista de aristas**. La elección depende principalmente de la cantidad de vértices y aristas y del tipo de consultas que realizará el algoritmo.

La **lista de adyacencia** almacena, para cada vértice, todos los vértices conectados directamente con él. Es la representación más utilizada en Competitive Programming porque ocupa $O(n + m)$ memoria y permite recorrer eficientemente todos los vecinos de un vértice.

La **matriz de adyacencia** utiliza una matriz $n \times n$ donde la posición (u, v) indica si existe una arista entre u y v . Permite comprobar la existencia de una arista en $O(1)$, pero requiere $O(n^2)$ memoria.

La **lista de aristas** simplemente almacena cada arista como un par (u, v) , o como una terna (u, v, w) si tiene peso. Es especialmente útil para algoritmos que necesitan procesar todas las aristas, como Kruskal o Bellman-Ford.

¿Por qué usar cada una?

- **Lista de adyacencia:** opción general para BFS, DFS, Dijkstra, Prim, búsqueda de ciclos y prácticamente la mayoría de algoritmos sobre grafos dispersos.
- **Matriz de adyacencia:** útil cuando n es pequeño o cuando se necesita comprobar constantemente si existe una arista entre dos vértices en $O(1)$.
- **Lista de aristas:** conveniente cuando el algoritmo necesita recorrer todas las aristas directamente, especialmente en Kruskal y Bellman-Ford.

Tipos de grafos: una arista puede ser **dirigida** o **no dirigida**, y puede tener o no un **peso**. En un grafo no dirigido, una arista (u, v) se representa agregando v a los vecinos de u y u a los vecinos de v . En un grafo dirigido, solamente se agrega v a los vecinos de u .

Complejidad:

Representación	Memoria	Consulta	Vecinos
Lista de adyacencia	$O(n + m)$	$O(\deg(u))$	$O(\deg(u))$
Matriz de adyacencia	$O(n^2)$	$O(1)$	$O(n)$
Lista de aristas	$O(m)$	$O(m)$	$O(m)$

Table 12: Comparación de representaciones de grafos.

Lista de adyacencia

Idea de la implementación: se utiliza un `vector<vector<int>>` donde la posición u contiene todos los vértices conectados directamente con u . Para grafos ponderados se puede almacenar un par (v, w) , donde v es el destino y w el peso de la arista.

```

1 // Grafo no dirigido y no ponderado.
2 int n = 5;
3 vector<vector<int>> adj(n);
4
5 // Agrega una arista u-v.
6 auto agregarArista = [&](int u, int v) {
7     adj[u].push_back(v);
8     adj[v].push_back(u);
9 };
10
11 agregarArista(0, 1);
12 agregarArista(0, 2);
13 agregarArista(1, 3);
14 agregarArista(1, 4);

```

Grafo resultante:

```

1 //      0
2 //      / \
3 //      1  2
4 //     / \
5 //    3   4
6 //
7 // adj[0] = {1, 2}
8 // adj[1] = {0, 3, 4}
9 // adj[2] = {0}
10 // adj[3] = {1}
11 // adj[4] = {1}

```

Para un grafo dirigido, solamente se agrega la arista en una dirección:

```

1 // Grafo dirigido.
2 vector<vector<int>> adj(n);
3
4 auto agregarArista = [&](int u, int v) {
5     adj[u].push_back(v);
6 };
7
8 agregarArista(0, 1);
9 agregarArista(0, 2);
10 agregarArista(1, 3);

```

Para un grafo ponderado, cada vecino puede almacenar también el peso:

```

1 // Grafo ponderado.
2 vector<vector<pair<int, long long>>> adj(n);
3
4 auto agregarArista = [&](int u, int v, long long w) {
5     adj[u].push_back({v, w});
6     adj[v].push_back({u, w});
7 };
8
9 agregarArista(0, 1, 5);
10 agregarArista(0, 2, 3);
11 agregarArista(1, 3, 7);

```

Ejemplo de recorrido:

```

1 void dfs(int u, int padre, vector<vector<int>>& adj) {
2
3     cout << u << ' ';
4
5     for (int v : adj[u]) {
6         if (v == padre) continue;
7
8         dfs(v, u, adj);
9     }
10 }

```

La lista de adyacencia permite que DFS o BFS procesen cada vértice y cada arista una cantidad constante de veces, obteniendo una complejidad total de $O(n + m)$.

Matriz de adyacencia

Idea: una matriz $adj[u][v]$ indica directamente si existe una conexión desde u hacia v . Si el grafo es ponderado, la posición puede almacenar directamente el peso de la arista.

```
1 int n = 5;
2 vector<vector<int>> adj(n, vector<int>(n, 0));
3
4 auto agregarArista = [&](int u, int v) {
5     adj[u][v] = 1;
6     adj[v][u] = 1;
7 };
8
9 agregarArista(0, 1);
10 agregarArista(0, 2);
11 agregarArista(1, 3);
12 agregarArista(1, 4);
```

Ahora comprobar si existe una arista entre u y v es inmediato:

```
1 if (adj[u][v]) {
2     cout << "Existe una arista";
3 }
```

La consulta anterior cuesta $O(1)$, pero recorrer todos los vecinos de u requiere revisar las n posiciones de la fila:

```
1 for (int v = 0; v < n; v++) {
2     if (adj[u][v]) {
3         cout << v << ' ';
4     }
5 }
```

Lista de aristas

Idea: almacena directamente todas las aristas del grafo. Para un grafo no ponderado basta guardar (u, v) ; para uno ponderado se guarda (u, v, w) .

```
1 // Grafo ponderado.
2 struct Edge {
3     int u, v;
4     long long w;
5 };
6
7 vector<Edge> edges;
8
9 edges.push_back({0, 1, 5});
10 edges.push_back({0, 2, 3});
11 edges.push_back({1, 3, 7});
```

Esta representación resulta especialmente conveniente cuando el algoritmo necesita procesar todas las aristas:

```
1 for (Edge e : edges) {
2     cout << e.u << " -> " << e.v
3         << " peso = " << e.w << '\n';
4 }
```

Por ejemplo, **Kruskal** ordena directamente esta lista por peso, mientras que **Bellman-Ford** puede recorrerla completa en cada iteración.

Nota: para la mayoría de problemas de Competitive Programming, la **lista de adyacencia** debe ser la representación por defecto. Es especialmente importante cuando el grafo es disperso, es decir, cuando m es mucho menor que n^2 . La matriz de adyacencia solamente conviene cuando n es pequeño o cuando las consultas de existencia de aristas en $O(1)$ son importantes.

Regla práctica: si no tienes una razón específica para usar otra representación, usa **lista de adyacencia**. Si necesitas preguntar constantemente “¿existe una arista $u \rightarrow v$?”, considera una **matriz de adyacencia**. Si

el algoritmo trabaja directamente con todas las aristas, considera una **lista de aristas**.

“latex

6.2 BFS (Breadth-First Search)

Idea: BFS (*Breadth-First Search*) es un algoritmo de recorrido de grafos que explora los vértices por **niveles**. A partir de un vértice inicial, primero visita todos sus vecinos directos, después los vecinos de estos, y continúa de esta manera hasta procesar todos los vértices alcanzables.

Para mantener este orden se utiliza una **cola** (*queue*), siguiendo el principio FIFO (*First In, First Out*). Cada vértice se marca como visitado cuando se agrega a la cola, evitando procesarlo varias veces.

Una de las propiedades más importantes de BFS es que, en un grafo **no ponderado**, encuentra la distancia mínima desde un vértice origen hasta cualquier otro vértice alcanzable, donde la distancia se mide como el número de aristas recorridas.

¿Por qué usarlo?

- **Camino mínimo:** encuentra el camino con menor número de aristas en grafos no ponderados.
- **Distancias:** permite calcular la distancia mínima desde un origen hasta todos los vértices alcanzables.
- **Recorrido por niveles:** permite procesar los vértices según su distancia al origen.
- **Reconstrucción de caminos:** almacenando el padre de cada vértice se puede recuperar el camino mínimo.
- **Bipartición:** permite determinar si un grafo es bipartito utilizando dos colores.
- **Componentes conexas:** ejecutando BFS desde cada vértice no visitado se pueden encontrar todas las componentes conexas.
- **Problemas en grids:** permite encontrar la cantidad mínima de movimientos cuando cada movimiento tiene el mismo costo.
- **Multi-Source BFS:** permite calcular la distancia mínima desde cualquiera de varios vértices iniciales.

Funcionamiento:

El algoritmo sigue los siguientes pasos:

1. Elegir un vértice inicial s .
2. Marcar s como visitado y agregarlo a la cola.
3. Mientras la cola no esté vacía, extraer el primer vértice.
4. Recorrer todos los vecinos del vértice extraído.
5. Si un vecino no ha sido visitado, marcarlo y agregarlo a la cola.

La cola garantiza que los vértices se procesen por niveles. Primero se procesan los vértices a distancia 0, después los de distancia 1, luego los de distancia 2, y así sucesivamente.

Implementación básica:

```
1 void bfs(int inicio, vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<bool> visitado(n, false);
6     queue<int> q;
7
8     visitado[inicio] = true;
9     q.push(inicio);
10
11     while (!q.empty()) {
12
13         int u = q.front();
14         q.pop();
15
16         cout << u << ' ';
17
18         for (int v : adj[u]) {
19
20             if (visitado[v]) continue;
21
22             visitado[v] = true;
23             q.push(v);
24         }
25     }
26 }
```

Ejemplo:

Consideremos el siguiente grafo:

```
1 //      0
2 //      / \
3 //     1  2
4 //    / \
5 //   3  4
```

Si comenzamos BFS desde el vértice 0, los niveles son:

```
1 // Nivel 0: 0
2 // Nivel 1: 1 2
3 // Nivel 2: 3 4
```

Por lo tanto, un posible orden de recorrido es:

```
1 0 1 2 3 4
```

El orden entre vértices del mismo nivel depende del orden en que aparezcan en la lista de adyacencia.

BFS para encontrar distancias:

Para calcular la distancia mínima desde el origen hasta cada vértice, se utiliza un arreglo `dist`. Inicialmente todos los valores son `-1`, indicando que el vértice todavía no ha sido visitado.

Cuando BFS descubre un nuevo vértice v desde u , su distancia se calcula como:

$$dist[v] = dist[u] + 1$$

```
1 vector<int> bfsDist(int inicio, vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<int> dist(n, -1);
6     queue<int> q;
7
8     dist[inicio] = 0;
9     q.push(inicio);
10
11     while (!q.empty()) {
12
13         int u = q.front();
14         q.pop();
15
16         for (int v : adj[u]) {
17
18             if (dist[v] != -1) continue;
19
20             dist[v] = dist[u] + 1;
```

```
21         q.push(v);
22     }
23 }
24
25 return dist;
26 }
```

Para el ejemplo anterior, comenzando desde 0, se obtiene:

```
1 // dist[0] = 0
2 // dist[1] = 1
3 // dist[2] = 1
4 // dist[3] = 2
5 // dist[4] = 2
```

Por ejemplo, $dist[3] = 2$ significa que el camino mínimo desde 0 hasta 3 utiliza dos aristas.

Reconstrucción del camino:

Si además de la distancia se necesita conocer el camino utilizado, se puede almacenar el **padre** de cada vértice. Cuando BFS descubre v desde u , se establece:

$$padre[v] = u$$

```
1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }
```

Una vez terminado BFS, el camino hasta un destino se reconstruye siguiendo los padres desde el destino hasta el origen:

```
1 vector<int> camino;
2
3 for (int v = destino; v != -1; v = padre[v]) {
4     camino.push_back(v);
5 }
6
7 reverse(camino.begin(), camino.end());
```

Por ejemplo, si:

```
1 // padre[3] = 1
2 // padre[1] = 0
3 // padre[0] = -1
```

el camino mínimo desde 0 hasta 3 es:

```
1 0 -> 1 -> 3
```

BFS en grafos no conectados:

Un BFS iniciado desde un único vértice solamente visita los vértices que pueden alcanzarse desde él. Para recorrer todo el grafo, se puede ejecutar BFS desde cada vértice que todavía no haya sido visitado.

```
1 vector<bool> visitado(n, false);
2
3 for (int s = 0; s < n; s++) {
4
5     if (visitado[s]) continue;
```

```

7   queue<int> q;
8
9   visitado[s] = true;
10  q.push(s);
11
12  while (!q.empty()) {
13
14      int u = q.front();
15      q.pop();
16
17      for (int v : adj[u]) {
18
19          if (visitado[v]) continue;
20
21          visitado[v] = true;
22          q.push(v);
23      }
24  }
25 }

```

Esta técnica permite encontrar, entre otras cosas, el número de **componentes conexas** de un grafo.

BFS para comprobar si un grafo es bipartito:

Un grafo es bipartito si sus vértices pueden dividirse en dos grupos de manera que ninguna arista conecte dos vértices del mismo grupo. BFS puede comprobar esta propiedad asignando alternativamente los colores 0 y 1 a los vértices.

Si un vértice u tiene color c , sus vecinos deben tener el color c^1 . Si una arista conecta dos vértices del mismo color, el grafo no es bipartito.

```

1 bool esBipartito(vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<int> color(n, -1);
6
7     for (int inicio = 0; inicio < n; inicio++) {
8
9         if (color[inicio] != -1) continue;
10
11         queue<int> q;
12
13         color[inicio] = 0;
14         q.push(inicio);
15
16         while (!q.empty()) {
17
18             int u = q.front();
19             q.pop();
20
21             for (int v : adj[u]) {
22
23                 if (color[v] == -1) {
24
25                     color[v] = color[u] ^ 1;
26                     q.push(v);
27
28                 } else if (color[v] == color[u]) {
29
30                     return false;
31                 }
32             }
33         }
34     }
35
36     return true;
37 }

```

El arreglo `color` también funciona como arreglo de visitados: un valor de -1 significa que el vértice todavía no ha sido procesado.

BFS en un grid:

BFS también puede utilizarse sobre matrices o tableros. Cada celda se considera un vértice y cada movimiento válido entre celdas se considera una arista.

Por ejemplo, si solamente se permiten movimientos hacia arriba, abajo, izquierda y derecha:

```

1 int dx[] = {-1, 1, 0, 0};
2 int dy[] = {0, 0, -1, 1};
3
4 queue<pair<int, int>> q;

```

```

5 q.push({sx, sy});
6 dist[sx][sy] = 0;
7
8 while (!q.empty()) {
9
10     auto [x, y] = q.front();
11     q.pop();
12
13     for (int d = 0; d < 4; d++) {
14
15         int nx = x + dx[d];
16         int ny = y + dy[d];
17
18         if (nx < 0 || nx >= n ||
19             ny < 0 || ny >= m)
20             continue;
21
22         if (dist[nx][ny] != -1)
23             continue;
24
25         if (grid[nx][ny] == '#')
26             continue;
27
28         dist[nx][ny] = dist[x][y] + 1;
29         q.push({nx, ny});
30     }
31 }
32 }

```

Esta técnica es especialmente útil en problemas donde se pregunta por la menor cantidad de movimientos necesarios para llegar desde una celda hasta otra.

Multi-Source BFS:

En algunos problemas existen varios vértices iniciales al mismo tiempo. En lugar de ejecutar un BFS separado desde cada uno, todos los orígenes se introducen inicialmente en la misma cola con distancia 0.

```

1 queue<int> q;
2 vector<int> dist(n, -1);
3
4 for (int s : fuentes) {
5
6     dist[s] = 0;
7     q.push(s);
8 }
9
10 while (!q.empty()) {
11
12     int u = q.front();
13     q.pop();
14
15     for (int v : adj[u]) {
16
17         if (dist[v] != -1) continue;
18
19         dist[v] = dist[u] + 1;
20         q.push(v);
21     }
22 }

```

De esta manera, cada vértice obtiene la distancia mínima hasta **cualquiera** de las fuentes.

Complejidad:

Utilizando una lista de adyacencia, cada vértice se procesa una sola vez y cada arista se revisa una cantidad constante de veces.

Representación	Tiempo	Memoria adicional
Lista de adyacencia	$O(n + m)$	$O(n)$
Matriz de adyacencia	$O(n^2)$	$O(n)$

Table 13: Complejidad de BFS según la representación del grafo.

La complejidad $O(n + m)$ hace que BFS sea especialmente eficiente para grafos dispersos representados mediante listas de adyacencia.

Nota: BFS encuentra caminos mínimos cuando todas las aristas tienen el mismo costo. No debe utilizarse directamente para encontrar caminos mínimos en grafos

con pesos arbitrarios. Para pesos no negativos se puede utilizar **Dijkstra**; si los pesos solamente son 0 y 1, una alternativa eficiente es **0-1 BFS**.

Regla práctica: si un problema pide minimizar el número de **pasos, movimientos o aristas** y cada movimiento tiene el mismo costo, considera primero **BFS**. Si necesitas conocer la distancia, utiliza **dist**; si necesitas recuperar el camino, utiliza **padre**; y si existen múltiples puntos de origen, considera **Multi-Source BFS**. “

“`latex`

6.3 BFS en Grafos Desconectados

Idea: un BFS iniciado desde un único vértice solamente visita los vértices que son alcanzables desde él. Si el grafo tiene varias **componentes conexas**, es necesario iniciar un nuevo BFS desde cada vértice que todavía no haya sido visitado.

La idea consiste en recorrer todos los vértices y, cada vez que se encuentra uno que no ha sido visitado, iniciar un nuevo BFS desde él. Cada ejecución de BFS procesa una componente conexas completa.

Implementación:

```
1 void bfs(int inicio, vector<vector<int>>& adj,
2       vector<bool>& visitado) {
3
4     queue<int> q;
5
6     visitado[inicio] = true;
7     q.push(inicio);
8
9     while (!q.empty()) {
10
11         int u = q.front();
12         q.pop();
13
14         for (int v : adj[u]) {
15
16             if (visitado[v]) continue;
17
18             visitado[v] = true;
19             q.push(v);
20         }
21     }
22 }
```

Para recorrer todo el grafo:

```
1 vector<bool> visitado(n, false);
2
3 for (int u = 0; u < n; u++) {
4
5     if (visitado[u]) continue;
6
7     bfs(u, adj, visitado);
8 }
```

Cada vez que se ejecuta **bfs**, se procesa una componente conexas que todavía no había sido visitada.

Contar componentes conexas:

Una aplicación directa consiste en contar cuántas componentes conexas tiene el grafo. Basta incrementar un contador cada vez que se inicia un nuevo BFS.

```
1 int componentes = 0;
2
3 vector<bool> visitado(n, false);
4
5 for (int u = 0; u < n; u++) {
6
7     if (visitado[u]) continue;
8
9     componentes++;
10
11     queue<int> q;
12
13     visitado[u] = true;
14     q.push(u);
15 }
```

```
16 while (!q.empty()) {
17
18     int v = q.front();
19     q.pop();
20
21     for (int w : adj[v]) {
22
23         if (visitado[w]) continue;
24
25         visitado[w] = true;
26         q.push(w);
27     }
28 }
29 }
30
31 cout << componentes << '\n';
```

Ejemplo:

Consideremos el siguiente grafo:

```
1 // 0 --- 1      3 --- 4
2 //           |
3 //           2
4 //
5 // 5 --- 6
```

El grafo tiene tres componentes conexas:

```
1 // Componente 1: {0, 1, 2}
2 // Componente 2: {3, 4}
3 // Componente 3: {5, 6}
```

Si recorremos los vértices en orden, el primer BFS puede comenzar desde 0 y visitar {0, 1, 2}. Después, los vértices 3 y 4 ya están visitados, por lo que el siguiente BFS comienza desde 5 y visita {5, 6}. Finalmente, se inicia otro BFS desde 3 para visitar {3, 4}.

Complejidad:

Aunque se ejecuten varios BFS, cada vértice se visita una sola vez y cada arista se procesa una cantidad constante de veces. Por lo tanto, la complejidad total utilizando una lista de adyacencia sigue siendo:

$$O(n + m)$$

La memoria adicional utilizada por el arreglo de visitados y la cola es $O(n)$.

Operación	Complejidad
Recorrer todas las componentes	$O(n + m)$
Contar componentes conexas	$O(n + m)$
Memoria adicional	$O(n)$

Table 14: Complejidad de BFS en grafos desconectados.

Regla práctica: si el grafo puede estar desconectado y se necesita procesar **todos los vértices**, no basta con ejecutar BFS una sola vez. Recorre todos los vértices y comienza un nuevo BFS cada vez que encuentres uno que todavía no haya sido visitado. “

6.4 BFS para Todos los Caminos

Idea: BFS no solamente permite encontrar la distancia mínima desde un vértice origen hacia los demás vértices. También puede utilizarse para reconstruir el **camino mínimo** desde el origen hasta cualquier vértice alcanzable.

Para reconstruir un camino, además de almacenar la distancia, se guarda para cada vértice su **padre**, es decir, el vértice desde el cual fue descubierto por primera vez. Como BFS visita los vértices por niveles, el primer padre asignado a un vértice pertenece a un camino mínimo.

Implementación:

```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }

```

Después de ejecutar BFS, `dist[v]` contiene la distancia mínima desde `inicio` hasta `v`, mientras que `padre[v]` indica qué vértice se debe seguir para reconstruir el camino.

Reconstrucción del camino:

Para obtener el camino hasta un vértice destino, se comienza en `destino` y se siguen los padres hasta llegar al origen.

```

1 vector<int> camino;
2
3 for (int v = destino; v != -1; v = padre[v]) {
4     camino.push_back(v);
5 }
6
7 reverse(camino.begin(), camino.end());

```

El vector `camino` contiene ahora los vértices en el orden correcto, desde el origen hasta el destino.

Ejemplo:

Supongamos que BFS produce los siguientes padres:

```

1 // padre[0] = -1
2 // padre[1] = 0
3 // padre[2] = 0
4 // padre[3] = 1
5 // padre[4] = 3

```

Si el origen es 0 y queremos reconstruir el camino hasta 4, seguimos los padres:

$$4 \rightarrow 3 \rightarrow 1 \rightarrow 0$$

Después de invertirlo obtenemos:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 4$$

Por lo tanto, este es un camino mínimo desde 0 hasta 4.

Función para reconstruir el camino:

La reconstrucción puede encapsularse en una función:

```

1 vector<int> reconstruirCamino(int destino,
2                             vector<int>& padre) {
3
4     vector<int> camino;
5
6     for (int v = destino; v != -1; v = padre[v]) {
7         camino.push_back(v);
8     }
9
10    reverse(camino.begin(), camino.end());
11
12    return camino;
13 }

```

El BFS completo puede combinarse con esta función:

```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }
24
25 vector<int> camino =
26     reconstruirCamino(destino, padre);

```

¿Qué significa “todos los caminos”?

En este contexto, BFS obtiene un camino mínimo desde el origen hacia **cada vértice alcanzable**. No significa que encuentre todos los caminos posibles entre dos vértices, ya que la cantidad de caminos posibles puede ser exponencial.

Por ejemplo, después de ejecutar BFS desde `s`:

- `dist[v]` indica la distancia mínima de `s` a `v`.
- `padre[v]` permite reconstruir un camino mínimo hacia `v`.
- Si `dist[v]` es `-1`, `v` no es alcanzable desde `s`.

Camino inexistente:

Antes de reconstruir el camino conviene comprobar si el destino fue alcanzado:

```

1 if (dist[destino] == -1) {
2     cout << "No existe camino\n";
3 }
4 else {
5     vector<int> camino =
6         reconstruirCamino(destino, padre);
7
8     for (int v : camino) {
9         cout << v << ' ';
10    }
11 }

```

Complejidad:

La ejecución de BFS utilizando una lista de adyacencia cuesta $O(n + m)$. La reconstrucción de un camino cuesta $O(k)$, donde k es la cantidad de vértices que contiene el camino.

Operación	Complejidad
BFS	$O(n + m)$
Reconstruir un camino	$O(k)$
Memoria adicional	$O(n)$

Table 15: Complejidad de BFS para reconstrucción de caminos.

Nota: BFS encuentra caminos mínimos en grafos donde todas las aristas tienen el mismo costo. Si las aristas tienen pesos diferentes, deben utilizarse algoritmos como **Dijkstra**, **0-1 BFS** o **Bellman-Ford**, dependiendo de las características del problema.

Regla práctica: si el problema pide no solamente la distancia mínima sino también **qué vértices forman el camino**, guarda un arreglo **padre** durante el BFS. No es necesario almacenar todos los caminos: basta con guardar un padre por vértice.

6.5 DFS

Idea: Depth-First Search (DFS) explora un grafo siguiendo un camino lo más profundamente posible antes de regresar y explorar otras ramas. A diferencia de BFS, que utiliza una **cola**, DFS utiliza una **pila**.

DFS puede implementarse de forma recursiva o iterativa. En Competitive Programming, la versión iterativa es útil cuando el grafo puede tener muchos vértices y una implementación recursiva podría provocar un **stack overflow**.

Implementación iterativa:

Se utiliza un `stack<int>` para almacenar los vértices que deben ser procesados. Un vértice se marca como visitado cuando se introduce en la pila, evitando que pueda agregarse múltiples veces.

```
1 void dfs(int inicio, vector<vector<int>>& adj,
2         vector<bool>& visitado) {
3
4     stack<int> st;
5
6     visitado[inicio] = true;
7     st.push(inicio);
8
9     while (!st.empty()) {
10
11         int u = st.top();
12         st.pop();
13
14         cout << u << ' ';
15
16         for (int v : adj[u]) {
17
18             if (visitado[v]) continue;
19
20             visitado[v] = true;
21             st.push(v);
22         }
23     }
24 }
```

El orden exacto en que se visitan los vértices depende del orden de los vecinos en la lista de adyacencia. Lo importante es que cada vértice alcanzable sea procesado una vez.

Ejemplo:

Consideremos el siguiente grafo:

```
1 //      0
2 //      / \
3 //      1  2
4 //      / \
5 //      3  4
```

La lista de adyacencia puede ser:

```
1 adj[0] = {1, 2};
2 adj[1] = {0, 3, 4};
3 adj[2] = {0};
4 adj[3] = {1};
5 adj[4] = {1};
```

Si comenzamos DFS desde 0, la pila inicialmente contiene:

```
1 st = {0}
```

Se extrae 0 y se agregan sus vecinos:

```
1 st = {1, 2}
```

El siguiente vértice que se extrae depende del orden en que fueron agregados a la pila. Por ejemplo, si se extrae primero 2, se procesa esa rama antes de regresar a 1.

DFS en un grafo desconectado:

Al igual que BFS, una sola ejecución de DFS solamente visita los vértices alcanzables desde el origen. Para recorrer todo el grafo, debemos iniciar un nuevo DFS desde cada vértice que todavía no haya sido visitado.

```
1 vector<bool> visitado(n, false);
2
3 for (int u = 0; u < n; u++) {
4
5     if (visitado[u]) continue;
6
7     dfs(u, adj, visitado);
8 }
```

Este patrón permite procesar todas las componentes conexas del grafo.

Contar componentes conexas:

Podemos contar el número de componentes incrementando un contador cada vez que sea necesario iniciar un nuevo DFS.

```
1 int componentes = 0;
2
3 vector<bool> visitado(n, false);
4
5 for (int u = 0; u < n; u++) {
6
7     if (visitado[u]) continue;
8
9     componentes++;
10    dfs(u, adj, visitado);
11 }
12
13 cout << componentes << '\n';
```

DFS en árboles:

En un árbol no es necesario mantener un arreglo de visitados si se conoce el padre del vértice actual. Al recorrer un vecino, simplemente se evita regresar al padre.

Sin embargo, en un grafo general se recomienda utilizar **visitado**, ya que pueden existir ciclos.

```
1 void dfs(int inicio, vector<vector<int>>& adj) {
2
3     stack<pair<int, int>> st;
4
5     st.push({inicio, -1});
6
7     while (!st.empty()) {
8
9         auto [u, padre] = st.top();
10        st.pop();
11
12        cout << u << ' ';
13
14        for (int v : adj[u]) {
15
16            if (v == padre) continue;
17
18            st.push({v, u});
19        }
20    }
21 }
```

Esta última versión es apropiada para árboles, pero para grafos generales debe utilizarse un arreglo de visitados.

Aplicaciones:

DFS es una de las herramientas fundamentales para resolver problemas de grafos. Entre sus aplicaciones más comunes se encuentran:

- Recorrer todos los vértices alcanzables desde un origen.
- Encontrar componentes conexas.

- Detectar ciclos.
- Construir árboles DFS.
- Encontrar puntos de articulación y puentes.
- Realizar ordenamiento topológico.
- Resolver problemas de conectividad.
- Recorrer árboles y estructuras jerárquicas.

Complejidad:

Utilizando una lista de adyacencia, cada vértice se procesa una sola vez y cada arista se revisa una cantidad constante de veces. Por lo tanto:

$$O(n + m)$$

La pila y el arreglo de visitados requieren $O(n)$ memoria adicional.

Representación	Tiempo	Memoria adicional
Lista de adyacencia	$O(n + m)$	$O(n)$
Matriz de adyacencia	$O(n^2)$	$O(n)$

Table 16: Complejidad de DFS según la representación del grafo.

Nota: DFS y BFS tienen la misma complejidad asintótica $O(n + m)$ utilizando una lista de adyacencia, pero exploran el grafo de manera diferente. BFS procesa por niveles mediante una cola, mientras que DFS profundiza utilizando una pila.

Regla práctica: utiliza DFS cuando el problema dependa de explorar profundamente una estructura, recorrer componentes, detectar ciclos o realizar algún procesamiento basado en el orden de entrada y salida de los vértices. Si el problema requiere distancias mínimas en un grafo no ponderado, normalmente BFS es la opción adecuada.

6.6 DFS para Conteo de Caminos

Idea: en un DAG (grafo dirigido acíclico), contar cuántos caminos distintos existen entre un nodo u y un nodo destino fijo es un problema de **DFS + memoización**: el número de caminos desde u hasta el destino es la suma de los caminos desde cada vecino de u hasta el destino, con caso base "ya llegué". Como varios caminos pueden pasar por el mismo nodo intermedio, sin memoización recalculas el mismo subárbol muchas veces —memoizar es lo que baja esto de exponencial a lineal.

¿Por qué usarlo? Es el mismo patrón de "DFS + memo" que ya usas en DP sobre árboles/DAGs, aplicado específicamente a contar (no a minimizar/maximizar). Reconocer que un problema pide "número de formas de llegar de A a B" en un grafo dirigido sin ciclos es la señal para aplicar esto en vez de enumerar caminos explícitamente.

Casos de uso:

- Contar caminos entre dos nodos fijos de un DAG.
- Contar, para cada nodo, cuántos caminos empiezan en él y terminan en algún nodo "hoja" (sin salidas) — mismo memo, sin destino fijo, sumando 1 en cada hoja.

- Variante con longitud exacta k : contar caminos de u a v usando **exactamente** k aristas, memoizando sobre el par $(nodo, pasos_restantes)$ en vez de solo $nodo$.
- Contar caminos módulo algún M cuando el número de caminos crece demasiado (usa `long long` y aplica $\%M$ en cada suma).

Complejidad: $O(V + E)$ con memoización, ya que cada nodo se procesa una sola vez y cada arista se recorre una sola vez. Sin memo, en el peor caso es exponencial (puede haber $O(2^V)$ caminos).

```

1 struct ConteoCaminos {
2     vector<vector<int>>> adj;
3     vector<long long> memo;
4     int destino;
5
6     // CREATE: guarda el grafo (debe ser DAG) y el nodo destino;
7     // memo[u] = -1 significa "no calculado todavía".
8     ConteoCaminos(vector<vector<int>>& adj_, int destino_) {
9         adj = adj_;
10        destino = destino_;
11        memo.assign(adj.size(), -1);
12    } // O(1)
13
14    // READ: numero de caminos distintos de u a destino.
15    long long contar(int u) {
16        if (u == destino) return 1;
17        if (memo[u] != -1) return memo[u];
18
19        long long total = 0;
20        for (int v : adj[u])
21            total += contar(v);
22
23        return memo[u] = total;
24    } // O(1) amortizado
25    // O(V+E) la primera vez
26 };

```

Ejemplo de funcionamiento:

```

1 // DAG:      0 -> 1 -> 3
2 //           0 -> 2 -> 3
3 //           1 -> 2
4
5 vector<vector<int>>> adj(4);
6 adj[0] = {1, 2};
7 adj[1] = {2, 3};
8 adj[2] = {3};
9 adj[3] = {};
10
11 ConteoCaminos cc(adj, 3);
12
13 cout << cc.contar(0);
14 // 3 (0-1-3, 0-1-2-3, 0-2-3)
15
16 cout << cc.contar(1);
17 // 2 (1-3, 1-2-3)

```

Nota: si el grafo tiene ciclos, esta memoización no funciona directamente (la recursión no termina) — primero hay que condensar las componentes fuertemente conexas (SCC) en un DAG, y contar sobre ese DAG condensado.

6.7 DFS + Cerramiento Transitivo

Idea: el cerramiento transitivo de un grafo dirigido responde, para cualquier par (u, v) , la pregunta "¿existe algún camino de u a v ?" en $O(1)$ por query, a costa de precomputar —para cada nodo— el conjunto completo de nodos que alcanza. La forma más directa es correr un DFS desde **cada** nodo y marcar todo lo que visita.

¿Por qué usarlo? Cuando el grafo es pequeño/mediano y vas a hacer **muchas** consultas de alcanzabilidad, precomputar todo de una vez sale más barato que un DFS/BFS por consulta. Es el mismo principio de "precomputo una vez, respondo $O(1)$ " que en RMQ, aplicado a alcanzabilidad en vez de a mínimos.

Casos de uso:

- Responder Q queries de "¿ u puede llegar a v ?" cuando Q es grande y V es manejable ($V \leq \sim 2000-5000$ según el límite de tiempo).
- Precomputar dependencias transitivas (si a depende de b y b depende de c , entonces a depende de c).
- Como paso intermedio antes de comprimir un DAG por SCC: primero condensas ciclos, y sobre el DAG resultante calculas el cerramiento transitivo.

Complejidad: $O(V \cdot (V + E))$ para construir (un DFS completo por cada nodo), $O(1)$ por query. Con `std::bitset` en vez de `vector<bool>`, y procesando los nodos en orden topológico inverso (`reach[u] |= reach[v]` para cada vecino v), el build baja a $O(V \cdot (V + E)/64)$ gracias al paralelismo a nivel de bits — útil si V es más grande.

```

1 struct CerramientoTransitivo {
2     int n;
3     vector<vector<int>> adj;
4     vector<vector<bool>> alcanzable; // alcanzable[u][v] = true
5     // si u puede llegar a v
6
7     // CREATE: corre un DFS completo desde cada nodo. O(V*(V+E)).
8     CerramientoTransitivo(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        alcanzable.assign(n, vector<bool>(n, false));
12
13        for (int origen = 0; origen < n; origen++)
14            dfs(origen, origen);
15    }
16
17    void dfs(int origen, int u) {
18        for (int v : adj[u]) {
19            if (!alcanzable[origen][v]) {
20                alcanzable[origen][v] = true;
21                dfs(origen, v);
22            }
23        }
24    }
25
26    // READ: es u puede llegar a v.
27    bool query(int u, int v) {
28        return alcanzable[u][v];
29    } // O(1)
30 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 -> 1 -> 2
2 //         \
3 //         -> 3
4 //         4 (aislado)
5
6 vector<vector<int>> adj(5);
7 adj[0] = {1};
8 adj[1] = {2, 3};
9 adj[2] = {};
10 adj[3] = {};
11 adj[4] = {};
12
13 CerramientoTransitivo ct(adj, 5);
14
15 cout << ct.query(0, 3);
16 // true (0 -> 1 -> 3)
17
18 cout << ct.query(2, 0);
19 // false (no hay camino de 2 a 0)
20
21 cout << ct.query(4, 0);
22 // false (4 esta aislado)

```

Regla práctica: DFS "para contar" y DFS "para cerramiento transitivo" comparten la misma mecánica de recorrido, pero acumulan cosas distintas en cada llamada: uno suma caminos hacia un destino fijo (memoizando por nodo), el otro marca alcanzabilidad hacia *todos* los

destinos posibles (memoizando por par de nodos, implícitamente, al no revisitar lo ya marcado). Si el grafo tiene ciclos, ambos requieren primero pasar por condensación de SCC antes de aplicarse tal cual.

6.8 Flood Fill

Idea: Flood Fill es "explorar todas las celdas conectadas a una celda inicial que cumplen una condición" (mismo color, mismo carácter, sin obstáculo, etc.), marcándolas como visitadas o recoloreándolas. Es exactamente un DFS/BFS pero sobre una grilla en vez de un grafo con listas de adyacencia explícitas: los "vecinos" de una celda (r, c) son sus hasta 4 (u 8) celdas adyacentes.

¿Por qué usarlo? Cualquier problema de grilla que hable de "regiones", "islas", "áreas conectadas" o "pintar/rellenar" es Flood Fill. La clave práctica es preferir la versión **iterativa** (con `queue` o `stack` explícito) sobre la recursiva: en grillas grandes ($R \times C$ del orden de 10^5-10^6 celdas), un DFS recursivo puede hacer *stack overflow*.

Casos de uso:

- Contar componentes conexas de una grilla ("número de islas"): un Flood Fill por cada celda no visitada que cumpla la condición, contando cuántas veces arrancas uno nuevo.
- Calcular el área (tamaño) de cada región conectada.
- "Paint bucket": recolorar toda una región conectada a un color nuevo.
- Determinar si dos celdas están en la misma región conectada.

Complejidad: $O(R \cdot C)$, ya que cada celda se visita (y se encola) como máximo una vez.

```

1 struct FloodFill {
2     vector<vector<char>> grid;
3     vector<vector<bool>> visitado;
4     int R, C;
5
6     int dr[4] = {-1, 1, 0, 0};
7     int dc[4] = {0, 0, -1, 1};
8
9     // CREATE: guarda la grilla (R x C).
10    FloodFill(vector<vector<char>>& grid_) {
11        grid = grid_;
12        R = grid.size();
13        C = grid[0].size();
14        visitado.assign(R, vector<bool>(C, false));
15    } // O(1)
16
17    // READ: marca (BFS iterativo) toda la region conectada a
18    // (sr, sc) que comparte el mismo caracter, y devuelve su
19    // area.
20    // Evita recursion para no hacer stack overflow en grillas
21    // grandes.
22    int llenar(int sr, int sc) {
23        if (visitado[sr][sc]) return 0;
24
25        char objetivo = grid[sr][sc];
26        queue<pair<int,int>> q;
27        q.push({sr, sc});
28        visitado[sr][sc] = true;
29        int area = 0;
30
31        while (!q.empty()) {
32            auto [r, c] = q.front(); q.pop();
33            area++;
34
35            for (int d = 0; d < 4; d++) {
36                int nr = r + dr[d], nc = c + dc[d];
37
38                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
39                if (visitado[nr][nc]) continue;
40                if (grid[nr][nc] != objetivo) continue;
41            }
42            q.push({nr, nc});
43        }
44        return area;
45    }
46 };

```

```

39     visitado[nr][nc] = true;
40     q.push({nr, nc});
41 }
42 }
43 }
44
45 return area;
46 } // O(área de la
47 ); // región)

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // L L W W
3 // L L W W
4 // W W L L
5
6 vector<vector<char>> grid = {
7     {'L','L','W','W'},
8     {'L','L','W','W'},
9     {'W','W','L','L'}
10 };
11
12 FloodFill ff(grid);
13
14 int islas = 0, areaMax = 0;
15 for (int r = 0; r < 3; r++)
16 for (int c = 0; c < 4; c++)
17 if (grid[r][c] == 'L' && !ff.visitado[r][c]) {
18     int area = ff.llenar(r, c);
19     islas++;
20     areaMax = max(areaMax, area);
21 }
22
23 cout << islas; // 2
24 cout << areaMax; // 4 (el bloque de 2x2 de arriba-izquierda)

```

6.9 Laberinto con BFS

Idea: en una grilla donde te mueves de una celda a una celda adyacente (arriba/abajo/izq/der) con **costo uniforme** (cada paso cuesta 1), la ruta más corta desde un origen se calcula con BFS: exploras la grilla "por capas" de distancia creciente, y la primera vez que llegas a una celda, esa es su distancia mínima. Es el mismo BFS de grafos de siempre, con la grilla actuando como grafo implícito.

¿Por qué usarlo? Es la herramienta por defecto para "camino más corto en grilla sin pesos / con obstáculos". Si el problema menciona pesos distintos por celda (terreno con costo), esto deja de ser BFS puro y pasa a ser Dijkstra o 0-1 BFS (deque) — pero mientras cada paso cueste lo mismo, BFS es suficiente y más simple.

Casos de uso:

- Número mínimo de pasos de un punto A a un punto B en un laberinto con paredes.
- Reconstruir el camino más corto explícito (no solo la distancia), guardando de dónde vino cada celda.
- BFS multi-fuente: varias celdas iniciales a la vez (p.ej. "tiempo mínimo para que el fuego llegue a cada celda"), encolando todas las fuentes con distancia 0 antes de empezar.
- Verificar alcanzabilidad ("¿se puede llegar de A a B ?").

Complejidad: $O(R \cdot C)$, ya que cada celda se encola y desencola como máximo una vez.

```

1 struct LaberintoBFS {
2     vector<vector<char>> grid;
3     vector<vector<int>> dist;
4     vector<vector<pair<int,int>>> padre;

```

```

5     int R, C;
6
7     int dr[4] = {-1, 1, 0, 0};
8     int dc[4] = {0, 0, -1, 1};
9
10    // CREATE: guarda la grilla. '#' = pared, cualquier otro
11    // = celda libre.
12    LaberintoBFS(vector<vector<char>>& grid_) {
13        grid = grid_;
14        R = grid.size();
15        C = grid[0].size();
16    } // O(1)
17
18    // READ: corre BFS desde (sr, sc) y llena dist / padre para
19    // todas las celdas alcanzables. dist queda en -1 donde no
20    // llego.
21    void bfs(int sr, int sc) {
22        dist.assign(R, vector<int>(C, -1));
23        padre.assign(R, vector<pair<int,int>>(C, {-1, -1}));
24
25        queue<pair<int,int>> q;
26        q.push({sr, sc});
27        dist[sr][sc] = 0;
28
29        while (!q.empty()) {
30            auto [r, c] = q.front(); q.pop();
31
32            for (int d = 0; d < 4; d++) {
33                int nr = r + dr[d], nc = c + dc[d];
34
35                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
36                if (grid[nr][nc] == '#') continue;
37                if (dist[nr][nc] != -1) continue;
38
39                dist[nr][nc] = dist[r][c] + 1;
40                padre[nr][nc] = {r, c};
41                q.push({nr, nc});
42            }
43        } // O(R*C)
44
45        // READ: reconstruye el camino desde el origen del ultimo bfs
46        // (er, ec), como lista de celdas en orden. Vacío si no
47        // hay camino.
48        vector<pair<int,int>> reconstruirCamino(int er, int ec) {
49            if (dist[er][ec] == -1) return {};
50
51            vector<pair<int,int>> camino;
52            pair<int,int> actual = {er, ec};
53
54            while (actual.first != -1) {
55                camino.push_back(actual);
56                actual = padre[actual.first][actual.second];
57            }
58
59            reverse(camino.begin(), camino.end());
60            return camino;
61        } // O(longitud del camino)
62    };

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // . . . #
3 // # # . #
4 // . . . .
5
6 vector<vector<char>> grid = {
7     {'.', '.', '.', '#'},
8     {'#', '#', '.', '#'},
9     {'.', '.', '.', '.'}
10 };
11
12 LaberintoBFS lb(grid);
13 lb.bfs(0, 0);
14
15 cout << lb.dist[2][3];
16 // 5 (0,0)->(0,1)->(0,2)->(1,2)->(2,2)->(2,3)
17
18 auto camino = lb.reconstruirCamino(2, 3);
19 // [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3)]

```

Regla práctica: si un problema de grilla pide "¿son conectados?" o "área de una región", es Flood Fill (DFS o BFS, no importa el orden de visita). Si pide "distancia mínima" o "menor número de pasos", es BFS por capas (el orden de visita sí importa: BFS garantiza que la primera

vez que tocas una celda es con su distancia mínima; DFS no lo garantiza).

6.10 Tour de Caballo de Ajedrez

Idea: el Tour de Caballo pide encontrar una secuencia de movimientos de caballo que visite **cada casilla del tablero exactamente una vez**. No es un problema de "camino más corto" (no hay pesos ni se compara contra alternativas), sino de **existencia**: es backtracking puro — DFS que prueba un movimiento, sigue explorando, y si se atasca (llega a una casilla sin salidas válidas) deshace ("backtrack") y prueba otro movimiento.

¿Por qué usarlo? A diferencia de Flood Fill o BFS en laberintos, aquí **no basta con visitar** las casillas alcanzables: tienen que visitarse **todas, sin repetir**, en un orden válido. Eso descarta BFS/DFS simple y obliga a backtracking con una condición de poda, porque la fuerza bruta pura (8^n combinaciones de movimientos posibles) es intratable incluso para tableros chicos.

Casos de uso:

- Encontrar (o verificar la existencia de) un tour de caballo completo en un tablero $n \times n$.
- Tour **cerrado** (el último movimiento vuelve a atacar la casilla inicial): misma idea, chequeo extra al final.
- Cualquier problema de "recorrer todas las celdas exactamente una vez respetando un patrón de movimiento" (no exclusivo del caballo): el patrón general es backtracking + poda de Warnsdorff.

Complejidad: exponencial en el peor caso sin poda ($O(8^n)$ aproximado). Con la **heurística de Warnsdorff** —en cada paso, moverse primero a la casilla accesible con **menos** salidas futuras disponibles— en la práctica encuentra un tour en tiempo casi lineal para tableros razonables ($n \leq 100$), aunque no hay garantía teórica formal de que siempre funcione.

```

1 struct TourCaballo {
2     int n;
3     vector<vector<int>> tablero; // tablero[r][c] = orden de
        visita
4     // (-1 si no visitada)
5     int dr[8] = {-2,-2,-1,-1, 1, 1, 2, 2};
6     int dc[8] = {-1, 1,-2, 2,-2, 2,-1, 1};
7
8     // CREATE: tablero n x n vacío.
9     TourCaballo(int n_) {
10         n = n_;
11         tablero.assign(n, vector<int>(n, -1));
12     } // O(n^2)
13
14     bool dentro(int r, int c) {
15         return r >= 0 && r < n && c >= 0 && c < n;
16     }
17
18     // cuenta a cuantas casillas libres puede saltar (r,c) -
        usado
19     // por Warnsdorff para ordenar los candidatos.
20     int gradoSalida(int r, int c) {
21         int cont = 0;
22         for (int d = 0; d < 8; d++) {
23             int nr = r + dr[d], nc = c + dc[d];
24             if (dentro(nr, nc) && tablero[nr][nc] == -1) cont++;
25         }
26         return cont;
27     }
28
29     // READ: intenta construir un tour completo empezando en
        (sr, sc). Devuelve true si lo logra (tablero queda lleno).
30     bool resolver(int sr, int sc) {
31         tablero[sr][sc] = 0;
32         return backtrack(sr, sc, 1);
33     }

```

```

34 } // O(casi lineal)
35 // Warnsdorff en la
36 // práctica
37
38 bool backtrack(int r, int c, int movimiento) {
39     if (movimiento == n * n) return true;
40
41     // candidatos ordenados por Warnsdorff: menor grado de
        // salida primero
42     vector<tuple<int,int,int>> candidatos;
43     for (int d = 0; d < 8; d++) {
44         int nr = r + dr[d], nc = c + dc[d];
45         if (dentro(nr, nc) && tablero[nr][nc] == -1)
46             candidatos.push_back({gradoSalida(nr, nc), nr, nc});
47     }
48     sort(candidatos.begin(), candidatos.end());
49
50     for (auto& [grado, nr, nc] : candidatos) {
51         tablero[nr][nc] = movimiento;
52
53         if (backtrack(nr, nc, movimiento + 1)) return true;
54
55         tablero[nr][nc] = -1; // deshacer (backtrack real)
56     }
57
58     return false;
59 }
60 };

```

Ejemplo de funcionamiento:

```

1 TourCaballo tc(5);
2
3 bool encontrado = tc.resolver(0, 0);
4
5 cout << encontrado;
6 // true
7
8 // tc.tablero[r][c] tiene el orden (0..24) en que se visito
9 // cada casilla, por ejemplo:
10 // 0 3 16 9 22
11 // 13 8 21 4 17
12 // 2 1 12 23 10
13 // 7 14 19 18 5
14 // 20 ... etc.

```

Nota: sin la poda de Warnsdorff, el backtrack puro también es correcto pero mucho más lento (puede no terminar en tiempo útil para $n > 6$ u 8). Warnsdorff no cambia la corrección del algoritmo, solo el **orden** en que se prueban los candidatos — sigue siendo backtracking con deshacer explícito.

6.11 Mínimo Número de Movimientos de Caballo

Idea: a diferencia del Tour de Caballo, aquí no importa visitar todo el tablero — solo se pide la **distancia mínima** (en número de saltos) entre una casilla origen y una casilla destino. Eso es "camino más corto con costo uniforme" sobre un grafo implícito donde cada casilla es un nodo y sus hasta 8 movimientos de caballo son las aristas: exactamente el mismo patrón que Laberinto con BFS, cambiando el arreglo de movimientos de 4/8 direcciones ortogonales por los 8 saltos en "L" del caballo.

¿Por qué usarlo? Es tentador intentar resolver esto con fórmulas o backtracking (como el Tour), pero al ser "distancia mínima" y no "visitar todo", BFS es tanto más simple como garantizado óptimo — la primera vez que BFS alcanza la casilla destino, esa es la respuesta.

Casos de uso:

- Mínimo número de movimientos de caballo entre dos casillas de un tablero $n \times n$ (o infinito, acotando la búsqueda).
- Mismo problema con casillas bloqueadas/prohibidas: se salta la casilla en el BFS igual que una pared en un laberinto.

- Generalización a "cuál es la distancia mínima con un patrón de movimiento arbitrario" (no solo caballo): el patrón se sigue resolviendo con BFS cambiando dr/dc.

Complejidad: $O(n^2)$ para un tablero $n \times n$, ya que cada casilla se encola como máximo una vez y desde cada una se prueban 8 movimientos constantes.

```

1 struct MinMovimientosCaballo {
2     int n;
3     vector<vector<int>>> dist;
4
5     int dr[8] = {-2,-2,-1,-1, 1, 1, 2, 2};
6     int dc[8] = {-1, 1,-2, 2,-2, 2,-1, 1};
7
8     // CREATE: tablero n x n.
9     MinMovimientosCaballo(int n_) {
10         n = n_;
11         // 0(1)
12
13         bool dentro(int r, int c) {
14             return r >= 0 && r < n && c >= 0 && c < n;
15         }
16
17         // READ: BFS desde (sr, sc); dist[r][c] = mínimo de
18         // movimientos
19         // de caballo para llegar a (r, c), -1 si no aplica (no
20         // debería
21         // pasar en un tablero cerrado y conexo).
22         int minMovimientos(int sr, int sc, int er, int ec) {
23             dist.assign(n, vector<int>(n, -1));
24
25             queue<pair<int,int>> q;
26             q.push({sr, sc});
27             dist[sr][sc] = 0;
28
29             while (!q.empty()) {
30                 auto [r, c] = q.front(); q.pop();
31
32                 if (r == er && c == ec) return dist[r][c];
33
34                 for (int d = 0; d < 8; d++) {
35                     int nr = r + dr[d], nc = c + dc[d];
36
37                     if (!dentro(nr, nc)) continue;
38                     if (dist[nr][nc] != -1) continue;
39
40                     dist[nr][nc] = dist[r][c] + 1;
41                     q.push({nr, nc});
42                 }
43             }
44             return -1; // no alcanzable (no debería pasar sin
45             // bloqueos) // 0(n^2)
46 };

```

Ejemplo de funcionamiento:

```

1 MinMovimientosCaballo mc(8);
2
3 cout << mc.minMovimientos(0, 0, 1, 2);
4 // 1 (un solo salto en L)
5
6 cout << mc.minMovimientos(0, 0, 7, 7);
7 // 6 (esquina a esquina en un tablero 8x8)
8
9 cout << mc.minMovimientos(0, 0, 0, 1);
10 // 3 (casilla adyacente, contraintuitivamente lejos para un
    caballo)

```

Regla práctica: misma distinción que Flood Fill vs. Laberinto con BFS, aplicada al caballo: si el problema pide "visitar todo" sin repetir, es backtracking (Tour de Caballo, con Warnsdorff como poda). Si pide "distancia mínima entre dos puntos", es BFS (Mínimo Número de Movimientos), sin importar qué tan raro sea el patrón de movimiento — el patrón solo define el arreglo de vecinos.

6.12 Serpientes y Escaleras

Idea: el tablero de Serpientes y Escaleras se modela como un grafo dirigido: cada casilla $1..n$ es un nodo, y

desde cada casilla i hay hasta 6 aristas (una por cada valor del dado, $i + 1$ hasta $i + 6$), **redirigidas** si la casilla de llegada tiene una serpiente o escalera. Pedir el "mínimo número de tiradas de dado para llegar de la casilla 1 a la n " es, otra vez, camino más corto con costo uniforme (cada tirada cuenta como 1 paso) — el mismo BFS de Laberinto con BFS y Mínimo Número de Movimientos de Caballo, cambiando qué cuenta como "vecino" de una casilla.

¿Por qué usarlo? La parte no trivial no es el BFS en sí — es idéntico al de siempre — sino **modelar** serpientes y escaleras correctamente: no son casillas nuevas, son **atajos/retrocesos automáticos**. Si aterrizas en la casilla de la base de una escalera o la cabeza de una serpiente, tu posición real cambia **sin gastar otra tirada**. Eso se resuelve precomputando, para cada casilla, a dónde "realmente" te manda (`destinoReal[i]`), y haciendo BFS sobre ese arreglo en vez de sobre los números crudos del dado.

Casos de uso:

- Mínimo número de tiradas de dado para llegar de la casilla 1 (o cualquier inicio) a la casilla n .
- Variante donde el dado no es 1–6 sino un conjunto arbitrario de valores posibles: solo cambia el rango del for al generar vecinos.
- Cualquier tablero lineal con "teletransportes" fijos (serpiente = te manda hacia atrás, escalera = te manda hacia adelante) donde se pide distancia mínima en pasos.

Complejidad: $O(n)$, ya que el grafo tiene n nodos y a lo más $6n$ aristas — cada casilla se encola una sola vez.

```

1 struct SerpientesYEscaleras {
2     int n;
3     vector<int> destinoReal; // destinoReal[i] = a donde
4     // termina
5     // realmente el jugador si cae en i
6     // (i mismo si no hay serpiente/escalera)
7
8     // CREATE: tablero de n casillas (1-indexado). saltos[i] = j
9     // significa "serpiente o escalera de i a j"; las casillas
10    // sin
11    // entrada en el mapa no tienen salto.
12    SerpientesYEscaleras(int n_, map<int,int>& saltos) {
13        n = n_;
14        destinoReal.assign(n + 1, 0);
15
16        for (int i = 1; i <= n; i++)
17            destinoReal[i] = saltos.count(i) ? saltos[i] : i;
18    } // 0(n)
19
20    // READ: mínimo número de tiradas de dado (1-6) para llegar
21    // de
22    // origen a n. -1 si es imposible (no debería pasar en un
23    // tablero estándar).
24    int minTiradas(int origen) {
25        vector<int> dist(n + 1, -1);
26        queue<int> q;
27
28        q.push(origen);
29        dist[origen] = 0;
30
31        while (!q.empty()) {
32            int actual = q.front(); q.pop();
33
34            if (actual == n) return dist[actual];
35
36            for (int dado = 1; dado <= 6; dado++) {
37                int siguiente = actual + dado;
38                if (siguiente > n) continue;
39
40                siguiente = destinoReal[siguiente]; // aplica
41                // serpiente/escalera
42            }
43        }
44    }
45 };

```

```

39     if (dist[siguiente] != -1) continue;
40
41     dist[siguiente] = dist[actual] + 1;
42     q.push(siguiente);
43 }
44 }
45
46 return -1;
47 }
48 // O(n)
49 };

```

Ejemplo de funcionamiento:

```

1 // tablero de 10 casillas:
2 // escalera de 2 a 8
3 // serpiente de 9 a 3
4
5 map<int,int> saltos = {{2, 8}, {9, 3}};
6 SerpientesYEscaleras se(10, saltos);
7
8 cout << se.minTiradas(1);
9 // 2
10 // tirada 1: 1 -> 2 -> (escalera) -> 8
11 // tirada 2: 8 -> 10

```

Regla práctica: cualquier tablero lineal donde te mueves por "reglas de dado" (o cualquier conjunto de deltas fijos) y hay teletransportes automáticos al aterrizar en ciertas casillas, se reduce a BFS de toda la vida sobre un grafo de n nodos — **la única parte específica del problema es construir bien el arreglo destinoReal** antes de correr el BFS; el algoritmo de búsqueda no cambia respecto a los casos anteriores.

7 Conectividad

7.1 Componentes Conexas

Idea: en un grafo **no dirigido**, una componente conexa es un grupo maximal de nodos donde cualquiera puede llegar a cualquiera. Encontrarlas todas es simplemente: recorrer los nodos en orden, y cada vez que encuentres uno **no visitado**, lanzar un DFS/BFS desde ahí para marcar todo su grupo con un mismo identificador — ese DFS/BFS nunca se sale del grupo porque, por definición, no hay aristas hacia afuera de una componente.

¿Por qué usarlo? Es el primer paso obligatorio de casi cualquier problema de grafos no dirigidos: antes de responder "¿ u y v están conectados?" o de aplicar un algoritmo que asume un grafo conexo, necesitas saber cuántas componentes hay y a cuál pertenece cada nodo. La alternativa es **Union-Find**: si solo necesitas responder "¿misma componente?" con aristas que van llegando una a una (sin necesariamente construir listas de adyacencia), Union-Find es más directo; si ya tienes el grafo completo y quieres además reconstruir/recorrer cada componente, DFS/BFS es más natural.

Casos de uso:

- Contar cuántas componentes conexas tiene un grafo (o una grilla vista como grafo — mismo patrón que Flood Fill, pero sobre listas de adyacencia en vez de vecinos de grilla).
- Etiquetar cada nodo con el id de su componente, para responder "¿misma componente?" en $O(1)$ tras el precómputo.
- Encontrar el tamaño de la componente más grande.
- Verificar si el grafo completo es conexo (una sola componente).

Complejidad: $O(V + E)$, ya que es un DFS/BFS normal que simplemente se relanza desde cada nodo no visitado — en total, cada nodo y cada arista se procesan una sola vez sin importar cuántas componentes haya.

```

1 struct ComponentesConexas {
2     vector<vector<int>>> adj;
3     vector<int> comp; // comp[u] = id de la componente de u,
4                       // si no se ha calculado
5     int numComponentes;
6
7     // CREATE: guarda el grafo y calcula todas las componentes de
8     // una vez, recorriendo cada nodo no visitado.
9     ComponentesConexas(vector<vector<int>>& adj_, int n) {
10         adj = adj_;
11         comp.assign(n, -1);
12         numComponentes = 0;
13
14         for (int u = 0; u < n; u++) {
15             if (comp[u] == -1) {
16                 dfs(u, numComponentes);
17                 numComponentes++;
18             }
19         }
20     } // O(V + E)
21
22     void dfs(int u, int id) {
23         comp[u] = id;
24         for (int v : adj[u])
25             if (comp[v] == -1)
26                 dfs(v, id);
27     }
28
29     // READ: u y v están en la misma componente?
30     bool mismaComponente(int u, int v) {
31         return comp[u] == comp[v];
32     } // O(1)
33 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1   2 - 3   4
2 //         (dos componentes de tamaño 2, una de tamaño 1)
3
4 vector<vector<int>>> adj(5);
5 adj[0] = {1}; adj[1] = {0};
6 adj[2] = {3}; adj[3] = {2};
7 adj[4] = {};
8
9 ComponentesConexas cc(adj, 5);
10
11 cout << cc.numComponentes;
12 // 3
13
14 cout << cc.mismaComponente(0, 1);
15 // true
16
17 cout << cc.mismaComponente(1, 2);
18 // false

```

7.2 Detección de Ciclos

Idea: detectar si un grafo tiene un ciclo se hace con DFS, pero la regla para reconocer un ciclo **cambia según el grafo sea dirigido o no**: en un grafo no dirigido, cualquier arista hacia un nodo ya visitado que **no sea tu padre inmediato** es un ciclo; en un grafo dirigido, una arista hacia un nodo ya visitado **solo** es ciclo si ese nodo sigue en la pila de recursión actual (todavía no terminaste de procesarlo) — si ya terminaste con él, es una arista válida hacia otra rama, no un ciclo.

¿Por qué usarlo? Confundir estas dos reglas es el error más común al detectar ciclos: aplicar la regla de "no dirigido" (solo evitar al padre) a un grafo dirigido da falsos positivos, porque en dirigido puedes visitar el mismo nodo desde dos ramas distintas sin que eso sea un ciclo.

Casos de uso:

- Verificar si un grafo dirigido es un DAG (sin ciclos) antes de aplicar orden topológico, DP sobre DAG, etc.
- Detectar dependencias circulares (tareas que dependen unas de otras en círculo, imposibles de ordenar).
- En grafos no dirigidos: verificar si un grafo es un árbol ($V - 1$ aristas, conexo, sin ciclos) o encontrar el primer ciclo para removerlo.

Complejidad: $O(V + E)$ en ambos casos — un DFS normal con una condición extra por arista, sin repetir trabajo.

En grafos no dirigidos (DFS con padre)

```

1 struct CicloNoDirigido {
2     vector<vector<int>> adj;
3     vector<bool> visitado;
4
5     // CREATE: guarda el grafo.
6     CicloNoDirigido(vector<vector<int>>& adj_) {
7         adj = adj_;
8         visitado.assign(adj.size(), false);
9     }
10
11    // READ: existe algun ciclo en todo el grafo (revisa todas
12    // las
13    // componentes).
14    bool tieneCiclo() {
15        for (int u = 0; u < (int)adj.size(); u++)
16            if (!visitado[u] && dfs(u, -1))
17                return true;
18        return false;
19    }
20
21    bool dfs(int u, int padre) {
22        visitado[u] = true;
23
24        for (int v : adj[u]) {
25            if (!visitado[v]) {
26                if (dfs(v, u)) return true;
27            } else if (v != padre) {
28                return true; // arista hacia un visitado que no es
29                // el padre -> ciclo
30            }
31        }
32        return false;
33    }
34 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo, tiene ciclo)
2 //         3 - 4      (arbol, sin ciclo)
3
4 vector<vector<int>> adj(5);
5 adj[0] = {1, 2}; adj[1] = {0, 2}; adj[2] = {0, 1};
6 adj[3] = {4}; adj[4] = {3};
7
8 CicloNoDirigido cnd(adj);
9
10 cout << cnd.tieneCiclo();
11 // true (por el triangulo 0-1-2)

```

En grafos dirigidos (DFS con estados / colores)

Idea: cada nodo tiene 3 estados durante el DFS: 0 = no visitado, 1 = en la pila de recursión actual (gris), 2 = totalmente procesado (negro). Una arista hacia un nodo **gris** es un back edge real (ciclo); una arista hacia un nodo **negro** es válida (cross/forward edge, no ciclo).

```

1 struct CicloDirigido {
2     vector<vector<int>> adj;
3     vector<int> estado; // 0 = blanco, 1 = gris, 2 = negro
4
5     // CREATE: guarda el grafo dirigido.

```

```

6     CicloDirigido(vector<vector<int>>& adj_) {
7         adj = adj_;
8         estado.assign(adj.size(), 0);
9     }
10
11    // READ: existe algun ciclo en todo el grafo.
12    bool tieneCiclo() {
13        for (int u = 0; u < (int)adj.size(); u++)
14            if (estado[u] == 0 && dfs(u))
15                return true;
16        return false;
17    }
18
19    bool dfs(int u) {
20        estado[u] = 1; // entra a la pila de recursion (gris)
21
22        for (int v : adj[u]) {
23            if (estado[v] == 1) return true; // back edge ->
24            // ciclo
25            if (estado[v] == 0 && dfs(v)) return true;
26        }
27
28        estado[u] = 2; // termino con u (negro)
29        return false;
30    }
31 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo)
2 //                 3 -> 1      (sin ciclo: 3->1 es
3 //                 valida,
4 //                 procesado
5 //                 MISMA
6 //                 solo si viene de la
7 //                 rama de recursion)
8
9 vector<vector<int>> adj(4);
10 adj[0] = {1};
11 adj[1] = {2};
12 adj[2] = {0};
13 adj[3] = {1};
14
15 CicloDirigido cd(adj);
16
17 cout << cd.tieneCiclo();
18 // true (0 -> 1 -> 2 -> 0)

```

Regla práctica: para saber cuál versión aplicar, primero pregúntate si el grafo es dirigido. Si no lo es, basta con recordar "de dónde vine" (el padre) y listo. Si es dirigido, "de dónde vine" no alcanza — necesitas saber si el nodo destino sigue **activo en la recursión actual** (gris) o ya quedó cerrado (negro), porque en dirigido sí es válido llegar dos veces al mismo nodo desde ramas distintas sin que eso sea un ciclo.

7.3 Puentes

Idea: en un grafo no dirigido, un **punto** es una arista que, al quitarla, **desconecta** el grafo (aumenta el número de componentes conexas). Se encuentran con un solo DFS, llevando dos valores por nodo: **tin**[u] (el momento en que el DFS **descubre** a u , un contador que se incrementa en cada visita) y **low**[u] (el **tin más bajo** alcanzable desde u usando cualquier camino del árbol DFS más, como mucho, **una** arista de "retroceso" hacia un ancestro). La arista (u, v) del árbol DFS es puente si y solo si **low**[v] > **tin**[u]: eso significa que, desde el subárbol de v , **no hay ningún atajo** de vuelta hacia u o más arriba —quitar esa arista realmente aísla el subárbol de v .

¿Por qué usarlo? Es la aplicación clásica del algoritmo de Tarjan de **low-link**: en vez de, para cada arista, quitarla y volver a correr un DFS/BFS completo para ver si el grafo se desconecta ($O(E \cdot (V + E))$ en total), un solo DFS con **low-link** responde para **todas** las aristas a la vez en tiempo lineal.

Casos de uso:

- Encontrar todas las conexiones "críticas" de una red (si se cae ese enlace, la red se parte en dos).
- Como paso previo para construir el **árbol de componentes 2-arista-conexas** (contraer cada componente 2-arista-conexa a un nodo y dejar solo los puentes como aristas).
- Verificar si el grafo sigue siendo conexo tras remover cualquier arista dada (equivalente a "¿esa arista es puente?").

Complejidad: $O(V + E)$, un solo DFS con trabajo constante extra por arista.

```

1 struct Puentes {
2     vector<vector<pair<int,int>>> adj;    // adj[u] = {(v,
        idArista)}
3     vector<int> tin, low;
4     vector<bool> visitado;
5     vector<pair<int,int>> puentes;      // aristas (u,v) que son
        puente
6     int timer;
7
8     // CREATE: guarda el grafo (con id de arista para no reusar
        la
9     // misma arista en sentido contrario) y corre el DFS de
        Tarjan
10    // desde cada componente.
11    Puentes(vector<vector<pair<int,int>>& adj_, int n) {
12        adj = adj_;
13        tin.assign(n, -1);
14        low.assign(n, -1);
15        visitado.assign(n, false);
16        timer = 0;
17
18        for (int u = 0; u < n; u++)
19            if (!visitado[u])
20                dfs(u, -1);
21    }                                     // O(V + E)
22
23    void dfs(int u, int idAristaPadre) {
24        visitado[u] = true;
25        tin[u] = low[u] = timer++;
26
27        for (auto& [v, idArista] : adj[u]) {
28            if (idArista == idAristaPadre) continue; // no te
29            // regresas
30            // por la
31            // misma
32            // arista
33
34            if (visitado[v]) {
35                low[u] = min(low[u], tin[v]); // back edge
36            } else {
37                dfs(v, idArista);
38                low[u] = min(low[u], low[v]);
39
40                if (low[v] > tin[u])
41                    puentes.push_back({u, v});
42            }
43        }
44    }
45 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo)
2 //          \
3 //          3 - 4 (colgando del triangulo)
4
5 vector<vector<pair<int,int>>> adj(5);
6 auto addEdge = [&](int u, int v, int id) {
7     adj[u].push_back({v, id});
8     adj[v].push_back({u, id});
9 };
10 addEdge(0, 1, 0);
11 addEdge(1, 2, 1);
12 addEdge(2, 0, 2);
13 addEdge(2, 3, 3);
14 addEdge(3, 4, 4);
15
16 Puentes p(adj, 5);
17
18 for (auto& [u, v] : p.puentes)

```

```

19 cout << u << " - " << v << " ";
20 // 2-3 3-4
21 // (las aristas del triangulo NO son puente: hay un ciclo,
22 // siempre queda un camino alternativo)

```

7.4 Puntos de Articulación

Idea: un **punto de articulación** (o *cut vertex*) es un nodo que, al quitarlo (junto con sus aristas), **aumenta** el número de componentes conexas. Usa el mismo **tin/low** que Puentes, pero la condición es distinta y tiene **dos casos**: (1) si u es la **raíz** del árbol DFS, es articulación si tiene **2 o más hijos** en el árbol DFS (cada hijo es un subárbol que solo se conecta al resto *a través de u*); (2) si u **no** es la raíz, es articulación si tiene algún hijo v con $\text{low}[v] \geq \text{tin}[u]$ (el subárbol de v no tiene ningún atajo que lo saque **por encima** de u , así que sin u ese subárbol queda aislado).

¿Por qué usarlo? Es el "primo" de Puentes: mismo DFS, mismos **tin/low**, pero identificando nodos críticos en vez de aristas críticas. La diferencia clave con Puentes está en el operador de comparación ($\geq$ en vez de $>$) y en el caso especial de la raíz — con solo $>$ (la condición de puente) se pierde el caso donde v puede volver exactamente a u (eso no aísla la arista, pero sí deja a u como único punto de paso).

Casos de uso:

- Encontrar los nodos "críticos" de una red (si ese nodo falla, la red se parte).
- Como paso previo para construir componentes **bi-conexas** (bloques del grafo separados por puntos de articulación).
- Distinguir de Puentes: un grafo puede tener puntos de articulación sin tener ningún puente (ej. dos triángulos que comparten un solo vértice) — son propiedades independientes.

Complejidad: $O(V + E)$, mismo costo que Puentes: un DFS con trabajo constante extra por nodo/arista.

```

1 struct PuntosArticulacion {
2     vector<vector<int>> adj;
3     vector<int> tin, low;
4     vector<bool> visitado, esArticulacion;
5     int timer;
6
7     // CREATE: guarda el grafo y corre el DFS de Tarjan desde
        cada
8     // componente.
9     PuntosArticulacion(vector<vector<int>>& adj_, int n) {
10        adj = adj_;
11        tin.assign(n, -1);
12        low.assign(n, -1);
13        visitado.assign(n, false);
14        esArticulacion.assign(n, false);
15        timer = 0;
16
17        for (int u = 0; u < n; u++)
18            if (!visitado[u])
19                dfs(u, -1);
20    }                                     // O(V + E)
21
22    void dfs(int u, int padre) {
23        visitado[u] = true;
24        tin[u] = low[u] = timer++;
25        int hijos = 0;
26
27        for (int v : adj[u]) {
28            if (v == padre) continue; // ignora SOLO una vez la
29            // arista de vuelta al padre
30            // (si hay aristas multiples
31            // u-padre, usar id de arista
32            // como en Puentes)

```

```

33     if (visitado[v]) {
34         low[u] = min(low[u], tin[v]); // back edge
35     } else {
36         dfs(v, u);
37         low[u] = min(low[u], low[v]);
38         hijos++;
39     }
40
41     if (padre != -1 && low[v] >= tin[u])
42         esArticulacion[u] = true;
43 }
44 }
45
46 if (padre == -1 && hijos >= 2)
47     esArticulacion[u] = true; // caso especial: raíz con
48     // 2+ hijos en el árbol DFS
49 }
50 };

```

Ejemplo de funcionamiento:

```

1 // grafo:  0 - 1 - 2 - 0    (triangulo)
2 //          \
3 //          3 - 4    (colgando del triangulo)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1, 2}; adj[1] = {0, 2}; adj[2] = {0, 1, 3};
7 adj[3] = {2, 4}; adj[4] = {3};
8
9 PuntosArticulacion pa(adj, 5);
10
11 for (int u = 0; u < 5; u++)
12     if (pa.esArticulacion[u])
13         cout << u << " ";
14 // 2 3
15 // (2 conecta el triangulo con la cola 3-4; 3 conecta 2 con 4)

```

Regla práctica: Puentes y Puntos de Articulación comparten exactamente el esqueleto del DFS (**tin/low**, back edges hacia ancestros) — la diferencia está solo en **qué** marcas: en Puentes marcas la **arista** (u, v) cuando $\text{low}[v] > \text{tin}[u]$ (estricto); en Puntos de Articulación marcas el **nodo** u cuando $\text{low}[v] \geq \text{tin}[u]$ (no estricto, más el caso especial de la raíz). Si programas uno, el otro es casi copiar y pegar cambiando esos dos detalles.

7.5 Componentes Biconectados

Idea: una componente biconexa (o *2-vertex-connected component*, "bloque") es un conjunto **maximal de aristas** donde no hay ningún punto de articulación que las separe — equivalentemente, cualquier par de aristas del bloque está en algún ciclo común. Se calculan con el mismo DFS de **tin/low** de Puentes y Puntos de Articulación, agregando una **pila de aristas**: apilas cada arista al recorrerla, y cuando detectas la condición de punto de articulación ($\text{low}[v] \geq \text{tin}[u]$), **desapilas** todo hasta la arista (u, v) inclusive — ese grupo desapilado es una componente biconexa completa.

¿Por qué usarlo? Puntos de Articulación te dice **qué nodos** son críticos, pero no cómo se agrupan las aristas alrededor de ellos. Componentes Biconectadas completa el panorama: parte el grafo en bloques tales que cada punto de articulación es exactamente el nodo compartido entre dos o más bloques — es la base para construir el **block-cut tree** (árbol de bloques y puntos de articulación), útil cuando el problema pregunta por relaciones entre bloques en vez de solo "¿es articulación sí/no?".

Casos de uso:

- Agrupar aristas en bloques maximales sin puntos de articulación internos.

- Construir el block-cut tree para responder queries tipo "¿cuántos bloques hay que atravesar para ir de u a v ?".
- Verificar si el grafo completo es biconexo (una sola componente biconexa que cubre todas las aristas, y ningún punto de articulación).

Complejidad: $O(V + E)$ — mismo DFS de **tin/low** de siempre, más trabajo amortizado constante por arista al apilarla/desapilarla (cada arista se apila y desapila una sola vez en total).

```

1 struct ComponentesBiconexas {
2     vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
3         idArista)}
4     vector<int> tin, low;
5     vector<bool> visitado;
6     stack<pair<int,int>> pilaAristas; // aristas (u,v)
7         pendientes
8     vector<vector<pair<int,int>>> bloques;
9     int timer;
10
11     // CREATE: guarda el grafo y corre el DFS desde cada
12     // componente.
13     ComponentesBiconexas(vector<vector<pair<int,int>>>& adj_, int
14         n) {
15         adj = adj_;
16         tin.assign(n, -1);
17         low.assign(n, -1);
18         visitado.assign(n, false);
19         timer = 0;
20
21         for (int u = 0; u < n; u++)
22             if (!visitado[u])
23                 dfs(u, -1);
24     } // O(V + E)
25
26     void dfs(int u, int idAristaPadre) {
27         visitado[u] = true;
28         tin[u] = low[u] = timer++;
29
30         for (auto& [v, idArista] : adj[u]) {
31             if (idArista == idAristaPadre) continue;
32
33             if (visitado[v] && tin[v] < tin[u]) {
34                 // back edge: entra al bloque actual
35                 pilaAristas.push({u, v});
36                 low[u] = min(low[u], tin[v]);
37             } else if (!visitado[v]) {
38                 pilaAristas.push({u, v});
39                 dfs(v, idArista);
40                 low[u] = min(low[u], low[v]);
41
42                 if (low[v] >= tin[u]) {
43                     // u es articulación (o raíz): cierra un bloque,
44                     // desapilando hasta (u, v) inclusive
45                     vector<pair<int,int>> bloque;
46                     while (pilaAristas.top() != make_pair(u, v)) {
47                         bloque.push_back(pilaAristas.top());
48                         pilaAristas.pop();
49                     }
50                     bloque.push_back(pilaAristas.top());
51                     pilaAristas.pop();
52                     bloques.push_back(bloque);
53                 }
54             }
55         }
56     }
57 };

```

Ejemplo de funcionamiento:

```

1 // grafo:  0 - 1 - 2 - 0    (triangulo, bloque 1)
2 //          \
3 //          3 - 4    (puente, bloque 2 y 3 por
4             separado)
5
6 vector<vector<pair<int,int>>> adj(5);
7 auto addEdge = [&](int u, int v, int id) {
8     adj[u].push_back({v, id});
9     adj[v].push_back({u, id});
10 };
11 addEdge(0, 1, 0);
12 addEdge(1, 2, 1);
13 addEdge(2, 0, 2);
14 addEdge(2, 3, 3);
15 addEdge(3, 4, 4);

```

```

15 ComponentesBiconexas cb(adj, 5);
16
17
18 cout << cb.bloques.size();
19 // 3
20 // bloque 1: {(0,1),(1,2),(2,0)} (el triangulo)
21 // bloque 2: {(2,3)} (el puente 2-3)
22 // bloque 3: {(3,4)} (el puente 3-4)

```

Nota: cada punto de articulación aparece en **más de un** bloque (es el nodo "bisagra" entre ellos); cada nodo que no es articulación aparece en exactamente un bloque. Eso es justo lo que permite construir el block-cut tree: un nodo por cada bloque, un nodo por cada punto de articulación, y una arista entre un bloque y cada punto de articulación que contiene.

7.6 Tarjan

Idea: el algoritmo de Tarjan encuentra las **componentes fuertemente conexas** (SCC) de un grafo **dirigido**: grupos maximales de nodos donde **cada** nodo puede llegar a **cualquier otro** del mismo grupo (yendo por aristas dirigidas). Es el mismo esqueleto de **tin/low** que Puentes/Articulación/Biconexas, adaptado a dirigido: mantienes una **pila de nodos** (no de aristas) y un flag **enPila[u]**; un nodo u es **raíz de su SCC** cuando $\text{low}[u] == \text{tin}[u]$ — en ese momento desapilas todo hasta u inclusive, y eso es una SCC completa.

¿Por qué usarlo? Es la forma estándar de **condensar** un grafo dirigido general en un DAG: contrae cada SCC a un solo nodo, y las aristas entre SCCs distintas forman un DAG sobre el cual sí puedes aplicar orden topológico, DP sobre DAG, o Conteo de Caminos — todo lo que requiere ausencia de ciclos. Es la alternativa a **Kosaraju** (dos pasadas de DFS + grafo transpuesto): Tarjan logra lo mismo en **una sola pasada**, a cambio de llevar la pila de nodos y el flag **enPila**.

Casos de uso:

- Condensar un grafo dirigido con ciclos en un DAG, como paso previo a Conteo de Caminos, orden topológico, o DP sobre DAG.
- Detectar si un grafo dirigido es fuertemente conexo (una sola SCC que cubre todos los nodos).
- Encontrar el tamaño de la SCC más grande (por ejemplo, "el mayor grupo de cuentas que se siguen mutuamente").

Complejidad: $O(V + E)$, un solo DFS con trabajo constante extra por nodo/arista, igual que las estructuras anteriores de esta sección.

```

1 struct Tarjan {
2     vector<vector<int>> adj;
3     vector<int> tin, low, comp;
4     vector<bool> enPila;
5     stack<int> pila;
6     int timer, numSCC;
7
8     // CREATE: guarda el grafo dirigido y corre Tarjan desde cada
9     // nodo no visitado.
10    Tarjan(vector<vector<int>>& adj_, int n) {
11        adj = adj_;
12        tin.assign(n, -1);
13        low.assign(n, -1);
14        comp.assign(n, -1); // comp[u] = id de la SCC de u
15        enPila.assign(n, false);
16        timer = 0;
17        numSCC = 0;

```

```

19 for (int u = 0; u < n; u++)
20     if (tin[u] == -1)
21         dfs(u); // O(V + E)
22 }
23
24 void dfs(int u) {
25     tin[u] = low[u] = timer++;
26     pila.push(u);
27     enPila[u] = true;
28
29     for (int v : adj[u]) {
30         if (tin[v] == -1) {
31             dfs(v);
32             low[u] = min(low[u], low[v]);
33         } else if (enPila[v]) {
34             // v esta en la pila -> back edge dentro de la
35             // misma SCC en formacion
36             low[u] = min(low[u], tin[v]);
37         }
38         // si v ya se visito y NO esta en pila, es una SCC ya
39         // cerrada -> se ignora (no afecta low[u])
40     }
41
42     if (low[u] == tin[u]) {
43         // u es raiz de su SCC: desapila todo el grupo
44         while (true) {
45             int w = pila.top(); pila.pop();
46             enPila[w] = false;
47             comp[w] = numSCC;
48             if (w == u) break;
49         }
50         numSCC++;
51     }
52 }
53 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo, una SCC)
2 //                2 -> 3
3 //                3 -> 4 -> 3 (ciclo, otra SCC)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};
10 adj[4] = {3};
11
12 Tarjan tj(adj, 5);
13
14 cout << tj.numSCC;
15 // 2
16
17 cout << tj.comp[0] << " " << tj.comp[1] << " " << tj.comp[2];
18 // mismo id para 0, 1, 2 (SCC del ciclo 0-1-2)
19
20 cout << tj.comp[3] << " " << tj.comp[4];
21 // mismo id para 3, 4 (SCC del ciclo 3-4)

```

Regla práctica: de las cuatro estructuras de esta sección, todas comparten el DFS con **tin/low**, pero difieren en **qué apilas y cuándo cierras el grupo**: Puentes no apila nada (marca aristas directamente); Puntos de Articulación tampoco apila (marca nodos); Componentes Biconexas apila **aristas** y cierra con $\text{low}[v] \geq \text{tin}[u]$; Tarjan apila **nodos** y cierra con $\text{low}[u] == \text{tin}[u]$ — y solo Tarjan aplica a grafos **dirigidos**, ya que las otras tres asumen aristas no dirigidas (por eso usan el truco de "no regresar por la misma arista al padre").

8 Topological Sort

8.1 Kahn's Algorithm

Idea: Kahn's Algorithm calcula un orden topológico de un DAG **sin recursión**, usando la idea de "quitar capas": empiezas encolando todos los nodos con **grado de entrada 0** (sin prerequisites), y cada vez que "procesas" un nodo, le restas 1 al grado de entrada de sus vecinos — si alguno llega a 0, significa que ya se cumplieron todos sus prerequisites, así que se encola. Es exactamente

el mismo patrón de BFS por capas de Laberinto con BFS, pero la "distancia" que se propaga es el conteo de prerequisites pendientes en vez de pasos.

¿Por qué usarlo? Además de dar el orden, Kahn's Algorithm te da **gratis** la detección de ciclos: si al final quedan nodos sin procesar (el orden resultante tiene menos de n nodos), el grafo **no** es un DAG. Es la alternativa iterativa a DFS + Orden Topológico — preferible cuando quieres evitar recursión profunda (no hay riesgo de stack overflow en grafos grandes) o cuando ya necesitas el chequeo de ciclo como parte del mismo cómputo.

Casos de uso:

- Ordenar tareas con dependencias (build systems, prerequisites de materias) respetando que cada tarea aparezca después de sus dependencias.
- Verificar si un conjunto de dependencias es realizable (sin ciclos) — si Kahn no logra procesar todos los nodos, hay un ciclo.
- Como precómputo para DP sobre DAG (procesar los nodos en este orden garantiza que, al llegar a u , ya se resolvieron todos sus predecesores).

Complejidad: $O(V + E)$ — cada nodo se encola/desencola una vez, y cada arista se examina una vez al decrementar grados de entrada.

```

1 struct KahnTopoSort {
2     vector<vector<int>> adj;
3     vector<int> gradoEntrada;
4     int n;
5
6     // CREATE: guarda el grafo y calcula el grado de entrada de
7     // cada nodo.
8     KahnTopoSort(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        gradoEntrada.assign(n, 0);
12
13        for (int u = 0; u < n; u++)
14            for (int v : adj[u])
15                gradoEntrada[v]++;
16    } // O(V + E)
17
18    // READ: devuelve el orden topologico, o un vector vacio si
19    // el grafo tiene un ciclo (no es DAG).
20    vector<int> ordenar() {
21        vector<int> grado = gradoEntrada; // copia para no mutar
22        // el original
23        queue<int> q;
24
25        for (int u = 0; u < n; u++)
26            if (grado[u] == 0)
27                q.push(u);
28
29        vector<int> orden;
30        while (!q.empty()) {
31            int u = q.front(); q.pop();
32            orden.push_back(u);
33
34            for (int v : adj[u]) {
35                grado[v]--;
36                if (grado[v] == 0)
37                    q.push(v);
38            }
39        }
40
41        if ((int)orden.size() != n) return {}; // habia un ciclo
42        return orden;
43    } // O(V + E)
44 };

```

Ejemplo de funcionamiento:

```

1 // DAG: 0 -> 1 -> 3
2 //      0 -> 2 -> 3
3

```

```

4 vector<vector<int>> adj(4);
5 adj[0] = {1, 2};
6 adj[1] = {3};
7 adj[2] = {3};
8 adj[3] = {};
9
10 KahnTopoSort kts(adj, 4);
11 auto orden = kts.ordenar();
12
13 for (int u : orden) cout << u << " ";
14 // 0 1 2 3 (o 0 2 1 3, ambos validos)

```

8.2 DFS + Orden Topológico

Idea: la versión recursiva del orden topológico usa una propiedad simple del **post-orden** de un DFS: cuando terminas de procesar completamente un nodo u (ya visitaste todo lo alcanzable desde él), **todos** sus sucesores ya están en la lista de resultado. Empujando cada nodo a una pila justo al **salir** de su llamada recursiva, y luego leyendo la pila de arriba hacia abajo, obtienes el orden topológico completo.

¿Por qué usarlo? Es más directo de escribir que Kahn cuando ya tienes DFS a mano y **sabes** de antemano que el grafo es un DAG (si no estás seguro, hay que combinarlo con Detección de Ciclos en grafos dirigidos, o simplemente usar Kahn, que detecta el ciclo como parte del mismo proceso). Es también la base de Tarjan para condensar SCCs en un DAG y luego recorrerlas en orden topológico.

Casos de uso:

- Mismo uso que Kahn (ordenar tareas con dependencias, DP sobre DAG), cuando el grafo ya se sabe acíclico de antemano.
- Recorrer las SCCs de un grafo condensado (via Tarjan) en orden topológico, por ejemplo para propagar valores de SCCs "fuente" hacia SCCs "destino".
- Cuando además del orden necesitas otra información que ya calculas naturalmente en post-orden de un DFS (tamaños de subárbol, etc.) y quieres aprovechar el mismo recorrido.

Complejidad: $O(V + E)$, un DFS normal con una operación constante extra (push a la pila) al salir de cada nodo.

```

1 struct DFSTopoSort {
2     vector<vector<int>> adj;
3     vector<bool> visitado;
4     stack<int> pila;
5     int n;
6
7     // CREATE: guarda el grafo (debe ser DAG).
8     DFSTopoSort(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        visitado.assign(n, false);
12    } // O(1)
13
14    // READ: devuelve el orden topologico completo del grafo.
15    vector<int> ordenar() {
16        for (int u = 0; u < n; u++)
17            if (!visitado[u])
18                dfs(u);
19
20        vector<int> orden;
21        while (!pila.empty()) {
22            orden.push_back(pila.top());
23            pila.pop();
24        }
25        return orden;
26    } // O(V + E)
27 };

```

```

28 void dfs(int u) {
29     visitado[u] = true;
30
31     for (int v : adj[u])
32         if (!visitado[v])
33             dfs(v);
34
35     pila.push(u); // post-orden: u entra a la pila solo
36                 // despues de procesar todos sus sucesores
37 }
38 };

```

Ejemplo de funcionamiento:

```

1 // DAG: 0 -> 1 -> 3
2 //       0 -> 2 -> 3
3
4 vector<vector<int>> adj(4);
5 adj[0] = {1, 2};
6 adj[1] = {3};
7 adj[2] = {3};
8 adj[3] = {};
9
10 DFSTopoSort dts(adj, 4);
11 auto orden = dts.ordenar();
12
13 for (int u : orden) cout << u << " ";
14 // 0 1 2 3 (o 0 2 1 3, ambos válidos)

```

Regla práctica: Kahn y DFS + Orden Topológico dan el mismo tipo de resultado (un orden válido, no necesariamente único) con el mismo costo $O(V + E)$ — la diferencia es de implementación, no de poder: Kahn es iterativo y detecta ciclos como efecto secundario del proceso (si no logra vaciar la cola, hay ciclo); DFS es recursivo y necesita la comprobación de ciclo por separado si el grafo no se sabe acíclico de antemano. En la práctica, usa el que ya tengas más a mano según si el problema ya trae DFS con `tin/low` (Tarjan) o no.

8.3 Detección de Ciclos con Topological Sort

Idea: un DAG (grafo dirigido acíclico) **siempre** admite al menos un orden topológico completo que incluye a todos sus n nodos. Si un grafo dirigido **tiene** un ciclo, ningún nodo del ciclo puede procesarse primero (cada uno depende de que otro del mismo ciclo se procese antes), así que Kahn's Algorithm se **atasca**: la cola se vacía sin haber procesado esos nodos. Eso convierte la detección de ciclos en algo casi gratis — basta con correr Kahn y comparar el tamaño del orden resultante contra n .

¿Por qué usarlo? Es la alternativa **iterativa** al DFS con estados/colores (blanco/gris/negro) de Detección de Ciclos: en vez de rastrear explícitamente qué nodos están "en la pila de recursión actual", Kahn detecta el ciclo **implícitamente** — cualquier nodo con grado de entrada > 0 al final del proceso forma parte de (o depende de) un ciclo. Es la opción natural cuando de todas formas ibas a calcular un orden topológico y solo necesitas saber, de paso, si el grafo es válido.

Casos de uso:

- Verificar si un conjunto de dependencias es realizable (build systems, prerrequisitos de materias, tareas con "depende de") antes de intentar ordenarlas.
- Identificar **cuáles** nodos específicos quedan atrapados en un ciclo (los que nunca llegan a grado de entrada 0), útil para reportarle al usuario exactamente qué dependencias circulares hay que romper.

- Como chequeo previo obligatorio antes de aplicar DP sobre DAG o Conteo de Caminos, que asumen ausencia de ciclos.

Complejidad: $O(V + E)$ — es literalmente Kahn's Algorithm sin ningún costo extra; el chequeo de ciclo es solo comparar `orden.size()` contra n al final.

```

1 struct DeteccionCiclosTopo {
2     vector<vector<int>> adj;
3     vector<int> gradoEntrada;
4     int n;
5
6     // CREATE: guarda el grafo dirigido y calcula el grado de
7     // entrada de cada nodo.
8     DeteccionCiclosTopo(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        gradoEntrada.assign(n, 0);
12
13        for (int u = 0; u < n; u++)
14            for (int v : adj[u])
15                gradoEntrada[v]++;
16    } // O(V + E)
17
18    // READ: true si el grafo tiene al menos un ciclo (no es DAG).
19
20    bool tieneCiclo() {
21        vector<int> grado = gradoEntrada;
22        queue<int> q;
23
24        for (int u = 0; u < n; u++)
25            if (grado[u] == 0)
26                q.push(u);
27
28        int procesados = 0;
29        while (!q.empty()) {
30            int u = q.front(); q.pop();
31            procesados++;
32
33            for (int v : adj[u]) {
34                grado[v]--;
35                if (grado[v] == 0)
36                    q.push(v);
37            }
38        }
39
40        return procesados != n; // si faltó alguno, está en un
41                               // ciclo // O(V + E)
42
43    // READ: devuelve los nodos que quedan atrapados en (o
44    // de) algun ciclo. Vacío si el grafo es un DAG.
45    vector<int> nodosEnCiclo() {
46        vector<int> grado = gradoEntrada;
47        vector<bool> procesado(n, false);
48        queue<int> q;
49
50        for (int u = 0; u < n; u++)
51            if (grado[u] == 0)
52                q.push(u);
53
54        while (!q.empty()) {
55            int u = q.front(); q.pop();
56            procesado[u] = true;
57
58            for (int v : adj[u]) {
59                grado[v]--;
60                if (grado[v] == 0)
61                    q.push(v);
62            }
63        }
64
65        vector<int> enCiclo;
66        for (int u = 0; u < n; u++)
67            if (!procesado[u])
68                enCiclo.push_back(u);
69
70        return enCiclo; // O(V + E)
71    };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 1 (ciclo entre 1 y 2)
2 //                 3 -> 0 (3 depende del ciclo)
3 // tambien
4 vector<vector<int>> adj(4);
5 adj[0] = {1};

```

```

6  adj[1] = {2};
7  adj[2] = {1};
8  adj[3] = {0};
9
10 DeteccionCiclosTopo dct(adj, 4);
11
12 cout << dct.tieneCiclo();
13 // true
14
15 for (int u : dct.nodosEnCiclo()) cout << u << " ";
16 // 0 1 2
17 // (3 SI se procesa: nunca depende directamente de que 1 o 2
18 // "terminen", solo apunta hacia 0, que queda atrapado)

```

Regla práctica: si ya vas a calcular un orden topológico con Kahn, la detección de ciclos viene **incluida sin costo adicional** — nunca hace falta un DFS separado con blanco/gris/negro solo para chequear ciclos si de todas formas vas a correr Kahn después. Resérvate el DFS con colores (de Detección de Ciclos) para cuando **no** necesitas el orden topológico en sí, solo la respuesta sí/no, y prefieres evitar la cola/vector extra de grados de entrada.

9 Single Source Shortest Path (SSSP)

9.1 BFS para Caminos Más Cortos

Idea: en cualquier grafo (no solo grillas) donde **todas** las aristas tienen el mismo costo (o no tienen costo explícito), BFS desde un nodo origen calcula la distancia mínima a **todos** los demás nodos en una sola pasada. Es exactamente el mismo BFS de Laberinto con BFS y Serpientes y Escaleras, pero formulado de forma general sobre listas de adyacencia en vez de sobre una grilla o un tablero: la garantía de optimalidad viene de que BFS visita los nodos **en orden creciente de distancia** — la primera vez que llegas a un nodo, ya no hay forma de llegar más rápido después.

¿Por qué usarlo? Es el caso base de SSSP: si el grafo no tiene pesos (o todos los pesos son iguales), **no uses Dijkstra** — es estrictamente más caro ($O(E \log V)$ con heap vs. $O(V + E)$ de BFS) para resolver exactamente el mismo problema. Dijkstra solo se vuelve necesario cuando los pesos son distintos entre sí; con pesos solo 0 y 1, hay una versión intermedia (0-1 BFS) que sigue siendo $O(V + E)$.

Casos de uso:

- Distancia mínima (en número de aristas) desde un nodo origen a todos los demás, en cualquier grafo no pesado dirigido o no dirigido.
- Verificar si el grafo es **bipartito**: BFS por capas, coloreando cada capa con un color distinto al de la anterior; si una arista conecta dos nodos de la misma capa, no es bipartito.
- Reconstruir el camino más corto explícito, guardando **padre[v]** cada vez que se descubre un nodo por primera vez (igual que en Laberinto con BFS).

Complejidad: $O(V + E)$ — cada nodo se encola una sola vez, cada arista se examina una sola vez.

```

1 struct BFSCaminoMasCorto {
2     vector<vector<int>>> adj;
3     vector<int> dist, padre;
4     int n;

```

```

5
6 // CREATE: guarda el grafo (puede ser dirigido o no).
7 BFSCaminoMasCorto(vector<vector<int>>>& adj_, int n_) {
8     adj = adj_;
9     n = n_;
10 } // O(1)
11
12 // READ: corre BFS desde origen y llena dist/padre para todos
13 // los nodos alcanzables. dist queda en -1 donde no se llega.
14 void bfs(int origen) {
15     dist.assign(n, -1);
16     padre.assign(n, -1);
17
18     queue<int> q;
19     q.push(origen);
20     dist[origen] = 0;
21
22     while (!q.empty()) {
23         int u = q.front(); q.pop();
24
25         for (int v : adj[u]) {
26             if (dist[v] != -1) continue;
27
28             dist[v] = dist[u] + 1;
29             padre[v] = u;
30             q.push(v);
31         }
32     }
33 } // O(V + E)
34
35 // READ: reconstruye el camino desde el origen del ultimo bfs
36 // hasta destino. Vacío si no hay camino.
37 vector<int> reconstruirCamino(int destino) {
38     if (dist[destino] == -1) return {};
39
40     vector<int> camino;
41     for (int u = destino; u != -1; u = padre[u])
42         camino.push_back(u);
43
44     reverse(camino.begin(), camino.end());
45     return camino;
46 } // O(longitud del camino)
47 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 3
2 //                0 -> 2 -> 3
3 //                1 -> 4
4
5 vector<vector<int>>> adj(5);
6 adj[0] = {1, 2};
7 adj[1] = {3, 4};
8 adj[2] = {3};
9 adj[3] = {};
10 adj[4] = {};
11
12 BFSCaminoMasCorto bcc(adj, 5);
13 bcc.bfs(0);
14
15 cout << bcc.dist[3];
16 // 2
17
18 cout << bcc.dist[4];
19 // 2
20
21 auto camino = bcc.reconstruirCamino(4);
22 // [0, 1, 4]

```

9.2 0-1 BFS

Idea: cuando las aristas de un grafo tienen **solo dos pesos posibles** (0 y 1), BFS normal ya no funciona directamente — una arista de peso 0 no debería "avanzar una capa" — pero tampoco hace falta Dijkstra completo. La solución es usar un **deque** en vez de una **queue**: las aristas de peso 1 se insertan al **final** (**push_back**, como BFS normal), y las de peso 0 se insertan al **principio** (**push_front**), garantizando que el deque se mantenga ordenado por distancia en todo momento, igual que una cola de prioridad pero sin el costo de un heap.

¿Por qué usarlo? Es el punto intermedio exacto entre BFS (todas las aristas cuestan 1) y Dijkstra (pesos

arbitrarios): si reconoces que un problema solo tiene dos costos posibles —muy común en variantes de "esta acción es gratis, esta otra cuesta 1" (ej. romper una pared cuesta 1, moverse a una celda libre cuesta 0)— 0-1 BFS resuelve en $O(V + E)$ en vez de $O(E \log V)$ de Dijkstra, sin la complejidad de mantener un heap.

Casos de uso:

- Laberintos donde romper una pared cuesta 1 movimiento extra pero moverse por una celda libre es gratis (costo 0).
- Grafos donde una arista representa "usar el mismo tipo de transporte" (costo 0, ya estás en esa línea) vs. "cambiar de tipo de transporte" (costo 1).
- Cualquier variante de camino más corto donde el costo de cada arista es binario (0 o 1), sin importar la interpretación concreta del dominio.

Complejidad: $O(V + E)$ — cada nodo se procesa (se saca definitivamente del deque) una sola vez, igual que BFS normal; la diferencia con BFS es solo **dónde** se inserta cada nodo nuevo.

```

1 struct ZeroOneBFS {
2     vector<vector<pair<int,int>>> adj;    // adj[u] = {(v, peso)},
3     // peso es 0 o 1
4     vector<int> dist;
5     int n;
6
7     // CREATE: guarda el grafo con pesos 0/1.
8     ZeroOneBFS(vector<vector<pair<int,int>>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11    }                                     // O(1)
12
13    // READ: corre 0-1 BFS desde origen y llena dist para todos
14    // los nodos alcanzables. dist queda en INT_MAX donde no se llega.
15
16    void bfs01(int origen) {
17        dist.assign(n, INT_MAX);
18        deque<int> dq;
19
20        dist[origen] = 0;
21        dq.push_back(origen);
22
23        while (!dq.empty()) {
24            int u = dq.front(); dq.pop_front();
25
26            for (auto& [v, peso] : adj[u]) {
27                if (dist[u] + peso >= dist[v]) continue;
28
29                dist[v] = dist[u] + peso;
30
31                if (peso == 0)
32                    dq.push_front(v); // gratis: se procesa ya,
33                    // en la misma "capa"
34                else
35                    dq.push_back(v); // cuesta 1: se procesa
36                    // despues, como BFS normal
37            }
38        }                                     // O(V + E)
39    };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 --(1)--> 1 --(0)--> 2 --(1)--> 3
2 //         0 --(1)--> 3   (ruta directa mas cara)
3
4 vector<vector<pair<int,int>>> adj(4);
5 adj[0] = {{1, 1}, {3, 1}};
6 adj[1] = {{2, 0}};
7 adj[2] = {{3, 1}};
8 adj[3] = {};
9
10 ZeroOneBFS zbfz(adj, 4);
11 zbfz.bfs01(0);
12
13 cout << zbfz.dist[3];
14 // 2

```

```

15 // via 0 -(1)-> 1 -(0)-> 2 -(1)-> 3, costo total 1+0+1=2,
16 // mejor que la arista directa 0->3 de costo 1... espera, esa
17 // (ejemplo ilustrativo: 0-1 BFS igual encuentra el minimo real
18 // entre ambas opciones, sea cual sea)

```

Regla práctica: la pregunta que decide entre estas tres opciones de SSSP es "¿cuántos valores de peso distintos hay?": uno solo (todas las aristas iguales) → BFS normal; dos (0 y 1) → 0-1 BFS con deque; tres o más, o pesos arbitrarios → ya no alcanza ninguna de las dos, hace falta Dijkstra (con heap) de la siguiente sección.

9.3 Dijkstra

Idea: cuando las aristas tienen pesos **no negativos** arbitrarios (no solo 0 y 1), la garantía de "BFS visita en orden creciente de distancia" ya no se sostiene con una simple cola —hace falta procesar siempre el nodo con la **menor distancia tentativa** conocida hasta el momento. Dijkstra hace exactamente eso con una **cola de prioridad** (min-heap): saca el nodo más cercano, lo marca como definitivo, y **relaja** sus aristas (si $\text{dist}[u] + \text{peso} < \text{dist}[v]$, actualiza y vuelve a meter v al heap). Es la generalización directa de BFS y 0-1 BFS al caso de pesos arbitrarios no negativos.

¿Por qué usarlo? Es el algoritmo por defecto de "camino más corto desde un origen" en cuanto los pesos dejan de ser uniformes o binarios —y sigue siendo correcto y eficiente mientras **no haya pesos negativos** (con negativos, un nodo ya marcado "definitivo" podría en realidad mejorar después, lo cual rompe la garantía del heap; para eso está Bellman-Ford).

Casos de uso:

- Camino más corto en un grafo con costos distintos por arista (distancias reales, costos de tiempo/dinero variables).
- Cuando además de la distancia se necesita el camino explícito: se reconstruye igual que en BFS, guardando **padre**[v] cada vez que se relaja una arista con éxito.
- Como subrutina dentro de problemas más grandes que piden "la ruta más barata" sobre un grafo ya construido (ej. redes de transporte, problemas de flujo con costo).

Complejidad: $O(E \log V)$ con **priority_queue** binaria (cada arista puede provocar, como mucho, una inserción al heap), o $O(V^2)$ con un arreglo simple sin heap —preferible solo en grafos densos ($E \approx V^2$) donde el heap no aporta.

```

1 struct Dijkstra {
2     vector<vector<pair<int,int>>> adj;    // adj[u] = {(v, peso)}
3     vector<long long> dist;
4     vector<int> padre;
5     int n;
6
7     // CREATE: guarda el grafo (pesos deben ser >= 0).
8     Dijkstra(vector<vector<pair<int,int>>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11    }                                     // O(1)
12
13    // READ: corre Dijkstra desde origen y llena dist/padre para
14    // todos los nodos alcanzables. dist queda en LLONG_MAX donde
15    // no se llega.
16    void dijkstra(int origen) {
17        dist.assign(n, LLONG_MAX);

```

```

18 padre.assign(n, -1);
19
20 // min-heap por distancia: {dist, nodo}
21 priority_queue<pair<long long,int>,
22 vector<pair<long long,int>>,
23 greater<>> pq;
24
25 dist[origen] = 0;
26 pq.push({0, origen});
27
28 while (!pq.empty()) {
29     auto [d, u] = pq.top(); pq.pop();
30
31     if (d > dist[u]) continue; // entrada obsoleta
32     // (ya se mejoro despues)
33
34     for (auto& [v, peso] : adj[u]) {
35         if (dist[u] + peso >= dist[v]) continue;
36
37         dist[v] = dist[u] + peso;
38         padre[v] = u;
39         pq.push({dist[v], v});
40     }
41 } // O(E log V)
42
43 // READ: reconstruye el camino desde el origen del ultimo
44 // dijkstra() hasta destino. Vacio si no hay camino.
45 vector<int> reconstruirCamino(int destino) {
46     if (dist[destino] == LLONG_MAX) return {};
47
48     vector<int> camino;
49     for (int u = destino; u != -1; u = padre[u])
50         camino.push_back(u);
51
52     reverse(camino.begin(), camino.end());
53     return camino;
54 } // O(longitud del camino)
55 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 --(4)--> 1
2 //         0 --(1)--> 2 --(2)--> 1
3 //         1 --(1)--> 3
4
5 vector<vector<pair<int,int>>> adj(4);
6 adj[0] = {{1, 4}, {2, 1}};
7 adj[1] = {{3, 1}};
8 adj[2] = {{1, 2}};
9 adj[3] = {};
10
11 Dijkstra dj(adj, 4);
12 dj.dijkstra(0);
13
14 cout << dj.dist[1];
15 // 3 (via 0 --(1)--> 2 --(2)--> 1, mas barato que 0-->1 directo,
16 // costo 4)
17
18 cout << dj.dist[3];
19 // 4
20
21 auto camino = dj.reconstruirCamino(3);
22 // [0, 2, 1, 3]

```

Nota: el chequeo `if (d > dist[u]) continue;` es esencial y fácil de olvidar — como no se puede *decrease-key* directamente en un `priority_queue` de C++, cada mejora de `dist[v]` mete una **nueva** entrada al heap en vez de actualizar la vieja; sin ese chequeo, se reprocesan entradas obsoletas (sigue dando la respuesta correcta, pero más lento).

9.4 Bellman-Ford

Idea: cuando el grafo puede tener **pesos negativos**, Dijkstra deja de ser correcto (marcar un nodo como "definitivo" demasiado pronto puede ignorar una ruta más barata que aparece después vía una arista negativa). Bellman-Ford resuelve esto sin heap, de fuerza bruta pero correcta: **relaja todas las aristas, $V - 1$ veces**. Después de k rondas, `dist[]` garantiza ser correcta para todo camino más corto de **como mucho k aristas**;

como el camino más corto simple más largo posible tiene $V - 1$ aristas, $V - 1$ rondas bastan para converger.

¿Por qué usarlo? Es la única opción "estándar" de SSSP que tolera pesos negativos, y además detecta **ciclos negativos**: si tras las $V - 1$ rondas todavía se puede relajar alguna arista, ese alivio ocurre dentro de (o alcanzable desde/hacia) un ciclo de peso negativo — en ese caso la distancia mínima ni siquiera está bien definida (se puede hacer arbitrariamente pequeña dando vueltas al ciclo).

Casos de uso:

- Camino más corto en grafos con aristas de peso negativo (pero sin ciclos negativos alcanzables desde el origen).
- Detectar la **existencia** de un ciclo negativo en el grafo (independientemente de querer o no el camino más corto).
- Identificar **qué** nodos son alcanzables desde un ciclo negativo (su distancia real es $-\infty$), corriendo una ronda V extra y propagando desde las aristas que aún relajan.

Complejidad: $O(V \cdot E)$ — $V - 1$ rondas, cada una revisando todas las E aristas; una ronda extra opcional para detectar el ciclo negativo.

```

1 struct BellmanFord {
2     struct Arista { int u, v, peso; };
3     vector<Arista> aristas;
4     vector<long long> dist;
5     int n;
6
7     // CREATE: guarda el grafo como lista plana de aristas (mas
8     // simple que listas de adyacencia para relajar todas de una).
9
10    BellmanFord(vector<Arista>& aristas_, int n_) {
11        aristas = aristas_;
12        n = n_;
13    } // O(1)
14
15    // READ: corre Bellman-Ford desde origen. Devuelve false si
16    // hay un ciclo negativo alcanzable desde origen (dist no es
17    // confiable en ese caso); true si termino normalmente.
18    bool bellmanFord(int origen) {
19        dist.assign(n, LLONG_MAX);
20        dist[origen] = 0;
21
22        for (int ronda = 0; ronda < n - 1; ronda++) {
23            bool huboMejora = false;
24
25            for (auto& [u, v, peso] : aristas) {
26                if (dist[u] == LLONG_MAX) continue;
27                if (dist[u] + peso >= dist[v]) continue;
28
29                dist[v] = dist[u] + peso;
30                huboMejora = true;
31            }
32
33            if (!huboMejora) break; // convergio antes de V-1
34            // rondas, ya no hace falta
35            // seguir
36
37            // ronda V: si todavia se puede relajar algo, hay ciclo
38            // negativo alcanzable
39            for (auto& [u, v, peso] : aristas) {
40                if (dist[u] == LLONG_MAX) continue;
41                if (dist[u] + peso < dist[v]) return false;
42            }
43
44            return true;
45        } // O(V * E)
46    };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 --(4)--> 1

```

```

2 //      0 -(-5)-> 2
3 //      1 -(-3)-> 2
4 //      2 -(2)-> 3
5
6 vector<BellmanFord::Arista> aristas = {
7     {0, 1, 4}, {0, 2, 5}, {1, 2, -3}, {2, 3, 2}
8 };
9
10 BellmanFord bf(aristas, 4);
11 bool ok = bf.bellmanFord(0);
12
13 cout << ok;
14 // true (sin ciclo negativo)
15
16 cout << bf.dist[2];
17 // 1 (via 0 -(-4)-> 1 -(-3)-> 2, mejor que 0->2 directo, costo
18 // 5)
19
20 cout << bf.dist[3];
21 // 3

```

Regla práctica: la pregunta que decide entre Dijkstra y Bellman-Ford es "¿puede haber pesos negativos?" — si no, Dijkstra siempre gana ($O(E \log V)$ vs. $O(V \cdot E)$, casi siempre mucho más rápido). Solo cae a Bellman-Ford cuando el problema explícitamente permite pesos negativos, o cuando lo que realmente se pide es detectar un ciclo negativo (algo que Dijkstra ni siquiera puede notar).

9.5 Shortest Path en Grafos Binarios

Idea: variante clásica de camino más corto sobre una **matriz binaria** $n \times n$: las celdas con 0 son libres, las celdas con 1 son obstáculo, y el movimiento permitido es a las **8 direcciones** (incluyendo diagonales), no solo las 4 ortogonales de Laberinto con BFS. Sigue siendo BFS —costo uniforme por paso—, el único cambio real es el arreglo de direcciones (dr/dc de 8 en vez de 4) y la condición de bloqueo ($grid[r][c] == 1$ en vez de '#').

¿Por qué usarlo? El error típico es intentar resolverlo con la plantilla de 4 direcciones (copiada de Laberinto con BFS) sin darse cuenta de que el enunciado permite diagonales — eso da una respuesta **mayor** a la real, porque una diagonal puede ahorrarse dos pasos ortogonales. Vale la pena tenerlo separado justamente para recordar: **siempre confirma cuántas direcciones permite el problema** antes de fijar el arreglo dr/dc .

Casos de uso:

- El problema clásico "Shortest Path in Binary Matrix": de $(0,0)$ a $(n-1, n-1)$ moviéndose en 8 direcciones, evitando celdas con 1.
- Cualquier grilla donde el movimiento diagonal esté explícitamente permitido (juegos tipo tablero, mapas de ajedrez-rey).
- Variantes donde además se cuenta como "path length" el número de **celdas visitadas** (no de aristas) — solo cambia si **dist** arranca en 0 o en 1 y si se suma 1 al final.

Complejidad: $O(n^2)$, igual que cualquier BFS sobre una grilla — cada celda se encola una sola vez, y desde cada una se prueban 8 vecinos constantes.

```

1 struct ShortestPathBinario {
2     vector<vector<int>> grid; // 0 = libre, 1 = obstaculo
3     vector<vector<int>> dist;
4     int n;
5
6     int dr[8] = {-1,-1,-1, 0, 0, 1, 1, 1};

```

```

7     int dc[8] = {-1, 0, 1,-1, 1,-1, 0, 1};
8
9     // CREATE: guarda la matriz binaria n x n.
10    ShortestPathBinario(vector<vector<int>>& grid_) {
11        grid = grid_;
12        n = grid.size();
13    } // O(1)
14
15    bool dentro(int r, int c) {
16        return r >= 0 && r < n && c >= 0 && c < n;
17    }
18
19    // READ: numero minimo de celdas para ir de (0,0) a (n-1,n-1)
20    // (contando ambas puntas). -1 si no hay camino, o si alguna
21    // de las dos puntas ya esta bloqueada.
22    int shortestPath() {
23        if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
24
25        dist.assign(n, vector<int>(n, -1));
26        queue<pair<int,int>> q;
27
28        q.push({0, 0});
29        dist[0][0] = 1; // cuenta celdas, no aristas
30
31        while (!q.empty()) {
32            auto [r, c] = q.front(); q.pop();
33
34            if (r == n - 1 && c == n - 1) return dist[r][c];
35
36            for (int d = 0; d < 8; d++) {
37                int nr = r + dr[d], nc = c + dc[d];
38
39                if (!dentro(nr, nc)) continue;
40                if (grid[nr][nc] == 1) continue;
41                if (dist[nr][nc] != -1) continue;
42
43                dist[nr][nc] = dist[r][c] + 1;
44                q.push({nr, nc});
45            }
46        }
47
48        return -1;
49    } // O(n^2)
50 }

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // 0 1
3 // 1 0
4
5 vector<vector<int>> grid1 = {{0,1},{1,0}};
6 ShortestPathBinario sp1(grid1);
7 cout << sp1.shortestPath();
8 // 2 (diagonal directa (0,0) -> (1,1))
9
10 // grid:
11 // 0 0 0
12 // 1 1 0
13 // 1 1 0
14
15 vector<vector<int>> grid2 = {{0,0,0},{1,1,0},{1,1,0}};
16 ShortestPathBinario sp2(grid2);
17 cout << sp2.shortestPath();
18 // 4

```

9.6 Shortest Path desde Múltiples Orígenes

Idea: cuando el problema pregunta, para **cada** celda/nodo, "¿cuál es tu distancia al más cercano de estos k puntos de interés?" (en vez de a un único origen), la solución NO es correr BFS k veces (una por origen) y quedarte con el mínimo —eso cuesta $O(k \cdot (V + E))$ —. En vez de eso, encolas **todas** las fuentes a la vez con distancia 0 antes de empezar el BFS. Como BFS sigue expandiéndose por capas sin importar cuántas semillas tenga, la primera vez que una celda se visita es, garantizado, desde su fuente más cercana.

¿Por qué usarlo? Es el mismo BFS de siempre con **un solo cambio:** en vez de un push inicial, son varios. Esa simplicidad es justo la trampa — es fácil no darse cuenta de que el problema es "multi-fuente" y en vez de

eso escribir un BFS por fuente, que sigue siendo correcto pero mucho más lento e innecesario.

Casos de uso:

- "Rotting Oranges": tiempo mínimo para que **todas** las celdas se "pudran", empezando con varias celdas ya podridas a la vez.
- "Walls and Gates": distancia de cada celda vacía a la puerta más cercana, con varias puertas en la grilla.
- Distancia de cada nodo de un grafo al miembro más cercano de un conjunto dado (ej. "estación de bomberos más cercana" cuando hay varias estaciones).
- Cualquier variante de Laberinto con BFS o Serpientes y Escaleras donde el origen no es un solo punto sino un conjunto.

Complejidad: $O(V + E)$ (o $O(R \cdot C)$ en grillas) — exactamente igual que BFS de una sola fuente, ya que el número de fuentes iniciales no cambia cuántas veces se visita cada nodo (sigue siendo una vez).

```

1 struct BFSMultiOrigen {
2     vector<vector<char>> grid; // 'X' = fuente, '#' = pared,
3     // '.' = celda libre
4     vector<vector<int>> dist;
5     int R, C;
6
7     int dr[4] = {-1, 1, 0, 0};
8     int dc[4] = {0, 0, -1, 1};
9
10    // CREATE: guarda la grilla.
11    BFSMultiOrigen(vector<vector<char>>& grid_) {
12        grid = grid_;
13        R = grid.size();
14        C = grid[0].size();
15    } // O(1)
16
17    // READ: BFS simultaneo desde TODAS las celdas 'X'. dist[r][c]
18    // queda con la distancia a la fuente 'X' mas cercana, -1 si
19    // es inalcanzable (o si (r,c) es pared).
20    void bfsMultiOrigen() {
21        dist.assign(R, vector<int>(C, -1));
22        queue<pair<int,int>> q;
23
24        // paso clave: encolar TODAS las fuentes de una vez,
25        // todas con distancia 0
26        for (int r = 0; r < R; r++)
27            for (int c = 0; c < C; c++)
28                if (grid[r][c] == 'X') {
29                    dist[r][c] = 0;
30                    q.push({r, c});
31                }
32
33        while (!q.empty()) {
34            auto [r, c] = q.front(); q.pop();
35
36            for (int d = 0; d < 4; d++) {
37                int nr = r + dr[d], nc = c + dc[d];
38
39                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
40                if (grid[nr][nc] == '#') continue;
41                if (dist[nr][nc] != -1) continue;
42
43                dist[nr][nc] = dist[r][c] + 1;
44                q.push({nr, nc});
45            }
46        }
47    } // O(R * C)
48 };

```

Ejemplo de funcionamiento:

```

1 // grid:
2 // X . . .
3 // . # . .
4 // . . . X

```

```

5 vector<vector<char>> grid = {
6     {'X', '.', '.', '.', '.'},
7     {'.', '#', '.', '.', '.'},
8     {'.', '.', '.', 'X', '.'}
9 };
10
11 BFSMultiOrigen bmo(grid);
12 bmo.bfsMultiOrigen();
13
14 cout << bmo.dist[1][2];
15 // 2 (mas cerca de la X de abajo-derecha: 2 pasos)
16
17 cout << bmo.dist[0][1];
18 // 1 (mas cerca de la X de arriba-izquierda)
19

```

Regla práctica: si el enunciado dice "desde el más cercano de estos puntos" (en plural), es multi-fuente — encola todo al principio, un solo BFS. Si dice "desde este punto específico", es BFS de una sola fuente (Laberinto con BFS de siempre). Nunca hace falta correr BFS una vez por fuente; eso multiplica el costo por k sin ganar nada, ya que un solo BFS multi-fuente da exactamente la misma respuesta.

9.7 Shortest Path en Grafos Multietapa

Idea: un grafo multietapa (*multistage graph*) tiene sus nodos divididos en "etapas" $0, 1, \dots, k$, y las aristas **solo** van de una etapa a la siguiente (nunca hacia atrás ni saltando etapas). Esa estructura ya es un DAG por construcción, así que el camino más corto de la etapa 0 a la etapa k se resuelve con **DP sobre DAG**, no con Dijkstra: procesando las etapas de **atrás hacia adelante**, $dp[u] =$ costo mínimo desde u hasta el destino final $= \min_{v \in \text{sig. etapa}} (\text{peso}(u, v) + dp[v])$.

¿Por qué usarlo? Dijkstra funcionaría igual de correcto aquí, pero es **trabajo de más**: no necesitas un heap ni relajaciones repetidas cuando ya sabes de antemano el orden exacto en que hay que procesar los nodos (etapa por etapa, de atrás para adelante) — es el mismo principio de "aprovecha la estructura de DAG" que Conteo de Caminos y DFS + Orden Topológico, aplicado a suma de pesos en vez de conteo.

Casos de uso:

- Redes por capas explícitas (ej. "elige una opción por cada nivel/fase, minimizando el costo total").
- Cualquier problema que se pueda reformular como "etapas" aunque no se llamen así explícitamente — por ejemplo, DP sobre posiciones en una línea donde cada posición solo conecta con posiciones estrictamente mayores.
- Como caso particular más simple antes de generalizar a Dijkstra, cuando en el enunciado el grafo **no** resulta ser estrictamente por etapas.

Complejidad: $O(V + E)$ — un solo recorrido en orden topológico inverso (o simplemente por etapa decreciente si las etapas ya vienen dadas), cada arista se relaja una sola vez.

```

1 struct GrafoMultietapa {
2     vector<vector<pair<int,int>>> adj; // adj[u] = {(v, peso)},
3     // solo hacia la
4     // siguiente etapa
5     vector<long long> dp; // dp[u] = costo minimo de u al
6     // destino
7     int n, destino;
8 };

```

```

8 // CREATE: guarda el grafo (ya organizado por etapas) y el
9 // destino final (ultima etapa).
10 GrafoMultietapa(vector<vector<pair<int,int>>>& adj_, int n_,
11 int destino_) {
12     adj = adj_;
13     n = n_;
14     destino = destino_;
15     dp.assign(n, -1); // -1 = no calculado
16 } // O(1)
17
18 // READ: costo minimo de u a destino, procesando en orden
19 // topologico inverso (los nodos deben pasarse ya ordenados
20 // de la ultima etapa a la primera, o usar DFS + Orden
21 // Topologico
22 // para obtener ese orden si no viene dado).
23 long long costoMinimo(int u) {
24     if (u == destino) return 0;
25     if (dp[u] != -1) return dp[u];
26
27     long long mejor = LLONG_MAX;
28     for (auto& [v, peso] : adj[u]) {
29         long long candidato = costoMinimo(v);
30         if (candidato == LLONG_MAX) continue;
31         mejor = min(mejor, peso + candidato);
32     }
33
34     return dp[u] = mejor;
35 } // O(1) amortizado
36
37 // O(V+E) la primera
38 // vez
39 };

```

Ejemplo de funcionamiento:

```

1 // etapas: 0 -> {1, 2} -> {3} -> 4
2 //
3 // 0-(2)->1 0-(3)->2
4 // 1-(1)->3 2-(4)->3
5 // 3-(2)->4
6
7 vector<vector<pair<int,int>>> adj(5);
8 adj[0] = {{1, 2}, {2, 3}};
9 adj[1] = {{3, 1}};
10 adj[2] = {{3, 4}};
11 adj[3] = {{4, 2}};
12 adj[4] = {};
13
14 GrafoMultietapa gm(adj, 5, 4);
15
16 cout << gm.costoMinimo(0);
17 // 5 (via 0 -(2)-> 1 -(1)-> 3 -(2)-> 4, costo 2+1+2=5)

```

Nota: si las etapas **no** vienen explícitas en el enunciado, primero corre DFS + Orden Topológico (o Kahn's Algorithm) para conseguir un orden válido; la DP de arriba es genérica para cualquier DAG, "multietapa" es solo el caso donde ese orden ya viene dado en capas.

9.8 Shortest Path con Exactamente k Aristas

Idea: variante donde no basta con "el camino más corto" —se pide el camino más corto usando **exactamente** k aristas, ni más ni menos (incluso si eso obliga a "desperdiciar" pasos yendo y viniendo). Ya no es un problema de un solo número por nodo, sino de un número por **par** (*nodo, aristas_usadas*): $dp[i][v]$ = costo mínimo para llegar a v usando **exactamente** i aristas desde el origen. Se llena con k rondas de relajación tipo Bellman-Ford, una por cada arista adicional permitida.

¿Por qué usarlo? BFS y Dijkstra normales asumen implícitamente "cuantas menos aristas, mejor" — aquí esa suposición se rompe: forzar **exactamente** k aristas puede dar una respuesta **peor** que el camino más corto real (si el óptimo real usa menos de k aristas), o incluso no tener solución si v no es alcanzable en exactamente k pasos. Por eso hace falta la dimensión extra de "pasos usados" en vez de solo "distancia mínima".

Casos de uso:

- "Cheapest Flights Within K Stops": costo mínimo de vuelos usando como máximo (o exactamente) k escalas.
- Cualquier variante donde el número de pasos/transiciones esté **acotado** o **fijado** explícitamente, no solo minimizado.
- Con k muy grande (ej. $k \sim 10^9$) y el grafo pequeño, esta misma DP se puede acelerar con **exponenciación de matrices** en el semianillo (min, +) — fuera del alcance de esta sección, pero el mismo principio de "DP por pasos" es la base.

Complejidad: $O(k \cdot E)$ — k rondas, cada una relajando todas las aristas una vez (igual que Bellman-Ford, pero limitado a k rondas en vez de $V - 1$, y guardando el resultado de **cada** ronda en vez de solo la última).

```

1 struct ShortestPathKAristas {
2     struct Arista { int u, v, peso; };
3     vector<Arista> aristas;
4     int n;
5
6     // CREATE: guarda el grafo como lista plana de aristas.
7     ShortestPathKAristas(vector<Arista>& aristas_, int n_) {
8         aristas = aristas_;
9         n = n_;
10    } // O(1)
11
12    // READ: costo minimo de origen a destino usando EXACTAMENTE
13    // k
14    // aristas. LLONG_MAX si no es posible.
15    long long costoExactoK(int origen, int destino, int k) {
16        vector<long long> dist(n, LLONG_MAX);
17        dist[origen] = 0;
18
19        for (int paso = 0; paso < k; paso++) {
20            vector<long long> nuevaDist(n, LLONG_MAX);
21
22            for (auto& [u, v, peso] : aristas) {
23                if (dist[u] == LLONG_MAX) continue;
24                nuevaDist[v] = min(nuevaDist[v], dist[u] + peso);
25            }
26
27            dist = nuevaDist; // IMPORTANTE: se reemplaza
28            // completo, no se relaja sobre
29            // el mismo arreglo -- cada ronda
30            // usa SOLO la ronda anterior, para
31            // no colar caminos de menos de
32            // "paso+1" aristas
33        }
34
35        return dist[destino];
36    } // O(k * E)
37 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 -(1)-> 1 -(1)-> 2
2 //         0 -(4)-> 2 (ruta directa mas cara)
3
4 vector<ShortestPathKAristas::Arista> aristas = {
5     {0, 1, 1}, {1, 2, 1}, {0, 2, 4}
6 };
7
8 ShortestPathKAristas spk(aristas, 3);
9
10 cout << spk.costoExactoK(0, 2, 1);
11 // 4 (con exactamente 1 arista, solo existe la ruta directa
12 // 0->2)
13
14 cout << spk.costoExactoK(0, 2, 2);
15 // 2 (con exactamente 2 aristas: 0 -> 1 -> 2, costo 1+1=2)

```

Regla práctica: si el problema dice "como máximo k escalas/aristas", basta con tomar el **mínimo sobre todas las rondas** $0..k$ (no solo la ronda k). Si dice **exactamente** k , hay que usar el valor de la ronda k tal cual, incluso si alguna ronda anterior ya había encontrado un costo menor — la diferencia entre "como máximo" y "exactamente" es literalmente la diferencia entre tomar el mínimo acumulado o solo el último valor.

10 All Pairs Shortest Path (APSP)

10.1 Floyd-Warshall

Idea: Floyd-Warshall calcula la distancia más corta entre todos los pares de nodos a la vez, en vez de desde un solo origen. La idea central es **DP sobre "nodos intermedios permitidos"**: $\text{dist}[i][j]$ empieza siendo el peso directo de la arista $i \rightarrow j$ (o ∞ si no existe), y luego, para cada nodo k de 0 a $n-1$, se pregunta "¿mejora $\text{dist}[i][j]$ si me permito pasar por k ?" —es decir, $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$. Al terminar de iterar k sobre todos los nodos, cada $\text{dist}[i][j]$ ya considera **cualquier** nodo intermedio posible, así que es la distancia mínima real.

¿Por qué usarlo? Es la alternativa directa a correr Dijkstra (o Bellman-Ford, si hay pesos negativos) desde cada nodo: V llamadas a Dijkstra cuestan $O(V \cdot E \log V)$, que en grafos densos ($E \approx V^2$) es peor que los $O(V^3)$ de Floyd-Warshall, además de que Floyd-Warshall es mucho más simple de programar (tres for anidados, sin heap) y —a diferencia de correr Dijkstra V veces— tolera pesos **negativos** (aunque no ciclos negativos).

Casos de uso:

- Todas las distancias par a par cuando V es pequeño ($V \lesssim 400-500$, por el costo $O(V^3)$).
- Calcular el **cerramiento transitivo** de un grafo con pesos: reemplazando min por OR lógico y + por AND, la misma estructura de tres for resuelve alcanzabilidad en vez de distancia (alternativa a DFS + Cerramiento Transitivo cuando ya tienes Floyd-Warshall a mano).
- Detectar si existe un ciclo negativo en **cualquier** parte del grafo: si al terminar $\text{dist}[i][i] < 0$ para algún i , ese nodo está en (o alcanza) un ciclo negativo.
- Encontrar el "diámetro" del grafo (la mayor distancia mínima entre cualquier par de nodos alcanzables).

Complejidad: $O(V^3)$ tiempo, $O(V^2)$ espacio — los tres for anidados son fijos sin importar cuántas aristas tenga el grafo (a diferencia de Dijkstra/Bellman-Ford, que dependen de E).

```
1 struct FloydWarshall {
2     vector<vector<long long>>> dist;
3     int n;
4
5     const long long INF = LLONG_MAX / 2; // evita overflow al
        sumar
6
7     // CREATE: inicializa dist[i][j] con el peso directo de cada
8     // arista (INF si no hay arista directa, 0 en la diagonal).
9     FloydWarshall(vector<vector<pair<int,int>>>& adj, int n_) {
10         n = n_;
11         dist.assign(n, vector<long long>(n, INF));
12
13         for (int i = 0; i < n; i++)
14             dist[i][i] = 0;
15
16         for (int u = 0; u < n; u++)
17             for (auto& [v, peso] : adj[u])
18                 dist[u][v] = min(dist[u][v], (long long)peso);
19         // min por si hay aristas multiples
20         // entre el mismo par
21     } // O(V + E)
22
23     // READ: corre Floyd-Warshall sobre dist, dejando la
24     // distancia
25     // minima real entre cada par. Devuelve false si detecta un
```

```
25 // ciclo negativo en algun lugar del grafo.
26 bool ejecutar() {
27     for (int k = 0; k < n; k++)
28         for (int i = 0; i < n; i++)
29             for (int j = 0; j < n; j++)
30                 if (dist[i][k] < INF && dist[k][j] < INF)
31                     dist[i][j] = min(dist[i][j],
32                                     dist[i][k] + dist[k][j]);
33
34     for (int i = 0; i < n; i++)
35         if (dist[i][i] < 0)
36             return false; // ciclo negativo detectado
37
38     return true;
39 } // O(V^3)
40
```

Ejemplo de funcionamiento:

```
1 // grafo: 0 -(3)-> 1
2 //          1 -(1)-> 2
3 //          2 -(-2)-> 0 (ciclo, pero de peso total 3+1-2=2,
4 //                    positivo)
5 //          0 -(10)-> 2 (ruta directa mas cara)
6
7 vector<vector<pair<int,int>>> adj(3);
8 adj[0] = {{1, 3}, {2, 10}};
9 adj[1] = {{2, 1}};
10 adj[2] = {{0, -2}};
11
12 FloydWarshall fw(adj, 3);
13 bool ok = fw.ejecutar();
14
15 cout << ok;
16 // true (no hay ciclo negativo)
17
18 cout << fw.dist[0][2];
19 // 4 (via 0 -(3)-> 1 -(1)-> 2, mejor que la ruta directa de
20 //    10)
21
22 cout << fw.dist[2][1];
23 // -1 (via 2 -(-2)-> 0 -(3)-> 1, costo -2+3=1... revisar signo:
24
25 // en este grafo especifico da 1, no -1 -- el punto es que
26 // dist[i][j] SIEMPRE queda como el minimo real
27 // considerando
28 // cualquier nodo intermedio, incluso via el ciclo)
```

Nota sobre el orden de los for: el orden k primero, luego i , luego j no es arbitrario —es lo que garantiza la correctitud de la DP. k representa "qué nodos intermedios ya se me permite usar", así que debe ser el bucle más externo: al procesar k , todas las combinaciones $\text{dist}[i][j]$ ya reflejan poder pasar por cualquier nodo $< k$, y solo entonces se evalúa si pasar también por k mejora algo. Cambiar el orden de los for (aunque compile y "parezca" correcto) rompe esa garantía y puede dar distancias mayores a las reales.

Regla práctica: Floyd-Warshall gana cuando necesitas **todas** las distancias par a par y V es chico; cuando solo necesitas las distancias desde **un** origen (o pocos), correr Dijkstra (o Bellman-Ford con pesos negativos) desde cada uno de esos orígenes específicos casi siempre es más rápido que Floyd-Warshall completo, salvo que ya de por sí necesites la matriz completa para otra cosa.

11 Minimum Spanning Tree (MST)

11.1 Disjoint Set Union (DSU)

Idea: DSU (también llamado Union-Find) mantiene una partición de n elementos en conjuntos disjuntos, soportando dos operaciones: **find**(x) (¿a qué conjunto pertenece x ? — devuelve un "representante" del conjunto) y **union**(x, y) (fusiona los conjuntos de x y y). Cada elemento apunta a un **padre**; el representante de un conjunto es la raíz de ese árbol (el nodo que es padre de sí mismo). Sin optimizaciones, esos árboles pueden

degenerar en una cadena larga; dos trucos lo evitan: **unión por rango/tamaño** (al fusionar, el árbol más chico cuelga del más grande, no al revés) y **compresión de camino** (en cada `find`, aplanar el camino recorrido para que futuras consultas sean casi $O(1)$).

¿Por qué usarlo? Es la estructura correcta cuando las aristas/relaciones **van llegando una a una** y solo necesitas responder "¿misma componente?" o "¿ya conectados?" sin reconstruir el grafo completo cada vez — a diferencia de Componentes Conexas (DFS), que necesita la lista de adyacencia completa de antemano. Con las dos optimizaciones juntas, cada operación es prácticamente $O(1)$ amortizado (formalmente $O(\alpha(n))$), la función inversa de Ackermann, que para cualquier n realista es ≤ 4).

Casos de uso:

- Kruskal (siguiente sección): decidir si agregar una arista crearía un ciclo, sin reconstruir el grafo.
- Detección de ciclos incremental: procesar aristas una por una y detectar el momento exacto en que se forma el primer ciclo.
- Conectividad dinámica *offline*: responder consultas de "¿ u y v conectados?" intercaladas con la llegada de nuevas aristas (unión), sin volver a correr DFS cada vez.
- Agrupar elementos bajo una relación de equivalencia dada incrementalmente (ej. "estas cuentas son la misma persona").

Complejidad: $O(\alpha(n))$ amortizado por operación (`find` y `union`) con ambas optimizaciones — prácticamente constante en la práctica.

```

1 struct DSU {
2     vector<int> padre, rango;
3
4     // CREATE: n elementos, cada uno en su propio conjunto.
5     DSU(int n) {
6         padre.resize(n);
7         rango.assign(n, 0);
8         iota(padre.begin(), padre.end(), 0); // padre[i] = i
9     } // O(n)
10
11     // READ: representante del conjunto de x, con compresión de
12     // camino (aplana el camino recorrido).
13     int find(int x) {
14         if (padre[x] != x)
15             padre[x] = find(padre[x]); // compresión de camino
16         return padre[x];
17     } // O(alpha(n))
18     // amortizado
19
20     // WRITE: fusiona los conjuntos de x e y. Devuelve false si
21     // ya estaban en el mismo conjunto (util para detectar ciclos).
22     bool unir(int x, int y) {
23         int rx = find(x), ry = find(y);
24         if (rx == ry) return false; // ya conectados -> formaría
25         // ciclo
26
27         // unión por rango: el árbol más chico cuelga del más
28         // grande
29         if (rango[rx] < rango[ry]) swap(rx, ry);
30         padre[ry] = rx;
31         if (rango[rx] == rango[ry]) rango[rx]++;
32
33         return true;
34     } // O(alpha(n))
35     // amortizado
36
37     // READ: x e y están en el mismo conjunto?
38     bool mismoConjunto(int x, int y) {
39         return find(x) == find(y);
40     } // O(alpha(n))
41     // amortizado
42 };

```

Ejemplo de funcionamiento:

```

1 DSU dsu(6); // {0} {1} {2} {3} {4} {5}
2
3 dsu.unir(0, 1); // {0,1} {2} {3} {4} {5}
4 dsu.unir(2, 3); // {0,1} {2,3} {4} {5}
5 dsu.unir(1, 2); // {0,1,2,3} {4} {5}
6
7 cout << dsu.mismoConjunto(0, 3);
8 // true (0 y 3 quedaron en el mismo conjunto tras las uniones)
9
10 cout << dsu.mismoConjunto(0, 4);
11 // false
12
13 cout << dsu.unir(0, 3);
14 // false (0 y 3 YA estaban conectados -> esta unión formaría
15 // un ciclo si (0,3) fuera una arista de grafo)

```

Nota: el valor de retorno `bool` de `unir` no es cos-mético — es exactamente la señal que usa Kruskal para decidir si una arista pertenece al MST: si `unir(u,v)` devuelve `false`, esa arista cerraría un ciclo y se descarta.

11.2 Kruskal

Idea: Kruskal construye un MST (árbol de expansión mínima) con una estrategia **greedy**: ordena **todas** las aristas por peso ascendente, y las va agregando una por una **solo si** no forman un ciclo con las ya agregadas (es decir, si sus dos extremos están en componentes distintas). Ese chequeo de "¿misma conjunto?" es exactamente DSU — Kruskal es, en esencia, "ordenar aristas + DSU" pegados.

¿Por qué usarlo? Es el algoritmo de MST más simple de razonar y programar cuando tienes la lista de aristas completa de antemano (no una matriz de adyacencia densa): el orden importa más que la estructura del grafo, así que basta con un `sort` y un DSU. La alternativa es Prim (crece un solo árbol desde un nodo, agregando la arista más barata que lo conecta con algo nuevo) — Prim suele preferirse en grafos **densos** representados como matriz de adyacencia; Kruskal suele preferirse en grafos **dispersos** representados como lista de aristas.

Casos de uso:

- Conectar n puntos (ciudades, computadoras, etc.) con el costo total mínimo, dado el costo de cada conexión posible.
- Encontrar el **cuello de botella** mínimo de un grafo: la arista más pesada del MST es el mínimo posible "peso máximo de arista" entre cualquier árbol que conecte todos los nodos (útil en problemas tipo "minimiza la arista más cara del camino").
- Verificar si un grafo es conexo: si al final el MST tiene menos de $n - 1$ aristas, el grafo original no era conexo.
- Como filtro previo: procesar aristas en orden de peso y usar DSU para responder "¿en qué momento (con qué peso máximo) se conectan u y v ?" — variante de consultas offline.

Complejidad: $O(E \log E)$, dominado por el `sort` inicial de las aristas; el procesamiento con DSU es $O(E \cdot \alpha(V))$, prácticamente lineal.

```

1 struct Kruskal {
2     struct Arista { int u, v, peso; };
3     vector<Arista> aristas;

```

```

4  int n;
5
6  // CREATE: guarda el grafo como lista plana de aristas.
7  Kruskal(vector<Arista>& aristas_, int n_) {
8      aristas = aristas_;
9      n = n_;
10 } // O(1)
11
12 // READ: construye el MST. Devuelve {costoTotal,
13 // aristasDelMST}.
14 // Si el grafo no es conexo, aristasDelMST tiene menos de n-1
15 // aristas (MST parcial, uno por componente -- un "MST forest
16 // ").
17 pair<long long, vector<Arista>> construirMST() {
18     sort(aristas.begin(), aristas.end(),
19         [](Arista& a, Arista& b) { return a.peso < b.peso; });
20
21     DSU dsu(n);
22     long long costoTotal = 0;
23     vector<Arista> mst;
24
25     for (auto& e : aristas) {
26         if (dsu.unir(e.u, e.v)) { // solo si NO forma ciclo
27             costoTotal += e.peso;
28             mst.push_back(e);
29
30             if ((int)mst.size() == n - 1) break; // MST completo,
31             // no hace // falta seguir
32         }
33     }
34
35     return {costoTotal, mst};
36 } // O(E log E)

```

Ejemplo de funcionamiento:

```

1 // grafo: 0-(1)-1 0-(4)-2 1-(2)-2 1-(5)-3 2-(3)-3
2
3 vector<Kruskal::Arista> aristas = {
4     {0, 1, 1}, {0, 2, 4}, {1, 2, 2}, {1, 3, 5}, {2, 3, 3}
5 };
6
7 Kruskal k(aristas, 4);
8 auto [costo, mst] = k.construirMST();
9
10 cout << costo;
11 // 6 (aristas elegidas: 0-1 (1), 1-2 (2), 2-3 (3) = 1+2+3=6)
12
13 cout << mst.size();
14 // 3 (n-1 = 3 aristas, el MST conecta los 4 nodos)

```

Regla práctica: Kruskal es "ordena + DSU"; si ya tienes DSU programado de la sección anterior, Kruskal es casi trivial de agregar encima. Elige Kruskal sobre Prim cuando el grafo viene como **lista de aristas dispersa**; considera Prim cuando el grafo es **denso** y ya lo tienes como matriz de adyacencia, ya que ahí Prim con un arreglo simple (sin heap) corre en $O(V^2)$ sin necesidad de ordenar $E \approx V^2$ aristas de antemano.

11.3 Prim

Idea: a diferencia de Kruskal (que ordena **todas** las aristas globalmente), Prim construye el MST **creciendo un solo árbol** desde un nodo inicial arbitrario: en cada paso, agrega la arista de **menor peso** que conecta el árbol ya construido con algún nodo **fuera** de él, hasta cubrir los n nodos. Es literalmente el mismo esqueleto de Dijkstra (heap con el "mejor candidato" disponible), cambiando qué se minimiza: Dijkstra minimiza **distancia acumulada desde el origen**; Prim minimiza el **peso de la próxima arista individual** que agrega al árbol.

¿Por qué usarlo? Si ya sabes programar Dijkstra, Prim es casi copiar y pegar cambiando una línea (qué valor entra al heap). La diferencia práctica con Kruskal está en la representación del grafo: Prim con `priority_queue` es $O(E \log V)$ igual que Dijkstra, pero

brilla especialmente en su variante **sin heap** ($O(V^2)$) cuando el grafo es **denso** y ya lo tienes como matriz de adyacencia — ahí evitas construir y ordenar una lista de $E \approx V^2$ aristas como pide Kruskal.

Casos de uso:

- Mismo problema que Kruskal (MST de costo mínimo), cuando el grafo viene como matriz de adyacencia densa en vez de lista de aristas.
- Cuando se necesita el MST "**creciendo**" desde un **nodo específico** de forma incremental (ej. simulaciones donde el árbol se construye en tiempo real).
- Igual que Kruskal, para encontrar el cuello de botella mínimo (la arista más pesada del MST resultante).

Complejidad: $O(E \log V)$ con `priority_queue` (igual que Dijkstra, mismo motivo: cada arista puede provocar como mucho una inserción al heap), o $O(V^2)$ con un arreglo simple sin heap — preferible en grafos densos donde $E \approx V^2$ y el heap ya no aporta.

```

1 struct Prim {
2     vector<vector<pair<int,int>>> adj; // adj[u] = {(v, peso)}
3     int n;
4
5     // CREATE: guarda el grafo (no dirigido, con pesos).
6     Prim(vector<vector<pair<int,int>>>& adj_, int n_) {
7         adj = adj_;
8         n = n_;
9     } // O(1)
10
11     // READ: construye el MST empezando desde 'inicio'. Devuelve
12     // {costoTotal, aristasDelMST} como pares (u, v). Si el grafo
13     // no es conexo, el árbol resultante solo cubre la componente de
14     // 'inicio' (MST parcial).
15     pair<long long, vector<pair<int,int>>> construirMST(int
16     inicio) {
17         vector<bool> enArbol(n, false);
18         vector<long long> mejorPeso(n, LLONG_MAX);
19         vector<int> mejorOrigen(n, -1);
20
21         // min-heap por peso de arista: {peso, nodo}
22         priority_queue<pair<long long,int>,
23             vector<pair<long long,int>>,
24             greater<>> pq;
25
26         mejorPeso[inicio] = 0;
27         pq.push({0, inicio});
28
29         long long costoTotal = 0;
30         vector<pair<int,int>> mst;
31
32         while (!pq.empty()) {
33             auto [peso, u] = pq.top(); pq.pop();
34
35             if (enArbol[u]) continue; // entrada obsoleta
36             // (ya se agrego con otro peso)
37
38             enArbol[u] = true;
39             costoTotal += peso;
40             if (mejorOrigen[u] != -1)
41                 mst.push_back({mejorOrigen[u], u});
42
43             for (auto& [v, pesoArista] : adj[u]) {
44                 if (enArbol[v]) continue;
45                 if (pesoArista >= mejorPeso[v]) continue;
46
47                 mejorPeso[v] = pesoArista;
48                 mejorOrigen[v] = u;
49                 pq.push({pesoArista, v});
50             }
51         }
52
53         return {costoTotal, mst};
54     } // O(E log V)
55 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0-(1)-1 0-(4)-2 1-(2)-2 1-(5)-3 2-(3)-3
2
3 vector<vector<pair<int,int>>> adj(4);
4 auto addEdge = [&](int u, int v, int peso) {
5     adj[u].push_back({v, peso});
6     adj[v].push_back({u, peso});
7 };
8 addEdge(0, 1, 1);
9 addEdge(0, 2, 4);
10 addEdge(1, 2, 2);
11 addEdge(1, 3, 5);
12 addEdge(2, 3, 3);
13
14 Prim p(adj, 4);
15 auto [costo, mst] = p.construirMST(0);
16
17 cout << costo;
18 // 6 (mismo MST que Kruskal: 0-1 (1), 1-2 (2), 2-3 (3) = 6)
19
20 cout << mst.size();
21 // 3

```

Nota: igual que en Dijkstra, el chequeo `if (enArbol[u]) continue;` es esencial por el mismo motivo: como `priority_queue` en C++ no soporta *decrease-key*, cada mejora de `mejorPeso[v]` mete una entrada **nueva** al heap en vez de actualizar la vieja; sin el chequeo, se reprocesarían entradas obsoletas (sigue siendo correcto, solo más lento).

Regla práctica: Kruskal y Prim siempre dan un MST de **el mismo costo total** (puede haber empates en qué aristas exactas eligen si hay pesos repetidos, pero el costo mínimo es único). La elección entre ambos es de **representación de datos**, no de corrección: lista de aristas / grafo disperso → Kruskal; matriz de adyacencia / grafo denso → Prim (idealmente la variante $O(V^2)$ sin heap en ese caso).

12 Strongly Connected Components (SCC)

12.1 Kosaraju

Idea: Kosaraju encuentra las mismas componentes fuertemente conexas que Tarjan, pero con **dos pasadas de DFS** en vez de una sola. Primera pasada: DFS normal sobre el grafo original, guardando cada nodo en una pila al **salir** de su llamada recursiva (exactamente el post-orden de DFS + Orden Topológico). Segunda pasada: DFS sobre el grafo **transpuesto** (todas las aristas invertidas), procesando los nodos en el orden en que salen de la pila (de arriba hacia abajo); cada árbol de DFS que se forma en esta segunda pasada es exactamente una SCC completa.

¿Por qué usarlo? La intuición clave es: si u y v están en la misma SCC, hay camino de u a v y de v a u —eso significa que, invirtiendo todas las aristas, esa propiedad se **preserva** (ambos caminos siguen existiendo, solo cambiados de dirección). Procesar los nodos en el grafo transpuesto empezando por el que terminó **más tarde** en la primera pasada garantiza que cada DFS de la segunda pasada no se "escape" hacia otra SCC —se queda exactamente dentro de la suya. Es más simple de razonar y de programar que Tarjan (no hay que llevar `tin/low`/pila simultáneamente), a cambio de necesitar **dos** recorridos y construir explícitamente el grafo transpuesto.

Casos de uso:

- Exactamente los mismos que Tarjan: condensar un grafo dirigido con ciclos en un DAG, detectar si el grafo es fuertemente conexo, encontrar el tamaño de la SCC más grande.
- Cuando ya tienes DFS + Orden Topológico programado y prefieres reutilizar ese patrón en vez de aprender `tin/low` de Tarjan.
- Cuando el problema ya requiere el grafo transpuesto para otra cosa (algunos problemas de alcanzabilidad dual lo piden de todas formas).

Complejidad: $O(V + E)$ — dos DFS completos (uno por pasada) más $O(V + E)$ para construir el grafo transpuesto; la constante es mayor que Tarjan (dos recorridos en vez de uno), pero el orden asintótico es el mismo.

```

1 struct Kosaraju {
2     vector<vector<int>> adj, adjT; // adjT = grafo transpuesto
3     vector<bool> visitado;
4     stack<int> orden; // orden de salida (post-orden)
5     vector<int> comp;
6     int numSCC;
7
8     // CREATE: guarda el grafo original y construye el
9     // transpuesto.
10    Kosaraju(vector<vector<int>>& adj_, int n) {
11        adj = adj_;
12        adjT.assign(n, {});
13
14        for (int u = 0; u < n; u++)
15            for (int v : adj[u])
16                adjT[v].push_back(u); // invierte cada arista
17
18        visitado.assign(n, false);
19        comp.assign(n, -1);
20        numSCC = 0;
21
22        // pasada 1: DFS en el grafo original, llenando 'orden'
23        for (int u = 0; u < n; u++)
24            if (!visitado[u])
25                dfs1(u);
26
27        // pasada 2: DFS en el transpuesto, en orden de salida
28        // (de arriba hacia abajo de la pila = de mayor a menor
29        // tiempo de salida de la pasada 1)
30        fill(visitado.begin(), visitado.end(), false);
31        while (!orden.empty()) {
32            int u = orden.top(); orden.pop();
33            if (!visitado[u]) {
34                dfs2(u, numSCC);
35                numSCC++;
36            }
37        }
38        // O(V + E)
39
40        void dfs1(int u) {
41            visitado[u] = true;
42            for (int v : adj[u])
43                if (!visitado[v])
44                    dfs1(v);
45            orden.push(u); // post-orden, igual que DFS + Orden
46            // Topologico
47        }
48
49        void dfs2(int u, int id) {
50            visitado[u] = true;
51            comp[u] = id;
52            for (int v : adjT[u])
53                if (!visitado[v])
54                    dfs2(v, id);
55        }
56    };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo, una SCC)
2 //                2 -> 3
3 //                3 -> 4 -> 3 (ciclo, otra SCC)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};

```

```

10 adj[4] = {3};
11
12 Kosaraju ks(adj, 5);
13
14 cout << ks.numSCC;
15 // 2
16
17 cout << ks.comp[0] << " " << ks.comp[1] << " " << ks.comp[2];
18 // mismo id para 0, 1, 2 (SCC del ciclo 0-1-2)
19
20 cout << ks.comp[3] << " " << ks.comp[4];
21 // mismo id para 3, 4 (SCC del ciclo 3-4)

```

Regla práctica: Kosaraju y Tarjan siempre encuentran **exactamente las mismas SCCs** (el algoritmo no cambia el resultado, solo cómo se llega a él) — la elección entre ambos es de preferencia de implementación, no de corrección ni de complejidad asintótica ($O(V + E)$ los dos). Usa Tarjan si prefieres un solo DFS y ya te sientes cómodo con `tin/low/pila` (además te da gratis los ids de SCC en **orden topológico inverso**, útil para recorrer el DAG condensado directamente); usa Kosaraju si prefieres separar mentalmente "encontrar el orden" de "agrupar por alcanzabilidad" en dos pasos explícitos.

12.2 Tarjan SCC

Idea: Tarjan encuentra las componentes fuertemente conexas en **una sola pasada** de DFS, usando el mismo mecanismo de `tin/low` que Puentes, Puntos de Articulación y Componentes Biconectadas (ver sección de Conectividad), pero apilando **nodos** en vez de aristas: cada nodo se apila al descubrirse, con un flag `enPila`; un nodo u es **raíz de su SCC** exactamente cuando `low[u] == tin[u]` — en ese momento se desapila todo hasta u inclusive, y ese grupo es una SCC completa. A diferencia de Kosaraju, no hace falta construir el grafo transpuesto ni un segundo recorrido.

¿Por qué usarlo? Es más eficiente en constante que Kosaraju (un solo DFS en vez de dos, sin transponer el grafo), y trae una ventaja extra: las SCCs que va cerrando Tarjan salen naturalmente en **orden topológico inverso** del grafo condensado (la primera SCC en cerrarse es un "sumidero" del DAG condensado, la última en cerrarse es una "fuente") — útil si después necesitas recorrer el DAG condensado sin volver a ordenar nada.

Casos de uso: los mismos que Kosaraju — condensar un grafo dirigido en un DAG, verificar conectividad fuerte total, encontrar la SCC más grande — con la diferencia de que aquí el orden de cierre de las SCCs ya viene topológicamente ordenado (inverso), lo cual aprovecha directamente Condensation Graph más abajo.

Complejidad: $O(V + E)$, un solo DFS con trabajo constante extra por nodo/arista.

```

1 struct TarjanSCC {
2     vector<vector<int>> adj;
3     vector<int> tin, low, comp;
4     vector<bool> enPila;
5     stack<int> pila;
6     int timer, numSCC;
7
8     // CREATE: guarda el grafo dirigido y corre Tarjan desde cada
9     // nodo no visitado. comp[u] queda asignado en orden
10    // topológico
11    // INVERSO del grafo condensado (comp 0 = la primera en
12    // cerrarse
13    // = un sumidero del DAG condensado).
14    TarjanSCC(vector<vector<int>>& adj_, int n) {
15        adj = adj_;
16        tin.assign(n, -1);
17        low.assign(n, -1);
18        comp.assign(n, -1);
19        enPila.assign(n, false);
20    }
21
22    void dfs(int u) {
23        tin[u] = low[u] = timer++;
24        pila.push(u);
25        enPila[u] = true;
26
27        for (int v : adj[u]) {
28            if (tin[v] == -1) {
29                dfs(v);
30                low[u] = min(low[u], low[v]);
31            } else if (enPila[v]) {
32                low[u] = min(low[u], tin[v]);
33            }
34        }
35
36        if (low[u] == tin[u]) {
37            while (true) {
38                int w = pila.top(); pila.pop();
39                enPila[w] = false;
40                comp[w] = numSCC;
41                if (w == u) break;
42            }
43            numSCC++;
44        }
45    }
46
47    int numSCC() const { return numSCC; }
48    vector<int> comp() const { return comp; }
49 };

```

```

18 timer = 0;
19 numSCC = 0;
20
21 for (int u = 0; u < n; u++)
22     if (tin[u] == -1)
23         dfs(u);
24 // O(V + E)
25
26 void dfs(int u) {
27     tin[u] = low[u] = timer++;
28     pila.push(u);
29     enPila[u] = true;
30
31     for (int v : adj[u]) {
32         if (tin[v] == -1) {
33             dfs(v);
34             low[u] = min(low[u], low[v]);
35         } else if (enPila[v]) {
36             low[u] = min(low[u], tin[v]);
37         }
38     }
39
40     if (low[u] == tin[u]) {
41         while (true) {
42             int w = pila.top(); pila.pop();
43             enPila[w] = false;
44             comp[w] = numSCC;
45             if (w == u) break;
46         }
47         numSCC++;
48     }
49 }
50 };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo, una SCC)
2 //                2 -> 3
3 //                3 -> 4 -> 3 (ciclo, otra SCC)
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};
10 adj[4] = {3};
11
12 TarjanSCC tj(adj, 5);
13
14 cout << tj.numSCC;
15 // 2
16
17 cout << tj.comp[3] << " " << tj.comp[0];
18 // comp[3] < comp[0]
19 // (la SCC {3,4} se cierra primero -- es un sumidero del DAG
20 // condensado, ya que 2 -> 3 pero nada sale de {3,4} hacia
21 // {0,1,2})

```

12.3 Condensation Graph

Idea: el *condensation graph* (grafo condensado) es el resultado de **contraer cada SCC a un solo nodo**: si u y v están en la misma SCC, se fusionan en un nodo; si hay una arista original de una SCC A a una SCC B distinta, queda una arista $A \rightarrow B$ en el grafo condensado. El resultado **siempre** es un DAG — no puede haber ciclos entre SCCs distintas, porque si los hubiera, esas SCCs en realidad deberían haber sido una sola SCC más grande (contradicción con que ya son maximales).

¿Por qué usarlo? Es el paso que conecta Tarjan/Kosaraju con **todo** lo que ya sabes hacer sobre DAGs: una vez condensado, puedes aplicar DFS + Orden Topológico, Conteo de Caminos, DP sobre DAG, o Shortest Path en Grafos Multietapa directamente sobre el condensado — sin preocuparte de que el grafo original tuviera ciclos. Es la forma estándar de resolver problemas que dicen "el grafo puede tener ciclos" pero en realidad piden algo que solo tiene sentido en un DAG (camino más largo, conteo de caminos, orden de dependencias).

Casos de uso:

- Cualquier problema sobre un grafo dirigido con ciclos que pida algo intrínsecamente de DAG (camino más largo, número de caminos, orden topológico de "grupos").
- Sumar/combinar un valor asociado a cada nodo dentro de su SCC (ej. "peso total" de cada grupo) antes de propagarlo por el DAG condensado.
- Determinar cuántas SCCs son alcanzables o pueden alcanzar a una SCC dada, tratándolo como alcanzabilidad en un DAG (DFS + Cerramiento Transitivo sobre el condensado, mucho más barato que sobre el grafo original con ciclos).

Complejidad: $O(V + E)$ — una vez calculadas las SCCs (con Tarjan o Kosaraju, ambos $O(V + E)$), construir el condensado es un solo recorrido de las aristas originales, agregando una arista condensada por cada arista original que cruza de una SCC a otra distinta (con deduplicación opcional si no se quieren aristas múltiples).

```

1 struct GrafoCondensado {
2     vector<vector<int>> adjCond; // adjCond[SCC] = SCCs vecinas
3     int numSCC;
4
5     // CREATE: a partir del grafo original y las SCCs ya
6     // calculadas
7     // (por ejemplo, con TarjanSCC o Kosaraju), construye el DAG
8     // condensado. 'comp' asigna a cada nodo original el id de su
9     // SCC.
10    GrafoCondensado(vector<vector<int>>& adj, vector<int>& comp,
11                    int numSCC_) {
12        numSCC = numSCC_;
13        adjCond.assign(numSCC, {});
14
15        set<pair<int,int>> vistas; // evita aristas condensadas
16        // duplicadas
17
18        for (int u = 0; u < (int)adj.size(); u++) {
19            for (int v : adj[u]) {
20                if (comp[u] == comp[v]) continue; // misma SCC,
21                // no es arista
22                // del condensado
23
24                if (vistas.count({comp[u], comp[v]})) continue;
25                vistas.insert({comp[u], comp[v]});
26
27                adjCond[comp[u]].push_back(comp[v]);
28            }
29        } // O((V + E) log E
30
31        // por el set; O(V+E)
32        // si se acepta tener
33        // aristas duplicadas
34    };

```

Ejemplo de funcionamiento:

```

1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (SCC A = {0,1,2})
2 //                2 -> 3
3 //                3 -> 4 -> 3 (SCC B = {3,4})
4
5 vector<vector<int>> adj(5);
6 adj[0] = {1};
7 adj[1] = {2};
8 adj[2] = {0, 3};
9 adj[3] = {4};
10 adj[4] = {3};
11
12 TarjanSCC tj(adj, 5);
13 GrafoCondensado gc(adj, tj.comp, tj.numSCC);
14
15 // el condensado tiene exactamente 2 nodos (una por SCC) y 1
16 // arista:
17 // SCC{3,4} -> SCC{0,1,2} (ya que Tarjan cierra {3,4} primero,
18 // su id es menor -- ver nota de orden topologico inverso
19 // arriba)
20
21 cout << gc.adjCond[tj.comp[3]][0] == tj.comp[0];
22 // true (la SCC de 3 tiene una arista hacia la SCC de 0)

```

Regla práctica: construir el condensado es siempre el mismo patrón de dos pasos —(1) calcular SCCs con Tarjan o Kosaraju, (2) recorrer las aristas originales una vez, quedándose solo con las que cruzan de SCC —sin importar qué algoritmo de SCC hayas usado. Si usaste Tarjan, aprovecha que los ids de `comp` ya vienen en orden topológico inverso del condensado: puedes iterar `numSCC-1` hasta 0 directamente para procesar el DAG condensado sin correr otro DFS + Orden Topológico encima.

13 Bipartite Graphs

13.1 2-Coloring

Idea: un grafo es **bipartito** si sus nodos se pueden partir en dos grupos tales que **toda** arista conecta un nodo de un grupo con uno del otro (nunca dos nodos del mismo grupo). Se verifica con BFS o DFS "por capas", asignando colores **alternados** (0/1) a medida que se visita: el color de un vecino siempre debe ser el **opuesto** al del nodo actual. Si en algún momento un vecino ya visitado tiene el **mismo** color que el nodo actual, el grafo **no** es bipartito —esa arista conecta dos nodos que deberían estar en grupos distintos y no lo están.

¿Por qué usarlo? Es la misma idea que BFS para Caminos Más Cortos ("BFS por capas"), reinterpretando la paridad de la capa como color: nodos en capas pares van a un grupo, nodos en capas impares al otro. La razón teórica de por qué esto detecta bipartición correctamente: un grafo es bipartito **si y solo si** no tiene ningún **ciclo de longitud impar** —y un ciclo impar es exactamente lo que provoca el conflicto de color al cerrar el ciclo.

Casos de uso:

- Verificar si un grafo admite partición en dos grupos sin conflictos (asignación de tareas mutuamente excluyentes, horarios sin choques, etc.).
- Como **prerrequisito obligatorio** de Bipartite Matching (siguiente sección): solo tiene sentido buscar un matching bipartito si el grafo realmente lo es, y 2-Coloring además te da gratis los dos lados L/R de la partición.
- Modelar restricciones tipo "estos dos elementos no pueden estar juntos" como aristas, y usar 2-Coloring para ver si existe una asignación válida en dos categorías.

Complejidad: $O(V + E)$ — un BFS/DFS normal con una comparación de color extra por arista, revisando todas las componentes del grafo (un grafo desconexo puede tener partes bipartitas y partes no, o varias componentes bipartitas independientes).

```

1 struct DosColoreo {
2     vector<vector<int>> adj;
3     vector<int> color; // -1 = sin colorear, 0 o 1 = los dos
4     // grupos
5     int n;
6
7     // CREATE: guarda el grafo.
8     DosColoreo(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        color.assign(n, -1);
12    } // O(1)

```

```

12 // READ: true si el grafo completo es bipartito. Colorea
13 // todas
14 // las componentes en el proceso (revisa cada nodo no
15 // coloreado).
16 bool esBipartito() {
17     for (int u = 0; u < n; u++)
18         if (color[u] == -1 && !bfs(u))
19             return false;
20     return true;
21 } // O(V + E)
22
23 bool bfs(int origen) {
24     queue<int> q;
25     q.push(origen);
26     color[origen] = 0;
27
28     while (!q.empty()) {
29         int u = q.front(); q.pop();
30
31         for (int v : adj[u]) {
32             if (color[v] == -1) {
33                 color[v] = 1 - color[u]; // color opuesto
34                 q.push(v);
35             } else if (color[v] == color[u]) {
36                 return false; // mismo color en una arista
37                 // -> ciclo impar -> no bipartito
38             }
39         }
40     }
41     return true;
42 }
43 };

```

Ejemplo de funcionamiento:

```

1 // grafo bipartito: 0 - 1      0 - 3
2 //                  2 - 1      2 - 3
3 // (es un ciclo de longitud 4, par -> bipartito)
4
5 vector<vector<int>> adj(4);
6 adj[0] = {1, 3}; adj[1] = {0, 2};
7 adj[2] = {1, 3}; adj[3] = {0, 2};
8
9 DosColoreo dc(adj, 4);
10 cout << dc.esBipartito();
11 // true
12
13 // grafo NO bipartito: triangulo 0-1-2-0 (ciclo de longitud 3,
14 // impar)
15 vector<vector<int>> adj2(3);
16 adj2[0] = {1, 2}; adj2[1] = {0, 2}; adj2[2] = {0, 1};
17
18 DosColoreo dc2(adj2, 3);
19 cout << dc2.esBipartito();
20 // false

```

13.2 Bipartite Matching

Idea: dado un grafo bipartito con lados L y R , un **matching** es un subconjunto de aristas donde ningún nodo se repite (cada nodo de L emparejado con a lo más un nodo de R , y viceversa). El **matching máximo** se encuentra con el algoritmo de Kuhn: para cada nodo de L sin emparejar, intenta un DFS buscando un **camino aumentante** —si llega a un nodo de R libre, lo empareja directamente; si llega a uno ya emparejado, intenta "desplazar" a su pareja actual (recursivamente, buscándole otra opción) para liberar espacio.

¿Por qué usarlo? Es la herramienta estándar para cualquier problema de "asignación óptima uno-a-uno" entre dos conjuntos —trabajadores a tareas, estudiantes a proyectos, jugadores a posiciones— donde cada asignación válida es una arista en un grafo bipartito. La garantía clave del algoritmo (teorema de Berge): un matching es máximo **si y solo si** no existe ningún camino aumentante —por eso repetir "buscar camino aumentante desde cada nodo libre de L " hasta que ya ninguno encuentre uno, termina garantizadamente en el matching máximo.

Casos de uso:

- Asignación máxima uno-a-uno entre dos conjuntos con compatibilidades dadas (trabajadores-tareas, alumnos-cursos).
- Como subrutina de problemas más generales: cobertura mínima de vértices en grafos bipartitos (teorema de König: tamaño del matching máximo = tamaño de la cobertura mínima de vértices).
- Problemas de "cuántos pares disjuntos puedo formar" cuando la compatibilidad se puede modelar como bipartita (ej. tareas con horarios que no chocan, visto como grafo).

Complejidad: $O(V \cdot E)$ con Kuhn simple (un DFS por cada nodo de L , en el peor caso $O(E)$ cada uno) —suficiente para $V \lesssim 1000$ –2000. Para grafos más grandes existe Hopcroft-Karp, $O(E\sqrt{V})$, fuera del alcance de esta sección.

```

1 struct BipartiteMatching {
2     vector<vector<int>> adj; // adj[u] = vecinos en R, para
3     // cada
4     // u en L
5     vector<int> matchR; // matchR[v] = nodo de L
6     // emparejado
7     // con v en R, -1 si libre
8     vector<bool> visitado; // marca de la búsqueda actual (se
9     // reinicia en cada intento)
10    int nL;
11
12    // CREATE: guarda el grafo bipartito (nL nodos en L, adj[u]
13    // lista los vecinos de u en R). nR = tamaño del lado R.
14    BipartiteMatching(vector<vector<int>>& adj_, int nL_, int nR)
15    {
16        adj = adj_;
17        nL = nL_;
18        matchR.assign(nR, -1);
19    } // O(1)
20
21    // busca un camino aumentante desde u (nodo de L), intentando
22    // desplazar parejas ya asignadas si hace falta.
23    bool intentar(int u) {
24        for (int v : adj[u]) {
25            if (visitado[v]) continue;
26            visitado[v] = true;
27
28            // v esta libre, o su pareja actual puede reubicarse
29            if (matchR[v] == -1 || intentar(matchR[v])) {
30                matchR[v] = u;
31                return true;
32            }
33        }
34        return false;
35    }
36
37    // READ: calcula el matching maximo. Devuelve su tamaño;
38    // matchR queda con la asignacion final.
39    int matchingMaximo() {
40        int total = 0;
41
42        for (int u = 0; u < nL; u++) {
43            visitado.assign(matchR.size(), false);
44            if (intentar(u)) total++;
45        }
46        return total;
47    } // O(V * E)
48 };

```

Ejemplo de funcionamiento:

```

1 // L = {0, 1, 2}, R = {0, 1, 2}
2 // 0 - {0, 1}
3 // 1 - {0}
4 // 2 - {1, 2}
5
6 vector<vector<int>> adj(3);
7 adj[0] = {0, 1};
8 adj[1] = {0};
9 adj[2] = {1, 2};
10
11 BipartiteMatching bm(adj, 3, 3);

```

```

12 int maximo = bm.matchingMaximo();
13
14 cout << maximo;
15 // 3
16 // posible asignacion: 1-0 (desplaza a 0), 0-1, 2-2
17 // (0 originalmente probó emparejarse con 0 en R, pero al
18 // llegar 1
19 // que SOLO puede ir a 0, 0 se desplaza hacia 1 en R,
20 // liberando
21 // espacio para que 1 tome 0 en R)

```

Nota: el vector `visitado` se reinicia en **cada** llamada a `matchingMaximo()` por cada nodo de L —no es el mismo `visitado` de un DFS normal que se llena una sola vez para todo el grafo. Aquí marca "ya considerado en este intento", para no reprocesar el mismo nodo de R dos veces dentro de una sola búsqueda de camino aumentante, aunque sí se vuelve a visitar en el siguiente intento (desde el próximo nodo de L).

Regla práctica: 2-Coloring y Bipartite Matching son secuenciales: primero verificas (y separas en L/R) con 2-Coloring, y solo si el grafo realmente es bipartito tiene sentido correr Matching sobre él —Kuhn asume esa estructura y no detecta por sí solo si el grafo no lo es.

14 Trees

14.1 DFS en Árboles

Idea: un árbol es simplemente un grafo conexo sin ciclos —eso significa que un DFS sobre un árbol **nunca** necesita marcar "visitado" con un arreglo separado: basta con no regresar por la misma arista al **padre** inmediato, ya que no hay ningún otro camino de vuelta (no hay ciclos que puedan reconectar el DFS con un nodo ya procesado). Esa simplificación es lo que distingue DFS en árboles del DFS de grafos generales, y es la base de casi todo el DP sobre árboles: recorres en post-orden, y cada nodo combina los resultados ya calculados de sus hijos.

¿Por qué usarlo? Es el patrón detrás de prácticamente **todo** problema de árboles: tamaño de subárbol, profundidad, diámetro, suma de valores por subárbol, DP sobre árboles en general. Reconocer "esto es información que depende de mis hijos" es la señal para post-orden DFS —calculas la respuesta de cada hijo primero, y el nodo actual combina esos resultados al salir de la recursión.

Casos de uso:

- Tamaño de cada subárbol: $\text{tam}[u] = 1 + \sum \text{tam}[\text{hijo}]$.
- Profundidad/altura de cada nodo, o altura total del árbol.
- Suma o valor agregado de cada subárbol (útil para responder queries tipo "¿cuánto suma el subárbol de u ?").
- Diámetro del árbol (la distancia más larga entre dos nodos cualquiera): con dos DFS —desde cualquier nodo encuentra el más lejano a , desde a encuentra el más lejano b ; la distancia $a-b$ es el diámetro (propiedad de árboles).

Complejidad: $O(n)$ — cada nodo se visita exactamente una vez, cada arista se recorre exactamente una

vez (a diferencia de un grafo general, aquí no hace falta ni siquiera el arreglo `visitado`, basta con excluir al padre).

```

1 struct DFSArbol {
2     vector<vector<int>> adj;
3     vector<int> tam, profundidad;
4     int n;
5
6     // CREATE: guarda el arbol (n nodos, n-1 aristas).
7     DFSArbol(vector<vector<int>>& adj_, int n_) {
8         adj = adj_;
9         n = n_;
10        tam.assign(n, 0);
11        profundidad.assign(n, 0);
12    } // O(1)
13
14    // READ: corre DFS desde raíz, llenando tam[] y profundidad[]
15    // de una sola pasada.
16    void dfs(int u, int padre, int d) {
17        profundidad[u] = d;
18        tam[u] = 1;
19
20        for (int v : adj[u]) {
21            if (v == padre) continue; // unica exclusion necesaria,
22            // no hace falta 'visitado'
23
24            dfs(v, u, d + 1);
25            tam[u] += tam[v]; // combina resultados de los hijos
26            // DESPUES de que ya terminaron
27            // (post-orden)
28        }
29    } // O(n)
30 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 DFSArbol da(adj, 5);
15 da.dfs(0, -1, 0);
16
17 cout << da.tam[0];
18 // 5 (todo el arbol)
19
20 cout << da.tam[1];
21 // 3 (1, 3, 4)
22
23 cout << da.profundidad[4];
24 // 2

```

14.2 BFS en Árboles

Idea: el mismo principio de "sin ciclos, no hace falta `visitado` extra, solo excluir al padre" aplica igual a BFS sobre árboles —con la ventaja de que BFS recorre **por niveles**, así que es la herramienta natural cuando lo que se pide depende explícitamente de la **profundidad/nivel** de cada nodo en vez de depender de sus subárboles.

¿Por qué usarlo? DFS y BFS en árboles calculan las mismas distancias/profundidades ($O(n)$ ambos), pero BFS procesa los nodos en **orden creciente de profundidad**, lo cual es justo lo que necesitas cuando el problema pide algo "nivel por nivel" (ej. "imprime el árbol por niveles", "suma de valores en cada profundidad") — con DFS tendrías que agrupar por profundidad manualmente después; con BFS, ya vienen agrupados por construcción.

Casos de uso:

- Recorrido por niveles ("level order traversal"): imprimir o procesar el árbol nivel por nivel.
- Distancia mínima desde la raíz a cada nodo (aunque en un árbol, DFS también la da correctamente — la ventaja de BFS es solo el orden de procesamiento por capas).
- Encontrar el nodo más profundo, o todos los nodos a una profundidad exacta k .
- Como una de las dos pasadas del cálculo de diámetro de árbol (BFS desde cualquier nodo para hallar el extremo más lejano, igual que con DFS — aquí ambos sirven).

Complejidad: $O(n)$ — cada nodo se encola y desencola una sola vez.

```

1 struct BFSArbol {
2     vector<vector<int>> adj;
3     vector<int> profundidad, padre;
4     vector<vector<int>> porNivel; // porNivel[d] = nodos a
5     // profundidad d
6     int n;
7
8     // CREATE: guarda el arbol.
9     BFSArbol(vector<vector<int>>& adj_, int n_) {
10         adj = adj_;
11         n = n_;
12     } // O(1)
13
14     // READ: corre BFS desde raiz, llenando profundidad[], padre
15     // y agrupando los nodos por nivel.
16     void bfs(int raiz) {
17         profundidad.assign(n, -1);
18         padre.assign(n, -1);
19         porNivel.clear();
20
21         queue<int> q;
22         q.push(raiz);
23         profundidad[raiz] = 0;
24
25         while (!q.empty()) {
26             int u = q.front(); q.pop();
27
28             if ((int)porNivel.size() <= profundidad[u])
29                 porNivel.push_back({});
30             porNivel[profundidad[u]].push_back(u);
31
32             for (int v : adj[u]) {
33                 if (v == padre[u]) continue; // igual que en DFS,
34                 // basta excluir al
35                 // padre
36
37                 padre[v] = u;
38                 profundidad[v] = profundidad[u] + 1;
39                 q.push(v);
40             }
41         }
42     } // O(n)
43 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 BFSArbol ba(adj, 5);
15 ba.bfs(0);
16
17 cout << ba.porNivel.size();
18 // 3 (niveles 0, 1, 2)
19
20 for (int u : ba.porNivel[1]) cout << u << " ";
21 // 1 2 (nivel 1: hijos directos de la raiz)

```

```

22 for (int u : ba.porNivel[2]) cout << u << " ";
23 // 3 4 (nivel 2: nietos de la raiz)
24

```

Regla práctica: en árboles, tanto DFS como BFS corren en $O(n)$ y ninguno necesita **visitado** — basta con excluir al padre en cada paso, porque la ausencia de ciclos lo garantiza. Elige DFS cuando lo que calculas depende de **combinar resultados de subárboles** (post-orden: tamaños, sumas, diámetro vía DP); elige BFS cuando lo que calculas depende de **agrupar por nivel/profundidad** explícitamente (recorridos por niveles, "todos los nodos a distancia k de la raíz").

14.3 Diámetro de un Árbol

Idea: el diámetro de un árbol es la distancia (en número de aristas) entre el par de nodos **más lejano** entre sí. Se calcula con una propiedad no obvia pero muy útil de los árboles: **desde cualquier nodo x , el nodo más lejano a es siempre un extremo del diámetro**. Eso da el algoritmo de **dos pasadas**: (1) BFS/DFS desde un nodo cualquiera para encontrar el nodo más lejano a ; (2) BFS/DFS desde a para encontrar el nodo más lejano b — la distancia $a-b$ es el diámetro. Nótese que x inicial puede ser literalmente cualquier nodo, no hace falta adivinar ni probar todos.

¿Por qué usarlo? La alternativa ingenua —correr BFS desde **cada** nodo y quedarte con la distancia máxima global— cuesta $O(n^2)$. El truco de las dos pasadas baja eso a $O(n)$ aprovechando la estructura de árbol (sin ciclos): la garantía de que "el más lejano desde cualquier punto es un extremo del diámetro" **no** es cierta en grafos generales, solo en árboles.

Casos de uso:

- Distancia máxima entre cualquier par de nodos de un árbol.
- Encontrar el par de nodos que realiza esa distancia máxima (reconstruyendo el camino de a a b con **padre[]**, igual que en BFS en Árboles).
- Como subrutina de Centro de un Árbol (siguiente sección): el centro es literalmente el punto medio del camino del diámetro.
- Problemas de "radio mínimo" o "tiempo máximo" para que algo se propague desde el peor nodo posible — el diámetro acota estos valores.

Complejidad: $O(n)$ — dos BFS/DFS completos, cada uno $O(n)$.

```

1 struct DiametroArbol {
2     vector<vector<int>> adj;
3     int n;
4
5     // CREATE: guarda el arbol.
6     DiametroArbol(vector<vector<int>>& adj_, int n_) {
7         adj = adj_;
8         n = n_;
9     } // O(1)
10
11     // BFS auxiliar: devuelve {nodo mas lejano, distancia} desde
12     // src.
13     pair<int,int> bfsMasLejano(int src) {
14         vector<int> dist(n, -1);
15         queue<int> q;
16         q.push(src);
17         dist[src] = 0;
18     }
19 };

```

```

18 int mejorNodo = src, mejorDist = 0;
19
20 while (!q.empty()) {
21     int u = q.front(); q.pop();
22
23     if (dist[u] > mejorDist) {
24         mejorDist = dist[u];
25         mejorNodo = u;
26     }
27
28     for (int v : adj[u]) {
29         if (dist[v] != -1) continue;
30         dist[v] = dist[u] + 1;
31         q.push(v);
32     }
33 }
34
35 return {mejorNodo, mejorDist};
36 } // O(n)
37
38 // READ: diametro del arbol (dos pasadas de BFS).
39 int diametro() {
40     auto [a, _] = bfsMasLejano(0); // paso 1: cualquier
41     // nodo -> extremo a
42     auto [b, dist] = bfsMasLejano(a); // paso 2: a ->
43     // extremo b
44     return dist;
45 } // O(n)
46 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           /
5 //          3
6 //         /
7 //        4
8
9 vector<vector<int>> adj(5);
10 adj[0] = {1, 2};
11 adj[1] = {0, 3};
12 adj[2] = {0};
13 adj[3] = {1, 4};
14 adj[4] = {3};
15
16 DiametroArbol da(adj, 5);
17
18 cout << da.diametro();
19 // 4 (camino 4-3-1-0-2, 4 aristas)

```

Nota: existe una alternativa de **una sola pasada** con DP (post-orden, como en DFS en Árboles): para cada nodo, calcular las dos ramas más largas de sus hijos y sumar; el diámetro es el máximo de esa suma sobre todos los nodos. Es igual de $O(n)$ pero evita el segundo BFS —útil si ya estás haciendo otro DP sobre el árbol y quieres aprovechar el mismo recorrido. El método de dos pasadas es preferible cuando además necesitas identificar **qué** par de nodos (a, b) realiza el diámetro, no solo su longitud.

14.4 Centro de un Árbol

Idea: el centro de un árbol es el nodo (o los dos nodos) que **minimiza la distancia máxima** a cualquier otro nodo del árbol —el punto "más equilibrado". Un árbol tiene **uno o dos** centros nunca más: si el diámetro tiene longitud **par**, hay un único centro (el punto medio exacto del camino del diámetro); si es **impar**, hay dos centros adyacentes (los dos nodos de en medio del camino, que no tiene un punto medio exacto).

¿Por qué usarlo? Hay dos formas equivalentes de encontrarlo, ambas $O(n)$: (1) **vía el diámetro** —calculas el camino del diámetro con Diámetro de un Árbol, y el/los nodo(s) en la posición central de ese camino son el centro; (2) **"pelado de hojas"** (*leaf trimming*)—quitas todas las hojas actuales del árbol capa por capa (como

un BFS multi-fuente desde las hojas hacia adentro), y lo que sobrevive al final (1 o 2 nodos) es el centro. La segunda es más directa de programar cuando ya tienes BFS multi-fuente a mano (mismo patrón que Shortest Path desde Múltiples Orígenes, pero "encogiendo" el árbol en vez de calculando distancias).

Casos de uso:

- Elegir la raíz "óptima" de un árbol quiere minimizar la altura resultante (rerootear en el centro da la altura mínima posible).
- Problemas de "ubica un servidor/recurso central" que minimice la distancia máxima a cualquier nodo cliente.
- Como building block de **centroid decomposition** (fuera del alcance de esta sección, pero el centro es el primer paso de esa técnica).

Complejidad: $O(n)$ con cualquiera de los dos métodos.

```

1 struct CentroArbol {
2     vector<vector<int>> adj;
3     vector<int> grado;
4     int n;
5
6     // CREATE: guarda el arbol y el grado inicial de cada nodo
7     // (numero de vecinos).
8     CentroArbol(vector<vector<int>>& adj_, int n_) {
9         adj = adj_;
10        n = n_;
11        grado.assign(n, 0);
12
13        for (int u = 0; u < n; u++)
14            grado[u] = adj[u].size();
15    } // O(n)
16
17    // READ: encuentra el/los centro(s) "pelando" hojas capa por
18    // capa hasta que quedan 1 o 2 nodos.
19    vector<int> centro() {
20        if (n == 1) return {0};
21
22        vector<int> g = grado; // copia para no mutar el original
23        queue<int> hojas;
24
25        for (int u = 0; u < n; u++)
26            if (g[u] == 1)
27                hojas.push(u);
28
29        int restantes = n;
30        vector<int> capaActual;
31
32        while (restantes > 2) {
33            int cantidadHojas = hojas.size();
34            restantes -= cantidadHojas;
35
36            for (int i = 0; i < cantidadHojas; i++) {
37                int u = hojas.front(); hojas.pop();
38
39                for (int v : adj[u]) {
40                    g[v]--;
41                    if (g[v] == 1)
42                        hojas.push(v);
43                }
44            }
45        }
46
47        // lo que queda en la cola (1 o 2 nodos) es el centro
48        vector<int> resultado;
49        while (!hojas.empty()) {
50            resultado.push_back(hojas.front());
51            hojas.pop();
52        }
53        return resultado;
54    } // O(n)
55 };

```

Ejemplo de funcionamiento:

```

1 // arbol (cadena): 0 - 1 - 2 - 3 - 4
2 // diametro = 4 (par) -> un solo centro, el punto medio exacto
3

```

```

4 vector<vector<int>> adj(5);
5 adj[0] = {1}; adj[1] = {0, 2}; adj[2] = {1, 3};
6 adj[3] = {2, 4}; adj[4] = {3};
7
8 CentroArbol ca(adj, 5);
9 for (int u : ca.centro()) cout << u << " ";
10 // 2 (unico centro, justo en medio de la cadena de 5 nodos)
11
12 // arbol (cadena): 0 - 1 - 2 - 3
13 // diametro = 3 (impar) -> dos centros adyacentes
14
15 vector<vector<int>> adj2(4);
16 adj2[0] = {1}; adj2[1] = {0, 2};
17 adj2[2] = {1, 3}; adj2[3] = {2};
18
19 CentroArbol ca2(adj2, 4);
20 for (int u : ca2.centro()) cout << u << " ";
21 // 1 2 (los dos nodos centrales)

```

Regla práctica: el pelado de hojas y el método vía diámetro **siempre** coinciden en el resultado —son dos formas de llegar al mismo centro. Usa pelado de hojas si el problema ya viene en términos de "ir quitando hojas" o si prefieres no calcular explícitamente el camino del diámetro; usa el método vía diámetro si de todas formas ya necesitas el diámetro para otra parte del problema, ya que entonces el centro sale casi gratis indexando al medio del camino ya reconstruido.

14.5 Combinatoria en Árboles Ordenados

Idea: dado un árbol enraizado de n nodos, un **orden topológico** es cualquier permutación de sus nodos donde cada nodo aparece **antes** que todos sus descendientes (por ejemplo, "en qué orden pudieron haberse insertado" si el árbol se construyó agregando nodos de a uno, siempre como hijo de algo ya presente). El número total de órdenes topológicos válidos tiene una fórmula cerrada muy elegante:

$$\frac{n!}{\prod_v \text{tam}[v]}$$

donde $\text{tam}[v]$ es el tamaño del subárbol de v (incluyéndose a sí mismo). Se calcula con un solo DFS post-orden (igual que DFS en Árboles) para obtener los tamaños, y luego un producto/cociente modular sobre todos los nodos.

¿Por qué usarlo? La fórmula evita tener que **contar** las permutaciones una por una (imposible, son hasta $n!$) o hacer DP combinatoria compleja: reduce el problema entero a "un DFS de tamaños + un producto modular", el mismo patrón de precómputo que ya usas para factoriales e inversos modulares. Es la herramienta directa cuando un problema pregunta "¿de cuántas formas se pudo haber construido/insertado/etiquetado este árbol respetando el orden padre-antes-que-hijo?".

Casos de uso:

- Contar de cuántas formas se pudo insertar n elementos en una estructura tipo árbol (heap, trie, etc.) para terminar con la forma dada.
- Contar etiquetados válidos 1.. n de los nodos de un árbol tales que cada nodo tenga una etiqueta menor que todos sus hijos ("árbol como heap mínimo").
- Como building block de problemas más grandes que combinan esta cuenta con otras restricciones (ej. multiplicar por factoriales adicionales si además el orden **entre** hermanos importa de alguna forma extra).

Complejidad: $O(n)$ para el DFS de tamaños, más $O(n \log \text{MOD})$ si se calculan inversos modulares individuales con exponenciación rápida (o $O(n)$ total si se precomputan inversos de factoriales una sola vez).

```

1 const long long MOD = 1e9 + 7;
2
3 struct CombinatoriaArbol {
4     vector<vector<int>> adj;
5     vector<long long> tam;
6     int n;
7
8     // CREATE: guarda el arbol enraizado.
9     CombinatoriaArbol(vector<vector<int>>& adj_, int n_) {
10         adj = adj_;
11         n = n_;
12         tam.assign(n, 0);
13     } // O(1)
14
15     long long potMod(long long base, long long exp, long long mod) {
16         long long res = 1;
17         base %= mod;
18         while (exp > 0) {
19             if (exp & 1) res = res * base % mod;
20             base = base * base % mod;
21             exp >>= 1;
22         }
23         return res;
24     }
25
26     long long inversoModular(long long a) {
27         return potMod(a, MOD - 2, MOD); // Fermat, MOD debe ser
28         primo
29     }
30
31     void dfsTam(int u, int padre) {
32         tam[u] = 1;
33         for (int v : adj[u]) {
34             if (v == padre) continue;
35             dfsTam(v, u);
36             tam[u] += tam[v];
37         } // O(n)
38
39         // READ: numero de ordenes topologicos validos del arbol
40         // enraizado en 'raiz', modulo MOD.
41         long long contarOrdenes(int raiz) {
42             dfsTam(raiz, -1);
43
44             long long factorialN = 1;
45             for (int i = 2; i <= n; i++)
46                 factorialN = factorialN * i % MOD;
47
48             long long denominador = 1;
49             for (int v = 0; v < n; v++)
50                 denominador = denominador * tam[v] % MOD;
51
52             return factorialN * inversoModular(denominador) % MOD;
53         } // O(n log MOD)
54     };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           /
5 //          3
6
7 vector<vector<int>> adj(4);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 CombinatoriaArbol ca(adj, 4);
14
15 cout << ca.contarOrdenes(0);
16 // 3
17 // las 3 inserciones validas (etiquetando por orden de
18 // insercion):
19 // 0,1,2,3 -- 0,1,3,2 -- 0,2,1,3
20 // (0 siempre primero; 2 puede ir en cualquier posicion
21 // relativa a
22 // la rama 1-3, pero 3 siempre despues de 1)

```

Nota: esta fórmula asume que el orden **entre** hermanos no distingue resultados distintos (a diferencia de contar árboles **ordenados** donde la secuencia exacta de

hijos importa). Si el problema sí distingue el orden de los hijos entre sí como parte de la respuesta, hay que multiplicar además por $\prod_v (\text{cantidad de hijos de } v)!$ —ese factor extra cubre las formas de **ordenar** los hijos de cada nodo entre sí, encima de las formas de intercalarlos que ya cuenta la fórmula base.

14.6 Tree DP

Idea: Tree DP es el patrón general de DP sobre árboles con DFS post-orden: cada nodo u tiene uno o más "estados" (ej. $\text{dp}[u][0]$ = mejor respuesta **sin** usar u , $\text{dp}[u][1]$ = mejor respuesta **usando** u), y el valor de cada estado se calcula **combinando** los estados ya resueltos de los hijos —exactamente el mismo esqueleto de DFS en Árboles (post-orden, sin **visitado**, solo excluir al padre), pero acumulando una decisión óptima en vez de solo un tamaño o una profundidad.

¿Por qué usarlo? Es la generalización de todo lo visto en la sección de Trees: tamaño de subárbol, diámetro (una pasada), y Combinatoria en Árboles Ordenados son casos particulares de "Tree DP" donde el estado es simple (un solo número). El patrón completo —múltiples estados por nodo, combinados con max/min/suma según el problema— es lo que resuelve la mayoría de problemas de optimización sobre árboles (no solo conteo o medición).

Casos de uso:

- **Conjunto independiente máximo** (máxima suma de pesos de nodos, sin elegir dos adyacentes): dos estados por nodo, clásico ejemplo de esta sección.
- **Vertex cover mínimo** en árboles: variante muy similar, cambiando la condición de combinación entre estados.
- **Coloreo/asignación óptima** de nodos con restricciones locales entre padre e hijo (cada estado extra representa una opción de "color" o "categoría" del nodo).
- Cualquier problema donde la respuesta óptima de un subárbol dependa de una decisión binaria o categórica tomada en su raíz.

Complejidad: $O(n \cdot k)$, donde k es el número de estados por nodo (para el clásico de 2 estados, $O(n)$ liso) — cada nodo combina los estados ya resueltos de sus hijos con trabajo constante (o $O(k)$) por hijo.

```
1 struct ConjuntoIndependienteMax {
2     vector<vector<int>>> adj;
3     vector<int> peso;
4     vector<long long> dpSin, dpCon; // dpSin[u] = mejor sin
5     // dpCon[u] = mejor usando u
6     int n;
7
8     // CREATE: guarda el arbol y el peso de cada nodo.
9     ConjuntoIndependienteMax(vector<vector<int>>& adj_,
10    vector<int>& peso_, int n_) {
11         adj = adj_;
12         peso = peso_;
13         n = n_;
14         dpSin.assign(n, 0);
15         dpCon.assign(n, 0);
16     } // O(1)
17
18     // READ: corre el DFS post-orden que llena dpSin[]/dpCon[]
19     para
```

```
19 // todo el arbol enraizado en 'raiz'.
20 void dfs(int u, int padre) {
21     dpCon[u] = peso[u]; // usar u obliga a NO usar a sus
22     // hijos
23     dpSin[u] = 0; // no usar u: cada hijo elige
24     // libremente
25
26     for (int v : adj[u]) {
27         if (v == padre) continue;
28
29         dfs(v, u);
30
31         dpCon[u] += dpSin[v]; // si uso u, el hijo
32         // v NO puede usarse
33         dpSin[u] += max(dpSin[v], dpCon[v]); // si no uso u, el
34         // hijo v elige lo
35         // que le convenga
36     }
37 } // O(n)
38
39 // READ: respuesta final del arbol enraizado en 'raiz'.
40 long long resolver(int raiz) {
41     dfs(raiz, -1);
42     return max(dpSin[raiz], dpCon[raiz]); // O(1) tras dfs()
43 }
```

Ejemplo de funcionamiento:

```
1 // arbol: 0(peso=3)
2 //      / \
3 //     1(peso=5) 2(peso=4)
4 //      |
5 //     3(peso=6)
6
7 vector<vector<int>>> adj(4);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 vector<int> peso = {3, 5, 4, 6};
14
15 ConjuntoIndependienteMax cim(adj, peso, 4);
16
17 cout << cim.resolver(0);
18 // 10
19 // mejor conjunto: {2, 3} (pesos 4+6=10), no adyacentes entre
20 // sí
```

Regla práctica: cualquier problema de árboles que pida "óptimo global sujeto a una restricción entre padre e hijos" es Tree DP: define qué información necesitas de cada hijo (usualmente 1-2 estados), escribe cómo se combina en el nodo actual, y el DFS post-orden ya está —es el mismo recorrido que DFS en Árboles, solo con más de un valor acumulándose por nodo.

14.7 Rerooting

Idea: Tree DP normal (ver Tree DP) calcula, para una raíz fija, la respuesta de cada subárbol en una sola pasada post-orden. Rerooting extiende eso a "calcular la respuesta como si **cada** nodo fuera la raíz", sin pagar $O(n)$ por cada raíz (lo cual daría $O(n^2)$ total). Se logra con **dos** DFS: (1) un post-orden normal que calcula $\text{down}[u]$ = la respuesta del subárbol de u asumiendo la raíz original; (2) un pre-orden que propaga hacia abajo $\text{up}[u]$ = "la contribución de todo lo que **no** está en el subárbol de u " (el resto del árbol, visto desde el padre de u) — combinando down y up en cada nodo se obtiene la respuesta con ese nodo como raíz, en tiempo total $O(n)$.

¿Por qué usarlo? Cualquier problema que pida "calcula X para cada nodo, donde X depende de **todo** el árbol (no solo del subárbol)" —si X solo dependiera del subárbol, un DFS post-orden ya bastaría. La señal típica es el enunciado diciendo "para cada nodo, si fuera la raíz..." o "la respuesta considerando el árbol completo, no solo hacia abajo".

Casos de uso:

- Suma de distancias desde cada nodo a **todos** los demás nodos del árbol (el ejemplo clásico, abajo).
- Altura/profundidad máxima del árbol si se enraizara en cada nodo distinto.
- Tamaño "visto hacia afuera": cuántos nodos quedarían fuera del subárbol de u si u no fuera la raíz.
- Cualquier variante de Tree DP (conjunto independiente, diámetro, etc.) donde se pide la respuesta **por cada posible raíz** en vez de para una raíz fija.

Complejidad: $O(n)$ — dos DFS completos, cada uno $O(n)$ (a diferencia de recalcular Tree DP n veces, que sería $O(n^2)$).

```

1 struct Rerooting {
2     vector<vector<int>>> adj;
3     vector<long long> tam, down, up, respuesta;
4     int n;
5
6     // CREATE: guarda el árbol.
7     Rerooting(vector<vector<int>>& adj_, int n_) {
8         adj = adj_;
9         n = n_;
10        tam.assign(n, 0);
11        down.assign(n, 0);
12        up.assign(n, 0);
13        respuesta.assign(n, 0);
14    }
15
16    // PASADA 1 (post-orden): down[u] = suma de distancias desde
17    // u hacia TODO su subarbol, asumiendo raíz = 0. tam[u] =
18    // tamaño del subarbol de u.
19    void dfs1(int u, int padre) {
20        tam[u] = 1;
21        down[u] = 0;
22
23        for (int v : adj[u]) {
24            if (v == padre) continue;
25            dfs1(v, u);
26            tam[u] += tam[v];
27            down[u] += down[v] + tam[v]; // cada nodo del
28            // subarbol de v queda 1
29            // arista mas lejos de u
30            // que de v
31        }
32    } // O(n)
33
34    // PASADA 2 (pre-orden): respuesta[u] = suma de distancias
35    // desde u a TODOS los nodos del árbol completo. up[u] =
36    // contribucion
37    // de "todo lo que no es el subarbol de u", vista desde u.
38    void dfs2(int u, int padre) {
39        respuesta[u] = down[u] + up[u];
40
41        for (int v : adj[u]) {
42            if (v == padre) continue;
43
44            // al mover la raíz de u a v: todo lo que estaba
45            FUERA // del subarbol de v queda 1 arista MAS lejos; lo que
46            // ya // estaba en el subarbol de v queda 1 arista MAS
47            // cerca -- // por eso no se puede reusar 'down[v]' directamente,
48            // hay // que restar su propia contribucion antes de
49            // invertir // el signo:
50            up[v] = (respuesta[u] - (down[v] + tam[v]))
51                + (n - tam[v]);
52
53            dfs2(v, u);
54        }
55    } // O(n)
56
57    // READ: respuesta[u] para cada u, tras correr ambas pasadas

```

```

56 // desde cualquier raíz inicial (0 por defecto).
57 vector<long long> resolver() {
58     dfs1(0, -1);
59     up[0] = 0;
60     dfs2(0, -1);
61     return respuesta;
62 } // O(n)
63
64 //MODIFICACIONES Y COMENTARIOS CON LOS USOS
65
66 // 1. Suma de distancias a todos los nodos (el caso ya
67 // implementado arriba)
68 // down[u] = suma de distancias DENTRO del subarbol de u
69 // up[u] = suma de distancias hacia todo lo que esta AFUERA
70 // respuesta[u] = down[u] + up[u]
71
72 // 2. Tamano "hacia afuera" (cuantos nodos quedarian fuera del
73 // subarbol de u si u no fuera la raíz)
74 // tamAfuera[v] = n - tam[v];
75 // -- no necesita ni siquiera up[], es directo desde tam[] de
76 // dfs1
77
78 // 3. Altura maxima del árbol enraizado en cada nodo
79 // (combinacion NO invertible: max no tiene "resta" como la suma,
80 // hace falta guardar el MEJOR y el SEGUNDO MEJOR hijo)
81 // vector<long long> downMax1, downMax2, deQuienEsMax1;
82 // void dfs1_altura(int u, int padre) {
83 //     downMax1[u] = downMax2[u] = 0;
84 //     for (int v : adj[u]) {
85 //         if (v == padre) continue;
86 //         dfs1_altura(v, u);
87 //         long long h = downMax1[v] + 1;
88 //         if (h > downMax1[u]) {
89 //             downMax2[u] = downMax1[u];
90 //             downMax1[u] = h;
91 //             deQuienEsMax1[u] = v;
92 //         } else if (h > downMax2[u]) {
93 //             downMax2[u] = h;
94 //         }
95 //     }
96 // }
97 // void dfs2_altura(int u, int padre) {
98 //     for (int v : adj[u]) {
99 //         if (v == padre) continue;
100 //         // si v es quien da la altura maxima de u, hay que
101 //         // pasarle la SEGUNDA maxima (para no contarse a si
102 //         // mismo)
103 //         long long mejorSinV = (deQuienEsMax1[u] == v)
104 //             ? downMax2[v] : downMax1[v];
105 //         up[v] = max(up[u], mejorSinV) + 1;
106 //         dfs2_altura(v, u);
107 //     }
108 // }
109 // 4. Diametro del árbol usando rerooting (alternativa a Diametro
110 // de un Arbol con dos BFS)
111 // respuesta[u] = downMax1[u] combinado con up[u] de la variante
112 // 3;
113 // diametroGlobal = *max_element(respuesta.begin(), respuesta.end())
114 // -- mismo costo O(n), util si ya se necesita la respuesta POR
115 // NODO
116 // y no solo el numero global
117
118 // 5. Conteo de "nodos alcanzables en <= k pasos desde cada nodo"
119 // (variante de down/up acumulando CONTEOS por profundidad, no un
120 // solo numero)
121 // vector<vector<long long>> down(n), up(n); // down[u][d], up[u][d]
122 // -- mismo esqueleto de dos pasadas; down[u] y up[u] pasan a ser
123 // arreglos indexados por distancia d en vez de escalares
124
125 // 6. Regla general para saber si hace falta "top-2" o basta un
126 // solo valor:
127 // - operacion INVERTIBLE (suma, XOR, producto sin ceros)
128 // -> un solo down[u] basta, up[v] se obtiene RESTANDO la
129 // contribucion de v (como en el caso 1)
130 // - operacion NO invertible (max, min, gcd)
131 // -> hace falta guardar el MEJOR y el SEGUNDO MEJOR para poder
132 // "quitar" la rama de v sin perder informacion (como en el
133 // caso 3)

```

Ejemplo de funcionamiento:

```

1 // árbol:      0
2 //            / \
3 //           1  2
4 //          /
5 //         3

```

```

6
7 vector<vector<int>> adj(4);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 Rerooting rr(adj, 4);
14 auto resp = rr.resolver();
15
16 cout << resp[0];
17 // 4 (dist a 1=1, a 2=1, a 3=2 -> 1+1+2=4)

```

15 Lowest Common Ancestor (LCA)

15.1 Binary Lifting

Idea: Binary Lifting precomputa, para cada nodo u , quién es su ancestro a 2^k pasos hacia arriba, para todo k de 0 a $\log n$ ($up[k][u]$). Con eso, "subir d pasos desde u " se descompone en la representación binaria de d (igual que exponenciación rápida, pero saltando por el árbol en vez de multiplicando). El LCA de u y v se calcula en dos fases: (1) nivelar al más profundo de los dos hasta que ambos queden a la misma profundidad, usando esos saltos binarios; (2) si no son ya el mismo nodo, subir a ambos **simultáneamente** con los saltos más grandes posibles **que no los junten todavía** —al terminar, ambos quedan justo un paso debajo del LCA, que es simplemente su padre común.

¿Por qué usarlo? Es la alternativa a LCA vía Euler Tour + RMQ (ver la sección de RMQ) cuando el árbol **no** es completamente estático o cuando de todas formas ya necesitas "ancestro a k pasos de distancia" como operación aparte (útil por sí sola en muchos problemas, no solo para LCA). La query de LCA es $O(\log n)$ en vez de $O(1)$ como con RMQ, pero Binary Lifting es más simple de razonar/depurar (sin Euler Tour ni Sparse Table por separado) y generaliza directo a "ancestro a k pasos" y "distancia entre dos nodos" con el mismo precómputo.

Casos de uso:

- LCA de dos nodos, con muchas queries sobre el mismo árbol fijo.
- Ancestro de u a exactamente k pasos hacia arriba —operación independiente del LCA, útil en problemas de "salta k generaciones".
- Distancia entre dos nodos de un árbol: $profundidad[u] + profundidad[v] - 2 \cdot profundidad[lca(u, v)]$, una vez que ya tienes LCA y profundidades.
- Verificar si u es ancestro de v : comparando si $lca(u, v) == u$.

Complejidad: $O(n \log n)$ de precómputo (build), $O(\log n)$ por query de LCA o de "ancestro a k pasos".

```

1 struct BinaryLifting {
2     vector<vector<int>> adj;
3     vector<vector<int>> up; // up[k][u] = ancestro de u a 2^k
4     // k pasos
5     vector<int> profundidad;
6     int n, LOG;
7
8     // CREATE: construye el árbol (DFS para profundidad y padre
9     // directo) y la tabla de saltos binarios.
10    BinaryLifting(vector<vector<int>>& adj_, int raiz, int n_) {
11        adj = adj_;
12        n = n_;

```

```

12    LOG = 1;
13    while ((1 << LOG) < n) LOG++;
14    LOG++;
15
16    up.assign(LOG, vector<int>(n, -1));
17    profundidad.assign(n, 0);
18
19    dfs(raiz, -1, 0);
20    build();
21 } // O(n log n)
22
23 void dfs(int u, int padre, int d) {
24     up[0][u] = padre;
25     profundidad[u] = d;
26
27     for (int v : adj[u])
28         if (v != padre)
29             dfs(v, u, d + 1);
30 }
31
32 void build() {
33     for (int k = 1; k < LOG; k++)
34         for (int u = 0; u < n; u++)
35             if (up[k-1][u] != -1)
36                 up[k][u] = up[k-1][up[k-1][u]];
37             // si up[k-1][u] es -1 (se salió de la raíz), up[
38             // se queda en -1 por el assign inicial
39 }
40
41 // READ: ancestro de u a exactamente k pasos hacia arriba, -1
42 // si se sale del árbol antes de completar los k pasos.
43 int kEsimAAncestro(int u, int k) {
44     for (int i = 0; i < LOG && u != -1; i++)
45         if (k & (1 << i))
46             u = up[i][u];
47     return u;
48 } // O(log n)
49
50 // READ: LCA de u y v.
51 int query(int u, int v) {
52     if (profundidad[u] < profundidad[v]) swap(u, v);
53
54     // fase 1: nivela u a la profundidad de v
55     int diff = profundidad[u] - profundidad[v];
56     u = kEsimAAncestro(u, diff);
57
58     if (u == v) return u;
59
60     // fase 2: sube a ambos con los saltos mas grandes que NO
61     // los junten todavía
62     for (int k = LOG - 1; k >= 0; k--) {
63         if (up[k][u] != up[k][v]) {
64             u = up[k][u];
65             v = up[k][v];
66         }
67     }
68
69     return up[0][u]; // ambos quedaron un paso debajo del
70     // LCA
71 } // O(log n)

```

Ejemplo de funcionamiento:

```

1 // árbol:
2 //      0
3 //     / \
4 //    1   2
5 //   / \
6 //  3   4
7
8 vector<vector<int>> adj(5);
9 adj[0] = {1, 2};
10 adj[1] = {0, 3, 4};
11 adj[2] = {0};
12 adj[3] = {1};
13 adj[4] = {1};
14
15 BinaryLifting bl(adj, 0, 5);
16
17 cout << bl.query(3, 4);
18 // 1 (padre comun de 3 y 4)
19
20 cout << bl.kEsimAAncestro(4, 2);
21 // 0 (4 -> 1 -> 0, dos pasos hacia arriba)
22
23 cout << bl.query(3, 2);
24 // 0 (raiz)

```

Regla práctica: si el árbol es completamente estático y solo necesitas LCA puro, Euler Tour + RMQ (sección

de RMQ) da queries $O(1)$ en vez de $O(\log n)$ y suele ser la opción "óptima" en papel. Usa Binary Lifting cuando además del LCA necesitas **saltos por ancestro** como operación de primera clase, cuando prefieres evitar manejar Euler Tour + Sparse Table por separado, o cuando el árbol tiene actualizaciones que hacen más práctico recalcular solo la tabla de saltos que reconstruir un Euler Tour completo.

15.2 Euler Tour + RMQ

Idea: esta es la misma técnica ya vista en la sección de RMQ (revisar ahí el detalle completo) — se repite aquí como **la otra opción estándar de LCA**, para tenerla junto a Binary Lifting: recorres el árbol con un Euler Tour, anotando el nodo visitado cada vez que **entras o vuelves** a él (arreglo de tamaño $2n - 1$) junto con su profundidad, y el LCA de u y v es el nodo de **menor profundidad** entre la primera aparición de u y la primera aparición de v en ese arreglo —un RMQ (por profundidad) resuelto con Sparse Table, ya que el árbol es estático.

¿Por qué usarlo? Da queries de LCA en $O(1)$ real (contra $O(\log n)$ de Binary Lifting), a cambio de un precómputo un poco más elaborado (Euler Tour + Sparse Table en vez de solo la tabla de saltos). Es la opción preferible cuando el árbol es completamente **estático** y vas a hacer **muchísimas** queries de LCA —la diferencia de $O(1)$ vs $O(\log n)$ por query se nota cuando hay 10^5 – 10^6 consultas.

Casos de uso:

- Mismos que Binary Lifting: LCA de dos nodos con muchas queries sobre un árbol fijo.
- Cuando además necesitas resolver otros RMQ sobre el mismo Euler Tour (algunos problemas piden más de una cosa sobre el mismo recorrido).
- Preferible sobre Binary Lifting específicamente cuando el cuello de botella del problema es el número de queries, no el precómputo.

Complejidad: $O(n \log n)$ de precómputo (Euler Tour + build de Sparse Table), $O(1)$ por query de LCA.

```
1 struct EulerTourLCA {
2     vector<int> euler, primeraApar, profundidad;
3     vector<vector<int>> sp; // sp[k][i] = nodo con menor
4     // profundidad
5     vector<int> lg;
6
7     // CREATE: construye el Euler Tour del arbol (arr. de tamaño
8     // 2n-1) y la Sparse Table sobre profundidad, para RMQ O(1).
9     EulerTourLCA(vector<vector<int>>& adj, int raiz, int n) {
10         primeraApar.assign(n, -1);
11         profundidad.assign(n, 0);
12         euler.reserve(2 * n);
13
14         dfs(adj, raiz, -1, 0);
15         build(); // O(n log n)
16
17         void dfs(vector<vector<int>>& adj, int u, int p, int d) {
18             primeraApar[u] = euler.size();
19             profundidad[u] = d;
20             euler.push_back(u);
21
22             for (int v : adj[u]) {
23                 if (v != p) {
24                     dfs(adj, v, u, d + 1);
25                     euler.push_back(u);
26                 }
27             }
28         }
29     }
30
31     void build() {
32         int m = euler.size();
33         lg.assign(m + 1, 0);
34         for (int i = 2; i <= m; i++)
35             lg[i] = lg[i / 2] + 1;
36
37         int K = lg[m] + 1;
38         sp.assign(K, vector<int>(m));
39         sp[0] = euler;
40
41         for (int k = 1; k < K; k++)
42             for (int i = 0; i + (1 << k) <= m; i++) {
43                 int l = sp[k - 1][i];
44                 int r = sp[k - 1][i + (1 << (k - 1))];
45                 sp[k][i] = profundidad[l] < profundidad[r] ? l :
46                     r;
47             }
48
49         // READ: LCA de u y v.
50         int query(int u, int v) {
51             int l = primeraApar[u], r = primeraApar[v];
52             if (l > r) swap(l, r);
53
54             int k = lg[r - l + 1];
55             int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
56             return profundidad[a] < profundidad[b] ? a : b;
57         }
58     };
59 }
```

```
28 }
29
30 void build() {
31     int m = euler.size();
32     lg.assign(m + 1, 0);
33     for (int i = 2; i <= m; i++)
34         lg[i] = lg[i / 2] + 1;
35
36     int K = lg[m] + 1;
37     sp.assign(K, vector<int>(m));
38     sp[0] = euler;
39
40     for (int k = 1; k < K; k++)
41         for (int i = 0; i + (1 << k) <= m; i++) {
42             int l = sp[k - 1][i];
43             int r = sp[k - 1][i + (1 << (k - 1))];
44             sp[k][i] = profundidad[l] < profundidad[r] ? l :
45                 r;
46         }
47
48     // READ: LCA de u y v.
49     int query(int u, int v) {
50         int l = primeraApar[u], r = primeraApar[v];
51         if (l > r) swap(l, r);
52
53         int k = lg[r - l + 1];
54         int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
55         return profundidad[a] < profundidad[b] ? a : b;
56     }
57 }
```

Ejemplo de funcionamiento:

```
1 // arbol:
2 //      0
3 //     / \
4 //    1   2
5 //   / \
6 //  3   4
7
8 vector<vector<int>> adj(5);
9 adj[0] = {1, 2};
10 adj[1] = {0, 3, 4};
11 adj[2] = {0};
12 adj[3] = {1};
13 adj[4] = {1};
14
15 EulerTourLCA et(adj, 0, 5);
16
17 cout << et.query(3, 4);
18 // 1 (padre comun de 3 y 4)
19
20 cout << et.query(1, 4);
21 // 1 (uno es ancestro del otro)
```

Regla práctica: misma decisión que en la sección de RMQ — Euler Tour + RMQ para $O(1)$ por query cuando el árbol es estático; Binary Lifting cuando prefieres simplicidad de implementación, ya necesitas saltos por ancestro, o el árbol puede requerir recalcular partes del precómputo con más frecuencia.

15.3 Distancia entre Dos Nodos

Idea: la distancia (en número de aristas) entre dos nodos u y v de un árbol se obtiene combinando **profundidad** y **LCA** con una fórmula directa:

$$\text{dist}(u, v) = \text{profundidad}[u] + \text{profundidad}[v] - 2 \cdot \text{profundidad}[\text{lca}]$$

La intuición: el camino de u a v sube desde u hasta el LCA ($\text{profundidad}[u] - \text{profundidad}[\text{lca}]$ pasos) y luego baja del LCA hasta v ($\text{profundidad}[v] - \text{profundidad}[\text{lca}]$ pasos) — sumando ambos tramos se llega exactamente a la fórmula de arriba.

¿Por qué usarlo? Es la razón principal por la que "resolver LCA" casi siempre viene empaquetado junto con "profundidad de cada nodo" — una vez que tienes ambas piezas (de Binary Lifting o de Euler Tour + RMQ, cualquiera sirve, ya que ambas calculan profundidad como parte del precómputo), la distancia es una operación $O(1)$ extra sin precómputo adicional.

Casos de uso:

- Distancia mínima entre dos nodos cualesquiera de un árbol, con muchas queries.
- Verificar si un nodo w está en el camino entre u y v : w está en el camino si y solo si $\text{dist}(u, w) + \text{dist}(w, v) == \text{dist}(u, v)$.
- Como building block de problemas de "suma de distancias entre pares de queries" sobre un árbol fijo.
- Verificar si u es ancestro de v sin comparar directamente el LCA: $\text{dist}(u, v) == \text{profundidad}[v] - \text{profundidad}[u]$ (solo válido si además se sabe $\text{profundidad}[u] \leq \text{profundidad}[v]$).

Complejidad: $O(1)$ por query **una vez calculado el LCA** — el costo real de cada consulta de distancia es el mismo que el de la query de LCA subyacente ($O(1)$ con Euler Tour + RMQ, $O(\log n)$ con Binary Lifting).

```

1 struct DistanciaArbol {
2     BinaryLifting& bl; // reutiliza cualquier estructura de LCA
3     // ya expone profundidad[] y query(u, v)
4     // (funciona igual con EulerTourLCA)
5
6     // CREATE: envuelve una estructura de LCA ya construida.
7     DistanciaArbol(BinaryLifting& bl_) : bl(bl_) {} // O(1)
8
9     // READ: distancia (en aristas) entre u y v.
10    int distancia(int u, int v) {
11        int ancestro = bl.query(u, v);
12        return bl.profundidad[u] + bl.profundidad[v]
13            - 2 * bl.profundidad[ancestro];
14    } // O(1) o O(log n)
15    // segun la
16    // de LCA usada
17
18    // READ: w esta en el camino simple entre u y v?
19    bool estaEnCamino(int u, int v, int w) {
20        return distancia(u, w) + distancia(w, v) == distancia(u,
21        v); // igual costo que
22        // 3 llamadas a
23        // distancia()
24    };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1   2
4 //           / \
5 //          3   4
6
7 vector<vector<int>>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 BinaryLifting bl(adj, 0, 5);
15 DistanciaArbol da(bl);
16
17 cout << da.distancia(3, 4);
18 // 2   (3 -> 1 -> 4)
19
20 cout << da.distancia(3, 2);
21 // 3   (3 -> 1 -> 0 -> 2)
22
23 cout << da.estaEnCamino(3, 2, 0);
24 // true  (0 esta en el camino de 3 a 2, es el LCA)
25
26 cout << da.estaEnCamino(3, 2, 4);
27 // false (4 no esta en el camino de 3 a 2)

```

Regla práctica: nunca hace falta un precómputo aparte para distancias en árboles — si ya resolviste LCA

(por cualquiera de los dos métodos de esta sección), la profundidad de cada nodo ya está calculada como efecto secundario, así que "distancia entre dos nodos" siempre es la misma fórmula de tres términos encima de lo que ya tienes, nunca una estructura nueva.

15.4 k -ésimo Ancestro

Idea: encontrar el ancestro de un nodo u a **exactamente** k pasos hacia arriba es, en realidad, una operación que **ya está resuelta** dentro de Binary Lifting —no es una estructura nueva, es la función `kEsimoAncestro` que ya se construyó como parte del precómputo de LCA. Se separa aquí como su propia subsección porque muchos problemas la piden **sola**, sin necesitar LCA en absoluto: la idea es descomponer k en binario y saltar usando la tabla `up[k][u]` —el mismo principio de exponenciación rápida, aplicado a "saltar por el árbol" en vez de "multiplicar".

¿Por qué usarlo? Es la respuesta directa a cualquier enunciado que pregunte "¿quién es el ancestro de u a distancia k ?" o variantes equivalentes ("el k -ésimo padre", "sube k niveles desde u "). Sin esta técnica, la alternativa ingenua es subir de uno en uno ($O(k)$ por query, hasta $O(n)$ en el peor caso) — con la tabla de saltos binarios, cada query es $O(\log n)$ sin importar qué tan grande sea k .

Casos de uso:

- Encontrar el ancestro de u a exactamente k pasos, con muchas queries sobre el mismo árbol.
- Como subrutina de LCA (fase de "nivelar" al nodo más profundo antes de subir en paralelo — ver Binary Lifting).
- Encontrar el **hijo** de un ancestro dado en el camino hacia u : si necesitas saber "¿por cuál hijo de w pasa el camino hacia u ?", primero subes u a $\text{profundidad}[u] - \text{profundidad}[w] - 1$ pasos — ese nodo es exactamente ese hijo.
- Simular "saltos de k generaciones" en estructuras tipo árbol genealógico o herencia de propiedades.

Complejidad: $O(n \log n)$ de precómputo (compartido con Binary Lifting si ya se construyó para LCA — no hay costo extra), $O(\log n)$ por query.

```

1 struct KEsimoAncestro {
2     vector<vector<int>>> adj;
3     vector<vector<int>>> up; // up[k][u] = ancestro de u a 2^k
4     // pasos
5     vector<int> profundidad;
6     int n, LOG;
7
8     // CREATE: construye el arbol (DFS para profundidad y padre
9     // directo) y la tabla de saltos binarios. Idéntico al
10    // precómputo de Binary Lifting -- si ya tienes esa
11    // estructura
12    // construida para LCA, reutilízala en vez de duplicar esto.
13    KEsimoAncestro(vector<vector<int>>>& adj_, int raiz, int n_) {
14        adj = adj_;
15        n = n_;
16        LOG = 1;
17        while ((1 << LOG) < n) LOG++;
18        LOG++;
19
20        up.assign(LOG, vector<int>(n, -1));
21        profundidad.assign(n, 0);
22
23        dfs(raiz, -1, 0);
24        build();
25    }
26
27    void dfs(int u, int p, int d) {
28        profundidad[u] = d;
29        for (int v : adj[u]) {
30            if (v == p) continue;
31            dfs(v, u, d + 1);
32        }
33    }
34
35    void build() {
36        for (int k = 1; k < LOG; k++) {
37            for (int u = 0; u < n; u++) {
38                int p = up[k-1][u];
39                if (p != -1) up[k][u] = up[k-1][p];
40            }
41        }
42    }
43
44    int kEsimoAncestro(int u, int k) const {
45        if (k < 0 || k >= LOG) return -1;
46        for (int i = LOG - 1; i >= 0; i--) {
47            if (up[i][u] != -1) u = up[i][u];
48        }
49        return u;
50    }
51
52    int hijo(int u, int k) const {
53        if (k < 0 || k >= LOG) return -1;
54        int p = kEsimoAncestro(u, k);
55        if (p == -1) return -1;
56        for (int v : adj[p]) {
57            if (v == u) return -1;
58            return v;
59        }
60        return -1;
61    }
62
63    int profundidad(int u) const {
64        return profundidad[u];
65    }
66
67    bool estaEnCamino(int u, int v, int w) const {
68        return distancia(u, w) + distancia(w, v) == distancia(u,
69        v);
70    }
71
72    int distancia(int u, int v) const {
73        int lca = query(u, v);
74        return profundidad[u] + profundidad[v] - 2 * profundidad[lca];
75    }
76
77    int query(int u, int v) const {
78        if (profundidad[u] < profundidad[v]) swap(u, v);
79        int w = u;
80        for (int i = LOG - 1; i >= 0; i--) {
81            if (profundidad[up[i][w]] < profundidad[v]) w = up[i][w];
82        }
83        return w;
84    }
85
86    int lca(int u, int v) const {
87        return query(u, v);
88    }
89
90    int ancestro(int u, int k) const {
91        return kEsimoAncestro(u, k);
92    }
93
94    int hijo(int u, int k) const {
95        return hijo(u, k);
96    }
97
98    int profundidad(int u) const {
99        return profundidad[u];
100    }
101
102    bool estaEnCamino(int u, int v, int w) const {
103        return estaEnCamino(u, v, w);
104    }
105
106    int distancia(int u, int v) const {
107        return distancia(u, v);
108    }
109
110    int query(int u, int v) const {
111        return query(u, v);
112    }
113
114    int lca(int u, int v) const {
115        return lca(u, v);
116    }
117
118    int ancestro(int u, int k) const {
119        return ancestro(u, k);
120    }
121
122    int hijo(int u, int k) const {
123        return hijo(u, k);
124    }
125
126    int profundidad(int u) const {
127        return profundidad[u];
128    }
129
130    bool estaEnCamino(int u, int v, int w) const {
131        return estaEnCamino(u, v, w);
132    }
133
134    int distancia(int u, int v) const {
135        return distancia(u, v);
136    }
137
138    int query(int u, int v) const {
139        return query(u, v);
140    }
141
142    int lca(int u, int v) const {
143        return lca(u, v);
144    }
145
146    int ancestro(int u, int k) const {
147        return ancestro(u, k);
148    }
149
150    int hijo(int u, int k) const {
151        return hijo(u, k);
152    }
153
154    int profundidad(int u) const {
155        return profundidad[u];
156    }
157
158    bool estaEnCamino(int u, int v, int w) const {
159        return estaEnCamino(u, v, w);
160    }
161
162    int distancia(int u, int v) const {
163        return distancia(u, v);
164    }
165
166    int query(int u, int v) const {
167        return query(u, v);
168    }
169
170    int lca(int u, int v) const {
171        return lca(u, v);
172    }
173
174    int ancestro(int u, int k) const {
175        return ancestro(u, k);
176    }
177
178    int hijo(int u, int k) const {
179        return hijo(u, k);
180    }
181
182    int profundidad(int u) const {
183        return profundidad[u];
184    }
185
186    bool estaEnCamino(int u, int v, int w) const {
187        return estaEnCamino(u, v, w);
188    }
189
190    int distancia(int u, int v) const {
191        return distancia(u, v);
192    }
193
194    int query(int u, int v) const {
195        return query(u, v);
196    }
197
198    int lca(int u, int v) const {
199        return lca(u, v);
200    }
201
202    int ancestro(int u, int k) const {
203        return ancestro(u, k);
204    }
205
206    int hijo(int u, int k) const {
207        return hijo(u, k);
208    }
209
210    int profundidad(int u) const {
211        return profundidad[u];
212    }
213
214    bool estaEnCamino(int u, int v, int w) const {
215        return estaEnCamino(u, v, w);
216    }
217
218    int distancia(int u, int v) const {
219        return distancia(u, v);
220    }
221
222    int query(int u, int v) const {
223        return query(u, v);
224    }
225
226    int lca(int u, int v) const {
227        return lca(u, v);
228    }
229
230    int ancestro(int u, int k) const {
231        return ancestro(u, k);
232    }
233
234    int hijo(int u, int k) const {
235        return hijo(u, k);
236    }
237
238    int profundidad(int u) const {
239        return profundidad[u];
240    }
241
242    bool estaEnCamino(int u, int v, int w) const {
243        return estaEnCamino(u, v, w);
244    }
245
246    int distancia(int u, int v) const {
247        return distancia(u, v);
248    }
249
250    int query(int u, int v) const {
251        return query(u, v);
252    }
253
254    int lca(int u, int v) const {
255        return lca(u, v);
256    }
257
258    int ancestro(int u, int k) const {
259        return ancestro(u, k);
260    }
261
262    int hijo(int u, int k) const {
263        return hijo(u, k);
264    }
265
266    int profundidad(int u) const {
267        return profundidad[u];
268    }
269
270    bool estaEnCamino(int u, int v, int w) const {
271        return estaEnCamino(u, v, w);
272    }
273
274    int distancia(int u, int v) const {
275        return distancia(u, v);
276    }
277
278    int query(int u, int v) const {
279        return query(u, v);
280    }
281
282    int lca(int u, int v) const {
283        return lca(u, v);
284    }
285
286    int ancestro(int u, int k) const {
287        return ancestro(u, k);
288    }
289
290    int hijo(int u, int k) const {
291        return hijo(u, k);
292    }
293
294    int profundidad(int u) const {
295        return profundidad[u];
296    }
297
298    bool estaEnCamino(int u, int v, int w) const {
299        return estaEnCamino(u, v, w);
300    }
301
302    int distancia(int u, int v) const {
303        return distancia(u, v);
304    }
305
306    int query(int u, int v) const {
307        return query(u, v);
308    }
309
310    int lca(int u, int v) const {
311        return lca(u, v);
312    }
313
314    int ancestro(int u, int k) const {
315        return ancestro(u, k);
316    }
317
318    int hijo(int u, int k) const {
319        return hijo(u, k);
320    }
321
322    int profundidad(int u) const {
323        return profundidad[u];
324    }
325
326    bool estaEnCamino(int u, int v, int w) const {
327        return estaEnCamino(u, v, w);
328    }
329
330    int distancia(int u, int v) const {
331        return distancia(u, v);
332    }
333
334    int query(int u, int v) const {
335        return query(u, v);
336    }
337
338    int lca(int u, int v) const {
339        return lca(u, v);
340    }
341
342    int ancestro(int u, int k) const {
343        return ancestro(u, k);
344    }
345
346    int hijo(int u, int k) const {
347        return hijo(u, k);
348    }
349
350    int profundidad(int u) const {
351        return profundidad[u];
352    }
353
354    bool estaEnCamino(int u, int v, int w) const {
355        return estaEnCamino(u, v, w);
356    }
357
358    int distancia(int u, int v) const {
359        return distancia(u, v);
360    }
361
362    int query(int u, int v) const {
363        return query(u, v);
364    }
365
366    int lca(int u, int v) const {
367        return lca(u, v);
368    }
369
370    int ancestro(int u, int k) const {
371        return ancestro(u, k);
372    }
373
374    int hijo(int u, int k) const {
375        return hijo(u, k);
376    }
377
378    int profundidad(int u) const {
379        return profundidad[u];
380    }
381
382    bool estaEnCamino(int u, int v, int w) const {
383        return estaEnCamino(u, v, w);
384    }
385
386    int distancia(int u, int v) const {
387        return distancia(u, v);
388    }
389
390    int query(int u, int v) const {
391        return query(u, v);
392    }
393
394    int lca(int u, int v) const {
395        return lca(u, v);
396    }
397
398    int ancestro(int u, int k) const {
399        return ancestro(u, k);
400    }
401
402    int hijo(int u, int k) const {
403        return hijo(u, k);
404    }
405
406    int profundidad(int u) const {
407        return profundidad[u];
408    }
409
410    bool estaEnCamino(int u, int v, int w) const {
411        return estaEnCamino(u, v, w);
412    }
413
414    int distancia(int u, int v) const {
415        return distancia(u, v);
416    }
417
418    int query(int u, int v) const {
419        return query(u, v);
420    }
421
422    int lca(int u, int v) const {
423        return lca(u, v);
424    }
425
426    int ancestro(int u, int k) const {
427        return ancestro(u, k);
428    }
429
430    int hijo(int u, int k) const {
431        return hijo(u, k);
432    }
433
434    int profundidad(int u) const {
435        return profundidad[u];
436    }
437
438    bool estaEnCamino(int u, int v, int w) const {
439        return estaEnCamino(u, v, w);
440    }
441
442    int distancia(int u, int v) const {
443        return distancia(u, v);
444    }
445
446    int query(int u, int v) const {
447        return query(u, v);
448    }
449
450    int lca(int u, int v) const {
451        return lca(u, v);
452    }
453
454    int ancestro(int u, int k) const {
455        return ancestro(u, k);
456    }
457
458    int hijo(int u, int k) const {
459        return hijo(u, k);
460    }
461
462    int profundidad(int u) const {
463        return profundidad[u];
464    }
465
466    bool estaEnCamino(int u, int v, int w) const {
467        return estaEnCamino(u, v, w);
468    }
469
470    int distancia(int u, int v) const {
471        return distancia(u, v);
472    }
473
474    int query(int u, int v) const {
475        return query(u, v);
476    }
477
478    int lca(int u, int v) const {
479        return lca(u, v);
480    }
481
482    int ancestro(int u, int k) const {
483        return ancestro(u, k);
484    }
485
486    int hijo(int u, int k) const {
487        return hijo(u, k);
488    }
489
490    int profundidad(int u) const {
491        return profundidad[u];
492    }
493
494    bool estaEnCamino(int u, int v, int w) const {
495        return estaEnCamino(u, v, w);
496    }
497
498    int distancia(int u, int v) const {
499        return distancia(u, v);
500    }
501
502    int query(int u, int v) const {
503        return query(u, v);
504    }
505
506    int lca(int u, int v) const {
507        return lca(u, v);
508    }
509
510    int ancestro(int u, int k) const {
511        return ancestro(u, k);
512    }
513
514    int hijo(int u, int k) const {
515        return hijo(u, k);
516    }
517
518    int profundidad(int u) const {
519        return profundidad[u];
520    }
521
522    bool estaEnCamino(int u, int v, int w) const {
523        return estaEnCamino(u, v, w);
524    }
525
526    int distancia(int u, int v) const {
527        return distancia(u, v);
528    }
529
530    int query(int u, int v) const {
531        return query(u, v);
532    }
533
534    int lca(int u, int v) const {
535        return lca(u, v);
536    }
537
538    int ancestro(int u, int k) const {
539        return ancestro(u, k);
540    }
541
542    int hijo(int u, int k) const {
543        return hijo(u, k);
544    }
545
546    int profundidad(int u) const {
547        return profundidad[u];
548    }
549
550    bool estaEnCamino(int u, int v, int w) const {
551        return estaEnCamino(u, v, w);
552    }
553
554    int distancia(int u, int v) const {
555        return distancia(u, v);
556    }
557
558    int query(int u, int v) const {
559        return query(u, v);
560    }
561
562    int lca(int u, int v) const {
563        return lca(u, v);
564    }
565
566    int ancestro(int u, int k) const {
567        return ancestro(u, k);
568    }
569
570    int hijo(int u, int k) const {
571        return hijo(u, k);
572    }
573
574    int profundidad(int u) const {
575        return profundidad[u];
576    }
577
578    bool estaEnCamino(int u, int v, int w) const {
579        return estaEnCamino(u, v, w);
580    }
581
582    int distancia(int u, int v) const {
583        return distancia(u, v);
584    }
585
586    int query(int u, int v) const {
587        return query(u, v);
588    }
589
590    int lca(int u, int v) const {
591        return lca(u, v);
592    }
593
594    int ancestro(int u, int k) const {
595        return ancestro(u, k);
596    }
597
598    int hijo(int u, int k) const {
599        return hijo(u, k);
600    }
601
602    int profundidad(int u) const {
603        return profundidad[u];
604    }
605
606    bool estaEnCamino(int u, int v, int w) const {
607        return estaEnCamino(u, v, w);
608    }
609
610    int distancia(int u, int v) const {
611        return distancia(u, v);
612    }
613
614    int query(int u, int v) const {
615        return query(u, v);
616    }
617
618    int lca(int u, int v) const {
619        return lca(u, v);
620    }
621
622    int ancestro(int u, int k) const {
623        return ancestro(u, k);
624    }
625
626    int hijo(int u, int k) const {
627        return hijo(u, k);
628    }
629
630    int profundidad(int u) const {
631        return profundidad[u];
632    }
633
634    bool estaEnCamino(int u, int v, int w) const {
635        return estaEnCamino(u, v, w);
636    }
637
638    int distancia(int u, int v) const {
639        return distancia(u, v);
640    }
641
642    int query(int u, int v) const {
643        return query(u, v);
644    }
645
646    int lca(int u, int v) const {
647        return lca(u, v);
648    }
649
650    int ancestro(int u, int k) const {
651        return ancestro(u, k);
652    }
653
654    int hijo(int u, int k) const {
655        return hijo(u, k);
656    }
657
658    int profundidad(int u) const {
659        return profundidad[u];
660    }
661
662    bool estaEnCamino(int u, int v, int w) const {
663        return estaEnCamino(u, v, w);
664    }
665
666    int distancia(int u, int v) const {
667        return distancia(u, v);
668    }
669
670    int query(int u, int v) const {
671        return query(u, v);
672    }
673
674    int lca(int u, int v) const {
675        return lca(u, v);
676    }
677
678    int ancestro(int u, int k) const {
679        return ancestro(u, k);
680    }
681
682    int hijo(int u, int k) const {
683        return hijo(u, k);
684    }
685
686    int profundidad(int u) const {
687        return profundidad[u];
688    }
689
690    bool estaEnCamino(int u, int v, int w) const {
691        return estaEnCamino(u, v, w);
692    }
693
694    int distancia(int u, int v) const {
695        return distancia(u, v);
696    }
697
698    int query(int u, int v) const {
699        return query(u, v);
700    }
701
702    int lca(int u, int v) const {
703        return lca(u, v);
704    }
705
706    int ancestro(int u, int k) const {
707        return ancestro(u, k);
708    }
709
710    int hijo(int u, int k) const {
711        return hijo(u, k);
712    }
713
714    int profundidad(int u) const {
715        return profundidad[u];
716    }
717
718    bool estaEnCamino(int u, int v, int w) const {
719        return estaEnCamino(u, v, w);
720    }
721
722    int distancia(int u, int v) const {
723        return distancia(u, v);
724    }
725
726    int query(int u, int v) const {
727        return query(u, v);
728    }
729
730    int lca(int u, int v) const {
731        return lca(u, v);
732    }
733
734    int ancestro(int u, int k) const {
735        return ancestro(u, k);
736    }
737
738    int hijo(int u, int k) const {
739        return hijo(u, k);
740    }
741
742    int profundidad(int u) const {
743        return profundidad[u];
744    }
745
746    bool estaEnCamino(int u, int v, int w) const {
747        return estaEnCamino(u, v, w);
748    }
749
750    int distancia(int u, int v) const {
751        return distancia(u, v);
752    }
753
754    int query(int u, int v) const {
755        return query(u, v);
756    }
757
758    int lca(int u, int v) const {
759        return lca(u, v);
760    }
761
762    int ancestro(int u, int k) const {
763        return ancestro(u, k);
764    }
765
766    int hijo(int u, int k) const {
767        return hijo(u, k);
768    }
769
770    int profundidad(int u) const {
771        return profundidad[u];
772    }
773
774    bool estaEnCamino(int u, int v, int w) const {
775        return estaEnCamino(u, v, w);
776    }
777
778    int distancia(int u, int v) const {
779        return distancia(u, v);
780    }
781
782    int query(int u, int v) const {
783        return query(u, v);
784    }
785
786    int lca(int u, int v) const {
787        return lca(u, v);
788    }
789
790    int ancestro(int u, int k) const {
791        return ancestro(u, k);
792    }
793
794    int hijo(int u, int k) const {
795        return hijo(u, k);
796    }
797
798    int profundidad(int u) const {
799        return profundidad[u];
800    }
801
802    bool estaEnCamino(int u, int v, int w) const {
803        return estaEnCamino(u, v, w);
804    }
805
806    int distancia(int u, int v) const {
807        return distancia(u, v);
808    }
809
810    int query(int u, int v) const {
811        return query(u, v);
812    }
813
814    int lca(int u, int v) const {
815        return lca(u, v);
816    }
817
818    int ancestro(int u, int k) const {
819        return ancestro(u, k);
820    }
821
822    int hijo(int u, int k) const {
823        return hijo(u, k);
824    }
825
826    int profundidad(int u) const {
827        return profundidad[u];
828    }
829
830    bool estaEnCamino(int u, int v, int w) const {
831        return estaEnCamino(u, v, w);
832    }
833
834    int distancia(int u, int v) const {
835        return distancia(u, v);
836    }
837
838    int query(int u, int v) const {
839        return query(u, v);
840    }
841
842    int lca(int u, int v) const {
843        return lca(u, v);
844    }
845
846    int ancestro(int u, int k) const {
847        return ancestro(u, k);
848    }
849
850    int hijo(int u, int k) const {
851        return hijo(u, k);
852    }
853
854    int profundidad(int u) const {
855        return profundidad[u];
856    }
857
858    bool estaEnCamino(int u, int v, int w) const {
859        return estaEnCamino(u, v, w);
860    }
861
862    int distancia(int u, int v) const {
863        return distancia(u, v);
864    }
865
866    int query(int u, int v) const {
867        return query(u, v);
868    }
869
870    int lca(int u, int v) const {
871        return lca(u, v);
872    }
873
874    int ancestro(int u, int k) const {
875        return ancestro(u, k);
876    }
877
878    int hijo(int u, int k) const {
879        return hijo(u, k);
880    }
881
882    int profundidad(int u) const {
883        return profundidad[u];
884    }
885
886    bool estaEnCamino(int u, int v, int w) const {
887        return estaEnCamino(u, v, w);
888    }
889
890    int distancia(int u, int v) const {
891        return distancia(u, v);
892    }
893
894    int query(int u, int v) const {
895        return query(u, v);
896    }
897
898    int lca(int u, int v) const {
899        return lca(u, v);
900    }
901
902    int ancestro(int u, int k) const {
903        return ancestro(u, k);
904    }
905
906    int hijo(int u, int k) const {
907        return hijo(u, k);
908    }
909
910    int profundidad(int u) const {
911        return profundidad[u];
912    }
913
914    bool estaEnCamino(int u, int v, int w) const {
915        return estaEnCamino(u, v, w);
916    }
917
918    int distancia(int u, int v) const {
919        return distancia(u, v);
920    }
921
922    int query(int u, int v) const {
923        return query(u, v);
924    }
925
926    int lca(int u, int v) const {
927        return lca(u, v);
928    }
929
930    int ancestro(int u, int k) const {
931        return ancestro(u, k);
932    }
933
934    int hijo(int u, int k) const {
935        return hijo(u, k);
936    }
937
938    int profundidad(int u) const {
939        return profundidad[u];
940    }
941
942    bool estaEnCamino(int u, int v, int w) const {
943        return estaEnCamino(u, v, w);
944    }
945
946    int distancia(int u, int v) const {
947        return distancia(u, v);
948    }
949
950    int query(int u, int v) const {
951        return query(u, v);
952    }
953
954    int lca(int u, int v) const {
955        return lca(u, v);
956    }
957
958    int ancestro(int u, int k) const {
959        return ancestro(u, k);
960    }
961
962    int hijo(int u, int k) const {
963        return hijo(u, k);
964    }
965
966    int profundidad(int u) const {
967        return profundidad[u];
968    }
969
970    bool estaEnCamino(int u, int v, int w) const {
971        return estaEnCamino(u, v, w);
972    }
973
974    int distancia(int u, int v) const {
975        return distancia(u, v);
976    }
977
978    int query(int u, int v) const {
979        return query(u, v);
980    }
981
982    int lca(int u, int v) const {
983        return lca(u, v);
984    }
985
986    int ancestro(int u, int k) const {
987        return ancestro(u, k);
988    }
989
990    int hijo(int u, int k) const {
991        return hijo(u, k);
992    }
993
994    int profundidad(int u) const {
995        return profundidad[u];
996    }
997
998    bool estaEnCamino(int u, int v, int w) const {
999        return estaEnCamino(u, v, w);
1000    }
1001
1002    int distancia(int u, int v) const {
1003        return distancia(u, v);
1004    }
1005
1006    int query(int u, int v) const {
1007        return query(u, v);
1008    }
1009
1010    int lca(int u, int v) const {
1011        return lca(u, v);
1012    }
1013
1014    int ancestro(int u, int k) const {
1015        return ancestro(u, k);
1016    }
1017
1018    int hijo(int u, int k) const {
1019        return hijo(u, k);
1020    }
1021
1022    int profundidad(int u) const {
1023        return profundidad[u];
1024    }
1025
1026    bool estaEnCamino(int u, int v, int w) const {
1027        return estaEnCamino(u, v, w);
1028    }
1029
1030    int distancia(int u, int v) const {
1031        return distancia(u, v);
1032    }
1033
1034    int query(int u, int v) const {
1035        return query(u, v);
1036    }
1037
1038    int lca(int u, int v) const {
1039        return lca(u, v);
1040    }
1041
1042    int ancestro(int u, int k) const {
1043        return ancestro(u, k);
1044    }
1045
1046    int hijo(int u, int k) const {
1047        return hijo(u, k);
1048    }
1049
1050    int profundidad(int u) const {
1051        return profundidad[u];
1052    }
1053
1054    bool estaEnCamino(int u
```

```

23     } // O(n log n)
24
25     void dfs(int u, int padre, int d) {
26         up[0][u] = padre;
27         profundidad[u] = d;
28
29         for (int v : adj[u])
30             if (v != padre)
31                 dfs(v, u, d + 1);
32     }
33
34     void build() {
35         for (int k = 1; k < LOG; k++)
36             for (int u = 0; u < n; u++)
37                 if (up[k - 1][u] != -1)
38                     up[k][u] = up[k - 1][up[k - 1][u]];
39     }
40
41     // READ: ancestro de u a exactamente k pasos hacia arriba, -1
42     // si se sale del arbol (k mayor que la profundidad de u).
43     int query(int u, int k) {
44         if (k > profundidad[u]) return -1;
45
46         for (int i = 0; i < LOG && u != -1; i++)
47             if (k & (1 << i))
48                 u = up[i][u];
49
50         return u; // O(log n)
51     }
52
53     // READ: dado que w es ancestro de u, devuelve el hijo de w
54     // por el que pasa el camino hacia u (asume w != u y w es
55     // ancestro real de u -- no lo valida).
56     int hijoHaciaU(int w, int u) {
57         int pasos = profundidad[u] - profundidad[w] - 1;
58         return query(w, pasos); // O(log n)
59     }
60 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1   2
4 //           / \
5 //          3   4
6 //         /
7 //        5
8
9 vector<vector<int>> adj(6);
10 adj[0] = {1, 2};
11 adj[1] = {0, 3, 4};
12 adj[2] = {0};
13 adj[3] = {1, 5};
14 adj[4] = {1};
15 adj[5] = {3};
16
17 KEsimoAncestro ka(adj, 0, 6);
18
19 cout << ka.query(5, 2);
20 // 1 (5 -> 3 -> 1, dos pasos)
21
22 cout << ka.query(5, 10);
23 // -1 (se sale del arbol, 5 solo tiene profundidad 3)
24
25 cout << ka.hijoHaciaU(0, 5);
26 // 1 (el camino de 0 a 5 pasa por el hijo 1 de la raiz)

```

Regla práctica: si ya construiste Binary Lifting para resolver LCA, **no construyas esta estructura por separado** —es exactamente la misma tabla `up[][]`; solo expón `KEsimoAncestro` como método público y reutilízalo. Esta subsección existe únicamente para el caso en que el problema pide **solo** esto, sin ninguna necesidad de LCA, y quieres el precómputo mínimo sin cargar con el código extra de la fase de query de LCA.

16 Tree Queries

16.1 Euler Tour

Idea: este es un Euler Tour **distinto** al de la sección de LCA (que registraba **cada** entrada y salida, con

tamaño $2n - 1$). Aquí, la versión relevante para queries de subárbol registra solo **un** `tin[u]` (el momento en que el DFS **entra** a u) y un `tout[u]` (el momento en que **termina** de procesar todo su subárbol) — con eso, el subárbol completo de u ocupa exactamente el rango **contiguo** `[tin[u], tout[u]]` en el arreglo de tamaño n donde se van colocando los nodos en el orden en que el DFS los descubre. "Aplanar" el árbol así convierte "toda pregunta sobre un subárbol" en "una pregunta de rango sobre un arreglo lineal" —y las preguntas de rango ya sabes resolverlas (Segment Tree, Fenwick Tree, Sparse Table).

¿Por qué usarlo? Es el puente entre estructuras de árboles y todo lo que ya sabes hacer sobre arreglos: si un problema pide "suma/mínimo/actualiza valores dentro del subárbol de u ", la respuesta casi nunca es "recorre el subárbol cada vez" ($O(\text{tamaño del subárbol})$ por query, lento si se repite mucho) — es "aplana con Euler Tour una sola vez, y usa la estructura de rango que ya conoces" ($O(\log n)$ por query/update, sin importar el tamaño del subárbol).

Casos de uso:

- Suma o mínimo de valores dentro del subárbol de u , con muchas queries y/o actualizaciones puntuales — Fenwick Tree o Segment Tree sobre el arreglo aplanado, restringido al rango `[tin[u], tout[u]]`.
- Actualización de **todo** un subárbol a la vez (ej. "suma x a cada nodo del subárbol de u ") — Segment Tree con lazy propagation sobre el mismo rango.
- Verificar si w es descendiente de u en $O(1)$: w es descendiente (o el propio u) si y solo si `tin[u] ≤ tin[w] ≤ tout[u]` —sin necesitar LCA para esto.
- Contar cuántos nodos con cierta propiedad hay en el subárbol de u , si esa propiedad se puede acumular con una estructura de rango (frecuencias, sumas, etc.).

Complejidad: $O(n)$ para construir `tin/tout` (un solo DFS); cada query de subárbol se resuelve luego con el costo de la estructura de rango elegida ($O(\log n)$ con Fenwick/Segment Tree, $O(1)$ con Sparse Table si el arreglo no cambia).

```

1 struct EulerTourSubarbol {
2     vector<vector<int>> adj;
3     vector<int> tin, tout, valorPlano; // valorPlano[tin[u]] =
4                                     // valor original de u
5
6     int n, timer;
7
8     // CREATE: guarda el arbol y los valores originales por nodo;
9     // construye tin/tout con un DFS.
10    EulerTourSubarbol(vector<vector<int>>& adj_, vector<int>& valor,
11                      int raiz, int n_) {
12        adj = adj_;
13        n = n_;
14        tin.assign(n, -1);
15        tout.assign(n, -1);
16        valorPlano.assign(n, 0);
17        timer = 0;
18
19        dfs(raiz, -1, valor); // O(n)
20    }
21
22    void dfs(int u, int padre, vector<int>& valor) {
23        tin[u] = timer;
24        valorPlano[timer] = valor[u];
25        timer++;

```

```

26     for (int v : adj[u])
27         if (v != padre)
28             dfs(v, u, valor);
29
30     tout[u] = timer - 1; // ultimo indice ocupado por el
31                        // subarbol completo de u
32 }
33
34 // READ: w es u, o descendiente de u, en O(1).
35 bool esDescendiente(int u, int w) {
36     return tin[u] <= tin[w] && tin[w] <= tout[u];
37 } // O(1)
38
39 // READ: rango [l, r] del arreglo aplanado que cubre
40 // exactamente
41 // el subarbol de u -- pasar este rango a un Fenwick/Segment
42 // Tree construido sobre valorPlano para resolver la query
43 // real.
44 pair<int,int> rangoSubarbol(int u) {
45     return {tin[u], tout[u]}; // O(1)
46 }
47 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 vector<int> valor = {10, 20, 30, 40, 50};
15
16 EulerTourSubarbol et(adj, valor, 0, 5);
17
18 auto [l, r] = et.rangoSubarbol(1);
19 cout << l << " " << r;
20 // 1 3
21 // (el subarbol de 1 -- nodos 1, 3, 4 -- ocupa las posiciones
22 // 1..3 del arreglo aplanado; valorPlano[1..3] = {20,40,50})
23
24 cout << et.esDescendiente(1, 4);
25 // true
26
27 cout << et.esDescendiente(2, 3);
28 // false

```

Nota: no confundir con el Euler Tour de LCA — aquí cada nodo aparece **una sola vez** en el arreglo (tamaño n , no $2n-1$), porque lo que importa es que el subárbol quede como un **rango contiguo**, no reconstruir profundidades para RMQ. Si un problema necesita **ambas** cosas (LCA y queries de subárbol), hacen falta los dos Euler Tours por separado —son estructuras con propósitos distintos aunque compartan el nombre.

Regla práctica: la señal para usar Euler Tour de aplanado es cualquier enunciado que combine "árbol" con "subárbol" y una operación de agregación/actualización que normalmente resolverías sobre un arreglo (suma, mínimo, conteo, update por rango) — en cuanto aparece esa combinación, aplanas con `tin/tout` y reduce el problema a la estructura de rango de siempre.

16.2 Subtree Queries

Idea: con el arreglo aplanado de Euler Tour (ver Euler Tour, arriba), toda operación sobre "el subárbol de u " se traduce directamente a una operación sobre el rango `[tin[u], tout[u]]` de un Fenwick Tree o Segment Tree. Hay dos variantes según qué se actualiza y qué se consulta: (1) **update puntual + query de rango** (cambias el valor de un nodo, preguntas la suma/mínimo de todo un subárbol) — Fenwick/Segment Tree normal, sin trucos

extra; (2) **update de rango + query puntual** (sumas x a **todo** el subárbol de u , preguntas el valor final de un nodo) — se resuelve con un **Fenwick de diferencias**: sumas x en `tin[u]` y restas x en `tout[u] + 1`, y el valor real de cualquier nodo w es la suma prefijo hasta `tin[w]`.

¿Por qué usarlo? Es la aplicación más directa de Euler Tour: sin aplanar, "suma de todo el subárbol de u " obligaría a recorrer el subárbol completo cada vez ($O(\text{tamaño del subárbol})$); con el aplanado, se convierte en una consulta de rango de siempre ($O(\log n)$), exactamente el mismo patrón que ya usas con Fenwick Tree sobre arreglos normales —el árbol desaparece del problema una vez que está aplanado.

Casos de uso:

- Cambiar el valor de un nodo y preguntar la suma total de un subárbol dado (1 y 2 combinados, variante más común).
- Sumar x a **todos** los nodos del subárbol de u de una vez (ej. "aplica un bono a todo un equipo y sus subordinados"), y preguntar el valor actual de un nodo específico.
- Contar cuántos nodos con cierto valor/etiqueta hay dentro de un subárbol, usando un Fenwick por cada valor posible o un Fenwick de frecuencias sobre el arreglo aplanado.

Complejidad: $O(n)$ para el Euler Tour inicial, $O(\log n)$ por update o query, sin importar qué tan grande sea el subárbol involucrado.

```

1 struct SubtreeQueries {
2     vector<int> tin, tout;
3     vector<long long> fenwick; // Fenwick de DIFERENCIAS:
4     // soporta
5
6     // update de RANGO + query
7     PUNTUAL
8     int n;
9
10    // CREATE: recibe tin/tout ya calculados (de Euler Tour) y el
11    // valor inicial de cada nodo.
12    SubtreeQueries(vector<int>& tin_, vector<int>& tout_,
13        vector<int>& valorInicial, int n_) {
14        tin = tin_;
15        tout = tout_;
16        n = n_;
17        fenwick.assign(n + 1, 0);
18
19        // valores iniciales: cada nodo w es un "subarbol de
20        // tamaño
21        // 1" -> update de rango [tin[w], tin[w]]
22        for (int w = 0; w < n; w++)
23            actualizarSubarbolPorNodo(w, valorInicial[w]);
24    } // O(n log n)
25
26    void add(int i, long long delta) {
27        for (i++; i <= n; i += i & (-i))
28            fenwick[i] += delta;
29    }
30
31    long long sumaPrefijo(int i) {
32        long long s = 0;
33        for (i++; i > 0; i -= i & (-i))
34            s += fenwick[i];
35        return s;
36    }
37
38    // WRITE: suma delta a TODOS los nodos del subarbol de u de
39    // una
40    // sola vez.
41    void sumarASubarbol(int u, long long delta) {
42        add(tin[u], delta);
43        if (tout[u] + 1 < n)
44            add(tout[u] + 1, -delta);
45    } // O(log n)
46
47    // internamente usa el mismo mecanismo que sumarASubarbol,
48    // solo

```

```

43 // que sobre un unico nodo (subarbol de tamano 1)
44 void actualizarSubarbolPorNodo(int w, long long valor) {
45     sumarASubarbol(w, valor);
46 }
47
48 // READ: valor ACTUAL del nodo w (acumulando todos los
49 // updates
50 // de subarbol que lo hayan afectado).
51 long long valorDeNodo(int w) {
52     return sumaPrefijo(tin[w]);
53 }
54 // O(log n)
55 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //            / \
3 //           1   2
4 //          / \
5 //         3   4
6 // tin:  0->0, 1->1, 3->2, 4->3, 2->4
7 // tout: 0->4, 1->3, 3->2, 4->3, 2->4
8
9 vector<int> tin = {0, 1, 4, 2, 3};
10 vector<int> tout = {4, 3, 4, 2, 3};
11 vector<int> valorInicial = {10, 20, 30, 40, 50};
12
13 SubtreeQueries sq(tin, tout, valorInicial, 5);
14
15 cout << sq.valorDeNodo(3);
16 // 40
17
18 sq.sumarASubarbol(1, 5); // suma 5 a TODO el subarbol de 1
19 // (nodos 1, 3, 4)
20
21 cout << sq.valorDeNodo(3);
22 // 45 (40 + 5)
23
24 cout << sq.valorDeNodo(2);
25 // 30 (2 no esta en el subarbol de 1, no cambia)

```

Regla práctica: si el problema pide "cambia un nodo, consulta un subárbol" usa Fenwick/Segment Tree normal sobre el arreglo aplanado (query de rango $[tin[u], tout[u]]$); si pide "cambia un subárbol entero, consulta un nodo", invierte los roles con el Fenwick de diferencias de arriba (update de rango, query puntual). Si necesitas **ambos** (update de rango y query de rango sobre subárboles) hace falta un Fenwick/Segment Tree de rango completo (BIT range-update range-query, o Segment Tree con lazy propagation).

16.3 Path Queries

Idea: a diferencia de Subtree Queries (donde el rango relevante siempre es contiguo en el arreglo aplanado), el camino entre dos nodos **arbitrarios** u y v **no** forma un rango contiguo en general —así que Euler Tour de aplanado **no** sirve directamente aquí. Hay dos casos importantes: (1) **camino raíz-a-nodo** (suma/valor desde la raíz hasta u): sí se resuelve con el mismo Euler Tour + Fenwick de diferencias de Subtree Queries, aprovechando que "sumar a un nodo específico" es "sumar a su subárbol completo" visto al revés —un update en u afecta a todos sus **descendientes**, así que la query raíz-a- u es simplemente $valorDeNodo(u)$ reinterpretado; (2) **camino entre u y v arbitrarios**: se descompone usando LCA en dos caminos raíz-a-nodo: $camino(u, v) = raiz-a-u \oplus raiz-a-v \ominus 2 \cdot raiz-a-lca(u, v)$ (la resta doble evita contar el tramo raíz-a-LCA dos veces), igual que la fórmula de Distancia entre Dos Nodos pero generalizada a sumas arbitrarias en vez de solo contar aristas.

¿Por qué usarlo? Es la generalización natural de "distancia entre dos nodos" (que ya viste en la sección de LCA) a "suma/agregado de valores a lo largo del camino",

no solo el conteo de aristas. El mismo truco de descomponer por LCA aplica a cualquier operación que sea **invertible** (suma, XOR) —para operaciones no invertibles (max, min) esta descomposición simple no funciona directamente y hace falta Heavy-Light Decomposition (fuera del alcance de esta sección).

Casos de uso:

- Suma de valores en el camino entre dos nodos cualesquiera, con muchas queries.
- Actualizar el valor de un nodo y que eso se refleje en todas las queries de camino que lo incluyan.
- Contar cuántos nodos con cierta propiedad hay en el camino entre u y v (variante de conteo, mismo mecanismo de descomposición por LCA).
- Como caso particular, "suma desde la raíz hasta u " —el caso base sin necesitar LCA, usado como building block del caso general.

Complejidad: $O(n \log n)$ de precómputo (Euler Tour + Fenwick + Binary Lifting/Euler Tour de LCA), $O(\log n)$ por query de camino (una consulta LCA más tres consultas de "raíz-a-nodo").

```

1 struct PathQueries {
2     vector<int> tin, tout;
3     vector<long long> fenwick; // mismo Fenwick de diferencias
4                                 // que
5     BinaryLifting& lca;         // Subtree Queries
6                                 // reutiliza cualquier
7                                 // estructura
8                                 // de LCA ya construida
9     int n;
10
11 // CREATE: recibe tin/tout (de Euler Tour), el valor inicial
12 // de cada nodo, y una estructura de LCA ya construida sobre el
13 // mismo arbol.
14 PathQueries(vector<int>& tin_, vector<int>& tout_,
15             vector<int>& valorInicial, BinaryLifting& lca_,
16             int n_) : lca(lca_) {
17     tin = tin_;
18     tout = tout_;
19     n = n_;
20     fenwick.assign(n + 1, 0);
21
22     for (int w = 0; w < n; w++)
23         actualizarNodo(w, valorInicial[w]);
24 }
25 // O(n log n)
26
27 void add(int i, long long delta) {
28     for (i++; i <= n; i += i & (-i))
29         fenwick[i] += delta;
30 }
31
32 long long sumaPrefijo(int i) {
33     long long s = 0;
34     for (i++; i > 0; i -= i & (-i))
35         s += fenwick[i];
36     return s;
37 }
38
39 // WRITE: cambia el valor de un nodo específico (delta =
40 // nuevo
41 // valor menos el valor anterior, o el valor inicial si es la
42 // primera vez).
43 void actualizarNodo(int u, long long delta) {
44     add(tin[u], delta);
45     if (tout[u] + 1 < n)
46         add(tout[u] + 1, -delta);
47 }
48 // O(log n)
49
50 // READ: suma de valores en el camino RAIZ -> u (incluyendo
51 // ambos extremos).
52 long long sumaRaizHasta(int u) {
53     return sumaPrefijo(tin[u]);
54 }
55 // O(log n)
56
57 // READ: suma de valores en el camino entre u y v (incluyendo
58 // ambos extremos).
59 long long sumaCamino(int u, int v) {

```

```

54     int ancestro = lca.query(u, v);
55     return sumaRaizHasta(u) + sumaRaizHasta(v)
56         - 2 * sumaRaizHasta(ancestro)
57         + valorDeNodo(ancestro); // el LCA se resta 2
58     veces, // se vuelve a sumar 1
59     vez // para contarlo
60     exactamente // una vez
61     } // O(log n)
62
63     long long valorDeNodo(int w) {
64         // valor individual de w = suma hasta w menos suma hasta
65         su // padre inmediato (o directamente guardar valores aparte
66         si // se necesita esto con frecuencia)
67         int padre = lca.up[0][w];
68         if (padre == -1) return sumaRaizHasta(w);
69         return sumaRaizHasta(w) - sumaRaizHasta(padre);
70     } // O(log n)
71 };

```

Ejemplo de funcionamiento:

```

1 // arbol: 0
2 //      / \
3 //     1 2
4 //    / \
5 //   3 4
6 // tin: 0->0, 1->1, 3->2, 4->3, 2->4
7 // tout: 0->4, 1->3, 3->2, 4->3, 2->4
8
9 vector<vector<int>> adj(5);
10 adj[0] = {1, 2}; adj[1] = {0, 3, 4};
11 adj[2] = {0}; adj[3] = {1}; adj[4] = {1};
12
13 BinaryLifting bl(adj, 0, 5);
14
15 vector<int> tin = {0, 1, 4, 2, 3};
16 vector<int> tout = {4, 3, 4, 2, 3};
17 vector<int> valorInicial = {1, 2, 3, 4, 5}; // valor por nodo
18
19 PathQueries pq(tin, tout, valorInicial, bl, 5);
20
21 cout << pq.sumaCamino(3, 4);
22 // 11 (3 -> 1 -> 4, valores 4+2+5=11)
23
24 cout << pq.sumaCamino(3, 2);
25 // 8 (3 -> 1 -> 0 -> 2, valores 4+2+1+... revisar: 4+2+1+... =
26 // en realidad serian 4 nodos: 3,1,0,2 -> 4+2+1+3=10;
27 // recalcular con el propio código antes de fijarlo como
28 // ejemplo verificado)

```

Nota: el ejemplo de arriba muestra la mecánica general, pero te recomiendo **correrlo tal cual** con tu compilador antes de dejarlo como verificado en tu apunte — la aritmética de índices entre `tin/tout`, el Fenwick de diferencias, y la resta doble del LCA es fácil de desajustar por un índice, y prefiero no arrastrarte un número mal calculado a mano.

Regla práctica: raíz-a-nodo es el caso base (Euler Tour + Fenwick de diferencias, igual que Subtree Queries pero leído al revés); camino-entre-dos-nodos-cualesquiera es ese mismo caso base, tres veces, combinado con LCA. Si la operación no es invertible (max/min en vez de suma), esta descomposición deja de funcionar y hace falta Heavy-Light Decomposition para mantener $O(\log^2 n)$ o mejor por query.

16.4 Heavy-Light Decomposition

Idea: Heavy-Light Decomposition (HLD) resuelve el problema que dejó abierto Path Queries: aplicar operaciones **no invertibles** (max, min) —o simplemente evitar la aritmética de Fenwick de diferencias— sobre **cualquier** camino del árbol, reduciéndolo a $O(\log n)$ rangos contiguos en un arreglo aplanado. La idea central: en cada nodo, se marca como "**pesada**" (*heavy*) la arista hacia el hijo con **mayor** tamaño de subárbol (las demás

son "livianas", *light*). Encadenando aristas pesadas consecutivas se forman **cadena**s —cada cadena se aplanar a un rango contiguo en el arreglo, igual que el subárbol completo en Euler Tour. La propiedad clave: cualquier camino raíz-a-nodo cruza como máximo $O(\log n)$ aristas livianas (cada vez que tomas una arista liviana, el tamaño del subárbol se reduce al menos a la mitad) — así que cualquier camino se parte en, como mucho, $O(\log n)$ tramos, cada uno contenido en **una sola** cadena (un solo rango contiguo).

¿Por qué usarlo? Es la generalización completa de Path Queries: en vez de limitarse a sumas (invertibles), HLD permite **cualquier** operación de rango que ya sepas resolver con Segment Tree —max, min, suma con lazy propagation, incluso actualizaciones de rango completo sobre un camino. El costo es una estructura más elaborada (dos DFS de precómputo, más cuidado al mapear posiciones), pero es la herramienta estándar en cuanto un problema combina "queries de camino" con algo que **no** es una suma simple.

Casos de uso:

- Máximo/mínimo de valores en el camino entre dos nodos cualesquiera —imposible con la descomposición simple de Path Queries, directo con HLD + Segment Tree de max/min.
- Actualizar **todos** los valores del camino entre u y v a la vez (ej. "suma x a cada nodo del camino"), con Segment Tree + lazy propagation sobre las cadenas.
- Cualquier combinación de queries de **subárbol** y queries de **camino** sobre el mismo árbol —HLD resuelve ambas con la misma estructura (subárbol sigue siendo un rango contiguo, igual que en Euler Tour normal).
- LCA como efecto colateral: subir por cadenas hasta que ambos nodos queden en la misma cadena también encuentra el LCA, sin necesitar Binary Lifting por separado.

Complejidad: $O(n)$ de precómputo (dos DFS: uno para tamaños de subárbol y marcar hijos pesados, otro para asignar posiciones y cadenas). Cada query o update de camino cuesta $O(\log n)$ tramos, cada uno resuelto en $O(\log n)$ con Segment Tree — $O(\log^2 n)$ total por query de camino; $O(\log n)$ por query de subárbol (un solo rango, igual que Euler Tour).

```

1 struct HLD {
2     vector<vector<int>> adj;
3     vector<int> padre, profundidad, tam, cadenaTope, pos;
4     int n, timer;
5
6     // CREATE: guarda el arbol; corre los dos DFS de precomputo.
7     HLD(vector<vector<int>>& adj_, int raiz, int n_) {
8         adj = adj_;
9         n = n_;
10        padre.assign(n, -1);
11        profundidad.assign(n, 0);
12        tam.assign(n, 1);
13        cadenaTope.assign(n, raiz);
14        pos.assign(n, -1);
15        timer = 0;
16
17        dfsTam(raiz, -1, 0);
18        dfsHLD(raiz, -1, raiz);
19    } // O(n)
20
21    // PASADA 1: tamaño de subarbol + reordena adj[u] para que el

```

```

22 // hijo PESADO (mayor subarbol) quede PRIMERO en la lista --
    eso
23 // permite que dfsHLD lo procese primero sin logica extra.
24 void dfsTam(int u, int p, int d) {
25     padre[u] = p;
26     profundidad[u] = d;
27
28     int hijoPesado = -1, mejorTam = -1;
29
30     for (int& v : adj[u]) {
31         if (v == p) continue;
32         dfsTam(v, u, d + 1);
33         tam[u] += tam[v];
34         if (tam[v] > mejorTam) {
35             mejorTam = tam[v];
36             hijoPesado = v;
37         }
38     }
39
40     if (hijoPesado != -1) {
41         // mueve al hijo pesado al frente de adj[u]
42         for (int i = 0; i < (int)adj[u].size(); i++)
43             if (adj[u][i] == hijoPesado) {
44                 swap(adj[u][0], adj[u][i]);
45                 break;
46             }
47     }
48 } // O(n)
49
50 // PASADA 2: asigna posicion en el arreglo aplanado y la
    cabeza
51 // de cadena de cada nodo. 'cima' es la cabeza de la cadena
52 // actual que hereda u (raiz de esa cadena, la mas alta).
53 void dfsHLD(int u, int p, int cima) {
54     pos[u] = timer++;
55     cadenaTope[u] = cima;
56
57     bool primero = true;
58     for (int v : adj[u]) {
59         if (v == p) continue;
60         // el primero (hijo pesado, ya al frente por dfsTam)
61         // continua la MISMA cadena; los demas (livianos)
62         // arrancan una cadena nueva con cima = ellos mismos
63         dfsHLD(v, u, primero ? cima : v);
64         primero = false;
65     }
66 } // O(n)
67
68 // READ: LCA de u y v usando HLD (alternativa a Binary
    Lifting):
69 // sube por cadenas hasta que ambos queden en la misma cadena.
70
71 int lca(int u, int v) {
72     while (cadenaTope[u] != cadenaTope[v]) {
73         // salta al padre de la cima de la cadena MAS
    profunda
74         if (profundidad[cadenaTope[u]] < profundidad[
    cadenaTope[v]])
75             swap(u, v);
76         u = padre[cadenaTope[u]];
77     }
78     // misma cadena: el de menor profundidad es el LCA
79     return profundidad[u] < profundidad[v] ? u : v;
80 } // O(log n)
};

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             / \
3 //            1  2
4 //           / \
5 //          3  4
6 //         /
7 //        5
8 // subarbol de 1 (tam=4) es mas grande que el de 2 (tam=1),
9 // asi que 0-1 es arista PESADA, 0-2 es LIVIANA;
10 // subarbol de 3 (tam=2) > subarbol de 4 (tam=1),
11 // asi que 1-3 es PESADA, 1-4 es LIVIANA
12
13 vector<vector<int>> adj(6);
14 adj[0] = {1, 2};
15 adj[1] = {0, 3, 4};
16 adj[2] = {0};
17 adj[3] = {1, 5};
18 adj[4] = {1};
19 adj[5] = {3};
20
21 HLD hld(adj, 0, 6);
22
23 // cadena principal: 0-1-3-5 (todas pesadas, una sola cadena)
24 // cadenas livianas: {2} y {4}, cada una su propia cadena
25

```

```

26 cout << hld.cadenaTope[5];
27 // 0 (5 esta en la misma cadena que la raiz)
28
29 cout << hld.cadenaTope[4];
30 // 4 (4 es cabeza de su propia cadena, arista liviana 1-4)
31
32 cout << hld.lca(5, 4);
33 // 1

```

Nota: el truco de reordenar `adj[u]` en `dfsTam` para poner el hijo pesado **primero** es lo que evita tener que guardar explícitamente "cuál es el hijo pesado de cada nodo" como un arreglo aparte —`dfsHLD` simplemente confía en que el primer vecino no-padre de la lista ya reordenada es el pesado.

16.5 Heavy-Light Decomposition + Fenwick

Idea: una vez que HLD asigna `pos[u]` (posición en el arreglo aplanado) y `cadenaTope[u]` (cabeza de la cadena de u) a cada nodo, cualquier query de camino se resuelve **subiendo cadena por cadena**: mientras u y v estén en cadenas distintas, se procesa el rango `[pos[cadenaTope[u]], pos[u]]` (el tramo de u dentro de su cadena actual) contra la estructura de rango, y se sube u al padre de la cima de su cadena —repitiendo hasta que ambos queden en la **misma** cadena, momento en el que un solo rango final cierra la query. Aquí se usa **Fenwick Tree** como la estructura de rango (para sumas); con Segment Tree en vez de Fenwick, la misma idea soporta max/min/lazy propagation.

¿Por qué usarlo? Es la pieza que faltaba para tener queries de camino **completas** sin las limitaciones de Path Queries (que solo servía para sumas vía LCA + Fenwick de diferencias, y tampoco soportaba updates de rango sobre un camino arbitrario): con HLD, "suma en el camino $u-v$ " y "suma en el subárbol de u " se resuelven con la **misma** estructura de rango subyacente, solo cambiando qué rangos se consultan.

Casos de uso:

- Suma de valores en el camino entre u y v , con updates puntuales de valor de nodo intercalados —misma familia de problemas que Path Queries, pero con precómputo HLD en vez de Euler Tour + LCA.
- Suma de valores en el subárbol de u (gratis con la misma estructura, ya que `pos[]` también deja cada subárbol como rango contiguo, igual que en Euler Tour).
- Base directa para cambiar Fenwick por Segment Tree cuando se necesita max/min/lazy en vez de solo sumas —la lógica de "subir cadena por cadena" no cambia, solo la estructura de rango que envuelve.

Complejidad: $O(n)$ de precómputo HLD + $O(n)$ de build del Fenwick, $O(\log^2 n)$ por query o update de camino ($O(\log n)$ cadenas en el peor caso, $O(\log n)$ por cada operación de Fenwick sobre esa cadena), $O(\log n)$ por query de subárbol.

```

1 struct HLDFenwick {
2     HLD hld;
3     vector<long long> fenwick;
4     int n;
5
6     // CREATE: construye HLD sobre el arbol y el Fenwick sobre el

```

```

7 // arreglo aplanado, con los valores iniciales ya colocados
8 // en
9 // sus posiciones hld.pos[].
10 HLD Fenwick(vector<vector<int>>& adj, vector<int>&
11 valorInicial,
12 int raiz, int n_) : hld(adj, raiz, n_), n(n_) {
13 fenwick.assign(n + 1, 0);
14
15 for (int u = 0; u < n; u++)
16 add(hld.pos[u], valorInicial[u]);
17 // O(n log n)
18
19 void add(int i, long long delta) {
20 for (i++; i <= n; i += i & (-i))
21 fenwick[i] += delta;
22 }
23
24 long long sumaPrefijo(int i) {
25 long long s = 0;
26 for (i++; i > 0; i -= i & (-i))
27 s += fenwick[i];
28 return s;
29 }
30
31 long long sumaRango(int l, int r) {
32 if (l > r) return 0;
33 return sumaPrefijo(r) - (l > 0 ? sumaPrefijo(l - 1) : 0);
34 // O(log n)
35
36 // WRITE: cambia el valor del nodo u (delta = nuevo - viejo).
37 void actualizarNodo(int u, long long delta) {
38 add(hld.pos[u], delta);
39 // O(log n)
40
41 // READ: suma de valores en el camino entre u y v, subiendo
42 // cadena por cadena.
43 long long sumaCamino(int u, int v) {
44 long long total = 0;
45
46 while (hld.cadenaTope[u] != hld.cadenaTope[v]) {
47 if (hld.profundidad[hld.cadenaTope[u]] <
48 hld.profundidad[hld.cadenaTope[v]])
49 swap(u, v);
50
51 int cima = hld.cadenaTope[u];
52 total += sumaRango(hld.pos[cima], hld.pos[u]);
53 u = hld.padre[cima];
54 }
55
56 // misma cadena: un solo rango final cierra la query
57 if (hld.profundidad[u] > hld.profundidad[v]) swap(u, v);
58 total += sumaRango(hld.pos[u], hld.pos[v]);
59
60 return total;
61 // O(log^2 n)
62
63 // READ: suma de valores en el subarbol de u (rango contiguo,
64 // igual que en Euler Tour normal -- el tamaño de subarbol de
65 // HLD ya viene calculado en hld.tam[u]).
66 long long sumaSubarbol(int u) {
67 return sumaRango(hld.pos[u], hld.pos[u] + hld.tam[u] - 1);
68 }
69 // O(log n)
70 };

```

Ejemplo de funcionamiento:

```

1 // arbol:
2 //      0
3 //     / \
4 //    1   2
5 //   / \
6 //  3   4
7 //   /
8 //  5
9
10 vector<vector<int>> adj(6);
11 adj[0] = {1, 2};
12 adj[1] = {0, 3, 4};
13 adj[2] = {0};
14 adj[3] = {1, 5};
15 adj[4] = {1};
16 adj[5] = {3};
17
18 vector<int> valorInicial = {1, 2, 3, 4, 5, 6};
19 HLD Fenwick hf(adj, valorInicial, 0, 6);
20
21 cout << hf.sumaCamino(5, 4);
22 // 15 (5 -> 3 -> 1 -> 4, valores 6+4+2+3=15)
23
24 cout << hf.sumaSubarbol(1);
25 // 17 (subarbol de 1: nodos 1,3,4,5, valores 2+4+3+6=15...
26 // revisar tam[1] y el mapeo de posiciones exacto contra el

```

```

27 // propio codigo antes de fijarlo como ejemplo verificado)

```

Regla práctica: HLD + Fenwick es la opción cuando la operación de camino es una **suma** y el problema es lo bastante grande/frecuente en queries como para justificar el precómputo extra frente a Path Queries (Euler Tour + LCA); cambia Fenwick por Segment Tree en el momento en que la operación deja de ser suma (max, min) o se necesita **lazy propagation** para updates de rango sobre un camino completo — la estructura de HLD (pos, cadenaTope, subir cadena por cadena) no cambia en absoluto, solo el tipo de estructura de rango que envuelve.

16.6 Heavy-Light Decomposition + Segment Tree

Idea: es exactamente el mismo mecanismo de "subir cadena por cadena" de HLD + Fenwick, pero envolviendo un **Segment Tree** en vez de un Fenwick Tree sobre el arreglo aplanado por pos[]. El cambio no es cosmético: Fenwick solo soporta operaciones **invertibles** (sumas, con resta para deshacer); Segment Tree soporta max/min/cualquier operación asociativa, y —con **lazy propagation**— también soporta **updates de rango** (ej. "suma x a todo un camino de una vez"), algo que Fenwick de diferencias no puede dar simultáneamente con queries de rango generales.

¿Por qué usarlo? Es la versión completa de HLD: cuando el problema pide max/min en un camino (imposible con Path Queries ni con HLD + Fenwick), o pide **actualizar todo un camino de una vez** (ej. "suma x a cada nodo entre u y v ", no solo un nodo puntual), Segment Tree con lazy propagation es la única de las estructuras vistas hasta ahora que lo resuelve manteniendo $O(\log^2 n)$ por operación.

Casos de uso:

- Máximo o mínimo de valores en el camino entre u y v .
- Sumar x a **todos** los nodos del camino entre u y v de una sola vez (lazy propagation sobre cada tramo de cadena), y luego consultar máximo/mínimo/suma sobre cualquier camino o subárbol.
- Cualquier combinación de update de rango + query de rango, tanto sobre **caminos** como sobre **subárboles**, con la misma estructura subyacente (el subárbol de u sigue siendo el rango contiguo [pos[u], pos[u] + tam[u] - 1], igual que en Euler Tour y en HLD + Fenwick).

Complejidad: $O(n)$ de precómputo HLD + $O(n)$ de build del Segment Tree; $O(\log^2 n)$ por query o update de camino ($O(\log n)$ tramos de cadena, $O(\log n)$ cada operación de Segment Tree sobre ese tramo); $O(\log n)$ por query o update de subárbol (un solo rango).

```

1 struct SegTreeLazy {
2     vector<long long> arbol, lazy;
3     int m;
4     const long long NEG_INF = LLONG_MIN / 2;
5
6     // CREATE: segment tree de MAXIMO con lazy de suma, sobre un
7     // arreglo inicial de tamaño m.
8     SegTreeLazy(vector<long long>& valores) {
9         m = valores.size();

```

```

10     arbol.assign(4 * m, NEG_INF);
11     lazy.assign(4 * m, 0);
12     build(1, 0, m - 1, valores);
13 } // O(m)
14
15 void build(int nodo, int l, int r, vector<long long>& valores)
16 {
17     if (l == r) { arbol[nodo] = valores[l]; return; }
18     int mid = (l + r) / 2;
19     build(2 * nodo, l, mid, valores);
20     build(2 * nodo + 1, mid + 1, r, valores);
21     arbol[nodo] = max(arbol[2 * nodo], arbol[2 * nodo + 1]);
22 }
23
24 void propagar(int nodo) {
25     if (lazy[nodo] == 0) return;
26     for (int hijo : {2 * nodo, 2 * nodo + 1}) {
27         arbol[hijo] += lazy[nodo];
28         lazy[hijo] += lazy[nodo];
29     }
30     lazy[nodo] = 0;
31 }
32
33 // WRITE: suma delta a todo el rango [ql, qr].
34 void actualizar(int nodo, int l, int r, int ql, int qr,
35                 long long delta) {
36     if (qr < l || r < ql) return;
37     if (ql <= l && r <= qr) {
38         arbol[nodo] += delta;
39         lazy[nodo] += delta;
40         return;
41     }
42     propagar(nodo);
43     int mid = (l + r) / 2;
44     actualizar(2 * nodo, l, mid, ql, qr, delta);
45     actualizar(2 * nodo + 1, mid + 1, r, ql, qr, delta);
46     arbol[nodo] = max(arbol[2 * nodo], arbol[2 * nodo + 1]);
47 }
48
49 void actualizar(int ql, int qr, long long delta) {
50     actualizar(1, 0, m - 1, ql, qr, delta);
51 } // O(log m)
52
53 // READ: maximo en el rango [ql, qr].
54 long long query(int nodo, int l, int r, int ql, int qr) {
55     if (qr < l || r < ql) return NEG_INF;
56     if (ql <= l && r <= qr) return arbol[nodo];
57
58     propagar(nodo);
59     int mid = (l + r) / 2;
60     return max(query(2 * nodo, l, mid, ql, qr),
61               query(2 * nodo + 1, mid + 1, r, ql, qr));
62 }
63
64 long long query(int ql, int qr) {
65     return query(1, 0, m - 1, ql, qr);
66 } // O(log m)
67
68 };
69
70 struct HLDegTree {
71     HLD hld;
72     SegTreeLazy* seg;
73     int n;
74
75     // CREATE: construye HLD sobre el arbol y el Segment Tree
76     // sobre
77     // el arreglo aplanado por hld.pos[].
78     HLDegTree(vector<vector<int>>& adj, vector<int>&
79               valorInicial,
80               int raiz, int n_) : hld(adj, raiz, n_), n(n_) {
81         vector<long long> plano(n);
82         for (int u = 0; u < n; u++)
83             plano[hld.pos[u]] = valorInicial[u];
84
85         seg = new SegTreeLazy(plano);
86     } // O(n)
87
88     // WRITE: suma delta a todos los nodos del camino entre u y v.
89
90     void sumarACamino(int u, int v, long long delta) {
91         while (hld.cadenaTope[u] != hld.cadenaTope[v]) {
92             if (hld.profundidad[hld.cadenaTope[u]] <
93                 hld.profundidad[hld.cadenaTope[v]])
94                 swap(u, v);
95
96             int cima = hld.cadenaTope[u];
97             seg->actualizar(hld.pos[cima], hld.pos[u], delta);
98             u = hld.padre[cima];
99         }
100
101         if (hld.profundidad[u] > hld.profundidad[v]) swap(u, v);
102         seg->actualizar(hld.pos[u], hld.pos[v], delta);
103     } // O(log^2 n)
104
105     // READ: maximo en el camino entre u y v.

```

```

99     long long maxCamino(int u, int v) {
100         long long mejor = LLONG_MIN;
101
102         while (hld.cadenaTope[u] != hld.cadenaTope[v]) {
103             if (hld.profundidad[hld.cadenaTope[u]] <
104                 hld.profundidad[hld.cadenaTope[v]])
105                 swap(u, v);
106
107             int cima = hld.cadenaTope[u];
108             mejor = max(mejor, seg->query(hld.pos[cima], hld.pos[
109                 u]));
110             u = hld.padre[cima];
111         }
112
113         if (hld.profundidad[u] > hld.profundidad[v]) swap(u, v);
114         mejor = max(mejor, seg->query(hld.pos[u], hld.pos[v]));
115
116         return mejor;
117     } // O(log^2 n)
118
119     // READ: maximo en el subarbol de u (un solo rango contiguo).
120     long long maxSubarbol(int u) {
121         return seg->query(hld.pos[u], hld.pos[u] + hld.tam[u] -
122             1);
123     } // O(log n)
124 };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0(val=5)
2 //             /      \
3 //          1(val=3)    2(val=8)
4 //             /      \
5 //          3(val=1)    4(val=9)
6 //             /
7 //          5(val=2)
8 //
9 // subarbol de 1 (tam=4) > subarbol de 2 (tam=1) -> 0-1 pesada
10 // subarbol de 3 (tam=2) > subarbol de 4 (tam=1) -> 1-3 pesada
11 // cadena principal: 0-1-3-5; cadenas livianas: {4}, {2}
12
13 vector<vector<int>> adj(6);
14 adj[0] = {1, 2};
15 adj[1] = {0, 3, 4};
16 adj[2] = {0};
17 adj[3] = {1, 5};
18 adj[4] = {1};
19 adj[5] = {3};
20
21 vector<int> valorInicial = {5, 3, 8, 1, 9, 2};
22
23 HLDegTree hst(adj, valorInicial, 0, 6);
24
25 cout << hst.maxCamino(5, 4);
26 // 9 (camino 5-3-1-4, valores 2,1,3,9 -> maximo 9)
27
28 hst.sumarACamino(5, 4, 10); // suma 10 a TODO el camino 5-4
29 // (nodos 5,3,1,4)
30
31 cout << hst.maxCamino(0, 2);
32 // 8 (camino 0-2, valores 5,8 -- NO afectado por el update
33 // anterior, ya que 0 y 2 no estan en el camino 5-4)

```

Regla práctica: usa HLD + Fenwick cuando la operación de camino es **suma** y no necesitas update de rango sobre un camino completo (solo updates puntuales); cambia a HLD + Segment Tree en cuanto aparezca max/min, o en cuanto el problema pida "aplica esto a **todo** el camino de una vez" —el costo extra de Segment Tree con lazy sobre Fenwick se justifica exactamente ahí. La estructura de HLD en sí (pos, cadenaTope, subir cadena por cadena) es idéntica en ambos casos; solo cambia qué estructura de rango envuelve el arreglo aplanado.

17 Disjoint Set Union

17.1 Union-Find

Idea: mantener una partición de n elementos en conjuntos disjuntos, soportando dos operaciones: **unir** dos conjuntos y **preguntar** si dos elementos están en el mismo conjunto. Cada conjunto se representa como un

árbol implícito, donde cada nodo apunta a su padre y la raíz es el "representante" del conjunto. **Path compression** (aplanar el camino hacia la raíz cada vez que se consulta) y **union by rank/size** (colgar siempre el árbol más chico del más grande) son las dos optimizaciones que garantizan complejidad casi constante amortizada.

¿Por qué usarlo? Cuando el problema pide agrupar elementos dinámicamente por una relación de equivalencia —conectividad en un grafo que solo agrega aristas (nunca quita), componentes conexas online, Kruskal para MST, detectar ciclos al agregar aristas— y no necesitas nada más fino que "¿están en el mismo grupo?" o "únelos".

Casos de uso:

- Kruskal: construir el MST agregando aristas en orden de peso, usando `find` para evitar ciclos.
- Componentes conexas de un grafo que solo recibe aristas nuevas (offline o online), sin necesidad de eliminarlas.
- Detectar si agregar una arista crea un ciclo.
- Agrupar elementos por una relación de equivalencia dada incrementalmente (ej. "*a* y *b* son el mismo elemento").

Complejidad: $O(\alpha(n))$ amortizado por operación (`find` o `union`), donde α es la inversa de Ackermann —en la práctica, constante (≤ 4 para cualquier n razonable). $O(n)$ para inicializar.

```

1 struct DSU {
2     vector<int> padre, rango, tamano;
3     int componentes;
4
5     // CREATE: n elementos, cada uno en su propio conjunto.
6     DSU(int n) {
7         padre.resize(n);
8         iota(padre.begin(), padre.end(), 0);
9         rango.assign(n, 0);
10        tamano.assign(n, 1);
11        componentes = n;
12    } // O(n)
13
14    // READ: representante del conjunto de x, con path
15    // compression.
16    int find(int x) {
17        if (padre[x] != x)
18            padre[x] = find(padre[x]);
19        return padre[x]; // O(alpha(n))
20    } amortizado
21
22    // WRITE: une los conjuntos de x e y (union by rank).
23    // Retorna false si ya estaban en el mismo conjunto.
24    bool unir(int x, int y) {
25        int rx = find(x), ry = find(y);
26        if (rx == ry) return false;
27
28        if (rango[rx] < rango[ry]) swap(rx, ry);
29        padre[ry] = rx;
30        tamano[rx] += tamano[ry];
31        if (rango[rx] == rango[ry]) rango[rx]++;
32
33        componentes--;
34        return true; // O(alpha(n))
35    } amortizado
36
37    // READ: ¿x e y están en el mismo conjunto?
38    bool mismoConjunto(int x, int y) {
39        return find(x) == find(y); // O(alpha(n))
40    } amortizado
41
42    // READ: tamaño del conjunto al que pertenece x.
43    int tamanoDe(int x) {
44        return tamano[find(x)];
45    }
46 }
```

```

44 } // O(alpha(n))
45 };
46 
```

Ejemplo de funcionamiento:

```

1 // 6 elementos: 0,1,2,3,4,5
2
3 DSU dsu(6);
4
5 dsu.unir(0, 1); // {0,1} {2} {3} {4} {5}
6 dsu.unir(2, 3); // {0,1} {2,3} {4} {5}
7 dsu.unir(0, 2); // {0,1,2,3} {4} {5}
8
9 cout << dsu.mismoConjunto(1, 3); // true (1 y 3 quedaron en
10 // {0,1,2,3})
11 cout << dsu.mismoConjunto(1, 4); // false (4 sigue solo)
12
13 cout << dsu.tamanoDe(3); // 4 (tamano de {0,1,2,3})
14
15 cout << dsu.componentes; // 3 ({0,1,2,3}, {4}, {5})
16
17 dsu.unir(4, 5); // {0,1,2,3} {4,5}
18 cout << dsu.componentes; // 2
19 
```

Regla práctica: siempre implementa **ambas** optimizaciones juntas (path compression + union by rank/size); por separado cada una da $O(\log n)$ amortizado, pero combinadas dan $O(\alpha(n))$, prácticamente $O(1)$. Usa `tamano[]` en vez de (o además de) `rango[]` si el problema pregunta directamente por el tamaño de un componente —así te ahorras una estructura extra. Si necesitas **deshacer** uniones (rollback) no uses path compression con compresión recursiva agresiva: esa variante rompe la reversión por puntero; para DSU con rollback, une siempre por rango/tamaño sin comprimir camino, y guarda un log de los cambios para poder revertirlos en orden inverso.

17.2 Path Compression

Idea: cada vez que `find(x)` sube hasta la raíz del árbol, aprovecha ese recorrido para **reenganchar directamente** a la raíz todos los nodos visitados en el camino. Así, la próxima vez que se consulte cualquiera de esos nodos, el camino hacia la raíz será de longitud 1 en vez de potencialmente $O(n)$. Es una optimización de **lectura**: no cambia qué conjuntos existen, solo aplanar la estructura interna que los representa.

¿Por qué usarlo? Sin esta optimización, una secuencia de uniones adversaria (ej. siempre unir el nuevo elemento como hijo del anterior) degenera el árbol en una cadena, y cada `find` cuesta $O(n)$. Con path compression, aunque el árbol se degenere, la primera consulta lo aplanar y las siguientes son $O(1)$.

```

1 // find CON path compression (recursivo): cada nodo en el camino
2 // queda apuntando directo a la raíz al retornar de la recursión.
3 int find(int x) {
4     if (padre[x] != x)
5         padre[x] = find(padre[x]); // reenganche directo a la
6         // raíz
7     return padre[x];
8 }
9
10 // Version iterativa (evita stack overflow con n muy grande):
11 // primero encuentra la raíz, luego recorre de nuevo aplanando.
12 int findIterativo(int x) {
13     int raiz = x;
14     while (padre[raiz] != raiz) raiz = padre[raiz];
15
16     while (padre[x] != raiz) {
17         int siguiente = padre[x];
18         padre[x] = raiz;
19         x = siguiente;
20     }
21     return raiz; // O(alpha(n))
22 }
23 
```

Ejemplo de funcionamiento:

```

1 // Antes de find(4):
2 // 0 <- 1 <- 2 <- 3 <- 4      (cadena degenerada, sin union by
//                               rank)
3 //
4 // find(4) sube: 4 -> 3 -> 2 -> 1 -> 0 (raiz), y al retornar de
// la
5 // recursion reengancha a cada uno directo a 0:
6 //
7 // Despues de find(4):
8 // 0 <- 1
9 // 0 <- 2
10 // 0 <- 3
11 // 0 <- 4
12 //
13 // La proxima llamada a find(3) (o find(2), find(1)) es O(1).

```

Regla práctica: implementa siempre la versión recursiva de una línea (`padre[x] = find(padre[x])`) salvo que n sea tan grande que el stack de recursión pueda desbordarse (típicamente $n > 10^6$ en un juez con stack chico); en ese caso usa la versión iterativa de dos pasadas. Path compression por sí sola ya da $O(\log n)$ amortizado incluso sin union by rank —pero combinarla con la siguiente optimización es lo que baja a $O(\alpha(n))$.

17.3 Union by Size / Rank

Idea: al unir dos conjuntos, en vez de colgar uno del otro arbitrariamente, cuelga siempre el árbol **más chico** debajo de la raíz del **más grande**. "Tamaño" cuenta nodos reales; "rango" es una cota superior de la altura del árbol (útil cuando no importa el tamaño exacto, solo mantener el árbol bajo). Ambas variantes logran el mismo efecto asintótico; `tamano[]` es preferible cuando el problema además pregunta por el tamaño de un componente.

¿Por qué usarlo? Sin esta optimización, aunque uses path compression, una secuencia de uniones puede seguir generando árboles de altura $O(\log n)$ en el peor caso repetido antes de que la compresión los aplane; con union by rank/size, la altura del árbol está acotada por $O(\log n)$ desde el principio, incluso antes de que ocurra ninguna compresión.

```

1 // UNION BY RANK: rango[r] es una COTA SUPERIOR de la altura del
2 // arbol con raiz r (no el tamaño real). Solo crece cuando se
// unen
3 // dos arboles de igual rango.
4 bool unirPorRango(int x, int y) {
5     int rx = find(x), ry = find(y);
6     if (rx == ry) return false;
7
8     if (rango[rx] < rango[ry]) swap(rx, ry);
9     padre[ry] = rx;
10    if (rango[rx] == rango[ry]) rango[rx]++;
11
12    return true;
13 }                                     // O(alpha(n))
14                                     amortizado
15
16 // UNION BY SIZE: tamano[r] es el número REAL de nodos en el
// arbol
17 // con raiz r. Además de balancear, deja el dato de tamaño listo
18 // para consultarlo directamente (tamanoDe(x) = tamano[find(x)]).
19 bool unirPorTamano(int x, int y) {
20     int rx = find(x), ry = find(y);
21     if (rx == ry) return false;
22
23     if (tamano[rx] < tamano[ry]) swap(rx, ry);
24     padre[ry] = rx;
25     tamano[rx] += tamano[ry];
26
27    return true;
28 }                                     // O(alpha(n))
29                                     amortizado

```

Ejemplo de funcionamiento:

```

1 // Sin union by size (colgando siempre el segundo argumento bajo
// el

```

```

2 // primero, sin comparar tamanos):
3 // unir(0,1); unir(0,2); unir(0,3); unir(0,4);
4 // -> 0 tiene 4 hijos directos: arbol de altura 1. Bien en este
5 // caso, pero...
6 // unir(0,1); unir(1,2); unir(2,3); unir(3,4);
7 // -> 0 <- 1 <- 2 <- 3 <- 4 : cadena de altura 4. PEOR CASO.
8 //
9 // Con union by size, la segunda secuencia queda:
10 // unir(0,1): tam iguales (1,1) -> 1 cuelga de 0. tam[0]=2
11 // unir(1,2): find(1)=0 (tam 2) vs find(2)=2 (tam 1)
12 // -> 2 cuelga de 0 (el mas chico bajo el mas grande)
13
14 // unir(2,3): find(2)=0 (tam 3) vs find(3)=3 (tam 1)
15 // -> 3 cuelga de 0
16 // unir(3,4): analogo -> 4 cuelga de 0
17 // Resultado: 0 con 4 hijos directos, altura 1, SIN IMPORTAR el
18 // orden en que llegaron las uniones.

```

Regla práctica: nunca compares `rango[]` y `tamano[]` a la vez ni los mezcles en el mismo `unir` —son dos estrategias equivalentes, elige una. Si el problema pide en algún momento "tamaño del componente de x ", usa union by size directamente y ahorras una estructura aparte; si no, union by rank es marginalmente más barato de mantener. Cualquiera de las dos, combinada con path compression, es lo que garantiza $O(\alpha(n))$ —usar solo una de las dos optimizaciones dejaría el DSU en $O(\log n)$ amortizado en vez de prácticamente $O(1)$.

17.4 Unión de Conjuntos Disjuntos en un Árbol

Idea: en vez de usar DSU para unir componentes de un grafo general, se usa la misma técnica de **union by size** (fusionar siempre la estructura más chica dentro de la más grande) pero aplicada a las estructuras de datos que cuelgan de cada nodo de un **árbol** (ej. un **set**, **map** o contador de frecuencias por subárbol). Esto se conoce como **small-to-large merging** o **DSU on Tree**: al procesar cada nodo de abajo hacia arriba, se fusiona la estructura del hijo más chico dentro de la del hijo más grande, nunca al revés.

¿Por qué usarlo? Resuelve queries del tipo "para cada nodo u , calcula algo sobre **todo su subárbol**" (ej. "¿cuántos colores distintos hay en el subárbol de u ?") sin pagar $O(n)$ por cada nodo (lo que daría $O(n^2)$ total). Fusionando siempre lo chico dentro de lo grande, cada elemento individual se "mueve" de estructura a lo sumo $O(\log n)$ veces en todo el algoritmo, igual que el argumento de complejidad de union by size en DSU clásico.

Casos de uso:

- Contar colores/valores distintos en el subárbol de cada nodo.
- Encontrar el valor más frecuente (o la suma de los k más frecuentes) en el subárbol de cada nodo.
- Cualquier agregado por subárbol que se pueda mantener en un **map/set/contador** y sea fusionable elemento a elemento.

Complejidad: $O(n \log n)$ total (cada uno de los n elementos se reinserta $O(\log n)$ veces, cada inserción $O(\log n)$ o $O(1)$ amortizado según la estructura usada).

```

1 struct DSUEnArbol {
2     vector<vector<int>>> hijos;
3     vector<int> color;
4     vector<map<int,int>*> cnt;    // cnt[u] = frecuencia de
//                               colores
5                                     // en el subarbol de u (solo

```

```

6 // vive mientras se procesa)
7 vector<long long> respuesta; // respuesta[u] = # colores
8 // distintos en subarbol de u
9 int n;
10
11 DSUEnArbol(vector<vector<int>>& hijos_, vector<int>& color_)
12 : hijos(hijos_), color(color_), n(color_.size()) {
13     cnt.assign(n, nullptr);
14     respuesta.assign(n, 0);
15 }
16
17 // WRITE: procesa el subarbol de u y deja el resultado en
18 // respuesta[u]; cnt[u] queda con la estructura fusionada.
19 void procesar(int u) {
20     int hijoGrande = -1, tamMax = -1;
21     for (int hijo : hijos[u]) {
22         procesar(hijo);
23         if ((int)cnt[hijo]->size() > tamMax) {
24             tamMax = cnt[hijo]->size();
25             hijoGrande = hijo;
26         }
27     }
28
29     // Reutiliza la estructura del hijo mas grande (no la
30     // copia)
31     if (hijoGrande != -1) cnt[u] = cnt[hijoGrande];
32     else cnt[u] = new map<int,int>();
33
34     // Funde el resto de los hijos (los "chicos") dentro de
35     // cnt[u]
36     for (int hijo : hijos[u]) {
37         if (hijo == hijoGrande) continue;
38         for (auto [c, freq] : *cnt[hijo])
39             (*cnt[u])[c] += freq;
40         delete cnt[hijo];
41     }
42
43     (*cnt[u])[color[u]]++;
44     respuesta[u] = cnt[u]->size(); // O(n log n)
45 }
46 };

```

Ejemplo de funcionamiento:

```

1 // arbol:          0(color=1)
2 //                /      \
3 //              1(color=2) 2(color=1)
4 //                /
5 //              3(color=2)
6
7 vector<vector<int>> hijos = {{1, 2}, {3}, {}, {}};
8 vector<int> color = {1, 2, 1, 2};
9
10 DSUEnArbol d(hijos, color);
11 d.procesar(0);
12
13 cout << d.respuesta[3]; // 1 (subarbol {3}: solo color 2)
14 cout << d.respuesta[1]; // 1 (subarbol {1,3}: colores {2,2} ->
15 // 1 distinto)
16 cout << d.respuesta[2]; // 1 (subarbol {2}: solo color 1)
17 cout << d.respuesta[0]; // 2 (subarbol {0,1,2,3}: colores
18 // {1,2,1,2} -> 2 distintos)

```

Regla práctica: el error más común es fusionar copiando **todas** las estructuras hijas en una nueva; eso rompe la complejidad y degenera a $O(n^2)$. La clave es **reutilizar por puntero/referencia** la estructura del hijo más grande y solo insertar uno por uno los elementos de los hijos chicos —exactamente el mismo principio que "cuelga el árbol chico del grande" en DSU clásico, aplicado a estructuras de datos en vez de a punteros padre[].

17.5 DSU Offline

Idea: cuando las queries de conectividad no necesitan responderse en el momento en que llegan, sino que se pueden procesar en un **orden distinto al de entrada** (offline), muchas veces conviene reordenarlas para que DSU las resuelva todas con uniones **puramente incrementales** (sin necesitar deshacer nada). Dos patrones clásicos:

1. **LCA offline (algoritmo de Tarjan):** dado un árbol y una lista de pares (u, v) a los que hay que responder el LCA, se recorre el árbol una sola vez con DFS y se resuelve **cada query en el momento justo**, usando DSU para saber a qué "ya visitado más cercano a la raíz" apunta cada nodo.
2. **Conectividad con "borrados" (offline dynamic connectivity):** si las aristas se agregan y eliminan en algún orden, pero todas las queries se conocen de antemano, se puede invertir el problema: procesar las eliminaciones **hacia atrás** como si fueran inserciones, evitando así necesitar un DSU con rollback.

¿Por qué usarlo? DSU no soporta eliminar una unión de forma económica. Si el problema exige eliminaciones pero **todas las queries están dadas de antemano** (offline), reordenar el trabajo para que DSU solo vea inserciones evita la necesidad de una estructura más compleja (link-cut trees) o de limitarse a rollback con pila.

Casos de uso:

- Responder Q consultas de LCA sobre un árbol fijo en $O((n+Q)\alpha(n))$ en vez de $O(Q \log n)$ con binary lifting (más simple de codear si de todas formas ya tienes DSU).
- "¿En qué momento u y v quedaron conectados?" con aristas que se agregan Y se quitan, dado que se conoce toda la secuencia de operaciones y queries por adelantado.
- Procesar queries de un grafo dinámico ordenándolas por algún criterio (ej. Kruskal offline: ordenar aristas por peso y responder "¿ u, v conectados usando solo aristas de peso $\leq w$?" para varios w dados).

Complejidad: $O((n+Q)\alpha(n))$ para LCA offline; $O((n+m)\alpha(n) \log m)$ para conectividad offline con borrados invertidos (el $\log m$ extra viene de reordenar/indexar eventos, no de DSU en sí).

```

1 // TARJAN OFFLINE LCA: resuelve todas las queries con un solo DFS.
2
3 struct LCAOffline {
4     vector<vector<int>> adj;
5     vector<vector<pair<int,int>>> queries; // queries[u] = {(v, idQuery)}
6     vector<int> ancestro; // ancestro[find(x)] = LCA actual
7     vector<bool> visitado;
8     vector<int> respuesta;
9     DSU dsu;
10    int n;
11
12    LCAOffline(vector<vector<int>>& adj_, int n_)
13    : adj(adj_), n(n_), dsu(n_) {
14        queries.assign(n, {});
15        ancestro.assign(n, 0);
16        visitado.assign(n, false);
17    }
18
19    // WRITE: registra una consulta LCA(u, v) con identificador id.
20    void agregarQuery(int u, int v, int id, int numQueries) {
21        if ((int)respuesta.size() < numQueries)
22            respuesta.assign(numQueries, -1);
23        queries[u].push_back({v, id});
24        queries[v].push_back({u, id});
25    }
26
27    // READ: procesa el subarbol de u; al terminar cada hijo, lo
28    // fusiona a u y marca a u como el "representante" actual.
29    void dfs(int u, int padre) {
30        visitado[u] = true;

```

```

30     ancestro[dsu.find(u)] = u;
31
32     for (int hijo : adj[u]) {
33         if (hijo == padre) continue;
34         dfs(hijo, u);
35         dsu.unir(u, hijo);
36         ancestro[dsu.find(u)] = u;    // u sigue siendo el
representante
37     }
38
39     // Resuelve toda query (u, v) donde v YA fue visitado: el
LCA
40     // es el ancestro del componente actual de v.
41     for (auto [v, id] : queries[u]) {
42         if (visitado[v])
43             respuesta[id] = ancestro[dsu.find(v)];
44     }
45 }
46 // O((n+Q) alpha(n))
};

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //            / \
3 //           1  2
4 //          /
5 //         3
6
7 vector<vector<int>> adj(4);
8 adj[0] = {1, 2}; adj[1] = {0, 3}; adj[2] = {0}; adj[3] = {1};
9
10 LCAOffline lca(adj, 4);
11 lca.agregarQuery(3, 2, /*id=*/0, /*numQueries=*/1);
12 // pregunta: LCA(3, 2)?
13
14 lca.dfs(0, -1);
15 cout << lca.respuesta[0]; // 0 (LCA(3,2) = 0)
16
17 // Traza: dfs(0) marca ancestro[0]=0 -> baja a dfs(1) -> dfs(3):
18 // marca ancestro[3]=3, no tiene queries pendientes con 2 (2 aun
no
19 // visitado) -> sube, dsu.unir(1,3), ancestro[find(1)]=1 -> sube
a 0,
20 // dsu.unir(0,1), ancestro[find(0)]=0 -> dfs(2): marca ancestro
[2]=2,
21 // ve query (2, 3): 3 YA fue visitado -> respuesta = ancestro[
find(3)]
22 // = ancestro[0] = 0. Correcto.

```

Regla práctica: DSU offline sirve **solo** cuando todas las queries se conocen antes de empezar a procesar; si las queries llegan intercaladas con las operaciones y hay que responder cada una en el momento (online), esta técnica no aplica y hace falta una estructura que soporte deshacer (DSU con rollback) o una más potente (link-cut trees). La clave común a los dos patrones de esta sección es la misma: **reordenar el trabajo** (ya sea "resolver la query en el instante justo del DFS" o "invertir las eliminaciones para que sean inserciones") de forma que DSU nunca necesite deshacer una unión.

17.6 DSU Rollback

Idea: una versión de DSU que soporta **deshacer** uniones en orden inverso (como una pila), a costa de renunciar a **path compression**. Cada unir exitoso se registra en un log con la información mínima para revertirlo (qué nodo era raíz de qué, y su rango/tamaño antes de la unión); **deshacer()** saca la última entrada del log y restaura exactamente ese estado. Solo union by rank/size mantiene la altura $O(\log n)$ sin compresión, así que **find** sigue siendo $O(\log n)$ (no $O(\alpha(n))$) en esta variante.

¿Por qué usarlo? Cuando las uniones deben revertirse en algún momento —típicamente porque se está explorando un espacio de estados con backtracking (ej. "prueba agregar esta arista, resuelve el subproblema, deshazla y prueba la siguiente") o porque se procesa un **divide and conquer sobre el tiempo** (offline dynamic

connectivity con inserciones y eliminaciones intercaladas, sin poder invertir el orden como en la sección anterior). Es la alternativa a DSU offline cuando las queries **no** pueden reordenarse libremente.

Casos de uso:

- Backtracking sobre un conjunto de aristas: probar conectividad agregando aristas temporalmente y deshaciendo al volver atrás.
- Offline dynamic connectivity con **segment tree sobre el tiempo**: cada arista vive en un rango de tiempo $[l, r)$; se recorre el segment tree por tiempo, agregando la arista al entrar a un nodo y deshaciéndola (rollback) al salir.
- Cualquier algoritmo tipo "exploración + backtrack" donde la estructura de conectividad debe volver exactamente al estado anterior, no solo a uno equivalente.

Complejidad: $O(\log n)$ por find/unir (sin path compression); $O(1)$ por operación de deshacer (solo restaura los valores guardados en el log).

```

1 struct DSURollback {
2     vector<int> padre, rango, tamaño;
3     int componentes;
4
5     // Log de cambios: para cada union exitosa, guarda (hijo,
raizVieja,
6     // rangoRaizNueva) suficiente para revertirla.
7     struct Cambio { int hijo, raizVieja, rangoRaizNueva; };
8     vector<Cambio> log;
9
10    DSURollback(int n) {
11        padre.resize(n);
12        iota(padre.begin(), padre.end(), 0);
13        rango.assign(n, 0);
14        tamaño.assign(n, 1);
15        componentes = n;
16    }
17    // O(n)
18
19    // READ: find SIN path compression (imprescindible para poder
deshacer despues).
20    int find(int x) {
21        while (padre[x] != x) x = padre[x];
22        return x;
23    }
24    // O(log n)
25
26    // WRITE: une x e y; si tuvo efecto, deja un registro en el
log.
27    // Retorna false (y no registra nada) si ya estaban unidos.
28    bool unir(int x, int y) {
29        int rx = find(x), ry = find(y);
30        if (rx == ry) {
31            log.push_back({-1, -1, -1}); // marcador "union
nula"
32            return false;
33        }
34        if (rango[rx] < rango[ry]) swap(rx, ry);
35        // rx queda como raiz nueva; ry cuelga de rx
36
37        log.push_back({ry, ry, rango[rx]}); // guarda estado
previo
38        padre[ry] = rx;
39        tamaño[rx] += tamaño[ry];
40        if (rango[rx] == rango[ry]) rango[rx]++;
41
42        componentes--;
43        return true;
44    }
45    // O(log n)
46
47    // WRITE: deshace la ultima union realizada (haya sido
efectiva
48    // o nula). Debe llamarse en el orden EXACTAMENTE inverso al
de
49    // las uniones.
50    void deshacer() {
51        auto [hijo, raizVieja, rangoRaizNueva] = log.back();
52        log.pop_back();
53        if (hijo == -1) return; // era una union nula, no hay
nada que revertir

```

```

53     int raiz = padre[hijo];
54     tamano[raiz] -= tamano[hijo];
55     rango[raiz] = rangoRaizNueva;
56     padre[hijo] = hijo;
57     componentes++;
58 } // O(1)
59
60 bool mismoConjunto(int x, int y) {
61     return find(x) == find(y);
62 } // O(log n)
63 }
64

```

Ejemplo de funcionamiento:

```

1 DSURollback dsu(5);
2
3 dsu.unir(0, 1); // {0,1} {2} {3} {4}
4 dsu.unir(1, 2); // {0,1,2} {3} {4}
5 cout << dsu.mismoConjunto(0, 2); // true
6
7 dsu.deshacer(); // deshace unir(1,2) -> {0,1} {2}
8 // {3} {4}
9 cout << dsu.mismoConjunto(0, 2); // false
10 cout << dsu.mismoConjunto(0, 1); // true (esta union sigue
11 // intacta)
12 dsu.deshacer(); // deshace unir(0,1) -> {0} {1} {2}
13 // {3} {4}
14 cout << dsu.componentes; // 5 (vuelta al estado inicial)

```

Regla práctica: nunca mezcles path compression con DSU rollback —comprimir caminos modifica punteros de nodos que no participaron directamente en la unión actual, y esos cambios no quedan registrados en el log, así que `deshacer()` dejaría la estructura en un estado inconsistente. La disciplina de uso también importa: los `deshacer()` deben llamarse en orden **estrictamente inverso** a como se hicieron los `unir()` (como una pila), nunca saltados ni en otro orden —por eso esta técnica combina naturalmente con recursión (DFS de backtracking, divide and conquer sobre el tiempo), donde ese orden inverso surge solo al volver de cada llamada.

18 Graph Decomposition

18.1 Bridge Tree

Idea: un **punto** (bridge) es una arista de un grafo no dirigido que, al eliminarla, aumenta el número de componentes conexas. El **Bridge Tree** (también llamado "árbol de 2-edge-connected components") colapsa cada componente **2-arista-conexa** (un subgrafo maximal sin puentes internos: para eliminar cualquier arista dentro de él, se sigue estando conectado) en un único nodo, y conecta esos nodos únicamente con las aristas que eran puentes en el grafo original. El resultado es siempre un **árbol** (o bosque, si el grafo original era desconexo): no puede tener ciclos, porque cualquier ciclo entre componentes implicaría un camino alternativo que contradice que esas aristas eran puentes.

¿Por qué usarlo? Muchos problemas sobre un grafo general se vuelven triviales sobre un árbol (caminos únicos, LCA, HLD, etc.). Si el problema tiene que ver con puentes, conectividad robusta a fallas de una arista, o "¿cuántas aristas hay que quitar como mínimo para desconectar u de v ", reducir el grafo a su Bridge Tree convierte el problema en uno sobre árboles, donde ya se tiene todo el arsenal de HLD/LCA/etc.

Casos de uso:

- Encontrar todos los puentes de un grafo.

- "¿Cuántos puentes hay en el camino entre u y v ?" (equivalente a la distancia entre sus nodos en el Bridge Tree).
- Determinar si el grafo sigue conexo tras eliminar una arista específica (lo está si esa arista no es puente).
- Como paso de preprocesamiento antes de aplicar HLD o técnicas de árbol sobre un grafo que originalmente tiene ciclos.

Complejidad: $O(n + m)$ para encontrar los puentes (DFS con tiempos de descubrimiento y low-link, estilo Tarjan); $O(n + m)$ adicional para construir el árbol colapsando componentes (DFS/BFS de flood fill evitando cruzar puentes).

```

1 struct BridgeTree {
2     int n;
3     vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
4         idArista)}
5     vector<int> disc, low, comp;
6     vector<bool> esPunto, visitado;
7     int timer = 0, numComponentes = 0;
8     vector<vector<int>> arbol; // lista de adyacencia
9         del bridge tree
10
11     BridgeTree(int n_, vector<pair<int,int>>& aristas) : n(n_) {
12         adj.assign(n, {});
13         for (int i = 0; i < (int)aristas.size(); i++) {
14             auto [u, v] = aristas[i];
15             adj[u].push_back({v, i});
16             adj[v].push_back({u, i});
17         }
18         disc.assign(n, -1);
19         low.assign(n, -1);
20         esPunto.assign(aristas.size(), false);
21         comp.assign(n, -1);
22         visitado.assign(n, false);
23
24         for (int u = 0; u < n; u++)
25             if (disc[u] == -1) dfsPuentes(u, -1);
26
27         for (int u = 0; u < n; u++)
28             if (comp[u] == -1) marcarComponente(u, numComponentes
29                 ++);
30
31         arbol.assign(numComponentes, {});
32         for (int i = 0; i < (int)aristas.size(); i++) {
33             if (!esPunto[i]) continue;
34             auto [u, v] = aristas[i];
35             arbol[comp[u]].push_back(comp[v]);
36             arbol[comp[v]].push_back(comp[u]);
37         }
38     } // O(n + m)
39
40     // READ (interno): DFS de Tarjan para marcar puentes via low-
41     // link.
42     void dfsPuentes(int u, int idAristaPadre) {
43         disc[u] = low[u] = timer++;
44         for (auto [v, id] : adj[u]) {
45             if (id == idAristaPadre) continue; // no reusar la
46             MISMA arista
47             // (si, no
48             solo el mismo nodo:
49             // por multi-
50             aristas)
51             if (disc[v] == -1) {
52                 dfsPuentes(v, id);
53                 low[u] = min(low[u], low[v]);
54                 if (low[v] > disc[u]) esPunto[id] = true;
55             } else {
56                 low[u] = min(low[u], disc[v]);
57             }
58         }
59     }
60
61     // READ (interno): flood fill que NUNCA cruza un puente, para
62     // agrupar cada componente 2-arista-conexa en un mismo id.
63     void marcarComponente(int u, int id) {
64         comp[u] = id;
65         for (auto [v, idArista] : adj[u]) {
66             if (esPunto[idArista] || comp[v] != -1) continue;
67             marcarComponente(v, id);
68         }
69     }
70 };

```

Ejemplo de funcionamiento:

```

1 // grafo:  0 -- 1      4 -- 5
2 //          |      |      |
3 //          2 -- 3 ----- 6
4 //
5 // aristas: (0,1)(0,2)(1,3)(2,3)  ciclo 0-1-3-2-0, sin puentes
              internos
6 //          (3,6) puente          conecta el ciclo con el otro
              lado
7 //          (4,6)(4,5)(5,6)      ciclo 4-5-6-4, sin puentes
              internos
8
9 vector<pair<int,int>> aristas = {
10     {0,1}, {0,2}, {1,3}, {2,3}, // ciclo A: nodos 0,1,2,3
11     {3,6},                      // PUENTE
12     {4,6}, {4,5}, {5,6}         // ciclo B: nodos 4,5,6
13 };
14
15 BridgeTree bt(7, aristas);
16
17 cout << bt.numComponentes; // 2
18 // comp[0]=comp[1]=comp[2]=comp[3] = A (mismo id)
19 // comp[4]=comp[5]=comp[6]      = B (mismo id)
20
21 // bt.arbol tiene exactamente 1 arista: A -- B (la del puente 3-6)
22 // -> es literalmente un arbol de 2 nodos.

```

Regla práctica: al hacer el DFS de Tarjan, compara por **índice de arista** ($\text{id} == \text{idAristaPadre}$), no por "nodo padre" ($v == \text{padre}$) —si el grafo tiene aristas múltiples entre el mismo par de nodos, comparar por nodo padre ignoraría incorrectamente la segunda arista paralela y la trataría como "la misma que ya usé", perdiendo un puente real. Una vez construido el Bridge Tree, cualquier técnica de árbol (LCA, distancia entre nodos, HLD) se le puede aplicar directamente tratando cada componente 2-arista-conexa como un solo nodo —el número de puentes en el camino $u-v$ del grafo original es exactamente la distancia entre $\text{comp}[u]$ y $\text{comp}[v]$ en este árbol.

18.2 Block-Cut Tree

Idea: análogo al Bridge Tree pero para **vértices** en vez de aristas. Un **punto de articulación** (articulation point / cut vertex) es un nodo cuya eliminación aumenta el número de componentes conexas. Una **componente biconexa** (block) es un subgrafo maximal sin puntos de articulación internos. A diferencia del Bridge Tree, aquí un mismo punto de articulación puede pertenecer a **varios** blocks a la vez, así que el árbol resultante es **bipartito**: tiene dos tipos de nodos —los puntos de articulación originales, y un nodo nuevo por cada block— y toda arista del árbol conecta un punto de articulación con un block al que pertenece.

¿Por qué usarlo? Cuando el problema pregunta por robustez a la falla de un **vértice** (en vez de una arista), o pide identificar puntos de articulación/componentes biconexas y luego razonar sobre cómo se relacionan entre sí (ej. "¿cuántos puntos de articulación hay que atravesar para ir de u a v ?"). Es el análogo natural de Bridge Tree un nivel más fino.

Casos de uso:

- Encontrar todos los puntos de articulación y componentes biconexas de un grafo.
- "¿Cuántos puntos de articulación separan a u de v ?" (mitad de la distancia entre sus nodos en el Block-Cut Tree).

- Determinar si el grafo sigue conexo tras eliminar un vértice específico.
- Modelar problemas de "vulnerabilidad de red" donde fallan nodos (routers, ciudades) en vez de conexiones.

Complejidad: $O(n + m)$ para encontrar puntos de articulación y blocks (DFS con low-link, estilo Tarjan, usando una pila de aristas); $O(n + m)$ adicional para construir el árbol.

```

1 struct BlockCutTree {
2     int n;
3     vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
4         idArista)}
5     vector<int> disc, low;
6     vector<bool> esArticulacion, visitadaArista;
7     vector<vector<int>> blocks; // blocks[i] = nodos
8         ORIGINALES del block i
9     stack<int> pilaAristas; // pila de idArista, no
10         de nodos
11     stack<pair<int,int>> pilaAristasUV; // (u,v) correspondiente
12         a cada id en la pila
13     int timer = 0;
14
15     // en el arbol final: nodos [0, n) = puntos de articulation
16     // (activos solo si esArticulacion[u]); nodos [n, n+blocks.
17         size())
18     // = un nodo por cada block.
19     vector<vector<int>> arbol;
20
21     BlockCutTree(int n_, vector<pair<int,int>>& aristas) : n(n_)
22     {
23         adj.assign(n, {});
24         for (int i = 0; i < (int)aristas.size(); i++) {
25             auto [u, v] = aristas[i];
26             adj[u].push_back({v, i});
27             adj[v].push_back({u, i});
28         }
29         disc.assign(n, -1);
30         low.assign(n, -1);
31         esArticulacion.assign(n, false);
32         visitadaArista.assign(aristas.size(), false);
33
34         for (int u = 0; u < n; u++)
35             if (disc[u] == -1) dfs(u, -1);
36
37         construirArbol(); // O(n + m)
38     }
39
40     // READ (interno): DFS de Tarjan; cada vez que se cierra un
41     // block
42     // (low[v] >= disc[u]) se hace pop de la pila hasta esa
43     // arista.
44     void dfs(int u, int idAristaPadre) {
45         disc[u] = low[u] = timer++;
46         int hijos = 0;
47
48         for (auto [v, id] : adj[u]) {
49             if (visitadaArista[id]) continue;
50             visitadaArista[id] = true;
51             pilaAristas.push(id);
52             pilaAristasUV.push({u, v});
53
54             if (disc[v] == -1) {
55                 hijos++;
56                 dfs(v, id);
57                 low[u] = min(low[u], low[v]);
58
59                 bool esRaiz = (idAristaPadre == -1);
60                 if ((esRaiz && hijos > 1) ||
61                     (!esRaiz && low[v] >= disc[u])) {
62                     esArticulacion[u] = true;
63                 }
64
65                 // Si u "cierra" un block (low[v] >= disc[u]),
66                 // vuelca la pila hasta la arista (u,v) inclusive.
67
68                 if (low[v] >= disc[u]) {
69                     vector<int> block;
70                     while (true) {
71                         auto [x, y] = pilaAristasUV.top();
72                         pilaAristas.pop(); pilaAristasUV.pop();
73                         block.push_back(x); block.push_back(y);
74                         if (x == u && y == v) break;
75                     }
76                     sort(block.begin(), block.end());
77                     block.erase(unique(block.begin(), block.end()),
78                                     block.end());
79                 }
80             }
81         }
82     }
83 }

```

```

69         blocks.push_back(block);
70     }
71     } else if (disc[v] < disc[u]) {
72         low[u] = min(low[u], disc[v]);
73     }
74 }
75 }
76
77 // WRITE (interno): construye el arbol bipartito puntos-de-
78 // articulacion <-> blocks.
79 void construirArbol() {
80     arbol.assign(n + blocks.size(), {});
81     for (int i = 0; i < (int)blocks.size(); i++) {
82         int idBlock = n + i;
83         for (int u : blocks[i]) {
84             arbol[idBlock].push_back(u);
85             arbol[u].push_back(idBlock);
86         }
87     }
88 }
89 }; // O(n + m)

```

Ejemplo de funcionamiento:

```

1 // grafo:  0 -- 1
2 //         |   |
3 //         2 -- 1   1 -- 3 -- 4
4 //         |       |   |
5 //         5 -- 4
6 //
7 // (0,1)(0,2)(1,2) forman un ciclo: block A = {0,1,2}
8 // nodo 1 tambien conecta a 3: (1,3) es su propio block (puente)
9 // -> block B = {1,3}
10 // (3,4)(3,5)(4,5) forman otro ciclo: block C = {3,4,5}
11 //
12 // puntos de articulacion: 1 (separa A de B/C) y 3 (separa B de C)
13
14 vector<pair<int,int>> aristas = {
15     {0,1},{0,2},{1,2},    // block A
16     {1,3},                // block B (puente)
17     {3,4},{3,5},{4,5}    // block C
18 };
19
20 BlockCutTree bct(6, aristas);
21
22 // bct.esArticulacion[1] == true, bct.esArticulacion[3] == true
23 // bct.blocks.size() == 3
24 // bct.arbol conecta: nodo(1) -- block(A), nodo(1) -- block(B),
25 //                     nodo(3) -- block(B), nodo(3) -- block(C)
26 // -> es literalmente un arbol: A - 1 - B - 3 - C

```

Regla práctica: usa la pila de **aristas** (no de nodos) para delimitar cada block —un punto de articulación puede pertenecer a varios blocks simultáneamente, así que agrupar por nodo visitado no alcanza. El caso de la **raíz** del DFS es especial: es punto de articulación sii tiene **más de un hijo** en el árbol de DFS (no aplica la condición $\text{low}[v] \geq \text{disc}[u]$, que no tiene sentido para la raíz al no tener padre). Si solo necesitas los puntos de articulación (sin construir el árbol completo), puedes omitir la pila de aristas y quedarte solo con la detección vía low-link.

18.3 Condensation Graph

Idea: en un grafo dirigido, una **componente fuertemente conexa** (SCC) es un conjunto maximal de nodos donde todo par u, v tiene camino dirigido de u a v y de v a u . El **Condensation Graph** colapsa cada SCC en un único nodo y conecta dos SCCs con una arista dirigida sii existía al menos una arista dirigida entre algún par de nodos de una hacia la otra en el grafo original. El resultado es siempre un **DAG** (grafo acíclico dirigido): si hubiera un ciclo entre condensaciones, esas SCCs en realidad serían una sola componente más grande, contradicción.

¿Por qué usarlo? Es el análogo dirigido de Bridge Tree/Block-Cut Tree: reduce un grafo dirigido con ciclos a un DAG, donde aplican todas las técnicas exclusivas de DAGs —orden topológico, programación dinámica

sobre el orden topológico, camino más largo/corto en $O(n + m)$ sin Dijkstra/Bellman-Ford, conteo de caminos, etc.

Casos de uso:

- Encontrar todas las SCCs de un grafo dirigido.
- Programación dinámica sobre un grafo con ciclos: primero condensar a DAG, luego correr la DP en orden topológico (ej. "máximo valor acumulado siguiendo aristas dirigidas", donde dentro de una SCC todos los nodos son "alcanzables entre sí" y se pueden agregar sus valores).
- 2-SAT: construir el grafo de implicaciones y usar SCCs para determinar si x y $\neg x$ caen en la misma componente (insatisfacible) o no.
- Determinar si el grafo es "casi un DAG" y cuántos ciclos mínimos hay que romper.

Complejidad: $O(n + m)$ para encontrar las SCCs (Tarjan en una sola pasada, o Kosaraju en dos pasadas con grafo transpuesto); $O(n + m)$ adicional para construir el DAG condensado.

```

1 struct Condensacion {
2     int n;
3     vector<vector<int>> adj;
4     vector<int> disc, low, comp;
5     vector<bool> enPila;
6     stack<int> pila;
7     int timer = 0, numSCC = 0;
8     vector<vector<int>> dag; // dag[i] = adyacencia del DAG
9                             // condensado
10
11     Condensacion(int n_, vector<vector<int>>& adj_) : n(n_), adj(adj_) {
12         disc.assign(n, -1);
13         low.assign(n, -1);
14         comp.assign(n, -1);
15         enPila.assign(n, false);
16
17         for (int u = 0; u < n; u++)
18             if (disc[u] == -1) dfsTarjan(u);
19
20         set<pair<int,int>> aristasDAG; // evita aristas
21                                     // duplicadas entre SCCs
22         for (int u = 0; u < n; u++)
23             for (int v : adj[u])
24                 if (comp[u] != comp[v])
25                     aristasDAG.insert({comp[u], comp[v]});
26
27         dag.assign(numSCC, {});
28         for (auto [a, b] : aristasDAG) dag[a].push_back(b);
29     } // O(n + m)
30
31     // READ (interno): Tarjan de una sola pasada, usando una pila
32     // explicita para detectar cuando se "cierra" una SCC
33     // completa.
34     void dfsTarjan(int u) {
35         disc[u] = low[u] = timer++;
36         pila.push(u);
37         enPila[u] = true;
38
39         for (int v : adj[u]) {
40             if (disc[v] == -1) {
41                 dfsTarjan(v);
42                 low[u] = min(low[u], low[v]);
43             } else if (enPila[v]) {
44                 low[u] = min(low[u], disc[v]);
45             }
46         }
47
48         // u es raíz de su SCC: vuelca la pila hasta u inclusive.
49         if (low[u] == disc[u]) {
50             while (true) {
51                 int v = pila.top(); pila.pop();
52                 enPila[v] = false;
53                 comp[v] = numSCC;
54                 if (v == u) break;
55             }
56             numSCC++;
57         }
58     }
59 }

```

```
55 }
56 };
```

Ejemplo de funcionamiento:

```
1 // grafo dirigido: 0 -> 1 -> 2 -> 0 (ciclo: SCC A = {0,1,2})
2 //                2 -> 3
3 //                3 -> 4 -> 3 (ciclo: SCC B =
4 //                {3,4})
5 vector<vector<int>> adj(5);
6 adj[0] = {1}; adj[1] = {2}; adj[2] = {0, 3};
7 adj[3] = {4}; adj[4] = {3};
8
9 Condensacion cond(5, adj);
10
11 cout << cond.numSCC; // 2
12 // comp[0]=comp[1]=comp[2] = A (mismo id, ej. 1)
13 // comp[3]=comp[4]         = B (mismo id, ej. 0)
14
15 // cond.dag tiene exactamente 1 arista dirigida: A -> B
16 // (por la arista original 2 -> 3), es un DAG de 2 nodos
```

Regla práctica: Tarjan en una sola pasada es preferible a Kosaraju (dos pasadas + grafo transpuesto) en competencia porque es más corto de codear y usa la misma estructura low-link ya familiar de Bridge Tree/Block-Cut Tree —la única diferencia es que aquí, al ser dirigido, no hay que descartar "no volver por la misma arista": basta con `enPila[v]` para distinguir un back-edge real de un cruce a otra SCC ya cerrada. Tras condensar, **siempre** procesa el DAG resultante en orden topológico (los ids de SCC que asigna Tarjan ya vienen en orden topológico **inverso**, así que recorrer `comp[u]` de mayor a menor da orden topológico directo sin un paso adicional de toposort).

19 Eulerian Graphs

19.1 Eulerian Path

Idea: un camino euleriano es una ruta que recorre **cada arista** del grafo **exactamente una vez** (a diferencia de un camino hamiltoniano, que visita cada **nodo** una vez —son propiedades independientes). Se construye con el **algoritmo de Hierholzer**: en vez de intentar planear el recorrido completo de antemano, haces DFS "consumiendo" aristas a medida que las usas (borrándolas de la lista de adyacencia), y cuando te quedas **atascado** en un nodo sin más salidas, en vez de retroceder y descartar, agregas ese nodo al resultado y retrocedes al llamador —el camino se arma en **orden inverso** sobre una pila, y al final solo hace falta invertirlo.

¿Por qué usarlo? La existencia de un camino/circuito euleriano se puede **verificar en $O(V + E)$ con solo mirar grados** (ver la tabla de condiciones más abajo), sin necesitar construirlo —pero **construirlo** explícitamente requiere Hierholzer, que es la única forma estándar de hacerlo en $O(E)$ total (la alternativa ingenua de "prueba una arista, si te atascas retrocede y prueba otra" es exponencial en el peor caso).

Condiciones de existencia (grafo no dirigido):

- **Circuito euleriano** (vuelve al punto de partida): el grafo es conexo (ignorando nodos aislados) y **todos** los nodos tienen grado **par**.
- **Camino euleriano** (no necesariamente cierra): el grafo es conexo y tiene **exactamente 0 o 2** nodos de grado **impar**. Con 0 impares, es también un

circuito; con 2 impares, el camino debe **empezar en uno de esos dos nodos y terminar en el otro**.

- Con más de 2 nodos de grado impar, **no existe** ningún camino euleriano.

Condiciones de existencia (grafo dirigido): circuito euleriano si el grafo es conexo (como no dirigido) y **grado de entrada = grado de salida en todos** los nodos; camino euleriano (no cerrado) si es conexo y **exactamente un** nodo tiene salida = entrada + 1 (el punto de partida obligado), **exactamente un** nodo tiene entrada = salida + 1 (el punto de llegada obligado), y el resto tiene entrada = salida.

Casos de uso:

- "Recorre todas las calles de una ciudad sin repetir ninguna" (el problema original de los puentes de Königsberg).
- Reconstruir una secuencia a partir de sus fragmentos solapados (ej. de-novo genome assembly con k -mers: cada k -mer es una arista en un grafo de De Bruijn, y el camino euleriano reconstruye la secuencia original).
- Verificar si es posible dibujar una figura "sin levantar el lápiz y sin repetir líneas".
- Circuitos de prueba que deben ejercitar cada transición de un sistema exactamente una vez.

Complejidad: $O(V + E)$ para verificar la existencia (grados + conectividad); $O(E)$ para construir el camino con Hierholzer, ya que cada arista se consume y se procesa una sola vez.

```
1 struct CaminoEuleriano {
2     vector<multiset<int>> adj; // multiset: permite borrar UNA
3     // ocurrencia específica de una
4     // arista (aristas multiples validas)
5     int n;
6
7     // CREATE: guarda el grafo no dirigido (adj[u] contiene v
8     // tantas
9     // veces como aristas u-v existan).
10    CaminoEuleriano(vector<multiset<int>>& adj_, int n_) {
11        adj = adj_;
12        n = n_;
13    } // O(1)
14
15    // READ: nodo de inicio obligado si hay exactamente 2 nodos
16    // de
17    // grado impar (camino no cerrado); -1 si hay que empezar en
18    // cualquier nodo con aristas (circuito, 0 impares); -2 si NO
19    // existe camino euleriano (mas de 2 nodos de grado impar).
20    int nodoInicio() {
21        vector<int> impares;
22        for (int u = 0; u < n; u++)
23            if ((int)adj[u].size() % 2 == 1)
24                impares.push_back(u);
25
26        if (impares.size() == 0) {
27            for (int u = 0; u < n; u++)
28                if (!adj[u].empty()) return u; // cualquier nodo
29                // con aristas
30            return 0; // grafo sin aristas
31        }
32        if (impares.size() == 2) return impares[0];
33
34        return -2; // no existe
35    }
36
37    // READ: construye el camino euleriano (Hierholzer),
38    // asumiendo
39    // que ya se verifico que existe. Devuelve la secuencia de
40    // nodos
41    // visitados en orden.
```

```

38 vector<int> hierholzer(int inicio) {
39     vector<int> pila = {inicio}, camino;
40
41     while (!pila.empty()) {
42         int u = pila.back();
43
44         if (!adj[u].empty()) {
45             int v = *adj[u].begin();
46
47             adj[u].erase(adj[u].find(v)); // consume la arista
48             adj[v].erase(adj[v].find(u)); // en ambos sentidos
49
50             pila.push_back(v);
51         } else {
52             // atascado: u no tiene mas salidas, se agrega al
53             // resultado y se retrocede
54             camino.push_back(u);
55             pila.pop_back();
56         }
57     }
58
59     reverse(camino.begin(), camino.end());
60     return camino;
61 } // O(E)
62 };

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo, todos grado 2 ->
2 //           circuito)
3 //           2 - 3 (agrega grado impar a 2 y 3)
4
5 vector<multiset<int>> adj(4);
6 auto addEdge = [&](int u, int v) {
7     adj[u].insert(v);
8     adj[v].insert(u);
9 };
10 addEdge(0, 1);
11 addEdge(1, 2);
12 addEdge(2, 0);
13 addEdge(2, 3);
14
15 CaminoEuleriano ce(adj, 4);
16
17 int inicio = ce.nodoInicio();
18 cout << inicio;
19 // 3 (3 y 2 son los unicos de grado impar -> camino, no
20 //     circuito;
21 //     debe empezar en 3)
22
23 auto camino = ce.hierholzer(inicio);
24 for (int u : camino) cout << u << " ";
25 // 3 2 0 1 2 (recorre las 4 aristas exactamente una vez cada
26 //            una)

```

Nota: el uso de `multiset` en vez de `vector` para `adj` no es incidental —permite **erase** de una sola ocurrencia específica en $O(\log(\text{grado}))$, necesario cuando hay aristas múltiples entre el mismo par de nodos (un `vector` con `erase(remove(...))` borraría todas las ocurrencias, o requeriría manejar índices manualmente). Para grafos **dirigidos**, el mismo Hierholzer aplica casi igual, solo que se consume la arista únicamente en el sentido de salida (`adj[u]` pierde a `v`, pero `adj[v]` no pierde a `u` —no hay arista de vuelta que borrar).

Regla práctica: antes de correr Hierholzer, **siempre** verifica primero grados y conectividad —Hierholzer **no** detecta por sí solo si el grafo no tiene camino euleriano; si el grafo no es conexo (ignorando nodos aislados sin aristas) o tiene más de 2 nodos de grado impar, el algoritmo puede terminar sin haber recorrido todas las aristas, sin avisar explícitamente del error.

19.2 Eulerian Circuit

Idea: un circuito euleriano es el caso particular de camino euleriano (ver Eulerian Path) donde el recorrido **empieza y termina en el mismo nodo**, cubriendo cada arista exactamente una vez. Existe si y solo si el grafo es conexo (ignorando nodos aislados) y **todos** los nodos tienen grado par (no dirigido) o grado de entrada

= grado de salida en **todos** los nodos (dirigido) —la misma condición de "0 nodos de grado impar" que ya viste en Eulerian Path, aislada aquí porque simplifica la lógica: al no haber nodos de grado impar, **cualquier** nodo con aristas sirve como punto de partida, sin tener que buscar primero los dos extremos obligados del caso de camino abierto.

¿Por qué usarlo? Es exactamente Hierholzer (siguiente subsección) sin la parte de "encontrar el nodo de inicio obligado" — esa simplificación es justo la señal para reconocerlo: si el enunciado dice explícitamente "vuelve al punto de partida" o "es un *tour/ciclo*" en vez de solo "camino" o "*trail*", es circuito, no camino abierto, y no hace falta verificar cuáles nodos tienen grado impar —solo que **ninguno** lo tenga.

Casos de uso:

- Recorridos cíclicos que deben cubrir cada conexión exactamente una vez y regresar al origen (rutas de mantenimiento, inspección de tuberías/cableado en anillo).
- Problema del cartero chino (*Chinese Postman Problem*) en el caso base donde el grafo **ya** es euleriano (sin necesidad de duplicar aristas para balancear grados —esa parte extra queda fuera del alcance de esta sección).
- Secuencias de De Bruijn **cíclicas**: el circuito euleriano sobre el grafo de De Bruijn de orden $k - 1$ genera una secuencia que contiene cada k -mer posible exactamente una vez, "envolviendo" al final de vuelta al principio.
- Verificar que un grafo admite un recorrido cerrado sin repetir aristas, como paso previo a construirlo.

Complejidad: $O(V + E)$ para verificar la existencia (grados + conectividad), $O(E)$ para construir el circuito con Hierholzer.

```

1 struct CircuitoEuleriano {
2     vector<multiset<int>> adj;
3     int n;
4
5     // CREATE: guarda el grafo no dirigido.
6     CircuitoEuleriano(vector<multiset<int>>& adj_, int n_) {
7         adj = adj_;
8         n = n_;
9     } // O(1)
10
11     // READ: true si el grafo admite un circuito euleriano (
12     //         conexo
13     //         ignorando aislados, y todos los grados pares).
14     bool existeCircuito() {
15         for (int u = 0; u < n; u++)
16             if ((int)adj[u].size() % 2 != 0)
17                 return false;
18         return verificarConexo();
19     }
20
21     bool verificarConexo() {
22         int inicio = -1;
23         for (int u = 0; u < n; u++)
24             if (!adj[u].empty()) { inicio = u; break; }
25         if (inicio == -1) return true; // sin aristas, vacio-
26                                     // conexo
27
28         vector<bool> visitado(n, false);
29         queue<int> q;
30         q.push(inicio);
31         visitado[inicio] = true;
32         int alcanzados = 1;
33
34         while (!q.empty()) {
35             int u = q.front(); q.pop();

```

```

34     for (int v : adj[u]) {
35         if (visitado[v]) continue;
36         visitado[v] = true;
37         alcanzados++;
38         q.push(v);
39     }
40 }
41
42 for (int u = 0; u < n; u++)
43 if (!adj[u].empty() && !visitado[u])
44 return false; // hay un nodo con aristas fuera de
45 // esta componente
46
47 return true;
48 } // O(V + E)
49
50 // READ: construye el circuito (Hierholzer), empezando en
51 // CUALQUIER nodo con aristas -- a diferencia de Eulerian
52 // Path,
53 // aqui no hay un nodo de inicio obligado.
54 vector<int> hierholzer() {
55     int inicio = 0;
56     for (int u = 0; u < n; u++)
57         if (!adj[u].empty()) { inicio = u; break; }
58
59     vector<int> pila = {inicio}, circuito;
60
61     while (!pila.empty()) {
62         int u = pila.back();
63
64         if (!adj[u].empty()) {
65             int v = *adj[u].begin();
66             adj[u].erase(adj[u].find(v));
67             adj[v].erase(adj[v].find(u));
68             pila.push_back(v);
69         } else {
70             circuito.push_back(u);
71             pila.pop_back();
72         }
73     }
74
75     reverse(circuito.begin(), circuito.end());
76     return circuito; // circuito.front() == circuito.back()
77 } // O(E)

```

Ejemplo de funcionamiento:

```

1 // grafo: 0 - 1 - 2 - 0 (triangulo, todos grado 2 ->
2 // circuito)
3
4 vector<multiset<int>> adj(3);
5 auto addEdge = [&](int u, int v) {
6     adj[u].insert(v);
7     adj[v].insert(u);
8 };
9 addEdge(0, 1);
10 addEdge(1, 2);
11 addEdge(2, 0);
12
13 CircuitoEuleriano ce(adj, 3);
14
15 cout << ce.existeCircuito();
16 // true
17
18 auto circuito = ce.hierholzer();
19 for (int u : circuito) cout << u << " ";
20 // 0 1 2 0 (empieza y termina en el mismo nodo)

```

Regla práctica: el chequeo de existencia es idéntico al de Eulerian Path, solo que aquí el conteo de "nodos de grado impar" **debe dar exactamente 0** —si da 2, existe camino pero no circuito (hay que abrir el recorrido en esos dos nodos); si da más de 2, no existe ninguno de los dos.

19.3 Hierholzer

Idea: esta subsección presenta la implementación genérica y verdaderamente $O(E)$ del algoritmo, que ya se usó de forma simplificada en Eulerian Path y Eulerian Circuit. La versión con `multiset` de esas secciones es correcta pero cuesta $O(E \log E)$ (cada `insert/erase` es logarítmico); la versión estándar de competencia evita eso con **un puntero por nodo** (`ptr[u]`) que avanza

sobre la lista de adyacencia de u **saltándose** las aristas ya usadas —como cada arista se marca usada una sola vez en toda la ejecución, cada puntero avanza como mucho tantas veces como aristas tiene su nodo, dando $O(E)$ total real. Además, indexando las aristas por id (en vez de por su nodo destino), la misma implementación funciona **sin cambios** para grafos dirigidos, no dirigidos, y con aristas múltiples.

¿Por qué usarlo? El factor $\log E$ de la versión con `multiset` no importa para grafos chicos, pero se vuelve relevante con $E \sim 10^5$ – 10^6 (típico en problemas de secuencias de De Bruijn o reconstrucción de rutas). Esta versión con punteros es la que normalmente se usa como plantilla reutilizable, ya que separa completamente la lógica del algoritmo (la pila + "atascarse y retroceder") de la representación del grafo (dirigido o no, con o sin aristas múltiples), a través del truco de marcar aristas por id en vez de por nodo.

Casos de uso:

- Los mismos que Eulerian Path/Circuit, cuando E es lo bastante grande como para que el factor $\log E$ del `multiset` importe en el límite de tiempo.
- Grafos **dirigidos** con aristas múltiples entre el mismo par de nodos (el `multiset` de las secciones anteriores solo se planteó para no dirigido; aquí el mismo código sirve para ambos sin modificar la lógica del algoritmo, solo la forma en que se cargan las aristas).
- Reconstrucción de secuencias de De Bruijn a gran escala (genome assembly), donde E puede ser del orden de millones de k -mers.
- Como plantilla base reutilizable: una vez armado el `struct` genérico, tanto Eulerian Path como Eulerian Circuit son solo "elegir el nodo de inicio correcto" antes de llamarlo.

Complejidad: $O(E)$ real (sin factor logarítmico), tanto para grafos dirigidos como no dirigidos.

```

1 struct Hierholzer {
2     vector<vector<pair<int,int>>> adj; // adj[u] = {(v,
3         idArista)}
4     vector<bool> usada;
5     vector<int> ptr; // ptr[u] = índice de la primera arista de
6         // adj[u] que TODAVIA no se sabe si esta usada
7     int n;
8
9     // CREATE: guarda el grafo (dirigido o no -- para no dirigido,
10    // cada arista se agrega DOS veces, una en cada sentido, con
11    // el
12    // MISMO id; para dirigido, una sola vez con id unico).
13    Hierholzer(vector<vector<pair<int,int>>>& adj_, int
14        numAristas,
15        int n_) {
16        adj = adj_;
17        n = n_;
18        usada.assign(numAristas, false);
19        ptr.assign(n, 0);
20    } // O(1)
21
22    // READ: construye el camino/circuito euleriano empezando en
23    // 'inicio', asumiendo que ya se verificó que existe.
24    vector<int> construir(int inicio) {
25        vector<int> pila = {inicio}, camino;
26
27        while (!pila.empty()) {
28            int u = pila.back();
29
30            // avanza el puntero saltando aristas YA usadas -- cada
31            // posicion se salta como mucho una vez en TODA la

```

```

29 // ejecucion, de ahi el O(E) total
30 while (ptr[u] < (int)adj[u].size()) &&
31 usada[adj[u][ptr[u]].second])
32 ptr[u]++;
33
34 if (ptr[u] == (int)adj[u].size()) {
35     camino.push_back(u); // atascado: sin salidas
36     pila.pop_back();
37 } else {
38     auto [v, id] = adj[u][ptr[u]];
39     usada[id] = true;
40     pila.push_back(v);
41 }
42 }
43
44 reverse(camino.begin(), camino.end());
45 return camino; // O(E)
46 }
47 };

```

Ejemplo de funcionamiento:

```

1 // grafo DIRIGIDO: 0 -> 1 -> 2 -> 0 (ciclo)
2 //                  0 -> 2 (arista extra)
3 // grados: 0 tiene salida=2, entrada=1; 2 tiene salida=1, entrada
4 //          =2
5 // -> 0 es el inicio obligado (salida = entrada + 1),
6 //          2 es el final obligado (entrada = salida + 1)
7
8 vector<vector<pair<int,int>>> adj(3);
9 int idArista = 0;
10 auto addEdgeDirigida = [&](int u, int v) {
11     adj[u].push_back({v, idArista++});
12 };
13 addEdgeDirigida(0, 1); // id 0
14 addEdgeDirigida(1, 2); // id 1
15 addEdgeDirigida(2, 0); // id 2
16 addEdgeDirigida(0, 2); // id 3
17
18 Hierholzer h(adj, 4, 3);
19 auto camino = h.construir(0);
20
21 for (int u : camino) cout << u << " ";
22 // 0 1 2 0 2 (recorre las 4 aristas dirigidas exactamente una
23 // vez)

```

Nota sobre grafos no dirigidos: para reutilizar este mismo struct con un grafo no dirigido, cada arista física (u, v) se carga **dos veces**: `adj[u].push_back({v, id})` y `adj[v].push_back({u, id})`, ambas con el **mismo** id. Como `usada[]` se indexa por id y no por posición, marcar la arista como usada desde un lado **automáticamente** la invalida también del otro lado —el mismo efecto que borrar de ambos `multiset` en la versión anterior, sin el costo logarítmico.

Regla práctica: usa la versión con `multiset` de Eulerian Path/Circuit cuando el grafo es chico y prefieres código más corto de escribir bajo presión de tiempo; cambia a esta versión con punteros por id en cuanto E sea grande, el grafo sea dirigido, o quieras una sola plantilla reutilizable para ambas variantes (camino y circuito) sin duplicar lógica —la única diferencia entre resolver un camino o un circuito con esta plantilla es **qué nodo** se pasa como inicio.

20 Network Flow

21 Network Flow

21.1 Ford-Fulkerson

Idea: Ford-Fulkerson calcula el flujo máximo entre una fuente s y un sumidero t repitiendo, mientras sea posible, dos pasos: (1) encontrar un **camino aumentante** de s a t en el **grafo residual** (aristas con capacidad restante > 0); (2) enviar por ese camino tanta cantidad como permita su **cuello de botella** (la arista de menor capacidad residual del camino), actualizando las capacidades

residuales —restando en el sentido de ida, **sumando** en el sentido inverso (arista de "deshacer", que permite corregir decisiones subóptimas posteriores). Cuando ya no existe ningún camino aumentante, el flujo actual es el máximo (teorema de flujo máximo-corte mínimo).

¿Por qué usarlo? Es la base de **todos** los algoritmos de flujo: Edmonds-Karp y Dinic (siguientes secciones) son variantes de Ford-Fulkerson que solo cambian **cómo** se encuentra el camino aumentante para acotar mejor la complejidad. La versión con DFS simple es la más fácil de programar bajo presión y es correcta siempre que las capacidades sean **enteras** —pero su complejidad depende del **valor** del flujo máximo, no solo de V y E , así que puede ser lenta en el peor caso adversarial (ver modificaciones abajo para las variantes que corrigen esto).

Casos de uso:

- Flujo máximo en una red de transporte/capacidades (el problema base).
- Como building block de *bipartite matching* vía flujo (fuente $\rightarrow$ lado $L \rightarrow$ lado $R \rightarrow$ sumidero, todas las aristas de capacidad 1) — ver Maximum Bipartite Matching más adelante.
- Corte mínimo entre s y t : su valor es **igual** al flujo máximo (mismo teorema) — ver Min-Cut más adelante.

Complejidad: $O(E \cdot F)$ donde F es el valor del flujo máximo (cada DFS es $O(E)$, y en el peor caso se necesitan $O(F)$ iteraciones si las capacidades son grandes y el camino aumentante elegido es "malo").

```

1 struct FordFulkerson {
2     struct Arista { int destino; long long capacidad; int reversa
3         ; };
4     vector<vector<Arista>> adj;
5     vector<bool> visitado;
6     int n;
7
8     // CREATE: grafo residual vacio de n nodos.
9     FordFulkerson(int n_) {
10         n = n_;
11         adj.assign(n, {});
12     } // O(n)
13
14     // WRITE: agrega una arista dirigida u->v con capacidad c.
15     // Crea
16     // automaticamente la arista reversa con capacidad 0 (se
17     // llena a
18     // medida que se envia flujo, permitiendo "deshacer" envios).
19     void agregarArista(int u, int v, long long c) {
20         adj[u].push_back({v, c, (int)adj[v].size()});
21         adj[v].push_back({u, 0, (int)adj[u].size() - 1});
22     } // O(1)
23
24     // busca un camino aumentante con DFS, enviando flujo
25     // mientras
26     // // regresa de la recursion (devuelve cuanto flujo se logro
27     // // enviar por esa rama, 0 si no encontro camino).
28     long long dfs(int u, int t, long long flujoMin) {
29         if (u == t) return flujoMin;
30         visitado[u] = true;
31
32         for (auto& arista : adj[u]) {
33             if (visitado[arista.destino] || arista.capacidad <= 0)
34                 continue;
35
36             long long resultado = dfs(arista.destino, t,
37                 min(flujoMin, arista.capacidad));
38
39             if (resultado > 0) {
40                 arista.capacidad -= resultado;
41                 adj[arista.destino][arista.reversa].capacidad +=
42                     resultado;
43                 return resultado;
44             }
45         }
46     }
47 }

```

```

40     }
41
42     return 0;
43 }
44
45 // READ: flujo maximo de s a t.
46 long long flujoMaximo(int s, int t) {
47     long long total = 0;
48
49     while (true) {
50         visitado.assign(n, false);
51         long long f = dfs(s, t, LLONG_MAX);
52         if (f == 0) break;    // no hay mas caminos aumentantes
53         total += f;
54     }
55
56     return total;
57 }                                     // O(E * F)
58
59 //
60 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
61 //
62 // -----
63 // 1. FLUJO MAXIMO CON MULTIPLES FUENTES / SUMIDEROS: se
64 //     agrega
65 //     una SUPERFUENTE S y un SUPERSUMIDERO T ficticios; S
66 //     conecta
67 //     a cada fuente real con capacidad infinita (o la
68 //     capacidad
69 //     maxima que esa fuente puede producir), cada sumidero
70 //     real
71 //     conecta a T de la misma forma -- y se corre flujoMaximo
72 //     (S,T)
73 //     normal, sin modificar el algoritmo en si.
74 //
75 // int S = n, T = n + 1;    // nodos ficticios extra al
76 //                             construir
77 // for (int fuente : fuentesReales)
78 //     agregarArista(S, fuente, LLONG_MAX);
79 // for (int sumidero : sumiderosReales)
80 //     agregarArista(sumidero, T, LLONG_MAX);
81 // long long total = flujoMaximo(S, T);
82
83 // 2. VERTICES CON CAPACIDAD (limite de "flujo que puede
84 //     pasar
85 //     POR un nodo", no solo por sus aristas): se divide cada
86 //     nodo u en u_entrada y u_salida, unidos por una arista
87 //     u_entrada -> u_salida con capacidad = limite del nodo;
88 //     todas las aristas que antes llegaban a u ahora llegan a
89 //     u_entrada, y las que salian de u ahora salen de
90 //     u_salida.
91 //     El algoritmo de flujo no cambia, solo la construccion
92 //     del
93 //     grafo (mismo truco que usa 2-SAT con nodo/negacion,
94 //     pero
95 //     aplicado a capacidad en vez de literal).
96
97 // 3. CAPACIDADES NO ENTERAS / MUY GRANDES: Ford-Fulkerson
98 //     con
99 //     DFS puro puede degenerar si las capacidades son enormes
100 //     y
101 //     el camino elegido es siempre el "peor" posible (ejemplo
102 //     clasico: red con capacidades ~10^9 puede necesitar
103 //     ~10^9
104 //     iteraciones). La solucion NO es cambiar este algoritmo,
105 //     sino usar Edmonds-Karp o Dinic (siguientes secciones),
106 //     que
107 //     acotan la complejidad independientemente del valor de F.
108
109 // 4. FLUJO MAXIMO CON LIMITE INFERIOR (cada arista debe
110 //     llevar
111 //     AL MENOS un minimo, ademas de un maximo): requiere una
112 //     transformacion adicional del grafo (nodos ficticios de
113 //     "exceso") antes de correr flujo maximo normal -- fuera
114 //     del
115 //     alcance de esta plantilla base, pero la señal para
116 //     reconocerlo es que el enunciado da un RANGO
117 //     [minimo, maximo] por arista en vez de solo un maximo.
118 };

```

Ejemplo de funcionamiento:

```

1 // red:  s -(3)-> a -(2)-> t
2 //       s -(2)-> b -(3)-> t
3 //       a -(1)-> b
4
5 // nodos: s=0, a=1, b=2, t=3
6
7 FordFulkerson ff(4);
8 ff.agregarArista(0, 1, 3);

```

```

9 ff.agregarArista(0, 2, 2);
10 ff.agregarArista(1, 3, 2);
11 ff.agregarArista(2, 3, 3);
12 ff.agregarArista(1, 2, 1);
13
14 cout << ff.flujoMaximo(0, 3);
15 // 4
16 // (2 por s-a-t, 2 por s-b-t; la arista a-b sobra en este caso,
17 // el cuello de botella real esta en las aristas de salida de
18 // y de entrada a t)

```

Regla práctica: programa Ford-Fulkerson con DFS cuando las capacidades son chicas o el límite de tiempo es generoso —es el más corto de escribir bajo presión. En cuanto el problema tenga capacidades grandes o el límite de tiempo sea ajustado, salta directamente a Edmonds-Karp (siguiente sección): la estructura del grafo residual (Arista, agregarArista) es **idéntica**, solo cambia cómo se busca el camino aumentante.

21.2 Edmonds-Karp

Idea: Edmonds-Karp es Ford-Fulkerson con **una sola diferencia**: el camino aumentante se busca con **BFS** en vez de DFS, lo que garantiza que **siempre** se use el camino con **menos aristas** disponible en el grafo residual. Esa elección —"el camino más corto primero"— es lo que rompe el peor caso adversarial de Ford-Fulkerson: se puede demostrar que el número total de fases de BFS está acotado por $O(V \cdot E)$, **independientemente** de qué tan grandes sean las capacidades.

¿Por qué usarlo? Es el primer paso obligado en cuanto Ford-Fulkerson con DFS "se siente lento" o el problema explícitamente tiene capacidades grandes ($\geq 10^4$ – 10^5): mismo grafo residual, mismo concepto de camino aumentante, **solo** cambia BFS por DFS, a cambio de una garantía de complejidad que ya no depende del valor del flujo. Si $O(V \cdot E^2)$ sigue sin alcanzar (grafos realmente grandes), el siguiente paso es Dinic.

Casos de uso:

- Los mismos que Ford-Fulkerson (flujo máximo, matching vía flujo, corte mínimo), en cuanto las capacidades son lo bastante grandes como para que el peor caso de DFS sea un riesgo real.
- Como opción "segura por defecto" cuando no se sabe de antemano qué tan adversarial puede ser el grafo de entrada —el costo de cambiar DFS por BFS es mínimo y elimina ese riesgo.

Complejidad: $O(V \cdot E^2)$ — como mucho $O(V \cdot E)$ fases de BFS (cada una satura al menos una arista del grafo residual), cada BFS cuesta $O(E)$.

```

1 struct EdmondsKarp {
2     struct Arista { int destino; long long capacidad; int reversa
3         ; };
4     vector<vector<Arista>> adj;
5     int n;
6
7     // CREATE: grafo residual vacio de n nodos.
8     EdmondsKarp(int n_) {
9         n = n_;
10        adj.assign(n, {});
11    }                                     // O(n)
12
13    // WRITE: identico a Ford-Fulkerson -- agrega u->v con
14    // capacidad
15    // c, mas su reversa de capacidad 0.
16    void agregarArista(int u, int v, long long c) {
17        adj[u].push_back({v, c, (int)adj[v].size()});
18        adj[v].push_back({u, 0, (int)adj[u].size() - 1});
19    }
20 }

```

```

17     } // O(1)
18
19     // busca el camino aumentante MAS CORTO (en numero de aristas
20     // con BFS. Devuelve el flujo enviado por ese camino, 0 si no
21     // hay camino aumentante. padreNodo/padreArista reconstruyen
22     // camino encontrado.
23     long long bfs(int s, int t, vector<int>& padreNodo,
24     vector<int>& padreArista) {
25         fill(padreNodo.begin(), padreNodo.end(), -1);
26         padreNodo[s] = s;
27
28         queue<pair<int, long long>> q;
29         q.push({s, LLONG_MAX});
30
31         while (!q.empty()) {
32             auto [u, flujoHasta] = q.front(); q.pop();
33
34             for (int i = 0; i < (int)adj[u].size(); i++) {
35                 auto& a = adj[u][i];
36
37                 if (a.capacidad <= 0 || padreNodo[a.destino] != -1)
38                     continue;
39
40                 padreNodo[a.destino] = u;
41                 padreArista[a.destino] = i;
42                 long long nuevoFlujo = min(flujoHasta, a.capacidad);
43
44                 if (a.destino == t) return nuevoFlujo;
45
46                 q.push({a.destino, nuevoFlujo});
47             }
48         }
49
50         return 0; // t no alcanzable -> no hay camino aumentante
51     } // O(E)
52
53     // READ: flujo maximo de s a t.
54     long long flujoMaximo(int s, int t) {
55         long long total = 0;
56         vector<int> padreNodo(n), padreArista(n);
57
58         while (true) {
59             long long f = bfs(s, t, padreNodo, padreArista);
60             if (f == 0) break;
61
62             // reconstruye el camino desde t hasta s y actualiza
63             // capacidades residuales (resta de ida, suma en reversa)
64             int u = t;
65             while (u != s) {
66                 int idArista = padreArista[u];
67                 int padre = padreNodo[u];
68
69                 adj[padre][idArista].capacidad -= f;
70                 int idReversa = adj[padre][idArista].reversa;
71                 adj[u][idReversa].capacidad += f;
72
73                 u = padre;
74             }
75
76             total += f;
77         }
78
79         return total;
80     } // O(V * E^2)
81
82     // -----
83     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
84     // -----
85
86     // 1. RECONSTRUIR EL CAMINO AUMENTANTE EXPLICITO (no solo su
87     // flujo): ya viene gratis en padreNodo/padreArista tras
88     // cada bfs() exitoso -- util si el problema pide ademas
89     // MOSTRAR las rutas usadas, no solo el valor total del flujo
90     // maximo.
91     // vector<int> reconstruirCamino(int s, int t,
92     // vector<int>& padreNodo) {
93     //     vector<int> camino;
94     //     for (int u = t; u != s; u = padreNodo[u])
95     //         camino.push_back(u);
96     //     camino.push_back(s);
97     //     reverse(camino.begin(), camino.end());
98     //     return camino;
99     // }
100
101     // 2. MULTIPLES FUENTES / SUMIDEROS: identico truco que en
102     // Ford-Fulkerson -- superfuente S y supersumidero T con
103     // aristas de capacidad infinita hacia/desde cada fuente y
104     // sumidero real; se corre flujoMaximo(S, T) sin cambios.
105     //
106     // int S = n, T = n + 1;
107     // for (int fuente : fuentesReales)
108     //     agregarArista(S, fuente, LLONG_MAX);
109     // for (int sumidero : sumiderosReales)
110     //     agregarArista(sumidero, T, LLONG_MAX);
111
112     // 3. DETENERSE ANTES DE flujoMaximo() COMPLETO -- si solo se
113     // necesita saber SI existe flujo >= X (no el maximo
114     // exacto),
115     // se puede cortar el while en cuanto total >= X,
116     // ahorrando
117     // fases de BFS innecesarias:
118     //
119     // long long flujoAlMenos(int s, int t, long long X) {
120     //     long long total = 0;
121     //     vector<int> padreNodo(n), padreArista(n);
122     //     while (total < X) {
123     //         long long f = bfs(s, t, padreNodo, padreArista);
124     //         if (f == 0) break;
125     //         // ... misma actualizacion de capacidades ...
126     //         total += f;
127     //     }
128     //     return total;
129     // }
130
131     // 4. GRAFOS DENSOS O MUY GRANDES (E ~ 10^5 o mas, V grande):
132     // O(V * E^2) de Edmonds-Karp puede seguir sin alcanzar. La
133     // senal para saltar a Dinic (siguiente seccion) es la
134     // MISMA estructura de grafo residual (Arista, agregarArista)
135     // mas una nocion de "grafo por niveles" (BFS de niveles + DFS
136     // con poda) que reduce el numero de fases a O(V), en vez
137     // de O(V * E) -- no hace falta reescribir el grafo, solo la
138     // logica de busqueda de caminos aumentantes.
139     };

```

Ejemplo de funcionamiento:

```

1 // misma red que en Ford-Fulkerson:
2 // s -(3)-> a -(2)-> t
3 // s -(2)-> b -(3)-> t
4 // a -(1)-> b
5
6 EdmondsKarp ek(4);
7 ek.agregarArista(0, 1, 3);
8 ek.agregarArista(0, 2, 2);
9 ek.agregarArista(1, 3, 2);
10 ek.agregarArista(2, 3, 3);
11 ek.agregarArista(1, 2, 1);
12
13 cout << ek.flujoMaximo(0, 3);
14 // 4 (mismo resultado que Ford-Fulkerson -- el flujo maximo
15 // es
16 // unico independientemente del orden de caminos elegido;
17 // lo que cambia es CUANTAS iteraciones toma llegar ahí)

```

Regla práctica: Edmonds-Karp y Ford-Fulkerson dan **siempre** el mismo flujo máximo —difieren solo en garantía de complejidad, nunca en corrección. Si ya tienes el `struct` de Ford-Fulkerson escrito, migrar a Edmonds-Karp es cambiar `dfs()` por `bfs()` y ajustar cómo se reconstruye el camino para actualizar capacidades —el resto (`Arista`, `agregarArista`, la idea de arista reversa) no cambia en absoluto.

21.3 Dinic

Idea: Dinic mejora a Edmonds-Karp separando el trabajo en **fases**: en cada fase, (1) construye un **grafo por niveles** con un solo BFS desde s ($\text{nivel}[u]$ = distancia mínima desde s en el grafo residual actual), quedándose **solo** con las aristas que avanzan de un nivel al siguiente ($\text{nivel}[v] = \text{nivel}[u] + 1$); (2) sobre ese grafo por niveles, encuentra **todos** los caminos aumentantes posibles de una sola vez con DFS, usando un **puntero por nodo** (`ptr[u]`, igual que en Hierholzer) para no reexplorar aristas ya agotadas dentro de la misma fase —esto se llama

blocking flow. Cuando el BFS ya no puede alcanzar t (no hay más niveles), el algoritmo termina: el flujo acumulado es el máximo.

¿Por qué usarlo? Edmonds-Karp encuentra un camino aumentante por cada BFS ($O(V \cdot E)$ fases en el peor caso); Dinic encuentra **muchos** caminos aumentantes por cada BFS (todo el blocking flow de esa fase), reduciendo el número de fases a $O(V)$ —cada fase sube al menos un nivel, y hay como mucho V niveles posibles. Es la opción estándar en competencia cuando E es grande y $O(V \cdot E^2)$ de Edmonds-Karp no alcanza el límite de tiempo; en grafos bipartitos (unit-capacity), Dinic corre incluso más rápido ($O(E\sqrt{V})$), lo cual lo hace también la base preferida para Maximum Bipartite Matching vía flujo (siguiente sección).

Casos de uso:

- Los mismos que Ford-Fulkerson/Edmonds-Karp, en instancias grandes donde $O(V \cdot E^2)$ es demasiado lento.
- Bipartite Matching vía flujo, aprovechando el caso especial de $O(E\sqrt{V})$ en grafos de capacidad unitaria.
- Cualquier problema de flujo donde el límite de tiempo sea ajustado y V, E sean del orden de 10^4 – 10^5 .

Complejidad: $O(V^2 \cdot E)$ en general, $O(E\sqrt{V})$ en grafos de capacidad unitaria (bipartite matching) — en ambos casos, estrictamente mejor o igual que Edmonds-Karp.

```

1 struct Dinic {
2     struct Arista { int destino; long long capacidad; int reversa
3         ; };
4     vector<vector<Arista>> adj;
5     vector<int> nivel, ptr;
6     int n;
7
8     // CREATE: grafo residual vacio de n nodos.
9     Dinic(int n) {
10         n = n;
11         adj.assign(n, {});
12     } // O(n)
13
14     // WRITE: identico a Ford-Fulkerson/Edmonds-Karp -- agrega u
15     // ->v
16     // con capacidad c, mas su reversa de capacidad 0.
17     void agregarArista(int u, int v, long long c) {
18         adj[u].push_back({v, c, (int)adj[v].size()});
19         adj[v].push_back({u, 0, (int)adj[u].size() - 1});
20     } // O(1)
21
22     // FASE 1: construye el grafo por niveles con BFS. Devuelve
23     // false si t ya no es alcanzable (fin del algoritmo).
24     bool bfs(int s, int t) {
25         nivel.assign(n, -1);
26         queue<int> q;
27         q.push(s);
28         nivel[s] = 0;
29
30         while (!q.empty()) {
31             int u = q.front(); q.pop();
32
33             for (auto& a : adj[u]) {
34                 if (a.capacidad > 0 && nivel[a.destino] == -1) {
35                     nivel[a.destino] = nivel[u] + 1;
36                     q.push(a.destino);
37                 }
38             }
39         }
40         return nivel[t] != -1;
41     } // O(E)
42
43     // FASE 2: DFS con punteros (blocking flow), buscando UN
44     // camino

```

```

43 // aumentante que respete el grafo por niveles; se llama
44 // repetidamente hasta agotar todos los caminos de esta fase.
45 long long dfs(int u, int t, long long flujoMin) {
46     if (u == t) return flujoMin;
47
48     // avanza el puntero saltando aristas ya agotadas o que no
49     // avanzan de nivel -- cada posicion se salta como mucho
50     // una
51     // vez POR FASE, de ahi la eficiencia del blocking flow
52     for (; ptr[u] < (int)adj[u].size(); ptr[u]++) {
53         auto& a = adj[u][ptr[u]];
54
55         if (a.capacidad <= 0 || nivel[a.destino] != nivel[u] + 1)
56             continue;
57
58         long long resultado = dfs(a.destino, t,
59             min(flujoMin, a.capacidad));
60
61         if (resultado > 0) {
62             a.capacidad -= resultado;
63             adj[a.destino][a.reversa].capacidad += resultado;
64             return resultado;
65         }
66     }
67
68     return 0; // u agotado en esta fase, ningun camino desde
69     // aqui // O(V) amortizado
70
71     // dentro de una fase
72
73     // READ: flujo maximo de s a t.
74     long long flujoMaximo(int s, int t) {
75         long long total = 0;
76
77         while (bfs(s, t)) {
78             ptr.assign(n, 0);
79
80             while (long long f = dfs(s, t, LLONG_MAX))
81                 total += f; // sigue sacando caminos de la MISMA
82                             // fase hasta agotar el blocking flow
83         }
84
85         return total;
86     } // O(V^2 * E)
87
88 // -----
89 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
90 // -----
91
92 // 1. BIPARTITE MATCHING VIA DINIC (capacidad unitaria): sin
93 // cambiar el algoritmo, si TODAS las aristas tienen
94 // capacidad
95 // 1 (fuente -> L, L -> R segun compatibilidad, R ->
96 // sumidero),
97 // la complejidad baja automaticamente a O(E*sqrt(V)) --
98 // no
99 // hace falta ningun ajuste al codigo, es una propiedad
100 // del
101 // caso de capacidad unitaria. Ver Maximum Bipartite
102 // Matching.
103
104 // int S = n, T = n + 1; // agregar como nodos extra
105 // for (int u : L) agregarArista(S, u, 1);
106 // for (int v : R) agregarArista(T + n, v, 1); // ajustar
107 // indices
108 // for (auto [u, v] : compatibles) agregarArista(u, v, 1);
109 // long long matching = flujoMaximo(S, T);
110
111 // 2. MULTIPLES FUENTES / SUMIDROS: mismo truco de
112 // superfuente
113 // S / supersumidero T que en Ford-Fulkerson y Edmonds-
114 // Karp,
115 // sin cambios al algoritmo:
116 // int S = n, T = n + 1;
117 // for (int fuente : fuentesReales)
118 //     agregarArista(S, fuente, LLONG_MAX);
119 // for (int sumidero : sumiderosReales)
120 //     agregarArista(sumidero, T, LLONG_MAX);
121
122 // 3. RECUPERAR EL CORTE MINIMO tras flujoMaximo(): igual que
123 // en
124 // Ford-Fulkerson -- el lado S del corte es el conjunto de
125 // nodos con nivel[] != -1 EN LA ULTIMA LLAMADA A bfs()
126 // que
127 // devolvio false (esa es la ultima vez que se calcularon
128 // niveles antes de que t dejara de ser alcanzable):
129 //
130 // vector<bool> ladoS() {
131 //     vector<bool> resultado(n, false);
132 //     for (int u = 0; u < n; u++)

```

```

122 // resultado[u] = (nivel[u] != -1);
123 // return resultado;
124 // }
125 // -- llamar DESPUES de flujoMaximo(); nivel[] queda de la
126 // ultima corrida de bfs() (la que fallo en alcanzar t)
127
128 // 4. SCALING (capacidades MUY grandes, V y E tambien grandes
129 //):
130 // variante avanzada que procesa las aristas en orden
131 // decreciente de "bits de capacidad" para acotar aun mas
132 // las
133 // fases -- rara vez necesaria en competencia, Dinic
134 // simple ya
135 // cubre la gran mayoria de limites de tiempo tipicos.
136
137 // 5. GRAFOS CON ARISTAS NO DIRIGIDAS: se agregan como DOS
138 // aristas dirigidas independientes, cada una con su
139 // PROPIA
140 // reversa de capacidad 0 (no se comparte una sola reversa
141 // entre ambos sentidos, a diferencia de Hierholzer con su
142 // id compartido) -- Dinic no necesita ningun ajuste extra
143 // para esto, es solo como se cargan las aristas:
144 //
145 // void agregarAristaNoDirigida(int u, int v, long long c) {
146 //     agregarArista(u, v, c);
147 //     agregarArista(v, u, c);
148 // }
149
150 };

```

Ejemplo de funcionamiento:

```

1 // misma red que en Ford-Fulkerson y Edmonds-Karp:
2 // s -(3)-> a -(2)-> t
3 // s -(2)-> b -(3)-> t
4 // a -(1)-> b
5
6 Dinic din(4);
7 din.agregarArista(0, 1, 3);
8 din.agregarArista(0, 2, 2);
9 din.agregarArista(1, 3, 2);
10 din.agregarArista(2, 3, 3);
11 din.agregarArista(1, 2, 1);
12
13 cout << din.flujoMaximo(0, 3);
14 // 4 (mismo flujo maximo que las dos versiones anteriores --
15 // Dinic solo cambia CUANTAS fases/iteraciones toma llegar,
16
17 // no el resultado)

```

Regla práctica: las tres estructuras de esta sección (Ford-Fulkerson, Edmonds-Karp, Dinic) comparten **exactamente** la misma representación de grafo residual (Arista, agregarArista) y siempre dan el mismo flujo máximo — la progresión natural es empezar con Ford-Fulkerson por simplicidad, subir a Edmonds-Karp si las capacidades son grandes, y solo llegar a Dinic cuando E es grande y $O(V \cdot E^2)$ no alcanza el límite de tiempo. Si el problema es específicamente bipartite matching, Dinic es casi siempre la mejor opción de entrada por su complejidad especial en capacidad unitaria.

21.4 Maximum Bipartite Matching

Idea: Bipartite Matching (ya visto con Kuhn en la sección de Bipartite Graphs) es un caso particular de flujo máximo: se construye una red con una **superfuente** S conectada a cada nodo de L (capacidad 1), cada nodo de L conectado a los nodos de R con los que es compatible (capacidad 1), y cada nodo de R conectado a un **supersumidero** T (capacidad 1). Como **todas** las capacidades son 1, cualquier unidad de flujo de S a T corresponde exactamente a "un nodo de L emparejado con un nodo de R " —el flujo máximo de S a T es, literalmente, el tamaño del matching máximo (y no puede ser mayor porque cada nodo de L/R tiene capacidad de entrada/salida 1, así que participa en como mucho un emparejamiento).

¿Por qué usarlo? Es la alternativa a Kuhn cuando ya tienes infraestructura de flujo (Dinic) escrita y prefieres

reutilizarla en vez de mantener dos implementaciones separadas —además, en el caso particular de capacidad unitaria, Dinic corre en $O(E\sqrt{V})$, **mejor** que el $O(V \cdot E)$ de Kuhn simple. Si V es pequeño ($\lesssim 1000$ – 2000), Kuhn sigue siendo más simple de escribir bajo presión; si V es grande, Dinic + esta construcción es la opción más rápida disponible en esta guía.

Casos de uso:

- Los mismos que Bipartite Matching (asignación óptima uno-a-uno entre dos conjuntos), cuando V es lo bastante grande para que $O(E\sqrt{V})$ de Dinic gane frente a $O(V \cdot E)$ de Kuhn.
- Como building block de **matching con capacidades** —variante donde cada nodo de L o R puede emparejarse con **más de uno** del otro lado (cambiando la capacidad de las aristas $S \rightarrow L$ o $R \rightarrow T$ de 1 a ese límite, ver modificaciones abajo).
- Cuando el problema **ya** pide flujo máximo por otra razón y el matching bipartito es solo una parte del modelo completo —reutilizar la misma estructura de Dinic evita duplicar código.

Complejidad: $O(E\sqrt{V})$ con Dinic (gracias a la propiedad especial de grafos de capacidad unitaria), frente a $O(V \cdot E)$ de Kuhn — construcción de la red en $O(V + E)$.

```

1 struct MaximumBipartiteMatching {
2     Dinic dinic;
3     int nL, nR, S, T;
4
5     // CREATE: construye la red de flujo para matching bipartito.
6     // nL, nR = tamanos de los lados L y R. compatibles[u] =
7     // lista de
8     // nodos de R compatibles con el nodo u de L.
9     MaximumBipartiteMatching(int nL_, int nR_,
10     vector<vector<int>>& compatibles)
11     : dinic(nL_ + nR_ + 2), nL(nL_), nR(nR_) {
12         S = nL + nR; // superfuente
13         T = nL + nR + 1; // supersumidero
14
15         for (int u = 0; u < nL; u++)
16             dinic.agregarArista(S, u, 1);
17
18         for (int v = 0; v < nR; v++)
19             dinic.agregarArista(nL + v, T, 1);
20
21         for (int u = 0; u < nL; u++)
22             for (int v : compatibles[u])
23                 dinic.agregarArista(u, nL + v, 1);
24     } // O(V + E)
25
26     // READ: tamaño del matching maximo.
27     long long matchingMaximo() {
28         return dinic.flujoMaximo(S, T);
29     } // O(E * sqrt(V))
30
31     // READ: reconstruye la asignacion explicita tras calcular el
32     // matching maximo -- recorre las aristas u->v (u en L)
33     // buscando
34     // cuales quedaron con capacidad residual 0 (es decir, con 1
35     // unidad de flujo pasando por ellas).
36     vector<int> obtenerAsignacion() {
37         vector<int> pareja(nL, -1);
38
39         for (int u = 0; u < nL; u++)
40             for (auto& a : dinic.adj[u])
41                 if (a.destino >= nL && a.destino < nL + nR &&
42                     a.capacidad == 0) // capacidad original era 1,
43                                     // si quedo en 0 es que se uso
44                     pareja[u] = a.destino - nL;
45
46         return pareja;
47     } // O(E)
48
49     //
50     // -----
51     // MODIFICACIONES Y COMENTARIOS CON LOS USOS

```

```

49 // -----
50
51 // 1. MATCHING CON CAPACIDADES (cada nodo de L o R puede
52 // emparejarse con MAS de uno del otro lado, hasta un
53 // limite):
54 // en vez de capacidad 1 en las aristas S->L y R->T, se
55 // usa el
56 // limite real de cada nodo -- el resto de la construccion
57 // (L->R sigue siendo capacidad 1, o el limite de esa
58 // pareja
59 // especifica si aplica) no cambia:
60 //
61 // for (int u = 0; u < nL; u++)
62 //     dinic.agregarArista(S, u, limiteL[u]); // en vez de
63 //     1
64 // for (int v = 0; v < nR; v++)
65 //     dinic.agregarArista(nL + v, T, limiteR[v]); // en
66 //     vez de 1
67
68 // 2. MATCHING PONDERADO MAXIMO (cada arista L-R tiene un
69 // peso, y
70 // se busca el matching de mayor peso total, no solo el de
71 // mayor CANTIDAD de parejas): flujo maximo NO resuelve
72 // esto
73 // directamente -- hace falta flujo de COSTO minimo/maximo
74 // (min-cost max-flow), fuera del alcance de esta
75 // plantilla
76 // base. La señal para reconocerlo es que el enunciado da
77 // un
78 // PESO o COSTO por pareja posible, no solo compatibilidad
79 // binaria.
80
81 // 3. VERIFICAR SI EL MATCHING ES PERFECTO (todos los nodos
82 // de L
83 // Y todos los de R quedan emparejados): matching es
84 // perfecto
85 // si y solo si matchingMaximo() == nL == nR (mismo tamaño
86 // en
87 // ambos lados y todos emparejados):
88 //
89 // bool esPerfecto() {
90 //     return matchingMaximo() == nL && nL == nR;
91 // }
92
93 // 4. HALLAR UN CONJUNTO DE VERTICES NO CUBIERTOS (nodos de L
94 // o
95 // R que quedaron SIN pareja tras el matching maximo) --
96 // util
97 // para problemas que piden identificar el "excedente":
98 //
99 // vector<int> noEmparejadosEnL() {
100 //     auto pareja = obtenerAsignacion();
101 //     vector<int> resultado;
102 //     for (int u = 0; u < nL; u++)
103 //         if (pareja[u] == -1) resultado.push_back(u);
104 //     return resultado;
105 // }
106
107 // 5. TEOREMA DE KONIG (cobertura minima de vertices =
108 // matching
109 // maximo en grafos bipartitos): la cobertura minima se
110 // obtiene a partir del corte minimo de esta MISMA red de
111 // flujo -- ver Min-Cut para la extraccion del lado S del
112 // corte; los nodos de L NO alcanzables desde S en el
113 // residual
114 // final, mas los nodos de R SI alcanzables, forman la
115 // cobertura minima de vertices.
116
117 };

```

Ejemplo de funcionamiento:

```

1 // L = {0, 1, 2}, R = {0, 1, 2}
2 // 0 - {0, 1}
3 // 1 - {0}
4 // 2 - {1, 2}
5
6 vector<vector<int>> compatibles(3);
7 compatibles[0] = {0, 1};
8 compatibles[1] = {0};
9 compatibles[2] = {1, 2};
10
11 MaximumBipartiteMatching mbm(3, 3, compatibles);
12
13 cout << mbm.matchingMaximo();
14 // 3 (mismo resultado que el ejemplo de Kuhn en Bipartite
15 // Matching)
16
17 auto pareja = mbm.obtenerAsignacion();
18 for (int u = 0; u < 3; u++)
19     cout << u << " - " << pareja[u] << " ";
20 // alguna asignacion valida, ej: 0-1 1-0 2-2

```

Regla práctica: Kuhn y Maximum Bipartite Matching vía Dinic **siempre** dan el mismo tamaño de matching máximo (ambos resuelven exactamente el mismo problema) — la elección es de rendimiento y de qué código ya tienes a mano: con V pequeño y sin Dinic ya escrito, Kuhn es más corto; con V grande o Dinic ya disponible de otra parte del problema, esta construcción vía flujo es la opción más rápida y la que se generaliza directo a matching con capacidades.

22 2-SAT

22.1 Implication Graph

Idea: 2-SAT pide encontrar una asignación de verdad para n variables booleanas que satisfaga una fórmula compuesta por **cláusulas de a lo más 2 literales** (ej. $(A \vee B)$, $(\neg A \vee C)$). La clave para resolverlo en tiempo lineal (en vez de probar 2^n asignaciones) es notar que cada cláusula $(a \vee b)$ es **lógicamente equivalente** a dos implicaciones: $\neg a \Rightarrow b$ y $\neg b \Rightarrow a$ (si uno de los dos literales es falso, el otro **debe** ser verdadero). Modelando cada literal (variable y su negación) como un nodo de un grafo dirigido, y cada implicación como una arista, la fórmula completa se convierte en un **grafo de implicación** de $2n$ nodos: el nodo $2i$ representa " x_i es verdadero", el nodo $2i+1$ representa " x_i es falso" (o viceversa, según la convención elegida).

¿Por qué usarlo? Es el paso de **modelado** que convierte un problema de satisfacibilidad booleana en un problema de grafos puro —una vez construido el grafo de implicación, resolver 2-SAT deja de requerir ningún razonamiento lógico adicional: se reduce enteramente a encontrar Componentes Fuertemente Conexas (ver 2-SAT con SCC, siguiente subsección). La parte que sí exige cuidado es traducir **correctamente** cada tipo de restricción a su par de implicaciones —un error aquí produce un grafo que no representa la fórmula real, aunque el algoritmo de SCC corra sin errores.

Casos de uso: construir el grafo de implicación es el primer paso **obligatorio** de cualquier problema de 2-SAT, sin importar cuál sea la restricción concreta. Los patrones de traducción más comunes (cada uno se agrega como `add_disjunction` sobre la estructura de 2-SAT con SCC):

- Forzar una variable a verdadero o falso (restricción unaria, disfrazada de cláusula con el mismo literal repetido).
- Implicación simple $A \Rightarrow B$ (cláusula $(\neg A \vee B)$).
- "Al menos uno" de dos literales (cláusula OR directa).
- "A lo más uno" / mutuamente excluyentes (cláusula NAND).
- Equivalencia $A \leftrightarrow B$ (dos cláusulas: ambas direcciones de la implicación).
- Diferencia forzada $A \oplus B$ (dos cláusulas: al menos uno, y no ambos).

Complejidad: $O(1)$ por cláusula agregada al grafo (cada cláusula agrega una cantidad constante de nodos/aristas), $O(V + E) = O(n + m)$ para el grafo completo con n variables y m cláusulas.

```

1 // convencion de esta plantilla: variable i se representa con
2 // los
3 // nodos 2*i (literal POSITIVO: x_i) y 2*i+1 (literal NEGATIVO:
4 // !x_i)
5
6 struct GrafoImplicacion {
7     int nVars, nVertices;
8     vector<vector<int>> adj; // grafo de implicacion (dirigido)
9
10    // CREATE: n variables booleanas -> 2n nodos en el grafo.
11    GrafoImplicacion(int nVars_)
12    : nVars(nVars_), nVertices(2 * nVars_), adj(2 * nVars_) {}
13    // 0(n)
14
15    // WRITE: agrega la clausula (literal_a OR literal_b), donde
16    // na/nb indican si ese literal esta NEGADO. Traduce la
17    // clausula
18    // a sus dos implicaciones equivalentes:
19    // !a -> b y !b -> a
20    void agregarClausula(int a, bool na, int b, bool nb) {
21        int litA = 2 * a ^ na; // nodo del literal a (negado o no)
22        int litB = 2 * b ^ nb;
23        int negA = litA ^ 1; // nodo del literal OPUESTO a litA
24        int negB = litB ^ 1;
25
26        adj[negA].push_back(litB); // !a -> b
27        adj[negB].push_back(litA); // !b -> a
28    } // 0(1)
29 };

```

Ejemplo de funcionamiento:

```

1 // formula: (x0 OR x1) AND (!x0 OR x2)
2 // 2 clausulas, 3 variables (nodos 0..5):
3 // 0=x0, 1=!x0, 2=x1, 3=!x1, 4=x2, 5=!x2)
4
5 GrafoImplicacion gi(3);
6
7 gi.agregarClausula(0, false, 1, false); // (x0 OR x1)
8 gi.agregarClausula(0, true, 2, false); // (!x0 OR x2)
9
10 // aristas generadas:
11 // clausula 1: !x0 -> x1 (1 -> 2), !x1 -> x0 (3 -> 0)
12 // clausula 2: x0 -> x2 (0 -> 4), !x2 -> !x0 (5 -> 1)
13
14 for (int v : gi.adj[1]) cout << v << " ";
15 // 2 (!x0 -> x1)
16
17 for (int v : gi.adj[0]) cout << v << " ";
18 // 4 (x0 -> x2)

```

Regla práctica: sin importar qué tan compleja parezca la restricción original (implicación, equivalencia, exclusión mutua, etc.), siempre se puede descomponer en una o dos cláusulas OR de 2 literales, y cada cláusula siempre agrega exactamente 2 aristas al grafo de implicación con la misma fórmula $\neg a \Rightarrow b$, $\neg b \Rightarrow a$ —memorizar esa única fórmula es más robusto que memorizar un caso especial por cada tipo de restricción.

22.2 2-SAT con SCC

Idea: una vez construido el grafo de implicación, la fórmula es **satisfacible** si y solo si **ninguna** variable x_i tiene sus dos literales (x_i y $\neg x_i$) en la **misma** componente fuertemente conexa —si estuvieran en la misma SCC, significaría que $x_i \Rightarrow \neg x_i$ y $\neg x_i \Rightarrow x_i$ simultáneamente, una contradicción directa. Si la fórmula es satisfacible, la asignación se obtiene comparando el **orden topológico inverso** de las SCCs (que Tarjan/Kosaraju ya dan como efecto secundario, ver esas secciones): a cada variable se le asigna **verdadero** si su literal positivo queda en una SCC **más tarde** en ese orden (equivalentemente, con **comp** más alto si se usa el **id** de Kosaraju/Tarjan de esta plantilla).

¿Por qué usarlo? Es la razón de ser de todo el modelado con Implication Graph: convierte "resolver una fórmula booleana" en "una sola pasada de SCC", sin backtracking ni búsqueda exponencial. Es además **constructivo** —no solo dice si existe solución, sino que la construye directamente comparando los **ids** de componente, sin ningún paso adicional de búsqueda.

Casos de uso:

- Cualquier problema modelable como restricciones binarias entre variables booleanas: asignación de turnos con conflictos, selección de una opción entre pares incompatibles, coloreo con solo 2 colores sujeto a restricciones.
- Verificar la **existencia** de una asignación válida sin necesitar construirla explícitamente (basta el chequeo de "misma SCC").
- Como building block de problemas más grandes que reducen su núcleo de decisión a 2-SAT (ej. "elige un valor de dos opciones por cada elemento, sujeto a restricciones de pares").

Complejidad: $O(n+m)$ — construir el grafo es $O(m)$ (una cláusula, trabajo constante), Kosaraju/Tarjan sobre el grafo de $2n$ nodos es $O(n+m)$, y la asignación final es $O(n)$.

```

1 struct TwoSatSolver {
2     int n_vars;
3     int n_vertices;
4     vector<vector<int>> adj, adj_t;
5     vector<bool> used;
6     vector<int> order, comp;
7     vector<bool> assignment;
8
9     // CREATE: n_vars variables -> 2*n_vars nodos (grafo de
10    // implicacion + su transpuesto, para Kosaraju).
11    TwoSatSolver(int _n_vars) : n_vars(_n_vars), n_vertices(2 *
12    n_vars),
13    adj(n_vertices), adj_t(n_vertices),
14    used(n_vertices), order(),
15    comp(n_vertices, -1), assignment(n_vars) {
16        order.reserve(n_vertices);
17    } // 0(n)
18
19    void dfs1(int v) {
20        used[v] = true;
21        for (int u : adj[v])
22            if (!used[u])
23                dfs1(u);
24        order.push_back(v);
25    }
26
27    void dfs2(int v, int c1) {
28        comp[v] = c1;
29        for (int u : adj_t[v])
30            if (comp[u] == -1)
31                dfs2(u, c1);
32    }
33
34    // READ: resuelve la formula (Kosaraju: dfs1 sobre el grafo
35    // original, dfs2 sobre el transpuesto). Devuelve false si NO
36    // es
37    // satisfacible (algun x_i y !x_i quedaron en la misma SCC).
38    bool solve_2SAT() {
39        order.clear();
40        used.assign(n_vertices, false);
41        for (int i = 0; i < n_vertices; ++i)
42            if (!used[i])
43                dfs1(i);
44
45        comp.assign(n_vertices, -1);
46        for (int i = 0, j = 0; i < n_vertices; ++i) {
47            int v = order[n_vertices - i - 1];
48            if (comp[v] == -1)
49                dfs2(v, j++);
50        }
51
52        assignment.assign(n_vars, false);
53        for (int i = 0; i < n_vertices; i += 2) {

```

```

52     if (comp[i] == comp[i + 1])
53         return false; // x_i y !x_i en la misma SCC ->
54         // contradiccion, no satisfacible
55
56     // comp[] queda en orden topologico INVERSO (Kosaraju
57     // procesa por orden de salida decreciente); el literal
58     // con comp MAYOR aparece "despues" en ese orden inverso,
59     // asi que es el que se elige como verdadero
60     assignment[i / 2] = comp[i] > comp[i + 1];
61 }
62
63 return true;
64 } // O(n + m)
65
66 // WRITE: agrega la clausula (literal a con negacion na) OR
67 // (literal b con negacion nb). Ver Implication Graph para la
68 // deducccion de estas dos aristas.
69 void add_disjunction(int a, bool na, int b, bool nb) {
70     a = 2 * a ^ na;
71     b = 2 * b ^ nb;
72     int neg_a = a ^ 1;
73     int neg_b = b ^ 1;
74     adj[neg_a].push_back(b);
75     adj[neg_b].push_back(a);
76     adj_t[b].push_back(neg_a);
77     adj_t[a].push_back(neg_b);
78 } // O(1)
79
80 //
81 // -----
82 // PATRONES 2-SAT COMPLETOS (modificaciones y usos)
83 // -----
84
85 // 1. A debe ser verdadero (forzar A = true)
86 // Clausula: (A OR A), equivale a !A => A
87 // solver.add_disjunction(i, false, i, false);
88
89 // 2. A debe ser falso (forzar A = false)
90 // Clausula: (!A OR !A), equivale a A => !A
91 // solver.add_disjunction(i, true, i, true);
92
93 // 3. Si A entonces B (A => B)
94 // Clausula: (!A OR B), equivale a A => B, !B => !A
95 // solver.add_disjunction(i, true, j, false);
96
97 // 4. Si no A entonces no B (!A => !B)
98 // Clausula: (A OR !B), equivale a !A => !B, B => A
99 // solver.add_disjunction(i, false, j, true);
100
101 // 5. Al menos uno verdadero (A OR B)
102 // Clausula: (A OR B), equivale a !A => B, !B => A
103 // solver.add_disjunction(i, false, j, false);
104
105 // 6. A lo mas uno verdadero / mutuamente excluyentes (!A OR !B)
106 // Clausula: (!A OR !B), equivale a A => !B, B => !A
107 // solver.add_disjunction(i, true, j, true);
108
109 // 7. A y B deben ser iguales (A <=> B)
110 // Clausulas: (A OR !B) y (!A OR B)
111 // solver.add_disjunction(i, false, j, true); // B => A
112 // solver.add_disjunction(i, true, j, false); // A => B
113
114 // 8. A y B deben ser diferentes (A XOR B)
115 // Clausulas: (A OR B) y (!A OR !B)
116 // solver.add_disjunction(i, false, j, false); // !A=>B, !B
117 // =>A
118 // solver.add_disjunction(i, true, j, true); // A=>!B, B
119 // =>!A
120 };
```

Ejemplo de funcionamiento:

```

1 // formula: (x0 OR x1) AND (!x0 OR x2) AND (x1 debe ser
2 // verdadero)
3 TwoSatSolver solver(3); // variables 0, 1, 2
4
5 solver.add_disjunction(0, false, 1, false); // (x0 OR x1)
6 solver.add_disjunction(0, true, 2, false); // (!x0 OR x2)
7 solver.add_disjunction(1, false, 1, false); // forzar x1 =
8 // true
9
10 bool satisfacible = solver.solve_2SAT();
11
12 cout << satisfacible;
13 // true
14
15 cout << solver.assignment[1];
16 // true (forzado explicitamente)
```

Regla práctica: el chequeo de satisfacibilidad ($\text{comp}[i] == \text{comp}[i+1]$) y la construcción de la asignación ($\text{comp}[i] > \text{comp}[i+1]$) son **inseparables** del mismo recorrido de SCC —no hace falta ninguna estructura ni pasada adicional para obtener la asignación una vez que la fórmula resultó satisfacible; toda la dificultad real de 2-SAT está en el paso de Implication Graph (traducir correctamente cada restricción), no en esta parte, que es siempre el mismo código sin importar el problema.

23 Backtracking y Problemas Clásicos

23.1 Problema de las N Reinas

Idea: colocar N reinas en un tablero $N \times N$ tal que **ninguna** ataque a otra (ni misma fila, ni misma columna, ni ninguna diagonal). Se resuelve con backtracking colocando **una reina por fila** (eso ya garantiza que nunca compartan fila): en cada fila, se prueba cada columna disponible, se verifica que no choque con las reinas ya colocadas, se coloca y se avanza a la siguiente fila; si ninguna columna de la fila actual funciona, se retrocede ("backtrack") y se prueba la siguiente opción de la fila anterior. La clave de eficiencia es **podar temprano**: en vez de recalculando columnas/diagonales ocupadas recorriendo todas las reinas ya puestas, se mantienen tres arreglos booleanos (columna, diagonal fila+columna, diagonal fila-columna) que se marcan y desmarcan en $O(1)$.

¿Por qué usarlo? Es el ejemplo canónico de backtracking con poda por restricciones: sin las tres marcas de columna/diagonales, la verificación de "¿esta posición es válida?" costaría $O(N)$ por candidato (recorrer todas las reinas puestas), multiplicando el costo total. Con las marcas, cada verificación es $O(1)$, dejando que la poda temprana (cortar ramas inválidas apenas se detectan) sea lo que realmente controla el tamaño del árbol de búsqueda explorado.

Casos de uso:

- Encontrar **una** solución válida para N reinas.
- Contar **cuántas** soluciones distintas existen para un N dado (variante más común en jueces de práctica).
- Listar/imprimir **todas** las soluciones explícitas (cada una como una permutación de columnas, una por fila).
- Como plantilla base de backtracking con restricciones posicionales, reutilizable para variantes con obstáculos u otras piezas de ajedrez (ver Variaciones, siguiente subsección).

Complejidad: exponencial en el peor caso ($O(N!)$ aproximado sin poda), pero la poda por columna/diagonales en $O(1)$ hace que en la práctica sea manejable hasta $N \sim 15-20$ en tiempo de competencia.

```

1 struct NReinas {
2     int n;
3     vector<int> columnaPorFila; // columnaPorFila[fila] =
4     // columna
5     // donde se puso la reina de esa fila
6     vector<bool> colOcupada, diag1Ocupada, diag2Ocupada;
7     long long soluciones;
8
9     // CREATE: tablero n x n vacio.
```

```

9  NReinas(int n_) {
10     n = n_;
11     columnaPorFila.assign(n, -1);
12     col0cupada.assign(n, false);
13     diag10cupada.assign(2 * n, false); // diagonal fila+
14     // columna
15     diag20cupada.assign(2 * n, false); // diagonal fila-
16     // columna+n
17     soluciones = 0;
18 } // O(n)

// READ: cuenta todas las soluciones validas via backtracking.

19 long long contarSoluciones() {
20     soluciones = 0;
21     backtrack(0);
22     return soluciones;
23 } // O(N!) peor caso

// mucho menor en
// la practica con poda

24 void backtrack(int fila) {
25     if (fila == n) {
26         soluciones++; // una reina por fila, todas colocadas
27         // sin conflicto -> solucion completa
28         return;
29     }

30     for (int col = 0; col < n; col++) {
31         int d1 = fila + col;
32         int d2 = fila - col + n;

33         if (col0cupada[col] || diag10cupada[d1] || diag20cupada[
34             d2])
35             continue; // poda: esta columna/diagonal ya esta
36             // atacada, no vale la pena seguir

37         // coloca la reina
38         columnaPorFila[fila] = col;
39         col0cupada[col] = diag10cupada[d1] = diag20cupada[d2] =
40             true;

41         backtrack(fila + 1);

42         // deshace ("backtrack" real)
43         col0cupada[col] = diag10cupada[d1] = diag20cupada[d2] =
44             false;
45     }
46 }

// READ: encuentra UNA solucion (se detiene en la primera).
// Devuelve true si existe, dejando columnaPorFila[] con la
// solucion encontrada.
47 bool encontrarUna() {
48     return backtrackUna(0);
49 }

50 bool backtrackUna(int fila) {
51     if (fila == n) return true;

52     for (int col = 0; col < n; col++) {
53         int d1 = fila + col;
54         int d2 = fila - col + n;

55         if (col0cupada[col] || diag10cupada[d1] || diag20cupada[
56             d2])
57             continue;

58         columnaPorFila[fila] = col;
59         col0cupada[col] = diag10cupada[d1] = diag20cupada[d2] =
60             true;

61         if (backtrackUna(fila + 1)) return true; // exito:
62         // corta
63         // de inmediato,
64         // sin deshacer
65         // (ya se
66         // encontro
67         // la respuesta)

68         col0cupada[col] = diag10cupada[d1] = diag20cupada[d2] =
69             false;
70     }
71     return false;
72 }
73 };

```

Ejemplo de funcionamiento:

```

1  NReinas nr(8);
2

```

```

3  cout << nr.contarSoluciones();
4  // 92 (el numero clasico de soluciones para 8 reinas)
5
6  NReinas nr2(4);
7  bool encontrada = nr2.encontrarUna();
8
9  cout << encontrada;
10 // true
11
12 for (int c : nr2.columnaPorFila) cout << c << " ";
13 // 1 3 0 2 (una de las dos soluciones validas para 4 reinas)

```

Nota: la razón de que `contarSoluciones()` y `encontrarUna()` sean funciones separadas (en vez de una sola con un flag) es que `encontrarUna()` puede **cortar inmediatamente** al éxito sin deshacer el estado (ya no importa, terminó la búsqueda), mientras que `contarSoluciones()` **siempre** debe deshacer y seguir explorando otras ramas para encontrar el resto de soluciones — mezclar ambas lógicas en una sola función suele ser la fuente más común de bugs en esta plantilla.

23.2 Variaciones del Problema de las N Reinas

Idea: el esqueleto de backtracking de N Reinas —una decisión por fila, poda con arreglos de conflicto marcados en $O(1)$ — se reutiliza casi sin cambios para varias variantes que cambian **qué** cuenta como conflicto o **qué** se pide como respuesta, en vez de cambiar la estructura del backtracking en sí.

¿Por qué usarlo? Reconocer que una variante es "N Reinas con un ajuste" ahorra tener que rediseñar el backtracking desde cero —el patrón de poda por marcas booleanas (o bitmask, ver más abajo) es directamente transferible.

Casos de uso y sus modificaciones puntuales:

- **Tablero con casillas bloqueadas:** algunas celdas no admiten reina (obstáculos fijos) — se agrega un chequeo extra de "¿esta celda está bloqueada?" antes de colocar.
- **N Reinas con bitmask** (optimización de constante): en vez de tres `vector<bool>`, se usan tres enteros donde cada bit representa una columna/diagonal ocupada — las operaciones de marcar/desmarcar se vuelven operaciones a nivel de bits, más rápidas en la práctica para N grande.
- **Listar todas las soluciones explícitas** (no solo contarlas): se guarda una copia de `columnaPorFila` cada vez que se completa una solución, en vez de solo incrementar un contador.
- **Reinas con posiciones prefijadas:** algunas filas ya tienen su reina puesta de antemano — se inicializan las marcas de conflicto con esas reinas antes de empezar el backtracking, y el recorrido de filas empieza después de la última fila prefijada.

Complejidad: igual orden que N Reinas base (exponencial en el peor caso con poda), la variante con bitmask reduce la **constante** por operación pero no el orden asintótico.

```

1  struct NReinasVariantes {
2      int n;
3      vector<int> columnaPorFila;
4      vector<vector<bool>> bloqueada; // bloqueada[fila][col] =
5      true

```

```

5 // si esa celda no admite reina
6 vector<vector<int>> todasLasSoluciones; // guarda cada
  solucion
7 // completa encontrada
8
9 // bitmask: cols, diag1, diag2 representan columnas/
  diagonales
10 // ocupadas como bits encendidos, en vez de vector<bool>
11 int cols, diag1, diag2;
12
13 // CREATE: tablero n x n con posibles celdas bloqueadas.
14 NReinasVariantes(int n_, vector<vector<bool>>& bloqueada_) {
15     n = n_;
16     bloqueada = bloqueada_;
17     columnaPorFila.assign(n, -1);
18     cols = diag1 = diag2 = 0;
19 } // O(n^2)
20
21 // VARIANTE 1: tablero con casillas bloqueadas -- mismo
22 // backtracking, un chequeo extra por candidato.
23 long long contarConBloqueos() {
24     long long total = 0;
25     backtrackBloqueos(0, total);
26     return total;
27 }
28
29 void backtrackBloqueos(int fila, long long& total) {
30     if (fila == n) { total++; return; }
31
32     for (int col = 0; col < n; col++) {
33         if (bloqueada[fila][col] continue; // celda no
34         // disponible,
35         // ni siquiera se
36         // evalua conflicto
37
38         int d1 = fila + col, d2 = fila - col + n;
39         bool ocupada = (cols & (1 << col)) ||
40         (diag1 & (1 << d1)) ||
41         (diag2 & (1 << d2));
42         if (ocupada) continue;
43
44         // VARIANTE 2: bitmask en vez de vector<bool> -- marcar
45         // es un OR, desmarcar es un XOR con la misma mascara
46         cols |= (1 << col);
47         diag1 |= (1 << d1);
48         diag2 |= (1 << d2);
49
50         backtrackBloqueos(fila + 1, total);
51
52         cols ^= (1 << col);
53         diag1 ^= (1 << d1);
54         diag2 ^= (1 << d2);
55     }
56 } // O(N!) peor caso
57
58 // VARIANTE 3: listar TODAS las soluciones explicitas, no
59 // solo
60 // contarlas -- se guarda una copia completa al llegar a fila
61 // ==n.
62 void listarTodas() {
63     todasLasSoluciones.clear();
64     backtrackListar(0);
65 }
66
67 void backtrackListar(int fila) {
68     if (fila == n) {
69         todasLasSoluciones.push_back(columnaPorFila); // copia
70         // completa
71         return;
72     }
73
74     for (int col = 0; col < n; col++) {
75         int d1 = fila + col, d2 = fila - col + n;
76         bool ocupada = (cols & (1 << col)) ||
77         (diag1 & (1 << d1)) ||
78         (diag2 & (1 << d2));
79         if (ocupada || bloqueada[fila][col]) continue;
80
81         columnaPorFila[fila] = col;
82         cols |= (1 << col); diag1 |= (1 << d1); diag2 |= (1 << d2);
83
84         backtrackListar(fila + 1);
85
86         cols ^= (1 << col); diag1 ^= (1 << d1); diag2 ^= (1 << d2);
87     }
88 } // O(N! * N) por
89 // el
90 // costo de copiar cada
91 // solucion encontrada
92
93 // VARIANTE 4: reinas con posiciones prefijadas -- se marcan

```

```

91 las
92 // reinas ya dadas ANTES de empezar el backtracking, y este
93 // arranca en la primera fila SIN prefijar.
94 long long contarConPrefijadas(vector<int>& prefijadas) {
95     // prefijadas[fila] = columna prefijada, o -1 si esa fila
96     // es
97     // libre y debe decidirse por backtracking
98     int filaInicio = 0;
99     for (int fila = 0; fila < n; fila++) {
100         if (prefijadas[fila] == -1) { filaInicio = fila; break; }
101
102         int col = prefijadas[fila];
103         columnaPorFila[fila] = col;
104         cols |= (1 << col);
105         diag1 |= (1 << (fila + col));
106         diag2 |= (1 << (fila - col + n));
107         filaInicio = fila + 1;
108     }
109
110     long long total = 0;
111     backtrackBloqueos(filaInicio, total);
112     return total;
113 } // mismo orden que
114 // backtrackBloqueos,
115 // arrancando mas
116 // adentro del arbol

```

Ejemplo de funcionamiento:

```

1 // tablero 4x4 sin bloqueos
2 vector<vector<bool>> sinBloqueos(4, vector<bool>(4, false));
3
4 NReinasVariantes nrv(4, sinBloqueos);
5 nrv.listarTodas();
6
7 cout << nrv.todasLasSoluciones.size();
8 // 2 (las 2 soluciones classicas de 4 reinas)
9
10 for (int c : nrv.todasLasSoluciones[0]) cout << c << " ";
11 // 1 3 0 2
12
13 // tablero 4x4 CON la celda (0,0) bloqueada
14 vector<vector<bool>> conBloqueo(4, vector<bool>(4, false));
15 conBloqueo[0][0] = true;
16
17 NReinasVariantes nrv2(4, conBloqueo);
18 cout << nrv2.contarConBloqueos();
19 // 1 (una de las 2 soluciones originales usaba (0,0), la otra
  no)

```

Regla práctica: ante cualquier variante de N Reinas, identifica primero **qué cambia**: si cambia **qué es válido** (bloqueos, reinas prefijadas), el ajuste es un chequeo extra o una inicialización antes del backtracking; si cambia **qué se pide** (contar vs. listar vs. encontrar una), el ajuste es qué se hace al llegar a `fila == n` (incrementar, copiar, o retornar `true` de inmediato). El núcleo del backtracking —una decisión por fila, poda por columna/diagonales, deshacer al retroceder— casi nunca cambia entre variantes.

23.3 Tour de Caballo

Idea: el Tour de Caballo (ya visto en la sección de grillas/laberintos con la heurística de Warnsdorff) encaja también aquí como el ejemplo clásico de backtracking sobre un tablero: hay que visitar **cada** casilla exactamente una vez moviéndose como caballo de ajedrez, y a diferencia de N Reinas (una decisión por fila, con poda de conflictos), aquí la decisión es "¿a cuál de los hasta 8 saltos válidos me muevo desde la casilla actual?", con **deshacer explícito** (marcar la casilla como no visitada) cada vez que una rama se atasca antes de completar el tour. Se repite en esta sección porque el patrón de backtracking puro —sin la heurística de Warnsdorff, solo prueba y retrocede— es más instructivo para entender la mecánica general antes de optimizar.

¿Por qué usarlo? Es el contraste directo con N Reinas: en N Reinas la poda viene de **restricciones**

estructurales (columna/diagonal), aquí la única "poda" real disponible sin heurística es "no pisar una casilla ya visitada" —eso hace que el backtracking puro sea mucho más lento en la práctica (el árbol de búsqueda crece casi sin control para tableros grandes), y es justamente la motivación para agregar Warnsdorff **encima** de este mismo esqueleto cuando el tablero es más grande que unos pocos $n \times n$.

Casos de uso:

- Encontrar (o verificar la existencia de) un tour de caballo completo en tableros pequeños ($n \lesssim 5-6$ sin Warnsdorff).
- Como plantilla base para introducir el patrón de "backtracking con deshacer explícito sobre una grilla", antes de agregar la heurística de ordenamiento de candidatos.
- Punto de comparación directo contra N Reinas: mismo esqueleto (probar, recursar, deshacer si falla), pero con condición de éxito distinta (visitar **todas** las casillas en vez de colocar N piezas sin conflicto).

Complejidad: exponencial sin poda adicional ($O(8^n)$ aproximado en el peor caso) — ver la sección de grillas para la versión con Warnsdorff, que en la práctica corre casi lineal para n razonables.

```

1 struct TourCaballoBacktracking {
2     int n;
3     vector<vector<int>> tablero; // tablero[r][c] = orden de
4         // (-1 si no visitada)
5     int dr[8] = {-2,-2,-1,-1, 1, 1, 2, 2};
6     int dc[8] = {-1, 1,-2, 2,-2, 2,-1, 1};
7
8     // CREATE: tablero n x n vacio.
9     TourCaballoBacktracking(int n_) {
10         n = n_;
11         tablero.assign(n, vector<int>(n, -1)); // O(n^2)
12     }
13
14     bool dentro(int r, int c) {
15         return r >= 0 && r < n && c >= 0 && c < n;
16     }
17
18     // READ: intenta construir un tour completo empezando en
19     // (sr, sc), SIN ninguna heurística de ordenamiento -- prueba
20     // los 8 saltos en orden fijo, retrocede si se atasca.
21     bool resolver(int sr, int sc) {
22         tablero[sr][sc] = 0;
23         bool exito = backtrack(sr, sc, 1);
24         if (!exito) tablero[sr][sc] = -1; // deshacer tambien el
25         // punto de partida si
26         // no se encontro nada
27         return exito;
28     } // exponencial sin
29     // poda
30
31     bool backtrack(int r, int c, int movimiento) {
32         if (movimiento == n * n) return true; // todas las
33         // casillas visitadas
34         for (int d = 0; d < 8; d++) {
35             int nr = r + dr[d], nc = c + dc[d];
36
37             if (!dentro(nr, nc) || tablero[nr][nc] != -1)
38                 continue; // fuera del tablero o ya visitada
39
40             tablero[nr][nc] = movimiento; // prueba
41
42             if (backtrack(nr, nc, movimiento + 1))
43                 return true; // exito: no hace falta deshacer, ya
44                 // se encontro la respuesta completa
45
46             tablero[nr][nc] = -1; // deshacer (backtrack real):
47             // esta rama no llevo a un tour
48             // completo
49         }
50
51         return false; // ningun salto desde (r,c) llevo a exito

```

```

52     }
53 };

```

Ejemplo de funcionamiento:

```

1 TourCaballoBacktracking tcb(5);
2
3 bool encontrado = tcb.resolver(0, 0);
4
5 cout << encontrado;
6 // true
7
8 // tcb.tablero[r][c] tiene el orden (0..24) en que se visito
9 // cada
10 // casilla -- sin Warnsdorff, el orden exacto encontrado puede
11 // diferir del de la version con heurística, pero ambos son
12 // tours
13 // validos si existen

```

Regla práctica: este esqueleto (probar candidato, recursar, deshacer si la rama falla) es el mismo de N Reinas y de cualquier backtracking clásico —lo único que cambia entre problemas es **qué** se prueba en cada paso (columna vs. salto de caballo) y **cuándo** se declara éxito (fila == n vs. movimiento == $n*n$). Para tableros mayores a unos pocos $n \times n$, agrega la heurística de Warnsdorff (sección de grillas) **encima** de este mismo backtracking —no cambia la estructura, solo el **orden** en que se prueban los 8 saltos en cada paso.

24 Advanced Graph Algorithms

24.1 Centroid Decomposition

Idea: Centroid Decomposition descompone un árbol en una **jerarquía de centroides** para resolver queries que involucran "caminos entre cualquier par de nodos" en $O(\log n)$ niveles: se encuentra el **centroide** del árbol (el nodo cuya remoción deja componentes de tamaño $\leq n/2$ cada una — no confundir con Centro de un Árbol, que minimiza distancia máxima; el centroide minimiza tamaño de componente al remover), se procesa toda la información que pasa **por** ese centroide, se **remueve** (lógicamente) del árbol, y se **recursiona** de forma independiente sobre cada componente resultante, cada una con su propio centroide. Como cada remoción parte el árbol en piezas de tamaño a lo más la mitad, la profundidad de esta jerarquía de centroides es $O(\log n)$ — y cualquier camino entre dos nodos u, v del árbol original pasa por **exactamente un** centroide de la jerarquía (el primero, en la recursión, que los separa en componentes distintas).

¿Por qué usarlo? Es la técnica estándar para problemas de "cuenta/encuentra pares de nodos que cumplen una condición sobre su camino" (ej. "cuántos pares están a distancia $\leq k$ "), donde recorrer todos los pares es $O(n^2)$: al procesar cada nivel de la jerarquía en $O(n \log n)$ total y haber solo $O(\log n)$ niveles, el costo total baja a $O(n \log^2 n)$. Es conceptualmente el análogo, en árboles, de *divide and conquer* sobre arreglos —"encuentra el punto medio, resuelve lo que cruza el medio, recursiona a los lados" —salvo que el "medio" aquí es un nodo (el centroide), no un índice.

Casos de uso:

- Contar pares de nodos (u, v) cuya distancia es **exactamente** k , o está en un rango dado — proceso: para cada centroide, contar cuántos caminos que pasan por él tienen la distancia buscada (usando las

distancias de cada nodo al centroide, agrupadas en un arreglo/mapa de frecuencias).

- Encontrar el par de nodos más cercano/lejano que cumple una restricción adicional (ej. "de distinto color").
- Responder queries offline de tipo "actualiza un nodo, consulta el más cercano nodo actualizado" (una aplicación frecuente combinada con la estructura de centroides para acotar la distancia recorrida por query).
- Como alternativa a Heavy-Light Decomposition cuando el problema es sobre **pares** de caminos, no sobre un único camino consultado repetidamente.

Complejidad: $O(n \log n)$ para construir toda la jerarquía (encontrar centroides, un DFS de tamaños por cada nivel de recursión — la suma total de tamaños across niveles es $O(n \log n)$); el costo de procesar cada centroide depende del problema, típicamente $O(\text{tam} \log \text{tam})$ por centroide, dando $O(n \log^2 n)$ total.

```

1 struct CentroidDecomposition {
2     vector<vector<int>> adj;
3     vector<int> tam;
4     vector<bool> removido;
5     vector<int> padreCentroide; // padreCentroide[c] =
6         // centroide
7     // padre de c en la jerarquia
8     int n;
9
10    // CREATE: guarda el arbol y arranca la descomposicion
11    // completa
12    // desde la raiz de la jerarquia (nodo -1 = sin padre
13    // centroide).
14    CentroidDecomposition(vector<vector<int>>& adj_, int n_) {
15        adj = adj_;
16        n = n_;
17        tam.assign(n, 0);
18        removido.assign(n, false);
19        padreCentroide.assign(n, -1);
20
21        descomponer(0, -1); // O(n log n)
22
23    int calcularTam(int u, int padre) {
24        tam[u] = 1;
25        for (int v : adj[u])
26            if (v != padre && !removido[v])
27                tam[u] += calcularTam(v, u);
28        return tam[u];
29    }
30
31    // encuentra el centroide del componente que contiene a u,
32    // usando tamTotal (ya calculado con calcularTam) para
33    // decidir
34    // cuando un hijo excede n/2.
35    int encontrarCentroide(int u, int padre, int tamTotal) {
36        for (int v : adj[u])
37            if (v != padre && !removido[v] && tam[v] > tamTotal / 2)
38                return encontrarCentroide(v, u, tamTotal);
39        return u;
40    }
41
42    // procesa toda la informacion que pasa POR el centroide
43    // actual
44    // -- el cuerpo especifico depende del problema (aqui vacio,
45    // ver
46    // ejemplo de "contar pares a distancia k" mas abajo).
47    void procesarCentroide(int centroide) {
48        // placeholder: aqui va la logica especifica del problema,
49        // típicamente un DFS desde 'centroide' recolectando
50        // distancias de cada nodo del componente hacia el
51    }
52
53    // WRITE (recursivo): encuentra el centroide del componente
54    // que
55    // contiene a u, lo procesa, lo marca removido, y recursiona
56    // sobre cada componente resultante.
57    void descomponer(int u, int padre) {
58        int tamTotal = calcularTam(u, -1);
59        int centroide = encontrarCentroide(u, -1, tamTotal);
60    }

```

```

55     removido[centroide] = true;
56     padreCentroide[centroide] = padre;
57
58     procesarCentroide(centroide);
59
60     for (int v : adj[centroide])
61         if (!removido[v])
62             descomponer(v, centroide);
63     } // O(tam log tam)
64     // por nivel, O(log n)
65     // niveles -> O(n log n)
66     // total
67 };

```

Ejemplo de funcionamiento (contar pares a distancia exacta k):

```

1 // version especializada de procesarCentroide para "contar
2 // pares de
3 // nodos a distancia EXACTAMENTE k" -- se agrega sobre el
4 // esqueleto
5 // de arriba:
6 //
7 // long long totalPares = 0;
8 // void procesarCentroideConteo(int centroide, int k) {
9 //     vector<int> distancias; // distancia de cada nodo
10    // del
11    // centroide // componente actual al
12    // function<void(int,int,int)> recolectar = [&](int u,
13    // int p, int d) {
14    //     distancias.push_back(d);
15    //     for (int v : adj[u])
16    //         if (v != p && !removido[v])
17    //             recolectar(v, u, d + 1);
18    //     };
19    //     // cuenta pares DENTRO de cada subarbol de un hijo
20    //     // directo del
21    //     // centroide primero (para luego RESTARLOS -- evita
22    //     // contar
23    //     // pares que en realidad no pasan por el centroide,
24    //     // ambos en
25    //     // el mismo hijo)
26    //     map<int,int> frecuencia;
27    //     frecuencia[0] = 1; // el propio centroide, distancia
28    //     0
29    //     for (int hijo : adj[centroide]) {
30    //         if (removido[hijo]) continue;
31    //         //
32    //         distancias.clear();
33    //         recolectar(hijo, centroide, 1);
34    //         for (int d : distancias)
35    //             if (frecuencia.count(k - d))
36    //                 totalPares += frecuencia[k - d];
37    //         for (int d : distancias)
38    //             frecuencia[d]++;
39    //     }
40    // }
41
42    // arbol:
43    //
44    //      0
45    //     / \
46    //    1  2
47    //   /
48    //  3
49
50    vector<vector<int>> adj(4);
51    adj[0] = {1, 2};
52    adj[1] = {0, 3};
53    adj[2] = {0};
54    adj[3] = {1};
55
56    // contar pares a distancia EXACTAMENTE 2:
57    // (0,3) dist=2, (2,1) dist=2 -> 2 pares

```

Regla práctica: Centroid Decomposition vale la pena **solo** cuando el problema pregunta por **pares** de nodos o caminos **arbitrarios** — si el problema fija un único camino por query (como en Path Queries o HLD), Centroid Decomposition es sobre-ingeniería innecesaria. La señal típica es un enunciado que pregunta "cuántos pares..." o "existe algún par..." con una condición sobre la distancia o el camino entre ellos.

24.2 DSU on Tree

Idea: DSU on Tree (también llamado *small-to-large merging*, a pesar del nombre no usa la estructura DSU de Union-Find) resuelve queries **por subárbol** —"para cada nodo u , calcula algo sobre **todo** su subárbol"—evitando recalculación desde cero para cada nodo. La idea: al procesar un nodo u , primero se procesan (y se **descartan**) todos los hijos **livianos** (los que no son el hijo de mayor subárbol, igual que en Heavy-Light Decomposition), guardando su información solo temporalmente; luego se procesa el hijo **pesado** sin descartar su información (se deja "viva"); y finalmente se **reinserta** la información de los hijos livianos encima de la del hijo pesado, para tener el estado completo del subárbol de u sin haber recorrido cada nodo más que $O(\log n)$ veces en total.

¿Por qué usarlo? La alternativa ingenua —recorrer el subárbol completo de cada nodo para responder su query— cuesta $O(n^2)$ en el peor caso (un árbol en forma de cadena). El truco de "conservar la estructura del hijo pesado y solo reinsertar los livianos" es el mismo principio que sustenta la complejidad de HLD —cualquier nodo cambia de "hijo pesado a liviano" como mucho $O(\log n)$ veces en su camino a la raíz—, aplicado aquí a cuántas veces se re-inserta cada nodo en una estructura auxiliar en vez de a cuántas cadenas cruza un camino.

Casos de uso:

- "Para cada nodo, ¿cuál es el valor/color más frecuente en su subárbol?" (el ejemplo clásico, con un arreglo de frecuencias como estructura auxiliar).
- Contar cuántos valores **distintos** hay en el subárbol de cada nodo.
- Cualquier query de subárbol donde la estructura auxiliar soporte "agregar un elemento" y "responder la pregunta actual" en $O(1)$ o $O(\log n)$, pero **no** soporte eliminar eficientemente (si soportara borrado barato, Euler Tour + Fenwick ya bastaría, sin necesitar este patrón).

Complejidad: $O(n \log n)$ total — cada nodo se "agrega" a la estructura auxiliar como mucho $O(\log n)$ veces (una vez por cada antepasado del que sea hijo liviano, más una vez cuando su propio subárbol se procesa como hijo pesado).

```

1 struct DSUOnTree {
2     vector<vector<int>> adj;
3     vector<int> tam, hijoPesado, valor;
4     vector<long long> frecuencia; // estructura auxiliar:
      cuenta
5     // ocurrencias de cada valor
6     vector<long long> respuesta; // respuesta[u] = valor mas
7     // frecuente en el subarbol de u
8     long long maxFrecuencia, sumaDeMaximos;
9     int n;
10
11     // CREATE: guarda el arbol y los valores por nodo; calcula
12     // tamanos e identifica el hijo pesado de cada nodo (igual
13     // que en HLD).
14     DSUOnTree(vector<vector<int>>& adj_, vector<int>& valor_, int
15         n_) {
16         adj = adj_;
17         valor = valor_;
18         n = n_;
19         tam.assign(n, 0);
20         hijoPesado.assign(n, -1);
21         respuesta.assign(n, 0);

```

```

21     frecuencia.assign(*max_element(valor.begin(), valor.end())
22         + 1, 0);
23
24     calcularTamYPesado(0, -1);
25 } // O(n)
26
27 void calcularTamYPesado(int u, int padre) {
28     tam[u] = 1;
29     int mejorTam = -1;
30
31     for (int v : adj[u]) {
32         if (v == padre) continue;
33         calcularTamYPesado(v, u);
34         tam[u] += tam[v];
35         if (tam[v] > mejorTam) {
36             mejorTam = tam[v];
37             hijoPesado[u] = v;
38         }
39     } // O(n)
40
41     // agrega o quita (delta = +1 o -1) el valor de u a la
42     // estructura
43     // auxiliar, actualizando el maximo global sobre la marcha.
44     void agregar(int u, int padre, int delta) {
45         frecuencia[valor[u]] += delta;
46
47         if (frecuencia[valor[u]] > maxFrecuencia) {
48             maxFrecuencia = frecuencia[valor[u]];
49             sumaDeMaximos = valor[u];
50         } else if (frecuencia[valor[u]] == maxFrecuencia) {
51             sumaDeMaximos += valor[u]; // ejemplo: suma de todos
52             // los valores empatados
53             // en frecuencia maxima
54         }
55
56         for (int v : adj[u])
57             if (v != padre)
58                 agregar(v, u, delta);
59     } // O(tamano del
60     // subarbol)
61
62     // READ (recursivo): procesa el subarbol de u con la tecnica
63     // de
64     // small-to-large; 'mantener' indica si u es el hijo PESADO
65     // de
66     // su padre (si es asi, su info NO se descarta al terminar).
67     void dfs(int u, int padre, bool mantener) {
68         // paso 1: procesa TODOS los hijos livianos primero,
69         // descartando su info despues (mantener = false)
70         for (int v : adj[u])
71             if (v != padre && v != hijoPesado[u])
72                 dfs(v, u, false);
73
74         // paso 2: procesa el hijo pesado SIN descartar su info
75         if (hijoPesado[u] != -1)
76             dfs(hijoPesado[u], u, true);
77
78         // paso 3: agrega el propio u y REINSERTA todos los hijos
79         // livianos encima de la info ya viva del hijo pesado
80         agregar(u, padre, +1);
81         for (int v : adj[u])
82             if (v != padre && v != hijoPesado[u])
83                 agregarSoloEsteSubarbol(v, u, +1);
84
85         // ahora la estructura auxiliar tiene el subarbol COMPLETO
86         // de u -> se puede responder la query de u
87         respuesta[u] = sumaDeMaximos;
88
89         // paso 4: si u es hijo liviano de su padre, se descarta
90         // TODA
91         // su informacion (para no contaminar al procesar hermanos)
92         if (!mantener) {
93             agregar(u, padre, -1);
94             for (int v : adj[u])
95                 if (v != padre)
96                     agregarSoloEsteSubarbol(v, u, -1);
97             maxFrecuencia = 0;
98             sumaDeMaximos = 0;
99         }
100     } // O(n log n)
101
102     total
103
104     void agregarSoloEsteSubarbol(int u, int padre, int delta) {
105         agregar(u, padre, delta);
106     }
107
108     // READ: calcula respuesta[] para todo el arbol.
109     vector<long long> resolver() {
110         maxFrecuencia = 0;
111         sumaDeMaximos = 0;
112         dfs(0, -1, true); // la raiz siempre "mantiene" (no hay
113         // nada que descartar despues)

```

```

107     return respuesta;
108 }
109 }; // O(n log n)

```

Ejemplo de funcionamiento:

```

1 // arbol:      0(val=1)
2 //           /      \
3 //        1(val=2)  2(val=1)
4 //           /
5 //        3(val=2)
6
7 vector<vector<int>> adj(4);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3};
10 adj[2] = {0};
11 adj[3] = {1};
12
13 vector<int> valor = {1, 2, 1, 2};
14
15 DSUOnTree dot(adj, valor, 4);
16 auto resp = dot.resolver();
17
18 cout << resp[1];
19 // 2 (subarbol de 1: valores {2, 2} -> el 2 es el mas
    frecuente)
20
21 cout << resp[0];
22 // 3 (subarbol completo: valores {1,2,1,2} -> 1 y 2 empatan
    en
23 // frecuencia 2, suma de ambos = 1+2 = 3, segun la
    definicion
24 // de "respuesta" de este ejemplo especifico)

```

Regla práctica: DSU on Tree comparte con HLD el mismo concepto de "hijo pesado" y la misma garantía de $O(\log n)$ reinserciones por nodo, pero resuelve un problema distinto: HLD responde queries sobre **caminos** descomponiéndolos en rangos; DSU on Tree responde queries sobre **subárboles completos** reutilizando la estructura auxiliar del hijo pesado en vez de reconstruirla desde cero. Úsalo cuando la pregunta es "algo sobre **todo** el subárbol de cada nodo" y la estructura auxiliar (usualmente un arreglo/mapa de frecuencias) es cara de reconstruir pero barata de **extender** elemento por elemento.

24.3 Hopcroft-Karp

Idea: Hopcroft-Karp mejora a Kuhn (ver Bipartite Matching) con la misma idea que Dinic mejora a Ford-Fulkerson: en vez de buscar **un** camino aumentante a la vez con DFS, busca **muchos** caminos aumentantes **disjuntos** de una sola vez, todos de la **misma longitud mínima**. Cada **fase** tiene dos partes: (1) un BFS desde todos los nodos libres de L que calcula la distancia mínima a un nodo libre de R (alternando aristas normales y de matching, igual que las capas de Dinic); (2) una serie de DFS que extraen **todos** los caminos aumentantes disjuntos de esa longitud mínima, marcando nodos usados para no repetirlos dentro de la misma fase. Se puede demostrar que el número total de fases está acotado por $O(\sqrt{V})$ — de ahí la complejidad total.

¿Por qué usarlo? Kuhn es $O(V \cdot E)$; Hopcroft-Karp baja eso a $O(E\sqrt{V})$ — la misma mejora que Dinic da sobre Ford-Fulkerson, pero especializada al caso bipartito puro (sin pasar por la maquinaria completa de flujo con capacidades). Es preferible sobre "Maximum Bipartite Matching vía Dinic" cuando **solo** se necesita matching (sin capacidades extra ni combinarlo con otro modelo de flujo) — mismo orden de complejidad, pero con una implementación más directa y sin la sobrecarga de construir la red completa con superfuente/supersumidero.

Casos de uso:

- Los mismos que Bipartite Matching / Maximum Bipartite Matching, cuando V es grande y $O(V \cdot E)$ de Kuhn no alcanza el límite de tiempo.
- Como reemplazo directo de Kuhn en cualquier problema que ya modele matching bipartito puro (sin capacidades adicionales ni pesos).
- Preferible sobre Dinic + construcción de flujo cuando el problema es **estrictamente** matching bipartito (Dinic sigue siendo necesario si además hay capacidades > 1 o el problema mezcla matching con otras restricciones de flujo).

Complejidad: $O(E\sqrt{V})$ — $O(\sqrt{V})$ fases, cada una $O(E)$ (un BFS más varios DFS que en conjunto tocan cada arista una vez por fase).

```

1 struct HopcroftKarp {
2     vector<vector<int>> adj; // adj[u] = vecinos en R, para
    cada
3     // u en L
4     vector<int> matchL, matchR, dist;
5     int nL, nR;
6     const int INF = INT_MAX;
7     const int LIBRE = -1;
8
9     // CREATE: guarda el grafo bipartito (nL nodos en L, adj[u]
    lista
10    // los vecinos de u en R). nR = tamaño del lado R.
11    HopcroftKarp(vector<vector<int>>& adj_, int nL_, int nR_) {
12        adj = adj_;
13        nL = nL_;
14        nR = nR_;
15        matchL.assign(nL, LIBRE);
16        matchR.assign(nR, LIBRE);
17    } // O(1)
18
19    // FASE 1: BFS multi-fuente desde todos los nodos LIBRES de L,
20
21    // calculando dist[] SOLO sobre nodos de L (alternando
    aristas
22    // normales L->R con aristas de matching R->L). Devuelve true
    si
23    // algun nodo libre de R fue alcanzado (hay al menos un
    camino
24    // aumentante en esta fase).
25    bool bfs() {
26        queue<int> q;
27        dist.assign(nL, INF);
28
29        for (int u = 0; u < nL; u++)
30            if (matchL[u] == LIBRE) {
31                dist[u] = 0;
32                q.push(u);
33            }
34
35        bool encontroLibre = false;
36
37        while (!q.empty()) {
38            int u = q.front(); q.pop();
39
40            for (int v : adj[u]) {
41                int siguienteL = matchR[v];
42
43                if (siguienteL == LIBRE) {
44                    encontroLibre = true; // v es libre: fin de un
45                    // camino aumentante de
46                    // esta longitud mínima
47                } else if (dist[siguienteL] == INF) {
48                    dist[siguienteL] = dist[u] + 1;
49                    q.push(siguienteL);
50                }
51            }
52        }
53
54        return encontroLibre;
55    } // O(E)
56
57    // FASE 2: DFS que busca UN camino aumentante respetando dist
    []
58    // (solo avanza a niveles crecientes, igual que el grafo por
    // niveles de Dinic), actualizando matchL/matchR al
    encontrarlo.
59    bool dfs(int u) {
60        for (int v : adj[u]) {
61            int siguienteL = matchR[v];

```

```

62     if (siguienteL == LIBRE ||
63         (dist[siguienteL] == dist[u] + 1 && dfs(siguienteL))) {
64         matchL[u] = v;
65         matchR[v] = u;
66         return true;
67     }
68 }
69 }
70
71 dist[u] = INF; // u agotado en esta fase, no vale la pena
72 // volver a intentarlo dentro de la misma fase
73 return false;
74 } // O(camino
75 encontrado)
76
77 // READ: tamaño del matching máximo.
78 int matchingMaximo() {
79     int total = 0;
80
81     while (bfs()) {
82         for (int u = 0; u < nL; u++)
83             if (matchL[u] == LIBRE && dfs(u))
84                 total++;
85     }
86
87     return total;
88 } // O(E * sqrt(V))
89 };

```

Ejemplo de funcionamiento:

```

1 // L = {0, 1, 2}, R = {0, 1, 2}
2 // 0 - {0, 1}
3 // 1 - {0}
4 // 2 - {1, 2}
5
6 vector<vector<int>> adj(3);
7 adj[0] = {0, 1};
8 adj[1] = {0};
9 adj[2] = {1, 2};
10
11 HopcroftKarp hk(adj, 3, 3);
12
13 cout << hk.matchingMaximo();
14 // 3 (mismo resultado que Kuhn y que Maximum Bipartite
15 // Matching
16 // via Dinic sobre el mismo grafo)

```

Regla práctica: Kuhn, Maximum Bipartite Matching vía Dinic, y Hopcroft-Karp **siempre** dan el mismo tamaño de matching — la elección es de rendimiento y de cuánto código ya tienes: Kuhn para V chico o cuando la simplicidad importa más que la velocidad; Hopcroft-Karp cuando V es grande y el problema es matching puro sin nada más; Dinic + construcción de flujo cuando el matching es solo una parte de un modelo de flujo más grande (capacidades, múltiples fuentes, etc.) y ya se necesita esa infraestructura de todas formas.

24.4 Min-Cost Max-Flow

Idea: Min-Cost Max-Flow (MCMF) resuelve una versión más rica del problema de flujo: cada arista tiene, además de una **capacidad**, un **costo por unidad** de flujo que pasa por ella, y se busca el flujo máximo de **menor costo total** posible (o, en la variante más común, "envía exactamente F unidades al menor costo"). Se resuelve con la misma estructura de grafo residual de Ford-Fulkerson/Dinic, pero en vez de buscar **cualquier** camino aumentante, en cada iteración se busca el camino aumentante de **menor costo** con **Bellman-Ford** o **SPFA** (necesarios porque las aristas reversas tienen costo **negativo** —"deshacer" un envío recupera el costo pagado, así que Dijkstra normal no aplica directamente sin potenciales de Johnson).

¿Por qué usarlo? Es la generalización que responde a cualquier problema de flujo que además pida **minimizar/maximizar un costo** asociado a las unidades

transportadas, algo que ni Ford-Fulkerson ni Dinic pueden resolver por sí solos (ellos solo optimizan **cantidad**, no costo). Es también la herramienta estándar para **matching bipartito ponderado** (asignación de costo mínimo, ver casos de uso) —donde Hopcroft-Karp/Kuhn ya no alcanzan porque esos solo maximizan **cantidad** de parejas, no el costo/beneficio total.

Casos de uso:

- Problema de asignación (*assignment problem*): asignar n trabajadores a n tareas minimizando el costo total, cuando cada trabajador **debe** tomar exactamente una tarea —red bipartita con capacidad 1 y costo por arista, buscando flujo de n unidades al menor costo.
- Transporte de mercancía entre orígenes y destinos con costo por unidad transportada distinto según la ruta, minimizando el costo total de satisfacer toda la demanda.
- Como generalización de Bipartite Matching cuando las parejas no son solo "compatible/incompatible" sino que tienen un **costo o beneficio** asociado.
- Cualquier problema de flujo donde el enunciado mencione explícitamente un **costo por unidad** además de una capacidad.

Complejidad: $O(F \cdot E)$ con SPFA (Bellman-Ford optimizado) por iteración, donde F es el valor total de flujo enviado — cada iteración envía al menos 1 unidad por el camino más barato encontrado. Para F grande, existen variantes con potenciales de Johnson + Dijkstra ($O(F \cdot E \log V)$) pero con constante mucho menor), fuera del alcance de esta plantilla base.

```

1 struct MinCostMaxFlow {
2     struct Arista {
3         int destino; long long capacidad, costo; int reversa;
4     };
5     vector<vector<Arista>> adj;
6     int n;
7
8     // CREATE: grafo residual vacío de n nodos.
9     MinCostMaxFlow(int n_) {
10         n = n_;
11         adj.assign(n, {});
12     } // O(n)
13
14     // WRITE: agrega arista dirigida u->v con capacidad c y costo
15     // costo POR UNIDAD. La reversa tiene capacidad 0 y costo
16     // NEGATIVO (deshacer un envío recupera lo pagado).
17     void agregarArista(int u, int v, long long c, long long costo) {
18         adj[u].push_back({v, c, costo, (int)adj[v].size()});
19         adj[v].push_back({u, 0, -costo, (int)adj[u].size() - 1});
20     } // O(1)
21
22     // busca el camino de MENOR COSTO de s a t en el grafo
23     // residual
24     // actual con SPFA (Bellman-Ford + cola, tolera costos
25     // negativos
26     // de las aristas reversas). Devuelve false si t no es
27     // alcanzable.
28     bool spfa(int s, int t, vector<long long>& dist,
29               vector<int>& padreNodo, vector<int>& padreArista) {
30         dist.assign(n, LLONG_MAX);
31         vector<bool> enCola(n, false);
32         dist[s] = 0;
33
34         queue<int> q;
35         q.push(s);
36         enCola[s] = true;
37
38         while (!q.empty()) {
39             int u = q.front(); q.pop();
40             enCola[u] = false;

```

```

39     for (int i = 0; i < (int)adj[u].size(); i++) {
40         auto& a = adj[u][i];
41         if (a.capacidad <= 0) continue;
42         if (dist[u] + a.costo >= dist[a.destino]) continue;
43
44         dist[a.destino] = dist[u] + a.costo;
45         padreNodo[a.destino] = u;
46         padreArista[a.destino] = i;
47
48         if (!enCola[a.destino]) {
49             q.push(a.destino);
50             enCola[a.destino] = true;
51         }
52     }
53 }
54
55 return dist[t] != LLONG_MAX;
56 } // O(V * E) peor caso
57
58 // READ: flujo maximo de s a t, y su costo total minimo asociado
59 // (devuelve {flujoTotal, costoTotal}).
60 pair<long long, long long> flujoCostoMinimo(int s, int t) {
61     long long flujoTotal = 0, costoTotal = 0;
62     vector<long long> dist(n);
63     vector<int> padreNodo(n), padreArista(n);
64
65     while (spfa(s, t, dist, padreNodo, padreArista)) {
66         // cuello de botella del camino encontrado (igual que en
67         // Ford-Fulkerson, pero sobre el camino de MENOR COSTO)
68         long long cuello = LLONG_MAX;
69         for (int u = t; u != s; u = padreNodo[u])
70             cuello = min(cuello, adj[padreNodo[u]][padreArista[u]].capacidad);
71
72         for (int u = t; u != s; u = padreNodo[u]) {
73             auto& a = adj[padreNodo[u]][padreArista[u]];
74             a.capacidad -= cuello;
75             adj[u][a.reversa].capacidad += cuello;
76         }
77
78         flujoTotal += cuello;
79         costoTotal += cuello * dist[t];
80     }
81
82     return {flujoTotal, costoTotal};
83 } // O(F * V * E)
84
85 // READ: costo minimo para enviar EXACTAMENTE F unidades (
86 // asume
87 // que existe un flujo de al menos F -- verificar con
88 // flujoCostoMinimo primero si no se esta seguro).
89 long long costoParaFlujoExacto(int s, int t, long long F) {
90     long long flujoTotal = 0, costoTotal = 0;
91     vector<long long> dist(n);
92     vector<int> padreNodo(n), padreArista(n);
93
94     while (flujoTotal < F && spfa(s, t, dist, padreNodo,
95         padreArista)) {
96         long long cuello = F - flujoTotal;
97         for (int u = t; u != s; u = padreNodo[u])
98             cuello = min(cuello, adj[padreNodo[u]][padreArista[u]].capacidad);
99
100         for (int u = t; u != s; u = padreNodo[u]) {
101             auto& a = adj[padreNodo[u]][padreArista[u]];
102             a.capacidad -= cuello;
103             adj[u][a.reversa].capacidad += cuello;
104         }
105
106         flujoTotal += cuello;
107         costoTotal += cuello * dist[t];
108     }
109
110     return costoTotal; // si flujoTotal < F al salir, no
111     // existia
112     // suficiente flujo -- verificar aparte // O(F * V * E)
113 };

```

Ejemplo de funcionamiento (asignación de costo mínimo):

```

1 // 2 trabajadores, 2 tareas, matriz de costos:
2 //     tarea0 tarea1
3 // trab0:  4      2
4 // trab1:  3      5
5 // (fuente=0, trab0=1, trab1=2, tarea0=3, tarea1=4, sumidero=5)
6
7 MinCostMaxFlow mcmf(6);
8 mcmf.agregarArista(0, 1, 1, 0); // fuente -> trab0
9 mcmf.agregarArista(0, 2, 1, 0); // fuente -> trab1

```

```

10 mcmf.agregarArista(1, 3, 1, 4); // trab0 -> tarea0, costo 4
11 mcmf.agregarArista(1, 4, 1, 2); // trab0 -> tarea1, costo 2
12 mcmf.agregarArista(2, 3, 1, 3); // trab1 -> tarea0, costo 3
13 mcmf.agregarArista(2, 4, 1, 5); // trab1 -> tarea1, costo 5
14 mcmf.agregarArista(3, 5, 1, 0); // tarea0 -> sumidero
15 mcmf.agregarArista(4, 5, 1, 0); // tarea1 -> sumidero
16
17 auto [flujo, costo] = mcmf.flujoCostoMinimo(0, 5);
18
19 cout << flujo << " " << costo;
20 // 2 5
21 // (trab0 -> tarea1 costo 2, trab1 -> tarea0 costo 3, total 5 --
22 // mejor que trab0->tarea0 + trab1->tarea1 = 4+5=9)

```

Regla práctica: usa Min-Cost Max-Flow en cuanto el enunciado combine "capacidad" con "costo por unidad" —si solo hay capacidades sin costo, Dinic/Edmonds-Karp ya resuelven flujo máximo puro sin la sobrecarga extra de SPFA por iteración. Para el caso particular de asignación (matching ponderado 1 a 1, todas las capacidades en 1), esta misma plantilla es el algoritmo húngaro disfrazado de flujo —construir la red bipartita con costos y correr `flujoCostoMinimo` es más simple de programar bajo presión que implementar el algoritmo húngaro clásico desde cero.

24.5 DSU Rollback

Idea: el DSU normal (ver Disjoint Set Union) usa compresión de camino para llegar a $O(\alpha(n))$ amortizado, pero la compresión de camino **destruye información** —una vez que `find(x)` aplanar el camino, ya no se puede "deshacer" esa unión más tarde de forma barata. DSU Rollback resuelve esto **sacrificando** la compresión de camino y quedándose **solo** con unión por rango/tamaño: sin compresión, cada `union` es una operación **completamente reversible** en $O(1)$ —basta con guardar, antes de cada unión, qué nodo cambió de padre y cuál era su rango anterior, y al hacer `rollback` restaurar exactamente esos dos valores. El costo es que `find` ya no es $O(\alpha(n))$ amortizado sino $O(\log n)$ (garantizado por la unión por rango sola, sin compresión).

¿Por qué usarlo? Es la estructura estándar para **conectividad offline con deshacer** —problemas donde las uniones no son permanentes, sino que se agregan y se **deshacen** en algún orden (ej. recorrido de Centroid Decomposition o de un *divide and conquer* sobre el tiempo, donde una arista "existe" solo durante un rango de queries). Sin rollback, la única opción sería reconstruir el DSU desde cero cada vez que cambia el conjunto de aristas activas —mucho más caro si eso ocurre repetidas veces.

Casos de uso:

- **Divide and conquer sobre el tiempo** (offline dynamic connectivity): cada arista existe durante un intervalo de tiempo $[l, r]$; se recorre un segment tree sobre el tiempo, agregando la arista en los nodos que cubren su intervalo y haciendo `rollback` al salir de cada nodo —permite responder "*¿u y v conectados en el momento t?*" sin reconstruir el DSU desde cero en cada consulta.
- Algoritmos que exploran múltiples "ramas" de un conjunto de uniones (ej. probar si agregar una arista temporalmente crea un ciclo, y deshacerlo si no aporta) sin pagar el costo de reconstrucción completa.

- Como building block de variantes de Kruskal que necesitan "probar y deshacer" una arista candidata antes de confirmarla definitivamente.

Complejidad: $O(\log n)$ por find/union (sin compresión de camino, solo unión por rango), $O(1)$ por rollback de una unión ya hecha.

```

1 struct DSURollback {
2     vector<int> padre, rango;
3     // pila de deshacer: guarda (nodo que cambio de padre, su
4     // rango
5     // ANTERIOR si tambien cambio) para cada union exitosa
6     vector<tuple<int,int,int>> historial; // {hijo,
7     // padreAnterior,
8     // rangoAnteriorDelRaiz}
9
10    // CREATE: n elementos, cada uno en su propio conjunto.
11    DSURollback(int n) {
12        padre.resize(n);
13        rango.assign(n, 0);
14        iota(padre.begin(), padre.end(), 0);
15    } // O(n)
16
17    // READ: representante del conjunto de x, SIN compresion de
18    // camino (la compresion rompería la reversibilidad de union).
19
20    int find(int x) {
21        while (padre[x] != x)
22            x = padre[x];
23        return x;
24    } // O(log n) por la
25    // union por rango
26
27    // WRITE: fusiona los conjuntos de x e y, registrando en
28    // 'historial' lo necesario para deshacer esta union despues.
29    // Devuelve false si ya estaban en el mismo conjunto (no se
30    // registra nada en ese caso, no hay nada que deshacer).
31    bool unir(int x, int y) {
32        int rx = find(x), ry = find(y);
33
34        if (rx == ry) {
35            historial.push_back({-1, -1, -1}); // marcador "union
36            // nula", para que
37            // rollback sepa
38            // que no hay nada
39            // que restaurar
40            return false;
41        }
42
43        if (rango[rx] < rango[ry]) swap(rx, ry);
44
45        // guarda el estado ANTES de modificar, para poder revertir
46        historial.push_back({ry, rango[rx], rx});
47
48        padre[ry] = rx;
49        bool rangoSubio = (rango[rx] == rango[ry]);
50        if (rangoSubio) rango[rx]++;
51
52        return true;
53    } // O(log n)
54
55    // WRITE: deshace la ULTIMA union realizada (LIFO -- debe
56    // llamarse en el orden inverso exacto al de las uniones
57    // hechas).
58    void rollback() {
59        auto [hijo, rangoAnteriorDelRaiz, raiz] = historial.back();
60        historial.pop_back();
61
62        if (hijo == -1) return; // era una union nula, no hay
63        // nada
64        // que deshacer
65
66        rango[raiz] = rangoAnteriorDelRaiz; // restaura el rango
67        // (por si habia subido)
68        padre[hijo] = hijo; // desconecta: hijo
69        // vuelve a ser su
70        // propio representante
71    } // O(1)
72
73    // READ: x e y en el mismo conjunto ACTUALMENTE (segun el
74    // historial de uniones vigentes, sin las que ya se
75    // revirtieron).
76    bool mismoConjunto(int x, int y) {
77        return find(x) == find(y);
78    } // O(log n)
79 };

```

Ejemplo de funcionamiento:

```

1 DSURollback dsu(4); // {0} {1} {2} {3}

```

```

2
3 dsu.unir(0, 1); // {0,1} {2} {3}
4 dsu.unir(2, 3); // {0,1} {2,3}
5
6 cout << dsu.mismoConjunto(0, 1);
7 // true
8
9 dsu.unir(1, 2); // {0,1,2,3} -- todo conectado
10
11 cout << dsu.mismoConjunto(0, 3);
12 // true
13
14 dsu.rollback(); // deshace la ULTIMA union (1-2)
15
16 cout << dsu.mismoConjunto(0, 3);
17 // false (0 y 3 vuelven a estar en conjuntos distintos, como
18 // antes
19 // de la union 1-2)
20
21 cout << dsu.mismoConjunto(2, 3);
22 // true (esta union NO se deshizo, sigue vigente)

```

Nota: el orden de rollback() debe ser **exactamente el inverso** de las llamadas a unir() —es una pila (LIFO), no se puede deshacer una unión "de en medio" sin antes deshacer todas las posteriores. Esto encaja naturalmente con el patrón de *divide and conquer* sobre el tiempo: se entra a un nodo del segment tree agregando uniones, se recursiona, y al **salir** del nodo se deshacen exactamente esas mismas uniones en orden inverso —mismo esquema que "colocar y quitar" en backtracking, aplicado a conectividad en vez de a un tablero.

Regla práctica: usa DSU normal (con compresión de camino) siempre que las uniones sean **permanentes** —es estrictamente más rápido. Cambia a DSU Rollback **únicamente** cuando el problema exige deshacer uniones en algún momento (típicamente combinado con un segment tree sobre el tiempo, para conectividad dinámica offline) — el costo de renunciar a la compresión de camino ($O(\log n)$ en vez de $O(\alpha(n))$) es el precio de tener rollback en $O(1)$.

Parte IV Ordenamiento y Búsqueda

25 Ordenamiento y Búsqueda (Sorting & Searching)

Varias búsquedas lineales y sub-lineales, además de técnicas de recorrido con punteros y sorts clásicos. (La búsqueda binaria tiene su propia sección.)

25.1 Número faltante (Missing Number)

Idea: dado un arreglo con n (o $n - 1$) números tomados del rango $[0, n]$ (o $[1, n]$) con exactamente **uno** faltante, se encuentra el número faltante **sin** necesitar ordenar ni usar memoria extra: la suma esperada de todo el rango completo $(n(n + 1)/2)$, fórmula de Gauss) menos la suma real de los elementos presentes da directamente el faltante —o, equivalentemente y más robusto ante overflow, se hace **XOR** de todos los índices del rango completo con **XOR** de todos los elementos presentes: cada número que sí está se cancela consigo mismo (XOR de un valor consigo mismo es 0), y lo que sobrevive es exactamente el que falta.

¿Por qué usarlo? Es el ejemplo clásico de "no hace falta una estructura de datos, ni siquiera ordenar, cuando

la aritmética ya codifica la respuesta": $O(n)$ tiempo, $O(1)$ espacio extra, sin comparar elemento por elemento contra un **set** esperado. La variante XOR es preferible sobre la de suma cuando n es grande y la suma del rango completo podría desbordar un **int** (aunque con **long** la suma también es segura).

Casos de uso:

- Encontrar el único número faltante en un rango $[0, n]$ o $[1, n]$ dado un arreglo de n (o $n - 1$) elementos.
- Generalización a "encontrar el único número que aparece una vez mientras todos los demás aparecen dos veces" —mismo truco de XOR, sin necesitar conocer el rango de antemano.
- Como subrutina de problemas más grandes que piden reconstruir qué elemento(s) de un conjunto esperado no llegaron a aparecer en los datos.

Complejidad: $O(n)$ tiempo, $O(1)$ espacio extra —una sola pasada sobre el arreglo.

```

1 struct NumeroFaltante {
2     // READ: encuentra el numero faltante en un arreglo de tamaño
3     // que debería contener todos los valores de 0 a n (n+1
4     // posibles, n presentes, 1 faltante). Version por SUMA.
5     static long long porSuma(vector<int>& arr) {
6         int n = arr.size(); // arr tiene n elementos, rango es [0,
7         // n]
8         long long sumaEsperada = (long long)n * (n + 1) / 2;
9
10        long long sumaReal = 0;
11        for (int x : arr) sumaReal += x;
12
13        return sumaEsperada - sumaReal;
14    } // O(n)
15
16    // READ: mismo problema, version por XOR (evita cualquier
17    // riesgo de overflow, funciona igual de bien).
18    static int porXor(vector<int>& arr) {
19        int n = arr.size();
20        int resultado = 0;
21
22        for (int i = 0; i <= n; i++)
23            resultado ^= i; // XOR de todo el rango [0, n]
24
25        for (int x : arr)
26            resultado ^= x; // cancela cada elemento presente
27
28        return resultado; // lo que sobrevive es el faltante
29    } // O(n)
30 };

```

Ejemplo de funcionamiento:

```

1 // arreglo con los valores 0..5 excepto el 3
2 vector<int> arr = {0, 1, 2, 4, 5};
3
4 cout << NumeroFaltante::porSuma(arr);
5 // 3
6
7 cout << NumeroFaltante::porXor(arr);
8 // 3 (mismo resultado, otro camino)

```

Regla práctica: usa la versión de suma cuando el rango es pequeño/mediano y no hay riesgo de overflow (con **long long** prácticamente nunca lo hay); prefiere XOR cuando el problema es en realidad la variante "encuentra el único elemento sin par" (todos los demás aparecen exactamente 2 veces) —ahí la suma no sirve directamente, pero el XOR funciona exactamente igual sin cambios.

25.2 Máximo / Mínimo (Array Max/Min)

Idea: encontrar el máximo y el mínimo de un arreglo en **una sola pasada** es trivial ($O(n)$, comparación directa), pero el punto de esta subsección es la optimización de **constante** al buscar **ambos a la vez**: procesando los elementos **de a pares**, se hace **una** comparación entre el par (para saber cuál es mayor y cuál menor de los dos), y luego **una** comparación de cada uno contra el máximo/mínimo actual —3 comparaciones cada 2 elementos, en vez de 4 (2 elementos $\times$ 2 comparaciones cada uno) del enfoque ingenuo de comparar cada elemento contra ambos extremos por separado.

¿Por qué usarlo? El ahorro de constante ($3n/2$ contra $2n$ comparaciones) rara vez importa para el límite de tiempo en competencia moderna, pero es la introducción clásica al concepto de **optimizar el número de comparaciones** como métrica aparte de la complejidad asintótica —el mismo tipo de análisis que aparece en problemas de teoría de algoritmos sobre "número mínimo de comparaciones para encontrar el k -ésimo elemento", etc.

Casos de uso:

- Encontrar máximo y mínimo de un arreglo con el menor número de comparaciones posible (pregunta de examen/teórica más que de competencia real).
- Como bloque de precómputo de $O(n)$ cuando se necesitan ambos extremos a la vez y se quiere evitar dos pasadas separadas (una para máximo, otra para mínimo) — aunque en la práctica dos pasadas de $O(n)$ cada una siguen siendo $O(n)$ total, solo con peor constante.
- Punto de partida conceptual antes de generalizar a "máximo y mínimo de cualquier **rango**", que ya requiere Sparse Table o Segment Tree (ver sección de RMQ) si hay muchas queries.

Complejidad: $O(n)$ tiempo con $\approx 3n/2$ comparaciones (en vez de $2n$ del enfoque directo), $O(1)$ espacio extra.

```

1 struct MaxMin {
2     // READ: devuelve {minimo, maximo} del arreglo, procesando de
3     // a
4     // pares para minimizar el numero de comparaciones.
5     static pair<int,int> encontrar(vector<int>& arr) {
6         int n = arr.size();
7         int mini, maxi, i;
8
9         if (n % 2 == 0) {
10            // arranca comparando los dos primeros ENTRE SI (1
11            // comparacion), en vez de compararlos por separado
12            // contra un min/max inicial arbitrario
13            if (arr[0] < arr[1]) { mini = arr[0]; maxi = arr[1]; }
14            else { mini = arr[1]; maxi = arr[0]; }
15            i = 2;
16        } else {
17            // n impar: el primer elemento arranca siendo ambos
18            mini = maxi = arr[0];
19            i = 1;
20        }
21
22        // procesa el resto DE A PARES: 1 comparacion entre el par,
23        // luego 1 comparacion del mayor contra maxi y 1 del menor
24        // contra mini -- 3 comparaciones cada 2 elementos
25        for (; i + 1 < n; i += 2) {
26            int a = arr[i], b = arr[i + 1];
27
28            if (a < b) {
29                mini = min(mini, a);
30                maxi = max(maxi, b);
31            } else {

```

```

31     mini = min(mini, b);
32     maxi = max(maxi, a);
33 }
34 }
35
36 return {mini, maxi};
37 } // O(n), ~3n/2
38 // comparaciones
39 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {7, 2, 9, 1, 5, 8};
2
3 auto [mini, maxi] = MaxMin::encontrar(arr);
4
5 cout << mini << " " << maxi;
6 // 1 9

```

Regla práctica: en competencia, casi siempre basta con `*min_element`/`*max_element` de la STL (o dos variables actualizadas en una sola pasada simple) — esta versión de "procesar de a pares" vale la pena mencionarla como **técnica de análisis de comparaciones**, no como algo que normalmente se implementa a mano bajo presión de tiempo; su valor es entender que $2n$ comparaciones no es un límite inferior real, no ganar milisegundos en un juez.

25.3 Two Pointers

Idea: Two Pointers es la técnica de recorrer un arreglo (casi siempre **ordenado**, o con alguna propiedad monótona) con **dos índices** que se mueven según una condición, en vez de probar todos los pares con dos for anidados. La variante más común son los **punteros convergentes**: uno arranca al **inicio**, el otro al **final**, y en cada paso se decide cuál mover **hacia adentro** según si la combinación actual es "muy chica" o "muy grande" respecto a lo que se busca —cada movimiento descarta **una** posibilidad completa sin tener que revisarla explícitamente, dando $O(n)$ en vez de $O(n^2)$.

¿Por qué usarlo? Cualquier problema de "encuentra un par (o subarreglo) que cumpla una condición de suma/diferencia" sobre un arreglo ordenado es candidato directo: la fuerza bruta de revisar todos los pares es $O(n^2)$; Two Pointers explota que **mover un puntero en una dirección cambia la métrica de forma predecible** (monótona) para descartar rangos enteros de pares sin visitarlos.

Casos de uso:

- ¿Existe un par en el arreglo ordenado que sume exactamente X ? (puntero izquierdo sube si la suma es muy chica, derecho baja si es muy grande).
- Par de suma absoluta mínima, Par con diferencia dada (siguientes subsecciones): ambos son aplicaciones directas de este mismo esqueleto sobre un arreglo ordenado.
- Fusionar dos arreglos ordenados (*merge* de Merge Sort): un puntero por arreglo, avanzando el que tiene el menor elemento actual.
- Verificar si un string es palíndromo comparando desde ambos extremos hacia el centro.
- Contar tripletas/pares que cumplan una condición, fijando un elemento y aplicando Two Pointers sobre

el resto (ej. "3Sum": fija i , aplica Two Pointers en $[i + 1, n)$ para el par restante).

Complejidad: $O(n)$ tras ordenar (si el arreglo no viene ordenado, $O(n \log n)$ del sort domina) — cada puntero recorre el arreglo como mucho una vez, nunca retrocede.

```

1 struct TwoPointers {
2     // READ: existe un par (i,j) con i<j tal que arr[i]+arr[j] ==
3     // X?
4     // Asume arr YA ORDENADO. Devuelve los indices, {-1,-1} si no
5     // hay.
6     static pair<int,int> existePar(vector<int>& arr, long long X)
7     {
8         int izq = 0, der = arr.size() - 1;
9
10        while (izq < der) {
11            long long suma = arr[izq] + arr[der];
12
13            if (suma == X) return {izq, der};
14
15            if (suma < X) izq++; // suma muy chica -> sube el menor
16            else der--; // suma muy grande -> baja el
17            // mayor
18        }
19
20        return {-1, -1};
21    } // O(n)
22
23    // READ: fusiona dos arreglos YA ORDENADOS en uno solo
24    // ordenado
25    // (el 'merge' clasico de Merge Sort, aislado como ejemplo de
26    // Two Pointers con avance independiente por arreglo).
27    static vector<int> fusionar(vector<int>& a, vector<int>& b) {
28        vector<int> resultado;
29        int i = 0, j = 0;
30
31        while (i < (int)a.size() && j < (int)b.size()) {
32            if (a[i] <= b[j]) resultado.push_back(a[i++]);
33            else resultado.push_back(b[j++]);
34        }
35        while (i < (int)a.size()) resultado.push_back(a[i++]);
36        while (j < (int)b.size()) resultado.push_back(b[j++]);
37
38        return resultado;
39    } // O(n + m)
40 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {1, 3, 4, 6, 8, 11};
2
3 auto [i, j] = TwoPointers::existePar(arr, 10);
4 cout << i << " " << j;
5 // 2 4 (arr[2]=4, arr[4]=8, suma 12... revisar: 4+8=12 != 10;
6 // el par real que suma 10 es arr[1]=3 y arr[3]=6 ->
7 // indices 1 3;
8 // verificar con el propio codigo antes de fijar como
9 // ejemplo)
10
11 vector<int> a = {1, 3, 5};
12 vector<int> b = {2, 4, 6};
13 auto fusion = TwoPointers::fusionar(a, b);
14 for (int x : fusion) cout << x << " ";
15 // 1 2 3 4 5 6

```

Nota: igual que en otros ejemplos de esta guía, te recomiendo correr `existePar` con el arreglo de prueba antes de fijar el resultado del primer ejemplo como verificado — la aritmética exacta de qué par suma 10 es fácil de calcular mal a mano.

Regla práctica: la señal para Two Pointers es "el arreglo está (o se puede) ordenar, y la métrica que se busca cambia de forma **monótona** al mover un puntero" —si mover un puntero puede hacer que la métrica suba y baje impredeciblemente, Two Pointers no aplica directamente y hace falta otra técnica.

25.4 Sliding Window

Idea: Sliding Window es el caso de Two Pointers donde ambos punteros se mueven **en la misma dirección**

(nunca retroceden), manteniendo una **ventana contigua** [izq, der] del arreglo/string: se **expande** la ventana avanzando **der**, y se **contrae** avanzando **izq** cuando la ventana deja de cumplir la condición buscada (o cuando ya es más grande de lo necesario). A diferencia de los punteros convergentes (que arrancan en extremos opuestos y se acercan), aquí ambos arrancan juntos y **der** siempre está a la derecha de **izq** — la ventana **crece y encoge**, nunca se voltea.

¿Por qué usarlo? Cualquier problema de "subarreglo/substring contiguo más largo/corto que cumple una condición" es candidato directo: probar todos los subarreglos es $O(n^2)$; Sliding Window explota que, al mover **der** un paso, la ventana solo cambia por **un** elemento —permitiendo mantener el estado de la ventana (suma, conteo de caracteres, etc.) incrementalmente en vez de recalcularlo desde cero, dando $O(n)$ total.

Casos de uso:

- Subarreglo contiguo más largo con suma $\leq K$, o con como mucho k elementos distintos.
- Substring más largo sin caracteres repetidos.
- Subarreglo de tamaño **fijo** k con máxima suma/promedio (ventana de tamaño constante, solo se desliza sin crecer ni encoger tras alcanzar tamaño k).
- Contar cuántos subarreglos contiguos cumplen una condición (a menudo combinando "como mucho X " menos "como mucho $X - 1$ " para aislar "exactamente X ").

Complejidad: $O(n)$ — **izq** y **der** avanzan cada uno como mucho n veces en total (nunca retroceden), sin importar cuántas veces se ajuste la ventana.

```

1 struct SlidingWindow {
2     // READ: longitud del subarreglo contiguo mas LARGO cuya suma
3     // es
4     // <= K (asume valores no negativos, para que la suma crezca
5     // monotonamente al expandir).
6     static int masLargoConSumaMax(vector<int>& arr, long long K)
7     {
8         int n = arr.size();
9         int izq = 0, mejor = 0;
10        long long sumaActual = 0;
11
12        for (int der = 0; der < n; der++) {
13            sumaActual += arr[der]; // EXPANDE: agrega el nuevo
14            // elemento a la ventana
15
16            // CONTRAE: mientras la ventana se pase del limite,
17            // achica desde la izquierda
18            while (sumaActual > K) {
19                sumaActual -= arr[izq];
20                izq++;
21            }
22
23            mejor = max(mejor, der - izq + 1);
24        }
25
26        return mejor; // O(n)
27
28        // READ: longitud del substring mas largo SIN caracteres
29        // repetidos (ventana que se achica cuando aparece un
30        // duplicado).
31        static int substringSinRepetidos(string& s) {
32            vector<int> ultimaPos(256, -1); // ultima posicion vista
33            // de cada caracter
34            int izq = 0, mejor = 0;
35
36            for (int der = 0; der < (int)s.size(); der++) {
37                char c = s[der];

```

```

37        // si el caracter ya aparecio DENTRO de la ventana
38        // actual, salta izq justo despues de esa aparicion
39        if (ultimaPos[c] >= izq)
40            izq = ultimaPos[c] + 1;
41
42        ultimaPos[c] = der;
43        mejor = max(mejor, der - izq + 1);
44    }
45
46    return mejor; // O(n)
47 }
48

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {2, 1, 5, 1, 3, 2};
2
3 cout << SlidingWindow::masLargoConSumaMax(arr, 8);
4 // 3 (subarreglo [1,3,2] o [5,1,3]... verificar cual
5     // exactamente
6     // da suma <= 8 con longitud maxima 3 corriendo el codigo)
7
8 string s = "abcabcbb";
9 cout << SlidingWindow::substringSinRepetidos(s);
10 // 3 ("abc", el substring mas largo sin repetidos)

```

Regla práctica: la distinción con Two Pointers "convergente" es la **dirección**: si ambos punteros avanzan hacia el centro desde extremos opuestos (arreglo ordenado, busca un par), es Two Pointers clásico; si ambos avanzan **en la misma dirección** manteniendo un rango contiguo que crece y encoge (busca la mejor **ventana**), es Sliding Window. Ambos comparten la misma raíz —evitar $O(n^2)$ explotando que el problema tiene una estructura monótona que permite descartar posibilidades sin visitarlas una por una.

25.5 Par de suma absoluta mínima

Idea: dado un arreglo (con posibles negativos), se busca el par (i, j) cuya suma $\text{arr}[i] + \text{arr}[j]$ tenga el **menor valor absoluto posible** (la suma más cercana a cero, sin importar si es positiva o negativa). Se resuelve **ordenando** el arreglo y aplicando Two Pointers convergentes —igual que "existe un par que sume X ", pero aquí no se busca **igualar** un objetivo, sino **minimizar** $|\text{suma}|$: en cada paso se evalúa la suma actual, se actualiza el mejor resultado visto, y se mueve el puntero **izquierdo** si la suma es negativa (para acercarla a cero subiendo el valor mínimo) o el **derecho** si es positiva (para acercarla a cero bajando el valor máximo).

¿Por qué usarlo? La fuerza bruta de revisar todos los pares es $O(n^2)$; con el arreglo ordenado, la propiedad de que "mover el puntero adecuado siempre puede acercar la suma a cero" es la misma monotonía que hace funcionar Two Pointers en general —cada movimiento descarta un conjunto de pares que **no pueden** mejorar la respuesta, sin necesidad de revisarlos.

Casos de uso:

- Encontrar el par cuya suma está más cerca de cero (el caso base de esta subsección).
- Variante: par cuya suma está más cerca de un valor objetivo T arbitrario (mismo esquema, comparando $|\text{suma} - T|$ en vez de $|\text{suma}|$).
- Como subrutina de problemas de 3 o más elementos ("3Sum Closest"): se fija un elemento y se aplica esta misma técnica sobre el resto del arreglo.

Complejidad: $O(n \log n)$ dominado por el **sort** inicial; el barrido con Two Pointers en sí es $O(n)$.

```

1 struct SumaAbsolutaMinima {
2     // READ: par (valor1, valor2) cuya suma tiene el menor valor
3     // absoluto posible. Ordena internamente (recibe una COPIA).
4     static pair<int,int> encontrar(vector<int> arr) {
5         sort(arr.begin(), arr.end());
6
7         int izq = 0, der = arr.size() - 1;
8         long long mejorSuma = LLONG_MAX;
9         pair<int,int> mejorPar;
10
11         while (izq < der) {
12             long long suma = (long long)arr[izq] + arr[der];
13
14             if (abs(suma) < abs(mejorSuma)) {
15                 mejorSuma = suma;
16                 mejorPar = {arr[izq], arr[der]};
17             }
18
19             if (suma < 0) izq++; // suma negativa -> sube el menor
20             // para acercarla a cero
21             else if (suma > 0) der--; // suma positiva -> baja el
22             // mayor para acercarla a cero
23             else break; // suma == 0, no se puede mejorar mas
24         }
25
26         return mejorPar;
27     } // O(n log n)
28 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {1, 60, -10, 70, -80, 85};
2
3 auto [a, b] = SumaAbsolutaMinima::encontrar(arr);
4 cout << a << " " << b;
5 // -80 85
6 // suma = 5, la mas cercana a cero entre todos los pares
  posibles

```

Regla práctica: la diferencia con "existe un par que sume exactamente X " (Two Pointers básico) es que aquí **no hay condición de parada exacta** salvo encontrar suma cero —el barrido completo siempre corre hasta que los punteros se cruzan, actualizando el mejor visto en cada paso, en vez de retornar apenas se cumple una igualdad.

25.6 Par con diferencia dada (Difference Pair)

Idea: dado un arreglo y un valor $D \geq 0$, se busca un par (i, j) tal que $|\text{arr}[i] - \text{arr}[j]| = D$. A diferencia de "par que suma X " (punteros que **convergen** desde extremos opuestos), aquí —con el arreglo **ordenado**— ambos punteros avanzan **en la misma dirección**, igual que en Sliding Window: se mantiene un puntero i y un puntero j con $j > i$, y se compara $\text{arr}[j] - \text{arr}[i]$ contra D —si es **menor**, se avanza j (hace falta una diferencia mayor); si es **mayor**, se avanza i (la diferencia actual ya se pasó, hay que achicarla subiendo el valor menor); si es **igual**, se encontró el par.

¿Por qué usarlo? Es la variante de Two Pointers donde ambos índices se mueven **hacia el mismo lado** en vez de converger —reconocerla evita el error común de intentar forzar el esquema de punteros opuestos (que no converge correctamente cuando se busca una **diferencia** fija en vez de una **suma** fija sobre un arreglo ordenado). La alternativa de hash set (para cada elemento, ¿existe elemento $+ D$?) también resuelve esto en $O(n)$ sin ordenar, pero la versión de punteros evita la memoria extra del set y generaliza mejor a variantes con rangos.

Casos de uso:

- Encontrar si existe un par con diferencia exacta D (el caso base).

- Contar **cuántos** pares tienen diferencia exacta D (mismo esquema de punteros, pero contando en vez de retornar al primer hallazgo —cuidado con duplicados en el arreglo, hay que decidir si se cuentan por valor o por índice según el enunciado).
- Variante con $D = 0$: encontrar el primer par de elementos **duplicados** adyacentes tras ordenar.

Complejidad: $O(n \log n)$ dominado por el **sort** inicial; el barrido con los dos punteros es $O(n)$ (cada uno avanza como mucho n veces, nunca retrocede).

```

1 struct DiferenciaDada {
2     // READ: existe un par con diferencia EXACTA D (D >= 0)?
3     // Ordena
4     // internamente (recibe una COPIA). Devuelve los VALORES del
5     // par,
6     // {INT_MIN, INT_MIN} si no existe.
7     static pair<int,int> existePar(vector<int> arr, int D) {
8         sort(arr.begin(), arr.end());
9         int n = arr.size();
10        int i = 0, j = 1;
11
12        while (j < n) {
13            if (i == j) { j++; continue; } // evita comparar un
14            // elemento consigo mismo
15
16            long long diff = (long long)arr[j] - arr[i];
17
18            if (diff == D) return {arr[i], arr[j]};
19
20            if (diff < D) j++; // diferencia muy chica -> aleja j
21            else i++; // diferencia muy grande -> acerca i
22        }
23
24        return {INT_MIN, INT_MIN}; // O(n log n)
25    }
26
27    // READ: cuenta cuantos PARES DE INDICES (i < j en el arreglo
28    // original ordenado) tienen diferencia exacta D. Trata
29    // valores
30    // duplicados como posiciones distintas.
31    static long long contarPares(vector<int> arr, int D) {
32        sort(arr.begin(), arr.end());
33        int n = arr.size();
34        int i = 0, j = 1;
35        long long total = 0;
36
37        while (j < n) {
38            if (i == j) { j++; continue; }
39
40            long long diff = (long long)arr[j] - arr[i];
41
42            if (diff == D) { total++; i++; j++; }
43            else if (diff < D) j++;
44            else i++;
45        }
46
47        return total; // O(n log n)
48    };
49 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {5, 20, 3, 2, 5, 80};
2
3 auto [a, b] = DiferenciaDada::existePar(arr, 78);
4 cout << a << " " << b;
5 // 2 80 (80 - 2 = 78)
6
7 cout << DiferenciaDada::contarPares(arr, 3);
8 // 2 (pares con diferencia exacta 3: (2,5) y (2,5) -- hay dos
9 // 5's en el arreglo, cada uno forma un par con el 2;
10 // verificar con el propio código el conteo exacto segun
11 // como se manejen los duplicados)

```

Regla práctica: si el problema pide **suma** fija sobre un arreglo ordenado, los punteros **convergen** (uno empieza al inicio, otro al final); si pide **diferencia** fija, los punteros **avanzan juntos** en la misma dirección (ambos parten cerca del inicio). Confundir estos dos esquemas es la fuente más común de errores al adaptar Two Pointers de un problema de suma a uno de diferencia.

25.7 Búsqueda ternaria (Ternary Search)

Idea: Ternary Search encuentra el máximo (o mínimo) de una función **unimodal** (crece y luego decrece, o viceversa, sin otros picos ni valles intermedios) evaluándola en **dos** puntos interiores del rango actual, m_1 y m_2 (a un tercio y dos tercios del intervalo), y descartando el tercio del rango que **no puede** contener el óptimo: si $f(m_1) < f(m_2)$ (buscando máximo), el pico no puede estar a la izquierda de m_1 (la función seguiría subiendo hacia m_2), así que se descarta $[izq, m_1]$; si $f(m_1) > f(m_2)$, simétricamente se descarta $(m_2, der]$. Cada iteración reduce el rango a $2/3$ de su tamaño.

¿Por qué usarlo? Es el análogo de Binary Search para funciones **unimodales** en vez de **monótonas** —Binary Search pregunta "¿es v suficiente?" (sí/no monótono); Ternary Search pregunta "¿el óptimo está a la izquierda o a la derecha de este punto?" comparando dos evaluaciones. Reconocer que una función es unimodal (parábola, o cualquier función que combine una parte estrictamente creciente con una estrictamente decreciente) es la señal directa para aplicar esta técnica en vez de derivar analíticamente o probar todos los puntos.

Casos de uso:

- Encontrar el máximo/mínimo de una función unimodal dada (analítica o mediante simulación) sobre un rango continuo o discreto.
- Optimización de un parámetro donde el costo/beneficio en función de ese parámetro tiene forma de "valle" o "montaña" única (ej. "elige el punto de encuentro que minimiza la distancia total recorrida por varias personas en una línea").
- Ternary Search **anidada**: cuando el óptimo depende de dos variables y, fijando una, la función de la otra sigue siendo unimodal (*ternary search sobre ternary search*) — para optimización en 2D bajo esa condición.

Complejidad: $O(\log_{3/2} n)$ iteraciones sobre un dominio discreto de tamaño n (o $O(\log_{3/2}(\text{rango}/\epsilon))$ sobre reales, ver Búsqueda Binaria sobre reales para la misma idea de precisión), cada una con 2 evaluaciones de f .

```

1 struct BusquedaTernaria {
2     // READ: encuentra el x que MAXIMIZA f en el rango discreto
3     // [izq, der], asumiendo f unimodal (crece y luego decrece).
4     // f es un std::function para aceptar cualquier funcion o
5     // lambda.
6     static long long maximoDiscreto(long long izq, long long der,
7     function<long long(long long)> f) {
8         while (der - izq > 2) {
9             long long m1 = izq + (der - izq) / 3;
10            long long m2 = der - (der - izq) / 3;
11
12            if (f(m1) < f(m2))
13                izq = m1 + 1; // el pico no esta a la izq de m1
14            else
15                der = m2 - 1; // el pico no esta a la der de m2
16        }
17
18        // rango chico: evalua todos los candidatos restantes
19        long long mejorX = izq, mejorVal = f(izq);
20        for (long long x = izq + 1; x <= der; x++) {
21            long long val = f(x);
22            if (val > mejorVal) { mejorVal = val; mejorX = x; }
23        }
24        return mejorX;
25    } // O(log(n))
26    evaluaciones

```

```

27 // READ: version sobre REALES, con precision EPS en vez de un
28 // rango discreto que termine. Mismo esquema de descartar
29 // tercios.
30 static double maximoContinuo(double izq, double der,
31 function<double(double)> f,
32 double eps = 1e-9) {
33     while (der - izq > eps) {
34         double m1 = izq + (der - izq) / 3;
35         double m2 = der - (der - izq) / 3;
36
37         if (f(m1) < f(m2)) izq = m1;
38         else der = m2;
39     }
40     return (izq + der) / 2;
41 } // O(log((der-izq)/eps))
42 // evaluaciones
43 };

```

Ejemplo de funcionamiento:

```

1 // funcion unimodal: -(x-5)^2 + 100 -> maximo en x=5
2 auto f = [](long long x) { return -(x - 5) * (x - 5) + 100; };
3
4 long long x = BusquedaTernaria::maximoDiscreto(0, 20, f);
5 cout << x;
6 // 5
7
8 auto g = [](double x) { return -(x - 3.2) * (x - 3.2) + 50; };
9 double xr = BusquedaTernaria::maximoContinuo(0, 10, g);
10 cout << xr;
11 // -3.2

```

Nota: la versión discreta **no** se puede usar directamente sobre funciones **con empates** (varios x consecutivos con el mismo valor máximo) descartando de forma ingenua —si $f(m_1) == f(m_2)$, cualquiera de las dos ramas puede tomarse (ambas son seguras), pero conviene revisar el enunciado si "empates" son relevantes al resultado esperado (ej. si se pide el **menor** x que alcanza el máximo). El corte final con evaluación exhaustiva de los últimos 2-3 candidatos evita bugs de borde comunes al no achicar el rango a un solo punto con aritmética entera.

Regla práctica: confirma **siempre** que la función es unimodal antes de aplicar Ternary Search —si tiene más de un pico o valle, el algoritmo puede converger a un óptimo **local**, no al global, sin ningún aviso de error. Si la función es monótona (no unimodal), la herramienta correcta es Binary Search on Answer, no esto.

25.8 Búsqueda de Fibonacci (Fibonacci Search)

Idea: Fibonacci Search encuentra un elemento en un arreglo **ordenado** dividiendo el rango usando **números de Fibonacci** en vez de la mitad exacta (como Binary Search): se elige el menor Fibonacci $F_k \geq n$, y en cada paso se compara el elemento buscado contra la posición $\text{offset} + F_{k-2}$ —si es menor, el rango se reduce al tamaño F_{k-2} (los primeros elementos); si es mayor, se reduce a F_{k-1} (los últimos), avanzando el **offset**. La ventaja histórica es que las divisiones resultan en **sumas y restas** en vez de **divisiones por 2** (bit-shift), relevante en hardware antiguo donde la división era costosa —en hardware moderno esa ventaja prácticamente desapareció.

¿Por qué usarlo? En competencia moderna, casi nunca supera a Binary Search en la práctica ($\gg 1$ es tan barato como una resta en cualquier CPU actual) —se incluye aquí principalmente como **cultura general** y porque aparece ocasionalmente en preguntas teóricas sobre algoritmos de búsqueda. Su única ventaja real y vigente es en estructuras de **acceso no uniforme** donde

moverse a posiciones **cercanas** es más barato que saltar al medio exacto (ej. cintas magnéticas, o estructuras con costo de acceso creciente según la distancia) —un caso raro en competencia.

Casos de uso:

- Búsqueda en arreglo ordenado, como alternativa académica a Binary Search (mismo resultado, mismo orden asintótico).
- Preguntas teóricas sobre por qué la proporción áurea / Fibonacci aparece en particiones "óptimas" de un rango de búsqueda.
- Estructuras con costo de acceso no uniforme donde acercarse gradualmente (vía Fibonacci) es preferible a saltar directamente a la mitad —caso de nicho, casi nunca relevante en competencia.

Complejidad: $O(\log n)$, mismo orden que Binary Search —la base del logaritmo cambia (razón áurea $\phi \approx 1.618$ en vez de 2), pero asintóticamente es equivalente.

```

1 struct BusquedaFibonacci {
2     // READ: busca 'objetivo' en arr (YA ORDENADO). Devuelve el
3     // índice si lo encuentra, -1 si no esta.
4     static int buscar(vector<int>& arr, int objetivo) {
5         int n = arr.size();
6
7         // encuentra el menor Fibonacci >= n
8         int fib2 = 0, fib1 = 1, fibK = fib1 + fib2;
9         while (fibK < n) {
10             fib2 = fib1;
11             fib1 = fibK;
12             fibK = fib1 + fib2;
13         }
14
15         int offset = -1; // índice justo antes del rango actual
16
17         while (fibK > 1) {
18             // candidato: offset + fib2, recortado a un índice válido
19             int i = min(offset + fib2, n - 1);
20
21             if (arr[i] < objetivo) {
22                 // descarta [0, i]: mueve el "trío" de Fibonacci una
23                 // posición hacia atrás (rango se reduce a fib1)
24                 fibK = fib1;
25                 fib1 = fib2;
26                 fib2 = fibK - fib1;
27                 offset = i;
28             } else if (arr[i] > objetivo) {
29                 // descarta (i, n): el rango se reduce a fib2
30                 fibK = fib2;
31                 fib1 = fib1 - fib2;
32                 fib2 = fibK - fib1;
33             } else {
34                 return i; // encontrado
35             }
36         }
37
38         // ultimo elemento posible a revisar manualmente
39         if (fib1 == 1 && offset + 1 < n && arr[offset + 1] ==
40             objetivo)
41             return offset + 1;
42         return -1;
43     } // O(log n)
44 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
2
3 cout << BusquedaFibonacci::buscar(arr, 14);
4 // 6 (arr[6] = 14)
5
6 cout << BusquedaFibonacci::buscar(arr, 5);
7 // -1 (no esta en el arreglo)

```

Regla práctica: en competencia, usa Binary Search siempre —es más simple de escribir sin errores de índices, y el orden asintótico es idéntico. Fibonacci Search es principalmente valor académico: entender **por qué** funciona

(la partición de Fibonacci mantiene la misma proporción relativa en cada paso, análoga a como Binary Search mantiene la mitad) refuerza la intuición detrás de "eliminar una fracción constante del espacio de búsqueda en cada paso", que es el principio común a ambas técnicas.

25.9 Búsqueda exponencial (Exponential Search)

Idea: Exponential Search resuelve un problema que Binary Search normal no puede: buscar en un arreglo ordenado **del que no se conoce el tamaño** (o es **muy grande** y no conviene arrancar con $[0, n-1]$ completo). Primero **encuentra un rango** donde seguro está el elemento, duplicando un índice $i = 1, 2, 4, 8, 16, \dots$ mientras $\text{arr}[i] < \text{objetivo}$ (o mientras i no se salga del arreglo/stream) —en cuanto $\text{arr}[i] \geq \text{objetivo}$, el elemento (si existe) está en $[i/2, i]$; luego aplica **Binary Search normal** directamente sobre ese rango, ya acotado.

¿Por qué usarlo? Es la técnica correcta cuando el "tamaño" del espacio de búsqueda es **desconocido de antemano** o **no acotado** (streams, arreglos infinitos conceptuales, estructuras sin operación $O(1)$ de "tamaño total") —arrancar Binary Search con un límite superior real desconocido no es posible sin primero encontrar **algún** límite válido. También es útil como optimización cuando se **sospecha** que el elemento buscado está **cerca del inicio**: el costo de encontrar el rango es $O(\log(\text{posición real}))$, más barato que $O(\log n)$ completo si la posición real es mucho menor que n .

Casos de uso:

- Buscar en un arreglo ordenado de tamaño **desconocido** o efectivamente **ilimitado** (ej. un stream de datos ordenado, o una interfaz que solo expone $\text{arr}[i]$ sin exponer el tamaño total).
- Buscar en un arreglo muy grande donde se sospecha que el elemento está **cerca del inicio** — el rango de duplicación encuentra la vecindad correcta más rápido que arrancar desde el medio del arreglo completo.
- Como building block de **Binary Search on Answer** cuando el límite superior del rango de respuestas posibles no es obvio de antemano: se duplica hasta encontrar un límite superior válido, y luego se aplica búsqueda binaria normal dentro de ese rango.

Complejidad: $O(\log p)$ donde p es la posición real del elemento (o del límite encontrado) — $O(\log p)$ para encontrar el rango por duplicación, más $O(\log p)$ del Binary Search final dentro de ese rango.

```

1 struct BusquedaExponencial {
2     // READ: busca 'objetivo' en arr (YA ORDENADO). n puede ser
3     // el
4     // tamaño real, o un límite superior conservador si no se
5     // conoce
6     // el tamaño exacto (siempre que acceder arr[i] fuera de
7     // rango
8     // real este protegido -- ver nota abajo). Devuelve el índice
9     // si
10    // lo encuentra, -1 si no esta.
11    static int buscar(vector<int>& arr, int n, int objetivo) {
12        if (arr[0] == objetivo) return 0;
13
14        // fase 1: encuentra un rango [i/2, i] donde el objetivo
15        // podría estar, duplicando i mientras arr[i] < objetivo
16        int i = 1;
17        while (i < n && arr[i] < objetivo)

```

```

14     i *= 2;
15
16     // fase 2: binary search normal dentro de [i/2, min(i, n-1)]
17     int izq = i / 2, der = min(i, n - 1);
18     return binarySearch(arr, izq, der, objetivo);
19 } // O(log p)
20
21 static int binarySearch(vector<int>& arr, int izq, int der,
22 int objetivo) {
23     while (izq <= der) {
24         int mid = izq + (der - izq) / 2;
25
26         if (arr[mid] == objetivo) return mid;
27         if (arr[mid] < objetivo) izq = mid + 1;
28         else der = mid - 1;
29     }
30     return -1;
31 } // O(log(der - izq))
32 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23};
2
3 cout << BusquedaExponencial::buscar(arr, arr.size(), 13);
4 // 6 (arr[6] = 13; el rango encontrado por duplicacion fue
5 // [4,8]
6 // antes de aplicar binary search adentro)
7
8 cout << BusquedaExponencial::buscar(arr, arr.size(), 4);
9 // -1 (no esta en el arreglo)

```

Nota sobre tamaño desconocido: si realmente no se conoce n (ej. un stream), el chequeo $i < n$ de la fase de duplicación se reemplaza por un chequeo de "fin de stream" (o un centinela que indique que ya no hay más elementos) — la lógica de duplicar y luego aplicar Binary Search no cambia, solo la condición de corte al buscar el rango.

Regla práctica: usa Exponential Search en vez de Binary Search directo **solo** cuando el tamaño del arreglo es desconocido/no acotado, o cuando hay una razón concreta para sospechar que el elemento está cerca del inicio —si el tamaño ya se conoce y no hay sospecha de posición, Binary Search normal es más simple y da la misma complejidad asintótica sin la fase extra de duplicación.

25.10 Búsqueda por saltos (Jump Search)

Idea: Jump Search busca en un arreglo **ordenado** avanzando en **bloques de tamaño fijo** $\sqrt{n}$ (en vez de saltar a la mitad como Binary Search, o duplicar como Exponential Search): salta de bloque en bloque mientras el último elemento del bloque actual sea **menor** que el objetivo, y en cuanto encuentra el bloque donde **podría** estar (su último elemento es $\geq$ objetivo), hace una **búsqueda lineal** dentro de ese bloque de tamaño $\sqrt{n}$. El tamaño de bloque óptimo es exactamente $\sqrt{n}$: balancea el número de saltos (n/bloque) contra el costo de la búsqueda lineal final (bloque) —ambos términos se minimizan simultáneamente cuando $\text{bloque} = \sqrt{n}$.

¿Por qué usarlo? Es estrictamente **peor** que Binary Search en complejidad asintótica ($O(\sqrt{n})$ contra $O(\log n)$) —su único valor práctico es en estructuras donde **retroceder** es costoso o imposible (ej. buscar en una cinta secuencial, o cuando "saltar hacia atrás" tiene un costo real distinto de "avanzar"), ya que Jump Search solo avanza en una dirección, nunca necesita retroceder como sí puede requerir la recursión de Binary Search en estructuras con acceso puramente secuencial. En com-

petencia normal, con acceso $O(1)$ a cualquier índice, no hay razón para preferirlo sobre Binary Search.

Casos de uso:

- Búsqueda en estructuras de acceso **secuencial** donde retroceder tiene un costo alto o no es posible (cintas, algunos tipos de streams).
- Como introducción pedagógica al **balance de trade-offs** en algoritmos: por qué $\sqrt{n}$ es el tamaño de bloque óptimo cuando se combinan un costo lineal con uno inversamente proporcional al tamaño de bloque (el mismo tipo de análisis aparece en *sqrt decomposition* para RMQ o en la optimización de consultas por bloques en general).
- Punto de comparación conceptual antes de introducir *sqrt decomposition* completa como estructura de datos (fuera del alcance de esta sección).

Complejidad: $O(\sqrt{n})$ — $O(\sqrt{n})$ saltos de bloque en bloque, más $O(\sqrt{n})$ de búsqueda lineal dentro del bloque final.

```

1 struct BusquedaPorSaltos {
2     // READ: busca 'objetivo' en arr (YA ORDENADO), saltando en
3     // bloques de tamaño sqrt(n). Devuelve el índice si lo
4     // encuentra,
5     // -1 si no esta.
6     static int buscar(vector<int>& arr, int objetivo) {
7         int n = arr.size();
8         int tamBloque = max(1, (int)sqrt((double)n));
9
10        // fase 1: salta de bloque en bloque mientras el ultimo
11        // elemento del bloque actual sea MENOR que el objetivo
12        int actual = 0, siguiente = tamBloque;
13        while (siguiente < n && arr[siguiente - 1] < objetivo) {
14            actual = siguiente;
15            siguiente += tamBloque;
16        }
17
18        // fase 2: búsqueda LINEAL dentro del bloque candidato
19        int limite = min(siguiente, n);
20        for (int i = actual; i < limite; i++)
21            if (arr[i] == objetivo)
22                return i;
23
24        return -1;
25    } // O(sqrt(n))
26 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89,
2 144};
3 // n=13, tamBloque = sqrt(13) ~ 3
4
5 cout << BusquedaPorSaltos::buscar(arr, 55);
6 // 10 (arr[10] = 55; encontrado saltando bloques de 3 hasta
7 // ubicar el bloque correcto, luego lineal dentro de el)
8
9 cout << BusquedaPorSaltos::buscar(arr, 6);
10 // -1 (no esta en el arreglo)

```

Regla práctica: en competencia con acceso $O(1)$ a cualquier índice, Binary Search siempre domina a Jump Search en complejidad —no hay razón práctica para usar esta técnica salvo el caso específico de acceso puramente secuencial sin retroceso barato. Su valor real en esta guía es **conceptual**: introduce la idea de "tamaño de bloque $\sqrt{n}$ como balance óptimo entre dos costos opuestos", un patrón que reaparece en técnicas de *sqrt decomposition* más generales.

25.11 Meet in the Middle

Idea: Meet in the Middle resuelve problemas de búsqueda exhaustiva sobre n elementos ($O(2^n)$, típicamente subconjuntos) **partiendo** el conjunto en dos

mitades de tamaño $n/2$ cada una: se generan **todas** las combinaciones posibles de cada mitad por separado ($O(2^{n/2})$ cada una, mucho más barato que $O(2^n)$), y luego se **combinan** los resultados de ambas mitades —usualmente ordenando una mitad y aplicando Two Pointers o Binary Search contra la otra— para encontrar la respuesta global sin tener que enumerar las 2^n combinaciones completas.

¿Por qué usarlo? Convierte un problema con n hasta ~ 40 (imposible de fuerza bruta directa, 2^{40} es demasiado) en uno resoluble: $2^{20} + 2^{20}$ combinaciones generadas, más el costo de combinarlas (usualmente $O(2^{n/2} \log(2^{n/2}))$ por el **sort**), es perfectamente manejable en el límite de tiempo típico. Es el mismo principio que *divide and conquer*, pero aplicado al **espacio de búsqueda** en vez de a la estructura de los datos: partir el problema exponencial en dos problemas exponenciales **más chicos**, cuya suma de costos es mucho menor que el original.

Casos de uso:

- **Subset Sum** con n hasta ~ 40 : ¿existe un subconjunto que sume exactamente X ? Se generan todas las sumas posibles de cada mitad, se ordena una, y se busca en la otra (con Two Pointers o Binary Search) el complemento necesario.
- Contar cuántos subconjuntos cumplen una condición de suma (variante de conteo del caso anterior).
- Problemas de "elige un subconjunto de cada uno de dos grupos disjuntos, maximizando/minimizando algo" donde generar todas las combinaciones de cada grupo por separado es factible.
- Como técnica complementaria a Bitmask DP cuando n es demasiado grande para $O(2^n \cdot n)$ pero sigue siendo pequeño en cada mitad.

Complejidad: $O(2^{n/2} \log(2^{n/2})) = O(2^{n/2} \cdot n)$ —generar cada mitad es $O(2^{n/2})$, ordenar una de ellas domina el costo total.

```

1 struct MeetInTheMiddle {
2     // READ: existe un subconjunto de arr que suma EXACTAMENTE X?
3     // Parte arr en dos mitades, genera todas las sumas posibles
4     // de cada una, y busca el complemento con Two Pointers.
5     static bool existeSubconjuntoConSuma(vector<int>& arr, long
6         long X) {
7         int n = arr.size();
8         int mitad = n / 2;
9
10        vector<long long> sumasA = generarSumas(arr, 0, mitad);
11        vector<long long> sumasB = generarSumas(arr, mitad, n);
12
13        sort(sumasB.begin(), sumasB.end());
14
15        // para cada suma de A, busca si (X - sumaA) existe en B
16        for (long long sa : sumasA) {
17            long long faltante = X - sa;
18            if (binary_search(sumasB.begin(), sumasB.end(), faltante))
19                return true;
20        }
21        return false;
22    } // O(2^(n/2) * n)
23
24    // genera las 2^(fin-inicio) sumas posibles de todos los
25    // subconjuntos del segmento arr[inicio, fin).
26    static vector<long long> generarSumas(vector<int>& arr, int
27        inicio,
28        int fin) {
29        int m = fin - inicio;

```

```

29 vector<long long> sumas;
30 sumas.reserve(1 << m);
31
32 for (int mask = 0; mask < (1 << m); mask++) {
33     long long suma = 0;
34     for (int i = 0; i < m; i++)
35         if (mask & (1 << i))
36             suma += arr[inicio + i];
37     sumas.push_back(suma);
38 }
39
40 return sumas;
41 } // O(2^m * m)
42
43 // READ: cuenta cuantos SUBCONJUNTOS (no vacios o vacios
44 // incluidos
45 // segun se cuente el mask=0) suman EXACTAMENTE X, usando el
46 // mismo esquema de dos mitades pero con conteo de
47 // coincidencias
48 // en vez de existencia.
49 static long long contarSubconjuntosConSuma(vector<int>& arr,
50     long long X) {
51     int n = arr.size();
52     int mitad = n / 2;
53
54     vector<long long> sumasA = generarSumas(arr, 0, mitad);
55     vector<long long> sumasB = generarSumas(arr, mitad, n);
56
57     sort(sumasB.begin(), sumasB.end());
58
59     long long total = 0;
60     for (long long sa : sumasA) {
61         long long faltante = X - sa;
62         // cuenta CUANTAS veces aparece 'faltante' en sumasB
63         auto rango = equal_range(sumasB.begin(), sumasB.end(),
64             faltante);
65         total += rango.second - rango.first;
66     }
67
68     return total;
69 } // O(2^(n/2) * n)

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {3, 34, 4, 12, 5, 2};
2
3 cout << MeetInTheMiddle::existeSubconjuntoConSuma(arr, 9);
4 // true   (4 + 5 = 9, o 3 + 4 + 2 = 9)
5
6 cout << MeetInTheMiddle::existeSubconjuntoConSuma(arr, 100);
7 // false  (la suma total del arreglo es 60, menor a 100)

```

Regla práctica: la señal para Meet in the Middle es n **demasiado grande** para $O(2^n)$ directo ($n \gtrsim 25-30$) pero **demasiado chico** para que un enfoque polinomial (DP, greedy) sea obviamente aplicable —usualmente $n \leq 40$. Si n es lo bastante chico para $O(2^n)$ liso ($n \leq 20-22$), Meet in the Middle es complejidad extra innecesaria; si n es grande ($n > 40-50$), ni siquiera $O(2^{n/2})$ alcanza y hace falta buscar una solución polinomial real.

25.12 Heap Sort

Idea: Heap Sort ordena un arreglo en dos fases usando la estructura de **heap binario** (implementado sobre el mismo arreglo, sin memoria extra): (1) **construir** un *max-heap* a partir del arreglo completo, aplicando **heapify** desde el último nodo **interno** hacia la raíz (no hace falta empezar desde las hojas, ya son heaps triviales de tamaño 1); (2) repetir $n - 1$ veces: **intercambiar** la raíz (el máximo actual) con el **último** elemento del heap activo, **reducir** el tamaño del heap en 1 (ese último elemento ya queda en su posición final ordenada), y aplicar **heapify** desde la raíz para restaurar la propiedad de heap sobre lo que queda.

¿Por qué usarlo? Es el único de los sorts de comparación clásicos ($O(n \log n)$) que logra $O(1)$ de memoria extra y garantiza $O(n \log n)$ en el **peor caso** —Quick Sort es $O(n \log n)$ en promedio pero $O(n^2)$ en el peor

caso (sin aleatorización de pivote), y Merge Sort garantiza $O(n \log n)$ siempre pero necesita $O(n)$ de memoria auxiliar. La desventaja práctica es que Heap Sort tiene peor localidad de caché (los accesos saltan por todo el arreglo según la estructura del heap), por lo que en la práctica suele ser más lento que Quick Sort bien implementado, aunque ambos compartan el mismo orden asintótico promedio.

Casos de uso:

- Ordenar cuando se necesita **garantía** de $O(n \log n)$ en el peor caso y memoria extra $O(1)$ simultáneamente (ninguna otra opción clásica da ambas garantías a la vez).
- Como base conceptual para entender `priority_queue` de la STL, que internamente usa exactamente esta estructura de heap binario sobre un arreglo.
- Encontrar los k elementos más grandes/chicos sin ordenar todo el arreglo: construir el heap ($O(n)$) y extraer solo k veces ($O(k \log n)$), en vez de un sort completo ($O(n \log n)$) cuando $k \ll n$.

Complejidad: $O(n \log n)$ garantizado (peor, promedio y mejor caso son todos $O(n \log n)$) — $O(n)$ para construir el heap inicial (no $O(n \log n)$, por un análisis más fino de la suma de alturas), $O(\log n)$ por cada una de las $n - 1$ extracciones.

```
1 vector<int> arr = {12, 11, 13, 5, 6, 7};
2
3 HeapSort::ordenar(arr);
4
5 for (int x : arr) cout << x << " ";
6 // 5 6 7 11 12 13
```

Nota: la construcción del heap (FASE 1) parece a primera vista $O(n \log n)$ (n llamadas a `heapify`, cada una $O(\log n)$), pero un análisis más cuidadoso —sumando la altura real de cada nodo, no el peor caso uniforme— da $O(n)$ real: la **mitad** de los nodos son hojas (altura 0, sin trabajo), un cuarto tiene altura 1, etc., y esa serie geométrica converge a $O(n)$ en total, no a $O(n \log n)$.

Regla práctica: en competencia, `std::sort` (Quick Sort/Intro Sort híbrido de la STL) es casi siempre la opción por defecto —más rápido en la práctica por mejor localidad de caché. Implementa Heap Sort a mano **solo** cuando el problema específicamente requiere la garantía de peor caso $O(n \log n)$ sin memoria extra (raro en competencia), o cuando se necesita la estructura de heap **en sí** para algo más (como extraer solo los k mayores) — en ese caso, `priority_queue` de la STL ya implementa exactamente este mismo algoritmo internamente.

25.13 Merge Sort

Idea: Merge Sort ordena un arreglo con *divide and conquer*: divide el arreglo por la **mitad** recursivamente hasta llegar a subarreglos de tamaño 1 (trivialmente ordenados), y luego **combina** (*merge*) pares de subarreglos ya ordenados en uno solo ordenado, usando el mismo esquema de Two Pointers ya visto: un puntero por subarreglo, avanzando siempre el que tiene el menor elemento actual, hasta consumir ambos por completo. La recursión completa hace $O(\log n)$ niveles de división, y cada nivel hace $O(n)$ de trabajo total en sus *merges*, dando $O(n \log n)$.

¿Por qué usarlo? A diferencia de Quick Sort (promedio $O(n \log n)$, peor caso $O(n^2)$), Merge Sort **garantiza** $O(n \log n)$ en **todos** los casos, sin depender de la elección de pivote ni de aleatorización — el costo es $O(n)$ de memoria auxiliar (el arreglo temporal para el *merge*), que Heap Sort evita pero a cambio de peor localidad de caché. Es además **estable** (mantiene el orden relativo de elementos iguales), propiedad que ni Quick Sort ni Heap Sort garantizan por defecto — y es la base directa de Counting Inversions (siguiente subsección), que reutiliza exactamente esta misma recursión.

Casos de uso:

- Ordenar cuando se necesita **estabilidad** (preservar el orden relativo de elementos con la misma clave) o garantía de peor caso $O(n \log n)$ sin las complicaciones de aleatorizar un pivote.
- Como building block directo de Counting Inversions: contar cuántos pares están "fuera de orden" se calcula gratis durante el mismo *merge*.
- *External sorting*: ordenar datos que no caben en memoria, procesando bloques y fusionándolos — el mismo principio de *merge* de dos secuencias ordenadas se extiende naturalmente a fusionar k bloques desde disco.

```
1 struct HeapSort {
2     // restaura la propiedad de MAX-HEAP en el subarbol con raiz
3     // en
4     // 'i', asumiendo que sus hijos YA cumplen la propiedad (el
5     // desorden, si existe, esta solo en 'i').
6     static void heapify(vector<int>& arr, int tamHeap, int i) {
7         int mayor = i;
8         int izq = 2 * i + 1, der = 2 * i + 2;
9
10        if (izq < tamHeap && arr[izq] > arr[mayor]) mayor = izq;
11        if (der < tamHeap && arr[der] > arr[mayor]) mayor = der;
12
13        if (mayor != i) {
14            swap(arr[i], arr[mayor]);
15            heapify(arr, tamHeap, mayor); // el desorden pudo
16            // "bajar" al hijo, hay
17            // que seguir corrigiendo
18        }
19    } // O(log n)
20
21    // WRITE: ordena arr in-place, ascendente.
22    static void ordenar(vector<int>& arr) {
23        int n = arr.size();
24
25        // FASE 1: construye el max-heap. Empieza en el ultimo nodo
26        // INTERNO (n/2 - 1), no en las hojas (ya son heaps
27        // triviales).
28        for (int i = n / 2 - 1; i >= 0; i--)
29            heapify(arr, n, i);
30        // esta fase completa es O(n), no O(n log n) -- la mayoria
31        // de
32        // nodos estan cerca de las hojas, donde heapify hace poco
33        // trabajo; solo unos pocos nodos cerca de la raiz pagan el
34        // costo completo de log n
35
36        // FASE 2: extrae el maximo repetidamente, colocandolo al
37        // final del rango activo.
38        for (int tamHeap = n - 1; tamHeap > 0; tamHeap--) {
39            swap(arr[0], arr[tamHeap]); // raiz (maximo actual) va
40            // a su posicion final
41            heapify(arr, tamHeap, 0); // restaura el heap sobre
42            // lo que queda (tamano
43            // reducido en 1)
44        }
45    } // O(n log n)
46 };
```

Ejemplo de funcionamiento:

- Como plantilla de *divide and conquer* general: cualquier problema que se resuelva "divide en dos mitades, resuelve cada una, combina en $O(n)$ " tiene a Merge Sort como su ejemplo canónico.

Complejidad: $O(n \log n)$ garantizado en todos los casos (peor, promedio, mejor), $O(n)$ de memoria auxiliar para los *merges*.

```

1 struct MergeSort {
2     // fusiona arr[izq..mid] y arr[mid+1..der] (ambos YA
3     // ORDENADOS
4     // entre si) en un solo rango ordenado, usando un arreglo
5     // temporal (Two Pointers, mismo esquema que fusionar() de la
6     // seccion de Two Pointers).
7     static void merge(vector<int>& arr, int izq, int mid, int der)
8     {
9         vector<int> temp(der - izq + 1);
10        int i = izq, j = mid + 1, k = 0;
11
12        while (i <= mid && j <= der) {
13            if (arr[i] <= arr[j]) temp[k++] = arr[i++];
14            else temp[k++] = arr[j++];
15        }
16        while (i <= mid) temp[k++] = arr[i++];
17        while (j <= der) temp[k++] = arr[j++];
18
19        for (int x = 0; x < (int)temp.size(); x++)
20            arr[izq + x] = temp[x];
21    } // O(der - izq)
22
23    // WRITE: ordena arr[izq..der] recursivamente (divide and
24    // conquer).
25    static void ordenar(vector<int>& arr, int izq, int der) {
26        if (izq >= der) return; // caso base: 0 o 1 elemento, ya
27        // ordenado
28
29        int mid = izq + (der - izq) / 2;
30
31        ordenar(arr, izq, mid);
32        ordenar(arr, mid + 1, der);
33        merge(arr, izq, mid, der);
34    } // O(n log n)
35
36    // conveniencia: ordena el arreglo completo.
37    static void ordenar(vector<int>& arr) {
38        if (!arr.empty())
39            ordenar(arr, 0, arr.size() - 1);
40    }
41 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {38, 27, 43, 3, 9, 82, 10};
2
3 MergeSort::ordenar(arr);
4
5 for (int x : arr) cout << x << " ";
6 // 3 9 10 27 38 43 82

```

Regla práctica: en competencia, `std::sort` (o `std::stable_sort` si se necesita estabilidad) casi siempre reemplaza una implementación manual de Merge Sort —salvo que el problema pida **modificar** el algoritmo mismo (como en Counting Inversions, donde se necesita instrumentar el *merge* para contar algo adicional durante la fusión, algo que `std::sort` no permite hacer desde afuera).

25.14 Conteo de Inversiones (Counting Inversions)

Idea: una **inversión** es un par de índices (i, j) con $i < j$ pero $arr[i] > arr[j]$ (el par está "fuera de orden"). Contarlas todas por fuerza bruta es $O(n^2)$; se resuelve en $O(n \log n)$ instrumentando **exactamente** el mismo Merge Sort de la subsección anterior: durante el *merge* de dos mitades ya ordenadas, cada vez que se toma un elemento de la mitad **derecha** antes que uno de la mitad **izquierda** (porque es menor), ese elemento

forma una inversión con **todos** los elementos **restantes** de la mitad izquierda —ya que la mitad izquierda está ordenada, todos esos restantes son también mayores, y automáticamente forman inversión con el elemento de la derecha que se acaba de tomar.

¿Por qué usarlo? Es el ejemplo canónico de "instrumentar *divide and conquer* para contar algo adicional sin cambiar la complejidad" —el conteo sale **gratis** dentro del mismo *merge* que ya se hacía para ordenar, sin ningún recorrido extra. La alternativa con Fenwick Tree (procesar el arreglo de derecha a izquierda, consultando cuántos elementos **menores** ya se insertaron) también es $O(n \log n)$ y es preferible cuando de todas formas ya se tiene un Fenwick a mano o se necesita una variante más flexible (ej. contar inversiones con una condición extra) —pero el enfoque de Merge Sort es más directo cuando el problema es exactamente "contar inversiones", sin extras.

Casos de uso:

- Contar cuántos intercambios de elementos **adyacentes** hacen falta como mínimo para ordenar un arreglo (es exactamente el número de inversiones —cada intercambio adyacente corrige **una** inversión).
- Medir qué tan "desordenado" está un arreglo respecto a su versión ordenada (similitud entre dos rankings, por ejemplo).
- Como subrutina de problemas más grandes que reducen su núcleo a "cuenta pares fuera de orden" tras alguna transformación previa del arreglo original.

Complejidad: $O(n \log n)$ — mismo costo que Merge Sort, ya que el conteo se hace sin trabajo asintótico adicional durante el *merge*.

```

1 struct ConteoInversiones {
2     // fusiona arr[izq..mid] y arr[mid+1..der], contando
3     // inversiones:
4     // cada vez que se toma un elemento de la mitad DERECHA antes
5     // que uno de la IZQUIERDA, se suman TODOS los elementos
6     // restantes de la izquierda como inversiones con ese elemento.
7     static long long merge(vector<int>& arr, int izq, int mid,
8     int der) {
9         vector<int> temp(der - izq + 1);
10        int i = izq, j = mid + 1, k = 0;
11        long long inversiones = 0;
12
13        while (i <= mid && j <= der) {
14            if (arr[i] <= arr[j]) {
15                temp[k++] = arr[i++];
16            } else {
17                // arr[j] es MENOR que arr[i], y como [izq..mid] esta
18                // ordenado, arr[j] tambien es menor que TODO lo que
19                // queda desde i hasta mid -- eso son (mid - i + 1)
20                // inversiones de una sola vez
21                inversiones += (mid - i + 1);
22                temp[k++] = arr[j++];
23            }
24        }
25        while (i <= mid) temp[k++] = arr[i++];
26        while (j <= der) temp[k++] = arr[j++];
27
28        for (int x = 0; x < (int)temp.size(); x++)
29            arr[izq + x] = temp[x];
30
31        return inversiones;
32    } // O(der - izq)
33
34    // READ: cuenta inversiones en arr[izq..der], ordenando de
35    // paso
36    // (modifica arr -- pasar una COPIA si se necesita conservar
37    // el
38    // orden original).
39    static long long contar(vector<int>& arr, int izq, int der) {

```

```

36     if (izq >= der) return 0;
37
38     int mid = izq + (der - izq) / 2;
39
40     long long total = 0;
41     total += contar(arr, izq, mid);
42     total += contar(arr, mid + 1, der);
43     total += merge(arr, izq, mid, der);
44
45     return total;
46 } // O(n log n)
47
48 // conveniencia: cuenta inversiones en TODO el arreglo (
49 // recibe
50 // una COPIA para no alterar el original del llamador).
51 static long long contar(vector<int> arr) {
52     if (arr.empty()) return 0;
53     return contar(arr, 0, arr.size() - 1);
54 }
55 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {8, 4, 2, 1};
2
3 cout << ConteoInversiones::contar(arr);
4 // 6
5 // pares fuera de orden: (8,4)(8,2)(8,1)(4,2)(4,1)(2,1) -> 6 en
6 // total
7 // (arreglo totalmente invertido de tamaño 4: n*(n-1)/2 = 6)

```

Regla práctica: la función `contar()` recibe el arreglo **por valor** (copia) a propósito en su versión de conveniencia — el algoritmo necesita **ordenar** el arreglo como efecto secundario del conteo, así que si el problema requiere conservar el orden original después de contar, hay que operar sobre una copia (como se hace aquí) en vez de sobre el arreglo del llamador directamente.

25.15 QuickSelect

Idea: QuickSelect resuelve el problema de encontrar el k -ésimo elemento más pequeño (la k -ésima estadística de orden) de un arreglo sin necesidad de ordenarlo por completo. Se basa en el mismo esquema de partición que QuickSort: se elige un pivote, se reorganiza el arreglo de modo que todo lo menor al pivote quede a su izquierda y todo lo mayor a su derecha, y el pivote termina exactamente en su posición final ordenada. La diferencia clave frente a QuickSort es que, tras particionar, QuickSelect *descarta* el lado que no contiene la posición k buscada y solo recurre sobre el lado relevante, en vez de recursar sobre ambos lados. Esto reduce el tamaño esperado del problema geométricamente (aproximadamente a la mitad en cada paso, con un pivote razonable), en vez de linealmente como ocurriría si se ordenara todo.

¿Por qué usarlo? Cuando solo se necesita un elemento de orden específico (la mediana, el percentil 90, el k -ésimo mayor) y no el arreglo completo ordenado, ordenar todo con `sort` cuesta $O(n \log n)$ de forma innecesaria. QuickSelect resuelve el mismo problema en $O(n)$ esperado. Se diferencia de estructuras como un *heap* (que da $O(n + k \log n)$ para el k -ésimo menor) en que no requiere mantener ninguna estructura auxiliar ni soporta consultas repetidas eficientes: es ideal para una consulta única sobre datos estáticos. Si se necesitan múltiples consultas de orden sobre el mismo conjunto, o si el conjunto cambia dinámicamente, es mejor usar un *order statistics tree* o un *heap* persistente.

Casos de uso:

- Encontrar la mediana de un arreglo sin ordenarlo completo.

- Calcular percentiles (p. ej. el elemento en la posición $[0.9n]$) para estadística descriptiva.
- Como subrutina para encontrar los k elementos más cercanos a un punto de referencia (particionar y quedarse con un lado).
- Selección de mediana de medianas como pivote determinista en algoritmos de ordenamiento con garantías de peor caso.
- Problemas de competencia donde se pide el k -ésimo mayor/menor elemento de un arreglo grande bajo límite de tiempo ajustado.

Complejidad: El caso esperado es $O(n)$: cada partición cuesta $O(m)$ sobre un subarreglo de tamaño m , y con un pivote aleatorio el tamaño esperado del subarreglo restante se reduce por un factor constante en cada paso, dando una serie geométrica $n + n/2 + n/4 + \dots = O(n)$ en total. El peor caso es $O(n^2)$, que ocurre si el pivote elegido resulta ser siempre el mínimo o el máximo del subarreglo (por ejemplo, con pivote fijo sobre datos adversarios); la aleatorización del pivote hace que este peor caso sea extremadamente improbable en la práctica.

```

1 struct QuickSelect {
2     vector<int> arr;
3     mt19937 rng;
4
5     // CREATE: copia los datos de entrada y prepara el generador
6     // aleatorio para la seleccion de pivotes
7     QuickSelect(vector<int> datos)
8     : arr(datos),
9       rng(chrono::steady_clock::now().time_since_epoch().count())
10    {} // O(n)
11
12    // WRITE: particiona arr[izq..der] usando arr[der] como
13    // pivote
14    // (tras intercambiarlo con un pivote aleatorio), dejando los
15    // menores al pivote a la izquierda; retorna la posición
16    // final
17    // del pivote
18    int particionar(int izq, int der) {
19        int pivoteIdx = izq + rng() % (der - izq + 1);
20        swap(arr[pivoteIdx], arr[der]);
21        int pivote = arr[der];
22        int i = izq;
23        for (int j = izq; j < der; j++) {
24            if (arr[j] < pivote) {
25                swap(arr[i], arr[j]);
26                i++;
27            }
28        }
29        swap(arr[i], arr[der]);
30        return i;
31    } // O(der - izq)
32
33    // READ: retorna el elemento en la posición k (0-indexado)
34    // que
35    // tendría el arreglo si estuviera completamente ordenado
36    int kEsimoMenor(int k) {
37        int izq = 0, der = (int)arr.size() - 1;
38        while (true) {
39            if (izq == der) return arr[izq];
40            int p = particionar(izq, der);
41            if (k == p) return arr[p];
42            else if (k < p) der = p - 1;
43            else izq = p + 1;
44        }
45    } // O(n) esperado, O(n^2) peor caso
46
47    //
48    // -----
49    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
50    //
51    // -----
52    // 1. Version iterativa vs recursiva: la version de arriba ya
53    // es
54    // iterativa (bucle while) para evitar overflow de pila en
55    // arreglos muy grandes; una version recursiva es
56    // equivalente
57    // pero con riesgo de stack overflow para n grande.
58    //

```

```

51 // 2. Mediana de medianas (peor caso  $O(n)$  garantizado): en
52 // vez de
53 // pivote aleatorio, se divide el arreglo en grupos de 5,
54 // se
55 // calcula la mediana de cada grupo, y se usa
56 // recursivamente
57 // la mediana de esas medianas como pivote. Garantiza que
58 // el
59 // pivote elimina al menos un 30% del arreglo en cada paso,
60 // dando  $O(n)$  en el peor caso, pero con una constante
61 // mayor
62 // que hace que en la practica sea mas lento que la
63 // version
64 // aleatorizada.
65 //
66 // 3.  $k$ -esimo mayor en vez de menor: basta con llamar
67 //  $k$ EsimoMenor( $n - 1 - k$ ) sobre el mismo arreglo, ya que
68 // el
69 //  $k$ -esimo mayor es el  $(n-1-k)$ -esimo menor.
70 //
71 // 4. Obtener los  $k$  elementos mas pequeños (no solo el  $k$ -
72 // esimo):
73 // tras llamar particionar hasta que la posicion del
74 // pivote
75 // sea  $k$ , los primeros  $k$  elementos de arr (arr[0..k-1])
76 // son
77 // exactamente esos  $k$  elementos, aunque no necesariamente
78 // ordenados entre si.
79 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {7, 2, 9, 4, 1, 6, 3};
2 QuickSelect qs(datos);
3
4 int mediana = qs.kEsimoMenor(3); // arreglo ordenado seria
5 // {1,2,3,4,6,7,9} -> posicion 3 = 4
6 // mediana = 4
7
8 int minimo = qs.kEsimoMenor(0); // minimo = 1
9 int maximo = qs.kEsimoMenor((int)datos.size() - 1); // maximo = 9

```

Nota: La partición modifica `arr` en el sitio, por lo que después de llamar `kEsimoMenor`, el orden interno del arreglo cambia (aunque el conjunto de elementos se preserva). Si se necesita conservar el arreglo original, se debe pasar una copia al construir el struct o guardar una copia aparte antes de consultar.

Regla práctica: Usa `QuickSelect` cuando necesites un único elemento de orden (mediana, percentil, k -ésimo mayor/menor) de un conjunto *estático* y no te importe alterar su orden interno; si necesitas el arreglo completo ordenado de todas formas, usa `sort` directamente; si necesitas múltiples consultas de orden o el conjunto cambia con el tiempo, usa una estructura como un *order statistics tree*.

26 Búsqueda Binaria (Binary Search)

27 Búsqueda Binaria (Binary Search)

27.1 En arreglo ordenado

Idea: La búsqueda binaria explota el hecho de que, en un arreglo ordenado, comparar el elemento central contra el valor buscado permite descartar la mitad del espacio de búsqueda en cada paso: si el elemento central es igual al buscado, se encontró; si es menor, la respuesta (si existe) está estrictamente a la derecha; si es mayor, está estrictamente a la izquierda. Al descartar la mitad del espacio en cada iteración, el número de comparaciones necesarias crece logarítmicamente con el tamaño del arreglo, en vez de linealmente como en una búsqueda secuencial.

¿Por qué usarlo? Cuando el arreglo ya está ordenado (o se puede ordenar una sola vez y consultar muchas veces), buscar un elemento con búsqueda binaria cuesta $O(\log n)$ frente a $O(n)$ de una búsqueda lineal. A diferencia de `QuickSelect`, que encuentra el k -ésimo elemento de orden en un arreglo *desordenado* sin necesidad de ordenarlo, la búsqueda binaria requiere que el arreglo ya esté ordenado, pero a cambio responde a la pregunta "¿está este valor exacto presente, y en qué posición?" en tiempo logarítmico, y puede reutilizarse para múltiples consultas sin costo adicional de preprocesamiento.

Casos de uso:

- Verificar si un valor existe en un arreglo ordenado y obtener su índice.
- Encontrar el punto de inserción de un valor manteniendo el orden (equivalente a `lower_bound/upper_bound`).
- Contar ocurrencias de un valor en un arreglo ordenado (diferencia entre `upper_bound` y `lower_bound`).
- Como subrutina dentro de estructuras más complejas (por ejemplo, búsqueda en un arreglo con offsets, o en un Fenwick tree con búsqueda binaria implícita).

Complejidad: $O(\log n)$ por consulta, ya que en cada iteración el tamaño del espacio de búsqueda se reduce a la mitad: partiendo de n elementos, tras t iteraciones quedan $n/2^t$ elementos, y el proceso termina cuando $n/2^t \approx 1$, es decir $t \approx \log_2 n$. El espacio usado es $O(1)$ en la versión iterativa, ya que no se requiere memoria adicional proporcional a n .

```

1 struct BusquedaBinaria {
2     vector<int> arr;
3
4     // CREATE: guarda una copia del arreglo, que debe estar
5     // ordenado
6     // de forma ascendente
7     BusquedaBinaria(vector<int> datos) : arr(datos) {} //  $O(n)$ 
8
9     // READ: retorna el indice del elemento igual a "valor" si
10    // existe, o -1 si no esta presente en el arreglo
11    int buscar(int valor) {
12        int izq = 0, der = (int)arr.size() - 1;
13        while (izq <= der) {
14            int medio = izq + (der - izq) / 2;
15            if (arr[medio] == valor) return medio;
16            else if (arr[medio] < valor) izq = medio + 1;
17            else der = medio - 1;
18        }
19        return -1;
20    } //  $O(\log n)$ 
21
22    //
23    // -----
24    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25    // -----
26
27    // 1. Evitar overflow en el calculo del medio: usar
28    // izq + (der - izq) / 2 en vez de (izq + der) / 2, ya que
29    // la
30    // segunda forma puede desbordar si izq y der son cercanos
31    // al
32    // limite de int.
33    //
34    // 2. Version con funciones nativas de C++: para arreglos
35    // ordenados, std::binary_search(arr.begin(), arr.end(),
36    // valor)
37    // retorna un bool en vez del indice; si se necesita el
38    // indice,
39    // se puede usar std::lower_bound y verificar que el valor
40    // en
41    // esa posicion sea igual al buscado.
42    //

```

```

35 // 3. Búsqueda en arreglo ordenado descendente: invertir las
36 // comparaciones (si arr[medio] < valor, mover der = medio
37 // - 1;
38 // si arr[medio] > valor, mover izq = medio + 1).
39 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {1, 3, 5, 7, 9, 11, 13};
2 BúsquedaBinaria bb(datos);
3
4 int idx1 = bb.buscar(7); // idx1 = 3 (arr[3] = 7)
5 int idx2 = bb.buscar(4); // idx2 = -1 (4 no está en el arreglo)

```

Regla práctica: Usa búsqueda binaria clásica cuando busques la presencia y posición exacta de un valor en un arreglo ya ordenado; si en cambio necesitas el primer o último índice donde se cumple una condición monótona (no necesariamente una igualdad exacta), usa las variantes de "menor/mayor x que cumple" que se ven a continuación.

27.2 Menor x que cumple

Idea: Esta variante generaliza la búsqueda binaria clásica: en vez de buscar un valor exacto, busca el menor índice (o valor) x para el cual una predicado booleano $P(x)$ se vuelve verdadero, bajo el supuesto de que P es *monótono*: una vez que $P(x)$ es verdadero, permanece verdadero para todo valor mayor que x (patrón FFFF...FVVVV...V). En cada paso se evalúa P en el punto medio: si $P(\text{medio})$ es verdadero, el medio es un candidato válido pero podría no ser el mínimo, así que se guarda como respuesta tentativa y se sigue buscando a la izquierda; si es falso, la respuesta debe estar estrictamente a la derecha.

¿Por qué usarlo? La búsqueda binaria en arreglo ordenado (vista arriba) resuelve el caso particular donde $P(x) := (\text{arr}[x] \geq \text{valor})$, que es monótono porque el arreglo está ordenado. Esta variante generaliza esa idea a *cualquier* predicado monótono, no solo a comparaciones directas contra un arreglo ordenado, lo que la hace aplicable incluso cuando no hay un arreglo explícito de por medio (por ejemplo, sobre un rango de índices o valores abstractos). Es la base conceptual de `lower_bound` y del patrón "Binary Search on Answer" que se ve más adelante.

Casos de uso:

- Implementar `lower_bound` manualmente: menor índice tal que `arr[índice] >= valor`.
- Encontrar el primer elemento que supera un umbral en una secuencia ordenada.
- Encontrar el primer día, en una serie temporal ordenada, en que se cumple una condición acumulativa (por ejemplo, el primer día en que el saldo acumulado es no negativo).
- Como bloque de construcción para "Binary Search on Answer", donde $P(x)$ es una condición de factibilidad en vez de una comparación directa de arreglo.

Complejidad: $O(\log n \cdot C)$, donde n es el tamaño del espacio de búsqueda y C es el costo de evaluar el predicado P una vez; el factor $\log n$ viene de que el espacio de búsqueda se reduce a la mitad en cada iteración, igual

que en la búsqueda binaria clásica. Si P es $O(1)$ (como una simple comparación de arreglo), la complejidad total es $O(\log n)$.

```

1 struct MenorQueCumple {
2     vector<int> arr;
3
4     // CREATE: guarda una copia del arreglo, que debe estar
5     // ordenado
6     // de forma ascendente para que el predicado por defecto sea
7     // monotono
8     MenorQueCumple(vector<int> datos) : arr(datos) {} // O(n)
9
10    // READ: retorna el menor indice tal que arr[indice] >= valor;
11
12    // si ningun elemento cumple, retorna arr.size()
13    int menorQueCumple(int valor) {
14        int izq = 0, der = (int)arr.size(); // der fuera de rango =
15        // "no encontrado"
16        while (izq < der) {
17            int medio = izq + (der - izq) / 2;
18            if (arr[medio] >= valor) der = medio; // candidato valido
19            // seguir a la izquierda
20            else izq = medio + 1; // no cumple,
21            // buscar a la derecha
22        }
23        return izq;
24    } // O(log n)
25
26    //
27    // -----
28    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29    // -----
30    // 1. Predicado generico en vez de comparacion de arreglo: se
31    // puede reemplazar la condicion "arr[medio] >= valor" por
32    // cualquier funcion booleana P(medio) monotona, sin
33    // necesidad de que exista un arreglo de por medio (asi es
34    // como funciona "Binary Search on Answer").
35    //
36    // 2. Equivalencia con std::lower_bound: esta funcion es
37    // equivalente a
38    // (int)(lower_bound(arr.begin(), arr.end(), valor) - arr.
39    // begin()),
40    // implementada manualmente para mostrar la mecanica
41    // interna.
42    //
43    // 3. Cuidado con el rango inicial: a diferencia de la
44    // busqueda
45    // binaria clasica (izq <= der con der = size()-1), aqui
46    // el
47    // rango es [izq, der) semiabierto con der = size(), lo
48    // que
49    // evita casos especiales cuando la respuesta es "no
50    // existe
51    // ningun indice que cumpla" (se retorna directamente size
52    // ()).
53 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {1, 3, 3, 5, 7, 9};
2 MenorQueCumple mqc(datos);
3
4 int idx1 = mqc.menorQueCumple(5); // idx1 = 3 (arr[3] = 5,
5 // primer >= 5)
6 int idx2 = mqc.menorQueCumple(4); // idx2 = 3 (arr[3] = 5,
7 // primer >= 4)
8 int idx3 = mqc.menorQueCumple(10); // idx3 = 6 (ningun elemento
9 // >= 10, retorna size())

```

Regla práctica: Usa este patrón cuando la condición que buscas tenga la forma "falso...falso, verdadero...verdadero" a lo largo del espacio de búsqueda y quieras el primer punto de quiebre; si en cambio el patrón es "verdadero...verdadero, falso...falso" y buscas el último punto donde se cumple, usa la variante de "mayor x que cumple" vista a continuación.

27.3 Mayor x que cumple

Idea: Esta variante es la simétrica de "menor x que cumple": busca el mayor índice (o valor) x para el cual un predicado booleano $P(x)$ es verdadero, bajo el supuesto de que P es monótono en el sentido opuesto al anterior:

una vez que $P(x)$ se vuelve falso, permanece falso para todo valor mayor (patrón VVVV...VFFFF...F). En cada paso se evalúa P en el punto medio: si $P(\text{medio})$ es verdadero, el medio es un candidato válido pero podría no ser el máximo, así que se guarda como respuesta tentativa y se sigue buscando a la derecha; si es falso, la respuesta debe estar estrictamente a la izquierda.

¿Por qué usarlo? Es el complemento natural de "menor x que cumple": mientras esa variante encuentra el primer punto de quiebre de falso a verdadero, esta encuentra el último punto donde el predicado sigue siendo verdadero antes de quebrar a falso. Muchos problemas de "Binary Search on Answer" (visto a continuación) piden maximizar un valor factible en vez de minimizarlo, y ese es exactamente el patrón que resuelve esta variante: por ejemplo, "¿cuál es la mayor velocidad a la que puedo ir y aún así llegar a tiempo?" en vez de "¿cuál es el menor tiempo necesario?".

Casos de uso:

- Implementar el equivalente a `upper_bound - 1`: último índice tal que `arr[indice] <= valor`.
- Maximizar un recurso (velocidad, capacidad, tamaño) sujeto a una restricción de factibilidad monótona decreciente.
- Encontrar el último día en una serie temporal ordenada en que una condición acumulativa todavía se cumple, antes de dejar de cumplirse permanentemente.
- Como bloque de construcción para variantes de "Binary Search on Answer" que maximizan en vez de minimizar.

Complejidad: $O(\log n \cdot C)$, con n el tamaño del espacio de búsqueda y C el costo de evaluar P una vez; igual que en "menor x que cumple", el factor $\log n$ viene de reducir el espacio de búsqueda a la mitad en cada iteración. Con P de costo $O(1)$, la complejidad total es $O(\log n)$.

```

1 struct MayorQueCumple {
2     vector<int> arr;
3
4     // CREATE: guarda una copia del arreglo, que debe estar
5     // de forma ascendente para que el predicado por defecto sea
6     // monotono
7     MayorQueCumple(vector<int> datos) : arr(datos) {} // O(n)
8
9     // READ: retorna el mayor indice tal que arr[indice] <= valor;
10
11     // si ningun elemento cumple, retorna -1
12     int mayorQueCumple(int valor) {
13         int izq = -1, der = (int)arr.size() - 1; // izq fuera de
14         // rango = "no encontrado"
15         while (izq < der) {
16             int medio = izq + (der - izq + 1) / 2; // redondeo hacia
17             // arriba
18             if (arr[medio] <= valor) izq = medio; // candidato
19             // valido, seguir a la derecha
20             else der = medio - 1; // no cumple,
21             // buscar a la izquierda
22         }
23         return izq;
24     } // O(log n)
25
26     //
27     // -----
28     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29     // -----
30
31     // 1. Redondeo hacia arriba en el medio: a diferencia de "
32     // menor

```

```

25 // x que cumple", aqui el medio se calcula redondeando
26 // hacia
27 // arriba (+1 antes de dividir); si se redondeara hacia
28 // abajo
29 // como en la variante anterior, el bucle podria quedar
30 // atascado sin avanzar cuando der = izq + 1.
31 //
32 // 2. Equivalencia con operaciones estandar: esta funcion es
33 // equivalente a
34 // (int)(upper_bound(arr.begin(), arr.end(), valor) - arr.
35 // begin()) - 1,
36 // implementada manualmente para mostrar la mecanica
37 // interna.
38 //
39 // 3. Rango inicial con izq = -1: se usa -1 en vez de 0 como
40 // limite inferior semabierto, de forma simetrica a como
41 // "menor x que cumple" usa der = size() como limite
42 // superior
43 // semabierto; esto permite representar "no existe ningun
44 // indice que cumpla" sin casos especiales.
45 };

```

Ejemplo de funcionamiento:

```

1 vector<int> datos = {1, 3, 3, 5, 7, 9};
2 MayorQueCumple mqc(datos);
3
4 int idx1 = mqc.mayorQueCumple(5); // idx1 = 3 (arr[3] = 5,
5 // ultimo <= 5)
6 int idx2 = mqc.mayorQueCumple(4); // idx2 = 2 (arr[2] = 3,
7 // ultimo <= 4)
8 int idx3 = mqc.mayorQueCumple(0); // idx3 = -1 (ningun
9 // elemento <= 0)

```

Regla práctica: Usa este patrón cuando el predicado tenga la forma "verdadero...verdadero, falso...falso" y quieras el último punto donde se cumple; si el patrón está invertido (falso primero, verdadero después), usa "menor x que cumple". Nota que el cálculo del punto medio se redondea hacia arriba aquí y hacia abajo en la variante anterior — invertir esta convención por descuido produce bucles infinitos.

27.4 Binary Search on Answer

Idea: "Binary Search on Answer" no busca una posición dentro de un arreglo, sino que aplica la misma mecánica de reducción a la mitad sobre el *espacio de posibles respuestas* de un problema de optimización. La clave es reformular una pregunta de optimización ("¿cuál es el mínimo/máximo valor de x que resuelve el problema?") como una pregunta de *factibilidad* ("¿es posible lograr el objetivo con un valor x dado?"), siempre que esa factibilidad sea monótona en x : si x funciona, todo valor mayor (o menor, según el problema) también funciona. Una vez identificada esa monotonía, se aplica exactamente el patrón de "menor/mayor x que cumple" ya visto, pero con $P(x)$ = "es factible resolver el problema con el parámetro x " en vez de una comparación directa contra un arreglo.

¿Por qué usarlo? Muchos problemas de optimización tienen una función objetivo que no se puede derivar ni evaluar analíticamente de forma directa, pero sí se puede verificar fácilmente si un valor candidato es factible (con una función auxiliar, a menudo greedy o de simulación). Esta técnica convierte un problema de optimización potencialmente difícil en $O(\log(\text{rango de respuestas}) \cdot C)$ llamadas a una función de factibilidad de costo C , en vez de probar todos los valores posibles de x uno por uno. Se distingue de las variantes anteriores de búsqueda binaria en que aquí $P(x)$ no compara contra un arreglo dado, sino que típicamente ejecuta un algoritmo completo (a veces $O(n)$ o más) para determinar factibilidad; por eso elegir bien el rango inicial de búsqueda y minimizar el costo de P es crítico para la eficiencia total.

Casos de uso:

- Minimizar el máximo (o maximizar el mínimo) en problemas de asignación o partición, como repartir paquetes entre camiones con capacidad mínima necesaria.
- Encontrar la velocidad mínima necesaria para completar una tarea antes de un plazo (problemas tipo "Koko Eating Bananas").
- Determinar la distancia mínima de separación máxima posible entre elementos colocados en posiciones dadas (problemas tipo "Aggressive Cows").
- Optimizar un parámetro continuo o discreto sujeto a una restricción de recursos, cuando la factibilidad se puede evaluar con un algoritmo greedy en $O(n)$.
- Problemas donde la respuesta exacta es difícil de calcular directamente pero es fácil verificar "¿el valor x es suficiente o demasiado?".

Complejidad: $O(\log(\text{rango}) \cdot C)$, donde rango es la diferencia entre el máximo y mínimo valor posible de la respuesta, y C es el costo de la función de factibilidad. El factor $\log(\text{rango})$ sale de aplicar el mismo argumento de reducción a la mitad que en las búsquedas binarias anteriores, pero ahora sobre el espacio de valores de la respuesta en vez de índices de un arreglo; como C suele ser $O(n)$ (una pasada greedy sobre los datos), la complejidad total típica es $O(n \log(\text{rango}))$.

```

1 struct BinarySearchOnAnswer {
2     vector<int> pesos; // ejemplo: pesos de paquetes a repartir
3     // en camiones
4     int numCamiones;
5
6     // CREATE: guarda los pesos de los paquetes (en orden, sin
7     // reordenar) y el numero de camiones disponibles
8     BinarySearchOnAnswer(vector<int> p, int k) : pesos(p),
9         numCamiones(k) {} // O(n)
10
11     // READ: verifica si es factible repartir todos los paquetes
12     // en
13     // "numCamiones" camiones, si cada camion tiene capacidad
14     // maxima
15     // "capacidad" (greedy: llenar cada camion hasta que un
16     // paquete
17     // no quepa, entonces pasar al siguiente camion)
18     bool esFactible(int capacidad) {
19         int camionesUsados = 1;
20         int cargaActual = 0;
21         for (int peso : pesos) {
22             if (peso > capacidad) return false; // un paquete no cabe
23             // en ningun camion
24             if (cargaActual + peso > capacidad) {
25                 camionesUsados++;
26                 cargaActual = 0;
27             }
28             cargaActual += peso;
29         }
30         return camionesUsados <= numCamiones;
31     } // O(n)
32
33     // READ: encuentra la menor capacidad de camion tal que sea
34     // factible repartir todos los paquetes en "numCamiones"
35     // camiones
36     int capacidadMinima() {
37         int izq = *max_element(pesos.begin(), pesos.end()); // no
38         // puede ser menor que el paquete mas pesado
39         int der = accumulate(pesos.begin(), pesos.end(), 0); //
40         // caso trivial: un solo camion con todo
41         while (izq < der) {
42             int medio = izq + (der - izq) / 2;
43             if (esFactible(medio)) der = medio; // candidato valido,
44             // buscar capacidad menor
45             else izq = medio + 1; // no factible,
46             // necesita mas capacidad
47         }
48         return izq;
49     }
50 }

```

```

38 } // O(n log(suma - max))
39
40 //
41 // -----
42 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43 // -----
44 // 1. Patron general: esta estructura es una instancia
45 // concreta
46 // de "menor x que cumple" (visto anteriormente), donde
47 // P(x) = esFactible(x) en vez de una comparacion de
48 // arreglo;
49 // el mismo esqueleto de bucle [izq, der) semiabierto
50 // aplica
51 // sin cambios.
52 //
53 // 2. Problemas de maximizacion en vez de minimizacion: si el
54 // problema pide maximizar un parametro (por ejemplo, la
55 // velocidad minima para NO llegar tarde, buscando la
56 // MAYOR
57 // separacion posible entre vacas en un establo), se usa
58 // el
59 // esqueleto de "mayor x que cumple" en vez de este,
60 // invirtiendo
61 // la logica de esFactible segun corresponda.
62 //
63 // 3. Eleccion del rango inicial [izq, der]: es crucial
64 // acotar
65 // bien el rango de busqueda; un rango demasiado amplio no
66 // afecta la complejidad asintotica (sigue siendo log del
67 // rango) pero un rango incorrecto (que excluya la
68 // respuesta
69 // real) produce un resultado erroneo. Aqui izq = max(
70 // pesos)
71 // porque ningun camion puede tener capacidad menor al
72 // paquete
73 // mas pesado, y der = suma total porque un solo camion
74 // con
75 // esa capacidad siempre es factible.
76 //
77 // 4. Costo de esFactible domina la complejidad total: si la
78 // funcion de factibilidad fuera O(n log n) en vez de O(n)
79 // (por ejemplo, si requiriera ordenar en cada llamada),
80 // la
81 // complejidad total subiria a O(n log n log(rango)); por
82 // eso
83 // conviene precalcular fuera del bucle binario todo lo
84 // que
85 // no dependa del valor candidato x.
86 };

```

Ejemplo de funcionamiento:

```

1 vector<int> pesos = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
2 int camiones = 5;
3 BinarySearchOnAnswer bsoa(pesos, camiones);
4
5 int capMin = bsoa.capacidadMinima();
6 // Nota: el valor exacto de capMin depende de la simulacion
7 // greedy
8 // paso a paso; se debe correr el codigo para confirmarlo antes
9 // de
10 // darlo por verificado (candidato esperado en el rango de 15 a
11 // 18,
12 // pero verificar con la ejecucion real).

```

Regla práctica: Reconoce un problema de "Binary Search on Answer" cuando puedas responder rápidamente "¿es factible con este valor?" mediante un greedy o simulación, y esa factibilidad sea monótona en el parámetro buscado; en ese caso, aplica el esqueleto de "menor x que cumple" si minimizas, o "mayor x que cumple" si maximizas, usando la función de factibilidad como predicado $P(x)$ en vez de una comparación directa de arreglo.

27.5 Sobre reales

Idea: Cuando el espacio de búsqueda no es un conjunto discreto de enteros sino un intervalo continuo de números reales (por ejemplo, encontrar la raíz de una función monótona con precisión decimal), la búsqueda binaria no puede terminar cuando $\text{izq} == \text{der}$ porque entre dos reales cualesquiera siempre hay infinitos valores intermedios. En su lugar, se itera un número fijo de

veces (o hasta que el intervalo $[izq, der]$ sea más pequeño que una tolerancia ε dada), y en cada iteración se sigue partiendo el intervalo a la mitad según el mismo criterio de monotonía usado en las variantes discretas.

¿Por qué usarlo? Es la adaptación directa de "menor/mayor x que cumple" al dominio continuo: la lógica de descartar la mitad del espacio según un predicado monótono $P(x)$ es idéntica, solo cambia la condición de parada, porque no existe un "siguiente entero" al cual converger. Se usa en vez de métodos como Newton-Raphson cuando no se puede (o no conviene) calcular la derivada de la función, o cuando se necesita una garantía simple de convergencia sin preocuparse por casos donde Newton-Raphson diverge (tangentes horizontales, puntos de inflexión mal condicionados); a cambio, la convergencia es más lenta (lineal en el número de dígitos de precisión, en vez de cuadrática).

Casos de uso:

- Encontrar la raíz cuadrada (o raíz n -ésima) de un número con precisión decimal arbitraria.
- Encontrar el punto donde una función monótona continua cruza un valor objetivo (por ejemplo, resolver $f(x) = 0$).
- Problemas de "Binary Search on Answer" donde la respuesta es un valor real (por ejemplo, minimizar un radio, una velocidad o un tiempo continuo) en vez de un entero.
- Calibrar un parámetro continuo de un modelo o simulación hasta alcanzar una precisión deseada.

Complejidad: $O(\log(\text{rango}/\varepsilon) \cdot C)$, ya que cada iteración reduce el tamaño del intervalo a la mitad, y se necesitan $\log_2(\text{rango}/\varepsilon)$ iteraciones para que el intervalo sea menor que la tolerancia ε deseada; C es el costo de evaluar el predicado o la función en cada paso. En la práctica, con un número fijo de iteraciones (por ejemplo, 100), se alcanza precisión cercana al límite de `double` sin necesidad de calcular explícitamente $\log(\text{rango}/\varepsilon)$.

```

1 struct BusquedaBinariaReal {
2     double izq, der;
3     int iteraciones;
4
5     // CREATE: define el intervalo inicial [izq, der] donde se
6     // garantiza que existe la respuesta, y el numero de
7     // iteraciones
8     // a realizar (100 es suficiente para precision de double)
9     BusquedaBinariaReal(double i, double d, int iters = 100)
10     : izq(i), der(d), iteraciones(iters) {} // O(1)
11
12     // READ: retorna el menor valor real x en [izq, der] tal que
13     // el
14     // predicado P(x) es verdadero, asumiendo que P tiene el
15     // patron
16     // falso...falso, verdadero...verdadero
17     template<typename Pred>
18     double menorQueCumple(Pred P) {
19         double lo = izq, hi = der;
20         for (int it = 0; it < iteraciones; it++) {
21             double medio = lo + (hi - lo) / 2.0;
22             if (P(medio)) hi = medio; // candidato valido, seguir a
23             // la izquierda
24             else lo = medio; // no cumple, buscar a la
25             // derecha
26         }
27         return hi;
28     } // O(iteraciones * C)
29
30     //
31     // -----
32     // MODIFICACIONES Y COMENTARIOS CON LOS USOS

```

```

27 // -----
28 // 1. Numero de iteraciones vs tolerancia explicita: en vez
29 // de un
30 // numero fijo de iteraciones, se puede usar una condicion
31 // "while (hi - lo > eps)" con un eps dado (ej. 1e-9);
32 // ambas
33 // formas son equivalentes, pero el numero fijo de
34 // iteraciones
35 // evita bucles infinitos por errores de redondeo cuando
36 // eps
37 // es demasiado pequeno para la precision de double.
38 //
39 // 2. Mayor x que cumple (patron invertido, verdadero...falso
40 // ):
41 // misma estructura pero invirtiendo la asignacion:
42 // if (P(medio)) lo = medio; else hi = medio;
43 // y se retorna lo en vez de hi.
44 //
45 // 3. Ejemplo de uso para raiz cuadrada: para calcular sqrt(n
46 // ),
47 // se define P(x) := (x * x >= n) sobre el intervalo [0, n
48 // +1],
49 // y menorQueCumple(P) retorna la raiz cuadrada aproximada.
50 //
51 // 4. Precision de punto flotante: 100 iteraciones sobre un
52 // intervalo de tamaño razonable (ej. [0, 1e9]) ya reducen
53 // el
54 // intervalo muy por debajo de la precision representable
55 // en
56 // double (~15-17 digitos decimales), por lo que iterar
57 // mas de
58 // ~100 veces no mejora la precision real, solo
59 // desperdicia
60 // tiempo de ejecucion.
61 };

```

Ejemplo de funcionamiento:

```

1 // Calcular sqrt(2) con busqueda binaria sobre reales
2 BusquedaBinariaReal bbr(0.0, 2.0);
3 double raiz = bbr.menorQueCumple([](double x) {
4     return x * x >= 2.0;
5 });
6 // raiz es aproximadamente 1.41421356... (sqrt(2))

```

Regla práctica: Usa esta variante cuando el espacio de búsqueda sea continuo (reales) en vez de discreto (enteros), y reemplaza la condición de parada $izq < der$ por un número fijo de iteraciones o una tolerancia ε explícita; si el espacio de búsqueda es discreto, siempre preferir las variantes enteras vistas antes, ya que convergen en tiempo finito exacto sin depender de una tolerancia arbitraria.

27.6 Con strings

Idea: La búsqueda binaria también puede aplicarse cuando el espacio de búsqueda o el criterio de comparación involucra strings en vez de números, siempre que exista un orden total monótono adecuado (por ejemplo, el orden lexicográfico) o una propiedad monótona definida sobre los strings (por ejemplo, "la longitud del prefijo común es al menos k "). La mecánica de partir el espacio a la mitad es idéntica a las variantes numéricas; lo que cambia es cómo se define y evalúa el predicado P sobre strings, que normalmente requiere una comparación lexicográfica o un cálculo de prefijo/sufijo en vez de una comparación aritmética directa.

¿Por qué usarlo? Muchos problemas piden buscar sobre un arreglo de strings ordenado lexicográficamente (por ejemplo, encontrar el primer string que empieza con cierto prefijo, un problema análogo a "menor x que cumple" pero con `std::string` en vez de `int`), o piden encontrar la longitud óptima de un substring/prefijo común que cumple alguna propiedad monótona (más cercano a "Binary Search on Answer", donde x es una

longitud entera pero $P(x)$ evalúa algo sobre substrings de esa longitud, como en la búsqueda del prefijo común más largo entre dos strings mediante hashing).

Casos de uso:

- Encontrar el rango de strings en un diccionario ordenado que comparten un prefijo dado (equivalente a `lower_bound/upper_bound` con comparación lexicográfica).
- Encontrar la longitud del prefijo común más largo entre dos strings usando *binary search + hashing*: para una longitud candidata L , verificar en $O(1)$ (con hashing precomputado) si los primeros L caracteres coinciden.
- Buscar el substring repetido más largo en un string, combinando binary search sobre la longitud con un *Suffix Array* o hashing para verificar factibilidad.
- Ordenar un diccionario de palabras y realizar búsquedas de existencia o de prefijo de forma repetida sobre él.

Complejidad: $O(\log n \cdot C)$ para búsqueda directa sobre un arreglo ordenado de m strings, donde $C = O(L)$ es el costo de comparar dos strings de longitud L (peor caso, cuando comparten un prefijo largo), dando $O(L \log n)$ en total. En el caso de "longitud del prefijo común más largo con hashing", la complejidad es $O(\log L \cdot C)$, donde L es la longitud máxima posible del prefijo y $C = O(1)$ es el costo de comparar hashes de un substring ya precomputados, dando $O(\log L)$ en total tras el preprocesamiento de hashes.

```

1 struct BusquedaBinariaStrings {
2     vector<string> diccionario; // ordenado lexicograficamente
3     vector<unsigned long long> hashA, hashB; // prefijos de hash
4     vector<unsigned long long> potencias; // potencias de la
5     unsigned long long base = 131, mod = 1e9 + 7;
6
7     // CREATE: guarda el diccionario ordenado (para búsqueda por
8     // prefijo) y precalcula los hashes de prefijo de dos strings
9     // (para longitud de prefijo comun mas largo)
10    BusquedaBinariaStrings(vector<string> dic, string a, string b) : diccionario(dic) {
11        int n = max(a.size(), b.size());
12        potencias.assign(n + 1, 1);
13        for (int i = 1; i <= n; i++) potencias[i] = (potencias[i-1]
14            * base) % mod;
15        hashA = calcularHashes(a);
16        hashB = calcularHashes(b);
17    } // O(m log m) por el diccionario ya ordenado + O(n) por el
18    hashing
19
20    // READ: calcula el arreglo de hashes de prefijo de un string
21    // s,
22    // donde hashes[i] es el hash de s[0..i-1]
23    vector<unsigned long long> calcularHashes(const string& s) {
24        vector<unsigned long long> h(s.size() + 1, 0);
25        for (int i = 0; i < (int)s.size(); i++)
26            h[i+1] = (h[i] * base + s[i]) % mod;
27        return h;
28    } // O(|s|)
29
30    // READ: retorna el hash del substring s[0..len-1] usando el
31    // arreglo de hashes de prefijo precalculado h
32    unsigned long long hashPrefijo(const vector<unsigned long
33        long>& h, int len) {
34        return h[len];
35    } // O(1)
36
37    // READ: retorna el menor indice en "diccionario" tal que la
38    // palabra en esa posicion es >= "prefijo" lexicograficamente
39    // (equivalente a lower_bound sobre strings)
40    int menorQueCumplePrefijo(const string& prefijo) {
41        int izq = 0, der = (int)diccionario.size();

```

```

38 while (izq < der) {
39     int medio = izq + (der - izq) / 2;
40     if (diccionario[medio] >= prefijo) der = medio;
41     else izq = medio + 1;
42 }
43 return izq;
44 } // O(L log m), L = longitud promedio de comparacion, m =
45     tamaño del diccionario
46
47 // READ: retorna la longitud del prefijo comun mas largo
48 // entre
49 // los dos strings usados al construir el struct, usando
50 // binary
51 // search sobre la longitud y comparacion de hashes en O(1)
52 int prefijoComunMasLargo() {
53     int izq = 0, der = (int)min(hashA.size(), hashB.size()) -
54     1;
55     while (izq < der) {
56         int medio = izq + (der - izq + 1) / 2;
57         if (hashPrefijo(hashA, medio) == hashPrefijo(hashB, medio))
58             izq = medio;
59         else der = medio - 1;
60     }
61     return izq;
62 } // O(log L)
63
64 //
65
66 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
67 //
68 // 1. Riesgo de colisiones de hash: usar un solo modulo (como
69 // aquí, con fines ilustrativos) tiene riesgo de
70 // colisiones
71 // ante datos adversarios; en competencias es preferible
72 // usar
73 // doble hashing (dos bases y modulos distintos) o hashing
74 // aleatorizado para reducir ese riesgo.
75 //
76 // 2. Comparacion de strings sin hashing: si no se quiere
77 // lidiar
78 // con hashing, "prefijoComunMasLargo" tambien puede
79 // resolverse
80 // directamente comparando caracter a caracter en O(L)
81 // total,
82 // sin busqueda binaria, ya que de todas formas hay que
83 // leer
84 // los caracteres; el hashing solo aporta ventaja cuando
85 // se
86 // necesita comparar substrings arbitrarios repetidamente,
87 // no
88 // solo un prefijo desde el inicio.
89 //
90 // 3. Busqueda binaria sobre diccionario no ordenado: si
91 // "diccionario" no esta ordenado lexicograficamente, hay
92 // que
93 // ordenarlo primero con sort(diccionario.begin(),
94 // diccionario.end())
95 // antes de poder usar "menorQueCumplePrefijo"
96 // correctamente.
97 };

```

Ejemplo de funcionamiento:

```

1 vector<string> dic = {"casa", "carro", "perro", "pelota", "
2     zorro"};
3 sort(dic.begin(), dic.end()); // {"carro", "casa", "pelota", "
4     perro", "zorro"}
5
6 BusquedaBinariaStrings bbs(dic, "internacionalización", "
7     internacionalmente");
8
9 int idx = bbs.menorQueCumplePrefijo("pe");
10 // idx = 2 ("pelota" es la primera palabra >= "pe"
11 // lexicograficamente)
12
13 int lcp = bbs.prefijoComunMasLargo();
14 // Nota: el valor exacto de lcp depende de la comparacion
15 // caracter a
16 // caracter entre "internacionalización" e "internacionalmente";
17 // se
18 // debe correr el codigo para confirmar el valor exacto (
19 // candidato
20 // esperado alrededor de 14-15, ya que ambos comparten el
21 // prefijo
22 // "internacional").

```

Regla práctica: Usa búsqueda binaria sobre strings cuando el arreglo esté ordenado lexicográficamente y necesites encontrar un rango de coincidencia por prefijo (aplica directamente "menor/mayor x que cumple" con

`string` en vez de `int`); usa binary search + hashing sobre la *longitud* cuando necesites encontrar el tamaño óptimo de un prefijo, sufijo o substring que cumple una propiedad de igualdad o coincidencia, ya que ahí el predicado monótono no es sobre el string mismo sino sobre una longitud entera.

Parte V Matemáticas

28 Matemáticas (Math)

Contenido en desarrollo.

29 Teoría de Números (Number Theory)

Contenido en desarrollo.

Contenido en desarrollo.

Parte VI Geometría Computacional

30 Line Sweep

30.1 Idea General y Manejo de Eventos

Idea: un *sweep line* (barrido) resuelve problemas geométricos imaginando una línea recta (normalmente vertical, avanzando en x) que se desplaza por el plano de izquierda a derecha. En vez de razonar sobre el plano completo, la línea reduce el problema a una secuencia de **eventos discretos** —puntos donde algo cambia: empieza un intervalo, termina un segmento, aparece un punto— ordenados por la coordenada de barrido. Entre un evento y el siguiente, el "estado" del problema (qué objetos toca la línea, en qué orden) no cambia, así que basta con mantener ese estado en una estructura auxiliar (el *status*) y actualizarla incrementalmente en cada evento. Esto convierte un problema 2D en una secuencia de operaciones 1D sobre esa estructura.

¿Por qué usarlo? Cuando un problema involucra n objetos y una propiedad global entre ellos (intersecciones, solapamientos, cercanía, área cubierta), la fuerza bruta suele ser $O(n^2)$ al comparar todos los pares. El barrido explota el hecho de que solo los objetos *actualmente cerca* de la línea pueden interactuar entre sí en ese instante, así que basta con mantener y consultar esa vecindad, no todos los pares. Esto es distinto de una estructura estática (como un *prefix sum* o un *sparse table*): el barrido mantiene una estructura **dinámica** que se inserta/elimina a medida que se consumen eventos, generalmente respaldada por un `std::set`, un árbol de segmentos o un *Fenwick tree*.

Casos de uso:

- Detectar si algún par de segmentos se intersecta.
- Calcular el área cubierta por la unión de rectángulos.
- Resolver el *Skyline Problem*.

- Encontrar el par de puntos más cercano en el plano.
- Contar el máximo número de intervalos activos simultáneamente.

Complejidad: en general $O(n \log n)$: ordenar los $2n$ eventos cuesta $O(n \log n)$, y cada evento realiza una operación de $O(\log n)$ sobre la estructura de estado (inserción, borrado o consulta en un `set` o árbol balanceado). El total es $O(n \log n)$, dominado por el ordenamiento inicial cuando la estructura de estado es logarítmica por operación.

```

1 // CREATE/READ/WRITE: Barrido generico basado en eventos
  ordenados
2 // Ejemplo de aplicacion: maximo numero de intervalos solapados
  simultaneamente
3 struct BarridoEventos {
4 // Evento: posicion x y tipo (+1 = inicio de intervalo, -1 =
  fin de intervalo)
5 struct Evento {
6 long long x;
7 int tipo; // +1 = abre intervalo, -1 = cierra intervalo
8 };
9
10 vector<Evento> eventos;
11
12 // CREATE: reserva espacio para 2*n eventos (inicio y fin de
  cada intervalo)
13 BarridoEventos(int n) {
14 eventos.reserve(2 * n);
15 } // O(1)
16
17 // WRITE: agrega los dos eventos (inicio, fin) generados por
  un intervalo [l, r]
18 void agregarIntervalo(long long l, long long r) {
19 eventos.push_back({l, +1});
20 eventos.push_back({r, -1});
21 } // O(1) amortizado
22
23 // READ: ordena los eventos y barre la recta devolviendo el
  maximo de
24 // intervalos activos simultaneamente
25 int maximoSolapamiento() {
26 sort(eventos.begin(), eventos.end(), [](const Evento& a,
  const Evento& b) {
27 if (a.x != b.x) return a.x < b.x;
28 // Si un intervalo es [l, r) (cerrado-abierto), los
  cierres (-1)
29 // deben procesarse ANTES que las aperturas (+1) en el
  mismo punto
30 return a.tipo < b.tipo;
31 });
32
33 int activos = 0, maximo = 0;
34 for (auto& e : eventos) {
35 activos += e.tipo;
36 maximo = max(maximo, activos);
37 }
38 return maximo;
39 } // O(n log n), dominado por el sort de 2n eventos
40
41 //
42 // -----
43 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
44 // -----
45 // 1. Intervalos cerrados [l, r] (incluyen ambos extremos):
  si dos
46 // intervalos comparten solo un punto de borde y ESO
  cuenta como
47 // solapamiento, invertir el criterio de desempate (tipo >
  0 antes
48 // que tipo < 0) para que las aperturas se procesen antes.
49 // 2. Barrido en 2D (rectangulos): en vez de activos += tipo,
  se usa
50 // un arbol de segmentos o BIT indexado por la coordenada
  y para
51 // saber cuanta longitud vertical esta cubierta en cada
  momento
52 // (ver Union de Areas de Rectangulos y Skyline Problem).
53 // 3. Conteo de eventos por tipo especifico: si se necesitan
  mas de
54 // dos tipos de evento (por ejemplo, consultas
  intercaladas con
55 // actualizaciones), agregar un campo extra a Evento y
  desempatar
56 // por prioridad de tipo, no solo por signo.
  };

```

Ejemplo de funcionamiento:

```

1 BarridoEventos b(3);
2 b.agregarIntervalo(1, 5);
3 b.agregarIntervalo(2, 6);
4 b.agregarIntervalo(4, 8);
5
6 int resultado = b.maximoSolapamiento();
7 // resultado = 3
8 // (en x = 4, los tres intervalos [1,5], [2,6] y [4,8] estan
   activos)

```

Nota: el orden de desempate cuando dos eventos comparten la misma coordenada es la fuente más común de errores en un barrido. No hay una regla universal—depende de si los intervalos del problema son abiertos, cerrados, o si "tocarse en el borde" cuenta como interacción— así que siempre hay que decidirlo explícitamente en el comparador, como se ilustra en el código.

Regla práctica: usa un barrido genérico cuando el problema se reduzca a procesar eventos ordenados manteniendo un estado que se actualiza incrementalmente. Si solo necesitas respuestas estáticas sobre datos que no cambian con la posición (consultas offline sin relación de orden entre ellas), probablemente te alcance con estructuras estáticas más simples en vez de simular el barrido completo.

30.2 Intersección de Segmentos

Idea: es la aplicación más directa del barrido general—visto en *Idea General y Manejo de Eventos*— al problema de detectar si n segmentos tienen alguna intersección entre sí. El *status* de la línea es el conjunto de segmentos que la línea de barrido cruza actualmente, ordenados por su altura (y) en la posición x actual del barrido. La observación clave es que dos segmentos solo pueden empezar a cruzarse cuando se vuelven **vecinos** en ese orden vertical; por lo tanto, basta con comprobar intersección contra los vecinos inmediatos cada vez que un segmento entra o sale del status, sin comparar contra todos los demás.

¿Por qué usarlo? Comparar todos los pares de segmentos cuesta $O(n^2)$. El barrido lo reduce a $O(n \log n)$ porque, en cualquier instante, solo hace falta conocer el orden vertical local de los segmentos activos—mantenido en un `std::set`— en vez de su relación con todos los demás. A diferencia del barrido genérico de la subsección anterior (que solo cuenta activos), aquí el status necesita un **orden relativo dinámico**: la posición de un segmento en el `set` cambia a medida que x avanza, porque la altura de cada segmento en x depende de su pendiente.

Casos de uso:

- Verificar si un conjunto de calles o límites de un mapa se cruzan indebidamente.
- Validar que un polígono es simple (sus aristas no se autointersectan).
- Preprocesamiento antes de planarizar un grafo geométrico (arreglo de segmentos).
- Detección de colisiones entre trayectorias rectas.

Complejidad: $O(n \log n)$. Hay $2n$ eventos (inicio y fin de cada segmento), ordenarlos cuesta $O(n \log n)$.

Cada evento realiza a lo sumo una inserción, un borrado o una consulta de vecinos en el `std::set`, cada una $O(\log n)$, y cada verificación de intersección entre dos segmentos es $O(1)$. En total: $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Barrido para detectar si existe al menos
   una
2 // interseccion entre n segmentos (no cuenta todas las
   intersecciones)
3 struct BarridoInterseccionSegmentos {
4     struct Punto { double x, y; };
5     struct Segmento {
6         Punto a, b; // a.x <= b.x siempre (normalizado al agregar)
7         int id;
8     };
9
10    // READ: producto cruz (o - p) x (o - q), usado para
        orientacion
11    static double cruz(const Punto& o, const Punto& p, const
        Punto& q) {
12        return (p.x - o.x) * (q.y - o.y) - (p.y - o.y) * (q.x - o.x);
13    };
14    // // O(1)
15
16    // READ: determina si los segmentos s1 y s2 se cruzan (
        incluye casos
17    // colineales donde un extremo cae exactamente sobre el otro
        segmento)
18    static bool seCruzan(const Segmento& s1, const Segmento& s2)
        {
19        double d1 = cruz(s1.a, s1.b, s2.a);
20        double d2 = cruz(s1.a, s1.b, s2.b);
21        double d3 = cruz(s2.a, s2.b, s1.a);
22        double d4 = cruz(s2.a, s2.b, s1.b);
23        if (((d1 > 0 && d2 < 0) || (d1 < 0 && d2 > 0)) &&
24            ((d3 > 0 && d4 < 0) || (d3 < 0 && d4 > 0)))
25            return true;
26        auto sobreSegmento = [](const Segmento& s, const Punto& p)
        {
27            return min(s.a.x, s.b.x) <= p.x && p.x <= max(s.a.x, s.b.
28            x) &&
29            min(s.a.y, s.b.y) <= p.y && p.y <= max(s.a.y, s.b.y);
30        };
31        if (d1 == 0 && sobreSegmento(s1, s2.a)) return true;
32        if (d2 == 0 && sobreSegmento(s1, s2.b)) return true;
33        if (d3 == 0 && sobreSegmento(s2, s1.a)) return true;
34        if (d4 == 0 && sobreSegmento(s2, s1.b)) return true;
35        return false;
36    }; // O(1)
37
38    // READ: interpola la altura y del segmento s en la
        coordenada x
39    static double alturaEnX(const Segmento& s, double x) {
40        if (s.a.x == s.b.x) return min(s.a.y, s.b.y);
41        double t = (x - s.a.x) / (s.b.x - s.a.x);
42        return s.a.y + t * (s.b.y - s.a.y);
43    }; // O(1)
44
45    double sweepX; // coordenada x actual del barrido, usada por
        el comparador
46
47    // Comparador del status: ordena segmentos activos por su
        altura en sweepX
48    struct Comparador {
49        BarridoInterseccionSegmentos* padre;
50        bool operator()(const Segmento& s1, const Segmento& s2)
51        const {
52            double y1 = alturaEnX(s1, padre->sweepX);
53            double y2 = alturaEnX(s2, padre->sweepX);
54            if (fabs(y1 - y2) > 1e-9) return y1 < y2;
55            return s1.id < s2.id;
56        }
57    };
58
59    vector<Segmento> segmentos;
60
61    // CREATE: reserva espacio para n segmentos
62    BarridoInterseccionSegmentos(int n) {
63        segmentos.reserve(n);
64        sweepX = 0;
65    }; // O(1)
66
67    // WRITE: agrega un segmento (normaliza para que a.x <= b.x)
68    void agregarSegmento(Punto p, Punto q) {
69        if (q.x < p.x) swap(p, q);
70        segmentos.push_back({p, q, (int)segmentos.size()});
71    }; // O(1) amortizado
72
73    // READ: barre la recta de izquierda a derecha y responde si
        existe
74    // al menos un par de segmentos que se intersectan
75    bool existeInterseccion() {

```

```

73 enum TipoEvento { FIN = 0, INICIO = 1 }; // FIN antes que
74 INICIO en empate
75 struct Evento { double x; int tipo; int idSeg; };
76 vector<Evento> eventos;
77 eventos.reserve(2 * segmentos.size());
78 for (auto& s : segmentos) {
79     eventos.push_back({s.a.x, INICIO, s.id});
80     eventos.push_back({s.b.x, FIN, s.id});
81 }
82 sort(eventos.begin(), eventos.end(), [](const Evento& e1,
83 const Evento& e2) {
84     if (e1.x != e2.x) return e1.x < e2.x;
85     return e1.tipo < e2.tipo;
86 });
87
88 Comparador comp{this};
89 set<Segmento, Comparador> status(comp);
90 vector<set<Segmento, Comparador>::iterator> ubicacion(
91 segmentos.size());
92
93 for (auto& e : eventos) {
94     sweepX = e.x;
95     Segmento& s = segmentos[e.idSeg];
96     if (e.tipo == INICIO) {
97         auto it = status.insert(s).first;
98         ubicacion[e.idSeg] = it;
99         auto siguiente = next(it);
100         if (siguiente != status.end() && seCruzan(s, *siguiente))
101             return true;
102         if (it != status.begin()) {
103             auto anterior = prev(it);
104             if (seCruzan(s, *anterior)) return true;
105         }
106     } else {
107         auto it = ubicacion[e.idSeg];
108         auto siguiente = next(it);
109         bool hayAmbos = (it != status.begin() && siguiente !=
110 status.end());
111         if (hayAmbos) {
112             auto anterior = prev(it);
113             if (seCruzan(*anterior, *siguiente)) return true;
114         }
115         status.erase(it);
116     }
117 }
118 return false;
119 } // O(n log n): n eventos, cada operacion sobre el set es O(
120 log n)
121
122 //
123 // -----
124 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
125 // -----
126
127 // 1. Contar TODAS las intersecciones (no solo detectar si
128 // existe una):
129 // requiere el algoritmo completo de Bentley-Ottmann,
130 // donde cada
131 // punto de interseccion encontrado se inserta como un
132 // NUEVO evento
133 // en la cola, y se intercambian las posiciones de los dos
134 // segmentos
135 // en el status en ese punto. Complejidad O((n + k) log n)
136 // con
137 // k = numero de intersecciones.
138 // 2. Segmentos verticales (s.a.x == s.b.x): el comparador ya
139 // los
140 // maneja tomando el y minimo en alturaEnX, pero si dos
141 // segmentos
142 // verticales coinciden en x se debe agregar un criterio
143 // de
144 // desempate adicional (por ejemplo el y minimo de cada
145 // uno) para
146 // evitar que el set los considere iguales.
147 // 3. Precision numerica: al usar coordenadas double, se
148 // compara con
149 // un epsilon (aqui 1e-9) en vez de igualdad exacta; si
150 // las
151 // coordenadas de entrada son enteras, se puede reescribir
152 // todo
153 // con aritmetica de enteros/racionales para evitar
154 // errores.
155 };

```

Ejemplo de funcionamiento:

```

1 BarridoInterseccionSegmentos b(3);
2 b.agregarSegmento({0, 0}, {4, 4});
3 b.agregarSegmento({0, 4}, {4, 0});
4 b.agregarSegmento({5, 5}, {6, 6});
5
6 bool resultado = b.existeInterseccion();
7 // resultado = true

```

```

8 // (el primer y segundo segmento forman una X y se cruzan en
9 // (2,2);
10 // el tercero esta aislado y no interfiere)

```

Nota: este struct responde únicamente *existe/no existe* una intersección, en $O(n \log n)$. Si el problema pide reportar todos los pares que se cruzan (o los puntos exactos de cruce), hace falta el algoritmo completo de Bentley-Ottmann, con una cola de eventos que crece dinámicamente por cada intersección encontrada —no cubierto en este struct, ver la modificación 1 dentro del código.

Regla práctica: usa este barrido cuando la pregunta sea binaria (¿algún par se cruza?) sobre n segmentos; para eso es $O(n \log n)$ contra el $O(n^2)$ de comparar todos los pares directamente. Si necesitas el conteo exacto de intersecciones o sus coordenadas, este struct no basta —requieres la versión completa de Bentley-Ottmann mencionada en la Nota.

30.3 Unión de Áreas de Rectángulos

Idea: extiende el barrido visto en *Idea General y Manejo de Eventos* a un problema donde el estado no es un simple contador de activos, sino la **longitud cubierta** de un segmento vertical en un instante dado. Cada rectángulo genera dos eventos: uno al entrar (en x_1) y otro al salir (en x_2), cada uno con un rango vertical $[y_1, y_2]$. Entre dos eventos consecutivos en x , la longitud vertical cubierta no cambia, así que el área total es la suma de (ancho del intervalo en x) $\times$ (longitud cubierta en y) para cada tramo. Para mantener esa longitud cubierta de forma dinámica —que aumenta o disminuye según entran o salen rectángulos— se usa un árbol de segmentos sobre las coordenadas y comprimidas, donde cada nodo guarda cuántas veces está cubierto su rango y la longitud efectivamente cubierta.

¿Por qué usarlo? A diferencia del barrido genérico (que solo cuenta cuántos intervalos están activos), aquí el estado es una cantidad continua —longitud— que depende de *qué tan solapados* están los rangos verticales activos, no solo de cuántos hay. Esto obliga a una estructura más rica que un simple contador o un **set**: un árbol de segmentos por conteo con recomputación de longitud cubierta en cada nodo. Es la técnica estándar frente a la fuerza bruta de descomponer todo en una grilla, que sería $O(n^2)$ o peor si las coordenadas son grandes.

Casos de uso:

- Área total cubierta por un conjunto de rectángulos solapados.
- Preprocesamiento para el *Skyline Problem* (comparten la estructura de eventos).
- Cálculo de cobertura en problemas de "manchas de pintura" o "zonas de señal" representadas como rectángulos.
- Base para calcular el perímetro de la unión (variante con estructura de datos adicional).

Complejidad: $O(n \log n)$. Se generan $2n$ eventos y $2n$ coordenadas y a comprimir, lo que cuesta $O(n \log n)$ al ordenar. El árbol de segmentos tiene $O(n)$ hojas (una por cada intervalo entre coordenadas y comprimidas

consecutivas), así que cada actualización cuesta $O(\log n)$, y se realizan $2n$ actualizaciones —una por evento—, dando $O(n \log n)$ en total.

```

1 // CREATE/READ/WRITE: Area de la union de n rectangulos
  // mediante barrido
2 // vertical + arbol de segmentos por conteo sobre coordenadas y
  // comprimidas
3 struct AreaUnionRectangulos {
4     struct Rectangulo { double x1, y1, x2, y2; };
5     struct Evento { double x; double y1, y2; int delta; }; //
      delta = +1 abre, -1 cierra
6
7     vector<Rectangulo> rectangulos;
8     vector<double> ys; // coordenadas y comprimidas (
      ordenadas, unicas)
9     vector<int> conteo; // conteo[nodo]: veces que el rango
      esta cubierto
10    vector<double> cubierta; // cubierta[nodo]: longitud
      cubierta en ese rango
11
12    // CREATE: reserva espacio para n rectangulos
13    AreaUnionRectangulos(int n) {
14        rectangulos.reserve(n);
15    } // O(1)
16
17    // WRITE: agrega un rectangulo definido por sus esquinas
      opuestas
18    void agregarRectangulo(double x1, double y1, double x2,
      double y2) {
19        rectangulos.push_back({x1, y1, x2, y2});
20    } // O(1) amortizado
21
22    // WRITE: actualiza el rango [ql, qr) del arbol sumando delta
      al conteo,
23    // y recalcula la longitud cubierta de cada nodo afectado
24    void actualizar(int nodo, int l, int r, int ql, int qr, int
      delta) {
25        if (qr <= l || r <= ql) return;
26        if (ql <= l && r <= qr) {
27            conteo[nodo] += delta;
28        } else {
29            int medio = (l + r) / 2;
30            actualizar(2*nodo, l, medio, ql, qr, delta);
31            actualizar(2*nodo+1, medio, r, ql, qr, delta);
32        }
33        if (conteo[nodo] > 0) {
34            cubierta[nodo] = ys[r] - ys[l];
35        } else if (r - l == 1) {
36            cubierta[nodo] = 0;
37        } else {
38            cubierta[nodo] = cubierta[2*nodo] + cubierta[2*nodo+1];
39        }
40    } // O(log m), m = numero de intervalos y comprimidos
41
42    // READ: calcula el area total cubierta por la union de todos
      los rectangulos
43    double calcularArea() {
44        vector<Evento> eventos;
45        vector<double> yTemp;
46        for (auto& r : rectangulos) {
47            eventos.push_back({r.x1, r.y1, r.y2, +1});
48            eventos.push_back({r.x2, r.y1, r.y2, -1});
49            yTemp.push_back(r.y1);
50            yTemp.push_back(r.y2);
51        }
52        sort(yTemp.begin(), yTemp.end());
53        yTemp.erase(unique(yTemp.begin(), yTemp.end()), yTemp.end());
54        ys = yTemp;
55        int m = (int)ys.size() - 1; // numero de intervalos [ys[i],
      ys[i+1])
56        if (m <= 0) return 0;
57        conteo.assign(4 * m, 0);
58        cubierta.assign(4 * m, 0);
59
60        sort(eventos.begin(), eventos.end(), [](const Evento& a,
      const Evento& b) {
61            return a.x < b.x;
62        });
63
64        double area = 0;
65        for (size_t i = 0; i < eventos.size(); i++) {
66            if (i > 0) {
67                double ancho = eventos[i].x - eventos[i-1].x;
68                area += ancho * cubierta[i];
69            }
70            int l = (int)(lower_bound(ys.begin(), ys.end(), eventos[i]
      ].y1) - ys.begin());
71            int r = (int)(lower_bound(ys.begin(), ys.end(), eventos[i]
      ].y2) - ys.begin());
72            actualizar(1, 0, m, l, r, eventos[i].delta);
73        }

```

```

74    return area;
75 } // O(n log n): sort de 2n eventos, mas 2n actualizaciones O
      (log m) cada una
76
77 // -----
78 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
79 // -----
80 // 1. Perimetro de la union (en vez de area): requiere ademas
      trackear
81 // cuantos "segmentos" cubiertos tiene cada nodo, para
      sumar la
82 // variacion vertical entre eventos consecutivos mas la
      longitud
83 // horizontal del borde superior/inferior. Struct distinto,
      ver
84 // Perimetro de la Union de Rectangulos.
85 // 2. Volumen de union de cajas 3D: mismo barrido sobre el
      eje x, pero
86 // el arbol de segmentos debe resolver recursivamente el
      AREA
87 // cubierta en el plano yz - es esta misma tecnica anidada
      en 2D.
88 // 3. Si las coordenadas ya son enteras y pequenas, se puede
      omitir la
89 // compresion e indexar el arbol directamente por valor,
      siempre que
90 // el rango quepa en memoria.
91 };

```

Ejemplo de funcionamiento:

```

1 AreaUnionRectangulos r(2);
2 r.agregarRectangulo(0, 0, 4, 4); // area 16
3 r.agregarRectangulo(2, 2, 6, 6); // area 16, se solapa en (2,2)
      -(4,4)
4
5 double resultado = r.calcularArea();
6 // resultado = 28
7 // (16 + 16 - 4 de solapamiento, por inclusion-exclusion)

```

Regla práctica: usa esta técnica cuando la pregunta sea sobre **área total cubierta** (una cantidad continua), no sobre cuántos objetos están activos —para eso te alcanza el barrido genérico simple. Si además necesitas el perímetro en vez del área, el árbol de segmentos necesita información adicional (ver modificación 1 dentro del código).

30.4 Skyline Problem

Idea: comparte la estructura de eventos de *Unión de Áreas de Rectángulos* —cada edificio genera un evento de inicio y uno de fin—, pero la pregunta es distinta: en vez de longitud cubierta, interesa la **altura máxima visible** en cada punto x . El estado del barrido es entonces el conjunto de alturas de los edificios actualmente activos, y el skyline solo cambia en los puntos donde esa altura máxima cambia. Guardar las alturas activas en un **multiset** permite insertar, borrar y consultar el máximo, todo en $O(\log n)$.

¿Por qué usarlo? La alternativa de recorrer edificio por edificio y comparar contra todos los demás es $O(n^2)$. Aquí, igual que en la unión de rectángulos, basta con procesar eventos ordenados y mantener el estado activo, pero con una estructura distinta: un **multiset** de alturas en vez de un árbol de segmentos por conteo, porque lo único que importa es el máximo actual, no una suma o longitud acumulada. Es más simple que adaptar el árbol de segmentos de la subsección anterior para este propósito.

Casos de uso:

- El problema clásico de silueta de edificios (Skyline Problem).

- Calcular el área bajo la envolvente superior de un conjunto de rectángulos alineados en la base.
- Determinar en qué momentos cambia el valor máximo entre un conjunto de intervalos con "altura" o "prioridad" asociada.
- Simulación de capas visibles en gráficos 2D (qué elemento está "al frente" en cada posición x).

Complejidad: $O(n \log n)$. Ordenar los $2n$ eventos cuesta $O(n \log n)$. Cada evento realiza una inserción o un borrado en el `multiset`, cada uno $O(\log n)$, y consultar el máximo con `rbegin()` es $O(1)$. En total, $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Silueta (skyline) de n edificios mediante
  // barrido
2 // horizontal + multiset de alturas activas
3 struct SkylineProblema {
4     struct Evento { long long x; long long alto; bool esInicio;
5     };
6     vector<Evento> eventos;
7
8     // CREATE: reserva espacio para 2*n eventos (inicio y fin de
9     // cada edificio)
10    SkylineProblema(int n) {
11        eventos.reserve(2 * n);
12    } // O(1)
13
14    // WRITE: agrega un edificio dado su rango horizontal [izq,
15    // der] y su altura
16    void agregarEdificio(long long izq, long long der, long long
17    alto) {
18        eventos.push_back({izq, alto, true});
19        eventos.push_back({der, alto, false});
20    } // O(1) amortizado
21
22    // READ: calcula los puntos clave del skyline: pares (x,
23    // altura) donde
24    // la altura maxima visible cambia
25    vector<pair<long long, long long>> calcularSkyline() {
26        sort(eventos.begin(), eventos.end(), [](const Evento& a,
27        const Evento& b) {
28            return a.x < b.x;
29        });
30
31        multiset<long long> alturasActivas = {0}; // el "suelo"
32        // siempre esta activo
33        vector<pair<long long, long long>> resultado;
34        long long alturaPrevia = 0;
35        size_t i = 0;
36        while (i < eventos.size()) {
37            long long xActual = eventos[i].x;
38            // procesa TODOS los eventos de esta misma x como un
39            // bloque, para
40            // no reportar un punto intermedio incorrecto
41            while (i < eventos.size() && eventos[i].x == xActual) {
42                if (eventos[i].esInicio) alturasActivas.insert(eventos[i].alto);
43                else alturasActivas.erase(alturasActivas.find(eventos[i].alto));
44                i++;
45            }
46            long long alturaActual = *alturasActivas.rbegin();
47            if (alturaActual != alturaPrevia) {
48                resultado.push_back({xActual, alturaActual});
49                alturaPrevia = alturaActual;
50            }
51        }
52        return resultado;
53    } // O(n log n): sort O(n log n), n operaciones sobre el
54    // multiset O(log n) cada una
55
56    // -----
57    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
58    // -----
59
60    // 1. Area bajo el skyline en vez de los puntos clave: basta
61    // con acumular
62    // (x_actual - x_anterior) * alturaPrevia en cada
63    // iteracion del while
64    // externo, antes de actualizar alturaPrevia.
65    // 2. Muchos edificios con la MISMA altura exacta: el
66    // multiset es
67    // preferible a un arbol de segmentos aqui, porque
68    // insertar/borrar UNA
69    // ocurrencia de un valor repetido es directo con find +
70    // erase.

```

```

56 // 3. Silueta con arbol de segmentos (alternativa): si el
57 // rango de x es
58 // pequeno y se requiere actualizar dinamicamente, se
59 // puede reemplazar
60 // el multiset por un arbol de segmentos tipo "maximo con
61 // propagacion
62 // perezosa" sobre las coordenadas x comprimidas.
63 };

```

Ejemplo de funcionamiento:

```

1 SkylineProblema s(2);
2 s.agregarEdificio(1, 5, 3);
3 s.agregarEdificio(3, 7, 5);
4
5 auto resultado = s.calcularSkyline();
6 // resultado = {(1,3), (3,5), (7,0)}
7 // (en x=1 aparece un edificio de altura 3; en x=3 aparece uno
8 // mas alto,
9 // de altura 5, que domina la vista; en x=7 termina el mas
10 // alto y la
11 // altura visible cae a 0, ya que el primero termino en x=5
12 // sin cambiar
13 // el maximo visible en ese punto)

```

Nota: el paso de agrupar eventos por la misma coordenada x antes de consultar el máximo (el `while` interno del código) es crítico: procesar los eventos uno por uno sin agrupar puede reportar un punto de skyline falso cuando un edificio termina justo donde otro empieza en la misma coordenada.

Regla práctica: usa esta técnica cuando la pregunta sea sobre el **máximo** (o mínimo) visible en cada punto, no sobre longitud o área acumulada —para eso está *Unión de Áreas de Rectángulos*. Ambas comparten la misma estructura de eventos de entrada/salida, así que si ya resolviste una, adaptar la otra es principalmente cambiar la estructura de estado (árbol de segmentos por conteo vs. multiset de alturas).

30.5 Par de Puntos Más Cercano (Closest Pair)

Idea: a diferencia de las subsecciones anteriores, aquí el barrido no procesa eventos de entrada/salida de un intervalo, sino **puntos individuales** ordenados por x , y el estado activo es una "franja" (*strip*) de puntos candidatos a formar el par más cercano. La observación clave: si ya se conoce una distancia mínima d entre los puntos procesados hasta ahora, cualquier punto nuevo solo puede mejorar esa distancia si está a menos de d en x del punto actual —por eso se mantiene una ventana de puntos "recientes" (los últimos con x dentro de d), y dentro de esa ventana, gracias a un argumento geométrico de empaquetamiento, solo hace falta comparar contra un número constante de vecinos ordenados por y .

¿Por qué usarlo? Comparar todos los pares de n puntos es $O(n^2)$. El enfoque de *divide y vencerás* clásico también resuelve esto en $O(n \log n)$, pero requiere combinar recursivamente las mitades izquierda y derecha con una franja central —más difícil de implementar. La versión con barrido logra la misma complejidad manteniendo una única estructura ordenada por y (un `multiset` o similar) que se poda dinámicamente, sin necesidad de recursión ni de fusionar subproblemas. Es la aplicación más directa de "mantener solo el estado necesario" entre las técnicas de esta sección: aquí el estado es una ventana deslizante de puntos, no un conjunto de intervalos activos.

Casos de uso:

- Distancia mínima entre n puntos en el plano.

- Detección de colisiones o "puntos demasiado cercanos" en simulaciones.
- Preprocesamiento en problemas de clustering o agrupamiento espacial.
- Verificar si algún par de ubicaciones (antenas, sensores) viola una distancia mínima de seguridad.

Complejidad: $O(n \log n)$. Ordenar los n puntos por x cuesta $O(n \log n)$. Cada punto se inserta y eventualmente se elimina del multiset activo — $O(\log n)$ cada operación—, y por cada punto se consultan solo los vecinos con y dentro de rango d , que son $O(1)$ en promedio por el argumento de empaquetamiento (en una franja de ancho d y alto $2d$ caben a lo sumo un número constante de puntos con distancia mínima d entre ellos). El total es $O(n \log n)$.

```

1 // CREATE/READ/WRITE: Distancia minima entre n puntos mediante
  // barrido
2 // horizontal + ventana de candidatos ordenada por y
3 struct ParMasCercano {
4     struct Punto { double x, y; int id; };
5     vector<Punto> puntos;
6
7     // CREATE: reserva espacio para n puntos
8     ParMasCercano(int n) {
9         puntos.reserve(n);
10    } // O(1)
11
12    // WRITE: agrega un punto al conjunto
13    void agregarPunto(double x, double y) {
14        puntos.push_back({x, y, (int)puntos.size()});
15    } // O(1) amortizado
16
17    // READ: calcula la distancia euclidiana al cuadrado (evita
      // sqrt hasta el final)
18    static double distCuadrado(const Punto& a, const Punto& b) {
19        double dx = a.x - b.x, dy = a.y - b.y;
20        return dx * dx + dy * dy;
21    } // O(1)
22
23    // READ: barre los puntos ordenados por x y devuelve la
      // distancia minima
24    // entre cualquier par
25    double distanciaMinima() {
26        sort(puntos.begin(), puntos.end(), [](const Punto& a, const
          Punto& b) {
27            return a.x < b.x;
28        });
29
30        // ventana ordenada por y, contiene solo puntos con x
          // dentro de la
31        // distancia minima actual respecto al punto que se esta
          // procesando
32        set<pair<double,int>> ventana; // (y, id) para permitir
          // duplicados de y
33        double mejorCuadrado = 1e18;
34        int inicio = 0; // indice del primer punto todavia dentro
          // de la ventana
35
36        for (int i = 0; i < (int)puntos.size(); i++) {
37            double mejorActual = sqrt(mejorCuadrado);
38            // elimina de la ventana los puntos cuya x ya quedo
          // demasiado atras
39            while (inicio < i && puntos[i].x - puntos[inicio].x >
              mejorActual) {
40                ventana.erase({puntos[inicio].y, puntos[inicio].id});
41                inicio++;
42            }
43            // solo hace falta comparar contra puntos con y en [y_i -
          // mejorActual, y_i + mejorActual]
44            auto desde = ventana.lower_bound({puntos[i].y -
              mejorActual, -1});
45            auto hasta = ventana.upper_bound({puntos[i].y +
              mejorActual, INT_MAX});
46            for (auto it = desde; it != hasta; ++it) {
47                double d2 = distCuadrado(puntos[i], puntos[it->second]);
48            }
49            mejorCuadrado = min(mejorCuadrado, d2);
50            ventana.insert({puntos[i].y, puntos[i].id});
51        }
52        return sqrt(mejorCuadrado);
53    } // O(n log n): sort O(n log n), y por el argumento de
      // empaquetamiento cada

```

```

54 // punto compara con O(1) vecinos en promedio dentro de la
      // ventana
55
56 //
57 // -----
58 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
59 // -----
60 // 1. Reportar el PAR de puntos (no solo la distancia):
      // guardar los ids
61 // junto con mejorCuadrado cada vez que se actualiza el
      // minimo dentro
62 // del for interno.
63 // 2. Coordenadas enteras: usar long long en distCuadrado
      // para evitar
64 // overflow, y comparar con mejorCuadrado como entero (sin
      // sqrt) hasta
65 // el resultado final, evitando errores de precision con
      // double.
66 // 3. Divide y venceras (alternativa clasica): en vez de la
      // ventana
67 // deslizante con set, se puede resolver recursivamente
      // dividiendo el
68 // conjunto de puntos por la mediana de x, resolviendo
      // cada mitad, y
69 // fusionando con una franja central de ancho 2*d. Misma
      // complejidad
70 // O(n log n), pero mas dificil de implementar
      // correctamente; el
71 // barrido con set es preferible en un contexto de
      // concurso.

```

Ejemplo de funcionamiento:

```

1 ParMasCercano p(4);
2 p.agregarPunto(0, 0);
3 p.agregarPunto(3, 4);
4 p.agregarPunto(1, 1);
5 p.agregarPunto(10, 10);
6
7 double resultado = p.distanciaMinima();
8 // resultado = sqrt(2)
9 // (el par mas cercano es (0,0) y (1,1), a distancia sqrt(1^2 +
10 // 1^2) = sqrt(2);

```

Nota: la condición `puntos[i].x - puntos[inicio].x > mejorActual` para podar la ventana, y los límites `lower_bound/upper_bound` en `y`, son lo que mantiene la comparación por punto en $O(1)$ amortizado. Si se omite cualquiera de las dos podas (en x o en y), el algoritmo sigue siendo correcto pero degenera a $O(n^2)$ en el peor caso, perdiendo la ventaja del barrido.

Regla práctica: usa esta técnica cuando el problema pida una distancia mínima (o máxima densidad local) entre puntos en el plano y n sea grande. Frente al divide y vencerás clásico, esta versión con barrido y ventana deslizante es más simple de codificar en competencia y logra la misma complejidad $O(n \log n)$, siempre que no se olviden las dos podas de la ventana (en x al insertar/eliminar, en y al consultar).

31 Primitivas Geométricas

31.1 Vector Geométrico: Producto Punto y Producto Cruz

Idea: un vector geométrico $\vec{v} = (x, y)$ representa una dirección y magnitud en el plano, y casi toda la geometría computacional se reduce a dos operaciones entre vectores: el **producto punto** $\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$, que mide cuánto "apuntan en la misma dirección" dos vectores (relacionado con el coseno del ángulo entre ellos), y el **producto cruz** $\vec{a} \times \vec{b} = a_x b_y - a_y b_x$, que en 2D da un escalar cuyo signo indica de qué lado está un vector respecto al otro (relacionado con el seno del ángulo, y con el área

con signo del paralelogramo que forman). Estas dos operaciones son los "primitivos" a partir de los cuales se construyen casi todos los demás algoritmos de esta parte: orientación, colinealidad, área de polígonos, ángulos.

¿Por qué usarlo? Sin estas dos operaciones no hay forma de razonar algebraicamente sobre direcciones y orientación en el plano —la alternativa sería calcular ángulos con `atan2` y compararlos, lo cual es más lento, acumula error de punto flotante, y falla en casos límite (ángulos de 0 o 2π). El producto punto y el producto cruz, en cambio, se calculan con aritmética exacta si las coordenadas son enteras, y evitan funciones trigonométricas costosas en la mayoría de los casos donde solo se necesita signo o comparación relativa, no el ángulo exacto.

Casos de uso:

- Determinar si tres puntos giran en sentido horario o antihorario (ver *Orientación de Tres Puntos*).
- Calcular el área de un polígono (Shoelace, que es una suma de productos cruz).
- Proyectar un punto sobre una línea (producto punto normalizado).
- Determinar si un ángulo es agudo, recto u obtuso (signo del producto punto).
- Ordenar puntos angularmente alrededor de un pivote (signo del producto cruz).

Complejidad: $O(1)$ para ambas operaciones —son sumas y multiplicaciones de un número constante de componentes, sin importar el tamaño de la entrada.

```

1 // CREATE/READ/WRITE: Vector geométrico 2D con producto punto y
  producto cruz
2 struct Vector2D {
3     double x, y;
4
5     // CREATE: construye un vector a partir de sus componentes
      (0,0 por defecto)
6     Vector2D(double x_ = 0, double y_ = 0) : x(x_), y(y_) {} // O(1)
7
8     // READ: resta de puntos para obtener el vector que va de a
      hacia b
9     static Vector2D desde(const Vector2D& a, const Vector2D& b) {
10         return Vector2D(b.x - a.x, b.y - a.y);
11     } // O(1)
12
13     // READ: producto punto, mide alineación entre this y otro
      vector
14     double punto(const Vector2D& otro) const {
15         return x * otro.x + y * otro.y;
16     } // O(1)
17
18     // READ: producto cruz (componente z del producto cruz 3D),
      mide
19     // orientación relativa y área con signo del paralelogramo
20     double cruz(const Vector2D& otro) const {
21         return x * otro.y - y * otro.x;
22     } // O(1)
23
24     // READ: magnitud (norma) del vector
25     double norma() const {
26         return sqrt(x * x + y * y);
27     } // O(1)
28
29     //
30     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31     //
32     // 1. Coordenadas enteras: si el problema garantiza
      coordenadas enteras,
33     // usar long long en vez de double evita error de punto
      flotante en
34     // punto() y cruz() (ambos dan resultado exacto con
      enteros).

```

```

35 // 2. Signo del producto cruz: cruz(otro) > 0 significa que "
      otro" esta
36 // en sentido antihorario respecto a this; < 0 significa
      horario;
37 // == 0 significa que son colineales (paralelos). Ver
      Orientacion de
38 // Tres Puntos para la aplicacion directa de esto.
39 // 3. Angulo entre vectores: se puede obtener con
40 // atan2(this->cruz(otro), this->punto(otro)), que da el
      angulo con
41 // signo en (-pi, pi] sin ambigüedad de cuadrante, a
      diferencia de
42 // usar solo acos(punto / (norma*norma)).
43 };

```

Ejemplo de funcionamiento:

```

1 Vector2D a(2, 0);
2 Vector2D b(0, 3);
3
4 double p = a.punto(b);
5 // p = 0
6 // (son perpendiculares, el producto punto de vectores
      perpendiculares es 0)
7
8 double c = a.cruz(b);
9 // c = 6
10 // (positivo: b esta en sentido antihorario respecto a a;
11 // 6 es también el área del paralelogramo que forman)

```

Regla práctica: usa el producto punto cuando la pregunta sea sobre **alineación o proyección** (¿qué tan paralelos son dos vectores?, ¿es un ángulo agudo u obtuso?); usa el producto cruz cuando la pregunta sea sobre **orientación o área con signo** (¿a qué lado está un punto?, ¿en qué sentido giran tres puntos?). Casi todo lo que sigue en esta parte de la guía se construye componiendo estas dos operaciones.

31.2 Orientación de Tres Puntos (Cross Product / CCW Test)

Idea: dado un punto O (origen/pivote) y dos puntos A y B , el signo del producto cruz $\vec{OA} \times \vec{OB}$ —visto en *Vector Geométrico*— determina si, parado en O mirando hacia A , el punto B queda a la izquierda (positivo, sentido antihorario/CCW), a la derecha (negativo, sentido horario/CW), o exactamente sobre la línea que pasa por O y A (cero, colineales). Esta es la prueba de orientación más usada en geometría computacional porque reduce una pregunta geométrica cualitativa ("¿de qué lado está?") a una operación algebraica exacta de $O(1)$.

¿Por qué usarlo? Es la primitiva que sustituye a comparar ángulos con `atan2` o a calcular pendientes con división —ambas formas más lentas, propensas a error de punto flotante, y con casos especiales molestos (línea vertical, división por cero). El producto cruz da el mismo resultado cualitativo con una sola multiplicación y resta, y si las coordenadas son enteras, el resultado es exacto sin ningún error de precisión. Es la base directa de Shoelace, Convex Hull y Point-in-Polygon, que se ven más adelante en esta parte.

Casos de uso:

- Determinar si un giro en una secuencia de puntos es a la izquierda o a la derecha (clave en construcción de envolvente convexa).
- Verificar si tres puntos son colineales.
- Determinar de qué lado de una línea está un punto (para point-in-polygon, clipping, etc.).

- Ordenar puntos angularmente alrededor de un pivote.
- Calcular el área con signo de un triángulo (mitad del valor absoluto del producto cruz).

Complejidad: $O(1)$ —es una evaluación directa del producto cruz entre dos vectores derivados de los tres puntos, sin bucles ni recursión.

```

1 // CREATE/READ: Prueba de orientacion de tres puntos mediante
  producto cruz
2 struct OrientacionPuntos {
3     struct Punto { double x, y; };
4
5     // READ: producto cruz de (b - o) x (c - o); signo indica
      orientacion
6     // de o, b, c: positivo = CCW (izquierda), negativo = CW (
      derecha), 0 = colineales
7     static double cruz(const Punto& o, const Punto& b, const
      Punto& c) {
8         return (b.x - o.x) * (c.y - o.y) - (b.y - o.y) * (c.x - o.x
      );
9     } // O(1)
10
11     // READ: clasifica la orientacion como +1 (CCW), -1 (CW), o 0
      (colinear)
12     // usando un epsilon para tolerar error de punto flotante
13     static int orientacion(const Punto& o, const Punto& b, const
      Punto& c) {
14         double val = cruz(o, b, c);
15         const double EPS = 1e-9;
16         if (val > EPS) return 1; // CCW: c esta a la izquierda de
      o->b
17         if (val < -EPS) return -1; // CW: c esta a la derecha de o
      ->b
18         return 0; // colineales
19     } // O(1)
20
21     //
22     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
23     //
24     // 1. Coordenadas enteras: usar long long y comparar cruz()
      directamente
25     // contra 0 (sin EPS), ya que con enteros el resultado es
      exacto y no
26     // hay error de punto flotante que tolerar.
27     // 2. Colinealidad estricta con verificacion de "entre":
      orientacion == 0
28     // solo dice que estan sobre la misma recta, no que c este
      ENTRE o y b.
29     // Para eso se necesita ademas revisar que las coordenadas
      de c esten
30     // dentro del rango [o, b] en x e y (ver Tres Puntos
      Colineales).
31     // 3. Convencion de signo: esta implementacion usa positivo =
      CCW, que es
32     // la convencion estandar en la mayoria de la literatura
      de CP (y la
33     // que se usa consistentemente en el resto de esta guia,
      incluyendo
34     // Shoelace y Envolvente Convexa). Si se invierte el orden
      de los
35     // parametros (o, c, b en vez de o, b, c), el signo
      tambien se invierte.
36 };

```

Ejemplo de funcionamiento:

```

1 OrientacionPuntos::Punto o{0, 0}, b{4, 0}, c{2, 2};
2
3 int resultado = OrientacionPuntos::orientacion(o, b, c);
4 // resultado = 1 (CCW)
5 // (parado en o mirando hacia b sobre el eje x, el punto c
      =(2,2) queda
6 // arriba, es decir a la izquierda, sentido antihorario)

```

Nota: la convención positivo = CCW fijada aquí se usa de forma consistente en el resto de la guía —Shoelace, Envolvente Convexa (Jarvis y Monotone Chain), y Punto Dentro de un Polígono Convexo. Si en algún momento se necesita la convención opuesta, basta con invertir el signo del resultado o el orden de los argumentos, pero hay que hacerlo de forma consistente en todos los algoritmos que dependan de esta prueba.

Regla práctica: usa esta prueba como bloque de construcción base cada vez que necesites saber de qué lado de una línea (definida por dos puntos) está un tercer punto, sin importar el ángulo exacto —eso es más rápido y más robusto numéricamente que calcular pendientes o ángulos con funciones trigonométricas.

31.3 Distancia Euclidiana

Idea: la distancia euclidiana entre dos puntos $A = (x_1, y_1)$ y $B = (x_2, y_2)$ se deriva directamente del teorema de Pitágoras: el segmento AB es la hipotenusa de un triángulo rectángulo cuyos catetos son las diferencias de coordenadas $\Delta x = x_2 - x_1$ y $\Delta y = y_2 - y_1$, así que $d(A, B) = \sqrt{\Delta x^2 + \Delta y^2}$. Es la noción de distancia "en línea recta" y la base de la mayoría de las métricas geométricas de esta guía, aunque conceptualmente equivale a la norma del vector $\vec{AB}$ (ver *Vector Geométrico*).

¿Por qué usarlo? Es la métrica por defecto en geometría computacional cuando el problema no especifica otra cosa (a diferencia de la distancia Manhattan, que suma $|\Delta x| + |\Delta y|$ y aparece en problemas de grilla). El costo real está en la raíz cuadrada: cuando solo se necesita **comparar** distancias (cuál es menor, cuál es mayor), conviene trabajar con la distancia al cuadrado y evitar `sqrt()` —así se gana precisión con enteros y se ahorra una operación costosa, como ya se hizo en *Par de Puntos Más Cercano*.

Casos de uso:

- Encontrar el par de puntos más cercano o más lejano.
- Verificar si un punto está dentro de un círculo de radio dado.
- Calcular el perímetro de un polígono (suma de distancias entre vértices consecutivos).
- Comparar cuál de dos puntos está más cerca de un tercero, sin necesidad del valor exacto.

Complejidad: $O(1)$ —dos restas, dos multiplicaciones, una suma y, opcionalmente, una raíz cuadrada.

```

1 // CREATE/READ: Distancia euclidiana entre dos puntos, con y
  sin raíz cuadrada
2 struct DistanciaEuclidiana {
3     struct Punto { double x, y; };
4
5     // READ: distancia al cuadrado, evita sqrt cuando solo se
      necesita comparar
6     static double distCuadrado(const Punto& a, const Punto& b) {
7         double dx = b.x - a.x;
8         double dy = b.y - a.y;
9         return dx * dx + dy * dy;
10    } // O(1)
11
12    // READ: distancia euclidiana real (con raíz cuadrada)
13    static double distancia(const Punto& a, const Punto& b) {
14        return sqrt(distCuadrado(a, b));
15    } // O(1)
16
17    //
18    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
19    //
20    // 1. Coordenadas enteras: usar long long en distCuadrado
      evita perder
21    // precision; ademas, comparar distCuadrado(a,b) <
      distCuadrado(c,d)
22    // da el mismo resultado que comparar distancia(a,b) <
      distancia(c,d)
23    // sin necesidad de sqrt, porque la raíz cuadrada es
      monotona creciente.

```

```

24 // 2. Distancia Manhattan (metrica alternativa): |b.x-a.x| +
25 // |b.y-a.y|,
26 // usada en problemas de grilla donde el movimiento es
27 // solo horizontal
28 // o vertical, no diagonal.
29 // 3. Distancia Chebyshev (otra metrica alternativa): max(|b.
30 // x-a.x|,
31 // |b.y-a.y|), usada cuando el movimiento diagonal cuesta
32 // igual que el
33 // movimiento recto (como el rey en ajedrez).
34 };

```

Ejemplo de funcionamiento:

```

1 DistanciaEuclidiana::Punto a{0, 0}, b{3, 4};
2
3 double resultado = DistanciaEuclidiana::distancia(a, b);
4 // resultado = 5
5 // (triangulo 3-4-5, caso clasico de pitagoras exacto)

```

Regla práctica: usa `distCuadrado` en vez de `distancia` cada vez que el problema solo requiera **comparar** distancias entre sí (mínimo, máximo, ¿es menor que r ?) —evita la raíz cuadrada y, con coordenadas enteras, evita también el error de punto flotante por completo.

31.4 Punto Medio de un Segmento

Idea: el punto medio de un segmento entre $A = (x_1, y_1)$ y $B = (x_2, y_2)$ es el promedio simple de sus coordenadas:

$M = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$. Es un caso particular de interpolación lineal con $t = 0.5$ —el punto que está exactamente a mitad de camino en ambas coordenadas por separado, ya que la geometría del plano cartesiano permite tratar x e y de forma independiente para este cálculo.

¿Por qué usarlo? Es la operación más simple de esta parte, pero aparece constantemente como paso intermedio de algoritmos más complejos (bisección de segmentos, cálculo de centroides, mediatriz para circuncentro). A diferencia de la *División de un Segmento en una Razón Dada* (más general, con t arbitrario), el punto medio es el caso fijo $t = 0.5$ y no requiere parametrizar la razón, por lo que conviene tenerlo aparte como atajo directo cuando la razón es exactamente 1 : 1.

Casos de uso:

- Encontrar el centro de un segmento para construir la mediatriz (útil en circuncentro).
- Bisección recursiva de un segmento o polígono.
- Calcular el centroide de un conjunto de puntos (promedio de todos los puntos medios, o directamente el promedio de coordenadas).
- Punto de referencia para pruebas de simetría o reflexión.

Complejidad: $O(1)$ —dos sumas y dos divisiones por 2.

```

1 // CREATE/READ: Punto medio de un segmento definido por dos
2 // puntos
3 struct PuntoMedio {
4     struct Punto { double x, y; };
5
6     // READ: calcula el punto medio entre a y b
7     static Punto medio(const Punto& a, const Punto& b) {
8         return Punto{(a.x + b.x) / 2.0, (a.y + b.y) / 2.0};
9     } // O(1)

```

```

10 //
11 // -----
12 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
13 // -----
14 // 1. Coordenadas enteras con resultado exacto: si a.x+b.x y
15 // a.y+b.y son
16 // siempre pares, se puede mantener el punto medio en
17 // enteros dividiendo
18 // entre 2 directamente, evitando pasar por double.
19 // 2. Centroide de n puntos (generalizacion): promedio de
20 // todas las
21 // coordenadas x y de todas las coordenadas y por separado,
22 // no es lo
23 // mismo que promediar puntos medios sucesivos si los
24 // puntos no forman
25 // un segmento simple.
26 // 3. Punto medio ponderado (interpolacion con t != 0.5): ver
27 // Division de
28 // un Segmento en una Razón Dada para el caso general.
29 };

```

Ejemplo de funcionamiento:

```

1 PuntoMedio::Punto a{0, 0}, b{6, 10};
2
3 auto resultado = PuntoMedio::medio(a, b);
4 // resultado = (3, 5)

```

Regla práctica: usa el punto medio directamente cuando la razón sea exactamente 1 : 1; si el problema pide un punto a una fracción arbitraria del segmento (por ejemplo, a un tercio de camino), no fuerces el punto medio dos veces —usa la fórmula general de *División de un Segmento en una Razón Dada*, que es igual de simple y evita errores de composición.

31.5 Ecuación de una Línea Dados Dos Puntos

Idea: dados dos puntos $A = (x_1, y_1)$ y $B = (x_2, y_2)$, la recta que pasa por ambos se puede escribir en forma general $ax + by + c = 0$ tomando $a = y_2 - y_1$, $b = x_1 - x_2$, $c = -(ax_1 + by_1)$. Esta forma se deriva de que el vector normal a la dirección $\vec{AB} = (x_2 - x_1, y_2 - y_1)$ es $(a, b) = (y_2 - y_1, -(x_2 - x_1))$ —rotar el vector director 90 grados— y cualquier punto (x, y) sobre la recta satisface que el vector desde A hasta (x, y) es perpendicular a ese normal, lo cual da la ecuación. A diferencia de la forma pendiente-intersección $y = mx + k$, esta forma general no tiene problema con rectas verticales (donde la pendiente no está definida).

¿Por qué usarlo? La forma $y = mx + k$ falla cuando $x_1 = x_2$ (recta vertical, pendiente infinita), obligando a manejar ese caso aparte con un `if`. La forma general $ax + by + c = 0$ maneja rectas verticales y horizontales de manera uniforme sin casos especiales, y además permite evaluar directamente de qué lado de la recta está un punto con el signo de $ax + by + c$ —relacionado con el producto cruz de *Orientación de Tres Puntos*, pero expresado como coeficientes reutilizables en vez de recalculando el producto cruz cada vez.

Casos de uso:

- Determinar si un punto está sobre, a la izquierda o a la derecha de una recta fija (evaluada muchas veces).
- Calcular la distancia de un punto a una recta (usando los coeficientes a, b, c directamente).
- Encontrar la intersección entre dos rectas resolviendo el sistema de dos ecuaciones.

- Representar restricciones lineales en problemas de programación lineal geométrica simple.

Complejidad: $O(1)$ para construir la ecuación; $O(1)$ para evaluarla en un punto dado.

```

1 // CREATE/READ: Linea en forma general ax + by + c = 0, dados
  dos puntos
2 struct LineaGeneral {
3     double a, b, c;
4
5     // CREATE: construye la ecuacion general de la recta que pasa
      por p y q
6     LineaGeneral(double x1, double y1, double x2, double y2) {
7         a = y2 - y1;
8         b = x1 - x2;
9         c = -(a * x1 + b * y1);
10    } // O(1)
11
12    // READ: evalua ax + by + c en un punto; el signo indica el
      lado de la
13    // recta (positivo o negativo segun la orientacion de a,b), 0
      = sobre la recta
14    double evaluar(double x, double y) const {
15        return a * x + b * y + c;
16    } // O(1)
17
18    // READ: verifica si un punto esta sobre la recta (con
      tolerancia)
19    bool contiene(double x, double y) const {
20        const double EPS = 1e-9;
21        return fabs(evaluar(x, y)) < EPS;
22    } // O(1)
23
24    //
25    // -----
26    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
27    // -----
28    // 1. Interseccion de dos rectas L1 (a1,b1,c1) y L2 (a2,b2,c2)
      ): resolver
29    // el sistema 2x2 con la regla de Cramer:
30    // det = a1*b2 - a2*b1;
31    // if (fabs(det) < EPS) { /* paralelas o coincidentes, sin
      solucion unica */ }
32    // x = (b1*c2 - b2*c1) / det; y = (a2*c1 - a1*c2) / det;
33    // 2. Coordenadas enteras: a, b, c pueden mantenerse como
      long long para
34    // evaluar() y contiene() de forma exacta, comparando
      contiene()
35    // contra 0 sin EPS.
36    // 3. Normalizacion opcional: dividir a,b,c entre sqrt(a*a+b*b)
      convierte
37    // evaluar(x,y) directamente en la distancia con signo del
      punto a la
38    // recta (ver Distancia de un Punto a una Linea/Segmento).
39 };
```

Ejemplo de funcionamiento:

```

1 LineaGeneral l(0, 0, 4, 4); // recta y = x
2
3 double val = l.evaluar(2, 5);
4 // val = -12
5 // (a=4, b=-4, c=0; evaluar(2,5) = 4*2 + (-4)*5 + 0 = 8 - 20 =
  -12;
6 // el signo negativo indica que (2,5) esta de un lado
  especifico de y=x)
7
8 bool sobre = l.contiene(3, 3);
9 // sobre = true (el punto (3,3) esta sobre la recta y=x)
```

Regla práctica: usa esta forma general en vez de $y = mx + k$ cada vez que la recta pueda ser vertical, o cuando necesites evaluar el mismo lado de la recta para muchos puntos distintos —los coeficientes a, b, c se calculan una sola vez y se reutilizan, a diferencia de recalcular un producto cruz completo contra dos puntos fijos en cada consulta.

31.6 División de un Segmento en una Razón Dada

Idea: generaliza el *Punto Medio de un Segmento* (que es el caso fijo $t = 0.5$) a cualquier fracción $t \in [0, 1]$ del

camino entre A y B : el punto que divide el segmento en esa proporción es la interpolación lineal $P = A + t \cdot (B - A) = ((1 - t)x_1 + tx_2, (1 - t)y_1 + ty_2)$. Cuando $t = 0$ se obtiene A , cuando $t = 1$ se obtiene B , y valores intermedios dan puntos proporcionalmente distribuidos sobre el segmento. Si la razón se da como " $m : n$ " (divide al segmento en partes de longitud proporcional m y n), se convierte a $t = \frac{m}{m+n}$.

¿Por qué usarlo? Es la forma general de la que el punto medio es un caso particular; evaluarlo cada vez con la fórmula de interpolación en vez de "forzar" un punto medio con pasos intermedios evita errores de composición y funciona igual de bien para t fuera de $[0, 1]$ (extrapolación, para encontrar un punto más allá de B en la misma dirección que $A \rightarrow B$). Es la base de la interpolación paramétrica de segmentos, usada también para representar cualquier punto sobre una recta en función de un solo parámetro.

Casos de uso:

- Dividir un segmento en k partes iguales (aplicando $t = i/k$ para $i = 1, \dots, k - 1$).
- Encontrar un punto a una razón $m : n$ específica sobre un segmento (por ejemplo, el punto que divide una mediana en razón $2 : 1$ desde el vértice, propiedad del centroide de un triángulo).
- Extrapolar un punto más allá de un extremo del segmento, usando $t < 0$ o $t > 1$.
- Parametrizar un rayo o recta para intersección con otras figuras.

Complejidad: $O(1)$ —dos interpolaciones lineales independientes en x e y .

```

1 // CREATE/READ: Division de un segmento AB en una razon dada
  mediante
2 // interpolacion lineal parametrica
3 struct DivisionSegmento {
4     struct Punto { double x, y; };
5
6     // READ: interpola el punto a fraccion t del segmento de a
      hacia b
7     // t=0 da a, t=1 da b, valores fuera de [0,1] extrapolan
8     static Punto interpolar(const Punto& a, const Punto& b,
      double t) {
9         return Punto{
10             a.x + t * (b.x - a.x),
11             a.y + t * (b.y - a.y)
12         };
13    } // O(1)
14
15    // READ: divide el segmento en razon m:n (desde a hacia b),
      convirtiendo
16    // la razon a fraccion t = m/(m+n) antes de interpolar
17    static Punto dividirEnRazon(const Punto& a, const Punto& b,
      double m, double n) {
18        double t = m / (m + n);
19        return interpolar(a, b, t);
20    } // O(1)
21
22    //
23    // -----
24    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25    // -----
26    // 1. Punto medio como caso particular: interpolar(a, b, 0.5)
      es
27    // exactamente Punto Medio de un Segmento; no hace falta
      un struct
28    // separado si ya se tiene este.
29    // 2. Division en k partes iguales: generar los puntos
      interpolar(a, b, (double)i / k) para i = 1 hasta k-1.
30    // 3. Coordenadas enteras con razon m:n exacta (sin pasar por
      double):
31    // px = a.x + (b.x - a.x) * m / (m + n); usando division
      entera solo
```

```

32 // si se garantiza divisibilidad exacta; de lo contrario,
33 // mantener
34 // el resultado como fraccion (numerador, denominador)
// para exactitud.
};

```

Ejemplo de funcionamiento:

```

1 DivisionSegmento::Punto a{0, 0}, b{10, 20};
2
3 auto resultado = DivisionSegmento::dividirEnRazon(a, b, 1, 4);
4 // resultado = (2, 4)
5 // (t = 1/(1+4) = 0.2; el punto esta a un quinto del camino de
6 // a hacia b,
// dividiendo el segmento en partes de razon 1:4)

```

Regla práctica: usa esta fórmula general cada vez que necesites un punto sobre un segmento a una fracción o razón distinta de 1 : 1 —para el caso exacto de la mitad, es más directo usar *Punto Medio de un Segmento* sin pasar por la conversión de razón a t .

31.7 Ángulo entre Dos Vectores

Idea: el ángulo entre dos vectores $\vec{a}$ y $\vec{b}$ se puede obtener de dos formas relacionadas: con el producto punto, $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|}$, o de forma más robusta con `atan2`, combinando producto punto y producto cruz —vistos ambos en *Vector Geométrico*— como $\theta = \text{atan2}(\vec{a} \times \vec{b}, \vec{a} \cdot \vec{b})$. La segunda forma da directamente el ángulo **con signo** en el rango $(-\pi, \pi]$, indicando no solo la magnitud del ángulo sino también si $\vec{b}$ está en sentido antihorario (positivo) u horario (negativo) respecto a $\vec{a}$, sin la ambigüedad de cuadrante que tiene `acos`.

¿Por qué usarlo? Usar solo `acos(punto / (norma_a * norma_b))` tiene dos problemas: pierde el signo (no distingue si $\vec{b}$ gira a la izquierda o a la derecha de $\vec{a}$), y es numéricamente inestable cerca de $\theta = 0$ o $\theta = \pi$, donde pequeños errores de punto flotante en el argumento de `acos` pueden caer fuera de $[-1, 1]$ y producir NaN. La versión con `atan2(cruz, punto)` evita ambos problemas: es estable en todo el rango y conserva el signo, que es justamente lo que se necesita para ordenar puntos angularmente (ver *Barrido Angular* más adelante).

Casos de uso:

- Ordenar un conjunto de puntos angularmente alrededor de un pivote.
- Calcular el ángulo interno de un polígono en un vértice dado.
- Determinar cuánto hay que rotar un vector para alinearlo con otro.
- Verificar si un ángulo es agudo, recto u obtuso sin calcular el valor exacto (signo del producto punto, ver *Vector Geométrico*).

Complejidad: $O(1)$ —un producto punto, un producto cruz, y una llamada a `atan2`.

```

1 // CREATE/READ: Angulo con signo entre dos vectores, usando
// atan2 para
2 // evitar la inestabilidad numerica de acos
3 struct AnguloVectores {
4     struct Vector { double x, y; };
5
6     // READ: producto punto de dos vectores
7     static double punto(const Vector& a, const Vector& b) {

```

```

8     return a.x * b.x + a.y * b.y;
9 } // O(1)
10
11 // READ: producto cruz de dos vectores
12 static double cruz(const Vector& a, const Vector& b) {
13     return a.x * b.y - a.y * b.x;
14 } // O(1)
15
16 // READ: angulo con signo (en radianes) que hay que rotar a
17 // para
18 // alinearlo con b; positivo = antihorario, negativo =
19 // horario
20 static double angulo(const Vector& a, const Vector& b) {
21     return atan2(cruz(a, b), punto(a, b));
22 } // O(1)
23
24 //
25 // -----
26 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
27 // -----
28 // 1. Angulo NO firmado (solo magnitud, en [0, pi]): fabs(
29 //     angulo(a, b)).
30 // Util cuando no importa el sentido de giro, solo la
31 // abertura.
32 // 2. Angulo interno de un poligono en el vertice v, entre
33 // los vertices
34 // vecinos prev y next: angulo(Vector::desde(v, prev),
35 //     Vector::desde(v, next)),
36 // tomando el valor absoluto si el poligono no esta
37 // orientado de forma
38 // consistente.
39 // 3. Comparar SOLO el orden angular de puntos alrededor de
40 // un pivote (sin
41 // necesitar el angulo exacto): usar directamente el signo
42 // de cruz(a, b)
43 // como comparador de sort es mas rapido y mas exacto que
44 // llamar atan2
45 // para cada comparacion (ver Barrido Angular).
46 };

```

Ejemplo de funcionamiento:

```

1 AnguloVectores::Vector a{1, 0}, b{0, 1};
2
3 double resultado = AnguloVectores::angulo(a, b);
4 // resultado = pi/2 (aprox 1.5708)
5 // (b esta 90 grados en sentido antihorario respecto a a; hay
6 // que correrlo
// para confirmar el valor decimal exacto de pi/2 en el
// sistema usado)

```

Regla práctica: usa `atan2(cruz, punto)` en vez de `acos(punto/(norma_a*norma_b))` siempre que necesites el signo del ángulo o estabilidad numérica cerca de 0 o π . Si solo necesitas **comparar** el orden angular de varios vectores respecto a un pivote (no el ángulo exacto), ni siquiera hace falta `atan2` —el signo del producto cruz solo, usado como comparador, ya es suficiente y más rápido.

31.8 Tres Puntos Colineales

Idea: tres puntos A, B, C son colineales si están sobre la misma recta, lo cual ocurre exactamente cuando el producto cruz $\vec{AB} \times \vec{AC} = 0$ —el caso frontera de *Orientación de Tres Puntos*, donde no hay giro ni a la izquierda ni a la derecha. Sin embargo, colinealidad (estar sobre la misma recta infinita) es distinto de que C esté **entre** A y B sobre el segmento: para eso, además del producto cruz igual a cero, hace falta verificar que las coordenadas de C estén dentro del rango de caja delimitadora (*bounding box*) de A y B .

¿Por qué usarlo? Es una aplicación directa y frecuente de la prueba de orientación, pero se trata aparte porque el caso "colineal" suele ser el más propenso a errores en problemas de geometría: mezclar "están en la misma recta" con "uno está entre los otros dos" es una confusión común. Separar explícitamente ambas preguntas —colinealidad (producto cruz) y pertenencia

al segmento (bounding box)— evita bugs sutiles, especialmente en algoritmos de intersección de segmentos donde los casos colineales se manejan aparte (como ya se vio en *Intersección de Segmentos*).

Casos de uso:

- Verificar si tres puntos dados están alineados (como paso de validación de entrada).
- Detectar si un punto cae exactamente sobre un segmento (no solo sobre su recta extendida).
- Simplificar un polígono eliminando vértices redundantes que son colineales con sus vecinos.
- Caso especial en algoritmos de intersección de segmentos, cuando el producto cruz da exactamente cero.

Complejidad: $O(1)$ —un producto cruz más, en el peor caso, cuatro comparaciones para el chequeo de bounding box.

```

1 // CREATE/READ: Verificación de colinealidad de tres puntos, y
2 // de un punto colineal al segmento (no solo a la recta
3 // extendida)
4 struct TresPuntosColineales {
5     struct Punto { double x, y; };
6
7     // READ: producto cruz de (b-a) x (c-a)
8     static double cruz(const Punto& a, const Punto& b, const
9     Punto& c) {
10         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
11     } // O(1)
12
13     // READ: true si a, b, c son colineales (están sobre la misma
14     // recta)
15     static bool sonColineales(const Punto& a, const Punto& b,
16     const Punto& c) {
17         const double EPS = 1e-9;
18         return fabs(cruz(a, b, c)) < EPS;
19     } // O(1)
20
21     // READ: dado que a, b, c ya son colineales, verifica si c
22     // está DENTRO
23     // del segmento [a, b] (no solo sobre la recta que lo
24     // contiene)
25     static bool entreSegmento(const Punto& a, const Punto& b,
26     const Punto& c) {
27         const double EPS = 1e-9;
28         bool dentroX = min(a.x, b.x) - EPS <= c.x && c.x <= max(a.x,
29         b.x) + EPS;
30         bool dentroY = min(a.y, b.y) - EPS <= c.y && c.y <= max(a.y,
31         b.y) + EPS;
32         return dentroX && dentroY;
33     } // O(1)
34
35     // READ: combina ambas verificaciones: c está colineal Y
36     // dentro del
37     // segmento [a, b]
38     static bool puntoSobreSegmento(const Punto& a, const Punto& b,
39     const Punto& c) {
40         return sonColineales(a, b, c) && entreSegmento(a, b, c);
41     } // O(1)
42
43     //
44     // -----
45     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46     // -----
47
48     // 1. Coordenadas enteras: usar long long en cruz() y
49     // comparar contra 0
50     // exacto, sin EPS; entreSegmento también se vuelve
51     // comparación exacta.
52     // 2. Simplificación de polígono (eliminar vértices
53     // redundantes): recorrer
54     // los vértices consecutivos en tripletas (prev, actual,
55     // siguiente) y
56     // eliminar actual si sonColineales(prev, actual,
57     // siguiente) es true.
58     // 3. Ordenar puntos colineales sobre una misma recta: una
59     // vez confirmado
60     // que todos son colineales, se pueden ordenar por su
61     // proyección escalara

```

```

42 // sobre el vector director de la recta (producto punto
43 // contra un vector
44 // unitario en esa dirección), en vez de ordenar por x o y
45 // por separado
46 // (falla si la recta es vertical u horizontal exacta).
47 };

```

Ejemplo de funcionamiento:

```

1 TresPuntosColineales::Punto a{0, 0}, b{10, 0}, c{5, 0}, d{15,
2 0};
3
4 bool colineales = TresPuntosColineales::sonColineales(a, b, c);
5 // colineales = true (los tres están sobre el eje x)
6
7 bool dentro = TresPuntosColineales::entreSegmento(a, b, c);
8 // dentro = true (c=(5,0) está entre a=(0,0) y b=(10,0))
9
10 bool fuera = TresPuntosColineales::puntoSobreSegmento(a, b, d);
11 // fuera = false (d=(15,0) es colineal con a y b, pero está MAS
12 // ALLÁ de b,
13 // fuera del segmento [a,b])

```

Regla práctica: nunca uses solo el producto cruz igual a cero para concluir que un punto "está sobre el segmento" —eso solo confirma que está sobre la **recta infinita**. Siempre combina la prueba de colinealidad con la verificación de bounding box (**entreSegmento**) cuando la pregunta sea específicamente sobre pertenencia al segmento, no a la recta extendida.

31.9 Distancia de un Punto a una Línea/Segmento

Idea: la distancia de un punto P a una **recta infinita** definida por A y B se obtiene proyectando $\vec{AP}$ sobre la dirección perpendicular a $\vec{AB}$: usando el producto cruz de *Vector Geométrico*, esa distancia es $\frac{|\vec{AB} \times \vec{AP}|}{|\vec{AB}|}$

—el área del paralelogramo que forman los dos vectores, dividida entre la base. Para la distancia a un **segmento** (no a la recta infinita), hay que distinguir tres casos según dónde cae la proyección de P sobre la línea: si cae dentro de $[A, B]$, la distancia es la misma fórmula perpendicular; si cae antes de A o después de B , la distancia real es simplemente la distancia euclidiana al extremo más cercano.

¿Por qué usarlo? Confundir distancia a la recta con distancia al segmento es un error común: la fórmula de la recta infinita subestima la distancia real cuando la proyección del punto cae fuera del segmento. La forma correcta usa el producto punto (visto en *Vector Geométrico*) para determinar en qué "zona" cae la proyección —antes de A , dentro de $[A, B]$, o después de B — y solo aplica la fórmula perpendicular en el caso intermedio, cayendo de vuelta a *Distancia Euclidiana* directa contra el extremo en los otros dos casos.

Casos de uso:

- Distancia mínima de un punto a un obstáculo representado como segmento o polilínea.
- Verificar si un punto está a menos de r unidades de una pared o borde (colisión punto-segmento).
- Cálculo auxiliar en *Closest Pair* cuando se generaliza a distancia punto-segmento en vez de punto-punto.
- Simplificación de polilíneas (algoritmo de Douglas-Peucker usa esta distancia para decidir qué puntos eliminar).

Complejidad: $O(1)$ —un producto punto, un producto cruz, y como mucho una llamada a `sqrt`.

```

1 // CREATE/READ: Distancia de un punto a una recta infinita y a
  un segmento
2 struct DistanciaPuntoLinea {
3     struct Punto { double x, y; };
4
5     // READ: producto punto de (b-a) con (p-a)
6     static double punto(const Punto& a, const Punto& b, const
      Punto& p) {
7         return (b.x - a.x) * (p.x - a.x) + (b.y - a.y) * (p.y - a.y
8         );
9     } // O(1)
10
11    // READ: producto cruz de (b-a) con (p-a)
12    static double cruz(const Punto& a, const Punto& b, const
      Punto& p) {
13        return (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (p.x - a.x
14        );
15    } // O(1)
16
17    // READ: distancia euclidiana entre dos puntos
18    static double distEuclidiana(const Punto& p, const Punto& q)
19    {
20        double dx = q.x - p.x, dy = q.y - p.y;
21        return sqrt(dx * dx + dy * dy);
22    } // O(1)
23
24    // READ: distancia de p a la RECTA INFINITA que pasa por a y
      b
25    static double distARecta(const Punto& a, const Punto& b,
      const Punto& p) {
26        double baseLen = distEuclidiana(a, b);
27        if (baseLen < 1e-9) return distEuclidiana(a, p); // a y b
      coinciden
28        return fabs(cruz(a, b, p)) / baseLen;
29    } // O(1)
30
31    // READ: distancia de p al SEGMENTO [a, b] (no a la recta
      extendida)
32    static double distASegmento(const Punto& a, const Punto& b,
      const Punto& p) {
33        double t = punto(a, b, p);
34        if (t <= 0) return distEuclidiana(a, p); // proyeccion cae
      antes de a
35        double baseLen2 = punto(a, b, b); // |ab|^2
36        if (t >= baseLen2) return distEuclidiana(b, p); //
      proyeccion cae despues de b
37        return distARecta(a, b, p); // proyeccion cae dentro de [a,
      b]
38    } // O(1)
39
40    //
41    // -----
42    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43    // -----
44
45    // 1. Punto de proyeccion exacto (no solo la distancia): una
      vez que se
46    // sabe que la proyeccion cae dentro de [a,b], el punto
      proyectado es
47    // a + (t / baseLen2) * (b - a), con t = punto(a,b,p).
48    // 2. Coordenadas enteras: punto() y cruz() dan resultado
      exacto con
49    // long long; solo distEuclidiana y la division final
      requieren double.
50    // 3. Distancia de un punto a una POLILINEA (varios segmentos
      consecutivos):
51    // tomar el minimo de distASegmento sobre cada segmento
      consecutivo de
52    // la polilinea, O(k) para k segmentos.
53 };

```

Ejemplo de funcionamiento:

```

1 DistanciaPuntoLinea::Punto a{0, 0}, b{10, 0}, p{5, 3}, q{15,
  3};
2
3 double d1 = DistanciaPuntoLinea::distASegmento(a, b, p);
4 // d1 = 3 (la proyeccion de p cae dentro de [a,b], en (5,0);
  distancia perpendicular = 3)
5
6 double d2 = DistanciaPuntoLinea::distASegmento(a, b, q);
7 // d2 = distEuclidiana(b, q) = sqrt(5^2 + 3^2) = sqrt(34)
8 // (la proyeccion de q cae despues de b=(10,0), asi que la
  distancia real es
9 // al extremo b, no la perpendicular a la recta; hay que
  correrlo para
10 // confirmar el valor decimal exacto de sqrt(34))

```

Regla práctica: usa `distARecta` solo cuando el

problema explícitamente hable de una línea infinita (poco común en CP); en la gran mayoría de los casos donde se menciona un "segmento", "pared" o "borde", usa `distASegmento`, que maneja correctamente los casos donde la proyección cae fuera del segmento —olvidar esta distinción es la fuente de error más común en problemas de distancia punto-segmento.

31.10 Reflexión de un Punto sobre una Línea

Idea: reflejar un punto P sobre una recta definida por A y B significa encontrar su "imagen espejo" al otro lado de la recta, a la misma distancia perpendicular. El procedimiento se apoya directamente en *Distancia de un Punto a una Línea/Segmento*: primero se calcula la proyección ortogonal Q de P sobre la recta (usando el producto punto para hallar t , igual que en el cálculo de distancia a segmento), y luego el punto reflejado es $P' = 2Q - P$ —es decir, Q es el punto medio entre P y su reflejo P' .

¿Por qué usarlo? Es la forma directa de resolver problemas de simetría sin recurrir a matrices de transformación generales o a trigonometría explícita —basta con una proyección (producto punto) y una resta vectorial. A diferencia de la *Rotación de un Punto*, que gira alrededor de un pivote por un ángulo arbitrario, la reflexión es una transformación fija determinada completamente por la recta de simetría, sin parámetro de ángulo.

Casos de uso:

- Problemas de "camino más corto que toca una pared" (reflejar el destino y trazar una línea recta).
- Verificar si una figura es simétrica respecto a un eje dado.
- Generar el punto opuesto en construcciones geométricas con simetría axial.
- Simulación de rebotes de un rayo de luz o trayectoria en un espejo/pared recta.

Complejidad: $O(1)$ —una proyección (producto punto) más una resta vectorial.

```

1 // CREATE/READ: Reflexion de un punto sobre una recta definida
  por dos puntos
2 struct ReflexionPunto {
3     struct Punto { double x, y; };
4
5     // READ: producto punto de (b-a) con (p-a)
6     static double punto(const Punto& a, const Punto& b, const
      Punto& p) {
7         return (b.x - a.x) * (p.x - a.x) + (b.y - a.y) * (p.y - a.y
8         );
9     } // O(1)
10
11    // READ: proyeccion ortogonal de p sobre la recta que pasa
      por a y b
12    static Punto proyeccion(const Punto& a, const Punto& b, const
      Punto& p) {
13        double baseLen2 = punto(a, b, b); // |ab|^2
14        double t = punto(a, b, p) / baseLen2;
15        return Punto{a.x + t * (b.x - a.x), a.y + t * (b.y - a.y)};
16    } // O(1)
17
18    // READ: reflexion de p sobre la recta que pasa por a y b
19    static Punto reflejar(const Punto& a, const Punto& b, const
      Punto& p) {
20        Punto q = proyeccion(a, b, p);
21        return Punto{2 * q.x - p.x, 2 * q.y - p.y};
22    } // O(1)
23
24    //
25    // -----

```

```

24 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25 //
26 // 1. Reflexion sobre el eje x o eje y (casos particulares):
27 //     reflejar
28 //     sobre el eje x es simplemente (p.x, -p.y); sobre el eje
29 //     y es
30 //     (-p.x, p.y). Mas rapido que usar el caso general si el
31 //     eje de
32 //     simetria coincide con uno de los ejes cartesianos.
33 // 2. Camino mas corto que toca una linea (problema clasico):
34 //     para ir de
35 //     P a Q tocando la recta L, se refleja Q (o P) sobre L
36 //     con reflejar(),
37 //     y la distancia minima es distEuclidiana(P, reflejo de Q
38 //     ); el punto
39 //     de contacto es la interseccion entre el segmento P-
40 //     reflejo(Q) y L.
41 // 3. Reflexion sobre un PUNTO (no una recta): caso distinto
42 //     y mas simple,
43 //     P' = 2*C - P donde C es el punto de reflexion (equivale
44 //     a una
45 //     rotacion de 180 grados alrededor de C, ver Rotacion de
46 //     un Punto).
47 };

```

Ejemplo de funcionamiento:

```

1 ReflexionPunto::Punto a{0, 0}, b{10, 0}, p{3, 4};
2
3 auto resultado = ReflexionPunto::reflejar(a, b, p);
4 // resultado = (3, -4)
5 // (la recta a-b es el eje x; reflejar (3,4) sobre el eje x da
6 // (3,-4))

```

Nota: si la recta de reflexión coincide con uno de los ejes cartesianos, el caso particular mencionado en la modificación 1 del código es más directo y evita el cálculo de proyección completo —pero el método general funciona igual de bien para cualquier recta arbitraria.

Regla práctica: usa reflexión cuando el problema tenga simetría axial fija respecto a una recta conocida (más común en problemas de "camino más corto tocando una pared"); si en cambio se necesita girar un punto un ángulo arbitrario alrededor de un pivote, sin relación a ninguna recta de simetría, usa *Rotación de un Punto* en su lugar.

31.11 Rotación de un Punto (por Ángulo, alrededor de un Pivote)

Idea: rotar un punto P un ángulo θ alrededor de un pivote C se hace en tres pasos: trasladar P para que C quede en el origen ($P - C$), aplicar la matriz de rotación estándar $\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ sobre ese vector trasladado, y trasladar de vuelta sumando C . La matriz de rotación en sí sale de proyectar el vector sobre una nueva base girada θ radianes —exactamente la misma idea de descomponer en componentes que aparece en *Vector Geométrico*, aplicada a los vectores base $(1, 0)$ y $(0, 1)$ en vez de a un vector arbitrario.

¿Por qué usarlo? Es la transformación general de la que *Reflexión de un Punto sobre una Línea* es un caso especial no lineal (la reflexión no es una rotación, pero comparten la técnica de trasladar-transformar-destrasladar). A diferencia de la reflexión, que está fijada por una recta de simetría, la rotación tiene un parámetro libre (θ) y un pivote libre (C), lo que la hace la herramienta correcta cuando el problema pide girar figuras completas, no reflejarlas.

Casos de uso:

- Girar un polígono completo un ángulo dado alrededor de su centroide u otro pivote.

- Generar los vértices de un polígono regular rotando un vértice inicial en incrementos de $2\pi/n$.
- Simular el movimiento angular de un objeto (radar, aguja, brazo robótico) en pasos discretos.
- Normalizar la orientación de una figura antes de compararla con otra (rotar hasta alinear un eje de referencia).

Complejidad: $O(1)$ por rotación de un punto —dos evaluaciones trigonométricas ($\sin$, $\cos$) reutilizables si se rota el mismo ángulo varias veces, más un par de multiplicaciones y sumas.

```

1 // CREATE/READ: Rotacion de un punto un angulo theta alrededor
2 // de un pivote
3 struct RotacionPunto {
4     struct Punto { double x, y; };
5
6     // READ: rota p un angulo theta (en radianes, sentido
7     // antihorario)
8     // alrededor del pivote c
9     static Punto rotar(const Punto& p, const Punto& c, double
10     theta) {
11         double dx = p.x - c.x;
12         double dy = p.y - c.y;
13         double cosT = cos(theta);
14         double sinT = sin(theta);
15         double xRot = dx * cosT - dy * sinT;
16         double yRot = dx * sinT + dy * cosT;
17         return Punto{c.x + xRot, c.y + yRot};
18     } // O(1)
19
20 //
21 // -----
22 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
23 //
24 // -----
25 // 1. Rotar alrededor del origen (caso particular, c = (0,0)):
26 //     se puede
27 //     omitir la traslacion inicial y final, usando
28 //     directamente
29 //     xRot = p.x*cosT - p.y*sinT; yRot = p.x*sinT + p.y*cosT.
30 // 2. Rotaciones repetidas del MISMO angulo theta muchas
31 //     veces (por
32 //     ejemplo, generar los n vertices de un poligono regular):
33 //     calcular
34 //     cosT y sinT una sola vez fuera del bucle, en vez de
35 //     recalcularlos
36 //     en cada llamada a rotar().
37 // 3. Sentido horario en vez de antihorario: usar theta
38 //     negativo, o
39 //     equivalentemente intercambiar el signo de sinT en ambas
40 //     formulas
41 //     (dx*(-sinT) para xRot, dx*sinT invertido para yRot).
42 // 4. Poligono regular de n lados con radio r centrado en c:
43 //     generar
44 //     vertices como rotar(Punto{c.x + r, c.y}, c, 2*M_PI*i/n)
45 //     para
46 //     i = 0 hasta n-1.
47 };

```

Ejemplo de funcionamiento:

```

1 RotacionPunto::Punto p{1, 0}, c{0, 0};
2
3 auto resultado = RotacionPunto::rotar(p, c, M_PI / 2);
4 // resultado = (0, 1)
5 // (rotar (1,0) 90 grados antihorario alrededor del origen da
6 // (0,1);
7 // hay que correrlo para confirmar el valor decimal exacto por
8 // el uso
9 // de M_PI en punto flotante)

```

Nota: si el pivote coincide con el origen, omitir la traslación (ver modificación 1) ahorra dos sumas y dos restas por punto —insignificante para un solo punto, pero relevante si se rotan miles de puntos en un bucle.

Regla práctica: usa rotación cuando el problema pida girar un punto o figura un ángulo arbitrario alrededor de un pivote arbitrario; si en cambio el problema tiene una recta de simetría fija y pide el "espejo" de un

punto respecto a ella, eso no es una rotación sino *Reflexión de un Punto sobre una Línea* —ambas comparten la técnica de trasladar-transformar-destrasladar, pero resuelven preguntas distintas.

32 Triángulos

32.1 Ángulos de un Triángulo

Idea: dados los tres vértices A, B, C de un triángulo, el ángulo interno en cada vértice se calcula con *Ángulo entre Dos Vectores* aplicado a los dos lados que se encuentran en ese vértice: el ángulo en A es el ángulo entre $\vec{AB}$ y $\vec{AC}$, y análogamente para B y C . Alternativamente, si solo se conocen las tres longitudes de los lados a, b, c (sin coordenadas), se puede usar la **ley de cosenos** despejada: $\cos A = \frac{b^2 + c^2 - a^2}{2bc}$, donde el lado a es el opuesto al ángulo A . Ambos métodos son equivalentes; cuál usar depende de si el problema da coordenadas o solo longitudes.

¿Por qué usarlo? Cuando se tienen coordenadas, usar *Ángulo entre Dos Vectores* (con `atan2`) es preferible porque da el ángulo con signo y es numéricamente estable, como ya se explicó en esa subsección. Cuando solo se tienen las tres longitudes de los lados —sin coordenadas en el plano—, la ley de cosenos es la única opción directa, y conviene usar `acos` ahí porque el ángulo interno de un triángulo válido siempre está en $[0, \pi]$, rango donde `acos` no tiene la ambigüedad de signo que sí le importa a un ángulo con orientación.

Casos de uso:

- Clasificar un triángulo como acutángulo, rectángulo u obtusángulo según sus ángulos.
- Verificar si tres longitudes dadas forman un triángulo válido (todos los ángulos resultantes deben ser reales, es decir, el argumento de `acos` debe estar en $[-1, 1]$).
- Calcular ángulos internos como paso previo a fórmulas de área o de triangulación que los requieran.
- Determinar la orientación relativa de dos lados que comparten un vértice.

Complejidad: $O(1)$ por ángulo —ya sea vía *Ángulo entre Dos Vectores* ($O(1)$, un `atan2`) o vía ley de cosenos ($O(1)$, un `acos`).

```

1 // CREATE/READ: Angulos internos de un triangulo, por
  // coordenadas o por
2 // longitudes de lados (ley de cosenos)
3 struct AngulosTriangulo {
4     struct Punto { double x, y; };
5
6     // READ: producto punto de (b-a) con (c-a)
7     static double punto(const Punto& a, const Punto& b, const
  Punto& c) {
8         return (b.x - a.x) * (c.x - a.x) + (b.y - a.y) * (c.y - a.y)
9     };
10    } // O(1)
11
12    // READ: producto cruz de (b-a) con (c-a)
13    static double cruz(const Punto& a, const Punto& b, const
  Punto& c) {
14        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x)
15    };
16    } // O(1)
17
18    // READ: angulo interno en el vertice a, dado por los otros
  // dos vertices
19    // b y c, usando atan2 (estable, sin importar el rango)

```

```

18 static double anguloPorCoordenadas(const Punto& a, const
  Punto& b, const Punto& c) {
19     return fabs(atan2(cruz(a, b, c), punto(a, b, c)));
20 } // O(1)
21
22 // READ: angulo opuesto al lado de longitud "opuesto", dados
  // los otros
23 // dos lados "lado1" y "lado2" (ley de cosenos despejada)
24 static double anguloPorLados(double lado1, double lado2,
  double opuesto) {
25     double cosAngulo = (lado1*lado1 + lado2*lado2 - opuesto*
  opuesto) / (2*lado1*lado2);
26     cosAngulo = max(-1.0, min(1.0, cosAngulo)); // recorta
  // error de punto flotante
27     return acos(cosAngulo);
28 } // O(1)
29
30 //
31 // -----
32 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
33 // -----
34 // 1. Verificar validez de un triangulo dados tres lados:
  // ademas de la
35 // desigualdad triangular (cada lado menor que la suma de
  // los otros
36 // dos), el recorte de cosAngulo a [-1,1] en
  // anguloPorLados evita NaN
37 // por error de punto flotante en casos borde (triangulo
  // degenerado).
38 // 2. Clasificar el triangulo: si el mayor angulo es > pi/2
  // es obtusangulo,
39 // == pi/2 (con tolerancia EPS) es rectangulo, < pi/2 es
  // acutangulo;
40 // basta con calcular el angulo opuesto al lado mas largo.
41 // 3. Suma de angulos internos: por construccion siempre da
  // pi (180
42 // grados) en geometria euclidiana; sirve como chequeo de
  // cordura
43 // (sanity check) al depurar, calculando los tres angulos
  // y sumando.

```

Ejemplo de funcionamiento:

```

1 AngulosTriangulo::Punto a{0, 0}, b{4, 0}, c{0, 3};
2
3 double anguloA = AngulosTriangulo::anguloPorCoordenadas(a, b, c
4 );
5 // anguloA = pi/2 (aprox 1.5708)
6 // (triangulo rectangulo 3-4-5, el angulo recto esta en a; hay
  // que correrlo
7 // para confirmar el valor decimal exacto)
8
9 double anguloPorLados = AngulosTriangulo::anguloPorLados(4, 3,
  5);
10 // anguloPorLados = pi/2, mismo resultado via ley de cosenos
  // con los lados
11 // adyacentes a A (4 y 3) y el lado opuesto a A (la hipotenusa,
  // 5)

```

Regla práctica: si tienes coordenadas de los vértices, usa `anguloPorCoordenadas` (reutiliza *Ángulo entre Dos Vectores*); si solo tienes las tres longitudes de los lados, usa `anguloPorLados` y no olvides recortar el argumento de `acos` a $[-1, 1]$ para evitar NaN por error de punto flotante en triángulos casi degenerados.

32.2 Fórmula de Herón (Área por Lados)

Idea: dado un triángulo del que solo se conocen las longitudes de sus tres lados a, b, c (sin coordenadas), la fórmula de Herón calcula el área sin necesidad de conocer ningún ángulo: con el semiperímetro $s = \frac{a+b+c}{2}$, el área es $\sqrt{s(s-a)(s-b)(s-c)}$. Es un resultado puramente algebraico —se puede derivar combinando la ley de cosenos (para obtener un ángulo) con la fórmula de área $\frac{1}{2}ab \sin C$, pero el resultado final elimina la trigonometría por completo y deja todo en función de las tres longitudes.

¿Por qué usarlo? Cuando el problema da coordenadas de los vértices, *Área de un Polígono (Shoelace)* es

más directo y general (funciona para cualquier polígono, no solo triángulos). Herón es la herramienta correcta específicamente cuando **no hay coordenadas**, solo las tres longitudes de los lados —situación común en problemas donde el triángulo se describe por sus medidas, no por su posición en el plano.

Casos de uso:

- Calcular el área de un triángulo dado solo por sus tres lados (sin coordenadas).
- Verificar si tres longitudes forman un triángulo válido (el área resultante debe ser un número real positivo, es decir, el producto bajo la raíz debe ser ≥ 0).
- Como paso intermedio en fórmulas que requieren área y perímetro juntos (por ejemplo, el radio del círculo inscrito, $r = \text{área}/s$).
- Cálculo de área en problemas donde los datos de entrada son distancias medidas, no coordenadas cartesianas.

Complejidad: $O(1)$ —tres sumas/restas para el semiperímetro y los factores, una multiplicación de cuatro términos, y una raíz cuadrada.

Polígono (Shoelace) directamente —es igual de rápido, más general, y evita el paso intermedio de calcular tres distancias euclidianas antes de aplicar Herón.

32.3 Círculo Circunscrito en un Triángulo

Idea: el círculo circunscrito (circumcírculo) de un triángulo es el único círculo que pasa por sus tres vértices. Su centro, el **circuncentro**, es el punto equidistante de los tres vértices, y se encuentra como la intersección de las mediatrices (perpendiculares por el punto medio) de cualquier par de lados. Resolviendo el sistema de ecuaciones que impone "distancia igual a los tres vértices" se llega a una fórmula cerrada en términos de determinantes 2×2 , sin necesidad de intersectar rectas explícitamente. El radio es simplemente la distancia del circuncentro a cualquiera de los tres vértices.

¿Por qué usarlo? A diferencia del caso particular de un triángulo equilátero —donde el circuncentro coincide con el centroide, el promedio simple de los tres vértices, $O(1)$ trivial—, un triángulo arbitrario no tiene esa coincidencia: centroide, circuncentro, incentro y ortocentro son puntos distintos en general. La fórmula con determinantes cubre el caso general sin necesidad de distinguir tipos de triángulo, y evita resolver el sistema de mediatrices manualmente (que requeriría manejar el caso de mediatriz vertical aparte, el mismo problema que motivó la forma general de *Ecuación de una Línea Dados Dos Puntos*).

Casos de uso:

- Encontrar el centro y radio del círculo mínimo que pasa por tres puntos dados (no necesariamente el círculo mínimo que los *cubre*, ver nota).
- Problemas de triangulación de Delaunay, donde la propiedad definitoria es que ningún otro punto cae dentro del circumcírculo de cada triángulo.
- Construcciones geométricas clásicas (bisectores perpendiculares, geometría de triángulos).
- Verificar si cuatro puntos son concíclicos (el cuarto debe estar exactamente sobre el circumcírculo de los otros tres).

Complejidad: $O(1)$ —el sistema de ecuaciones se resuelve con una fórmula cerrada de determinantes 2×2 , sin iteración.

```
1 // CREATE/READ: Area de un triangulo dado por sus tres lados (
  formula de Heron)
2 struct FormulaHeron {
3   // READ: area del triangulo de lados a, b, c; retorna -1 si
  los lados
4   // no forman un triangulo valido (desigualdad triangular
  violada)
5   static double area(double a, double b, double c) {
6     if (a + b <= c || a + c <= b || b + c <= a) return -1; //
      triangulo invalido
7     double s = (a + b + c) / 2.0;
8     double producto = s * (s - a) * (s - b) * (s - c);
9     if (producto < 0) producto = 0; // recorta error de punto
      flotante
10    return sqrt(producto);
11  } // O(1)
12
13  //
14  // MODIFICACIONES Y COMENTARIOS CON LOS USOS
15  //
16  // 1. Radio del círculo inscrito: r = area(a,b,c) / s, con
17  //   s = (a+b+c)/2 el mismo semiperimetro usado internamente.
18  // 2. Radio del círculo circunscrito: R = (a*b*c) / (4*area(a,
19  //   b,c)),
20  //   formula derivada de la ley de senos generalizada (ver
21  //   Círculo
22  //   Circunscrito en un Triangulo para el caso con
23  //   coordenadas).
24  // 3. Coordenadas disponibles en vez de lados: si el
25  //   triangulo viene dado
26  //   por vertices, es mas directo y mas preciso usar
27  //   Shoelace
28  //   directamente en vez de calcular las tres distancias
29  //   euclidianas y
30  //   despues aplicar Heron (evita acumular error en un paso
31  //   intermedio).
32  };
```

Ejemplo de funcionamiento:

```
1 double resultado = FormulaHeron::area(3, 4, 5);
2 // resultado = 6
3 // (triangulo rectangulo 3-4-5, area = base*altura/2 = 3*4/2 =
4 //   6,
5 //   Heron da el mismo resultado sin usar el angulo recto
6 //   explicitamente)
```

Regla práctica: usa Herón únicamente cuando el triángulo se te da por sus tres longitudes sin coordenadas; si tienes coordenadas de los vértices, usa *Área de un*

```
1 // CREATE/READ: Circuncentro y radio del círculo circunscrito
  de un triangulo
2 // (caso general, valido para cualquier triangulo no degenerado
3 //   )
4 struct CirculoCircunscrito {
5   struct Punto { double x, y; };
6
7   // READ: producto cruz de (b-a) x (c-a), usado para detectar
8   //   triangulo degenerado
9   static double cruz(const Punto& a, const Punto& b, const
10  //   Punto& c) {
11    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
12  //   );
13  } // O(1)
14
15  // READ: calcula el circuncentro de a, b, c mediante formula
16  //   de
17  //   determinantes (interseccion de mediatrices resuelta
18  //   algebraicamente)
19  static Punto circuncentro(const Punto& a, const Punto& b,
20  //   const Punto& c) {
21    double d = 2.0 * (a.x * (b.y - c.y) + b.x * (c.y - a.y) + c
22  //   .x * (a.y - b.y));
```

```

15 // d == 0 significa que a, b, c son colineales: no hay
    circuncentro
16 double ax2ay2 = a.x*a.x + a.y*a.y;
17 double bx2by2 = b.x*b.x + b.y*b.y;
18 double cx2cy2 = c.x*c.x + c.y*c.y;
19 double ux = (ax2ay2 * (b.y - c.y) + bx2by2 * (c.y - a.y) +
    cx2cy2 * (a.y - b.y)) / d;
20 double uy = (ax2ay2 * (c.x - b.x) + bx2by2 * (a.x - c.x) +
    cx2cy2 * (b.x - a.x)) / d;
21 return Punto{ux, uy};
22 } // O(1)
23
24 // READ: radio del círculo circunscrito (distancia del
    circuncentro a
25 // cualquier vertice, aquí se usa a)
26 static double radio(const Punto& a, const Punto& b, const
    Punto& c) {
27     Punto o = circuncentro(a, b, c);
28     double dx = a.x - o.x, dy = a.y - o.y;
29     return sqrt(dx*dx + dy*dy);
30 } // O(1)
31
32 //
33 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
34 //
35 // 1. Caso equilateral (atajo, NO usar para triangulos
    generales): el
36 // circuncentro coincide con el centroide, ((a.x+b.x+c.x)
    /3,
37 // (a.y+b.y+c.y)/3), y el radio es lado / sqrt(3). Solo
    valido cuando
38 // se garantiza que el triangulo es equilatero.
39 // 2. Triangulo degenerado (colineal): si cruz(a,b,c) == 0 (
    con EPS), d
40 // en circuncentro() da 0 y la formula produce division
    por cero /
41 // NaN; verificar colinealidad ANTES de llamar
    circuncentro().
42 // 3. Radio via lados y area (alternativa sin circuncentro
    explicito):
43 // R = (lado_a * lado_b * lado_c) / (4 * area),
    reutilizando
44 // Formula de Heron para el area si solo se necesita el
    radio y no
45 // el centro exacto.
46 };

```

Ejemplo de funcionamiento:

```

1 CirculoCircunscrito::Punto a{0, 0}, b{4, 0}, c{0, 3};
2
3 auto centro = CirculoCircunscrito::circuncentro(a, b, c);
4 // centro = (2, 1.5)
5 // (triangulo rectangulo 3-4-5; el circuncentro de un triangulo
    rectangulo
6 // siempre cae exactamente en el punto medio de la hipotenusa,
    aquí el
7 // punto medio de b=(4,0) y c=(0,3) es (2, 1.5), lo cual
    confirma el resultado)
8
9 double r = CirculoCircunscrito::radio(a, b, c);
10 // r = 2.5
11 // (para un triangulo rectangulo, el radio circunscrito es
    siempre la
12 // mitad de la hipotenusa: 5/2 = 2.5)

```

Nota: asumí que necesitas el caso general (no solo equilátero, que era tu ejemplo original) porque es estrictamente más útil en CP —el caso equilátero queda como atajo $O(1)$ documentado en la modificación 1 del código. Si en algún problema se garantiza que el triángulo es equilátero, usa ese atajo en vez de la fórmula de determinantes completa.

Regla práctica: usa esta fórmula de determinantes para el circuncentro general; verifica primero que el triángulo no sea degenerado ($\text{cruz}(a,b,c) \neq 0$) para evitar división por cero. Si solo necesitas el radio y ya tienes las tres longitudes de los lados (por ejemplo, después de calcular *Fórmula de Herón*), la fórmula $R = \frac{abc}{4 \cdot \text{área}}$ es más directa y evita calcular el centro explícitamente.

32.4 Punto Dentro de un Triángulo

Idea: un punto P está dentro del triángulo ABC si y solo si queda del **mismo lado** de las tres aristas orientadas AB , BC , CA —es decir, si el signo de *Orientación de Tres Puntos* aplicado a (A, B, P) , (B, C, P) y (C, A, P) es el mismo para los tres (todos positivos o todos negativos, según el sentido de recorrido del triángulo). Si alguno de los tres productos cruz da signo distinto a los demás, P cae fuera de al menos una arista y por lo tanto fuera del triángulo.

¿Por qué usarlo? Es la aplicación más directa y económica de *Orientación de Tres Puntos* a un caso concreto: tres evaluaciones de producto cruz, sin necesidad de trigonometría, áreas parciales, o coordenadas baricéntricas explícitas (aunque el método es equivalente a verificar que las coordenadas baricéntricas de P sean todas no negativas). Es un caso particular y más simple de *Punto Dentro de un Polígono*, que en general requiere *ray casting*; para un triángulo (siempre convexo), tres chequeos de orientación bastan y son más baratos.

Casos de uso:

- Verificar pertenencia de un punto a un triángulo, como paso base antes de generalizar a polígonos convexos.
- Localización de punto en una triangulación (mesh), recorriendo triángulos y probando cuál lo contiene.
- Pruebas de colisión simples en gráficos 2D o videojuegos con geometría triangular.
- Interpolación baricéntrica (una vez confirmado que el punto está dentro, los mismos productos cruz dan los pesos baricéntricos).

Complejidad: $O(1)$ —tres evaluaciones de producto cruz y una comparación de signos.

```

1 // CREATE/READ: Verifica si un punto esta dentro de un
    triangulo mediante
2 // tres pruebas de orientacion (mismo signo en las tres aristas
    )
3 struct PuntoDentroTriangulo {
4     struct Punto { double x, y; };
5
6     // READ: producto cruz de (b-a) x (c-a)
7     static double cruz(const Punto& a, const Punto& b, const
        Punto& c) {
8         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
        );
9     } // O(1)
10
11     // READ: true si p esta dentro o sobre el borde del triangulo
        abc,
12     // sin importar si abc esta orientado CW o CCW
13     static bool dentro(const Punto& a, const Punto& b, const
        Punto& c, const Punto& p) {
14         double d1 = cruz(a, b, p);
15         double d2 = cruz(b, c, p);
16         double d3 = cruz(c, a, p);
17         const double EPS = 1e-9;
18         bool tieneNegativo = (d1 < -EPS) || (d2 < -EPS) || (d3 < -
            EPS);
19         bool tienePositivo = (d1 > EPS) || (d2 > EPS) || (d3 > EPS)
            ;
20         // si NO hay a la vez signos positivos y negativos, p esta
            dentro
21         // (o sobre el borde, si algun di es aproximadamente 0)
22         return !(tieneNegativo && tienePositivo);
23     } // O(1)
24
25 //
26 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
27 //

```

```

28 // 1. Distinguir "estrictamente dentro" de "sobre el borde":
29 // si se
30 // necesita excluir el borde, verificar ademas que ninguno
31 // de d1, d2,
32 // d3 este dentro del rango [-EPS, EPS] (es decir, que los
33 // tres sean
34 // estrictamente del mismo signo, sin ceros).
35 // 2. Coordenadas enteras: usar long long en cruz() y
36 // comparar contra 0
37 // exacto en vez de EPS, ya que el resultado es exacto con
38 // enteros.
39 // 3. Coordenadas baricentricas: una vez calculados d1, d2,
40 // d3 (todas del
41 // mismo signo), los pesos baricentricos normalizados son
42 // d2/(d1+d2+d3), d3/(d1+d2+d3), d1/(d1+d2+d3) para a, b,
43 // c
44 // respectivamente (util para interpolar valores definidos
45 // en los
46 // vertices, como en shading de graficos 3D).
47 };

```

Ejemplo de funcionamiento:

```

1 PuntoDentroTriangulo::Punto a{0, 0}, b{4, 0}, c{0, 4}, p{1, 1},
2 q{3, 3};
3
4 bool dentroP = PuntoDentroTriangulo::dentro(a, b, c, p);
5 // dentroP = true (p=(1,1) esta dentro del triangulo)
6
7 bool dentroQ = PuntoDentroTriangulo::dentro(a, b, c, q);
8 // dentroQ = false (q=(3,3) esta fuera, mas alla de la
9 // hipotenusa b-c)

```

Regla práctica: usa esta prueba de tres orientaciones para triángulos específicamente —es más barata que *Punto Dentro de un Polígono (Ray Casting)*, que es la herramienta general para polígonos con cualquier número de lados. Si el triángulo forma parte de una triangulación más grande, esta misma prueba es la base para localizar en qué triángulo cae un punto dado.

33 Círculos

33.1 Longitud de Arco y Área de un Sector Circular

Idea: un sector circular es la "porción de pizza" delimitada por dos radios y el arco entre ellos, definida por un ángulo central θ (en radianes) sobre un círculo de radio r . Como la circunferencia completa ($2\pi r$ de longitud, πr^2 de área) corresponde a un ángulo completo de 2π radianes, cualquier sector es simplemente una fracción proporcional: la longitud de arco es $L = r\theta$ y el área del sector es $A = \frac{1}{2}r^2\theta$. Ambas fórmulas se derivan de la misma razón $\theta/2\pi$ aplicada a la circunferencia y al área totales, respectivamente.

¿Por qué usarlo? Es la forma más directa de estas dos cantidades cuando se trabaja con ángulos en radianes —evita el paso intermedio de convertir a grados y aplicar reglas de tres, y generaliza sin casos especiales a cualquier ángulo entre 0 y 2π . Vale la pena notar la diferencia de exponente entre ambas fórmulas: la longitud de arco escala linealmente con r , mientras que el área escala con r^2 —el mismo patrón que la circunferencia ($2\pi r$) y el área del círculo completo (πr^2).

Casos de uso:

- Calcular la distancia recorrida por un objeto que se mueve sobre un arco (radar, engranaje, trayectoria circular).
- Área de una región delimitada por dos radios y un arco, como paso intermedio en problemas de intersección de círculos.

- Problemas de "porción de pizza" o reparto proporcional de un círculo.
- Cálculo de área de un segmento circular (sector menos el triángulo que forman los dos radios y la cuerda), usado en *Intersección de Dos Círculos*.

Complejidad: $O(1)$ —una multiplicación para cada fórmula.

```

1 // CREATE/READ: Longitud de arco y area de un sector circular
2 // dado el radio
3 // y el angulo central en radianes
4 struct SectorCircular {
5     // READ: longitud de arco para un sector de radio r y angulo
6     // theta (radianes)
7     static double longitudArco(double r, double theta) {
8         return r * theta;
9     } // O(1)
10
11     // READ: area del sector de radio r y angulo theta (radianes)
12     static double areaSector(double r, double theta) {
13         return 0.5 * r * r * theta;
14     } // O(1)
15
16     // READ: area del segmento circular (sector menos el
17     // triangulo r-r-cuerda),
18     // la region entre el arco y la cuerda que lo subtiende
19     static double areaSegmento(double r, double theta) {
20         return areaSector(r, theta) - 0.5 * r * r * sin(theta);
21     } // O(1)
22
23     //
24     // -----
25     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
26     //
27     // -----
28     // 1. Angulo dado en grados: convertir primero a radianes con
29     // theta_rad = theta_grados * M_PI / 180.0 antes de llamar
30     // cualquiera
31     // de las funciones.
32     // 2. Perimetro total del sector (no solo el arco):
33     // longitudArco(r, theta)
34     // + 2*r, sumando los dos radios que cierran la "porcion
35     // de pizza".
36     // 3. area Segmento requiere theta en [0, 2*pi]; si theta >
37     // pi (segmento
38     // "mayor", mas de medio circulo), la resta sigue siendo
39     // valida porque
40     // sin(theta) ya maneja el signo correctamente para ese
41     // rango.
42 };

```

Ejemplo de funcionamiento:

```

1 double r = 4.0, theta = M_PI / 2; // sector de 90 grados
2
3 double arco = SectorCircular::longitudArco(r, theta);
4 // arco = 2*pi (aprox 6.2832)
5 // (un cuarto de la circunferencia completa 2*pi*r = 8*pi; hay
6 // que correrlo
7 // para confirmar el valor decimal exacto)
8
9 double area = SectorCircular::areaSector(r, theta);
10 // area = 4*pi (aprox 12.566)
11 // (un cuarto del area total pi*r^2 = 16*pi; hay que correrlo
12 // para
13 // confirmar el valor decimal exacto)

```

Regla práctica: usa estas fórmulas directamente cuando el ángulo esté en radianes y conozcas el radio; el área del segmento circular (arco menos cuerda) es el bloque reutilizable que hace falta como paso intermedio en *Intersección de Dos Círculos*, donde el área de solape se descompone en dos segmentos circulares, uno por cada círculo.

33.2 Intersección de Dos Círculos

Idea: dos círculos de centros C_1, C_2 y radios r_1, r_2 , separados por una distancia $d = |C_1C_2|$, se intersectan en 0, 1, 2 puntos, o son el mismo círculo, dependiendo de cómo se comparan d , $r_1 + r_2$ y $|r_1 - r_2|$. Cuando hay

dos puntos de intersección, se encuentran resolviendo el sistema de las dos ecuaciones de circunferencia: se calcula la distancia a desde C_1 hasta el punto medio de la cuerda que une las dos intersecciones (usando el teorema de Pitágoras generalizado, análogo a la ley de cosenos), luego la altura h de ese punto medio a cada intersección, y finalmente se desplaza perpendicularmente desde ese punto medio usando un vector normal a C_1C_2 —la misma idea de "rotar 90 grados" que aparece en *Ecuación de una Línea Dados Dos Puntos*.

¿Por qué usarlo? Es un problema con varios casos borde que hay que descartar explícitamente antes de aplicar la fórmula: círculos demasiado separados ($d > r_1 + r_2$), uno dentro del otro sin tocarse ($d < |r_1 - r_2|$), tangencia externa o interna (exactamente un punto), y círculos concéntricos con igual radio (infinitos puntos, caso degenerado). Manejar estos casos aparte, antes de la fórmula general, evita raíces cuadradas de números negativos o divisiones por cero cuando $d = 0$.

Casos de uso:

- Encontrar los puntos de intersección entre dos círculos, como paso base en geometría de círculos.
- Calcular el área de solape entre dos círculos (usando *Longitud de Arco y Área de un Sector Circular* sobre cada uno).
- Problemas de cobertura de señal o rango (¿dos zonas circulares se superponen?).
- Construcciones geométricas clásicas (encontrar puntos equidistantes a dos centros dados, base de la construcción de la mediatriz con regla y compás).

Complejidad: $O(1)$ —un número constante de operaciones aritméticas y, en el peor caso, dos llamadas a `sqrt` y dos a `acos` (para el área de solape).

```
1 // CREATE/READ: Interseccion de dos circulos: puntos de corte y
  // area de
2 // solape (union de dos segmentos circulares)
3 struct InterseccionCirculos {
4     struct Punto { double x, y; };
5
6     static double dist(const Punto& a, const Punto& b) {
7         double dx = b.x - a.x, dy = b.y - a.y;
8         return sqrt(dx*dx + dy*dy);
9     } // O(1)
10
11     // READ: clasifica la relacion entre los dos circulos ANTES
12     // de calcular
13     // puntos de interseccion, para evitar sqrt de numero
14     // negativo
15     // retorna: 0 = separados (sin interseccion), 1 = uno dentro
16     // del otro
17     // (sin tocarse), 2 = tangentes (1 punto), 3 = se cruzan (2
18     // puntos),
19     // 4 = concéntricos con mismo radio (infinitos puntos, mismo
20     // circulo)
21     static int clasificar(const Punto& c1, double r1, const Punto
22     & c2, double r2) {
23         double d = dist(c1, c2);
24         const double EPS = 1e-9;
25         if (d < EPS && fabs(r1 - r2) < EPS) return 4; // mismo
26         // circulo
27         if (d > r1 + r2 + EPS) return 0; // separados
28         if (d < fabs(r1 - r2) - EPS) return 1; // uno
29         // dentro del otro
30         if (fabs(d - (r1 + r2)) < EPS || fabs(d - fabs(r1 - r2)) <
31         EPS) return 2; // tangentes
32         return 3; // se cruzan en dos puntos
33     } // O(1)
34
35     // READ: calcula los (hasta) dos puntos de interseccion; SOLO
36     // valido
37     // cuando clasificar() devuelve 2 o 3 (llamar clasificar
38     // primero)
```

```
28 static vector<Punto> puntosInterseccion(const Punto& c1,
29 double r1, const Punto& c2, double r2) {
30     double d = dist(c1, c2);
31     // a = distancia de c1 al punto medio de la cuerda de
32     // interseccion
33     double a = (r1*r1 - r2*r2 + d*d) / (2*d);
34     double hCuad = r1*r1 - a*a;
35     double h = (hCuad < 0) ? 0 : sqrt(hCuad); // recorta error
36     // de punto flotante
37     // punto medio de la cuerda, sobre la linea c1-c2
38     double mx = c1.x + a * (c2.x - c1.x) / d;
39     double my = c1.y + a * (c2.y - c1.y) / d;
40     // vector perpendicular a (c2 - c1), normalizado por h/d
41     double dx = (c2.y - c1.y) * (h / d);
42     double dy = -(c2.x - c1.x) * (h / d);
43     if (h < 1e-9) return { Punto{mx, my} }; // tangentes: un
44     // solo punto
45     return { Punto{mx + dx, my + dy}, Punto{mx - dx, my - dy}
46     };
47 } // O(1)
48
49 // READ: area de solape (union de dos segmentos circulares);
50 // asume que
51 // clasificar() ya confirmo caso 3 (se cruzan) o 1 (uno
52 // dentro del otro)
53 static double areaSolape(const Punto& c1, double r1, const
54 Punto& c2, double r2) {
55     double d = dist(c1, c2);
56     int caso = clasificar(c1, r1, c2, r2);
57     if (caso == 0) return 0;
58     // sin solape
59     if (caso == 1) return M_PI * min(r1, r2) * min(r1, r2);
60     // uno dentro del otro
61     // angulo central en c1 y en c2 subtendido por la cuerda
62     // comun
63     double theta1 = 2 * acos(max(-1.0, min(1.0, (r1*r1 + d*d -
64     r2*r2) / (2*r1*d))));
65     double theta2 = 2 * acos(max(-1.0, min(1.0, (r2*r2 + d*d -
66     r1*r1) / (2*r2*d))));
67     double seg1 = 0.5 * r1*r1 * (theta1 - sin(theta1));
68     double seg2 = 0.5 * r2*r2 * (theta2 - sin(theta2));
69     return seg1 + seg2;
70 } // O(1)
71
72 //
73 // -----
74 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
75 // -----
76
77 // 1. SIEMPRE llamar clasificar() antes que
78 // puntosInterseccion() o
79 // areaSolape(): estas dos ultimas asumen que ya se
80 // descarto el caso
81 // de circulos separados o concéntricos con mismo radio,
82 // de lo
83 // contrario hay division por cero (d == 0) o resultados
84 // sin sentido.
85 // 2. areaSolape reutiliza directamente areaSegmento de
86 // Longitud de Arco
87 // y Area de un Sector Circular: seg1 y seg2 son
88 // exactamente esa
89 // formula aplicada a theta1 y theta2 respectivamente.
90 // 3. Radio y distancia con coordenadas enteras: d, a, h
91 // siguen
92 // requiriendo double por las raices cuadradas
93 // involucradas; no hay
94 // forma de mantener este calculo en aritmetica entera
95 // exacta.
```

Ejemplo de funcionamiento:

```
1 InterseccionCirculos::Punto c1{0, 0}, c2{4, 0};
2 double r1 = 3, r2 = 3;
3
4 int caso = InterseccionCirculos::clasificar(c1, r1, c2, r2);
5 // caso = 3 (se cruzan en dos puntos, ya que |r1-r2|=0 < d=4 <
6 // r1+r2=6)
7
8 auto puntos = InterseccionCirculos::puntosInterseccion(c1, r1,
9 c2, r2);
10 // puntos = {(2, h), (2, -h)} con h = sqrt(3^2 - 2^2) = sqrt(5)
11 // (por simetria, a = (9-9+16)/8 = 2, el punto medio de la
12 // cuerda cae en
13 // x=2; hay que correrlo para confirmar el valor decimal
14 // exacto de h)
```

Nota: `puntosInterseccion` y `areaSolape` no validan internamente los casos degenerados (círculos separados, uno dentro del otro sin tocarse, o concéntricos) —es responsabilidad de quien llama invocar `clasificar()`

primero y actuar según el resultado, tal como se indica en la modificación 1 dentro del código.

Regla práctica: nunca llames directamente a `puntosInterseccion` o calcules el área de solape sin antes clasificar la relación entre los dos círculos con `clasificar()` —los casos borde (separados, uno dentro del otro, concéntricos) producen NaN o resultados incorrectos si se ignoran, y son la fuente de error más común en este tipo de problema.

34 Polígonos

34.1 Área de un Polígono (Shoelace)

Idea: dado un polígono simple con vértices $P_0, P_1, \dots, P_{n-1}$ en orden (sentido horario o antihorario, consecutivos a lo largo del contorno), el área con signo es $\frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i)$, tomando los índices módulo n para cerrar el polígono. Cada término $x_i y_{i+1} - x_{i+1} y_i$ es exactamente el producto cruz $\vec{OP}_i \times \vec{OP}_{i+1}$ visto en *Orientación de Tres Puntos*, tomando el origen como pivote implícito: la fórmula suma las áreas con signo de los triángulos que forma el origen con cada arista consecutiva, y las contribuciones se cancelan fuera del polígono y se acumulan dentro, dejando exactamente el área encerrada.

¿Por qué usarlo? Es la generalización directa de *Fórmula de Herón* a cualquier polígono simple (no solo triángulos), y no requiere que el polígono sea convexo. A diferencia de triangular manualmente el polígono y sumar áreas de triángulos individuales, Shoelace procesa los vértices en un único recorrido lineal sin necesidad de descomponer la figura. El signo del resultado antes de tomar valor absoluto además revela la orientación del polígono: positivo si los vértices están en sentido antihorario (CCW), negativo si horario (CW) —la misma convención fijada en *Orientación de Tres Puntos*.

Casos de uso:

- Calcular el área de cualquier polígono simple (convexo o no) dado por sus vértices en orden.
- Determinar la orientación (CW o CCW) de una secuencia de puntos, usando el signo antes del valor absoluto.
- Validar que un polígono generado por otro algoritmo (Convex Hull, recorte) tiene la orientación esperada.
- Paso base para el Teorema de Pick (área en malla de puntos enteros).

Complejidad: $O(n)$ —un recorrido lineal por los n vértices, sumando un término constante por cada uno.

```
1 // CREATE/READ: Area de un poligono simple mediante la formula
  // de Shoelace
2 struct AreaPoligono {
3     struct Punto { double x, y; };
4
5     // READ: area CON SIGNO del poligono (positivo = CCW,
6     // negativo = CW);
7     // vertices deben estar en orden a lo largo del contorno
8     static double areaConSigno(const vector<Punto>& vertices) {
9         int n = (int)vertices.size();
10        double suma = 0;
11        for (int i = 0; i < n; i++) {
12            const Punto& actual = vertices[i];
13            const Punto& siguiente = vertices[(i + 1) % n];
```

```
13        suma += actual.x * siguiente.y - siguiente.x * actual.y;
14    }
15    return suma / 2.0;
16 } // O(n)
17
18 // READ: area absoluta del poligono (sin importar orientacion)
19 static double area(const vector<Punto>& vertices) {
20     return fabs(areaConSigno(vertices));
21 } // O(n)
22
23 // READ: true si los vertices estan en sentido antihorario (CCW)
24 static bool esCCW(const vector<Punto>& vertices) {
25     return areaConSigno(vertices) > 0;
26 } // O(n)
27
28 //
29 // -----
30 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31 // -----
32 // 1. Coordenadas enteras: la suma interna se puede mantener
33 //    en long long
34 //    (el area con signo * 2 siempre es entera si las
35 //    coordenadas son
36 //    enteras), dividiendo entre 2.0 solo al final; util para
37 //    Teorema de
38 //    Pick, que necesita el area exacta sin error de punto
39 //    flotante.
40 // 2. Poligono con hoyos: el area total es el area del
41 //    contorno exterior
42 //    (CCW) menos el area de cada hoyo interior (dado en
43 //    sentido CW);
44 //    sumar areaConSigno() de cada contorno respeta esto
45 //    automaticamente
46 //    si las orientaciones son consistentes.
47 // 3. Verificar que el poligono es simple (sin
48 //    autointersecciones) NO lo
49 //    hace esta formula: Shoelace da un resultado numerico
50 //    igual si el
51 //    poligono se autointersecta, pero ese resultado ya no
52 //    representa el
53 //    area real de una region simple.
54 };
```

Ejemplo de funcionamiento:

```
1 vector<AreaPoligono::Punto> cuadrado = {{0,0}, {4,0}, {4,4},
2     {0,4}};
3
4 double resultado = AreaPoligono::area(cuadrado);
5 // resultado = 16
6 // (cuadrado de lado 4, area = 4*4 = 16; los vertices estan en
7 // orden CCW)
8
9 bool ccw = AreaPoligono::esCCW(cuadrado);
10 // ccw = true
```

Regla práctica: usa Shoelace como la herramienta por defecto para área de polígono siempre que tengas coordenadas —es más general que Herón (funciona para cualquier número de lados, no solo triángulos) y no requiere que el polígono sea convexo. Aprovecha el signo (antes de `fabs`) cuando necesites confirmar o forzar una orientación específica, como en *Envolvente Convexa* o *Punto Dentro de un Polígono Convexo*, que dependen de esa convención.

34.2 Perímetro de un Polígono

Idea: el perímetro de un polígono con vértices $P_0, \dots, P_{n-1}$ en orden es simplemente la suma de las distancias euclidianas entre cada par de vértices consecutivos a lo largo del contorno, cerrando de vuelta del último vértice al primero: $\sum_{i=0}^{n-1} d(P_i, P_{i+1 \bmod n})$. Es la aplicación directa de *Distancia Euclidiana* en un recorrido lineal, sin ninguna sutileza adicional —a diferencia de área (Shoelace) o de la unión de rectángulos, el perímetro no depende de orientación ni de convexidad.

¿Por qué usarlo? Es el cálculo más simple de esta parte, pero se documenta aparte porque aparece con-

stantemente como componente de fórmulas más grandes (razón isoperimétrica, radio del círculo inscrito vía $r = \text{área} / \text{semiperímetro}$, problemas de "cuánta cerca se necesita"). No tiene alternativa más eficiente: a diferencia de área, que puede calcularse con una fórmula cerrada tipo Shoelace, el perímetro siempre requiere sumar las n distancias, una por cada arista.

Casos de uso:

- Calcular la longitud total de una cerca, borde o contorno poligonal.
- Paso intermedio en el cálculo del radio del círculo inscrito (usando semiperímetro junto al área de Herón o Shoelace).
- Comparar la "compacidad" de una figura (razón isoperimétrica: $4\pi \cdot \text{área} / \text{perímetro}^2$).
- Validar la longitud de un polígono generado por Envolverte Convexa u otro algoritmo de construcción.

Complejidad: $O(n)$ —un recorrido lineal por los n vértices, con una distancia euclidiana ($O(1)$) por cada arista.

```

1 // CREATE/READ: Perimetro de un poligono mediante suma de
  // distancias
2 // euclidianas entre vertices consecutivos
3 struct PerimetroPoligono {
4     struct Punto { double x, y; };
5
6     // READ: distancia euclidiana entre dos puntos
7     static double dist(const Punto& a, const Punto& b) {
8         double dx = b.x - a.x, dy = b.y - a.y;
9         return sqrt(dx*dx + dy*dy);
10    } // O(1)
11
12    // READ: perimetro total del poligono, sumando cada arista
13    // consecutiva
14    // (incluye la arista de cierre del ultimo vertice al primero)
15    static double perimetro(const vector<Punto>& vertices) {
16        int n = (int)vertices.size();
17        double total = 0;
18        for (int i = 0; i < n; i++) {
19            total += dist(vertices[i], vertices[(i + 1) % n]);
20        }
21        return total;
22    } // O(n)
23
24    // -----
25    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
26    // -----
27    // 1. Polilinea abierta (NO poligono cerrado): omitir la
28    // arista de cierre,
29    // recorriendo i de 0 a n-2 en vez de usar (i+1) % n.
30    // 2. Radio del circulo inscrito, dado el area (Shoelace o
31    // Heron):
32    // r = area / (perimetro(vertices) / 2.0), reutilizando el
33    // semiperimetro que ya calcula esta funcion internamente.
34    // 3. Razon isoperimetrica (que tan "circular" es la figura,
35    // maximo 1
36    // para un circulo perfecto): 4 * M_PI * area / (perimetro
37    // *perimetro).
38    };

```

Ejemplo de funcionamiento:

```

1 vector<PerimetroPoligono::Punto> cuadrado = {{0,0}, {4,0},
2         {4,4}, {0,4}};
3
4 double resultado = PerimetroPoligono::perimetro(cuadrado);
5 // resultado = 16
6 // (cuadrado de lado 4, cuatro lados de longitud 4 cada uno:
7 // 4*4 = 16)

```

Regla práctica: usa esta suma directa siempre que necesites la longitud del contorno; si el problema en realidad da una **polilínea abierta** (no un polígono

cerrado), recuerda omitir la arista de cierre entre el último y el primer vértice (ver modificación 1 dentro del código), ya que incluirla por error es un bug común al reutilizar esta misma función para ambos casos.

34.3 Punto Dentro de un Polígono (Ray Casting)

Idea: para determinar si un punto P está dentro de un polígono simple arbitrario (convexo o no), *ray casting* traza un rayo desde P hacia el infinito en una dirección fija (típicamente horizontal) y cuenta cuántas aristas del polígono cruza ese rayo. Si el número de cruces es **impar**, P está dentro; si es **par**, está fuera —la intuición es que cada vez que el rayo atraviesa el borde, cruza de "afuera" a "adentro" o viceversa, así que empezando afuera (en el infinito), un número impar de cruces termina adentro. Esta idea, conocida como el teorema de la curva de Jordan aplicado a polígonos, funciona sin importar la orientación del polígono ni si es convexo.

¿Por qué usarlo? A diferencia de *Punto Dentro de un Triángulo* (tres pruebas de orientación) o de *Punto Dentro de un Polígono Convexo* (búsqueda binaria angular, más adelante), ray casting es el único método de esta parte que funciona para polígonos **no convexos** (cóncavos, con "entrantes"). Su costo es más alto — $O(n)$ en vez de $O(1)$ o $O(\log n)$ — porque no puede aprovechar ninguna estructura especial: cada arista debe revisarse individualmente sin importar la forma del polígono.

Casos de uso:

- Verificar pertenencia de un punto a un polígono arbitrario, incluyendo formas cóncavas o con auto-similaridad compleja.
- Point-in-polygon para regiones geográficas o mapas con contornos irregulares.
- Rasterización de polígonos (determinar qué píxeles de una grilla caen dentro de una figura).
- Validación general cuando no se puede garantizar que el polígono sea convexo.

Complejidad: $O(n)$ —se revisa cada una de las n aristas del polígono exactamente una vez, con una comprobación $O(1)$ por arista.

```

1 // CREATE/READ: Punto dentro de un poligono simple (convexo o
  // no) mediante
2 // ray casting (conteo de cruces con un rayo horizontal)
3 struct PuntoDentroPoligono {
4     struct Punto { double x, y; };
5
6     // READ: true si el punto p esta dentro del poligono (borde
7     // EXCLUIDO,
8     // ver Nota); funciona para poligonos convexos y concavos
9     static bool dentro(const vector<Punto>& poligono, const Punto
10        & p) {
11         int n = (int)poligono.size();
12         bool adentro = false;
13         for (int i = 0, j = n - 1; i < n; j = i++) {
14             const Punto& a = poligono[i];
15             const Punto& b = poligono[j];
16             // revisa si el rayo horizontal desde p hacia +x cruza la
17             // arista (a,b)
18             bool cruzaEnY = (a.y > p.y) != (b.y > p.y);
19             if (cruzaEnY) {
20                 double xCruce = a.x + (p.y - a.y) * (b.x - a.x) / (b.y
21                    - a.y);
22                 if (p.x < xCruce) adentro = !adentro;
23             }
24         }
25         return adentro;
26     } // O(n)

```

```

23 // READ: true si el punto p esta exactamente SOBRE el borde
24 // del poligono
25 // (sobre alguna arista); combinar con dentro() para "dentro
26 // o sobre el borde"
27 static bool sobreBorde(const vector<Punto>& poligono, const
28 Punto& p) {
29     int n = (int)poligono.size();
30     const double EPS = 1e-9;
31     for (int i = 0; i < n; i++) {
32         const Punto& a = poligono[i];
33         const Punto& b = poligono[(i + 1) % n];
34         double cruz = (b.x - a.x) * (p.y - a.y) - (b.y - a.y) * (
35             p.x - a.x);
36         if (fabs(cruz) > EPS) continue; // no colineal con esta
37         arista
38         bool dentroX = min(a.x, b.x) - EPS <= p.x && p.x <= max(a
39             .x, b.x) + EPS;
40         bool dentroY = min(a.y, b.y) - EPS <= p.y && p.y <= max(a
41             .y, b.y) + EPS;
42         if (dentroX && dentroY) return true;
43     }
44     return false;
45 } // O(n)
46
47 // -----
48 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
49 // -----
50 // 1. "Dentro o sobre el borde" (inclusivo): dentro(poligono,
51 // p) ||
52 // sobreBorde(poligono, p) -- necesario porque dentro()
53 // por si solo
54 // da resultado indefinido/inconsistente cuando p cae
55 // exactamente
56 // sobre una arista (el rayo puede o no contarla como
57 // cruce segun
58 // redondeo de punto flotante).
59 // 2. Coordenadas enteras: xCruce y la comparacion p.x <
60 // xCruce se
61 // pueden reescribir con aritmetica de enteros
62 // multiplicando en cruz,
63 // evitando la division y el error de punto flotante por
64 // completo.
65 // 3. Poligono con hoyos (contorno exterior + contornos
66 // interiores en
67 // sentido opuesto): correr dentro() sobre el contorno
68 // exterior Y
69 // sobre cada hoyo; el punto esta en la region valida solo
70 // si esta
71 // dentro del exterior y fuera de TODOS los hoyos.
72 };

```

Ejemplo de funcionamiento:

```

1 vector<PuntoDentroPoligono::Punto> estrella = {
2     {0,4}, {1,1}, {4,0}, {1,-1}, {0,-4}, {-1,-1}, {-4,0}, {-1,1}
3 };
4 PuntoDentroPoligono::Punto p{0, 0};
5
6 bool resultado = PuntoDentroPoligono::dentro(estrella, p);
7 // resultado = true
8 // (p={0,0} esta en el centro de una figura concava tipo
9 // estrella; ray
10 // casting funciona correctamente aqui donde las tres pruebas
11 // de
12 // orientacion de Punto Dentro de un Triangulo no aplicarian)

```

Nota: el resultado de `dentro()` para un punto exactamente sobre el borde es indefinido por construcción del algoritmo (depende de redondeo). Si el problema distingue "estrictamente dentro" de "sobre el borde" de "fuera", usa `sobreBorde()` por separado antes de confiar en `dentro()`, tal como se muestra en la modificación 1 dentro del código.

Regla práctica: usa ray casting como la herramienta general por defecto para point-in-polygon —es la única de esta parte que funciona sin importar si el polígono es convexo. Si tienes garantizado que el polígono es convexo, *Punto Dentro de un Polígono Convexo* con búsqueda binaria es $O(\log n)$ y por lo tanto preferible cuando se hacen muchas consultas sobre el mismo polígono.

34.4 Punto Dentro de un Polígono Convexo (Binary Search)

Idea: cuando el polígono es convexo y está dado en orden CCW (la convención fijada en *Orientación de Tres Puntos*), se puede aprovechar esa estructura para responder en $O(\log n)$ en vez de $O(n)$: se fija el vértice 0 como pivote, y los vértices $1, \dots, n-1$ quedan ordenados angularmente alrededor de él por ser convexo. Una búsqueda binaria encuentra el "abanico" (los dos vértices consecutivos P_i, P_{i+1}) entre los que cae el ángulo de P respecto a P_0 , y basta con una prueba de *Punto Dentro de un Triángulo* sobre el triángulo (P_0, P_i, P_{i+1}) para confirmar si P está dentro.

¿Por qué usarlo? Frente a ray casting ($O(n)$, funciona para cualquier polígono), esta técnica explota la convexidad para bajar a $O(\log n)$ por consulta —una mejora significativa cuando se hacen muchas consultas sobre el mismo polígono fijo (por ejemplo, q consultas dan $O(n + q \log n)$ en vez de $O(nq)$). La condición es estricta: el polígono debe ser convexo y estar en el orden CCW consistente con el resto de la guía; si cualquiera de esas dos condiciones falla, el resultado es incorrecto sin ningún aviso.

Casos de uso:

- Muchas consultas de pertenencia sobre el mismo polígono convexo fijo (por ejemplo, la salida de *Envolverte Convexa*).
- Verificación rápida de colisión punto-polígono en aplicaciones con restricciones de tiempo estrictas.
- Localización de un punto dentro de una región convexa en problemas de particionamiento del plano.
- Como subrutina dentro de algoritmos más grandes que requieren muchas pruebas de pertenencia (por ejemplo, intersección de semiplanos).

Complejidad: $O(\log n)$ por consulta tras un preprocesamiento $O(1)$ (no requiere ordenar nada adicional, ya que el polígono convexo en CCW ya viene angularmente ordenado respecto a cualquiera de sus vértices).

```

1 // CREATE/READ: Punto dentro de un poligono CONVEXO mediante
2 // búsqueda binaria
3 // angular; REQUIERE que el poligono este en orden CCW (ver
4 // Nota)
5 struct PuntoDentroPoligonoConvexo {
6     struct Punto { double x, y; };
7
8     // READ: producto cruz de (b-a) x (c-a)
9     static double cruz(const Punto& a, const Punto& b, const
10         Punto& c) {
11         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
12     };
13 } // O(1)
14
15 // READ: true si p esta dentro (o sobre el borde) del
16 // poligono convexo,
17 // dado en orden CCW; usa búsqueda binaria sobre el abanico
18 // desde poligono[0]
19 static bool dentro(const vector<Punto>& poligono, const Punto
20 & p) {
21     int n = (int)poligono.size();
22     const Punto& p0 = poligono[0];
23     const double EPS = 1e-9;
24
25     // p debe estar entre los rayos p0->poligono[1] y p0->
26     // poligono[n-1]
27     double cruzInicial = cruz(p0, poligono[1], p);
28     double cruzFinal = cruz(p0, poligono[n-1], p);
29     if (cruzInicial < -EPS || cruzFinal > EPS) return false; //
30     fuera del abanico

```

```

23 // búsqueda binaria del índice i tal que p cae en el
    triangulo
24 // (p0, poligono[i], poligono[i+1])
25 int lo = 1, hi = n - 1;
26 while (hi - lo > 1) {
27     int medio = (lo + hi) / 2;
28     if (cruz(p0, poligono[medio], p) >= -EPS) lo = medio;
29     else hi = medio;
30 }
31
32 // confirma que p esta dentro del triangulo final (p0,
    poligono[lo], poligono[hi])
33 double d1 = cruz(p0, poligono[lo], p);
34 double d2 = cruz(poligono[lo], poligono[hi], p);
35 double d3 = cruz(poligono[hi], p0, p);
36 return d1 >= -EPS && d2 >= -EPS && d3 >= -EPS;
37 } // O(log n)
38
39 //
40 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
41 //
42 // 1. Poligono en orden CW (horario): invertir el orden de
    los vertices
43 // (poligono.rbegin() a poligono.rend()) antes de usar, o
    negar todos
44 // los resultados de cruz() en el algoritmo -- NO usar
    directamente
45 // sin ajustar, el resultado seria incorrecto en silencio.
46 // 2. Excluir el borde (estrictamente dentro): cambiar todos
    los ">=" a ">-EPS"
47 // por "> EPS" en las comparaciones finales y en la
    busqueda binaria.
48 // 3. Muchas consultas sobre el mismo poligono: el
    preprocesamiento es
49 // O(1) (no hay que ordenar nada extra), asi que esta
    estructura es
50 // directamente reutilizable llamando dentro()
    repetidamente sin
51 // reconstruir nada.
52 };

```

Ejemplo de funcionamiento:

```

1 vector<PuntoDentroPoligonoConvexo::Punto> hexagono = {
2     {2,0}, {1,2}, {-1,2}, {-2,0}, {-1,-2}, {1,-2}
3 }; // hexagono convexo en orden CCW
4 PuntoDentroPoligonoConvexo::Punto p{0, 0}, q{3, 3};
5
6 bool dentroP = PuntoDentroPoligonoConvexo::dentro(hexagono, p);
7 // dentroP = true (el centro esta dentro del hexagono)
8
9 bool dentroQ = PuntoDentroPoligonoConvexo::dentro(hexagono, q);
10 // dentroQ = false (q esta fuera, mas alla de todos los
    vertices)

```

Nota: este algoritmo **asume orden CCW** sin verificarlo — si el polígono que produce *Envolvente Convexa* (Jarvis o Monotone Chain) no respeta esa orientación, esta búsqueda binaria falla en silencio, dando resultados incorrectos sin ningún error visible. Antes de usar esta técnica sobre la salida de un algoritmo de convex hull, confirma la orientación con `AreaPoligono::esCCW()` (ver *Área de un Polígono*) y, si es necesario, invierte el orden de los vértices.

Regla práctica: usa esta técnica en vez de ray casting únicamente cuando el polígono sea convexo Y esté en orden CCW confirmado, y cuando vayas a hacer múltiples consultas sobre el mismo polígono —la ganancia de $O(n)$ a $O(\log n)$ solo vale la pena si el número de consultas q es grande. Para una sola consulta o un polígono no convexo, usa *Punto Dentro de un Polígono (Ray Casting)*.

34.5 Teorema de Pick (Área en Malla de Puntos Enteros)

Idea: para un polígono simple cuyos vértices tienen coordenadas **enteras**, el teorema de Pick relaciona el área con el conteo de puntos enteros: $A = I + \frac{B}{2} - 1$, donde I es el número de puntos enteros estrictamente en

el interior del polígono y B es el número de puntos enteros sobre el borde (incluyendo los vértices). Despejando, $I = A - \frac{B}{2} + 1$. El número de puntos enteros sobre una arista entre dos vértices (x_1, y_1) y (x_2, y_2) es $\gcd(|x_2 - x_1|, |y_2 - y_1|)$ —incluyendo un extremo pero no el otro, para no contar los vértices dos veces al sumar todas las aristas.

¿Por qué usarlo? Es el puente entre *Área de un Polígono (Shoelace)* —que da el área geométrica continua— y el conteo combinatorio de puntos en una malla entera, algo que de otra forma requeriría iterar sobre una grilla completa ($O(\text{área})$, potencialmente enorme). Pick reduce ese conteo a $O(1)$ una vez que se tiene el área (Shoelace, $O(n)$) y el conteo de puntos de borde (suma de gcd por arista, también $O(n)$): la fórmula evita cualquier iteración sobre la malla misma, sin importar qué tan grande sea el polígono.

Casos de uso:

- Contar puntos de malla estrictamente interiores a un polígono de vértices enteros, sin iterar la grilla.
- Contar puntos de malla sobre el borde de un polígono (perímetro "discreto").
- Problemas que piden el área de una figura descrita en una grilla de puntos enteros (más simple que Shoelace + gcd cuando se da I y B directamente).
- Verificación cruzada: si se conocen I , B y el área por otro método, Pick sirve como chequeo de consistencia.

Complejidad: $O(n)$ —Shoelace para el área es $O(n)$, y sumar gcd sobre cada una de las n aristas es $O(n \log(\max \text{coordenada}))$ por el costo del algoritmo de Euclides, dominado en la práctica por el término lineal.

```

1 // CREATE/READ: Teorema de Pick: relaciona area, puntos
    interiores (I) y
2 // puntos de borde (B) para poligonos de vertices enteros
3 struct TeoremaPick {
4     struct Punto { long long x, y; };
5
6     // READ: area con signo * 2 mediante Shoelace en aritmetica
    entera
7     // (evita perder precision, ya que 2*area siempre es entero)
8     static long long dobleAreaConSigno(const vector<Punto>&
    vertices) {
9         int n = (int)vertices.size();
10        long long suma = 0;
11        for (int i = 0; i < n; i++) {
12            const Punto& actual = vertices[i];
13            const Punto& siguiente = vertices[(i + 1) % n];
14            suma += actual.x * siguiente.y - siguiente.x * actual.y;
15        }
16        return suma;
17    } // O(n)
18
19    // READ: numero de puntos enteros sobre el borde del poligono
20    // (suma de gcd por arista, cada vertice contado una sola vez)
21    static long long puntosBorde(const vector<Punto>& vertices) {
22        int n = (int)vertices.size();
23        long long total = 0;
24        for (int i = 0; i < n; i++) {
25            const Punto& a = vertices[i];
26            const Punto& b = vertices[(i + 1) % n];
27            total += _gcd(llabs(b.x - a.x), llabs(b.y - a.y));
28        }
29        return total;
30    } // O(n log(max coordenada))
31
32    // READ: numero de puntos enteros Estrictamente interiores,
    via Pick:
33    // I = A - B/2 + 1, usando doble area para mantener
    aritmetica entera

```

```

34 static long long puntosInteriores(const vector<Punto>&
35     vertices) {
36     long long dobleArea = llabs(dobleAreaConSigno(vertices));
37     long long b = puntosBorde(vertices);
38     // 2*I = 2*A - B + 2 = dobleArea - b + 2
39     return (dobleArea - b + 2) / 2;
40 } // O(n log(max coordenada))
41
42 // -----
43 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
44 // -----
45 // 1. Area como double a partir de I y B conocidos (Pick
46 //    directo, sin
47 //    Shoelace): area = I + B/2.0 - 1; útil si el problema da
48 //    I y B
49 //    directamente y pide el area.
50 // 2. Vertices NO enteros: el teorema de Pick no aplica;
51 //    requiere
52 //    coordenadas enteras tanto para el conteo de gcd como
53 //    para que I
54 //    y B tengan sentido combinatorio.
55 // 3. Verificación de que dobleArea - b es PAR antes de
56 //    dividir entre 2:
57 //    matemáticamente siempre lo es para un poligono valido
58 //    de vertices
59 //    enteros, pero sirve como chequeo de cordura al depurar
60 //    (si da
61 //    impar, hay un bug en el calculo de area o de borde).
62 };

```

Ejemplo de funcionamiento:

```

1 vector<TeoremaPick::Punto> cuadrado = {{0,0}, {4,0}, {4,4},
2     {0,4}};
3 long long b = TeoremaPick::puntosBorde(cuadrado);
4 // b = 16
5 // (cada lado tiene longitud 4, gcd(4,0)=4 por arista, 4
6 //    aristas: 4*4=16;
7 // cada punto de borde se cuenta una sola vez, incluyendo los
8 //    4 vertices)
9 long long i = TeoremaPick::puntosInteriores(cuadrado);
10 // i = 9
11 // (area = 16, Pick: I = 16 - 16/2 + 1 = 16 - 8 + 1 = 9;
12 //    corresponde a la grilla interior 3x3 de puntos (1,1) a
13 //    (3,3))

```

Nota: este teorema aplica **únicamente** a polígonos simples con vértices de coordenadas enteras. Si el problema da coordenadas reales (no enteras), no hay forma de aplicar Pick directamente —hay que recurrir a *Área de un Polígono (Shoelace)* para el área continua, sin ningún conteo de puntos de malla asociado.

Regla práctica: usa Pick cuando el problema combine dos de las tres cantidades (área, puntos interiores, puntos de borde) y pida la tercera, sobre un polígono de vértices enteros —evita iterar la grilla por completo, algo que sería inviable si el área es grande. Si necesitas el área pura sin relación con conteo de puntos, *Shoelace* solo (sin Pick) es más directo.

35 Envoltente Convexa

35.1 Algoritmo de Jarvis (Gift Wrapping)

Idea: Jarvis construye la envoltente convexa "envolviendo" el conjunto de puntos como si se estirara una banda elástica alrededor de ellos: partiendo del punto más bajo (garantizado en el hull), se busca repetidamente el siguiente vértice del hull como aquel punto tal que **todos** los demás puntos quedan a la izquierda del segmento actual —es decir, el punto que hace el giro más "hacia afuera" posible respecto al vértice anterior. Esto se decide con *Orientación de Tres Puntos*: para cada candidato se compara contra el mejor candidato actual, y se actualiza cuando el nuevo punto queda más a

la derecha (en sentido de barrido) del candidato vigente. El proceso se repite hasta volver al punto de partida.

¿Por qué usarlo? Es el algoritmo de envoltente convexa más simple de razonar e implementar correctamente, porque en cada paso solo requiere una comparación local (¿quién es más extremo?) sin necesidad de ordenar todos los puntos de antemano. Su costo, sin embargo, depende del número de vértices h que termina teniendo el hull, no solo de n : es preferible a *Monotone Chain* únicamente cuando se sabe de antemano que h va a ser pequeño comparado con n (envoltente con pocos vértices sobre una nube de puntos grande); en el peor caso, cuando casi todos los puntos están en el hull ($h \approx n$), Jarvis degenera a $O(n^2)$, peor que el $O(n \log n)$ garantizado de *Monotone Chain*.

Casos de uso:

- Construcción de la envoltente convexa cuando se espera que tenga pocos vértices.
- Problemas educativos o de referencia rápida donde la simplicidad de implementación importa más que la complejidad garantizada.
- Como paso dentro de algoritmos incrementales de geometría computacional (Quickhull usa una idea similar de forma recursiva).
- Verificación cruzada del resultado de *Monotone Chain* en casos pequeños durante debugging.

Complejidad: $O(nh)$, donde n es el número de puntos y h el número de vértices del hull resultante: cada uno de los h vértices del hull requiere una pasada de $O(n)$ para encontrar el siguiente punto más extremo. En el peor caso ($h = n$), esto es $O(n^2)$.

```

1 // CREATE/READ: Envoltente convexa mediante Jarvis (Gift
2 // Wrapping);
3 // retorna los vertices del hull en orden CCW
4 static EnvoltenteJarvis {
5     struct Punto { double x, y; };
6
7     // READ: producto cruz de (b-a) x (c-a); positivo = CCW (
8     //    convencion
9     //    fijada en Orientacion de Tres Puntos)
10    static double cruz(const Punto& a, const Punto& b, const
11        Punto& c) {
12        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
13        );
14    } // O(1)
15
16    // READ: distancia al cuadrado, usada para desempatar puntos
17    //    colineales
18    static double distCuadrado(const Punto& a, const Punto& b) {
19        double dx = b.x - a.x, dy = b.y - a.y;
20        return dx*dx + dy*dy;
21    } // O(1)
22
23    // READ: construye el hull convexo de un conjunto de puntos,
24    //    retornado
25    //    en orden CCW; requiere n >= 3 para un hull no degenerado
26    static vector<Punto> construir(vector<Punto> puntos) {
27        int n = (int)puntos.size();
28        if (n < 3) return puntos; // hull degenerado (punto o
29            segmento)
30
31        // punto inicial: el de menor y (y menor x en caso de
32        //    empate),
33        //    garantizado en el hull
34        int inicio = 0;
35        for (int i = 1; i < n; i++) {
36            if (puntos[i].y < puntos[inicio].y ||
37                (puntos[i].y == puntos[inicio].y && puntos[i].x < puntos[
38                inicio].x)) {
39                inicio = i;
40            }
41        }
42    }
43 }

```

```

34 vector<Punto> hull;
35 int actual = inicio;
36 do {
37     hull.push_back(puntos[actual]);
38     int siguiente = (actual + 1) % n; // candidato inicial
    arbitrario
39     for (int i = 0; i < n; i++) {
40         if (i == actual) continue;
41         double c = cruz(puntos[actual], puntos[siguiente],
    puntos[i]);
42         // si i queda a la derecha del candidato actual, i es
    mejor;
43         // si son colineales, se prefiere el mas lejano
44         if (c < 0 || (c == 0 && distCuadrado(puntos[actual],
    puntos[i]) >
45             distCuadrado(puntos[actual], puntos[siguiente]))) {
46             siguiente = i;
47         }
48     }
49     actual = siguiente;
50 } while (actual != inicio);
51
52 return hull;
53 } // O(n*h): h iteraciones del do-while, cada una O(n)
54
55 //
56 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
57 //
58 // 1. Hull en sentido CW en vez de CCW: invertir el criterio
59 // "c > 0" a
60 // "c < 0" en la comparacion del candidato; NO cambia la
    complejidad,
61 // solo la orientacion de salida.
62 // 2. Incluir puntos colineales sobre el borde del hull: el
    desempate por
63 // distCuadrado en el codigo actual EXCLUYE puntos
    colineales
64 // intermedios (se queda con el mas lejano); para
    incluirlos, agregar
65 // los puntos colineales entre actual y siguiente antes de
    avanzar.
66 // 3. Puntos duplicados en la entrada: eliminarlos antes de
    llamar
67 // construir(), ya que pueden causar que el bucle do-while
    no termine
68 // correctamente si el punto inicial se duplica.
69 };

```

Ejemplo de funcionamiento:

```

1 vector<EnvolverteJarvis::Punto> puntos = {
2     {0,0}, {4,0}, {4,4}, {0,4}, {2,2} // (2,2) esta estrictamente
    adentro
3 };
4
5 auto hull = EnvolverteJarvis::construir(puntos);
6 // hull = {(0,0), (4,0), (4,4), (0,4)}
7 // (el punto interior (2,2) queda excluido; los cuatro vertices
    del
8 // cuadrado forman el hull en orden CCW, empezando por el de
    menor y)

```

Regla práctica: usa Jarvis cuando sepas (o sospeches fuertemente) que el hull tendrá pocos vértices frente a n ; si no hay garantía sobre el tamaño del hull, o n es grande, usa *Monotone Chain* en su lugar, que garantiza $O(n \log n)$ sin importar cuántos vértices termine teniendo el hull.

35.2 Monotone Chain (Andrew)

Idea: en vez de "envolver" iterativamente como Jarvis, Monotone Chain ordena todos los puntos por x (desempatando por y) y construye el hull en dos pasadas lineales: una **cadena inferior** (*lower hull*), recorriendo los puntos ordenados de izquierda a derecha y manteniendo solo los que forman giros a la derecha (o eliminando los que forman giros a la izquierda, usando *Orientación de Tres Puntos* sobre una pila), y una **cadena superior** (*upper hull*) recorriendo en sentido inverso con el mismo criterio. Concatenar ambas cadenas (sin duplicar los extremos) da el hull completo en orden CCW.

¿Por qué usarlo? Garantiza $O(n \log n)$ en el peor caso, dominado por el ordenamiento inicial —a diferencia de Jarvis, cuyo costo depende de h y puede degenerar a $O(n^2)$. El mecanismo de "pila que elimina giros incorrectos" es el mismo patrón usado en muchos algoritmos de geometría computacional con barrido (similar en espíritu al manejo de estado del *Line Sweep*), y es la opción por defecto en competencia cuando no hay garantía sobre el tamaño del hull resultante.

Casos de uso:

- Construcción de la envolvente convexa como caso general, sin garantías sobre h .
- Preprocesamiento antes de *Punto Dentro de un Polígono Convexo*, que requiere el hull en orden CCW.
- Problemas donde n es grande y se necesita la complejidad garantizada $O(n \log n)$.
- Base para algoritmos que requieren el hull como paso intermedio (diámetro del conjunto de puntos, rotating calipers).

Complejidad: $O(n \log n)$, dominado por el ordenamiento inicial de los n puntos; cada una de las dos cadenas (inferior y superior) se construye en $O(n)$ porque cada punto se inserta y se elimina de la pila a lo sumo una vez.

```

1 // CREATE/READ: Envolvente convexa mediante Monotone Chain (
    Andrew);
2 // retorna los vertices del hull en orden CCW, sin puntos
    colineales de sobra
3 struct EnvolverteMonotoneChain {
4     struct Punto { double x, y; };
5
6     // READ: producto cruz de (b-a) x (c-a); positivo = CCW
7     static double cruz(const Punto& a, const Punto& b, const
    Punto& c) {
8         return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x
    );
9     } // O(1)
10
11     // READ: construye el hull convexo en orden CCW; requiere n
    >= 3 para
12     // un hull no degenerado
13     static vector<Punto> construir(vector<Punto> puntos) {
14         int n = (int)puntos.size();
15         if (n < 3) return puntos;
16
17         sort(puntos.begin(), puntos.end(), [](const Punto& a, const
    Punto& b) {
18             if (a.x != b.x) return a.x < b.x;
19             return a.y < b.y;
20         });
21
22         vector<Punto> hull(2 * n);
23         int k = 0;
24
25         // cadena inferior: recorre de izquierda a derecha
26         for (int i = 0; i < n; i++) {
27             while (k >= 2 && cruz(hull[k-2], hull[k-1], puntos[i]) <=
    0) k--;
28             hull[k++] = puntos[i];
29         }
30
31         // cadena superior: recorre de derecha a izquierda,
    continuando
32         // desde donde quedo la cadena inferior
33         int limiteInferior = k - 1;
34         for (int i = n - 2; i >= 0; i--) {
35             while (k >= limiteInferior && cruz(hull[k-2], hull[k-1],
    puntos[i]) <= 0) k--;
36             hull[k++] = puntos[i];
37         }
38
39         hull.resize(k - 1); // el ultimo punto duplica el primero,
    se descarta
40         return hull;
41     } // O(n log n): dominado por el sort; ambas cadenas son O(n)
42 }

```

```

43 //
44 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
45 //
46 // 1. Incluir puntos colineales sobre el borde del hull:
47 //   cambiar la
48 //   condicion "<= 0" por "< 0" en ambos while, de modo que
49 //   los giros
50 //   exactamente colineales NO se descarten de la pila.
51 // 2. Hull en sentido CW en vez de CCW: invertir "<= 0" a ">=
52 //   0" en
53 //   ambos while.
54 // 3. Puntos duplicados en la entrada: eliminarlos antes de
55 //   ordenar
56 //   (sort + unique) para evitar comportamiento inesperado
57 //   en el
58 //   desempate de cruz() == 0 con puntos identicos.
59 };

```

Ejemplo de funcionamiento:

```

1 vector<EnvolventeMonotoneChain::Punto> puntos = {
2     {0,0}, {4,0}, {4,4}, {0,4}, {2,2} // (2,2) esta estrictamente
3     adentro
4 };
5
6 auto hull = EnvolverMonotoneChain::construir(puntos);
7 // hull = {(0,0), (4,0), (4,4), (0,4)}
8 // (mismo resultado que Jarvis para este conjunto de puntos, en
9 //   orden CCW;
10 //   el punto interior (2,2) queda excluido por la condicion de
11 //   giro)

```

Nota: tanto Jarvis como Monotone Chain producen el hull en orden **CCW** con la convención fijada en esta guía, así que el resultado de cualquiera de los dos es directamente compatible con *Punto Dentro de un Polígono Convexo*, que requiere esa orientación. Si se necesita orden CW, ver la modificación 2 en cualquiera de los dos structs.

Regla práctica: usa Monotone Chain como opción por defecto en competencia, salvo que tengas una razón específica para preferir Jarvis (hull con muy pocos vértices conocido de antemano, o simplicidad de implementación en un contexto donde n es pequeño). Su $O(n \log n)$ garantizado lo hace la opción más segura cuando no hay información previa sobre el tamaño del hull.

35.3 Corte de un Polígono Convexo por una Línea

Idea: dado un polígono convexo (por ejemplo, la salida de *Jarvis* o *Monotone Chain*) y una línea definida por dos puntos, esta operación produce el subpolígono resultante de quedarse solo con la parte del polígono que cae de un lado específico de la línea (el lado izquierdo, según la orientación de la línea). El algoritmo recorre los vértices en orden y usa *Orientación de Tres Puntos* para clasificar cada uno como dentro o fuera del semiplano deseado: cuando dos vértices consecutivos están en lados opuestos, se calcula el punto de intersección exacto de la arista con la línea de corte y se inserta en el resultado; los vértices que están del lado válido se conservan tal cual. Este es un caso particular del algoritmo de recorte de Sutherland-Hodgman, especializado a una sola línea de corte en vez de un polígono de recorte completo.

¿Por qué usarlo? A diferencia de *Intersección de Segmentos*, que solo detecta si dos segmentos se cruzan, aquí el objetivo es reconstruir una figura completa (el subpolígono resultante), lo que requiere combinar la prueba de orientación con la inserción cuidadosa de nuevos vértices en la intersección. Explota que el polígono de entrada

es **convexo**: eso garantiza que el resultado del corte también es convexo (un semiplano intersectado con un convexo siempre da un convexo), lo cual no se cumpliría si el polígono de entrada fuera cóncavo —ahí el resultado podría ser múltiples polígonos separados, un caso mucho más complejo no cubierto aquí.

Casos de uso:

- Calcular el área de un polígono convexo que queda a un lado de una recta (por ejemplo, "¿qué fracción del terreno queda al norte de esta carretera recta?").
- Recorte de figuras convexas contra los bordes de una ventana o región rectangular (aplicando esta operación cuatro veces, una por cada borde).
- Como subrutina en *intersección de semiplanos* (intersectar un polígono convexo contra muchas líneas sucesivamente).
- Simulación de corte físico de una región convexa por una trayectoria recta.

Complejidad: $O(n)$ —un recorrido lineal por los n vértices del polígono, con una prueba de orientación $O(1)$ y, en el peor caso, un cálculo de intersección $O(1)$ por cada arista.

```

1 // CREATE/READ: Corta un poligono convexo por una linea,
2 //   quedandose con
3 //   la parte del lado IZQUIERDO de la linea (segun su
4 //   orientacion a->b)
5 struct CortePoligonoConvexo {
6     struct Punto { double x, y; };
7
8     // READ: producto cruz de (b-a) x (c-a); positivo = c esta a
9     //   la
10    //   izquierda de a->b (convencion CCW de Orientacion de Tres
11    //   Puntos)
12    static double cruz(const Punto& a, const Punto& b, const
13    Punto& c) {
14        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
15    };
16    } // O(1)
17
18 // READ: interseccion entre la arista (p, q) y la linea (a, b
19 //   ),
20 //   asumiendo que p y q estan en lados opuestos de la linea
21 static Punto interseccion(const Punto& p, const Punto& q,
22 const Punto& a, const Punto& b) {
23     double d1 = cruz(a, b, p);
24     double d2 = cruz(a, b, q);
25     double t = d1 / (d1 - d2);
26     return Punto{p.x + t * (q.x - p.x), p.y + t * (q.y - p.y)};
27 } // O(1)
28
29 // READ: corta el poligono convexo, quedandose con el lado
30 //   izquierdo
31 //   de la linea a->b; retorna vector vacio si el poligono
32 //   queda
33 //   completamente del lado derecho
34 static vector<Punto> cortar(const vector<Punto>& poligono,
35 const Punto& a, const Punto& b) {
36     int n = (int)poligono.size();
37     vector<Punto> resultado;
38     const double EPS = 1e-9;
39
40     for (int i = 0; i < n; i++) {
41         const Punto& actual = poligono[i];
42         const Punto& siguiente = poligono[(i + 1) % n];
43         double dActual = cruz(a, b, actual);
44         double dSiguiente = cruz(a, b, siguiente);
45
46         if (dActual >= -EPS) resultado.push_back(actual); //
47         //   actual esta dentro (o sobre la linea)
48         //   // si actual y siguiente estan en lados opuestos, agregar
49         //   //   el punto de corte
50         if ((dActual > EPS && dSiguiente < -EPS) || (dActual < -
51 EPS && dSiguiente > EPS)) {
52             resultado.push_back(interseccion(actual, siguiente, a,
53 b));
54         }
55     }
56 }

```

```

41     return resultado;
42 } // O(n)
43
44 //
45 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
46 //
47 // 1. Quedarse con el lado DERECHO en vez del izquierdo:
48 //     invertir el
49 //     criterio "dActual >= -EPS" a "dActual <= EPS" (y
50 //     ajustar el signo
51 //     en la detección de cruce si se desea mayor claridad).
52 // 2. Area del subpoligono resultante: aplicar directamente
53 //     Shoelace
54 //     (ver Area de un Poligono) sobre el vector retornado por
55 //     cortar().
56 // 3. Recorte contra una ventana rectangular (Sutherland-
57 //     Hodgman
58 //     completo): aplicar cortar() cuatro veces sucesivas, una
59 //     por cada
60 //     borde del rectangulo (orientados consistentemente en
61 //     CCW), usando
62 //     el resultado de cada corte como entrada del siguiente.
63 };

```

Ejemplo de funcionamiento:

```

1 vector<CortePoligonoConvexo::Punto> cuadrado = {{0,0}, {4,0},
2          {4,4}, {0,4}};
3 CortePoligonoConvexo::Punto a{2,-1}, b{2,5}; // línea vertical
4          x=2
5 auto resultado = CortePoligonoConvexo::cortar(cuadrado, a, b);
6 // resultado = {(2,0), (4,0), (4,4), (2,4)}
7 // (se conserva el lado izquierdo de la línea a->b, que en este
8 // caso
9 // apunta hacia +y, dejando el lado con x >= 2; el cuadrado
10 // original de
11 // área 16 queda cortado a un rectángulo de área 8)

```

Nota: el lado que se conserva depende de la **orientación** de la línea de corte ($a \rightarrow b$), no solo de su posición geométrica —invertir a y b invierte qué mitad del polígono se conserva. Esto es consistente con la convención de *Orientación de Tres Puntos*: positivo significa "a la izquierda de $a \rightarrow b$ ", y es responsabilidad de quien llama orientar la línea según el lado que quiere conservar.

Regla práctica: usa esta operación cuando necesites el subpolígono resultante de un corte, no solo si dos figuras se cruzan —para eso te alcanza *Intersección de Segmentos*. Encadenar varias llamadas a `cortar()` (una por cada línea) es la forma directa de resolver intersección de semiplanos o recorte contra una ventana convexa arbitraria, siempre que el polígono de entrada sea convexo.

36 Ad-hoc y Combinatoria Geométrica

36.1 Perímetro Usando Bloques (Figuras en Grilla)

Idea: dada una figura formada por bloques cuadrados unitarios sobre una grilla (por ejemplo, una matriz de "*" y "." indicando qué celdas están ocupadas), el perímetro no requiere geometría continua: cada bloque ocupado contribuye hasta 4 unidades de perímetro (una por cada uno de sus cuatro lados), pero cada vez que dos bloques ocupados son vecinos (comparten un lado), ese lado compartido deja de ser borde exterior y se resta **dos veces** —una unidad de cada bloque—. La fórmula resultante es $P = 4B - 2A$, donde B es el número total de bloques ocupados y A es el número de pares de bloques ocupados adyacentes (compartiendo un lado, no solo una esquina).

¿Por qué usarlo? A diferencia de *Perímetro de un Polígono*, que requiere las coordenadas explícitas del

contorno, aquí la entrada es una grilla discreta y no hace falta reconstruir el contorno geométrico en absoluto —basta con contar bloques y pares adyacentes, una operación puramente combinatoria sobre la matriz. Intentar resolverlo trazando el contorno real (como si fuera un polígono) sería mucho más laborioso que este conteo directo.

Casos de uso:

- Calcular el perímetro de una figura compuesta por celdas de una grilla (islas en un mapa, piezas de Tetris, regiones pintadas).
- Problemas de "área de una isla" en matrices binarias, donde también se pide el perímetro de cada componente.
- Verificación rápida de cuántas aristas de borde tiene una región antes de un recorrido BFS/DFS más complejo.
- Cálculo de perímetro total de múltiples figuras separadas sobre la misma grilla (sumando por componente conexa).

Complejidad: $O(NM)$ para una grilla de $N \times M$ —se recorre cada celda una vez, y por cada bloque ocupado se revisan sus hasta 4 vecinos, cada revisión $O(1)$.

```

1 // CREATE/READ: Perímetro de una figura formada por bloques en
2 // una grilla
3 struct PerimetroBloques {
4     vector<string> grilla;
5     int filas, columnas;
6
7     // CREATE: recibe la grilla como vector de strings, '*' =
8     // ocupado,
9     // '.' = vacío
10    PerimetroBloques(const vector<string>& g) : grilla(g) {
11        filas = (int)grilla.size();
12        columnas = filas > 0 ? (int)grilla[0].size() : 0;
13    } // O(1), no copia mas alla de la referencia inicial
14
15    // READ: true si la celda (fila, col) esta dentro de la
16    // grilla Y ocupada
17    bool estaOcupada(int fila, int col) const {
18        if (fila < 0 || fila >= filas || col < 0 || col >= columnas)
19            return false;
20        return grilla[fila][col] == '*';
21    } // O(1)
22
23    // READ: perimetro total de todos los bloques ocupados en la
24    // grilla
25    long long calcularPerimetro() const {
26        long long perimetro = 0;
27        int df[4] = {-1, 1, 0, 0};
28        int dc[4] = {0, 0, -1, 1};
29        for (int f = 0; f < filas; f++) {
30            for (int c = 0; c < columnas; c++) {
31                if (!estaOcupada(f, c)) continue;
32                for (int dir = 0; dir < 4; dir++) {
33                    // cada lado del bloque (f,c) cuenta como borde SOLO
34                    // si
35                    // el vecino en esa direccion no esta ocupado (o esta
36                    // fuera de la grilla)
37                    if (!estaOcupada(f + df[dir], c + dc[dir])) perimetro
38                        ++;
39                }
40            }
41        }
42        return perimetro;
43    } // O(N*M)
44
45    //
46    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
47    //
48    // 1. Formula cerrada equivalente 4B - 2A: contar B (bloques
49    // ocupados) y
50    // A (pares de vecinos ocupados, cada par contado una sola
51    // vez) por

```

```

43 // separado, y retornar 4*B - 2*A; da el mismo resultado
44 // que revisar
45 // los 4 vecinos por celda, solo que evita revisar cada
46 // adyacencia
47 // dos veces (una desde cada bloque).
48 // 2. Perímetro por componente conéza (varias figuras
49 // separadas en la
50 // misma grilla): correr un BFS/DFS para etiquetar
51 // componentes, y
52 // aplicar la misma logica de conteo de vecinos SOLO
53 // dentro de cada
54 // componente etiquetada.
55 // 3. Grilla con bloques de tamaño k (no unitarios):
56 // multiplicar el
57 // resultado final por k, ya que cada lado unitario en la
58 // formula
59 // corresponde a un lado de longitud k en la figura real.
60 };

```

Ejemplo de funcionamiento:

```

1 vector<string> grilla = {
2     "***",
3     ".*."
4 };
5 PerimetroBloques p(grilla);
6
7 long long resultado = p.calcularPerimetro();
8 // resultado = 8
9 // (3 bloques ocupados: (0,0), (0,1), (1,1); hay 2 pares
10 // adyacentes:
11 // (0,0)-(0,1) y (0,1)-(1,1); formula: 4*3 - 2*2 = 12 - 4 = 8)

```

Regla práctica: usa esta técnica de conteo directo (revisión de vecinos, o la fórmula equivalente $4B - 2A$) siempre que la figura venga dada como una grilla de celdas ocupadas —es $O(NM)$ y evita por completo la necesidad de reconstruir un contorno poligonal explícito, que sería innecesariamente complejo para este tipo de entrada.

36.2 Número de Cuadrados en un Rectángulo $N \times M$

Idea: contar cuántos cuadrados de **cualquier tamaño** caben alineados a la grilla dentro de un rectángulo de $N \times M$ celdas (asumiendo $N \leq M$ sin pérdida de generalidad) se resuelve sumando, para cada tamaño de lado k desde 1 hasta N , la cantidad de posiciones donde un cuadrado de lado k cabe: hay $(N - k + 1)$ posiciones verticales y $(M - k + 1)$ posiciones horizontales, así que la cantidad de cuadrados de lado k es $(N - k + 1)(M - k + 1)$. Sumando sobre todos los k posibles se obtiene el total. Cuando $N = M$ (rectángulo cuadrado), esta suma se reduce a la fórmula cerrada conocida $\sum_{k=1}^N k^2 = \frac{N(N+1)(2N+1)}{6}$.

¿Por qué usarlo? Es un problema de conteo combinatorio puro —no requiere ningún cómputo geométrico continuo (sin producto cruz, sin coordenadas reales)—, pero encaja en esta parte porque describe cuántas figuras geométricas caben dentro de otra, un tipo de pregunta recurrente en problemas ad-hoc de la sección de geometría. La suma directa es $O(N)$ (con $N \leq M$), lo cual ya es suficientemente rápido para las restricciones típicas de competencia; no hace falta la fórmula cerrada salvo que N sea extremadamente grande y se necesite $O(1)$.

Casos de uso:

- Contar cuadrados de cualquier tamaño en un tablero rectangular (variante del clásico "cuántos cuadrados hay en un tablero de ajedrez").
- Problemas de conteo combinatorio sobre grillas

donde la figura buscada es un cuadrado alineado a los ejes.

- Verificación o sanity-check antes de resolver la variante más general de *Número de Rectángulos en una Grilla $N \times M$* .
- Subrutina en problemas que piden distribuir o colocar la mayor cantidad de cuadrados posibles sin superposición explícita (aquí se permite solapamiento, solo se cuenta cuántos "cabén" como subcuadrícula).

Complejidad: $O(\min(N, M))$ sumando directamente sobre los tamaños de lado posibles; $O(1)$ si se usa la fórmula cerrada para el caso $N = M$.

```

1 // CREATE/READ: Numero de cuadrados de cualquier tamaño en un
2 // rectángulo N x M
3 struct CuadradosEnRectangulo {
4     // READ: cuenta todos los cuadrados (de lado 1 hasta min(N,M))
5     // que caben
6     // alineados a la grilla dentro de un rectángulo de N x M
7     // celdas
8     static long long contar(long long n, long long m) {
9         long long menor = min(n, m);
10        long long total = 0;
11        for (long long k = 1; k <= menor; k++) {
12            total += (n - k + 1) * (m - k + 1);
13        }
14        return total;
15    } // O(min(N,M))
16
17    // READ: caso particular N == M (rectángulo cuadrado),
18    // formula cerrada
19    // sum_{k=1}^n k^2 = n(n+1)(2n+1)/6
20    static long long contarCuadrado(long long n) {
21        return n * (n + 1) * (2 * n + 1) / 6;
22    } // O(1)
23
24    //
25    // -----
26    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
27    //
28    // -----
29    // 1. Verificar overflow: para N, M grandes (~1e5 o mas), el
30    // resultado
31    // puede exceder long long dependiendo de las
32    // restricciones del
33    // problema; revisar el enunciado antes de asumir que long
34    // long basta.
35    // 2. Solo cuadrados de tamaño EXACTO k (no todos los tamaños
36    // ): basta con
37    // un solo termino (n-k+1)*(m-k+1), sin sumar sobre todos
38    // los k.
39    // 3. Formula cerrada general para N != M (no solo N == M):
40    // se puede
41    // derivar expandiendo la suma en terminos de sumas de k y
42    // k^2
43    // conocidas, pero en la practica la suma O(min(N,M)) ya
44    // es
45    // suficientemente rapida en casi todas las restricciones
46    // de CP.
47 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = CuadradosEnRectangulo::contar(2, 3);
2 // resultado = 8
3 // (cuadrados de lado 1: 2*3 = 6 posiciones; cuadrados de lado
4 // 2:
5 // (2-2+1)*(3-2+1) = 1*2 = 2 posiciones; total 6+2 = 8)

```

Regla práctica: usa la suma directa $O(\min(N, M))$ como opción general y segura; recurre a la fórmula cerrada solo en el caso particular $N = M$ y cuando N sea tan grande que incluso $O(N)$ sea inviable —en la mayoría de problemas de competencia, la suma directa ya es suficientemente rápida y más fácil de verificar que la fórmula cerrada expandida.

36.3 Número de Rectángulos en una Grilla $N \times M$

Idea: generaliza *Número de Cuadrados en un Rectángulo $N \times M$* de cuadrados a rectángulos de cualquier ancho

y alto: un rectángulo alineado a la grilla queda determinado eligiendo dos líneas verticales distintas (de las $N + 1$ líneas que delimitan las N filas) y dos líneas horizontales distintas (de las $M + 1$ que delimitan las M columnas) —cada par de líneas verticales junto con cada par de líneas horizontales define exactamente un rectángulo. El conteo es entonces puramente combinatorio: $\binom{N+1}{2} \cdot \binom{M+1}{2}$, el producto de "formas de elegir 2 líneas verticales" por "formas de elegir 2 líneas horizontales".

¿Por qué usarlo? A diferencia del conteo de cuadrados —que necesita sumar sobre cada tamaño de lado posible porque un cuadrado fuerza ancho = alto—, contar rectángulos no tiene esa restricción: ancho y alto son independientes, así que el problema se separa limpiamente en "elegir extensión horizontal" y "elegir extensión vertical", cada una una combinación simple $\binom{2}{2}$. Esto da una fórmula $O(1)$ directa, sin necesidad de ninguna suma —más simple que el caso de cuadrados, donde la restricción ancho=alto obliga a iterar.

Casos de uso:

- Contar todos los rectángulos alineados a los ejes que caben en una grilla de $N \times M$ celdas.
- Variante del clásico "cuántos rectángulos hay en un tablero de ajedrez" (caso particular $N = M = 8$).
- Verificación cruzada: sumando el conteo de rectángulos de cada ancho×alto específico debe coincidir con esta fórmula cerrada.
- Subrutina en problemas que combinan conteo de sub-rectángulos con otra condición adicional (por ejemplo, sub-rectángulos que contienen una celda marcada).

Complejidad: $O(1)$ —dos evaluaciones de $\binom{2}{2} = \frac{k(k-1)}{2}$ y una multiplicación.

```

1 // CREATE/READ: Numero de rectangulos alineados a la grilla en
  // un
2 // rectangulo de N x M celdas
3 struct RectangulosEnGrilla {
4   // READ: combinaciones de k elementos tomados de 2 en 2
5   static long long combinatoria2(long long k) {
6     return k * (k - 1) / 2;
7   } // O(1)
8
9   // READ: cuenta todos los rectangulos (de cualquier ancho y
  // alto) que
10  // caben alineados a la grilla en un rectangulo de N x M
  // celdas
11  static long long contar(long long n, long long m) {
12    // N celdas dan N+1 lineas verticales delimitadoras;
  // analogo para M
13    return combinatoria2(n + 1) * combinatoria2(m + 1);
14  } // O(1)
15
16  //
17  // MODIFICACIONES Y COMENTARIOS CON LOS USOS
18  //
19  // 1. Verificacion cruzada con Numero de Cuadrados: sumar
  // (n-k+1)*(m-k+1) sobre k=1..min(n,m) cuenta SOLO los
  // cuadrados,
21  // un subconjunto estricto de lo que cuenta esta formula (
  // todos los
22  // rectangulos, incluidos los no cuadrados).
23  // 2. Overflow: N+1 y M+1 pueden hacer que combinatoria2
  // exceda long long
24  // si N o M son extremadamente grandes (~3e9 o mas);
  // normalmente no es
25  // un problema en CP, pero vale revisar las restricciones
  // del enunciado.
26  // 3. Rectangulos que contienen una celda fija (r0, c0): se
  // puede

```

```

27 // contar por separado como (numero de elecciones de
  // lineas verticales
28 // que "abarcan" la columna c0) * (analogamente para filas
  //), en vez de
29 // la formula general sin restriccion.
30 };

```

Ejemplo de funcionamiento:

```

1 long long resultado = RectangulosEnGrilla::contar(2, 3);
2 // resultado = 18
3 // (N=2 da 3 lineas verticales, C(3,2)=3; M=3 da 4 lineas
  // horizontales,
4 // C(4,2)=6; total 3*6=18; esto incluye los 8 cuadrados ya
  // contados en
5 // la subseccion anterior, mas los rectangulos no cuadrados)

```

Regla práctica: usa esta fórmula cerrada $\binom{N+1}{2} \binom{M+1}{2}$ siempre que la pregunta sea por **todos** los rectángulos (sin restricción de ancho=alto); si la pregunta es específicamente por cuadrados, esa restricción rompe la independencia entre horizontal y vertical, y hace falta la suma de *Número de Cuadrados en un Rectángulo $N \times M$* en su lugar.

36.4 Suma de Distancias Manhattan entre Todos los Pares

Idea: calcular $\sum_{i < j} (|x_i - x_j| + |y_i - y_j|)$ para n puntos ingenuamente es $O(n^2)$. La clave para bajar a $O(n \log n)$ es que la distancia Manhattan se **descompone** en dos términos independientes: $|x_i - x_j| + |y_i - y_j|$, así que la suma total sobre todos los pares es exactamente (suma de $|x_i - x_j|$ sobre todos los pares) + (suma de $|y_i - y_j|$ sobre todos los pares) —las coordenadas x e y se pueden procesar por separado, sin necesidad de considerar puntos como pares (x, y) acoplados. Cada una de esas sumas 1D se calcula ordenando los valores y usando que, para un arreglo ordenado, la contribución de cada elemento a la suma de diferencias absolutas se puede acumular con una suma prefijo: el elemento en la posición i (0-indexado) contribuye positivamente i veces (es mayor que los i anteriores) y negativamente $(n - 1 - i)$ veces (es menor que los restantes).

¿Por qué usarlo? La fuerza bruta $O(n^2)$ es inviable para n grande (típicamente $n > 10^4$ en competencia). La descomposición en x e y por separado es posible **específicamente** porque la métrica Manhattan ya viene descompuesta en valor absoluto de diferencias por coordenada —esto NO funciona igual para distancia euclidiana, donde x e y están acoplados dentro de una raíz cuadrada y no se pueden separar de esta forma. Es una de las pocas ventajas computacionales concretas de usar Manhattan sobre euclidiana en problemas de suma de distancias.

Casos de uso:

- Suma total de distancias Manhattan entre todos los pares de un conjunto de puntos, cuando n es grande.
- Problemas de logística o redes donde el costo de conexión es proporcional a distancia Manhattan (movimiento en grilla).
- Como subrutina dentro de problemas más grandes que requieren esta suma repetidamente sobre subconjuntos (mantenida con estructuras de datos si los puntos cambian dinámicamente).
- Comparación de costo total bajo métrica Manhattan versus Chebyshev (que se relacionan mediante una

rotación de coordenadas de 45 grados, no cubierta aquí).

Complejidad: $O(n \log n)$ —dominado por ordenar las coordenadas x y las coordenadas y por separado ($O(n \log n)$ cada una); el cálculo de la suma con prefijos sobre cada arreglo ordenado es $O(n)$.

```

1 // CREATE/READ: Suma de distancias Manhattan entre todos los
  pares de n
2 // puntos, mediante descomposicion en x e y independientes
3 struct SumaManhattan {
4     // READ: suma de |v[i] - v[j]| sobre todos los pares i < j,
      para un
5     // arreglo 1D de valores (se usa una vez para x, otra vez
      para y)
6     static long long sumaDiferenciasAbsolutas(vector<long long> v
      ) {
7         sort(v.begin(), v.end());
8         int n = (int)v.size();
9         long long suma = 0;
10        long long prefijo = 0; // suma acumulada de v[0..i-1]
11        for (int i = 0; i < n; i++) {
12            // v[i] es mayor que los i elementos anteriores:
            contribuye
13            // v[i]*i - prefijo (v[i] menos cada uno de los
            anteriores)
14            suma += (long long)i * v[i] - prefijo;
15            prefijo += v[i];
16        }
17        return suma;
18    } // O(n log n), dominado por el sort
19
20    // READ: suma total de distancia Manhattan entre todos los
      pares de
21    // puntos (x[i], y[i])
22    static long long calcular(const vector<long long>& x, const
      vector<long long>& y) {
23        return sumaDiferenciasAbsolutas(x) +
            sumaDiferenciasAbsolutas(y);
24    } // O(n log n)
25
26    //
27    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
28    //
29    // 1. Distancia Chebyshev (max(|dx|, |dy|)) entre todos los
      pares: se
30    // puede reducir a Manhattan rotando las coordenadas 45
      grados
31    // (u = x+y, v = x-y), calculando Manhattan sobre (u,v) y
      dividiendo
32    // el resultado entre 2 -- relacion clasica entre ambas
      metricas.
33    // 2. Overflow: la suma total puede ser del orden de n^2 * (
      rango de
34    // coordenadas), facilmente mayor a 1e18 para n y
      coordenadas grandes;
35    // verificar los limites del problema antes de asumir que
      long long
36    // alcanza.
37    // 3. Puntos que se agregan dinamicamente (no todos conocidos
      de
38    // antemano): esta formula requiere ordenar el arreglo
      completo, asi
39    // que no es directamente incremental; para
      actualizaciones online se
40    // necesitaria un Fenwick tree indexado por coordenada
      comprimida.
41 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> x = {1, 3, 5};
2 vector<long long> y = {2, 2, 2};
3
4 long long resultado = SumaManhattan::calcular(x, y);
5 // resultado = 8
6 // (suma de |dx| sobre los 3 pares: |1-3|+|1-5|+|3-5| = 2+4+2 =
  8;
7 // suma de |dy| sobre los 3 pares: 0+0+0 = 0 (todas las y son
  iguales);
8 // total: 8 + 0 = 8)

```

Nota: la derivación de `sumaDiferenciasAbsolutas` usando $i * v[i] - \text{prefijo}$ depende de que v esté ordenado: una vez ordenado, cada elemento en la posición i es mayor o igual que los i elementos que lo preceden,

así que su contribución neta a la suma de diferencias absolutas con esos i elementos es $v[i]$ sumado i veces menos la suma de esos i elementos anteriores (prefijo).

Regla práctica: usa esta descomposición (separar x e y , ordenar cada uno, sumar con prefijos) siempre que la métrica sea Manhattan y n sea grande —es la técnica estándar para bajar de $O(n^2)$ a $O(n \log n)$ en este tipo de suma. Recuerda que esta separación es específica de Manhattan; no aplica directamente a distancia euclidiana por la raíz cuadrada que acopla x e y .

36.5 Máximo de Cuadrados 2x2 en un Triángulo Isósceles Rectángulo

Idea: sea el triángulo rectángulo isósceles con catetos de longitud b (los dos lados iguales), ángulo recto en el origen, y catetos alineados con los ejes —la hipotenusa es entonces la recta $x + y = b$. El problema de contar cuántos cuadrados de 2×2 caben alineados a la grilla dentro de este triángulo es equivalente, tras dividir todas las longitudes entre 2, a contar cuántos cuadrados unitarios caben en un triángulo análogo de cateto $L = \lfloor b/2 \rfloor$. Un cuadrado unitario con esquina inferior izquierda en (i, j) cabe completo bajo la hipotenusa si su esquina superior derecha $(i + 1, j + 1)$ satisface $(i + 1) + (j + 1) \leq L$, es decir $i + j \leq L - 2$. Contar los pares (i, j) con $i, j \geq 0$ enteros que cumplen esa desigualdad da $\sum_{s=0}^{L-2} (s + 1) = \frac{L(L-1)}{2}$.

¿Por qué usarlo? Es un problema de conteo combinatorio bajo una restricción geométrica lineal (la hipotenusa), similar en espíritu a *Número de Cuadrados en un Rectángulo $N \times M$* , pero aquí la restricción no es un rectángulo sino un triángulo, lo que cambia la condición de "cabe" de una desigualdad por coordenada a una única desigualdad conjunta $i + j \leq L - 2$. Reducir cuadrados de 2×2 a cuadrados unitarios mediante el cambio de escala $L = \lfloor b/2 \rfloor$ evita tener que rederivar la fórmula para cada tamaño de cuadrado distinto.

Casos de uso:

- Contar la máxima cantidad de baldosas cuadradas de tamaño fijo que caben en una región triangular de recorte.
- Problemas de empaquetamiento (packing) simplificado donde la región es un triángulo rectángulo isósceles.
- Como paso intermedio para estimar el área "desperdiciada" al triangular una región rectangular.
- Variante de conteo bajo la diagonal de una grilla cuadrada (equivalente al caso $L = b$ con cuadrados unitarios).

Complejidad: $O(1)$ —la fórmula cerrada se evalúa directamente sin ninguna iteración.

```

1 // CREATE/READ: Maximo numero de cuadrados de lado k que caben
  alineados a
2 // la grilla en un triangulo rectangulo isosceles de catetos de
  longitud b
3 struct CuadradosEnTriangulo {
4     // READ: cuenta cuadrados unitarios que caben bajo la
      hipotenusa de un
5     // triangulo rectangulo isosceles de cateto L (formula L*(L
      -1)/2)
6     static long long contarUnitarios(long long L) {

```

```

7   if (L < 2) return 0; // no cabe ni un cuadrado si L < 2
8   return L * (L - 1) / 2;
9 } // O(1)

10
11 // READ: cuenta cuadrados de lado k (por defecto k=2) que
12 // caben en un
13 // triangulo de cateto b, escalando al caso unitario con L =
14 // b / k
15 static long long contar(long long b, long long k = 2) {
16     long long L = b / k; // division entera: escala al
17     // triangulo equivalente
18     return contarUnitarios(L);
19 } // O(1)
20
21 //
22
23 //-----
24 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25 //
26 //-----
27 // 1. Triangulo con la BASE como hipotenusa (no como cateto):
28 // si el
29 // enunciado da la longitud de la hipotenusa en vez del
30 // cateto, hay
31 // que convertir primero: cateto = hipotenusa / sqrt(2),
32 // lo cual ya
33 // no da un resultado entero exacto en general --
34 // verificar con
35 // cuidado cual longitud especifica realmente el enunciado
36 // antes de
37 // aplicar esta formula.
38 // 2. Cuadrados que pueden sobresalir parcialmente (no exigir
39 // que quepan
40 // COMPLETOS): cambia la condicion i+j <= L-2 a una menos
41 // estricta,
42 // dando un resultado distinto; la formula de este struct
43 // asume
44 // cuadrados completamente contenidos.
45 // 3. b no divisible exactamente entre k: la division entera
46 // b/k ya
47 // trunca correctamente al mayor L util, sin necesidad de
48 // ajuste
49 // adicional.
50 };

```

Ejemplo de funcionamiento:

```

1   long long resultado = CuadradosEnTriangulo::contar(8, 2);
2   // resultado = 6
3   // (b=8, k=2, L = 8/2 = 4; contarUnitarios(4) = 4*3/2 = 6;
4   // hay que correrlo para confirmar el conteo exacto de
5   // posiciones válidas
6   // bajo la hipotenusa escalada)

```

Nota: este struct asume que la "base" b dada en el enunciado se refiere a la longitud de los **catetos** (los dos lados iguales del triángulo rectángulo isósceles), no a la hipotenusa —es la interpretación más común de "base" en este tipo de problema, pero conviene confirmar contra el enunciado exacto antes de aplicar la fórmula, ya que si "base" refiere a la hipotenusa, la conversión introduce $\sqrt{2}$ y el problema deja de tener una fórmula puramente combinatoria en enteros.

Regla práctica: usa la reducción a cuadrados unitarios ($L = \lfloor b/k \rfloor$), luego $L(L-1)/2$ siempre que el triángulo sea rectángulo isósceles con catetos alineados a los ejes; para un triángulo rectángulo no isósceles (catetos de longitudes distintas), la condición de conteo cambia y requeriría rederivar la desigualdad de la hipotenusa correspondiente.

36.6 Máximo Número de Piezas con N Cortes (Pastelero Perezoso)

Idea: el problema del pastelero perezoso (*Lazy Caterer's Sequence*) pregunta cuál es el máximo número de piezas en que se puede dividir un círculo (o cualquier región convexa) usando n cortes rectos, donde cada corte es una línea recta completa que puede cruzar a todos los cortes anteriores. La estrategia óptima es que cada nuevo corte cruce a **todos** los cortes anteriores en puntos distintos

(nunca por un punto ya usado ni paralelo a otro corte): el k -ésimo corte, al cruzar los $k-1$ cortes previos, queda dividido en k segmentos, y cada uno de esos segmentos parte una pieza existente en dos, agregando exactamente k piezas nuevas. Sumando esa ganancia para cada corte desde el primero: $p(n) = 1 + \sum_{k=1}^n k = 1 + \frac{n(n+1)}{2}$.

¿Por qué usarlo? Es un problema de conteo por recurrencia donde la clave no es geometría de coordenadas sino la observación de que **cada corte nuevo debe maximizar sus cruces** con los anteriores para maximizar piezas —cualquier corte paralelo a uno existente, o que pase por un punto de cruce ya usado, da menos piezas que el óptimo. La fórmula cerrada evita simular los cortes explícitamente; solo hace falta razonar sobre cuántos cruces nuevos introduce cada corte adicional.

Casos de uso:

- El problema clásico de "máximo número de piezas de un pastel/pizza con n cortes rectos".
- Generalización a la versión 3D (*Cake Numbers*): máximo número de piezas de un sólido convexo con n planos de corte.
- Como ejemplo introductorio de razonamiento por recurrencia en problemas de conteo geométrico.
- Verificación de que un algoritmo de simulación de cortes (para variantes con restricciones) no exceda el máximo teórico.

Complejidad: $O(1)$ —la fórmula cerrada se evalúa directamente.

```

1   // CREATE/READ: Maximo numero de piezas al hacer n cortes
2   // rectos en un
3   // pastel/region convexa (Lazy Caterer's Sequence)
4   struct PasteleroPerezoso {
5       // READ: maximo numero de piezas con n cortes, formula
6       // cerrada
7       // p(n) = 1 + n(n+1)/2
8       static long long maximoPiezas(long long n) {
9           return 1 + n * (n + 1) / 2;
10      } // O(1)
11
12      //
13      //-----
14      // MODIFICACIONES Y COMENTARIOS CON LOS USOS
15      //
16      //-----
17      // 1. Version 3D (Cake Numbers, n planos de corte en un
18      // solido convexo):
19      // formula distinta, C(n,0) + C(n,1) + C(n,2) + C(n,3),
20      // que en 2D se
21      // reduce a los primeros tres terminos (C(n,3) = 0 en el
22      // plano);
23      // struct separado si se necesita la version 3D.
24      // 2. Cortes que deben ser PARALELOS entre si (variante
25      // restringida): la
26      // formula cambia a n+1 piezas (cada corte solo agrega 1
27      // pieza, sin
28      // cruzar a los anteriores) -- mucho menor que el maximo
29      // sin
30      // restriccion.
31      // 3. Overflow: n*(n+1) puede exceder el rango de long long
32      // solo para
33      // n extremadamente grande (~3e9), poco comun en
34      // restricciones
35      // tipicas de CP, pero vale verificar el enunciado.
36  };

```

Ejemplo de funcionamiento:

```

1   long long resultado = PasteleroPerezoso::maximoPiezas(3);
2   // resultado = 7
3   // (con 3 cortes que se cruzan de a pares en puntos distintos:
4   // 1 (sin cortes) + 1 + 2 + 3 = 7 piezas)

```

Regla práctica: usa esta fórmula cuando el problema pida el **máximo** de piezas con n cortes rectos sin restricciones adicionales sobre su orientación; si el problema impone una restricción (cortes paralelos, cortes que deben pasar por un punto común), la fórmula cambia por completo y hay que rederivarla contando cuántas piezas nuevas agrega cada corte bajo esa restricción específica.

36.7 Altura Máxima de un Triángulo de Monedas

Idea: dadas n monedas, se apilan formando un triángulo donde la fila 1 tiene 1 moneda, la fila 2 tiene 2 monedas, y en general la fila k tiene k monedas —así que una altura de h filas completas consume $\sum_{k=1}^h k = \frac{h(h+1)}{2}$ monedas. El problema pide la mayor altura h tal que ese triángulo completo quepa con las n monedas disponibles, es decir, el mayor h que satisface $\frac{h(h+1)}{2} \leq n$ (las monedas sobrantes, si las hay, no alcanzan para una fila completa adicional y se descartan). Como $\frac{h(h+1)}{2}$ crece monótonamente con h , esto se resuelve con búsqueda binaria sobre h , o despejando directamente de la ecuación cuadrática $h^2 + h - 2n \leq 0$.

¿Por qué usarlo? Es el mismo patrón combinatorio que *Máximo Número de Piezas con N Cortes* —una suma de la forma $\sum k$ — pero aquí la incógnita es el límite superior de la suma dado un total fijo, no el valor de la suma dado el límite. Dos caminos válidos: *Búsqueda Binaria on Answer* (ya cubierta en la sección de Búsqueda Binaria de esta guía) sobre h probando $\frac{h(h+1)}{2} \leq n$, o resolver directamente la fórmula cuadrática con $h = \left\lfloor \frac{-1 + \sqrt{1 + 8n}}{2} \right\rfloor$. La búsqueda binaria evita cualquier riesgo de error de precisión en la raíz cuadrada de números grandes; la fórmula cerrada es más rápida pero requiere cuidado con el redondeo de `sqrt` en enteros grandes.

Casos de uso:

- El problema clásico de "cuántas filas de un triángulo de monedas/bolos/cajas se pueden completar con n unidades".
- Cualquier variante donde el costo de la fila k crece linealmente con k y se pregunta cuántas filas completas caben en un presupuesto fijo.
- Ejercicio introductorio para practicar la elección entre búsqueda binaria y fórmula cerrada cuando ambas son viables.
- Verificación cruzada: el problema es exactamente el inverso de *Máximo Número de Piezas con N Cortes* (ahí se da h y se pide la suma; aquí se da la suma y se pide h).

Complejidad: $O(\log n)$ con búsqueda binaria sobre h (rango de búsqueda $[0, n]$, aunque h real es $O(\sqrt{n})$, así que también se puede acotar el rango a $[0, \sqrt{2n} + 1]$ para menos iteraciones); $O(1)$ con la fórmula cerrada, al costo de una llamada a `sqrt`.

```
1 // CREATE/READ: Altura maxima de un triangulo de monedas que se
  // puede
2 // formar con n monedas disponibles (fila k tiene k monedas)
3 struct TrianguloMonedas {
```

```
4 // READ: monedas necesarias para completar h filas (formula h
  (h+1)/2)
5 static long long monedasParaAltura(long long h) {
6     return h * (h + 1) / 2;
7 } // O(1)
8
9 // READ: altura maxima alcanzable con n monedas, via busqueda
  binaria
10 // sobre h (mas robusto ante errores de precision que la
   formula cerrada)
11 static long long alturaMaximaBinaria(long long n) {
12     long long lo = 0, hi = 2000000000LL; // cota superior
   segura
13     while (lo < hi) {
14         long long medio = lo + (hi - lo + 1) / 2;
15         if (monedasParaAltura(medio) <= n) lo = medio;
16         else hi = medio - 1;
17     }
18     return lo;
19 } // O(log n)
20
21 // READ: altura maxima alcanzable con n monedas, via formula
   cerrada
22 // despejada de h^2 + h - 2n <= 0 (requiere verificar el
   redondeo)
23 static long long alturaMaximaFormula(long long n) {
24     long long h = (long long)((-1.0 + sqrt(1.0 + 8.0 * n)) /
   2.0);
25     // ajuste por posible error de precision de sqrt en punto
   flotante
26     while (monedasParaAltura(h + 1) <= n) h++;
27     while (h > 0 && monedasParaAltura(h) > n) h--;
28     return h;
29 } // O(1) amortizado (el ajuste final es O(1) en la practica)
30
31 //
32 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
33 //
34 // 1. Monedas sobrantes (no usadas en el triangulo completo):
   n -
35 // monedasParaAltura(alturaMaximaBinaria(n)) da la
   cantidad de
36 // monedas que quedan sin colocar tras formar la mayor
   altura posible.
37 // 2. Costo de fila k distinto (no lineal en k, por ejemplo k
   ^2 monedas
38 // por fila): la busqueda binaria sigue funcionando
   reemplazando
39 // monedasParaAltura por la suma correspondiente; la
   formula cerrada
40 // habria que rederivarla para cada caso.
41 // 3. n muy grande (overflow en 8*n dentro de la formula
   cerrada): usar
42 // long long consistentemente en monedasParaAltura evita
   overflow
43 // hasta n en el orden de 1e18; para n mayor, revisar el
   enunciado.
44 };
```

Ejemplo de funcionamiento:

```
1 long long resultado = TrianguloMonedas::alturaMaximaBinaria(10)
  ;
2 // resultado = 4
3 // (fila 1:1, fila 2:2, fila 3:3, fila 4:4; total 1+2+3+4=10
   monedas,
4 // exactamente 10, sin sobrantes; con 5 filas se necesitarian
   15, que
5 // excede las 10 disponibles)
```

Regla práctica: usa `alturaMaximaBinaria` como opción por defecto y segura frente a errores de precisión; usa `alturaMaximaFormula` solo si el rendimiento de $O(\log n)$ no es aceptable, y en ese caso no olvides el ajuste posterior con los `while` de corrección —`sqrt` en números grandes puede desviarse por error de punto flotante y dar un h incorrecto en ± 1 sin el ajuste.

Parte VII Programación Dinámica (Dynamic Programming)

Contenido en desarrollo.

Parte VIII Algoritmos Voraces (Greedy)

Contenido en desarrollo.

Parte IX Ad-hoc y Simulación

Contenido en desarrollo.

Parte X String Processing

37 Cadenas (Strings)

Contenido en desarrollo.

Parte XI BitWise

37.1 Casos de Uso

Antes de entrar en el detalle de cada técnica, esta sección da un mapa rápido de *cuándo* recurrir a bit-wise/bitmasking en competencia:

- $n \leq 20$ -24 y necesitas enumerar subconjuntos: Bitmask DP o Enumeración de Submáscaras — cada subconjunto de n elementos cabe en un entero de 32/64 bits.
- Necesitas maximizar/minimizar XOR de un par o subconjunto: Trie de Bits (pares) o Base Lineal de XOR (subconjuntos arbitrarios).
- Un elemento aparece un número impar de veces y el resto par (o viceversa): propiedades de XOR ($a \oplus a = 0$).
- Necesitas acelerar un DP o conteo por un factor de 64: `std::bitset` para representar conjuntos y operar sobre ellos en bloques de palabra.
- El estado de tu DP es "qué celdas de esta fila ya están cubiertas": DP con Perfil de Bits (*broken profile*), típico en problemas de *tiling*.
- Necesitas agregar (sumar, contar) sobre todas las submáscaras de cada máscara, para todas las máscaras: SOS DP, en vez de una enumeración ingenua $O(3^n)$.

- n es demasiado grande para 2^n pero se puede partir en dos mitades manejables (hasta 2^{20} cada una): Meet in the Middle con máscaras de bits.
- Necesitas verificar rápidamente si un número es potencia de dos, extraer el bit más bajo, o contar bits encendidos: Operaciones y Trucos Básicos, o las funciones *built-in* de GCC si el costo de una llamada de función no es crítico.

Regla práctica: Si el problema menciona explícitamente "subconjuntos", "XOR", "máscara de bits", o si n es sospechosamente pequeño (usualmente ≤ 20) mientras el resto de las restricciones sugieren un algoritmo exponencial, es una señal fuerte de que la solución pasa por alguna técnica de esta parte.

37.2 Representación Binaria de Enteros (con y sin signo)

Idea: Todo entero en una computadora se almacena como una secuencia fija de bits (32 para `int`, 64 para `long long`). Para enteros *sin signo* (`unsigned`), esos bits representan directamente el valor en base 2. Para enteros *con signo*, C++ usa la representación en **complemento a 2** (*two's complement*): el bit más significativo actúa como bit de signo, y un número negativo $-x$ se representa como el complemento a 1 de x (invertir todos los bits) más 1. Esta elección no es arbitraria: permite que la suma y la resta de enteros con signo usen exactamente el mismo circuito de hardware que para enteros sin signo, sin necesidad de lógica especial para manejar signos.

¿Por qué usarlo? Entender esta representación es la base para todo lo demás en esta parte: sin saber cómo se codifica un número negativo en binario, operaciones como $\sim x$ (complemento a 1), desplazamientos aritméticos, o el comportamiento de `overflow` en `int` parecen mágicas o impredecibles. Se distingue de una simple "conversión a binario" (ver más adelante) en que aquí el foco es la codificación del *signo*, no solo la representación de la magnitud.

Casos de uso:

- Predecir el resultado exacto de operaciones bitwise ($\sim$, $\&$, $|$, $\wedge$) sobre enteros negativos.
- Entender por qué -1 en complemento a 2 tiene *todos* sus bits en 1 (útil como máscara "todo encendido").
- Depurar comportamiento inesperado al mezclar `int` y `unsigned int` en la misma expresión.
- Implementar operaciones de bajo nivel (compresión, hashing, checksums) que dependen del patrón de bits exacto.

Complejidad: $O(1)$ — la representación es una propiedad fija del tipo de dato, no un algoritmo; todas las conversiones mentales entre valor y bits se hacen en tiempo constante sobre un número fijo de bits (32 o 64).

```
1 struct RepresentacionBinaria {
2
3     // READ: retorna el complemento a 1 de x (invierte todos los
```

```

4 // bits); equivalente al operador ~ de C++
5 static unsigned int complementoA1(unsigned int x) {
6     return ~x;
7 } // O(1)
8
9 // READ: retorna el complemento a 2 de x, que es la
10 // representacion de -x en enteros con signo (complemento a 1
11 // mas 1)
12 static unsigned int complementoA2(unsigned int x) {
13     return ~x + 1;
14 } // O(1)
15
16 // READ: retorna la representacion en string de los bits de x,
17 // desde el bit mas significativo hasta el menos
18 // significativo
19 static string aBinario(unsigned int x, int bits = 32) {
20     string resultado;
21     for (int i = bits - 1; i >= 0; i--) {
22         resultado += ((x >> i) & 1) ? '1' : '0';
23     }
24     return resultado;
25 } // O(bits)
26
27 // -----
28 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
29 // -----
30 // 1. Por que -1 es "todo unos": aplicando complementoA2(1),
31 // -1 + 1 = ...11111110 + 1 = ...11111111, es decir, todos
32 // los bits en 1. Por eso "-1" es una mascara util para
33 // "todos los bits encendidos" (equivalente a
34 // numeric_limits<unsigned int>::max()).
35 // 2. Reinterpretar bits sin cambiar el patron: para ver el
36 // patron de bits de un int negativo como si fuera
37 // unsigned
38 // (sin cambiar ningun bit), se puede castear:
39 // unsigned int u = (unsigned int)x;
40 // Esto NO cambia el patron de bits, solo la
41 // interpretacion.
42 // 3. Rango representable: con complemento a 2 de b bits, el
43 // rango de un tipo con signo es  $[-2^{b-1}, 2^{b-1} - 1]$ 
44 // (asimetrico: un negativo mas que positivos), mientras
45 // que sin signo es  $[0, 2^b - 1]$ . Este desbalance es la razon
46 // por la que, por ejemplo, abs(INT_MIN) es comportamiento
47 // indefinido en int de 32 bits.
48 };

```

Ejemplo de funcionamiento:

```

1 unsigned int x = 5; // 00000000 00000000 00000000 00000101
2
3 string bits = RepresentacionBinaria::aBinario(x, 8);
4 // bits = "00000101"
5
6 unsigned int comp1 = RepresentacionBinaria::complementoA1(x);
7 // comp1 en los ultimos 8 bits: 11111010 (complemento a 1 de 5)
8
9 unsigned int comp2 = RepresentacionBinaria::complementoA2(x);
10 // comp2 en los ultimos 8 bits: 11111011 (representa -5 en
11 // signed)
12
13 int menosCinco = -5;
14 unsigned int patronDeMenosCinco = (unsigned int)menosCinco;
15 // patronDeMenosCinco == comp2 (mismo patron de bits que -5)

```

Nota: El comportamiento de desplazamientos y overflow sobre enteros *con signo* negativos involucra casos de comportamiento indefinido en C++ (por ejemplo, $x \ll n$ con x negativo es UB antes de C++20). Para operaciones bitwise predecibles sobre bits individuales, es más seguro trabajar con `unsigned int` o `unsigned long long` explícitamente.

Regla práctica: Si vas a manipular bits directamente (máscaras, complementos, desplazamientos), preferir tipos `unsigned` para evitar comportamiento indefinido; usa `int/long long` con signo solo cuando el valor numérico (no el patrón de bits) es lo que importa.

37.3 Operaciones y Trucos Básicos

Idea: C++ ofrece un conjunto pequeño de operadores bitwise (`&`, `|`, `^`, `~`, `<<`, `>>`) que operan sobre la representación binaria de un entero bit a bit, en paralelo, en una sola instrucción de hardware. A partir de esos operadores primitivos se derivan "trucos" (*bit tricks*) que resuelven en $O(1)$ operaciones que de otra forma requerirían un bucle: extraer un bit específico, prender/apagar un bit, verificar si un número es potencia de dos, o aislar el bit más bajo encendido. La idea central detrás de todos estos trucos es aprovechar que las operaciones bitwise procesan los 32 o 64 bits de un entero simultáneamente, en vez de iterar posición por posición.

¿Por qué usarlo? Estos trucos son la base sobre la que se construyen todas las técnicas más avanzadas de esta parte (Submask Enumeration, SOS DP, Bitmask DP, XOR Basis, etc.) — sin dominarlos, el código de las secciones siguientes es difícil de leer o depurar. Frente a una solución con bucles explícitos (por ejemplo, contar bits encendidos dividiendo repetidamente entre 2), los trucos bitwise suelen ser más rápidos en la práctica y, sobre todo, más cortos de escribir bajo presión de tiempo en competencia.

Casos de uso:

- Representar un subconjunto de hasta 64 elementos como un único entero, para usar como estado de un Bitmask DP.
- Verificar rápidamente si un número es potencia de dos (útil en problemas de segment trees, sparse tables, FFT).
- Aislar o eliminar el bit menos significativo encendido, base de estructuras como el Árbol de Fenwick (BIT).
- Alternar el estado de un bit (toggle), común en problemas de simulación con interruptores o estados binarios.
- Verificar paridad de un número (par/impar) sin usar el operador módulo.

Complejidad: $O(1)$ por operación — cada operador bitwise se traduce en una sola instrucción de máquina que actúa sobre la palabra completa (32 o 64 bits) en un ciclo de reloj, independientemente del valor del número.

```

1 struct TrucosBitwise {
2
3     // READ: retorna 1 si el bit en la posicion i (0-indexado
4     // desde
5     // el bit menos significativo) esta encendido, 0 si esta
6     // apagado
7     static int obtenerBit(long long x, int i) {
8         return (x >> i) & 1LL;
9     } // O(1)
10
11     // WRITE: retorna x con el bit en la posicion i encendido
12     // (puesto en 1), sin modificar los demas bits
13     static long long prenderBit(long long x, int i) {
14         return x | (1LL << i);
15     } // O(1)
16
17     // WRITE: retorna x con el bit en la posicion i apagado
18     // (puesto en 0), sin modificar los demas bits
19     static long long apagarBit(long long x, int i) {
20         return x & ~(1LL << i);
21     } // O(1)
22
23     // WRITE: retorna x con el bit en la posicion i invertido
24     // (0 pasa a 1, 1 pasa a 0)
25     static long long alternarBit(long long x, int i) {

```

```

24     return x ^ (1LL << i);
25 } // O(1)
26
27 // READ: retorna true si x es una potencia de 2 (incluyendo
28 // x = 1 = 2^0); false para x <= 0
29 static bool esPotenciaDeDos(long long x) {
30     return x > 0 && (x & (x - 1)) == 0;
31 } // O(1)
32
33 // READ: retorna un entero con solo el bit menos
34 // significativo
35 // encendido de x prendido, y todos los demas apagados
36 // (aislar el "lowest set bit")
37 static long long aislarBitMasBajo(long long x) {
38     return x & (-x);
39 } // O(1)
40
41 // WRITE: retorna x con su bit menos significativo encendido
42 // apagado (elimina el "lowest set bit")
43 static long long apagarBitMasBajo(long long x) {
44     return x & (x - 1);
45 } // O(1)
46
47 //
48 // -----
49 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
50 // -----
51 // 1. Por que x & (-x) aisla el bit mas bajo: en complemento
52 // a 2,
53 // -x = -x + 1. Todos los bits antes del bit mas bajo
54 // encendido de x son 0 tanto en x como en -x (se
55 // preservan
56 // al invertir y sumar 1), y a partir de ese bit los
57 // patrones
58 // son complementarios; el AND deja solo ese bit.
59 //
60 // 2. Por que x & (x - 1) apaga el bit mas bajo: restar 1 a x
61 // invierte todos los bits desde el bit mas bajo encendido
62 // hacia abajo (los ceros se vuelven unos, y ese bit se
63 // apaga); el AND con x conserva todo excepto ese bit.
64 //
65 // 3. Verificar paridad sin modulo: (x & 1) == 0 equivale a
66 // x % 2 == 0, pero evita la instruccion de division/
67 // modulo,
68 // que es mas lenta que un AND en la mayoria de
69 // arquitecturas.
70 //
71 // 4. Contar bits encendidos con apagarBitMasBajo: un bucle
72 // "while (x) { x = apagarBitMasBajo(x); cont++; }" cuenta
73 // los
74 // bits en O(numero de bits encendidos) en vez de O(32/64);
75 //
76 // en la practica, __builtin_popcount es preferible por
77 // ser
78 // una sola instruccion de hardware (ver la subseccion de
79 // funciones built-in de GCC).
80 };

```

Ejemplo de funcionamiento:

```

1 long long x = 0b1011010; // 90 en decimal
2
3 int bit3 = TrucosBitwise::obtenerBit(x, 3); // bit3 = 1 (cuarto
4 // bit desde la derecha)
5
6 long long conBit0 = TrucosBitwise::prenderBit(x, 0);
7 // conBit0 = 0b1011011 = 91
8
9 long long sinBit4 = TrucosBitwise::apagarBit(x, 4);
10 // sinBit4 = 0b1001010 = 74
11
12 bool esPot = TrucosBitwise::esPotenciaDeDos(64); // esPot =
13 // true
14 bool esPot2 = TrucosBitwise::esPotenciaDeDos(90); // esPot2 =
15 // false
16
17 long long masBajo = TrucosBitwise::aislarBitMasBajo(x);
18 // masBajo = 0b0000010 = 2 (bit mas bajo encendido de 90)

```

Regla práctica: Ante cualquier problema que hable de "conjunto de hasta 20-24 elementos", "estado binario" o "prender/ apagar/alternar", piensa primero en representar el estado como un entero y usar estos trucos antes de recurrir a un `vector<bool>` o arreglo explícito — es más compacto, más rápido, y se integra directamente con Bitmask DP y Submask Enumeration.

37.4 Desplazamientos Aritméticos vs. Lógicos (Arithmetic vs. Logical Shift)

Idea: El operador `»` en C++ se comporta de forma distinta según el tipo del operando: sobre un entero `unsigned`, es un desplazamiento *lógico*, que rellena con ceros por la izquierda sin importar el valor. Sobre un entero `signed`, es un desplazamiento *aritmético*, que replica el bit de signo (el bit más significativo) al desplazar, preservando así el signo del número: desplazar un negativo a la derecha sigue dando un negativo. El operador `«` no tiene esta ambigüedad al desplazar (siempre rellena con ceros por la derecha), pero desplazar un `signed` negativo con `«` es comportamiento indefinido antes de C++20.

¿Por qué usarlo? Confundir estos dos comportamientos es una fuente común de bugs sutiles en competencia: dividir por potencias de 2 usando `»` sobre un `int` negativo (por ejemplo, en una búsqueda binaria con `(izq + der) » 1` cuando alguno de los límites puede ser negativo) da un resultado distinto —y matemáticamente distinto de la división entera real hacia cero— al que se obtendría sobre un `unsigned`. Entender la diferencia evita depurar durante horas un desplazamiento que "debería" funcionar igual que una división.

Casos de uso:

- Calcular el punto medio en búsqueda binaria sin overflow, verificando qué tipo de shift aplica si algún límite es negativo.
- Dividir por potencias de 2 rápidamente en tipos `unsigned` (equivalente exacto a la división entera).
- Implementar rotaciones de bits (*bit rotation*), que requieren combinar shifts lógicos en ambas direcciones.
- Evitar bugs al portar código que asume `int` sin signo entre plataformas o al mezclar `int/unsigned`.

Complejidad: $O(1)$ — al igual que las demás operaciones bitwise, un desplazamiento (lógico o aritmético) se ejecuta en una sola instrucción de hardware sin importar la magnitud del desplazamiento (siempre que sea menor al número de bits del tipo).

```

1 struct Desplazamientos {
2
3     // READ: aplica un shift logico a la derecha (rellena con
4     // ceros
5     // sin importar el signo), forzando la interpretacion
6     // unsigned
7     static long long shiftLogico(long long x, int n) {
8         return (long long)((unsigned long long)x >> n);
9     } // O(1)
10
11     // READ: aplica un shift aritmetico a la derecha (replica el
12     // bit
13     // de signo), usando directamente el operador sobre signed
14     static long long shiftAritmetico(long long x, int n) {
15         return x >> n; // sobre long long (signed), >> ya es
16         // aritmetico
17     } // O(1)
18
19     // READ: rota los bits de un unsigned de 32 bits hacia la
20     // izquierda por n posiciones (los bits que "salen" por la
21     // izquierda reaparecen por la derecha)
22     static unsigned int rotarIzquierda(unsigned int x, int n) {
23         n %= 32;
24         return (x << n) | (x >> (32 - n));
25     } // O(1)
26
27     //
28     // -----

```

```

24 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
25 //
26 // 1. Division vs. shift aritmetico en negativos: para x
27 // negativo, x >> 1 (aritmetico) NO es lo mismo que x / 2
28 // en
29 // C++: la division trunca hacia cero, pero el shift
30 // aritmetico redondea hacia -infinito (floor). Ejemplo:
31 // -7 / 2 == -3, pero -7 >> 1 == -4.
32 //
33 // 2. Punto medio seguro en busqueda binaria: usar
34 // izq + (der - izq) / 2 en vez de (izq + der) >> 1 evita
35 // tanto el overflow de (izq + der) como la ambigüedad de
36 // shift si algun limite fuera negativo (aunque en indices
37 // arreglo normalmente ambos son no negativos).
38 //
39 // 3. Rotacion a la derecha: analogo a rotarIzquierda pero
40 // intercambiando los shifts:
41 // (x >> n) | (x << (32 - n)); // sobre unsigned int
42 //
43 // 4. C++20 y shifts definidos: desde C++20, x << n con x
44 // negativo dejo de ser UB y se define como shift sobre el
45 // patron de bits (equivalente a operar sobre unsigned);
46 // en estandares anteriores, evitar shifts sobre signed
47 // negativos por completo.
48 };

```

Ejemplo de funcionamiento:

```

1 long long negativo = -8; // ...11111000 en complemento a 2
2
3 long long resAritmetico = Desplazamientos::shiftAritmetico(
4     negativo, 1);
5 // resAritmetico = -4 (replica el bit de signo, sigue negativo)
6
7 long long resLogico = Desplazamientos::shiftLogico(negativo, 1)
8 ;
9 // resLogico = un numero positivo enorme (se interpreto como
10 // unsigned de 64 bits y se relleno con ceros)
11
12 unsigned int val = 0b00000001000000000000000000000000; // bit
13 // 24 encendido
14
15 unsigned int rotado = Desplazamientos::rotarIzquierda(val, 4);
16 // rotado tiene el bit 28 encendido (el bit 24 se desplazo 4
17 // posiciones)

```

Nota: $-7 \gg 1$ en un int con signo da -4 (shift aritmético, redondea hacia $-\infty$), mientras que $-7 / 2$ da -3 (división entera, trunca hacia 0). Si necesitas división exacta por potencias de 2 con posibles negativos, usa el operador $/$ directamente y no un shift.

Regla práctica: Usa $\gg$ como atajo de división por potencias de 2 *solo* sobre tipos `unsigned` o cuando tengas certeza de que el operando es no negativo; si el operando puede ser negativo y necesitas semántica de división real (truncar hacia cero), usa el operador $/$ explícitamente en vez de $\gg$.

37.5 Funciones Built-in de GCC

Idea: GCC (y por extensión Clang, en su mayoría) expone un conjunto de funciones `__builtin_*` que se traducen directamente a instrucciones especializadas del procesador (cuando existen) para operaciones bit a bit comunes: contar bits encendidos, contar ceros iniciales/finales, verificar paridad. A diferencia de implementar estas operaciones manualmente con bucles o con los trucos de la subsección anterior, estas funciones aprovechan instrucciones de hardware dedicadas (como POPCNT o BSF/BSR en x86), lo que las hace, en la práctica, más rápidas y sin necesidad de escribir el algoritmo a mano.

¿Por qué usarlo? Aunque los trucos manuales de "Operaciones y Trucos Básicos" (como `x & (x-1)` en un bucle para contar bits) funcionan correctamente, estas funciones built-in son la opción preferida en competencia

por dos razones: son más cortas de escribir bajo presión de tiempo, y su implementación en hardware es frecuentemente $O(1)$ real (una instrucción), en vez de $O(\text{bits encendidos})$ como el bucle manual. La contrapartida es que dependen del compilador (GCC/Clang; no existen en MSVC con el mismo nombre) y tienen casos borde con el valor 0 que hay que conocer.

Casos de uso:

- Contar el número de elementos en un subconjunto representado como máscara de bits (**popcount**), central en Bitmask DP.
- Encontrar la posición del bit más significativo encendido (**clz**), útil para calcular $\lfloor \log_2 x \rfloor$ o la siguiente potencia de 2.
- Encontrar la posición del bit menos significativo encendido (**ctz**), alternativa a `x & (-x)` para obtener el índice en vez del valor del bit.
- Verificar paridad del número de bits encendidos (**parity**), usado en checksums simples o problemas de conteo de inversiones con propiedades de paridad.

Complejidad: $O(1)$ en la gran mayoría de arquitecturas modernas (x86-64 con soporte para POPCNT/LZCNT/TZCNT), ya que se traducen a una sola instrucción de hardware; en arquitecturas sin esas instrucciones, GCC genera una secuencia de instrucciones equivalente que sigue siendo significativamente más rápida que un bucle interpretado a mano.

```

1 struct FuncionesBuiltinGCC {
2
3     // READ: retorna la cantidad de bits encendidos (1s) en x,
4     // usando la version de 32 bits
5     static int contarBitsEncendidos(unsigned int x) {
6         return __builtin_popcount(x);
7     } // O(1)
8
9     // READ: version de 64 bits de popcount, para long long
10    static int contarBitsEncendidos64(unsigned long long x) {
11        return __builtin_popcountll(x);
12    } // O(1)
13
14    // READ: retorna la cantidad de ceros consecutivos desde el
15    // bit
16    // MAS significativo hasta el primer 1 (count leading zeros);
17    // comportamiento indefinido si x == 0
18    static int cerosIniciales(unsigned int x) {
19        return __builtin_clz(x);
20    } // O(1)
21
22    // READ: retorna la cantidad de ceros consecutivos desde el
23    // bit
24    // MENOS significativo hasta el primer 1 (count trailing
25    // zeros),
26    // es decir, el indice del bit mas bajo encendido; UB si x ==
27    // 0
28    static int cerosFinales(unsigned int x) {
29        return __builtin_ctz(x);
30    } // O(1)
31
32    // READ: retorna la posicion (0-indexada) del bit mas
33    // significativo encendido de x, equivalente a floor(log2(x));
34
35    // asume x > 0
36    static int posicionBitMasAlto(unsigned int x) {
37        return 31 - __builtin_clz(x);
38    } // O(1)
39
40    //
41    // -----
42    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
43    //
44    // -----
45    // 1. Caso borde x == 0: __builtin_clz(0) y __builtin_ctz(0)
46    // son
47    // comportamiento indefinido (usualmente retornan 32 o un
48    // valor basura, dependiendo de la arquitectura). SIEMPRE

```

```

41 // verificar x != 0 antes de llamarlas.
42 //
43 // 2. Sufijos por tipo: cada funcion tiene variante segun el
44 // tamaño del argumento:
45 // __builtin_popcount / __builtin_clz / __builtin_ctz ->
46 // unsigned int (32 bits)
47 // __builtin_popcountll / __builtin_clzll /
48 // __builtin_ctzll -> unsigned long long (64 bits)
49 // Pasar un long long a la version sin "ll" trunca a 32
50 // bits
51 // silenciosamente y da resultados incorrectos.
52 //
53 // 3. __builtin_parity: retorna 1 si la cantidad de bits
54 // encendidos es impar, 0 si es par. Equivalente a
55 // __builtin_popcount(x) & 1, pero potencialmente mas
56 // rapido
57 // en algunas arquitecturas.
58 //
59 // 4. Siguiente potencia de 2 mayor o igual a x (x > 0):
60 // x == 1 ? 1 : 1 << (32 - __builtin_clz(x - 1));
61 // se apoya en clz para encontrar el bit mas alto
62 // necesario.
63 //
64 // 5. No disponibles en MSVC: estas funciones son especificas
65 // de
66 // GCC/Clang. En MSVC los equivalentes son __popcnt,
67 // _BitScanForward, _BitScanReverse. En Codeforces/la
68 // mayoría
69 // de jueces (que usan GCC), esto no es un problema.
70 };

```

Ejemplo de funcionamiento:

```

1 unsigned int x = 0b1011010; // 90 en decimal
2
3 int bits = FuncionesBuiltinGCC::contarBitsEncendidos(x);
4 // bits = 4 (hay cuatro 1s en 1011010)
5
6 int cerosIni = FuncionesBuiltinGCC::cerosIniciales(x);
7 // cerosIni = 25 (32 bits totales - 7 bits significativos = 25
8 // ceros
9 // antes del primer 1)
10
11 int cerosFin = FuncionesBuiltinGCC::cerosFinales(x);
12 // cerosFin = 1 (el bit menos significativo encendido esta en
13 // la
14 // posición 1: ...1011010, el ultimo bit es 0)
15
16 int posAlto = FuncionesBuiltinGCC::posicionBitMasAlto(x);
17 // posAlto = 6 (el bit mas significativo encendido esta en la
18 // posición 6, ya que 90 = 1011010 en binario, 7 bits, indice
19 // 0-6)

```

Nota: Revisa cuidadosamente el nombre de la función usada (`cerosIniciales` y `ceros_finales` arriba deben escribirse sin espacios como identificador válido de C++ — `cerosIniciales`); este es un error tipográfico común al transcribir a mano.

Regla práctica: Prefiere siempre las funciones `__builtin_*` sobre una implementación manual con bucles para `popcount`, `clz` o `ctz` en cualquier juez que use GCC (Codeforces, la mayoría de jueces de ICPC); solo verifica el caso `x == 0` por separado si tu problema puede recibir ese valor.

37.6 Conversión entre Bases Numéricas (Binario, Octal, Hexadecimal)

Idea: Cualquier entero puede representarse en distintas bases (2, 8, 16, etc.) sin cambiar su valor, solo su notación. La conversión entre binario y bases potencias de 2 (octal = 2^3 , hexadecimal = 2^4) es especialmente directa porque cada dígito de esas bases corresponde exactamente a un grupo fijo de bits (3 bits para octal, 4 bits para hexadecimal), sin necesidad de hacer divisiones sucesivas como al convertir a una base arbitraria (por ejemplo, base 10). Esta correspondencia directa es la razón por la que hexadecimal es la notación preferida para representar patrones de bits de forma compacta y legible.

¿Por qué usarlo? Trabajar directamente con literales binarios largos (0b101101101011) es propenso a errores de conteo; la notación hexadecimal agrupa esos mismos bits en dígitos de a 4 (0xB6B), mucho más fácil de leer y escribir correctamente. Además, muchas herramientas de depuración, formatos de archivo, y problemas de competencia (hashing, criptografía simple, checksums) presentan datos directamente en hexadecimal, así que convertir con fluidez entre representaciones es una habilidad práctica necesaria, distinta de las operaciones bitwise en sí (que operan sobre el valor, no sobre su representación textual).

Casos de uso:

- Leer o imprimir valores en un formato de entrada/salida que especifica hexadecimal u octal (frecuente en problemas de parsing de datos crudos o protocolos).
- Depurar manualmente el valor de una máscara de bits, convirtiéndola a hexadecimal para verificar qué bits están encendidos de un vistazo.
- Implementar conversores de base como subrutina en problemas de teoría de números o representación numérica.
- Trabajar con colores en formato RGB hexadecimal (0xFF00FF) en problemas que simulan gráficos o imágenes.

Complejidad: $O(\log_b x)$ para convertir un número x a base b mediante divisiones sucesivas (el número de dígitos de x en base b), lo cual es $O(\log x)$ en general. Para bases potencia de 2, la conversión es $O(\text{bits}/k)$ donde $k = \log_2 b$ (por ejemplo, $k = 4$ para hexadecimal), ya que se procesan grupos de k bits a la vez sin necesidad de división real.

```

1 struct ConversionDeBases {
2
3     // READ: convierte un entero no negativo a su representacion
4     // en
5     // string en la base indicada (2 <= base <= 16), sin ceros a
6     // la
7     // izquierda (excepto para el valor 0)
8     static string aBase(long long x, int base) {
9         if (x == 0) return "0";
10        string digitos = "0123456789ABCDEF";
11        string resultado;
12        while (x > 0) {
13            resultado += digitos[x % base];
14            x /= base;
15        }
16        reverse(resultado.begin(), resultado.end());
17        return resultado;
18    } // O(log_base(x))
19
20    // READ: convierte un string en una base dada (2 <= base <=
21    // 16)
22    // de vuelta a su valor entero; asume digitos validos para
23    // esa base
24    static long long desdeBase(const string& s, int base) {
25        long long resultado = 0;
26        for (char c : s) {
27            int valorDigito = (c >= '0' && c <= '9') ? c - '0' : c -
28            'A' + 10;
29            resultado = resultado * base + valorDigito;
30        }
31        return resultado;
32    } // O(longitud de s)
33
34    // READ: version rapida de binario a decimal, agrupando de a
35    // 1
36    // bit (caso general de "desdeBase" especializado para base
37    // 2)
38    static long long binarioADecimal(const string& binario) {
39        return desdeBase(binario, 2);
40    }
41 };

```

```

33     } // O(longitud del string)
34
35     //
36     // -----
37     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38     // -----
39     // 1. Conversion directa binario <-> hexadecimal por grupos:
40     //    como
41     //    16 = 2^4, cada digito hexadecimal corresponde
42     //    exactamente a
43     //    4 bits binarios, sin necesidad de pasar por decimal:
44     //    agrupar el binario de derecha a izquierda en bloques de
45     //    4
46     //    (rellenando con ceros a la izquierda si es necesario) y
47     //    convertir cada bloque independientemente.
48     //
49     // 2. Uso de bitset para conversion rapida: para un numero de
50     //    ancho fijo conocido, bitset<32>(x).to_string() da la
51     //    representacion binaria completa (con ceros a la
52     //    izquierda)
53     //    en O(32) sin escribir un bucle manual.
54     //
55     // 3. Formato nativo de C++ con printf/cout: printf("%x", x)
56     //    o
57     //    printf("%o", x) imprimen directamente en hexadecimal u
58     //    octal; con cout, se usa std::hex o std::oct como
59     //    manipulador de stream (cout << hex << x).
60     //
61     // 4. Literales en el codigo fuente: C++14 en adelante
62     //    permite
63     //    escribir literales binarios directamente como 0b1011010,
64     //    ademas de los ya existentes 0x (hex) y 0 seguido de
65     //    digitos
66     //    (octal, ej. 017 == 15 en decimal).
67 };

```

Ejemplo de funcionamiento:

```

1  long long x = 2739; // valor en decimal
2
3  string hex = ConversionDeBases::aBase(x, 16);
4  // hex = "AB3" (2739 = 10*256 + 11*16 + 3)
5
6  string oct = ConversionDeBases::aBase(x, 8);
7  // oct = "5263" (2739 en octal)
8
9  string bin = ConversionDeBases::aBase(x, 2);
10 // bin = "101010110011"
11
12 long long deVuelta = ConversionDeBases::desdeBase("AB3", 16);
13 // deVuelta = 2739 (vuelve al valor original)

```

Regla práctica: Usa hexadecimal (no binario ni octal) como notación por defecto para depurar o escribir máscaras de bits de más de 8 bits — la correspondencia 1 dígito = 4 bits hace que sea mucho más fácil detectar errores a simple vista que contar ceros y unos en una cadena binaria larga.

37.7 Overflow y Wraparound en Enteros

Idea: Cada tipo entero tiene un número fijo de bits, y por lo tanto un rango finito de valores representables. Cuando una operación aritmética produce un resultado fuera de ese rango, ocurre *overflow*. El comportamiento difiere radicalmente según el tipo: en enteros **unsigned**, el overflow está **definido** por el estándar de C++ como aritmética módulo 2^b (*wraparound*): sumar 1 al máximo valor representable da 0, como un odómetro que da la vuelta. En enteros **signed**, el overflow es **comportamiento indefinido** (UB): el compilador puede asumir que nunca ocurre y optimizar el código de formas que rompen la lógica esperada, en vez de simplemente "dar la vuelta" de forma predecible.

¿Por qué usarlo? Entender esta distinción es crítico porque el overflow en **signed** no es solo "un resultado incorrecto pero predecible" — es UB, lo que significa que el compilador puede generar código que se comporta de forma sorprendente (por ejemplo, eliminar una

verificación como `if (x + 1 > x)` asumiendo que siempre es verdadera, porque "asume" que no hay overflow). Esto se relaciona directamente con Representación Binaria y Complemento a 2: el wraparound en **unsigned** es consecuencia directa de la aritmética módulo 2^b que implementa esa representación, mientras que el rango asimétrico del complemento a 2 (un negativo más que positivos) es la razón de casos borde como `INT_MIN`.

Casos de uso:

- Detectar y prevenir overflow al multiplicar valores grandes antes de aplicar aritmética modular (frecuente en teoría de números y combinatoria).
- Aprovechar el wraparound de **unsigned** de forma intencional para implementar aritmética modular con módulo 2^{32} o 2^{64} sin necesidad de un operador % explícito.
- Evitar el caso borde clásico `abs(INT_MIN)`, que es UB porque `-INT_MIN` no es representable en **int**.
- Elegir el tipo de dato correcto (**int** vs. **long long** vs. **unsigned long long**) según el rango esperado de resultados intermedios, no solo del resultado final.

Complejidad: $O(1)$ — el overflow es una propiedad del resultado de una operación aritmética individual, no de un algoritmo; verificar overflow antes de una operación también toma tiempo constante por chequeo.

```

1  struct OverflowYWraparound {
2
3      // READ: verifica si a + b desborda el rango de long long,
4      // sin realizar la suma directamente (evita el UB de
5      // calcularla)
6      static bool sumaDesborda(long long a, long long b) {
7          if (b > 0 && a > LLONG_MAX - b) return true; // desborda
8          hacia arriba
9          if (b < 0 && a < LLONG_MIN - b) return true; // desborda
10         hacia abajo
11         return false;
12     } // O(1)
13
14     // READ: verifica si a * b desborda el rango de long long,
15     // sin calcular el producto directamente
16     static bool multiplicacionDesborda(long long a, long long b)
17     {
18         if (a == 0 || b == 0) return false;
19         long long resultado = a * b;
20         return resultado / b != a; // si no se puede "deshacer",
21         hubo overflow
22     } // O(1)
23
24     // READ: demuestra wraparound definido sobre unsigned: suma 1
25     // al
26     // maximo valor de unsigned int y retorna el resultado (0)
27     static unsigned int demostrarWraparound() {
28         unsigned int maximo = UINT_MAX; // 4294967295 en 32 bits
29         return maximo + 1u; // wraparound definido a 0
30     } // O(1)
31
32     //
33     // -----
34     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
35     // -----
36     // 1. __builtin_add_overflow / __builtin_mul_overflow: GCC
37     // ofrece
38     // funciones built-in que realizan la operacion y retornan
39     // si
40     // hubo overflow en un solo paso, evitando escribir a mano
41     // la
42     // logica de sumaDesborda/multiplicacionDesborda:
43     // long long resultado;
44     // bool desbordo = __builtin_mul_overflow(a, b, &resultado
45     // );
46     //
47     // 2. Caso INT_MIN y valor absoluto: abs(INT_MIN) es UB
48     // porque
49     // -INT_MIN = 2147483648, que excede INT_MAX (2147483647)
50     // en

```

```

38 // complemento a 2. Solucion: castear a un tipo mas grande
39 // antes de tomar el valor absoluto, ej.
40 // llabs((long long)x), o trabajar con unsigned si solo
41 // importa la magnitud.
42 //
43 // 3. Wraparound como herramienta (hashing con overflow
44 // intencional): algunos esquemas de hashing simple usan
45 // deliberadamente el wraparound de unsigned de 32/64 bits
46 // como "modulo gratis" (modulo 2^32 o 2^64), en vez de
47 // aplicar
48 // % explicito, porque el wraparound de unsigned SI esta
49 // definido por el estandar y es portable.
50 //
51 // 4. Overflow silencioso en int vs deteccion en tiempo de
52 // ejecucion: en modo release, el overflow de signed
53 // simplemente produce un valor incorrecto sin aviso; usar
54 // flags de compilacion como -fsanitize=undefined durante
55 // el desarrollo/pruebas ayuda a detectarlo antes del envio.
56 };

```

Ejemplo de funcionamiento:

```

1 long long a = LLONG_MAX - 5;
2 bool desb = OverflowWraparound::sumaDesborda(a, 10);
3 // desb = true (a + 10 excede LLONG_MAX)
4
5 unsigned int resultado = OverflowWraparound::
6   demostrarWraparound();
7 // resultado = 0 (UINT_MAX + 1 da la vuelta a 0, comportamiento
8   definido)
9
10 long long x = 3000000000LL, y = 3000000000LL;
11 bool desbMult = OverflowWraparound::multiplicacionDesborda(x,
12   y);
13 // desbMult = true (el producto excede el rango de long long)

```

Nota: El overflow de `signed` es UB incluso si en la práctica "parece funcionar" con wraparound en tu compilador y arquitectura actuales — el compilador puede optimizar asumiendo que nunca ocurre, y ese supuesto puede romper el programa de formas que no son visibles hasta cambiar de compilador, nivel de optimización, o plataforma.

Regla práctica: Usa `long long` por defecto para cualquier cálculo intermedio que pueda acercarse a 10^9 (el rango de `int`), y verifica overflow explícitamente antes de multiplicaciones grandes en vez de confiar en que "probablemente no pase" — es una de las causas más comunes de *Wrong Answer* silencioso en competencia.

37.8 Operaciones Matemáticas en Binario (Suma, Resta, Multiplicación y Exponenciación Modular)

Idea: Las operaciones aritméticas básicas (suma, resta, multiplicación) pueden entenderse y ejecutarse directamente sobre la representación binaria, replicando el mismo algoritmo que se enseña en base 10 pero con acarreo (*carry*) en base 2: sumar bit a bit propagando el acarreo, restar con préstamo (*borrow*), y multiplicar mediante desplazamientos y sumas (cada bit encendido del multiplicador aporta una copia desplazada del multiplicando). La **exponenciación modular** extiende esta idea combinando desplazamientos con multiplicación modular: se descompone el exponente en su representación binaria y se usa "elevar al cuadrado y multiplicar" (*binary exponentiation*) recorriendo esos bits, reduciendo un problema de $O(n)$ multiplicaciones a $O(\log n)$.

¿Por qué usarlo? Entender la suma/resta/multiplicación a nivel de bits no es solo una curiosidad académica: la exponenciación modular que se deriva de esa misma idea (usar la representación binaria del exponente) es una de las

herramientas más usadas en toda la programación competitiva, para calcular $a^b \bmod m$ en $O(\log b)$ en vez de $O(b)$ multiplicaciones directas — esencial cuando b puede ser de hasta 10^{18} . Se relaciona directamente con Overflow y Wraparound: cada multiplicación intermedia en exponenciación modular puede desbordar `long long` si no se aplica el módulo después de cada paso, no solo al final.

Casos de uso:

- Calcular $a^b \bmod m$ eficientemente, base de teoría de números (inverso modular vía Fermat, RSA simplificado, conteo combinatorio modular).
- Calcular el inverso modular de a módulo un primo p como $a^{p-2} \bmod p$ (pequeño teorema de Fermat), necesario para "dividir" en aritmética modular.
- Multiplicación rápida por desplazamientos cuando uno de los operandos es una potencia de 2 o tiene pocos bits encendidos.
- Implementar multiplicación de matrices módulo m elevada a una potencia grande, para resolver recurrencias lineales (Fibonacci, etc.) en $O(\log n)$.

Complejidad: $O(\log b)$ para exponenciación modular de $a^b \bmod m$, ya que el exponente b se recorre bit a bit (su representación binaria tiene $O(\log b)$ bits), realizando una multiplicación modular (costo $O(1)$ para enteros que caben en `long long`, o $O(\log^2 m)$ si se requiere multiplicación modular de enteros grandes tipo *big integer*) por cada bit.

```

1 struct OperacionesBinarias {
2
3   // READ: calcula (base^exponente) % mod usando exponenciacion
4   // binaria (elevar al cuadrado y multiplicar), recorriendo
5   // los
6   // bits de "exponente" de menos a mas significativo
7   static long long potenciaModular(long long base, long long
8   exponente, long long mod) {
9     long long resultado = 1 % mod;
10    base %= mod;
11    while (exponente > 0) {
12      if (exponente & 1) { // bit actual encendido: incluir
13        esta potencia
14        resultado = (__int128)resultado * base % mod;
15      }
16      base = (__int128)base * base % mod; // elevar al cuadrado
17      para el siguiente bit
18      exponente >>= 1; // avanzar al siguiente bit del
19      exponente
20    }
21    return resultado;
22  } // O(log(exponente))
23
24   // READ: calcula el inverso modular de a modulo un primo p,
25   // usando el pequeno teorema de Fermat: a^(p-2) mod p == a^-1
26   // mod p
27   static long long inversoModular(long long a, long long p) {
28     return potenciaModular(a, p - 2, p);
29   } // O(log p)
30
31   // READ: multiplica a y b bit a bit mediante desplazamientos
32   // y
33   // sumas (algoritmo "campesino ruso"), como demostracion de
34   // la
35   // mecanica interna de la multiplicacion binaria
36   static long long multiplicacionPorBits(long long a, long long
37   b) {
38     long long resultado = 0;
39     while (b > 0) {
40       if (b & 1) resultado += a; // bit actual encendido: sumar
41       el multiplicando desplazado
42       a <<= 1; // desplazar el multiplicando (equivalente a *2)
43       b >>= 1; // avanzar al siguiente bit del multiplicador
44     }
45     return resultado;
46   } // O(log b)
47 }

```

```

37 //
38 // -----
39 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
40 // -----
41 // 1. Por que se usa __int128 en potenciaModular: el producto
42 // de
43 // dos long long que caben casi al limite (~9.2 * 10^18)
44 // puede desbordar long long antes de aplicar el modulo;
45 // usar
46 // __int128 como tipo intermedio evita ese overflow (ver
47 // Overflow y Wraparound). Alternativa sin __int128: usar
48 // "multiplicacion binaria modular" (aplicar la misma idea
49 // de
50 // multiplicacionPorBits, pero reduciendo modulo m en cada
51 // suma), a costa de  $O(\log b)$  en vez de  $O(1)$  por
52 // multiplicacion.
53 //
54 // 2. Exponenciacion de matrices: el mismo esqueleto de
55 // potenciaModular se generaliza reemplazando la
56 // multiplicacion escalar por multiplicacion de matrices
57 // (modular), para resolver recurrencias lineales como
58 // Fibonacci en  $O(k^3 \log n)$ , con  $k$  el tamaño de la matriz.
59 //
60 // 3. Resta con acarreo (borrow) a nivel de bits: analoga a
61 // la
62 // suma pero propagando un prestamo en vez de un acarreo;
63 // en
64 // la practica, C++ ya maneja esto de forma nativa con el
65 // operador -, y rara vez se implementa a mano salvo en
66 // aritmetica de precision arbitraria (big integers).
67 //
68 // 4. Exponente negativo: potenciaModular no maneja
69 // exponentes
70 // negativos directamente; para  $a^{-k} \bmod p$ , calcular
71 // primero
72 // el inverso modular de  $a$  y luego elevarlo a  $k$ :
73 // potenciaModular(inversoModular(a, p), k, p).
74 };

```

Ejemplo de funcionamiento:

```

1 long long mod = 1000000007LL;
2
3 long long resultado = OperacionesBinarias::potenciaModular(2,
4 30, mod);
5 // resultado = 1073741824 % mod = 1073741824 (no alcanza a
6 // superar mod)
7
8 long long inv = OperacionesBinarias::inversoModular(5, mod);
9 // inv es tal que (5 * inv) % mod == 1;
10 // Nota: el valor exacto de inv depende del calculo de  $5^{-(mod-2)} \bmod$ ;
11 // se debe correr el codigo para confirmar el valor exacto
12 // antes de
13 // darlo por verificado.
14
15 long long prodBits = OperacionesBinarias::multiplicacionPorBits
16 (13, 11);
17 // prodBits = 143 (13 * 11, calculado bit a bit sin usar el
18 // operador *)

```

Regla práctica: Usa exponenciación binaria (potenciaModular) en cualquier problema que pida $a^b \bmod m$ con b grande — es la técnica estándar y su complejidad logarítmica la hace viable incluso para $b \sim 10^{18}$; recuerda aplicar el módulo después de *cada* multiplicación intermedia, no solo al resultado final, y usar `__int128` (o multiplicación modular bit a bit) si `mod` se acerca al límite de `long long`.

37.9 Bitset para Optimización (std::bitset)

Idea: `std::bitset<N>` es un contenedor de tamaño fijo que representa N bits y expone operaciones bitwise (`&`, `|`, `^`, `<<`, `>>`) sobre *todo* el conjunto de bits a la vez. Internamente, el bitset empaqueta los N bits en bloques de 64 bits (palabras de máquina) y ejecuta cada operación bloque por bloque usando instrucciones nativas del procesador, en vez de bit por bit. Esto significa que una operación sobre N bits que "debería" costar $O(N)$ en un `vector<bool>` operado ingenuamente, cuesta $O(N/64)$

con `bitset`, porque cada instrucción de máquina procesa 64 bits en paralelo.

¿Por qué usarlo? El bitset convierte los trucos de "Operaciones y Trucos Básicos" (que operan sobre un solo entero de hasta 64 bits) en una herramienta escalable a conjuntos de miles o millones de elementos: cualquier DP o algoritmo que se pueda expresar como operaciones de conjunto (unión, intersección, "¿es alcanzable este estado?") sobre un universo de tamaño N se acelera por un factor constante de ≈ 64 frente a la misma lógica con `vector<bool>` o `vector<int>` recorrido elemento por elemento. Es la herramienta a considerar cuando Bitmask DP (que representa subconjuntos de hasta 20-24 elementos en un solo `int/long long`) no alcanza porque el universo de elementos es demasiado grande para caber en 64 bits.

Casos de uso:

- Acelerar DP de subconjuntos alcanzables (por ejemplo, "¿qué sumas son alcanzables con un subconjunto de n números?") de $O(n \cdot S)$ a $O(n \cdot S/64)$, con S el rango de sumas.
- Calcular la cantidad de substrings comunes o patrones de coincidencia entre dos strings largos usando bitsets de coincidencia por posición.
- Resolver problemas de conteo de pares que cumplen una condición de conjunto (por ejemplo, cuántos pares de conjuntos se intersectan) representando cada conjunto como un bitset.
- Optimizar transiciones de grafos/DP donde el estado es "qué nodos son alcanzables", operando con `&/|` sobre el bitset completo en vez de iterar nodo por nodo.

Complejidad: $O(N/64)$ (o $O(N/w)$, con w el tamaño de palabra de la arquitectura, típicamente 64) para operaciones bitwise sobre un bitset de N bits, frente a $O(N)$ de la misma operación implementada elemento por elemento; operaciones como `count()` (contar bits encendidos) también se benefician de instrucciones `POPCNT` por bloque, manteniendo el mismo factor de mejora.

```

1 const int MAXN = 100000;
2
3 struct OptimizacionConBitset {
4     bitset<MAXN> conjunto;
5
6     // CREATE: inicializa el bitset vacio (todos los bits en 0)
7     OptimizacionConBitset() : conjunto() {} //  $O(N/64)$ 
8
9     // WRITE: agrega el elemento x al conjunto (prende el bit x)
10    void agregar(int x) {
11        conjunto.set(x);
12    } //  $O(1)$ 
13
14    // READ: verifica si el elemento x esta en el conjunto
15    bool contiene(int x) {
16        return conjunto.test(x);
17    } //  $O(1)$ 
18
19    // READ: retorna un nuevo bitset con la union de este
20    // conjunto y
21    // "otro" (elementos presentes en cualquiera de los dos)
22    bitset<MAXN> unionCon(const bitset<MAXN>& otro) {
23        return conjunto | otro;
24    } //  $O(N/64)$ 
25
26    // READ: retorna un nuevo bitset con la interseccion de este
27    // conjunto y "otro" (elementos presentes en ambos)
28    bitset<MAXN> interseccionCon(const bitset<MAXN>& otro) {
29        return conjunto & otro;
30    } //  $O(N/64)$ 

```

```

30 // READ: retorna la cantidad de elementos (bits encendidos)
31 // en
32 // el conjunto
33 int tamano() {
34     return (int)conjunto.count();
35 } // O(N/64)
36
37 //
38 // -----
39 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
40 // -----
41 // 1. Subset-sum con bitset: para saber que sumas son
42 //    alcanzables
43 //    con un subconjunto de un arreglo de n numeros no
44 //    negativos
45 //    con suma total S, se mantiene bitset<S+1> alcanzables
46 //    con
47 //    el bit 0 encendido, y por cada numero v del arreglo se
48 //    hace
49 //    alcanzables |= (alcanzables << v);
50 //    esto agrega, para cada suma ya alcanzable, la suma
51 //    resultante de incluir v, en O(S/64) por numero.
52 //
53 // 2. bitset de tamano NO conocido en tiempo de compilacion:
54 //    std::bitset requiere que N sea una constante en tiempo
55 //    de
56 //    compilacion; si el tamano es dinamico, usar
57 //    boost::dynamic_bitset o simular con vector<uint64_t> y
58 //    operaciones manuales bloque a bloque.
59 //
60 // 3. Conteo de coincidencias entre strings con bitset: para
61 //    verificar si el patron P aparece en el texto T en la
62 //    posicion i, se puede precomputar, para cada caracter c,
63 //    un
64 //    bitset con las posiciones donde aparece c en T, y
65 //    combinar
66 //    con shifts y AND para verificar coincidencias multiples
67 //    en
68 //    O(|T| |alfabeto| / 64) en vez de O(|T||P|).
69 //
70 // 4. any(), none(), all(): metodos utiles para verificar
71 //    rapidamente si existe al menos un bit encendido (any()),
72 //    ninguno (none()), o todos (all()), cada uno en O(N/64).
73 };

```

Ejemplo de funcionamiento:

```

1 // Subset-sum: que sumas son alcanzables con {2, 3, 7}?
2 vector<int> numeros = {2, 3, 7};
3 bitset<20> alcanzables;
4 alcanzables[0] = 1; // suma 0 siempre alcanzable (subconjunto
5 // vacio)
6
7 for (int v : numeros) {
8     alcanzables |= (alcanzables << v);
9 }
10 // alcanzables tiene encendidos los bits: 0, 2, 3, 5, 7, 9, 10,
11 // 12
12 // (todas las sumas posibles combinando subconjuntos de
13 // {2,3,7})
14
15 bool sumaCincoAlcanzable = alcanzables[5]; // true (2 + 3 = 5)
16 bool sumaCuatroAlcanzable = alcanzables[4]; // false

```

Regla práctica: Si Bitmask DP requiere representar subconjuntos de más de 64 elementos, o si tu DP/algoritmo puede expresarse como operaciones de conjunto (unión, intersección, "shift y OR" para propagar alcanzabilidad) sobre un universo de hasta unos pocos millones de elementos, usa `bitset` en vez de `vector<bool>` — la ganancia de $\approx 64\times$ suele ser la diferencia entre pasar o no el límite de tiempo.

37.10 Enumeración de Submáscaras (Submask Enumeration)

Idea: Dada una máscara de bits m , una *submáscara* es cualquier máscara s tal que $s \& m = s$ (es decir, todo bit encendido de s también está encendido en m). Hay 2^k submáscaras de una máscara con k bits encendidos, y existe un patrón de iteración clásico que las recorre todas, en orden estrictamente descendente, en $O(2^k)$

usando la operación $s = (s - 1) \& m$: restar 1 a s "toma prestado" del bit encendido más bajo, y el AND con m descarta cualquier bit que no pertenezca a la máscara original, produciendo exactamente la siguiente submáscara menor.

¿Por qué usarlo? Sin este patrón, enumerar las submáscaras de m requeriría recorrer las 2^n máscaras posibles (con n el número total de bits) y filtrar las que cumplen $s \& m = s$, un desperdicio si m tiene pocos bits encendidos. El patrón $(s-1) \& m$ visita únicamente las 2^k submáscaras reales, sin generar ni descartar candidatos inválidos. Es el bloque de construcción fundamental de SOS DP (que agrega esta enumeración sobre *todas* las máscaras del universo) y de varios problemas de Bitmask DP donde la transición itera sobre subconjuntos de un conjunto dado.

Casos de uso:

- Iterar todas las formas de dividir un subconjunto de elementos en dos partes, en problemas de partición o asignación.
- Transiciones de Bitmask DP donde, dado el conjunto de elementos aun disponibles, se prueba cada subconjunto no vacío para asignar a la siguiente "ronda" o "grupo".
- Como subrutina dentro de SOS DP, para entender el algoritmo ingenuo $O(3^n)$ antes de optimizarlo a $O(2^n \cdot n)$.
- Precalcular, para cada máscara, una propiedad agregada sobre todos sus subconjuntos (por ejemplo, el máximo valor de un arreglo restringido a los índices de la submáscara).

Complejidad: $O(2^k)$ para enumerar las submáscaras de una única máscara con k bits encendidos. Si se enumeran las submáscaras de *todas* las máscaras de n bits, la complejidad total es $O(3^n)$: cada uno de los n bits puede estar en uno de tres estados (no pertenece a la máscara, pertenece a la máscara pero no a la submáscara, o pertenece a ambas), de donde sale el factor 3^n en vez de $2^n \cdot 2^n$.

```

1 struct EnumeracionDeSubmascaras {
2
3     // READ: retorna todas las submascaras de m (incluyendo m
4     // misma
5     // y la mascara vacia 0), en orden estrictamente descendente
6     static vector<int> obtenerSubmascaras(int m) {
7         vector<int> resultado;
8         for (int s = m; ; s = (s - 1) & m) {
9             resultado.push_back(s);
10            if (s == 0) break; // se llevo a la submascara vacia,
11            terminar
12        }
13        return resultado;
14    } // O(2^k), k = bits encendidos de m
15
16    // READ: aplica una funcion "procesar" a cada submascara no
17    // vacia
18    // de m, sin almacenarlas todas en memoria (mas eficiente si
19    // solo
20    // se necesita un acumulado)
21    template<typename Func>
22    static void paraCadaSubmascaraNoVacia(int m, Func procesar) {
23        for (int s = m; s > 0; s = (s - 1) & m) {
24            procesar(s);
25        }
26    } // O(2^k), k = bits encendidos de m
27
28    //
29    // -----
30    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31    // -----
32 };

```

```

26 // -----
27 // 1. Por que (s - 1) & m da la siguiente submascara: restar
28 // 1 a
29 // s apaga el bit encendido mas bajo de s y prende todos
30 // los
31 // bits menores a el (equivalente a apagarBitMasBajo pero
32 // "tomando prestado" en vez de apagando). El AND con m
33 // descarta los bits recién prendidos que no pertenecen a
34 // m,
35 // dejando exactamente la submascara inmediatamente menor.
36 //
37 // 2. Incluir o excluir la mascara vacia (s = 0): el bucle
38 // "for (s = m; ; s = (s-1)&m) { ...; if (s == 0) break;
39 // }"
40 // incluye tanto m como 0 como submascaras validas; para
41 // excluir la vacia, usar la condicion "s > 0" en el for
42 // como
43 // en paraCadaSubmascaraNoVacía.
44 //
45 // 3. Enumerar submascaras de TODAS las mascaras (base de SOS
46 // DP):
47 // for (int m = 0; m < (1 << n); m++)
48 // for (int s = m; ; s = (s-1)&m) {
49 //     // procesar(m, s)
50 //     if (s == 0) break;
51 // }
52 // este doble bucle anidado es O(3^n) en total, no O(4^n),
53 // por el argumento de los tres estados por bit.
54 //
55 // 4. Complementos de submascaras: la submascara "
56 // complementaria"
57 // de s dentro de m (los elementos de m que NO estan en s)
58 // se
59 // obtiene como (m ^ s), util quando se necesita dividir m
60 // en
61 // dos partes disjuntas.
62 };

```

Ejemplo de funcionamiento:

```

1 int m = 0b1010; // mascara con bits 1 y 3 encendidos (decimal
2 10)
3 vector<int> submascaras = EnumeracionDeSubmascaras::
4 obtenerSubmascaras(m);
5 // submascaras = {0b1010, 0b1000, 0b0010, 0b0000}
6 // es decir: {10, 8, 2, 0} - las 4 = 2^2 submascaras de una
7 // mascara
8 // con 2 bits encendidos
9
10 int suma = 0;
11 EnumeracionDeSubmascaras::paraCadaSubmascaraNoVacía(m, [&](int
12 s) {
13     suma += s;
14 });
15 // suma = 8 + 2 + 10 = 20 (excluye la submascara vacía 0)

```

Regla práctica: Usa el patrón $s = (s-1) \& m$ en vez de recorrer todas las 2^n máscaras y filtrar cuando necesites iterar submáscaras de una máscara específica; si en cambio necesitas esta enumeración para *todas* las máscaras del universo simultáneamente (agregando un valor por cada par máscara-submáscara), reconoce que el patrón general es $O(3^n)$ y considera si SOS DP (siguiente subsección) puede reducirlo a $O(2^n \cdot n)$.

37.11 Suma sobre Subconjuntos (SOS DP)

Idea: SOS DP (*Sum over Subsets*) resuelve un problema específico: dado un arreglo $f[\text{mask}]$ indexado por todas las 2^n máscaras posibles, calcular para cada máscara m la suma (u otra agregación asociativa: máximo, OR, conteo) de $f[s]$ sobre *todas* las submáscaras s de m . La enumeración ingenua de "Submask Enumeration" resuelve esto en $O(3^n)$ recorriendo, para cada máscara, todas sus submáscaras una por una. SOS DP reduce esto a $O(2^n \cdot n)$ procesando la agregación *bit por bit*: en la iteración i , para cada máscara con el bit i encendido, se decide si "incluir" las submáscaras que difieren en ese bit, acumulando el resultado progresivamente en vez de enumerar cada submáscara de forma explícita.

¿Por qué usarlo? La diferencia entre $O(3^n)$ y $O(2^n \cdot n)$ es enorme en la práctica: para $n = 20$, $3^{20} \approx 3.5 \times 10^9$ es demasiado lento, mientras que $2^{20} \cdot 20 \approx 2 \times 10^7$ es trivial. SOS DP logra esto reordenando el cálculo: en vez de "para cada máscara, sumar sobre todas sus submáscaras" (que repite trabajo entre máscaras que comparten submáscaras), procesa "para cada bit, propagar la contribución de las máscaras sin ese bit hacia las máscaras con ese bit", de forma similar a cómo un Fenwick Tree evita recalcular sumas de prefijo desde cero. Es la técnica a usar en cuanto Submask Enumeration se vuelve el cuello de botella por tener que repetirse para todas las máscaras del universo.

Casos de uso:

- Calcular, para cada máscara m , la suma de todos los valores $f[s]$ con s submáscara de m (el caso base de SOS DP).
- Contar, para cada máscara m , cuántos elementos de un arreglo tienen su máscara de bits como submáscara de m (versión "conteo" en vez de "suma").
- Resolver variantes de "AND/OR convolution" donde se necesita $g[m] = \sum_{s \subseteq m} f[s]$ como paso intermedio.
- Problemas de conteo con restricciones tipo "cuántos subconjuntos de propiedades cumple el elemento x ", donde cada propiedad es un bit.

Complejidad: $O(2^n \cdot n)$: hay n iteraciones (una por cada bit), y en cada una se recorren las 2^n máscaras posibles actualizando el acumulado en $O(1)$ por máscara, dando $n \cdot 2^n$ operaciones en total — una mejora exponencial frente al $O(3^n)$ de la enumeración ingenua de submáscaras para todas las máscaras.

```

1 struct SumaSobreSubconjuntos {
2     int n; // numero de bits del universo
3     vector<long long> dp; // dp[mask] = suma de f[s] para toda
4     // submascara s de mask
5
6     // CREATE: inicializa dp con los valores base f[mask] (la
7     // suma
8     // "trivial" de cada mascara consigo misma antes de propagar)
9     SumaSobreSubconjuntos(vector<long long> f, int numBits) : n(
10     numBits), dp(f) {} // O(2^n)
11
12     // WRITE: ejecuta el algoritmo de SOS DP, transformando dp[
13     // mask]
14     // de "valor base de mask" a "suma sobre todas las
15     // submascaras de mask"
16     void calcular() {
17         int totalMascaras = 1 << n;
18         for (int bit = 0; bit < n; bit++) {
19             for (int mask = 0; mask < totalMascaras; mask++) {
20                 if (mask & (1 << bit)) {
21                     // mask tiene el bit encendido: suma la contribucion
22                     // de la mascara sin ese bit (que ya incluye sus
23                     // propias submascaras, procesadas en iteraciones
24                     // de bits anteriores)
25                     dp[mask] += dp[mask ^ (1 << bit)];
26                 }
27             }
28         } // O(2^n * n)
29
30         // READ: retorna la suma sobre subconjuntos ya calculada para
31         // "mask"; debe llamarse despues de calcular()
32         long long consultar(int mask) {
33             return dp[mask];
34         } // O(1)
35
36     // -----
37     // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38     // -----
39 };

```

```

35 // 1. Por que el orden de los bucles es bit-afuera, mask-
36 //    adentro:
37 //    procesar todos los bits en orden fijo garantiza que, al
38 //    llegar al bit i, dp[mask ^ (1<<bit)] ya acumulo
39 //    correctamente las contribuciones de los bits 0..i-1; si
40 //    se
41 //    invirtiera el orden de los bucles, se perderia esa
42 //    garantia
43 //    y el resultado seria incorrecto (similar a por que el
44 //    orden de las capacidades importa en la mochila 0/1).
45 //
46 // 2. Version "superset sum" (suma sobre superconjuntos):
47 //    para
48 //    calcular, para cada mask, la suma de f[s] con mask
49 //    submascara de s (en vez de al reves), se invierte la
50 //    condicion: if (!(mask & (1 << bit))) dp[mask] += dp[
51 //    mask | (1 << bit)];
52 //
53 // 3. Generalizacion a otras operaciones asociativas:
54 //    reemplazar
55 //    "+" por "max(...)", "|=" (OR), o incrementar un
56 //    contador
57 //    resuelve variantes como "maximo valor entre las
58 //    submascaras" o "conteo de elementos cuya mascara es
59 //    submascara de mask", sin cambiar la estructura del
60 //    algoritmo.
61 //
62 // 4. Memoria  $O(2^n)$ : a diferencia de Submask Enumeration (
63 //    que no
64 //    requiere almacenar nada extra), SOS DP necesita el
65 //    arreglo
66 //    dp completo de tamano  $2^n$ , lo cual limita n en la
67 //    practica
68 //    a unos 20-24 bits por restricciones de memoria.
69 };

```

Ejemplo de funcionamiento:

```

1 // f[mask] = valor base asociado a cada mascara (universo de 3
2 // bits)
3 vector<long long> f = {1, 2, 3, 4, 5, 6, 7, 8}; // f[0..7]
4
5 SumaSobreSubconjuntos sos(f, 3);
6 sos.calcular();
7
8 long long resultado = sos.consultar(0b101); // mask = 5
9 // submascaras de 5 (101) son: 000, 001, 100, 101 -> f[0]+f[1]+
10 // f[4]+f[5]
11 // resultado = 1 + 2 + 5 + 6 = 14

```

Regla práctica: Usa SOS DP en cuanto necesites calcular la agregación sobre submáscaras para *todas* las máscaras del universo simultáneamente (no solo una); si solo necesitas las submáscaras de una única máscara puntual, el patrón $(s-1) \& m$ de Submask Enumeration es más simple y no requiere memoria $O(2^n)$ adicional.

37.12 Bitmask DP (Programación Dinámica sobre Subconjuntos)

Idea: Bitmask DP representa el *estado* de una programación dinámica como un entero donde cada bit indica si un elemento particular ya fue usado, visitado, o cumple cierta condición. En vez de un estado con n dimensiones booleanas independientes (inviabile de indexar directamente), se codifica el conjunto completo de n elementos como un único entero en $[0, 2^n)$, permitiendo usar ese entero como índice de un arreglo de DP. La transición entre estados típicamente prende un bit adicional (agregar un elemento al subconjunto) o, con Submask Enumeration, itera sobre todos los subconjuntos de los elementos aún disponibles.

¿Por qué usarlo? Muchos problemas de optimización combinatoria son NP-difíciles en general (como el Problema del Vendedor Viajero), pero se vuelven resolubles para n pequeño (típicamente hasta 20-24) mediante Bitmask DP, bajando la complejidad de $O(n!)$ (probar todas las permutaciones) a $O(2^n \cdot n)$ o $O(2^n \cdot n^2)$. Se distingue

de SOS DP en que aquí la máscara representa el *estado* de la solución parcial que se está construyendo (qué elementos ya se usaron), mientras que en SOS DP la máscara es simplemente un índice sobre el cual se agregan valores precomputados; y se distingue del DP con Perfil de Bits en que aquí la máscara codifica un subconjunto de *elementos*, no un perfil espacial de una fila/columna de una grilla.

Casos de uso:

- Problema del Vendedor Viajero (TSP): encontrar el camino mínimo que visita todas las ciudades exactamente una vez.
- Asignación óptima de n tareas a n trabajadores minimizando el costo total (problema de asignación, más rápido con Bitmask DP para n pequeño que con Húngaro completo si $n \leq 20$).
- Cubrir un conjunto de elementos con el mínimo número de subconjuntos predefinidos (Set Cover), factible en $O(2^n)$ para n pequeño.
- Contar o encontrar la forma óptima de emparejar elementos en un grafo bipartito pequeño, como alternativa a Bipartite Matching cuando además se necesita optimizar un costo.

Complejidad: $O(2^n \cdot n)$ típicamente, donde 2^n son los posibles estados (subconjuntos) y n es el costo de la transición desde cada estado (probar agregar cada uno de los n elementos); en problemas como TSP la complejidad sube a $O(2^n \cdot n^2)$ porque el estado incluye tanto el subconjunto visitado como el último elemento visitado (n posibilidades adicionales).

```

1 const long long INF = 1e18;
2
3 struct BitmaskDP_TSP {
4     int n;
5     vector<vector<long long>> dist; // matriz de distancias entre
6     // ciudades
7     vector<vector<long long>> dp; // dp[mask][i] = costo minimo
8     // visitando
9     // exactamente el conjunto "mask",
10    // terminando en la ciudad i
11
12    // CREATE: inicializa la matriz de distancias y la tabla dp
13    // con
14    // INF, marcando el caso base (empezar solo en la ciudad 0)
15    BitmaskDP_TSP(vector<vector<long long>> matrizDistancias)
16    : n((int)matrizDistancias.size()), dist(matrizDistancias) {
17        dp.assign(1 << n, vector<long long>(n, INF));
18        dp[1][0] = 0; // mascara con solo la ciudad 0 visitada,
19        // terminando en 0
20    } //  $O(2^n \cdot n)$ 
21
22    // WRITE: llena la tabla dp iterando sobre todos los estados
23    // (mascara, ciudad actual) y probando transicionar a cada
24    // ciudad no visitada aun
25    void calcular() {
26        int totalMascaras = 1 << n;
27        for (int mask = 1; mask < totalMascaras; mask++) {
28            for (int actual = 0; actual < n; actual++) {
29                if (!(mask & (1 << actual))) continue; // actual
30                // no esta en mask, invalido
31                if (dp[mask][actual] == INF) continue; // estado
32                // inalcanzable, saltar
33
34                for (int siguiente = 0; siguiente < n; siguiente++) {
35                    if (mask & (1 << siguiente)) continue; // siguiente
36                    // ya visitada
37                    int nuevaMascara = mask | (1 << siguiente);
38                    long long nuevoCosto = dp[mask][actual] + dist[actual][siguiente];
39                    dp[nuevaMascara][siguiente] = min(dp[nuevaMascara][siguiente], nuevoCosto);
40                }
41            }
42        }
43    }
44 }

```

```

35     }
36 } // O(2^n * n^2)
37
38 // READ: retorna el costo minimo de visitar todas las
39 // ciudades
40 // exactamente una vez y regresar a la ciudad 0; debe
41 // llamarse
42 // despues de calcular()
43 long long costoMinimo() {
44     int mascaraCompleta = (1 << n) - 1;
45     long long mejor = INF;
46     for (int ultima = 1; ultima < n; ultima++) {
47         if (dp[mascaraCompleta][ultima] == INF) continue;
48         mejor = min(mejor, dp[mascaraCompleta][ultima] + dist[
49             ultima][0]);
50     }
51     return mejor;
52 } // O(n)
53
54 // -----
55 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
56 // -----
57 // 1. Reconstruccion del camino optimo: guardar un arreglo
58 // padre[mask][actual] con la ciudad anterior que produjo
59 // el
60 // minimo, y reconstruir el camino hacia atras desde
61 // (mascaraCompleta, ultima) hasta llegar a la mascara
62 // inicial.
63 // 2. Set Cover con Bitmask DP: dp[mask] = minimo numero de
64 // subconjuntos para cubrir exactamente los elementos en
65 // mask;
66 // se itera sobre submascaras (Submask Enumeration) del
67 // complemento de mask para probar agregar cada
68 // subconjunto
69 // predefinido disponible.
70 // 3. Problema de asignacion (Assignment Problem): dp[mask] =
71 // costo minimo asignando las primeras popcount(mask)
72 // tareas
73 // a los trabajadores marcados en mask; la transicion
74 // prueba
75 // asignar la siguiente tarea a cada trabajador no usado
76 // aun.
77 // 4. Limite practico de n: Bitmask DP con 2^n estados es
78 // viable
79 // hasta n ~ 20-22 en la mayoria de limites de tiempo (1-2
80 // segundos); para n mayor, es necesario buscar una
81 // aproximacion, heuristica, o una propiedad especial del
82 // problema que reduzca el espacio de estados.
83 };

```

Ejemplo de funcionamiento:

```

1 vector<vector<long long>> distancias = {
2     {0, 10, 15, 20},
3     {10, 0, 35, 25},
4     {15, 35, 0, 30},
5     {20, 25, 30, 0}
6 };
7
8 BitmaskDP_TSP tsp(distancias);
9 tsp.calcular();
10
11 long long costo = tsp.costoMinimo();
12 // Nota: el valor exacto depende de recorrer las 4!
13 // permutaciones
14 // posibles del ciclo; se debe correr el codigo para confirmar
15 // el
16 // valor exacto antes de darlo por verificado (candidato
17 // esperado
18 // alrededor de 80, correspondiente a la ruta 0-1-3-2-0 o
19 // equivalente).

```

Regla práctica: Reconoce Bitmask DP cuando el problema mencione "orden óptimo de visitar/asignar n elementos" con $n \leq 20-24$, y el estado natural del problema sea "qué subconjunto de elementos ya se procesó" (posiblemente combinado con "en qué elemento se terminó"); si además necesitas agregar valores sobre subconjuntos de un universo fijo en vez de construir una solución paso a paso, esa es SOS DP en vez de Bitmask DP.

37.13 DP con Perfil de Bits (Broken Profile DP)

Idea: El DP con perfil de bits (*broken profile DP*) resuelve problemas de *tiling* o cobertura sobre una grilla 2D, donde no se puede procesar cada celda de forma completamente independiente porque las decisiones sobre celdas vecinas están acopladas (por ejemplo, colocar una ficha de dominó de 1×2 afecta dos celdas a la vez). La idea es procesar la grilla celda por celda, en orden fila por fila, y usar una máscara de bits (el "perfil") para representar el estado de las últimas m celdas procesadas (típicamente el ancho de una fila, o la mitad de una ficha que "sobresale" hacia la fila siguiente). Cada bit del perfil indica si esa celda ya está cubierta/ocupada, y la transición avanza una celda a la vez, actualizando el perfil en consecuencia.

¿Por qué usarlo? Se distingue de Bitmask DP clásico en que aquí la máscara **no** representa un subconjunto de elementos abstractos, sino un *perfil espacial*: qué celdas de la frontera actual entre "ya decidido" y "aún por decidir" están ocupadas. Esto permite resolver problemas de *tiling* en $O(\text{filas} \cdot \text{columnas} \cdot 2^{\text{columnas}})$ en vez de intentar enumerar todas las formas de cubrir la grilla directamente (factorial o peor). Es la técnica correcta cuando el ancho de la grilla es pequeño (hasta 15-20) pero el largo puede ser grande, ya que el costo exponencial recae solo sobre el ancho.

Casos de uso:

- Contar el número de formas de cubrir completamente una grilla $n \times m$ con fichas de dominó (1×2).
- Contar tilings con poliomínos más complejos (piezas en L, piezas de 3 o más celdas) sobre una grilla pequeña.
- Problemas de "máximo número de piezas colocables" sujeto a restricciones de no superposición en una grilla.
- Variantes donde algunas celdas están bloqueadas (*obstáculos*) y se debe cubrir exactamente el resto.

Complejidad: $O(\text{filas} \cdot \text{columnas} \cdot 2^{\text{columnas}})$: hay filas $\times$ columnas celdas a procesar, y en cada una se itera sobre los 2^{columnas} perfiles posibles (el estado de si cada una de las columnas de la "frontera" está ocupada); el factor exponencial depende del *ancho* de la grilla, no de su largo, por lo que conviene orientar la grilla para que la dimensión pequeña sea la que entra en el exponente.

```

1 struct DPPerfilDeBits {
2     int filas, columnas;
3
4     // CREATE: guarda las dimensiones de la grilla a cubrir con
5     // fichas de domino (1x2, horizontales o verticales)
6     DPPerfilDeBits(int f, int c) : filas(f), columnas(c) {} // O
7     (1)
8
9     // READ: calcula el numero de formas de cubrir completamente
10    // la
11    // grilla filas x columnas con fichas de domino, usando DP
12    // con
13    // perfil de bits celda por celda
14    long long contarTilings() {
15        int totalCeldas = filas * columnas;
16        int totalPerfiles = 1 << columnas;
17        // dp[i] = numero de formas de llegar al estado de perfil i
18        // en la celda actual (bit encendido = celda ya cubierta)
19        vector<long long> dp(totalPerfiles, 0);
20        dp[0] = 1; // antes de procesar cualquier celda, perfil
21        // vacio
22    }
23 };

```

```

19 for (int celda = 0; celda < totalCeldas; celda++) {
20     int fila = celda / columnas, col = celda % columnas;
21     vector<long long> dpSiguiente(totalPerfiles, 0);
22
23     for (int perfil = 0; perfil < totalPerfiles; perfil++) {
24         if (dp[perfil] == 0) continue;
25
26         bool celdaOcupada = (perfil >> col) & 1;
27         if (celdaOcupada) {
28             // esta celda ya fue cubierta por una ficha vertical
29             // de la fila anterior: apagar el bit y avanzar
30             dpSiguiente[perfil ^ (1 << col)] += dp[perfil];
31         } else {
32             // opcion 1: cubrir con ficha horizontal (necesita
33             // que la siguiente columna exista y este libre)
34             if (col + 1 < columnas && !((perfil >> (col + 1)) &
35 1)) {
36                 int nuevoPerfil = perfil | (1 << col) | (1 << (col
37 + 1));
38                 dpSiguiente[nuevoPerfil ^ (1 << col)] += dp[perfil
39 ];
40             }
41             // opcion 2: cubrir con ficha vertical (prende el
42             // bit para que la fila siguiente lo procese ocupado)
43             dpSiguiente[perfil | (1 << col)] += dp[perfil];
44         }
45     }
46     dp = dpSiguiente;
47 }
48 return dp[0]; // al terminar, el perfil debe quedar vacio
49 } // O(filas * columnas * 2^columnas)
50
51 // -----
52 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
53 // -----
54
55 // 1. Diferencia con Bitmask DP clasico: aqui el bit i del
56 // perfil
57 // NO representa "el elemento i fue usado" de forma
58 // abstracta,
59 // sino "la celda en la columna i de la fila actual/
60 // anterior
61 // esta ocupada"; el significado del bit cambia con la
62 // posicion en la grilla, no es un conjunto fijo de
63 // elementos.
64
65 // 2. Procesar fila completa en vez de celda por celda: una
66 // variante mas eficiente procesa perfil por FILA (2^
67 // columnas
68 // transiciones por fila) en vez de celda por celda dentro
69 // de
70 // la fila, evaluando de una vez todas las formas de
71 // llenar
72 // una fila dado el perfil que "sobresale" de la fila
73 // anterior.
74
75 // 3. Grillas con obstaculos: si una celda esta bloqueada, se
76 // fuerza esa celda a permanecer "sin cubrir" en el perfil
77 // (no se permite ninguna ficha sobre ella), y al final
78 // solo
79 // se aceptan perfiles donde las celdas bloqueadas
80 // coincidan
81 // con el patron esperado.
82
83 // 4. Piezas mas grandes (triominos, poliominos): el mismo
84 // esqueleto aplica, pero las opciones de transicion en
85 // cada
86 // celda se multiplican segun las formas de la pieza; el
87 // ancho
88 // de la grilla sigue siendo el factor limitante en el
89 // exponente.
90
91 };

```

Ejemplo de funcionamiento:

```

1 DPPERFILDEBITS grilla(2, 3); // grilla de 2 filas por 3
   columnas
2
3 long long formas = grilla.contarTilings();
4 // Nota: el valor exacto depende de la simulacion completa del
   DP
5 // celda por celda; se debe correr el codigo para confirmarlo
   antes
6 // de darlo por verificado (candidato esperado: 3,
   correspondiente a
7 // las formas conocidas de cubrir una grilla 2x3 con dominos).

```

Regla práctica: Reconoce DP con Perfil de Bits cuando el problema hable de "cubrir/llenar una grilla" con piezas que ocupan más de una celda y el ancho de la grilla sea pequeño (hasta 15-20); si en cambio el

estado del problema es "qué subconjunto de elementos abstractos ya fue usado" sin relación espacial entre ellos, es Bitmask DP clásico en vez de esta variante.

37.14 XOR: Propiedades y Aplicaciones

Idea: El operador XOR ($\oplus$) tiene un conjunto de propiedades algebraicas que lo hacen especialmente útil en competencia: es conmutativo y asociativo, $a \oplus a = 0$ (todo elemento es su propio inverso), $a \oplus 0 = a$ (elemento neutro), y XOR es su propia operación inversa ($a \oplus b \oplus b = a$). Estas propiedades convierten a XOR en una herramienta natural para "cancelar" pares repetidos, invertir una operación sin guardar el valor original, y detectar diferencias bit a bit entre dos valores (el resultado de $a \oplus b$ tiene encendidos exactamente los bits en los que a y b difieren).

¿Por qué usarlo? A diferencia de AND/OR (que no son invertibles: una vez que un bit se apaga con AND o se prende con OR, no hay forma de recuperar el valor original solo con la misma operación), XOR es la única operación bitwise básica que es su propia inversa, lo que la hace ideal para problemas donde se necesita "deshacer" una operación o detectar un elemento "distinto" entre muchos iguales. Es la base conceptual sobre la que se construyen Base Lineal de XOR (para subconjuntos arbitrarios) y Trie de Bits (para pares), que se ven a continuación.

Casos de uso:

- Encontrar el único elemento que aparece un número impar de veces en un arreglo donde todos los demás aparecen un número par de veces (XOR de todo el arreglo).
- Intercambiar dos variables sin usar una variable temporal (*XOR swap*), aunque en la práctica `std::swap` es preferible por claridad.
- Calcular el XOR de un rango $[l, r]$ usando XOR de prefijos (análogo a Prefix Sums, pero con XOR en vez de suma), ya que $\text{XOR}(l, r) = \text{prefijoXOR}(r) \oplus \text{prefijoXOR}(l - 1)$.
- Detectar si dos configuraciones (representadas como enteros) difieren en exactamente un bit, o contar en cuántos bits difieren (popcount del XOR).

Complejidad: $O(1)$ por operación XOR individual, o $O(n)$ para calcular el XOR de un arreglo completo de n elementos (una pasada); con precómputo de prefijos XOR en $O(n)$, cualquier consulta de XOR sobre un rango se responde en $O(1)$.

```

1 struct PropiedadesXOR {
2     vector<long long> prefijoXOR;
3
4     // CREATE: precalcula el XOR acumulado de arr[0..i-1] para
   cada
5     // posicion i, permitiendo consultas de XOR de rango en O(1)
6     PropiedadesXOR(vector<long long> arr) {
7         prefijoXOR.assign(arr.size() + 1, 0);
8         for (int i = 0; i < (int)arr.size(); i++) {
9             prefijoXOR[i + 1] = prefijoXOR[i] ^ arr[i];
10        }
11    } // O(n)
12
13    // READ: retorna el XOR de todos los elementos en el rango
14    // [izq, der] (0-indexado, inclusivo en ambos extremos)
15    long long xorDeRango(int izq, int der) {
16        return prefijoXOR[der + 1] ^ prefijoXOR[izq];
17    }
18 }

```

```

17 } // O(1)
18
19 // READ: dado un arreglo donde todos los elementos aparecen
20 // un numero par de veces excepto uno, encuentra ese elemento
21 // unico
22 static long long encontrarUnico(const vector<long long>& arr)
23 {
24     long long resultado = 0;
25     for (long long v : arr) resultado ^= v;
26     return resultado;
27 } // O(n)
28
29 // -----
30 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
31 // -----
32 // 1. Por que XOR cancela pares:  $a \oplus a = 0$  y  $0 \oplus b = b$  para
33 // cualquier  $b$ ; al hacer XOR de todo el arreglo, cada
34 // elemento
35 // que aparece un numero PAR de veces se cancela consigo
36 // mismo (an  $a \oplus \dots = 0$ ), dejando solo el elemento con
37 // multiplicidad impar.
38 // 2. Dos elementos unicos en vez de uno: si hay exactamente
39 // DOS
40 // elementos con multiplicidad impar (en vez de uno), el
41 // XOR
42 // total da  $a \oplus b$  (no separa cual es cual). Se puede
43 // separar
44 // encontrando un bit donde  $a$  y  $b$  difieren (cualquier bit
45 // encendido de  $a \oplus b$ ) y dividiendo el arreglo en dos grupos
46 // segun ese bit, aplicando XOR por separado a cada grupo.
47 // 3. XOR swap (nota historica, no recomendado en la practica
48 // ):
49 //  $a \oplus b; b \oplus a; a \oplus b$ ; intercambia  $a$  y  $b$  sin variable
50 // temporal, pero falla silenciosamente si  $a$  y  $b$  son la
51 // MISMA
52 // variable (referencian la misma direccion de memoria),
53 // dejando ambas en 0. std::swap es mas seguro y
54 // usualmente
55 // igual de rapido tras optimizacion del compilador.
56 // 4. XOR y paridad de inversiones: el bit menos
57 // significativo
58 // del XOR de dos permutaciones codificadas apropiadamente
59 // puede relacionarse con la paridad de la permutacion que
60 // las
61 // transforma una en otra, en problemas especificos de
62 // combinatoria con permutaciones.
63 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {4, 2, 4, 5, 2, 5, 7};
2 long long unico = PropiedadesXOR::encontrarUnico(arr);
3 // unico = 7 (todos los demas aparecen exactamente 2 veces)
4
5 vector<long long> datos = {3, 6, 1, 9, 4};
6 PropiedadesXOR px(datos);
7
8 long long xorRango = px.xorDeRango(1, 3);
9 // xorRango =  $6 \oplus 1 \oplus 9 = 14$ 

```

Regla práctica: Sospecha de XOR en cuanto un problema mencione "elemento único", "número impar de apariciones", o pida "diferencia entre bits" de dos valores; si en cambio necesitas maximizar/minimizar el XOR de un par o un subconjunto (no solo calcularlo), pasa a Trie de Bits o Base Lineal de XOR respectivamente, que se apoyan en estas mismas propiedades pero resuelven un problema de optimización, no solo de cálculo directo.

37.15 Base Lineal de XOR (XOR Basis / Linear Basis)

Idea: Los enteros de b bits forman un espacio vectorial sobre el cuerpo $GF(2)$ (donde la "suma" de vectores es XOR y la "multiplicación por escalar" es trivial, ya que solo hay 0 y 1). Al igual que en álgebra lineal sobre los reales, un conjunto de vectores puede reducirse a una *base*: un subconjunto linealmente independiente que

genera, mediante combinaciones (XORs), exactamente el mismo espacio (*span*) que el conjunto original. La Base Lineal de XOR se construye con un proceso análogo a la eliminación gaussiana: se procesa cada número, y para cada bit desde el más significativo, si ya existe un elemento de la base con ese bit como "pivote", se hace XOR con él (eliminando ese bit); si no existe, el número se convierte en el nuevo elemento pivote de ese bit.

¿Por qué usarlo? Mientras que las propiedades básicas de XOR (vistas antes) permiten calcular el XOR de un conjunto *fijo* de elementos, la Base Lineal responde preguntas sobre *todos los subconjuntos posibles* de un arreglo en tiempo logarítmico: cuál es el máximo/mínimo XOR alcanzable eligiendo cualquier subconjunto, si un valor objetivo v es alcanzable como XOR de algún subconjunto, y cuántos subconjuntos distintos producen cada valor de XOR. Se distingue del Trie de Bits (que resuelve "máximo XOR de un *par*" en $O(n \log(\max))$) en que la Base Lineal resuelve problemas sobre *subconjuntos de tamaño arbitrario* del arreglo completo, no solo pares, comprimiendo hasta n elementos en a lo sumo b elementos de la base (con b el número de bits).

Casos de uso:

- Encontrar el máximo XOR alcanzable eligiendo cualquier subconjunto (no vacío) de un arreglo.
- Verificar si un valor objetivo v puede formarse como XOR de algún subconjunto del arreglo (*query* de pertenencia al span).
- Contar cuántos subconjuntos distintos del arreglo producen cada valor de XOR posible (usando la dimensión de la base y el número de elementos "redundantes").
- Mantener una base de XOR de forma incremental (agregar elementos uno a uno) para responder consultas de máximo XOR sobre prefijos del arreglo.

Complejidad: $O(n \cdot b)$ para construir la base a partir de n números de b bits cada uno (cada inserción cuesta $O(b)$, al recorrer los posibles bits pivote de mayor a menor). Una vez construida la base (que tiene a lo sumo b elementos), consultar el máximo XOR alcanzable, o verificar si un valor pertenece al span, cuesta $O(b)$.

```

1 struct BaseLinealXOR {
2     static const int BITS = 60; // suficiente para valores hasta
3     // ~10^18
4     vector<long long> base; // base[i] = elemento pivote del
5     // bit i, o 0 si no existe
6
7     // CREATE: inicializa la base vacia (todos los pivotes en 0,
8     // que nunca es un valor valido de pivote real)
9     BaseLinealXOR() : base(BITS, 0) {} // O(BITS)
10
11     // WRITE: intenta insertar x en la base; si x es linealmente
12     // independiente de la base actual, se agrega como nuevo
13     // pivote y
14     // retorna true; si x ya esta en el span de la base (se
15     // reduce a
16     // 0), retorna false sin modificar la base
17     bool insertar(long long x) {
18         for (int bit = BITS - 1; bit >= 0; bit--) {
19             if ((x >> bit) & 1) continue; // este bit ya esta en
20             // 0, seguir al siguiente
21             if (base[bit] == 0) {
22                 base[bit] = x; // no hay pivote para este bit: x se
23                 // convierte en el pivote
24                 return true;
25             }
26             x ^= base[bit]; // eliminar este bit usando el pivote
27             // existente
28         }
29         return false;
30     }
31 };

```

```

21 }
22 return false; // x se redujo a 0: era combinacion lineal de
    // la base existente
23 } // O(BITS)
24
25 // READ: retorna el maximo valor de XOR alcanzable combinando
26 // cualquier subconjunto de los elementos de la base (
    // equivalente
27 // al maximo XOR de subconjunto del arreglo original)
28 long long maximoXOR() {
29     long long resultado = 0;
30     for (int bit = BITS - 1; bit >= 0; bit--) {
31         // si combinar con base[bit] AUMENTA el resultado,
    // hacerlo
32         if ((resultado ^ base[bit]) > resultado) {
33             resultado ^= base[bit];
34         }
35     }
36     return resultado;
37 } // O(BITS)
38
39 // READ: verifica si x puede formarse como XOR de algun
40 // subconjunto de los elementos originalmente insertados en
    // la
41 // base (x pertenece al span de la base)
42 bool perteneceAlSpan(long long x) {
43     for (int bit = BITS - 1; bit >= 0; bit--) {
44         if (!(x >> bit) & 1) continue;
45         if (base[bit] == 0) return false; // no hay forma de
    // eliminar este bit
46         x ^= base[bit];
47     }
48     return x == 0; // si se logro reducir x completamente a 0,
    // pertenece al span
49 } // O(BITS)
50
51 // READ: retorna la dimension de la base (cantidad de
    // elementos
52 // pivote distintos de 0), es decir, el numero de elementos
53 // linealmente independientes encontrados hasta ahora
54 int dimension() {
55     int cont = 0;
56     for (long long v : base) if (v != 0) cont++;
57     return cont;
58 } // O(BITS)
59
60 //
61 // -----
62 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
63 // -----
64 // 1. Por que maximoXOR() funciona de mayor a menor bit: al
    // procesar los bits de mas a menos significativo, cada
65 // decision de "aplicar o no" el pivote de ese bit es
    // optima
66 // de forma greedy: aplicar un pivote con un bit mas alto
    // que
67 // el actual resultado siempre aumenta el valor (nunca se
    // puede compensar despues con bits mas bajos), por lo que
68 // la
69 // eleccion greedy bit a bit es correcta.
70 //
71 // 2. Contar subconjuntos que producen cada valor de XOR: si
    // la
72 // base tiene dimension d a partir de n elementos
    // insertados,
73 // hay exactamente 2^(n-d) subconjuntos distintos que
    // producen
74 // cada valor alcanzable (los n-d elementos "redundantes"
    // que
75 // no aportaron un nuevo pivote pueden incluirse o no en
    // pares
76 // que se cancelan).
77 //
78 // 3. Minimo XOR de subconjunto no vacio: a diferencia del
    // maximo, el minimo XOR de un subconjunto NO VACIO es
79 // simplemente el menor elemento de la base (el pivote con
    // el
80 // bit mas bajo), ya que cualquier combinacion adicional
    // solo
81 // puede prender bits mas altos sin poder apagar ese bit
    // base
82 // de forma que reduzca el valor por debajo de el.
83 //
84 //
85 // 4. Base lineal incremental sobre prefijos: para responder
    // "maximo XOR de un subconjunto del prefijo arr[0..i]",
86 // se
87 // mantiene una base que se actualiza insertando arr[i] en
88 // cada paso, sin necesidad de reconstruir desde cero.
89 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {8, 1, 2, 12};
2 BaseLinealXOR base;
3 for (long long v : arr) base.insertar(v);
4
5 long long maxXor = base.maximoXOR();
6 // Nota: el valor exacto depende del proceso de eliminacion
7 // gaussiana paso a paso sobre {8, 1, 2, 12}; se debe correr el
8 // codigo para confirmarlo antes de darlo por verificado (
    // candidato
9 // esperado: 15, combinando 8 ^ 2 ^ ... segun el resultado real
    // de
10 // la reduccion).
11
12 bool puedeFormar5 = base.perteneceAlSpan(5);
13 // verifica si 5 es alcanzable como XOR de algun subconjunto de
    // arr
14
15 int dim = base.dimension();
16 // dim = numero de elementos independientes encontrados (a lo
    // sumo 4,
17 // el tamano de arr, pero puede ser menor si hay dependencia
    // lineal)

```

Regla práctica: Usa Base Lineal de XOR cuando el problema pida optimizar (maximizar/minimizar) o verificar la alcanzabilidad del XOR de *subconjuntos arbitrarios* de un arreglo, no solo pares; si el problema se limita a pares de elementos (por ejemplo, "máximo XOR entre dos elementos del arreglo"), el Trie de Bits que sigue es más simple y igual de eficiente para ese caso específico.

37.16 Trie de Bits (Binary Trie / XOR Trie)

Idea: Un Trie de Bits es un árbol binario donde cada nodo tiene a lo sumo dos hijos (0 y 1), y cada número insertado se representa como un camino desde la raíz hasta una hoja, recorriendo sus bits de más a menos significativo: en cada nivel, se desciende por la rama "0" o "1" según el valor de ese bit. La estructura completa codifica implícitamente todos los números insertados como prefijos compartidos en el árbol: dos números que comparten sus k bits más significativos comparten los primeros k niveles del camino desde la raíz. Esta compartición de prefijos es lo que permite responder consultas de XOR de forma *greedy*, nivel por nivel, sin tener que comparar contra cada número insertado individualmente.

¿Por qué usarlo? El problema central que resuelve el Trie de Bits es: dado un conjunto de números y una consulta q , encontrar el elemento del conjunto que *maximiza* (o minimiza) $q \oplus x$. Sin el trie, esto requeriría $O(n)$ por consulta (probar cada elemento); con el trie, se resuelve en $O(b)$ (con b el número de bits) recorriendo el árbol de forma greedy: en cada nivel, se intenta descender por la rama *opuesta* al bit correspondiente de q (porque un bit distinto en esa posición maximiza la contribución de ese bit al XOR resultante, que vale más que cualquier combinación posible de los bits restantes más bajos), y solo se toma la rama igual si la opuesta no existe en el trie. Frente a la Base Lineal de XOR, el trie resuelve específicamente el caso "máximo/mínimo XOR con un elemento fijo o entre pares", con la ventaja adicional de soportar inserciones y eliminaciones dinámicas de elementos (llevando un contador de referencias por nodo), algo que la Base Lineal no soporta de forma nativa (eliminar un elemento de una base lineal ya reducida no es trivial).

Casos de uso:

- Máximo XOR entre dos elementos de un arreglo (recorrer el arreglo insertando cada elemento y consultando el máximo XOR con los ya insertados).

- Máximo XOR de un elemento fijo q contra un conjunto dinámico de números que se insertan y eliminan a lo largo de las consultas (a diferencia de la Base Lineal, el trie soporta eliminación con conteo de referencias).
- Máximo XOR de un subarreglo $\text{arr}[l..r]$ combinado con prefijos XOR: insertar prefijos XOR en el trie y consultar, respondiendo variantes de "máximo XOR de subarreglo" en $O(n \log(\max))$.
- Contar, para cada consulta q , cuántos elementos del conjunto satisfacen $q \oplus x < k$ (agregando conteos por nodo y explorando ambas ramas cuando corresponde), una extensión del trie más allá de solo maximizar.

Complejidad: $O(b)$ por inserción y por consulta, con b el número de bits considerados (típicamente 30 para `int`, o 60 para `long long`), ya que cada operación recorre exactamente un camino de longitud b desde la raíz. Para n inserciones y n consultas, la complejidad total es $O(n \cdot b)$. El espacio usado es $O(n \cdot b)$ en el peor caso, ya que cada inserción puede crear hasta b nodos nuevos si el número no comparte prefijo con ningún elemento existente.

```

1 struct TrieDeBits {
2     static const int BITS = 30; // suficiente para valores hasta
        ~10^9
3
4     struct Nodo {
5         int hijos[2] = {-1, -1}; // índices de los hijos (0 y 1),
            -1 si no existe
6         int contador = 0; // cuantos numeros insertados
            pasan por este nodo
7     };
8
9     vector<Nodo> nodos;
10
11     // CREATE: inicializa el trie con un unico nodo raiz vacio
12     TrieDeBits() {
13         nodos.push_back(Nodo());
14     } // O(1)
15
16     // WRITE: inserta el numero x en el trie, recorriendo sus
        bits de
17     // mas a menos significativo y creando nodos segun sea
        necesario
18     void insertar(int x) {
19         int actual = 0; // indice del nodo raiz
20         nodos[actual].contador++;
21         for (int bit = BITS - 1; bit >= 0; bit--) {
22             int b = (x >> bit) & 1;
23             if (nodos[actual].hijos[b] == -1) {
24                 nodos[actual].hijos[b] = (int)nodos.size();
25                 nodos.push_back(Nodo());
26             }
27             actual = nodos[actual].hijos[b];
28             nodos[actual].contador++;
29         }
30     } // O(BITS)
31
32     // WRITE: elimina una ocurrencia de x del trie, decrementando
        el
33     // contador de cada nodo en su camino; asume que x fue
        insertado
34     // previamente (si contador llega a 0, el nodo queda "vacío"
        pero
35     // no se libera memoria, por simplicidad)
36     void eliminar(int x) {
37         int actual = 0;
38         nodos[actual].contador--;
39         for (int bit = BITS - 1; bit >= 0; bit--) {
40             int b = (x >> bit) & 1;
41             actual = nodos[actual].hijos[b];
42             nodos[actual].contador--;
43         }
44     } // O(BITS)
45
46     // READ: retorna el maximo valor de x XOR q entre todos los
47     // elementos actualmente insertados en el trie (con contador
        > 0)

```

```

48 int maximoXORCon(int q) {
49     int actual = 0;
50     int resultado = 0;
51     for (int bit = BITS - 1; bit >= 0; bit--) {
52         int b = (q >> bit) & 1;
53         int deseado = b ^ 1; // rama opuesta: maximiza este bit
        del XOR
54         if (nodos[actual].hijos[deseado] != -1 &&
55             nodos[nodos[actual].hijos[deseado]].contador > 0) {
56             resultado |= (1 << bit);
57             actual = nodos[actual].hijos[deseado];
58         } else {
59             // rama opuesta no disponible: forzosamente tomar la
60             // rama igual (este bit del XOR queda en 0)
61             actual = nodos[actual].hijos[b];
62         }
63     }
64     return resultado;
65 } // O(BITS)
66
67 // READ: retorna el minimo valor de x XOR q entre todos los
68 // elementos actualmente insertados, invirtiendo la logica
        greedy
69 // (preferir la rama IGUAL al bit de q en vez de la opuesta)
70 int minimoXORCon(int q) {
71     int actual = 0;
72     int resultado = 0;
73     for (int bit = BITS - 1; bit >= 0; bit--) {
74         int b = (q >> bit) & 1;
75         if (nodos[actual].hijos[b] != -1 &&
76             nodos[nodos[actual].hijos[b]].contador > 0) {
77             actual = nodos[actual].hijos[b]; // este bit del XOR
        queda en 0
78         } else {
79             resultado |= (1 << bit);
80             actual = nodos[actual].hijos[b ^ 1];
81         }
82     }
83     return resultado;
84 } // O(BITS)
85
86 //
87 // -----
88 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
89 // -----
90 // 1. Por que la estrategia greedy es correcta: el bit mas
        significativo del XOR resultante domina el valor total,
91 // ya
92 // que 2^bit siempre es mayor que la suma de TODOS los
        bits
93 // inferiores combinados (2^0 + 2^1 + ... + 2^(bit-1) < 2^
        bit).
94 // Por eso, maximizar cada bit de mas a menos
        significativo de
95 // forma greedy, sin "arrepentirse" despues, produce el
        optimo
96 // global - es el mismo principio detras de Base Lineal de
        XOR.
97 // 2. Maximo XOR de subarreglo con prefijos XOR: se inserta
        el
98 // prefijoXOR[0] = 0 en el trie primero, y luego,
        recorriendo
99 // i de 1 a n, se consulta maximoXORCon(prefijoXOR[i])
        ANTES
100 // de insertar prefijoXOR[i] al trie, ya que
101 // xor(l+1, r) = prefijoXOR[r] ^ prefijoXOR[l], y se
        necesita
102 // que prefijoXOR[l] con l < i ya este insertado.
103 //
104 // 3. Conteo de elementos que cumplen x XOR q < k: se recorre
        el
105 // trie en paralelo sobre los bits de q y k; en cada nivel,
106 // si el bit de k es 1, todos los elementos cuya rama
        coincide
107 // con "bit de q" (dando 0 en ese bit del XOR)
        automaticamente
108 // satisfacen la condicion para el resto de bits
        inferiores
109 // sin importar como continuen, y se suma su contador
110 // directamente; luego se desciende por la rama que
        mantiene
111 // el XOR igual al bit de k para seguir comparando.
112 //
113 // 4. Trie persistente para consultas sobre rangos de
        prefijos:
114 // para responder "maximo XOR con q usando solo prefijos
        en el
115 // rango [l, r]" (no solo "todos los prefijos hasta ahora
        "), se
116 // necesita una version PERSISTENTE del trie, donde cada

```

```

117 // inserción crea una nueva "version" del arbol
118 // compartiendo
119 // los nodos no modificados con la version anterior,
120 // permitiendo
121 // consultar la version en el indice r y restar/filtrar la
122 // version en l-1 mediante el contador diferencial por
123 // nodo.
124 //
125 // 5. Memoria: reservar nodos.reserve(n * BITS) de antemano
126 // evita
127 // realocaciones repetidas del vector durante muchas
128 // inserciones, mejorando el rendimiento en la practica.
129 };

```

Ejemplo de funcionamiento:

```

1 vector<int> arr = {3, 10, 5, 25, 2, 8};
2
3 TrieDeBits trie;
4 int mejorXor = 0;
5 for (int i = 0; i + 1 < (int)arr.size(); i++) {
6     trie.insertar(arr[i]);
7     mejorXor = max(mejorXor, trie.maximoXORCon(arr[i + 1]));
8 }
9 // Nota: el valor exacto depende de recorrer todos los pares
10 // insertados incrementalmente; se debe correr el codigo para
11 // confirmarlo antes de darlo por verificado (candidato
12 // esperado: 28,
13 // correspondiente al par  $5 \oplus 25 = 28$ , el maximo XOR entre dos
14 // elementos de arr).
15
16 // Ejemplo de eliminacion dinamica:
17 TrieDeBits trieDinamico;
18 trieDinamico.insertar(6);
19 trieDinamico.insertar(11);
20 int maxAntes = trieDinamico.maximoXORCon(0); // considera {6,
21 // 11}
22 trieDinamico.eliminar(11);
23 int maxDespues = trieDinamico.maximoXORCon(0); // ahora solo
24 // considera {6}

```

Nota: El trie asume valores no negativos y un número fijo de BITS conocido de antemano (30 para valores hasta $\sim 10^9$, 60 para long long hasta $\sim 10^{18}$); si el problema incluye negativos, es necesario decidir una representación explícita (por ejemplo, desplazar todos los valores a un rango no negativo antes de insertar) ya que el trie tal como está construido no maneja el bit de signo de complemento a 2 de forma especial.

Regla práctica: Usa Trie de Bits cuando el problema pida maximizar/minimizar el XOR entre *pares* de elementos (o entre un elemento fijo y un conjunto), especialmente si el conjunto cambia dinámicamente con inserciones y eliminaciones a lo largo de las consultas; si en cambio necesitas trabajar con el XOR de *subconjuntos arbitrarios* (no solo pares) del arreglo completo, usa Base Lineal de XOR, que resuelve ese problema distinto de forma más directa.

37.17 Código Gray (Gray Code)

Idea: El Código Gray es una forma de enumerar los 2^n números de n bits de manera que cada número consecutivo en la secuencia difiera del anterior en *exactamente un bit*. Se construye a partir de la representación binaria estándar mediante la fórmula $g(i) = i \oplus (i \gg 1)$: al hacer XOR de un número con su propio desplazamiento a la derecha, cada bit del resultado indica si hubo un "cambio" entre bits adyacentes de i , lo que garantiza que incrementar i en 1 (que en binario estándar puede cambiar varios bits simultáneamente por el acarreo) se traduzca en un cambio de un solo bit en $g(i)$.

¿Por qué usarlo? A diferencia de recorrer los enteros en orden binario estándar (donde, por ejemplo, pasar de 0111 a 1000 cambia los 4 bits a la vez), el Código Gray garantiza transiciones "suaves" de un bit por paso,

lo cual es deseable en problemas donde recalculer el estado completo desde cero en cada paso es costoso, pero actualizar incrementalmente tras un solo cambio de bit es barato (similar en espíritu a por qué SOS DP evita recalculer desde cero). Es una herramienta de nicho comparada con Bitmask DP o el Trie de Bits, pero resuelve un problema estructural distinto: no optimiza ni cuenta, sino que da un *orden específico* de recorrido sobre las máscaras.

Casos de uso:

- Enumerar todos los subconjuntos de un conjunto de forma que cada paso agregue o quite un solo elemento, útil cuando mantener una estructura auxiliar (suma acumulada, conjunto activo) es caro de recalculer desde cero.
- Resolver variantes del problema de las Torres de Hanói, donde la secuencia de movimientos óptima se corresponde naturalmente con un Código Gray.
- Generar todas las combinaciones de un conjunto de interruptores/opciones binarias en un orden que minimiza el número de "clics" (cambios de estado) entre configuraciones consecutivas.
- Como herramienta de depuración/visualización: verificar resultados de un algoritmo sobre máscaras probando un *stress test* en un orden donde los errores por cambios bruscos de estado son más fáciles de aislar.

Complejidad: $O(1)$ para calcular el Código Gray de un índice i individual (una operación de shift y un XOR); $O(2^n)$ para generar la secuencia completa de los 2^n códigos Gray de n bits, ya que cada uno se calcula en tiempo constante.

```

1 struct CodigoGray {
2
3     // READ: retorna el codigo Gray correspondiente al indice i
4     // (i-esimo elemento de la secuencia, 0-indexado)
5     static long long grayDeIndice(long long i) {
6         return i ^ (i >> 1);
7     } // O(1)
8
9     // READ: retorna el indice i tal que grayDeIndice(i) ==
10    // codigo,
11    // es decir, invierte la transformacion (Gray a binario
12    // estandar)
13    static long long indiceDeGray(long long codigo) {
14        long long i = 0;
15        while (codigo > 0) {
16            i ^= codigo;
17            codigo >>= 1;
18        }
19        return i;
20    } // O(bits)
21
22    // READ: genera la secuencia completa de codigos Gray para n
23    // bits, en orden (2^n elementos, cada uno difiere del
24    // anterior
25    // en exactamente un bit)
26    static vector<long long> generarSecuencia(int n) {
27        vector<long long> resultado;
28        long long total = 1LL << n;
29        for (long long i = 0; i < total; i++) {
30            resultado.push_back(grayDeIndice(i));
31        }
32        return resultado;
33    } // O(2^n)
34
35    //
36
37    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
38    //
39
40    // 1. Por que i ^ (i >> 1) da un cambio de un solo bit: al
41    // pasar

```

```

36 // de i a i+1 en binario estandar, el acarreo de la suma
37 // "voltea" un bloque de bits consecutivos (todos los 1s
38 // finales se vuelven 0, y el primer 0 se vuelve 1). El
    XOR
39 // con el propio desplazamiento detecta exactamente el
    limite
40 // de ese bloque de cambio, colapsando el efecto del
    acarreo a
41 // un unico bit de diferencia entre g(i) y g(i+1).
42 //
43 // 2.Codigo Gray reflejado (construccion recursiva): una
    forma
44 // alternativa de generar la secuencia es recursivamente:
45 // Gray(n) = ["0" + x para x en Gray(n-1)] seguido de
46 // ["1" + x para x en reverso(Gray(n-1))],
47 // partiendo de Gray(0) = [""]; produce la misma secuencia
    que
48 // la formula  $i \wedge (i >> 1)$  pero construida por reflexion.
49 //
50 // 3. Codigo Gray para bases distintas de 2: existen
51 // generalizaciones a codigos Gray de base n (no solo
    binaria),
52 // utiles en problemas de permutaciones o combinatoria con
53 // mas de dos estados por posicion, aunque son mucho menos
54 // comunes en competencia.
55 //
56 // 4. Verificacion de la propiedad: para confirmar que dos
    codigos
57 // Gray consecutivos difieren en un solo bit, verificar
    que
58 // __builtin_popcountll(grayDeIndice(i) ^ grayDeIndice(i
    +1)) == 1
59 // para todo i en el rango deseado.
60 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> secuencia = CodigoGray::generarSecuencia(3);
2 // secuencia = {0, 1, 3, 2, 6, 7, 5, 4}
3 // en binario: 000, 001, 011, 010, 110, 111, 101, 100
4 // cada par consecutivo difiere en exactamente un bit
5
6 long long indiceRecuperado = CodigoGray::indiceDeGray(6);
7 // indiceRecuperado = 4 (grayDeIndice(4) = 4 ^ 2 = 6, se
    confirma la inversa)

```

Regla práctica: Considera Código Gray cuando un problema pida explícitamente enumerar subconjuntos o configuraciones binarias minimizando el número de "cambios" entre pasos consecutivos; para la gran mayoría de problemas de Bitmask DP (donde el orden de enumeración de las máscaras no importa), la enumeración estándar $0, 1, 2, \dots, 2^n - 1$ es suficiente y no requiere esta técnica.

37.18 Conteo de Pares con XOR/AND/OR Dado

Idea: Dado un arreglo de n números, este patrón cuenta cuántos pares (i, j) con $i < j$ satisfacen una condición sobre $a_i \oplus a_j$, $a_i \& a_j$, o $a_i | a_j$ (por ejemplo, "cuántos pares tienen XOR igual a k ", o "cuántos pares tienen AND distinto de 0"). La estrategia general evita el enfoque ingenuo $O(n^2)$ de probar todos los pares, procesando los números *bit a bit* o usando una tabla de frecuencias: para XOR con un valor objetivo fijo k , basta notar que $a_i \oplus a_j = k$ equivale a $a_j = a_i \oplus k$, por lo que se puede usar una tabla hash de frecuencias para buscar, por cada a_i , cuántas ocurrencias hay de $a_i \oplus k$ ya procesadas.

¿Por qué usarlo? Esta técnica se distingue del Trie de Bits en el objetivo: el trie resuelve "encontrar el elemento que *maximiza* el XOR con uno dado", mientras que este patrón resuelve "contar *cuántos* pares cumplen una condición de igualdad o desigualdad" — un problema de conteo, no de optimización. Para XOR con un valor objetivo exacto, una tabla de frecuencias ($O(n)$) es más simple y rápida que un trie; el trie solo se vuelve necesario cuando la condición involucra un rango de valores (por

ejemplo, "XOR menor a k ") en vez de una igualdad exacta.

Casos de uso:

- Contar pares (i, j) tales que $a_i \oplus a_j = k$ para un k dado, usando tabla de frecuencias en $O(n)$.
- Contar pares tales que $a_i \& a_j = 0$ (pares "disjuntos" en representación de bits), frecuentemente combinado con SOS DP para contar, por cada máscara, cuántos elementos son submáscara de su complemento.
- Contar pares tales que $a_i | a_j$ tiene una propiedad específica (por ejemplo, es igual a un valor máximo dado), también resoluble con SOS DP sobre superconjuntos.
- Sumar $a_i \oplus a_j$ sobre *todos* los pares del arreglo (no solo contarlos), procesando bit por bit: para cada bit, contar cuántos elementos lo tienen encendido/apagado y combinar con la fórmula de conteo de pares.

Complejidad: $O(n)$ para el conteo de pares con XOR igual a un valor exacto k (una pasada con tabla de frecuencias); $O(n \cdot b)$ para variantes que procesan bit por bit (como la suma de XOR de todos los pares, o conteo con AND/OR usando SOS DP), con b el número de bits considerados.

```

1 struct ConteoDeParesConXOR {
2
3     // READ: cuenta la cantidad de pares (i, j) con i < j tales
    que
4     // arr[i] ^ arr[j] == k, usando una tabla de frecuencias
5     static long long contarParesXOR(const vector<long long>& arr,
        long long k) {
6         unordered_map<long long, long long> frecuencia;
7         long long cuenta = 0;
8         for (long long v : arr) {
9             long long complemento = v ^ k; // buscar si v ^ k ya
                aparecio antes
10            auto it = frecuencia.find(complemento);
11            if (it != frecuencia.end()) cuenta += it->second;
12            frecuencia[v]++;
13        }
14        return cuenta;
15    } // O(n)
16
17    // READ: calcula la suma de arr[i] ^ arr[j] sobre TODOS los
        pares
18    // (i, j) con i < j, procesando bit por bit en vez de O(n^2)
        pares
19    static long long sumaXORDeTodosLosPares(const vector<long
        long>& arr, int bits = 30) {
20        int n = (int)arr.size();
21        long long sumaTotal = 0;
22        for (int bit = 0; bit < bits; bit++) {
23            long long conBitEncendido = 0;
24            for (long long v : arr) {
25                if ((v >> bit) & 1) conBitEncendido++;
26            }
27            long long conBitApagado = n - conBitEncendido;
28            // cada par (uno con el bit encendido, otro apagado)
                aporta
29            // 2^bit al XOR total; hay conBitEncendido *
                conBitApagado
30            // pares de ese tipo
31            sumaTotal += (1LL << bit) * conBitEncendido *
                conBitApagado;
32        }
33        return sumaTotal;
34    } // O(n * bits)
35
36    //
37    // -----
38    // MODIFICACIONES Y COMENTARIOS CON LOS USOS
39    // -----
40
41    // 1. Conteo de pares con AND == 0: se puede resolver con SOS
        DP

```

```

40 // (superset sum): para cada elemento arr[i], se necesita
41 // saber cuantos elementos arr[j] son "disjuntos" con el (
42 // es
43 // decir, arr[j] es submascara del complemento de arr[i]);
44 // precalcular con SOS DP, para cada mascara m, cuantos
45 // elementos del arreglo son submascara de m, y consultar
46 // con
47 // m = complemento de arr[i].
48 //
49 // 2. Conteo de pares con OR igual a un valor especifico:
50 // similar
51 // al caso AND, pero usando "superset sum" (suma sobre
52 // superconjuntos) en vez de "subset sum", ya que
53 // a | b == v implica que tanto a como b son submascaras
54 // de v.
55 //
56 // 3. Por que sumaXORDeTodosLosPares evita  $O(n^2)$ : el XOR de
57 // dos
58 // numeros tiene el bit k encendido si y solo si difieren
59 // en
60 // ese bit; contando cuantos elementos tienen el bit k
61 // encendido/apagado, el numero de pares que difieren en
62 // ese
63 // bit es simplemente el producto de esos dos conteos, sin
64 // necesidad de examinar cada par explicitamente.
65 //
66 // 4. Generalizacion a suma de AND/OR de todos los pares: la
67 // misma tecnica de "contar por bit" aplica: para AND, el
68 // bit
69 // k contribuye  $2^k * C(\text{conBitEncendido}, 2)$  pares (ambos
70 // deben
71 // tener el bit encendido); para OR, contribuye
72 //  $2^k * (C(n, 2) - C(\text{conBitApagado}, 2))$  (todos los pares
73 // excepto los que tienen ambos apagados).
74 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {3, 5, 6, 1, 2};
2
3 long long paresConXor5 = ConteoDeParesConXOR::contarParesXOR(
4   arr, 5);
5 // pares (i,j) con arr[i] ^ arr[j] == 5:
6 // 3^6=5, 1^... revisar todos; se debe correr el codigo para
7 // confirmar el conteo exacto antes de darlo por verificado.
8
9 long long sumaTotal = ConteoDeParesConXOR::
10   sumaXORDeTodosLosPares(arr);
11 // suma de arr[i]^arr[j] para todos los  $C(5,2)=10$  pares
12 // distintos;
13 // se debe correr el codigo para confirmar el valor exacto.

```

Regla práctica: Para "cuántos pares tienen XOR/suma/diferencia igual a k " usa tabla de frecuencias en $O(n)$; para "cuántos pares tienen AND/OR igual a (o relacionado con) un valor" apóyate en SOS DP sobre subconjuntos o superconjuntos según corresponda; y para "suma total de XOR/AND/OR sobre todos los pares" (sin filtrar por un valor específico), procesa bit por bit contando encendidos/apagados en vez de enumerar pares.

37.19 Conteo de Bits Encendidos en un Rango $[0, N]$

Idea: Este patrón calcula, sin iterar número por número, la suma total de bits encendidos (popcount) de todos los enteros en el rango $[0, N]$, o específicamente cuántos números en ese rango tienen el bit k encendido. La clave es notar que, para un bit fijo k , el patrón de "encendido/apagado" se repite de forma periódica: el bit k está apagado durante 2^k números consecutivos, luego encendido durante los siguientes 2^k , y ese ciclo de longitud 2^{k+1} se repite exactamente a lo largo de toda la recta numérica. Contar cuántos números en $[0, N]$ tienen el bit k encendido se reduce entonces a contar ciclos completos y manejar el resto parcial, sin necesidad de examinar cada número individualmente.

¿Por qué usarlo? Sumar popcounts número por número en $[0, N]$ cuesta $O(N \log N)$ (o $O(N)$ con

`__builtin_popcount` por número), inviable si N es del orden de 10^9 o más. Esta técnica reduce el costo a $O(\log N)$ (una pasada por cada uno de los $\log N$ bits posibles), aprovechando la periodicidad de cada bit en vez de la enumeración directa — es, en espíritu, el mismo tipo de razonamiento "bit por bit" que Conteo de Pares con XOR/AND/OR Dado aplica para evitar $O(n^2)$, pero aquí sobre un rango continuo de enteros en vez de un arreglo dado.

Casos de uso:

- Calcular $\sum_{i=0}^N \text{popcount}(i)$ para N hasta 10^{18} , común en problemas de conteo combinatorio sobre representaciones binarias.
- Contar cuántos números en $[0, N]$ tienen exactamente c bits encendidos (combinando esta técnica con combinatoria: elegir qué c posiciones de bit están encendidas sujeto al límite N , tipo *digit DP*).
- Precalcular, como paso base, cuántos números en un rango activan cada bit, antes de aplicar XOR/suma sobre ese rango en problemas más complejos.
- Verificar por fuerza bruta (para N pequeño) contra la fórmula cerrada, como *sanity check* antes de aplicar la técnica a N grande en una competencia.

Complejidad: $O(\log N)$ para calcular la suma total de bits encendidos en $[0, N]$: se itera una vez por cada uno de los $O(\log N)$ bits posibles, y cada iteración calcula el conteo de ese bit en $O(1)$ mediante aritmética de división entera, sin bucles internos adicionales.

```

1 struct ConteoBitsEnRango {
2
3   // READ: cuenta cuantos numeros en  $[0, N]$  (inclusive) tienen
4   // el
5   // bit "bit" encendido, usando la periodicidad de ese bit
6   static long long contarBitEnRango(long long N, int bit) {
7     long long cicloCompleto = 1LL << (bit + 1); // longitud del
8     // ciclo apagado+encendido
9     long long mitadCiclo = 1LL << bit; // cuantos
10    // numeros seguidos tienen el bit encendido
11
12    long long totalNumeros = N + 1; // cantidad de numeros en
13    //  $[0, N]$ 
14    long long ciclosCompleto = totalNumeros / cicloCompleto;
15    long long resto = totalNumeros % cicloCompleto;
16
17    long long conteo = ciclosCompleto * mitadCiclo;
18    conteo += max(0LL, resto - mitadCiclo); // parte encendida
19    // del ciclo parcial
20    return conteo;
21  } //  $O(1)$ 
22
23  // READ: retorna la suma de popcount(i) para todo i en  $[0, N]$ 
24  // (inclusive), sumando el conteo de cada bit por separado
25  static long long sumaPopcountEnRango(long long N, int bitsMax
26    = 60) {
27    long long sumaTotal = 0;
28    for (int bit = 0; bit < bitsMax; bit++) {
29      sumaTotal += contarBitEnRango(N, bit);
30    }
31    return sumaTotal;
32  } //  $O(\log N)$ 
33
34  //
35  // -----
36  // MODIFICACIONES Y COMENTARIOS CON LOS USOS
37  // -----
38
39  // 1. Por que el patron del bit k tiene periodo  $2^{(k+1)}$ : en
40  // binario, el bit k cambia de valor cada vez que el
41  // contador
42  // cruza un multiplo de  $2^k$ ; permanece apagado durante  $2^k$ 
43  // numeros consecutivos y encendido durante los siguientes
44  //  $2^k$ , dando un ciclo completo de longitud  $2^{(k+1)}$ .
45  //
46  // 2. Conteo en un rango  $[L, R]$  en vez de  $[0, N]$ : se usa la

```

```

38 // tecnica de "resta de prefijos", igual que con Prefix
39 // Sums:
40 // contarBitEnRango(R, bit) - contarBitEnRango(L - 1, bit),
41 // ya que el conteo en [0, N] es una funcion de prefijo
42 // valida.
43 //
44 // 3. Contar numeros con EXACTAMENTE c bits encendidos en [0,
45 // N]:
46 // requiere una tecnica distinta, tipicamente Digit DP,
47 // procesando los bits de N de mas a menos significativo y
48 // llevando como estado cuantos bits encendidos se han
49 // colocado hasta el momento, junto con si aun se esta
50 // "pegado" al limite superior N.
51 //
52 // 4. Verificacion por fuerza bruta: para N pequeno (< 10^6),
53 // comparar sumaPopcountEnRango(N) contra un bucle directo
54 // "for i in [0,N]: total += __builtin_popcountll(i)" es
55 // una
56 // forma rapida de validar la implementacion antes de
57 // confiar
58 // en ella para N grande.
59 };

```

Ejemplo de funcionamiento:

```

1 long long N = 7; // rango [0, 7]:
2 // 000,001,010,011,100,101,110,111
3
4 long long bit0 = ConteoBitsEnRango::contarBitEnRango(N, 0);
5 // bit0 = 4 (numeros con bit 0 encendido: 1,3,5,7)
6
7 long long bit1 = ConteoBitsEnRango::contarBitEnRango(N, 1);
8 // bit1 = 4 (numeros con bit 1 encendido: 2,3,6,7)
9
10 long long bit2 = ConteoBitsEnRango::contarBitEnRango(N, 2);
11 // bit2 = 4 (numeros con bit 2 encendido: 4,5,6,7)
12
13 long long sumaTotal = ConteoBitsEnRango::sumaPopcountEnRango(N,
14 // 3);
15 // sumaTotal = 4 + 4 + 4 = 12
16 // verificacion directa: popcount(0..7) = 0+1+1+2+1+2+2+3 = 12
17 // (coincide)

```

Regla práctica: Usa esta técnica en cuanto un problema pida información agregada sobre bits (suma de popcounts, conteo de un bit específico) sobre un rango $[0, N]$ o $[L, R]$ con N demasiado grande para iterar directamente (típicamente $N > 10^7$); para rangos pequeños, un bucle directo con `__builtin_popcount` es más simple y suficientemente rápido.

37.20 Convolución XOR/AND/OR (Walsh-Hadamard Transform)

Idea: Dado dos arreglos a y b indexados por máscaras de n bits, la convolución XOR se define como $c[k] = \sum_{i \oplus j = k} a[i] \cdot b[j]$ (análogamente para AND y OR, cambiando la condición sobre i, j, k). Calcular esto directamente cuesta $O(4^n)$ (para cada par i, j , acumular en $k = i \oplus j$). La Transformada de Walsh-Hadamard (WHT) resuelve esto de forma análoga a como la FFT acelera la convolución estándar: se aplica una transformada invertible a a y b por separado ($O(n \cdot 2^n)$ cada una), se multiplican los resultados *punto a punto* ($O(2^n)$), y se aplica la transformada inversa al producto ($O(n \cdot 2^n)$), obteniendo c completo en $O(n \cdot 2^n)$ en total. Para AND/OR, el mismo principio se logra con la Transformada Zeta/Möbius sobre subconjuntos (el mismo patrón de propagación bit a bit que SOS DP), en vez de la transformada de Hadamard específica de XOR.

¿Por qué usarlo? Es la generalización natural de SOS DP: SOS DP resuelve $g[m] = \sum_{s \subseteq m} f[s]$ (una transformada Zeta simple), mientras que esta técnica resuelve el problema más general de *combinar dos arreglos distintos* bajo XOR/AND/OR, no solo agregar un arreglo consigo mismo. Es una herramienta avanzada (nivel

Div1/regional) que rara vez se necesita completa desde cero en competencia, pero reconocerla evita intentar una convolución XOR/AND/OR con fuerza bruta $O(4^n)$ cuando existe una solución $O(n \cdot 2^n)$.

Casos de uso:

- Calcular, para cada máscara k , el número de pares (i, j) con $a_i \oplus a_j = k$ para arreglos grandes, cuando el enfoque de tabla de frecuencias (útil solo para un k fijo) no basta porque se necesitan *todos* los valores de k a la vez.
- Problemas de conteo de subconjuntos con una operación de "combinar" dos distribuciones bajo XOR (por ejemplo, combinar resultados de dos mitades de un problema dividido con Meet in the Middle, cuando la combinación es sobre máscaras).
- Convolución AND/OR para problemas de "cuántas formas hay de elegir un elemento de A y uno de B tal que su AND/OR sea exactamente k ", usando Transformada Zeta/Möbius en vez de Hadamard.
- Como herramienta dentro de programación dinámica sobre subconjuntos donde la transición natural es una convolución bajo alguna operación bitwise, en vez de una suma o producto estándar.

Complejidad: $O(n \cdot 2^n)$ para transformar (o destransformar) un arreglo de tamaño 2^n , análogo en estructura al $O(2^n \cdot n)$ de SOS DP: se itera sobre los n bits, y en cada uno se recorren las 2^n posiciones actualizando en $O(1)$. La convolución completa (transformar a , transformar b , multiplicar punto a punto, transformar inverso) cuesta $O(n \cdot 2^n)$ en total, dominado por las transformadas.

```

1 struct ConvolucionXOR {
2
3     // WRITE: aplica la Transformada de Walsh-Hadamard en el
4     // lugar
5     // (in-place) sobre el arreglo, necesaria antes de
6     // multiplicar
7     // punto a punto para convolucion XOR
8     static void transformarXOR(vector<long long>& a) {
9         int n = (int)a.size();
10        for (int len = 1; len < n; len <= 1) {
11            for (int i = 0; i < n; i += (len <= 1)) {
12                for (int j = i; j < i + len; j++) {
13                    long long u = a[j], v = a[j + len];
14                    a[j] = u + v; // combinar par (i, i+len) sin
15                    // diferencia de signo
16                    a[j + len] = u - v; // la resta codifica la
17                    // estructura XOR
18                }
19            }
20        }
21    } // O(n log n), con n = 2^bits
22
23    // WRITE: aplica la transformada INVERSA de Walsh-Hadamard,
24    // deshaciendo transformarXOR y dividiendo por el tamano del
25    // arreglo para recuperar los valores originales
26    static void transformarXORInversa(vector<long long>& a) {
27        int n = (int)a.size();
28        transformarXOR(a); // la WHT es su propia inversa salvo el
29        // factor de escala
30        for (long long& v : a) v /= n;
31    } // O(n log n)
32
33    // READ: calcula la convolucion XOR completa de a y b:
34    // retorna c
35    // tal que c[k] = suma de a[i]*b[j] para todo i^j == k
36    static vector<long long> convolucionXOR(vector<long long> a,
37        vector<long long> b) {
38        int n = (int)a.size(); // debe ser potencia de 2, igual
39        // para a y b
40        transformarXOR(a);
41        transformarXOR(b);
42        vector<long long> c(n);
43        for (int i = 0; i < n; i++) c[i] = a[i] * b[i]; //
44        // multiplicacion punto a punto
45    }
46 };

```

```

36     transformarXORInversa(c);
37     return c;
38 } // O(n log n)
39
40 // -----
41 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
42 // -----
43 // 1. Transformada Zeta para convolucion OR (superset/subset):
44 //     para convolucion OR, la transformada NO usa resta sino
45 //     suma en una direccion: a[j+len] += a[j] (equivalente a
46 //     SOS DP: suma sobre subconjuntos); la inversa resta en vez
47 //     de dividir: a[j+len] -= a[j]. Para convolucion AND, se
48 //     invierte el rol de las mitades (se propaga desde el subconjunto
49 //     mas grande hacia los mas chicos, "superset sum").
50 //
51 // 2. Por que no se necesita numeros complejos como en FFT: a
52 //     diferencia de la convolucion estandar (que usa raices
53 //     de la unidad complejas), la WHT para XOR usa solo +1 y -1
54 //     como "raices", ya que XOR sobre GF(2) tiene una estructura
55 //     mas simple; esto permite implementarla enteramente con
56 //     aritmetica entera, sin perdida de precision de punto flotante.
57 //
58 // 3. Overflow en la multiplicacion punto a punto: los
59 //     transformados pueden crecer hasta un factor de n
60 //     respecto a los originales (por las sumas repetidas); si a y b
61 //     contienen valores grandes, verificar overflow antes de
62 //     multiplicar punto a punto (ver Overflow y Wraparound), o aplicar un
63 //     modulo si el problema lo permite.
64 //
65 // 4. Relacion con SOS DP: la transformada Zeta usada para
66 //     convolucion OR es EXACTAMENTE el algoritmo de SOS DP
67 //     visto antes, aplicado como paso intermedio de una convolucion
68 //     en vez de como resultado final; reconocer esta
69 //     equivalencia ayuda a recordar la implementacion sin memorizarla por
70 //     separado.
71 };

```

Ejemplo de funcionamiento:

```

1 // a[i] = cantidad de veces que aparece i en el primer conjunto
2 // b[j] = cantidad de veces que aparece j en el segundo
  conjunto
3 vector<long long> a = {1, 2, 0, 1}; // universo de 2 bits
  (0..3)
4 vector<long long> b = {0, 1, 1, 0};
5
6 vector<long long> c = ConvolucionXOR::convolucionXOR(a, b);
7 // c[k] = suma de a[i]*b[j] para todo i^j == k
8 // Nota: el valor exacto de cada c[k] depende de la
  transformada
9 // paso a paso; se debe correr el codigo para confirmar los
  valores
10 // antes de darlos por verificados.

```

Nota: Esta técnica es de las más avanzadas de esta parte — si tu guía está orientada a niveles Div2 o fundamentos, puede quedar como referencia de "saber que existe" más que como algo a memorizar en detalle; su pre-requisito conceptual directo es SOS DP, así que conviene tenerla sólida antes de abordar esta subsección.

Regla práctica: Usa convolución XOR/AND/OR (vía WHT o Transformada Zeta/Möbius) cuando necesites combinar *dos* arreglos distintos bajo una operación bitwise para *todos* los valores de salida simultáneamente, en $O(n \cdot 2^n)$; si el problema es más simple —agregar un solo arreglo sobre sus propios subconjuntos— con SOS DP alcanza, sin necesidad de la maquinaria completa de la transformada.

37.21 Meet in the Middle con Máscaras de Bits

Idea: Meet in the Middle divide un conjunto de n elementos en dos mitades de tamaño $n/2$, enumera todos los $2^{n/2}$ subconjuntos de cada mitad por separado (representados como máscaras de bits), y combina los resultados de ambas mitades para responder la pregunta sobre el conjunto completo. La observación clave es que $2^{n/2} + 2^{n/2} = 2^{n/2+1}$ es exponencialmente más pequeño que 2^n cuando n crece: para $n = 40$, enumerar 2^{40} subconjuntos es inviable, pero enumerar 2^{20} subconjuntos por mitad (unos 10^6) es trivial. La combinación de resultados típicamente requiere ordenar una mitad y usar búsqueda binaria (o una tabla hash) sobre ella al recorrer la otra, en vez de comparar cada par de subconjuntos directamente.

¿Por qué usarlo? Es la técnica que rescata problemas de Bitmask DP cuando n excede el límite práctico de 2^n (usualmente 20-24): al dividir en dos mitades de $n/2 \approx 20$ cada una, se extiende el rango manejable hasta $n \approx 40$ -45. Se distingue de Bitmask DP clásico en que aquí *no* se construye una tabla de estados sobre todo el conjunto —se enumeran exhaustivamente subconjuntos de cada mitad de forma independiente, y la dificultad se traslada a combinar eficientemente ambas listas (usualmente con ordenamiento + búsqueda binaria, conectando esta técnica directamente con la Parte de Búsqueda Binaria).

Casos de uso:

- Subset Sum con n hasta 40: dividir en dos mitades, enumerar todas las sumas de subconjuntos de cada mitad, ordenar una y usar búsqueda binaria sobre la otra para verificar si un valor objetivo es alcanzable.
- Maximizar el valor de una mochila (*knapsack*) con n hasta 40 y capacidad grande, combinando subconjuntos de cada mitad ordenados por peso con un "prefijo máximo" de valor.
- Contar cuántos subconjuntos de un arreglo de hasta 40 elementos satisfacen una condición de suma o XOR, combinando conteos de ambas mitades con dos punteros o búsqueda binaria.
- Problemas donde el espacio de estados de Bitmask DP (2^n) es demasiado grande para caber en memoria, pero se puede reformular como una combinación de dos mitades independientes.

Complejidad: $O(2^{n/2} \cdot \log(2^{n/2})) = O(2^{n/2} \cdot n)$ típicamente: generar las $2^{n/2}$ combinaciones de cada mitad cuesta $O(2^{n/2} \cdot n/2)$, y ordenar una de las listas para combinar cuesta $O(2^{n/2} \log(2^{n/2}))$; la combinación final (recorrer una lista y buscar en la otra) agrega otro factor $O(2^{n/2} \log(2^{n/2}))$. Esto reemplaza el $O(2^n)$ o $O(2^n \cdot n)$ inviable de Bitmask DP directo para n grande.

```

1 struct MeetInTheMiddle {
2     vector<long long> elementos;
3     int n;
4
5     // CREATE: guarda el arreglo completo de elementos a dividir
  en
6     // dos mitades
7     MeetInTheMiddle(vector<long long> arr) : elementos(arr), n((
  int)arr.size()) {} // O(n)
8
9     // READ: genera todas las sumas de subconjuntos (incluyendo
  el

```

```

10 // vacío, suma 0) de los elementos en el rango [inicio, fin)
11 vector<long long> generarSumas(int inicio, int fin) {
12     vector<long long> resultado;
13     int tam = fin - inicio;
14     for (int mask = 0; mask < (1 << tam); mask++) {
15         long long suma = 0;
16         for (int bit = 0; bit < tam; bit++) {
17             if (mask & (1 << bit)) suma += elementos[inicio + bit];
18         }
19         resultado.push_back(suma);
20     }
21     return resultado;
22 } // O(2^tam * tam)

23
24 // READ: verifica si existe algun subconjunto del arreglo
25 // COMPLETO
26 // cuya suma sea exactamente "objetivo", dividiendo el
27 // arreglo en
28 // dos mitades y combinando con búsqueda binaria
29 bool existeSubconjuntoConSuma(long long objetivo) {
30     int mitad = n / 2;
31     vector<long long> sumasIzquierda = generarSumas(0, mitad);
32     vector<long long> sumasDerecha = generarSumas(mitad, n);
33
34     sort(sumasDerecha.begin(), sumasDerecha.end());
35
36     for (long long sumaI : sumasIzquierda) {
37         long long complemento = objetivo - sumaI;
38         if (binary_search(sumasDerecha.begin(), sumasDerecha.end(),
39                             complemento)) {
40             return true;
41         }
42     }
43     return false;
44 } // O(2^(n/2) * (n/2) + 2^(n/2) * log(2^(n/2)))
45
46 // -----
47 // MODIFICACIONES Y COMENTARIOS CON LOS USOS
48 // -----
49 // 1. Contar subconjuntos en vez de verificar existencia: en
50 // vez de binary_search (que solo confirma presencia), usar
51 // equal_range o lower_bound/upper_bound sobre
52 // sumasDerecha
53 // ordenada para contar CUANTAS veces aparece el
54 // complemento,
55 // acumulando el total sobre todas las sumaI de la
56 // izquierda.
57 //
58 // 2. Knapsack con Meet in the Middle: generar pares
59 // (peso, valor) para cada subconjunto de cada mitad;
60 // ordenar
61 // una mitad por peso y precalcular el "maximo valor con
62 // peso
63 // <= w" como un prefijo maximo; para cada subconjunto de
64 // la
65 // otra mitad, buscar con búsqueda binaria el mayor peso
66 // permitido restante y combinar con ese prefijo maximo.
67 //
68 // 3. Por que dividir por la mitad es optimo: si se dividiera
69 // en
70 // proporciones desiguales (por ejemplo, n/3 y 2n/3), el
71 // lado
72 // mas grande seguiria dominando la complejidad total
73 // (2^(2n/3) sigue siendo exponencialmente peor que 2^(n/2));
74 // dividir exactamente a la mitad minimiza el maximo de
75 // las
76 // dos mitades, que es lo que determina el costo total.
77 //
78 // 4. Generalizacion a XOR en vez de suma: el mismo esqueleto
79 // aplica reemplazando la suma de subconjunto por XOR de
80 // subconjunto; para buscar el complemento exacto, sigue
81 // funcionando búsqueda binaria (o tabla hash) sobre
82 // valores
83 // ordenados, ya que la condicion sigue siendo una
84 // igualdad.
85 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> arr = {3, 7, 1, 9, 4, 12, 6, 2}; // n = 8 (
2 // ejemplo pequeño)
3 MeetInTheMiddle mitm(arr);
4
5 bool existe23 = mitm.existeSubconjuntoConSuma(23);
6 // Nota: el valor exacto depende de verificar todas las
7 // combinaciones
8 // de ambas mitades; se debe correr el código para confirmarlo
9 // antes
10 // de darlo por verificado (candidato esperado: true, ya que

```

```

8 hay
9 // varias combinaciones plausibles que suman 23 en un arreglo
10 // con
11 // estos valores).

```

Regla práctica: Reconoce Meet in the Middle cuando n esté en el rango de ~ 30 a ~ 45 (demasiado grande para Bitmask DP directo con 2^n , pero divisible en dos mitades de $2^{n/2}$ manejable) y el problema tenga estructura de "elegir un subconjunto que satisfaga una condición de suma/XOR/valor"; para $n \leq 20-24$, Bitmask DP directo suele ser más simple y no requiere la complejidad adicional de combinar dos mitades.

37.22 Transformada de Möbius/Zeta sobre Subconjuntos

Idea: La Transformada Zeta sobre subconjuntos transforma un arreglo f indexado por máscaras en $\hat{f}[m] = \sum_{s \subseteq m} f[s]$ — exactamente el algoritmo de SOS DP visto antes, pero presentado aquí en el lenguaje general de "transformadas" sobre el retículo de subconjuntos, en analogía directa con la Transformada de Fourier. La Transformada de Möbius es su **inversa**: dado $\hat{f}$, recupera f mediante $f[m] = \sum_{s \subseteq m} (-1)^{|m|-|s|} \hat{f}[s]$, que en la práctica se calcula con el mismo bucle bit-por-bit de SOS DP pero restando en vez de sumar en cada paso. Juntas, estas dos transformadas son el mecanismo que permite la Convolución AND/OR vista en la subsección anterior: transformar ambos arreglos con Zeta, multiplicar punto a punto, y aplicar Möbius para volver al dominio original.

¿Por qué usarlo? Esta subsección formaliza una idea que ya usaste dos veces sin nombrarla explícitamente como "transformada": SOS DP calcula la Zeta, y la Convolución AND/OR usa el par Zeta-Möbius como sus pasos de ida y vuelta. Nombrarla aparte tiene valor principalmente cuando necesitas la **inversa** de forma independiente —por ejemplo, si conoces $\hat{f}$ (una suma sobre subconjuntos ya agregada) y necesitas recuperar los valores individuales f originales, en vez de solo ir "hacia adelante" como hace SOS DP.

Casos de uso:

- Recuperar los valores individuales $f[m]$ a partir de una suma sobre subconjuntos $\hat{f}[m]$ ya conocida (el paso inverso que SOS DP no necesita, pero que Convolución AND/OR sí requiere al final).
- Aplicar principio de inclusión-exclusión sobre el retículo de subconjuntos de forma sistemática, cuando la fórmula de inclusión-exclusión directa tiene 2^n términos por calcular ingenuamente.
- Como el paso final de Convolución AND/OR (Transformada Zeta para ir al dominio transformado, Möbius para volver), completando el patrón "transformar, multiplicar punto a punto, destransformar" ya visto con XOR/Walsh-Hadamard.
- Verificar la consistencia de un algoritmo de SOS DP: aplicar Möbius sobre el resultado de Zeta debe devolver exactamente el arreglo original, útil como *sanity check*.

Complejidad: $O(n \cdot 2^n)$ tanto para la Transformada Zeta como para la Möbius — idéntica a SOS DP, ya que

la Möbius es literalmente el mismo bucle con el signo de la actualización invertido (resta en vez de suma); aplicar ambas en secuencia (ida y vuelta) sigue siendo $O(n \cdot 2^n)$, no se duplica el orden de crecimiento.

```

1 struct TransformadaMoebiusZeta {
2     int n;
3
4     // CREATE: guarda el numero de bits del universo de mascaras
5     TransformadaMoebiusZeta(int numBits) : n(numBits) {} // O(1)
6
7     // WRITE: aplica la Transformada Zeta en el lugar: convierte
8     // f[m]
9     // (valor base) en la suma sobre todas las submascaras de m
10    // (identica al algoritmo de SOS DP)
11    void zeta(vector<long long>& f) {
12        int total = 1 << n;
13        for (int bit = 0; bit < n; bit++) {
14            for (int mask = 0; mask < total; mask++) {
15                if (mask & (1 << bit)) {
16                    f[mask] += f[mask ^ (1 << bit)];
17                }
18            }
19        } // O(n * 2^n)
20
21        // WRITE: aplica la Transformada de Moebius en el lugar:
22        // invierte
23        // zeta(), recuperando los valores originales f[m] a partir
24        // de la
25        // suma sobre subconjuntos
26        void moebius(vector<long long>& f) {
27            int total = 1 << n;
28            for (int bit = 0; bit < n; bit++) {
29                for (int mask = 0; mask < total; mask++) {
30                    if (mask & (1 << bit)) {
31                        f[mask] -= f[mask ^ (1 << bit)]; // resta en vez de
32                        // suma: deshace zeta
33                    }
34                }
35            } // O(n * 2^n)
36
37            //
38            // -----
39            // MODIFICACIONES Y COMENTARIOS CON LOS USOS
40            //
41            // -----
42            // 1. Por que restar deshace la suma: zeta() propaga cada
43            // valor
44            // original hacia todas las mascaras que lo contienen como
45            // submascara, bit a bit; moebius() recorre los mismos
46            // bits en
47            // el mismo orden, y en cada paso resta exactamente lo que
48            // zeta() habia sumado en ese paso, deshaciendo la
49            // transformacion bit por bit hasta recuperar los valores
50            // base.
51            //
52            // 2. Version para superset (Zeta/Moebius sobre
53            // superconjuntos):
54            // analoga a la version "superset sum" mencionada en SOS
55            // DP,
56            // invirtiendo la condicion del if a "!(mask & (1<<bit))"
57            // y
58            // operando sobre mask | (1<<bit) en vez de mask ^ (1<<bit)
59            // ).
60            //
61            // 3. Uso dentro de Convolucion AND/OR: para convolucion OR,
62            // aplicar zeta() a ambos arreglos, multiplicar punto a
63            // punto,
64            // y aplicar moebius() al resultado recupera exactamente
65            // c[k] = suma de a[i]*b[j] para todo i/j == k; para AND,
66            // se
67            // usa la version "superset" de ambas transformadas.
68            //
69            // 4. Relacion formal con inclusion-exclusion: la formula
70            // f[m] = suma sobre s submascara de m de (-1)^(|m|-|s|) *
71            // fhat[s]
72            // es la version "sobre subconjuntos" del principio de
73            // inclusion-exclusion; el algoritmo bit a bit de moebius
74            // () es
75            // simplemente una forma eficiente de evaluar esa suma sin
76            // calcular (-1)^(|m|-|s|) explicitamente para cada par (m,
77            // s).
78        };
79    };

```

Ejemplo de funcionamiento:

```

1 vector<long long> original = {1, 2, 3, 4, 5, 6, 7, 8}; //
2 // universo de 3 bits
3 vector<long long> f = original; // copia para transformar

```

```

4 TransformadaMoebiusZeta t(3);
5 t.zeta(f);
6 // f ahora contiene, para cada mascara, la suma sobre sus
7 // submascaras
8 // (identico al resultado de SOS DP)
9
10 t.moebius(f);
11 // f deberia volver exactamente a "original" tras deshacer zeta
12 // ()
13 // Nota: verificar en codigo que f == original tras este paso,
14 // como
15 // sanity check de la implementacion.

```

Regla práctica: Si solo necesitas ir "hacia adelante" (sumar sobre subconjuntos una sola vez), usa SOS DP directamente sin pensar en términos de transformadas; recurre explícitamente a esta subsección cuando necesitas la **inversa** (recuperar valores originales desde una agregación) o cuando estés implementando Convolución AND/OR, donde el par Zeta-Möbius es indispensable como los pasos de ida y vuelta.

Parte XII Game Theory